

Documentation PostgreSQL 17.10

Le groupe de développement mondial de PostgreSQL

Documentation PostgreSQL 17.10

Le groupe de développement mondial de PostgreSQL

Copyright © 1996–2026 The PostgreSQL Global Development Group

Avis juridique

Système de gestion de bases de données PostgreSQL (également connu sous le nom de Postgres, anciennement connu sous le nom de Postgres95)

Certaines parties sont protégées par le droit d'auteur © 1996-2026, PostgreSQL Global Development Group

Certaines parties sont protégées par le droit d'auteur © 1994, The Regents of the University of California

L'autorisation d'utiliser, de copier, de modifier et de distribuer ce logiciel et sa documentation à quelque fin que ce soit, sans frais et sans accord écrit, est accordée par la présente, à condition que la mention de droit d'auteur ci-dessus, le présent paragraphe et les deux paragraphes suivants figurent dans toutes les copies.

EN AUCUN CAS L'UNIVERSITÉ DE CALIFORNIE NE POURRA ÊTRE TENUE RESPONSABLE ENVERS QUICONQUE DES DOMMAGES DIRECTS, INDIRECTS, SPÉCIAUX, ACCESSOIRES OU CONSÉCUTIFS, Y COMPRIS LES PERTES DE PROFITS, DÉCOULANT DE L'UTILISATION DE CE LOGICIEL ET DE SA DOCUMENTATION, MÊME SI L'UNIVERSITÉ DE CALIFORNIE A ÉTÉ AVERTIE DE LA POSSIBILITÉ DE TELS DOMMAGES.

L'Université de Californie décline expressément toute garantie, y compris, mais sans s'y limiter, les garanties implicites de qualité marchande et d'adéquation à un usage particulier. Le logiciel fourni ci-dessous est fourni « en l'état », et l'Université de Californie n'a aucune obligation d'assurer la maintenance, le support, les mises à jour, les améliorations ou les modifications.

Table des matières

Préface	xxxii
1. Qu'est-ce que PostgreSQL ?	xxxii
2. Bref historique de PostgreSQL	xxxii
2.1. Le projet Berkeley POSTGRES	xxxiii
2.2. Postgres95	xxxiii
2.3. PostgreSQL	xxxiv
3. Conventions	xxxiv
4. Informations complémentaires	xxxiv
5. Consignes de signalement des bogues	xxxv
5.1. Identification des bogues	xxxv
5.2. Éléments à signaler	xxxvi
5.3. Où signaler les bogues	xxxvii
I. Tutoriel	1
1. Pour commencer	3
1.1. Installation	3
1.2. Principes fondamentaux de l'architecture	3
1.3. Création d'une base de données	3
1.4. Accès à une base de données	5
2. Le langage SQL	7
2.1. Introduction	7
2.2. Concepts	7
2.3. Création d'une nouvelle table	7
2.4. Remplissage d'un tableau avec des lignes	8
2.5. Interrogation d'une table	9
2.6. Jointures entre tables	11
2.7. Fonctions agrégées	13
2.8. Mises à jour	15
2.9. Suppressions	15
3. Fonctionnalités avancées	17
3.1. Introduction	17
3.2. Vues	17
3.3. Clés étrangères	17
3.4. Transactions	18
3.5. Fonctions de fenêtrage	20
3.6. Héritage	23
3.7. Conclusion	24
II. Le langage SQL	25
4. Syntaxe SQL	33
4.1. Structure lexicale	33
4.2. Expressions de valeurs	42
4.3. Appel de fonctions	56
5. Définition des données	59
5.1. Notions de base sur les tableaux	59
5.2. Valeurs par défaut	60
5.3. Colonnes d'identité	61
5.4. Colonnes générées	62
5.5. Contraintes	64
5.6. Colonnes du système	73
5.7. Modification des tableaux	74
5.8. Privilèges	76
5.9. Politiques de sécurité des lignes	81
5.10. Schémas	87
5.11. Héritage	92
5.12. Partitionnement des tables	96
5.13. Données étrangères	109

5.14. Autres objets de base de données	110
5.15. Suivi des dépendances	110
6. Manipulation des données	112
6.1. Insertion des données 112.....	
6.2. Mise à jour des données 113.....	
6.3. Suppression des données 114.....	
6.4. Retour des données des lignes modifiées	114
7. Requêtes	116
7.1. Aperçu	116
7.2. Expressions tabulaires	116
7.3. Listes de sélection	132
7.4. Combinaison de requêtes (UNION, INTERSECT, EXCEPT)	134
7.5. Tri des lignes (ORDER BY)	135
7.6. LIMITE et COMPENSATION	136
7.7. VALEURS Listes	136
7.8. Requêtes WITH (Expressions de table communes)	137
8. Types de données	147
8.1. Types numériques	148
8.2. Types monétaires	154
8.3. Types de caractères	154
8.4. Types de données binaires	157
8.5. Types de date/heure	159
8.6. Type booléen	168
8.7. Types énumérés	169
8.8. Types géométriques	171
8.9. Types d'adresses réseau	174
8.10. Types de chaînes binaires	176
8.11. Types de recherche textuelle	177
8.12. Type d'UUID	180
8.13. Type XML	180
8.14. Types JSON	182
8.15. Tableaux	192
8.16. Types composites	202
8.17. Types de gammes	208
8.18. Types de domaines	214
8.19. Types d'identificateurs d'objets	215
8h20. Type pg_Isn	217
8.21. Pseudo-types	218
9. Fonctions et opérateurs	220
9.1. Opérateurs logiques	220
9.2. Fonctions et opérateurs de comparaison	221
9.3. Fonctions et opérateurs mathématiques	225
9.4. Fonctions et opérateurs de chaînes de caractères	233
9.5. Fonctions et opérateurs de chaînes binaires	243
9.6. Fonctions et opérateurs de chaînes binaires	247
9.7. Appariement de motifs	249
9.8. Fonctions de mise en forme des types de données	268
9.9. Fonctions et opérateurs de date/heure	276
9.10. Fonctions de support des énumérations	293
9.11. Fonctions et opérateurs géométriques	294
9.12. Fonctions et opérateurs d'adressage réseau	301
9.13. Fonctions et opérateurs de recherche de texte	304
9.14. Fonctions UUID	310
9.15. Fonctions XML	311
9.16. Fonctions et opérateurs JSON	325
9.17. Fonctions de manipulation de séquences	358
9.18. Expressions conditionnelles	360
9.19. Fonctions et opérateurs de tableaux	363

9.20. Fonctions et opérateurs de plage/multiplage	366
9.21. Fonctions agrégées	372
9.22. Fonctions de fenêtrage	380
9.23. Fusion des fonctions de support	381
9.24. Expressions de sous-requête	382
9.25. Comparaisons de lignes et de tableaux	385
9.26. Fonctions de retour d'ensembles	387
9.27. Fonctions et opérateurs d'information système	391
9.28. Fonctions d'administration système	412
9.29. Fonctions de déclenchement	429
9.30. Fonctions de déclenchement d'événements	430
9.31. Fonctions d'information statistique	433
10. Conversion de type	434
10.1. Aperçu	434
10.2. Opérateurs	435
10.3. Fonctions	439
10.4. Stockage de valeur	443
10.5. UNION, CASE et constructions connexes	444
10.6. SÉLECTIONNER les colonnes de sortie	445
11. Index	447
11.1. Introduction	447
11.2. Types d'index	448
11.3. Index à plusieurs colonnes	450
11.4. Index et ORDER BY	451
11.5. Combinaison de plusieurs index	452
11.6. Index uniques	453
11.7. Indices sur les expressions	453
11.8. Index partiels	454
11.9. Numérisations d'index uniquement et index de couverture	457
11.10. Classes et familles d'opérateurs	459
11.11. Index et classements	461
11.12. Examen de l'utilisation de l'index	461
12. Recherche en texte intégral	463
12.1. Introduction	463
12.2. Tables et index	467
12.3. Contrôle de la recherche textuelle	469
12.4. Fonctionnalités supplémentaires	476
12.5. Analyseurs syntaxiques	482
12.6. Dictionnaires	484
12.7. Exemple de configuration	493
12.8. Test et débogage de la recherche de texte	495
12.9. Types d'index préférés pour la recherche textuelle	499
12.10. Support psql	500
12.11. Limitations	503
13. Contrôle de la concurrence	505
13.1. Introduction	505
13.2. Isolation des transactions	505
13.3. Verrouillage explicite	511
13.4. Contrôles de cohérence des données au niveau de l'application	517
13.5. Gestion des échecs de sérialisation	518
13.6. Avertissements	519
13.7. Verrouillage et index	519
14. Conseils de performance	521
14.1. Utilisation d'EXPLAIN	521
14.2. Statistiques utilisées par le planificateur	534
14.3. Contrôle du planificateur avec des clauses JOIN explicites	540
14.4. Remplissage d'une base de données	542
14.5. Réglages non durables	544

15. Requête parallèle	546
15.1. Fonctionnement des requêtes parallèles	546
15.2. Quand peut-on utiliser les requêtes parallèles ?	547
15.3. Plans parallèles	548
15.4. Sécurité parallèle	550
III. Administration du serveur	552
16. Installation à partir des binaires	559
17. Installation à partir du code source	560
17.1. Exigences	560
17.2. Obtention de la source	562
17.3. Compilation et installation avec Autoconf et Make	562
17.4. Construction et installation avec Meson	575
17.5. Configuration post-installation	586
17.6. Plateformes prises en charge	588
17.7. Remarques spécifiques à la plateforme	588
18. Installation et fonctionnement du serveur	595
18.1. Le compte utilisateur PostgreSQL	595
18.2. Création d'un cluster de base de données	595
18.3. Démarrage du serveur de base de données	597
18.4. Gestion des ressources du noyau	601
18.5. Arrêt du serveur	608
18.6. Mise à niveau d'un cluster PostgreSQL	609
18.7. Prévention de l'usurpation d'identité du serveur	612
18.8. Options de chiffrement	613
18.9. Sécuriser les connexions TCP/IP avec SSL	614
18.10. Sécurisation des connexions TCP/IP avec chiffrement GSSAPI	618
18.11. Sécuriser les connexions TCP/IP avec des tunnels SSH	618
18.12. Enregistrement du journal des événements sous Windows	619
19. Configuration du serveur	621
19.1. Paramétrage	621
19.2. Emplacements des fichiers	625
19.3. Connexions et authentification	626
19.4. Consommation des ressources	633
19.5. Journal de prévision	642
19.6. Réplication	653
19.7. Planification des requêtes	660
19.8. Signalement et journalisation des erreurs	667
19.9. Statistiques d'exécution	681
19.10. Aspirateur automatique	683
19.11. Paramètres par défaut de connexion client	684
19.12. Gestion des serrures	695
19.13. Compatibilité des versions et des plateformes	696
19.14. Gestion des erreurs	698
19.15. Options prédéfinies	699
19.16. Options personnalisées	701
19.17. Options pour développeurs	701
19.18. Options à découvrir	707
20. Authentification du client	708
20.1. Le fichier pg_hba.conf	708
20.2. Cartes des noms d'utilisateur	717
20.3. Méthodes d'authentification	718
20.4. Authentification de confiance	719
20.5. Authentification par mot de passe	720
20.6. Authentification GSSAPI	721
20.7. Authentification SSPI	722
20.8. Authentification d'identité	723
20.9. Authentification par les pairs	724
20.10. Authentification LDAP	724

20.11. Authentification RADIUS	727
certificat	728
PAM	728
BSD	729
d'authentification	729
données	731
données	731
rôles	732
rôles	734
rôles	735
prédéfinis	736
fonctions	738
données	739
d'ensemble	739
données	739
modèles	740
données	742
données	742
tables	742
Localisation	745
régionaux	745
classement	750
caractères	760
de données	770
routine	770
routine	779
journaux	780
restauration	782
SQL	782
fichiers	785
dans le temps (PITR)	786
réplication	798
798 26.2. Serveurs de secours avec transfert de journaux	801
Basculement	810
chaud	811
données	819
standard	819
cumulatives	820
verrous	860
d'avancement	861
dynamique	869
disque	878
transactions	880
Fiabilité	880
données	882
(WAL)	882
asynchrone	883
WAL	884
WAL	887
logique	889
Publication	889
Abonnement	890
logique	896
lignes	898
colonnes	906
Conflits	908
Restrictions	909

29.8. Architecture	910
29.9. Surveillance	911
29.10. Sécurité	911
29.11. Paramètres de configuration	912
29.12. Configuration rapide	912
30. Compilation juste-à-temps (JAT)	914
30.1. Qu'est-ce que la compilation JIT ?	914
30.2. Quand utiliser le JIT ?	914
30.3. Configuration	916
30.4. Extensibilité	916
31. Tests de régression	917
31.1. Exécution des tests	917
31.2. Évaluation du test	921
31.3. Fichiers de comparaison des variantes	923
31.4. Tests TAP	924
31.5. Couverture du test Examen	925
IV. Interfaces client	927
32. libpq — Bibliothèque C	932
32.1. Fonctions de contrôle de la connexion à la base de données	932
32.2. Fonctions d'état de la connexion	951
32.3. Fonctions d'exécution des commandes	958
32.4. Traitement asynchrone des commandes	974
32.5. Mode pipeline	978
32.6. Récupération des résultats de requête par blocs	983
32.7. Annulation des requêtes en cours	984
32.8. L'interface Fast-Path	989
32.9. Notification asynchrone	990
32.10. Fonctions associées à la commande COPY	991
32.11. Fonctions de contrôle	995
32.12. Fonctions diverses	997
32.13. Traitement des notifications	1001
32.14. Système d'événements	1002
32.15. Variables d'environnement	1008
32.16. Le fichier des mots de passe	1010
32.17. Fichier de service de connexion	1011
32.18. Recherche LDAP des paramètres de connexion	1011
32.19. Prise en charge SSL	1012
32.20. Comportement dans les programmes multithread	1016
32.21. Compilation de programmes libpq	1017
32.22. Exemples de programmes	1018
33. Objets volumineux	1030
33.1. Introduction	1030
33.2. Caractéristiques de mise en œuvre	1030
33.3. Interfaces client	1030
33.4. Fonctions côté serveur	1035
33.5. Exemple de programme	1036
34. ECPG — SQL intégré en C	1042
34.1. Le concept	1042
34.2. Gestion des connexions aux bases de données	1042
34.3. Exécution des commandes SQL	1046
34.4. Utilisation des variables hôtes	1049
34.5. SQL dynamique	1063
34.6. Bibliothèque pgtypes	1065
34.7. Utilisation des zones descriptives	1079
34.8. Gestion des erreurs	1092
34.9. Directives relatives aux préprocesseurs	1099
34.10. Traitement des programmes SQL embarqués	1101
34.11. Fonctions de la bibliothèque	1102

34.12. Objets volumineux	1103
34.13. Applications C++	1104
34.14. Commandes SQL intégrées	1108
34.15. Mode de compatibilité Informix	1132
34.16. Mode de compatibilité Oracle	1147
34.17. Intérieur	1147
35. Le schéma d'information	1150
35.1. Le schéma	1150
35.2. Types de données	1150
35.3. nom_catalogue schéma_information	1151
35.4. autorisations de rôles administrables	1151
35.5. rôles applicables	1151
35.6. attributs	1152
35.7. jeux de caractères	1154
35.8. check_constraint_routine_usage	1155
35.9. check_constraints	1155
35.10. classements	1156
35.11. applicabilité du jeu de caractères de collation	1156
35.12. utilisation_colonne	1157
35.13. column_domain_usage	1157
35.14. options_colonne	1158
35.15. privilèges de colonne	1158
35.16. column_udt_usage	1159
35.17. colonnes	1159
35.18. utilisation_colonne_contrainte	1162
35.19. Utilisation de la table des contraintes	1163
35.20. privilèges_type_de_données	1163
35.21. contraintes_domaine	1164
35.22. domain_udt_usage	1165
35.23. Domaines	1165
35.24. types_éléments	1167
35.25. enabled_roles	1169
35.26. options_foreign_data_wrapper	1169
35.27. wrappers de données étrangères	1170
35.28. options_serveur_étranger	1170
35.29. Serveurs étrangers	1170
35.30. options_table_étranger	1171
35.31. tables étrangères	1171
35.32. key_column_usage	1172
35.33. Paramètres	1172
35.34. contraintes référentielles	1174
35.35. role_column_grants	1175
35.36. role_routine_grants	1175
35.37. role_table_grants	1176
35.38. role_udt_grants	1177
35.39. role_usage_grants	1177
35.40. routine_column_usage	1178
35.41. privilèges de routine	1179
35.42. routine_routine_usage	1179
35.43. routine_sequence_usage	1180
35.44. routine_table_usage	1180
35.45. routines	1181
35.46. schémas	1185
35.47. séquences	1186
35.48. sql_features	1186
35.49. sql_implementation_info	1187
35.50. sql_parts	1187
35.51. sql_sizing	1188

35.52. contraintes_table	1188
35.53. privilèges_table	1189
35.54. Tables	1190
35.55. transformations	1190
35.56. triggered_update_columns	1191
35.57. déclencheurs	1192
35.58. udt_privileges	1193
35.59. privilèges_usage	1194
35.60. types_définis_par_l'utilisateur	1194
35.61. options_de_mappage_utilisateur	1196
35.62. mappages_utilisateur	1196
35.63. view_column_usage	1197
35.64. view_routine_usage	1197
35.65. view_table_usage	1198
35.66 vues	1198
V. Programmation serveur	1200
36. Extension de SQL	1206
36.1. Fonctionnement de l'extensibilité	1206
36.2. Le système de types PostgreSQL	1206
36.3. Fonctions définies par l'utilisateur	1209
36.4. Procédures définies par l'utilisateur	1210
36.5. Fonctions du langage de requête (SQL)	1210
36.6. Surcharge de fonction	1227
36.7. Catégories de volatilité fonctionnelle	1228
36.8. Fonctions du langage procédural	1229
36.9. Fonctions internes	1229
36.10. Fonctions du langage C	1230
36.11. Informations sur l'optimisation des fonctions	1253
36.12. Agrégats définis par l'utilisateur	1254
36.13. Types définis par l'utilisateur	1261
36.14. Opérateurs définis par l'utilisateur	1265
36.15. Informations sur l'optimisation de l'opérateur	1266
36.16. Extensions d'interface pour les index	1270
36.17. Regroupement d'objets liés dans une extension	1283
36.18. Infrastructure du bâtiment d'extension	1291
37. Déclencheurs	1296
37.1. Aperçu du comportement des déclencheurs	1296
37.2. Visibilité des modifications de données	1299
37.3. Écriture de fonctions de déclenchement en C	1299
37.4. Exemple complet de déclenchement	1302
38. Déclencheurs d'événements	1306
38.1. Aperçu du comportement des déclencheurs d'événements	1306
38.2. Matrice de déclenchement d'événement	1307
38.3. Écriture de fonctions de déclenchement d'événements en C	1310
38.4. Exemple complet de déclenchement d'événement	1312
38.5. Exemple de déclencheur d'événement de réécriture de table	1313
38.6. Exemple de déclencheur d'événement de connexion à une base de données	1314
39. Le système de règles	1316
39.1. L'arbre de requête	1316
39.2. Opinions et système de règles	1318
39.3. Vues matérialisées	1324
39.4. Règles relatives aux opérations INSERT, UPDATE et DELETE	1327
39.5. Règles et privilèges	1338
39.6. Règles et statut de commandement	1340
39.7. Règles contre déclencheurs	1340
40. Langages procéduraux	1344
40.1. Installation des langages procéduraux	1344
41. PL/pgSQL — Langage procédural SQL	1347

41.1. Aperçu	1347
41.2. Structure de PL/pgSQL	1348
41.3. Déclarations	1350
41.4. Expressions	1356
41.5. Énoncés de base	1357
41.6. Structures de contrôle	1365
41.7. Curseurs	1380
41.8. Gestion des transactions	1386
41.9. Erreurs et messages	1387
41.10. Fonctions de déclenchement	1389
41.11. PL/pgSQL sous le capot	1398
41.12. Conseils pour le développement en PL/pgSQL	1401
41.13. Migration depuis Oracle PL/SQL	1405
42. PL/Tcl — Langage procédural Tcl	1415
42.1. Aperçu	1415
42.2. Fonctions et arguments PL/Tcl	1415
42.3. Valeurs des données en PL/Tcl	1417
42.4. Données globales en PL/Tcl	1417
42.5. Accès aux bases de données depuis PL/Tcl	1418
42.6. Fonctions de déclenchement en PL/Tcl	1420
42.7. Fonctions de déclenchement d'événements en PL/Tcl	1422
42.8. Gestion des erreurs en PL/Tcl	1422
42.9. Sous-transactions explicites en PL/Tcl	1423
42.10. Gestion des transactions	1424
42.11. Configuration PL/Tcl	1425
42.12. Noms des procédures Tcl	1425
43. PL/Perl — Langage procédural Perl	1426
43.1. Fonctions et arguments PL/Perl	1426
43.2. Valeurs des données en PL/Perl	1431
43.3. Fonctions intégrées	1431
43.4. Valeurs globales en PL/Perl	1436
43.5. PL/Perl de confiance et non de confiance	1437
43.6. Déclencheurs PL/Perl	1438
43.7. Déclencheurs d'événements PL/Perl	1440
43.8. PL/Perl Sous le capot	1440
44. PL/Python — Langage procédural Python	1442
44.1. Fonctions PL/Python	1442
44.2. Valeurs des données	1443
44.3. Partage des données	1449
44.4. Blocs de code anonymes	1449
44.5. Fonctions de déclenchement	1449
44.6. Accès aux bases de données	1450
44.7. Sous-transactions explicites	1454
44.8. Gestion des transactions	1455
44.9. Fonctions utilitaires	1455
44.10. Python 2 contre Python 3	1456
44.11. Variables d'environnement	1456
45. Interface de programmation serveur	1458
45.1. Fonctions d'interface	1458
45.2. Fonctions de support d'interface	1500
45.3. Gestion de la mémoire	1509
45.4. Gestion des transactions	1519
45.5. Visibilité des modifications de données	1522
45.6. Exemples	1522
46. Processus des travailleurs en arrière-plan	1526
47. Décodage logique	1529
47.1. Exemples de décodage logique	1529
47.2. Concepts de décodage logique	1533

47.3. Interface du protocole de réplication en continu	1536
47.4. Interface SQL de décodage logique	1536
47.5. Catalogues système relatifs au décodage logique	1536
47.6. Modules de sortie de décodage logique	1536
47.7. Enregistreurs de sortie de décodage logique	1544
47.8. Support de la réplication synchrone pour le décodage logique	1544
47.9. Diffusion en continu de transactions volumineuses pour le décodage logique	1545
47.10. Prise en charge de la validation en deux phases pour le décodage logique	1546
48. Suivi de la progression de la réplication	1548
49. Modules d'archives	1549
49.1. Fonctions d'initialisation	1549
49.2. Rappels du module d'archivage	1549
VI. Référence	1552
I. Commandes SQL	1557
ABORTIR	1561
ALTER AGGREGATE	1562
ALTER COLLATION	1564
ALTER CONVERSION	1567
ALTER DATABASE	1569
MODIFIER LES PRIVILÈGES PAR DÉFAUT	1572
ALTER DOMAIN	1576
DÉCLENCHEUR D'ÉVÉNEMENT MODIFIÉ	1580
ALTER EXTENSION	1581
MODIFIER L'EMBALLAGE DE DONNÉES ÉTRANGÈRES	1585
MODIFIER LE TABLEAU ÉTRANGER	1587
FONCTION ALTER	1592
ALTER GROUP	1596
MODIFIER L'INDEX	1598
ALTER LANGUAGE	1601
MODIFIER UN GRAND OBJET	1602
MODIFIER LA VUE MATÉRIALISÉE	1603
ALTER OPÉRATEUR	1605
CLASSE D'OPÉRATEUR MODIFIÉE	1607
FAMILLE D'OPÉRATEURS ALTER	1608
MODIFIER LA POLITIQUE	1612
PROCÉDURE DE MODIFICATION	1614
ALTER PUBLICATION	1617
ALTER RÔLE	1620
MODIFIER LA ROUTINE	1624
MODIFIER LA RÈGLE	1626
MODIFIER LE SCHÉMA	1627
ALTER SEQUENCE	1628
ALTER SERVER	1631
STATISTIQUES ALTER	1633
MODIFIER L'ABONNEMENT	1634
ALTER SYSTEM	1637
MODIFIER LA TABLE	1639
ALTER TABLESPACE	1657
MODIFIER LA CONFIGURATION DE RECHERCHE DE TEXTE	1659
ALTER TEXT SEARCH DICTIONARY	1661
ALTER TEXT SEARCH PARSER	1663
MODÈLE DE RECHERCHE DE TEXTE ALTER	1664
ALTER TRIGGER	1665
ALTER TYPE	1667
ALTER USER	1672
MODIFIER LE MAPPAGE UTILISATEUR	1673
ALTER VIEW	1674
ANALYSER	1676

DÉBUT	1680
APPEL	1682
POINT DE CONTRÔLE	1684
FERMER	1685
CLUSTER	1686
COMMENTAIRE	1689
COMMIT	1694
COMMISSION PRÉPARÉE	1695
COPIE	1696
CRÉER UNE MÉTHODE D'ACCÈS	1707
CRÉER UN AGRÉGAT	1708
CRÉER UN CASTING	1716
CRÉER UN COLLAGE	1720
CRÉER UNE CONVERSION	1723
CRÉER UNE BASE DE DONNÉES	1725
CRÉER UN DOMAINE	1730
CRÉER UN DÉCLENCHEUR D'ÉVÉNEMENT	1733
CRÉER UNE EXTENSION	1735
CRÉER UN WRAPPER DE DONNÉES ÉTRANGÈRES	1738
CRÉER UNE TABLE ÉTRANGÈRE	1740
CRÉER UNE FONCTION	1745
CRÉER UN GROUPE	1754
CRÉER UN INDEX	1755
CRÉER UN LANGAGE	1764
CRÉER UNE VUE MATÉRIALISÉE	1767
CRÉER UN OPÉRATEUR	1769
CRÉER UNE CLASSE D'OPÉRATEUR	1772
CRÉER UNE FAMILLE D'OPÉRATEURS	1775
CRÉER UNE POLITIQUE	1776
CRÉER UNE PROCÉDURE	1783
CRÉER UNE PUBLICATION	1787
CRÉER UN RÔLE	1791
CRÉER UNE RÉGLE	1796
CRÉER UN SCHÉMA	1799
CRÉER UNE SÉQUENCE	1802
CRÉER UN SERVEUR	1806
CRÉER DES STATISTIQUES	1808
CRÉER UN ABONNEMENT	1812
CRÉER LA TABLE	1817
CRÉER LA TABLE COMME	1840
CRÉER UN ESPACE DE TABLE	1843
CRÉER UNE CONFIGURATION DE RECHERCHE DE TEXTE	1845
CRÉER UN DICTIONNAIRE DE RECHERCHE TEXTUELLE	1846
CRÉER UN ANALYSEUR DE RECHERCHE DE TEXTE	1848
CRÉER UN MODÈLE DE RECHERCHE DE TEXTE 1850.....	
CRÉER TRANSFORMER	1851
CRÉER UN DÉCLENCHEUR	1853
CRÉER TYPE 1860	
CRÉER UN UTILISATEUR	1869
CRÉER UNE CARTOGRAPHIE UTILISATEUR	1870
CRÉER UNE VUE	1872
DÉSALLOCUER	1878
DÉCLARER	1879
SUPPRIMER 1883.....	
JETER	1886
FAIRE	1887
MÉTHODE D'ACCÈS PAR GOUTTE	1889
DROP AGGREGATE	1890

SUPPRESSION DE CAST	1892 SUPPRESSION DE
COLLATION	1893 SUPPRESSION DE
CONVERSION	1894 SUPPRESSION DE BASE DE
DONNÉES	1895 SUPPRESSION DE
DOMAINE	1896 SUPPRESSION DE DÉCLENCHEUR
D'ÉVÉNEMENT	1897 SUPPRESSION
D'EXTENSION	1898 SUPPRESSION D'ENVELOPPE DE
DONNÉES ÉTRANGÈRES	1899 SUPPRESSION DE TABLE
ÉTRANGÈRE	1900 SUPPRESSION DE
FONCTION	1901 SUPPRESSION DE
GROUPE	1903 SUPPRESSION
D'INDEX	1904 SUPPRESSION DE
LANGUE	1906 SUPPRESSION DE VUE
MATÉRIALISÉE	1907 SUPPRESSION
D'OPÉRATEUR	1908 CLASSE D'OPÉRATEURS
DROP	1910 FAMILLE D'OPÉRATEURS
DROP	1912.PROPRIÉTÉ DE DROP
1914 POLITIQUE DROP	1915 PROCÉDURE
DROP	1916 PUBLICATION
DROP	1918 RÔLE
DROP	1919 ROUTINE
DROP	1920 RÈGLE
DROP	1922 SCHÉMA
DROP	1923 SÉQUENCE
DROP	1924 SERVEUR
DROP	1925 STATISTIQUES
DROP	1926 ABONNEMENT
DROP	1927 SUPPRIMER LA
TABLE	1929 SUPPRIMER L'ESPACE
TABLES	1930 SUPPRIMER LA CONFIGURATION DE
RECHERCHE TEXTUELLE	1931 SUPPRIMER LE DICTIONNAIRE DE RECHERCHE
TEXTUELLE	1932 SUPPRIMER L'ANALYSE ANALYTIQUE DE RECHERCHE
TEXTUELLE	1933 SUPPRIMER LE MODÈLE DE RECHERCHE
TEXTUELLE	1934 SUPPRIMER LA
TRANSFORMATION	1935 SUPPRIMER LE
DÉCLENCHEUR	1936 SUPPRIMER LE
TYPE	1937 SUPPRIMER
L'UTILISATEUR	1938 SUPPRIMER LE MAPPAGE DES
UTILISATEURS	1939 SUPPRIMER LA
VUE	1940
FIN	1941
EXÉCUTER	1942
EXPLAIN	1943
EXTRACTION	1949
OCTROI	1953 IMPORTATION SCHÉMA
ÉTRANGER	1959
INSÉRATION	1961
ÉCOUTE	1969
CHARGEMENT	1971
VERROUILLAGE	1972
FUSION	1975
DÉPLACEMENT	1982
NOTIFICATION	1984
PRÉPARATION	1987 PRÉPARATION DE LA
TRANSACTION	1990 RÉATTRIBUTION DE LA
PROPRIÉTÉ	1992 ACTUALISER LA VUE
MATÉRIALISÉE	1993

RÉINDEX	1995	
POINT DE SAUVEGARDE DE LA SORTIE	2000	
RÉINITIALISATION	2002	
RÉVOQUER	2003	
RETOUR EN ARRIÈRE	2008	
RÉINITIALISATION PRÉPARÉE	2009	
RESTAURATION AU POINT DE SAUVEGARDE	2010	
POINT DE SAUVEGARDE	2012	
LABEL DE SÉCURITÉ 2014		
SÉLECTIONNER	2017	
SÉLECTIONNER DANS	2039	
ENSEMBLE	2041	
DÉFINIR LES CONTRAINTES	2044	
DÉFINIR LE RÔLE	2045	
AUTORISATION DE SESSION	2047	
DÉFINIR LA TRANSACTION	2049	
SPECTACLE	2052	
DÉBUT DE LA TRANSACTION	2054	
TRONQUER	2055	
DÉSÉCOUTER	2057	
MISE À JOUR	2059	
VIDE	2064	
VALEURS	2069	
II. Applications clientes PostgreSQL	2072	
clusterdb	2073	
crééb	2076	
créer un utilisateur		2080
dropdb		2085
dropuser		2088
ecpg	2091	
pg_amcheck	2094	
pg_basebackup	2100	
pgbench	2109	
pg_combinebackup	2134	
pg_config	2137	
pg_dump	2140	
pg_dumpall	2156	
pg_isready	2163	
pg_receivewal	2165	
pg_recvlogical	2170	
pg_restore	2174	
pg_verifybackup	2184	
psql	2187	
réindexation de la base de données		2231
vacuumdb	2235	
III. Applications PostgreSQL Server	2240	
base de données d'initialisation		2241
pg_archivecleanup	2246	
pg_checksums	2248	
pg_controldata	2250	
pg_createsubscriber	2251	
pg_ctl	2256	
pg_resetwal	2262	
pg_rewind	2266	
pg_test_fsync	2270	
pg_test_timing	2271	
pg_upgrade	2275	
pg_waldump	2285	

pg_walsummary	2289
postgres	2290
VII. Intérieur	2297
50. Aperçu du fonctionnement interne de PostgreSQL	2303
50.1. Le chemin d'une requête	2303
50.2. Modalités d'établissement des connexions	2303
50.3. L'étape d'analyse syntaxique	2304
50.4. Le système de règles PostgreSQL	2305
50.5. Planificateur/Optimiseur	2305
50.6. Exécuteur testamentaire	2306
51. Catalogues système	2308
51.1. Aperçu	2308
51.2. pg_aggregate	2310
51.3. pg_am	2311
51.4. pg_amop	2312
51.5. pg_amproc	2313
51.6. pg_attrdef	2313
51.7. pg_attribute	2314
51.8. pg_authid	2316
51.9. pg_auth_members	2317
51.10. pg_cast	2317
51.11. pg_class	2318
51.12. pg_collation	2320
51.13. pg_constraint	2321
51.14. pg_conversion	2323
51.15. pg_database	2324
51.16. pg_db_role_setting	2325
51.17. pg_default_acl	2326
51.18. pg_depend	2326
51.19. pg_description	2328
51.20. pg_enum	2329
51.21. pg_event_trigger	2329
51.22. pg_extension	2330
51.23. pg_foreign_data_wrapper	2330
51.24. pg_foreign_server	2331
51.25. pg_foreign_table	2331
51.26. pg_index	2332
51.27. pg_héríte	2333
51.28. pg_init_privs	2334
51.29. pg_language	2334
51.30. pg_largeobject	2335
51.31. pg_largeobject_metadata	2335
51.32. pg_namespace	2336
51.33. pg_opclass	2336
51.34. pg_operator	2337
51.35. pg_opfamily	2338
51.36. pg_parameter_acl	2338
51.37. pg_partitioned_table	2339
51.38. pg_policy	2339
51.39. pg_proc	2340
51.40. pg_publication	2342
51.41. pg_publication_namespace	2343
51.42. pg_publication_rel	2343
51.43. pg_range	2344
51.44. pg_replication_origin	2344
51.45. pg_rewrite	2345
51.46. pg_seclabel	2345
51.47. pg_sequence	2346

51.48. pg_shdepend	2346
51.49. pg_shdescription	2347
51.50. pg_shseclabel	2348
51.51. pg_statistique	2348
51.52. pg_statistic_ext	2350
51.53. pg_statistic_ext_data	2350
51.54. pg_subscription	2351
51.55. pg_subscription_rel	2352
51.56. pg_tablespace	2353
51.57. pg_transform	2353
51.58. pg_trigger	2354
51.59. pg_ts_config	2355
51.60. pg_ts_config_map	2356
51.61. pg_ts_dict	2356
51.62. pg_ts_parser	2357
51.63. pg_ts_template	2357
51.64. pg_type	2358
51.65. pg_user_mapping	2361
52. Vues système	2362
52.1. Aperçu	2362
52.2. pg_available_extensions	2363
52.3. pg_available_extension_versions	2363
52.4. pg_backend_memory_contexts	2364
52.5. pg_config	2365
52.6. pg_cursors	2365
52.7. pg_file_settings	2366
52.8. pg_group	2366
52.9. pg_hba_file_rules	2367
52.10. pg_ident_file_mappings	2368
52.11. pg_indexes	2368
52.12. pg_locks	2369
52.13. pg_matviews	2371
52.14. pg_policies	2372
52.15. pg_prepared_statements	2372
52.16. pg_prepared_xacts	2373
52.17. pg_publication_tables	2374
52.18. pg_replication_origin_status	2374
52.19. pg_replication_slots	2375
52.20. pg_roles	2376
52.21. pg_rules	2377
52.22. pg_seclabels	2378
52.23. pg_sequences	2378
52.24. pg_settings	2379
52.25. pg_shadow	2381
52.26. pg_shmem_allocations	2382
52.27. pg_stats	2382
52.28. pg_stats_ext	2384
52.29. pg_stats_ext_exprs	2385
52.30. pg_tables	2387
52.31. pg_timezone_abbrevs	2387
52.32. pg_timezone_names	2388
52.33. pg_user	2388
52.34. pg_user_mappings	2389
52.35. pg_views	2389
52.36. pg_wait_events	2390
53. Protocole Frontend/Backend	2391
53.1. Aperçu	2391
53.2. Flux de messages	2392

53.3. Authentification SASL	2406
53.4. Protocole de réplication en continu	2408
53.5. Protocole de réplication en flux logique	2417
53.6. Types de données des messages	2419
53.7. Formats de messages	2420
53.8. Champs de messages d'erreur et d'avertissement	2436
53.9. Formats des messages de réplication logique	2438
53.10. Résumé des modifications depuis le protocole 2.0	2447
54. Conventions de codage PostgreSQL	2449
54.1. Mise en forme	2449
54.2. Signalement des erreurs au sein du serveur	2449
54.3. Guide de style des messages d'erreur	2453
54.4. Conventions de codage diverses	2457
55. Prise en charge de la langue maternelle	2459
55.1. Pour le traducteur	2459
55.2. À l'attention du programmeur	2461
56. Écriture d'un gestionnaire de langage procédural	2465
57. Écriture d'un wrapper de données externes	2467
57.1. Fonctions d'encapsulation de données externes	2467
57.2. Routines de rappel d'encapsulation de données externes	2467
57.3. Fonctions d'assistance pour l'encapsulation de données externes	2483
57.4. Planification des requêtes d'encapsulation de données étrangères	2484
57.5. Verrouillage des lignes dans les wrappers de données externes	2487
58. Écrire une méthode d'échantillonnage tabulaire	2489
58.1. Fonctions de support de la méthode d'échantillonnage	2489
59. Création d'un fournisseur de numérisation personnalisé	2492
59.1. Création de parcours de numérisation personnalisés	2492
59.2. Création de plans de numérisation personnalisés	2493
59.3. Exécution d'analyses personnalisées	2494
60. Optimiseur de requêtes génétiques	2497
60.1. La gestion des requêtes comme problème d'optimisation complexe	2497
60.2. Algorithmes génétiques	2497
60.3. Optimisation génétique des requêtes (GEQO) dans PostgreSQL	2498
60.4. Lectures complémentaires	2500
61. Définition de l'interface de méthode d'accès aux tables	2501
62. Définition de l'interface de méthode d'accès à l'index	2502
62.1. Structure de base de l'API pour les index	2502
62.2. Fonctions de la méthode d'accès à l'index	2505
62.3. Numérisation de l'index	2511
62.4. Considérations relatives au verrouillage des index	2512
62.5. Contrôles d'unicité de l'index	2513
62.6. Fonctions d'estimation des coûts indexés	2515
63. Journalisation anticipée pour les extensions	2518
63.1. Enregistrements WAL génériques	2518
63.2. Gestionnaires de ressources WAL personnalisés	2519
64. Méthodes d'accès à l'index intégrées	2521
64.1. Index des arbres B	2521
64.2. Index GiST	2527
64.3. Index SP-GiST	2544
64.4. Indices GIN	2557
64.5. Indices BRIN	2564
64.6. Index de hachage	2577
65. Stockage physique de la base de données	2579
65.1. Organisation des fichiers de base de données	2579
65.2. TOAST	2581
65.3. Carte de l'espace libre	2584
65.4. Carte de visibilité	2584
65.5. La bifurcation d'initialisation	2585

65.6. Mise en page de la base de données	2585
65.7. Tuples de type tas uniquement (HOT)	2588
66. Traitement des transactions	2589
66.1. Transactions et identifiants	2589
66.2. Transactions et verrouillage	2589
66.3. Sous-transactions	2589
66.4. Transactions en deux phases	2590
67. Déclarations du catalogue système et contenu initial	2591
67.1. Règles de déclaration du catalogue système	2591
67.2. Données initiales du catalogue système	2592
67.3. Format de fichier BKI	2597
67.4. Commandements BKI	2597
67.5. Structure du fichier Bootstrap BKI	2598
67.6. Exemple BKI	2599
68. Comment le planificateur utilise les statistiques	2600
68.1. Exemples d'estimation par lignes	2600
68.2. Exemples de statistiques multivariées	2605
68.3. Statistiques et sécurité du planificateur	2609
69. Format du manifeste de sauvegarde	2610
69.1. Objet de niveau supérieur du manifeste de sauvegarde	2610
69.2. Objet Fichier Manifeste de sauvegarde	2610
69.3. Objet de plage WAL du manifeste de sauvegarde	2611
VIII. Annexes	2612
A. Codes d'erreur PostgreSQL	2619
B. Prise en charge de la date et de l'heure	2628
B.1. Interprétation de la date et de l'heure saisies	2628
B.2. Gestion des horodatages invalides ou ambigus	2629
B.3. Mots clés Date/Heure	2630
B.4. Fichiers de configuration de date/heure	2631
B.5. Spécifications des fuseaux horaires POSIX	2632
B.6. Historique des unités	2634
B.7. Dates juliennes	2635
C. Mots clés SQL	2636
D. Conformité SQL	2661
D.1. Fonctionnalités prises en charge	2662
D.2. Fonctionnalités non prises en charge	2674
D.3. Limites XML et conformité à SQL/XML	2682
E. Notes de version	2686
E.1. Version 17.10	2686
E.2. Version 17.9	2692
E.3. Version 17.8	2693
E.4. Version 17.7	2698
E.5. Version 17.6	2704
E.6. Version 17.5	2711
E.7. Version 17.4	2716
E.8. Version 17.3	2717
E.9. Version 17.2	2723
E.10. Version 17.1	2724
E.11. Version 17	2729
E.12. Versions antérieures	2750
F. Modules et extensions supplémentaires fournis	2751
F.1. amcheck — outils de vérification de la cohérence des tables et des index	2753
F.2. auth_delay — pause en cas d'échec d'authentification	2759
F.3. auto_explain — enregistrer les plans d'exécution des requêtes lentes	2760
F.4. basebackup_to_shell — exemple de module « shell » pg_basebackup	2763
F.5. basic_archive — un exemple de module d'archivage WAL	2764
F.6. Bloom — Méthode d'accès à l'index du filtre Bloom	2765
F.7. btree_gin — Classes d'opérateurs GIN avec comportement d'arbre B	2769

F.8. btree_gist — Classes d'opérateurs GiST avec comportement d'arbre B	2770
F.9. citext — un type de chaîne de caractères insensible à la casse	2772
F.10. Cube — un type de données cubique multidimensionnel	2775
F.11. dblink — Connexion à d'autres bases de données PostgreSQL	2780
F.12. dict_int — exemple de dictionnaire de recherche plein texte pour les entiers	2812
F.13. dict_xsyn — exemple de dictionnaire de recherche plein texte de synonymes	2813
F.14. Distance terrestre — calculer les distances orthodromiques	2815
F.15. file_fdw — accéder aux fichiers de données du système de fichiers du serveur	2817
F.16. fuzzystmatch — déterminer les similarités et les distances entre chaînes de caractères	2820
F.17. hstore — Type de données clé/valeur hstore	2825
F.18. intagg — agrégateur et énumérateur d'entiers	2833
F.19. intarray — manipuler des tableaux d'entiers	2835
F.20. n'est pas — types de données pour les numéros normalisés internationaux (ISBN, EAN, UPC, etc.)	2839
F.21. lo — gérer les objets volumineux	2843
F.22. ltree — type de données hiérarchique arborescent	2845
F.23. pageinspect — inspection de bas niveau des pages de la base de données	2853
F.24. vérification du mot de passe — vérifier la robustesse du mot de passe	2864
F.25. pg_buffercache — inspecter l'état du cache de mémoire tampon PostgreSQL	2865
F.26. pgcrypto — fonctions cryptographiques	2869
F.27. pg_freespacemap — examiner la carte des espaces libres	2879
F.28. pg_prewarm — préchargement des données de relation dans les caches de mémoire tampon	2881
F.29. pgrowlocks — afficher les informations de verrouillage des lignes d'un tableau	2883
F.30. pg_stat_statements — Suivi des statistiques de planification et d'exécution SQL	2885
F.31. pgstattuple — obtenir des statistiques au niveau du tuple	2894
F.32. pg_surgery — effectuer une chirurgie de bas niveau sur les données relationnelles	2899
F.33. pg_trgm — prise en charge de la similarité de texte par correspondance de trigrammes	2901
F.34. pg_visibility — Informations et utilitaires de la carte de visibilité	2907
F.35. pg_walinspect — Inspection WAL de bas niveau	2909
F.36. postgres_fdw — accéder aux données stockées sur des serveurs PostgreSQL externes	2913
F.37. seg — un type de données pour les segments de ligne ou les intervalles à virgule flottante	2924
F.38. sepgsql — Sécurité SELinux, contrôle d'accès obligatoire (MAC) basé sur les étiquettes module	2927
F.39. spi — Interface de programmation serveur : fonctionnalités et exemples	2935
F.40. sslinfo — obtenir les informations SSL du client	2937
F.41. tablefunc — fonctions qui renvoient des tableaux (tableaux croisés et autres)	2939
F.42. tcn — une fonction de déclenchement pour notifier les écouteurs des modifications apportées au contenu du tableau	2949
F.43. test_decoding — Module de test/exemple basé sur SQL pour le décodage logique WAL	2951
F.44. tsm_system_rows — la méthode d'échantillonnage SYSTEM_ROWS pour TABLEAU D'EXEMPLE	2952
F.45. tsm_system_time — la méthode d'échantillonnage SYSTEM_TIME pour TABLESAM- PLE	2953
F.46. unaccent — un dictionnaire de recherche textuelle qui supprime les signes diacritiques	2954
F.47. uuid-oss — un générateur d'UUID	2957
F.48. xml2 — Fonctionnalités de requête XPath et XSLT	2959
G. Programmes supplémentaires fournis	2964
G.1. Applications clientes	2964
G.2. Applications serveur	2971
H. Projets externes	2972
H.1. Interfaces client	2972
H.2. Outils d'administration	2972
H.3. Langages procéduraux	2972
H.4. Extensions	2972
I. Le dépôt de code source	2973
I.1. Récupération du code source via Git	2973
J. Documentation	2974
J.1. DocBook	2974

J.2. Coffrets d'outils	2974
J.3. Création de la documentation avec Make	2976
J.4. Création de la documentation avec Meson	2978
J.5. Rédaction de la documentation	2978
J.6. Guide de style	2978
K. Limites de PostgreSQL	2981
L. Acronymes	2982
Glossaire M.	2989
N. Support des couleurs	3003
N° 1. Lorsque la couleur est utilisée	3003
N° 2. Configuration des couleurs	3003
O. Fonctionnalités obsolètes ou renommées	3004
O.1. Fichier recovery.conf fusionné dans postgresql.conf	3004
O.2. Rôles par défaut renommés en rôles prédéfinis	3004
O.3. pg_xlogdump renommé en pg_waldump	3004
O.4. pg_resetxlog renommé en pg_resetwal	3004
O.5. pg_receivexlog renommé en pg_receivewal	3004
Bibliographie	3006
Index	3008

Liste des figures

60.1. Structure d'un algorithme génétique	2498	64.1. Fonctionnement interne
d'un algorithme génétique	2562	
65.1. Mise en page	2587	

Liste des tables

4.1. Séquences d'échappement avec barre oblique inverse	36
4.2. Priorité des opérateurs (de la plus élevée à la plus basse)	41
5.1. Abréviations des privilèges ACL	79
5.2. Résumé des privilèges d'accès	80
8.1. Types de données	147
8.2. Types numériques	148
8.3. Types monétaires	154
8.4. Types de caractères	155
8.5. Types de caractères spéciaux	156
8.6. Types de données binaires	157
8.7. Octets échappés littéraux de bytea	158
8.8. Octets échappés de sortie bytea	158
8.9. Types de date/heure	159
8.10. Saisie de la date	160
8.11. Saisie de l'heure	161
8.12. Saisie du fuseau horaire	162
8.13. Saisie spéciale de la date et de l'heure	163
8.14. Styles d'affichage de la date et de l'heure	164
8.15. Conventions de classement des dates	164
8.16. Abréviations des unités d'intervalle ISO 8601	166
8.17. Entrée d'intervalle	167
8.18. Exemples de style de sortie par intervalles	168
8.19. Type de données booléen	168
8.20. Types géométriques	171
8.21. Types d'adresses réseau	174
8.22. Exemples d'entrée de type cidr	174
8.23. Types primitifs JSON et types PostgreSQL correspondants	183
8.24. Variables jsonpath	192
8.25. Accesseurs jsonpath	192
8.26. Types d'identificateurs d'objets	215
8.27. Pseudo-types	218
9.1. Opérateurs de comparaison	221
9.2. Prédicats de comparaison	221
9.3. Fonctions de comparaison	225
9.4. Opérateurs mathématiques	225
9.5. Fonctions mathématiques	227
9.6. Fonctions aléatoires	230
9.7. Fonctions trigonométriques	231
9.8. Fonctions hyperboliques	232
9.9. Fonctions et opérateurs de chaînes SQL	233
9.10. Autres fonctions et opérateurs de chaînes de caractères	236
9.11. Fonctions et opérateurs de chaînes binaires SQL	243
9.12. Autres fonctions de manipulation de chaînes binaires	245
9.13. Fonctions de conversion texte/chaîne binaire	246
9.14. Opérateurs de chaînes binaires	247
9.15. Fonctions de chaînes binaires	248
9.16. Opérateurs de correspondance d'expressions régulières	252
9.17. Atomes d'expressions régulières	258
9.18. Quantificateurs d'expressions régulières	259
9.19. Contraintes d'expressions régulières	259
9.20. Échappements de caractères dans les expressions régulières	261
9.21. Classe d'expressions régulières - Échappements abrégés	262
9.22. Échappements des contraintes d'expression régulière	262
9.23. Références arrières des expressions régulières	263
9.24. Lettres ARE à option intégrée	263

9.25. Équivalences des fonctions d'expressions régulières	267
9.26. Fonctions de mise en forme	268
9.27. Modèles de formatage de date et d'heure	269
9.28. Modificateurs de modèle pour la mise en forme de la date et de l'heure	271
9.29. Modèles de formatage numérique	273
9.30. Modificateurs de modèles pour la mise en forme numérique	275
9.31. Exemples de to_char	275
9.32. Opérateurs de date/heure	276
9.33. Fonctions Date/Heure	278
9.34. Variantes AT TIME FONE et AT LOCAL	289
9.35. Fonctions de support des énumérations	293
9.36. Opérateurs géométriques	294
9.37. Fonctions géométriques	297
9.38. Fonctions de conversion de type géométrique	298
9.39. Opérateurs d'adresses IP	301
9.40. Fonctions des adresses IP	302
9.41. Fonctions de l'adresse MAC	303
9.42. Opérateurs de recherche de texte	304
9.43. Fonctions de recherche de texte	305
9.44. Fonctions de débogage de la recherche de texte	309
9.45. Opérateurs JSON et JSONB	326
9.46. Opérateurs jsonb supplémentaires	327
9.47. Fonctions de création JSON	329
9.48. Fonctions de test SQL/JSON	331
9.49. Fonctions de traitement JSON	332
9.50. Opérateurs et méthodes jsonpath	344
9.51. Éléments d'expression de filtre jsonpath	348
9.52. Fonctions de requête SQL/JSON	350
9.53. Fonctions de séquence	358
9.54. Opérateurs de tableaux	363
9.55. Fonctions de tableaux	364
9.56. Opérateurs de tir	366
9.57. Opérateurs multigammes	368
9.58. Fonctions de plage	370
9.59. Fonctions multiplage	371
9.60. Fonctions agrégées à usage général	372
9.61. Fonctions agrégées pour les statistiques	376
9.62. Fonctions d'agrégation d'ensembles ordonnés	378
9.63. Fonctions agrégées d'ensembles hypothétiques	378
9.64. Opérations de groupement	379
9.65. Fonctions de fenêtrage à usage général	380
9.66. Fusion des fonctions de support	381
9.67. Fonctions génératrices de séries	387
9.68. Fonctions génératrices d'indices	389
9.69. Fonctions d'information de session	391
9.70. Fonctions de consultation des privilèges d'accès	394
9.71. Opérateurs aclitem	396
9.72. Fonctions de l'aclitem	396
9.73. Fonctions d'interrogation de la visibilité du schéma	397
9.74. Fonctions d'information du catalogue système	398
9.75. Propriétés de la colonne Index	403
9.76. Propriétés de l'index	403
9.77. Propriétés de la méthode d'accès à l'index	403
9.78. Drapeaux GUC	404
9.79. Informations sur les objets et fonctions d'adressage	404
9.80. Fonctions d'information des commentaires	405
9.81. Fonctions de vérification de la validité des données	405
9.82. Fonctions d'identification de transaction et d'informations d'instantané	406

9.83. Composants de l'instantané	407
9.84. Fonctions obsolètes d'identification de transaction et d'informations d'instantané	408
9.85. Fonctions d'information sur les transactions engagées	408
9.86. Fonctions de contrôle des données	409
9.87. pg_control_checkpoint Colonnes de sortie	409
9.88. pg_control_system Colonnes de sortie	410
9.89. pg_control_init Colonnes de sortie	410
9.90. pg_control_recovery Colonnes de sortie	410
9.91. Informations sur la version Fonctions	410
9.92. Fonctions de synthèse d'informations WAL	411
9.93. Fonctions de configuration	412
9.94. Fonctions de signalisation du serveur	412
9.95. Fonctions de contrôle de sauvegarde	414
9.96. Fonctions d'information de récupération	416
9.97. Fonctions de contrôle de récupération	417
9.98. Fonctions de synchronisation des instantanés	418
9.99. Fonctions de gestion de la réplication	419
9.100. Fonctions de taille des objets de base de données	422
9.101. Fonctions de localisation des objets de base de données	423
9.102. Fonctions de gestion du classement	424
9.103. Fonctions de partitionnement de l'information	424
9.104. Fonctions de maintenance de l'index	425
9.105. Fonctions génériques d'accès aux fichiers	426
9.106. Fonctions de verrouillage consultatif	428
9.107. Fonctions de déclenchement intégrées	429
9.108. Fonctions d'information de réécriture de tableau	432
12.1. Types de jetons par défaut de l'analyseur syntaxique	482
13.1. Niveaux d'isolation des transactions	506
13.2. Modes de verrouillage conflictuels	513
13.3. Verrous de ligne conflictuels	515
18.1. Paramètres IPC du système V	601
18.2. Utilisation des fichiers du serveur SSL	616
19.1. Modes de validation synchrone	644
19.2. Niveaux de gravité des messages	672
19.3. Clés et valeurs des entrées de journal JSON	679
19.4. Touche d'option courte	707
21.1. Rôles prédéfinis	736
23.1. Niveaux de regroupement des soins intensifs	756
23.2. Paramètres de regroupement des soins intensifs	757
23.3. Jeux de caractères PostgreSQL	760
23.4. Conversions intégrées de jeux de caractères client/serveur	765
23.5. Toutes les conversions de jeux de caractères intégrés	766
26.1. Matrice des fonctionnalités de haute disponibilité, d'équilibrage de charge et de réplication 800	
27.1. Vues statistiques dynamiques	821
27.2. Statistiques collectées : Vues	822
27.3. pg_stat_activity Afficher	825
27.4. Types d'événements d'attente	826
27.5. Événements d'attente de type Activité	827
27.6. Événements d'attente de type Bufferpin	828
27.7. Événements d'attente de type Client	828
27.8. Événements d'attente de type Extension	828
27.9. Événements d'attente de type Io	829
27.10. Événements d'attente de type IPC	831
27.11. Événements d'attente de type Verrouillage	834
27.12. Événements d'attente de type Lwlock	834
27.13. Événements d'attente de type Délai d'attente	837
27.14. pg_stat_replication Vue	839
27.15. pg_stat_replication_slots Vue	841

27.16. pg_stat_wal_receiver Vue	842
27.17. pg_stat_recovery_prefetch Vue	843
27.18. pg_stat_subscription Vue	843
27.19. pg_stat_subscription_stats Afficher	844
27.20. pg_stat_ssl Vue	844
27.21. Vue pg_stat_gssapi	845
27.22. Vue pg_stat_archiver	845
27.23. Vue pg_stat_io	846
27.24. pg_stat_bgwriter Vue	848
27.25. pg_stat_checkpointing Vue	849
27.26. pg_stat_wal Afficher	849
27.27. pg_stat_database Vue	850
27.28. pg_stat_database_conflicts Vue	852
27.29. pg_stat_all_tables Vue	852
27.30. pg_stat_all_indexes Afficher	854
27.31. pg_statio_all_tables Vue	855
27.32. pg_statio_all_indexes Afficher	855
27.33. pg_statio_all_sequences Vue	856
27.34. pg_stat_user_functions Vue	856
27.35. pg_stat_slru Vue	857
27.36. Fonctions statistiques supplémentaires	858
27.37. Fonctions statistiques par serveur dorsal	860
27.38. pg_stat_progress_analyze Afficher	861
27.39. ANALYSE Phases	862
27.40. pg_stat_progress_cluster Vue	862
27.41. Phases CLUSTER et VACUUM FULL	863
27.42. pg_stat_progress_copy Afficher	863
27.43. pg_stat_progress_create_index Vue	864
27.44. CRÉER UN INDEX Phases	865
27.45. pg_stat_progress_vacuum Vue	866
27.46. Phases du vide	867
27.47. pg_stat_progress_basebackup Afficher	868
27.48. Phases de sauvegarde de base	868
27.49. Sondes DTrace intégrées	869
27.50. Types définis utilisés dans les paramètres de sonde	875
29.1. MISE À JOUR Résumé de la transformation	899
32.1. Descriptions des modes SSL	1015
32.2. Utilisation des fichiers SSL Libpq/Client	1015
33.1. Fonctions objets volumineuses orientées SQL	1035
34.1. Correspondance entre les types de données PostgreSQL et les types de variables C	1051
34.2. Formats d'entrée valides pour PGTYPEStdate_from_asc	1069
34.3. Formats d'entrée valides pour PGTYPEStdate_fmt_asc	1071
34.4. Formats d'entrée valides pour rdefmtdate	1072
34.5. Formats d'entrée valides pour PGTYPEStimestamp_from_asc	1073
35.1. nom_catalogue_schéma_information Colonnes	1151
35.2. Colonnes d'autorisation des rôles administrables	1151
35.3. Colonnes applicables_roles	1151
35.4. Attributs Colonnes	1152
35.5. Colonnes des jeux de caractères	1154
35.6. check_constraint_routine_usage Colonnes	1155
35.7. check_constraints Colonnes	1156
35.8. classements Colonnes	1156
35.9. Colonnes d'applicabilité de l'ensemble de caractères de collation	1156
35.10. column_column_usage Colonnes	1157
35.11. column_domain_usage Colonnes	1157
35.12. options_colonnes Colonnes	1158
35.13. privilèges de colonne Colonnes	1158
35.14. column_udt_usage Colonnes	1159

35.15. Colonnes	1159	35.16. Utilisation des colonnes de
contrainte	1162	35.17. Utilisation des tables de contrainte
1163 35.18. Privilèges de type de données Colonnes	1164	35.19. Contraintes de
domaine Colonnes	1164	35.20. Utilisation des UDT de domaine
Colonnes	1165	35.21. Domaines
Colonnes	1165	35.22. Types d'éléments
Colonnes	1167	35.23. Rôles activés
Colonnes	1169	35.24. options_foreign_data_wrapper_options
Colonnes	1169	35.25. wrappers_foreign_data_wrappers
Colonnes	1170	35.26. options_foreign_server_options
Colonnes	1170	35.27. serveurs_foreign_servers
Colonnes	1171	35.28. options_foreign_table_options
Colonnes	1171	35.29. tables_foreign_tables
Colonnes	1171	35.30. utilisation_column_key
Colonnes	1172	35.31. paramètres
Colonnes	1173	35.32. contraintes_référentielles
Colonnes	1174	35.33. role_column_grants Colonnes
1175 35.34. role_routine_grants Colonnes	1176	35.35. role_table_grants
Colonnes	1176	35.36. role_udt_grants
Colonnes	1177	35.37. role_usage_grants
Colonnes	1177	35.38. routine_column_usage
Colonnes	1178	35.39. routine_privileges Colonnes
1179 35.40. routine_routine_usage Colonnes	1179	35.41. routine_sequence_usage
Colonnes	1180	35.42. routine_table_usage Colonnes
1180 35.43. routines Colonnes	1181	35.44. schemata
Colonnes	1185	35.45. sequences
Colonnes	1186	35.46. sql_features
Colonnes	1187	35.47. sql_implementation_info
Colonnes	1187	35.48. sql_parts Colonnes
1188 35.49. sql_sizing Colonnes	1188	35.50. table_constraints
Colonnes	1188	35.51. table_privileges
Colonnes	1189	35.52. Colonnes
tables	1190	35.53. Colonnes
transforms	1190	35.54. Colonnes
triggered_update_columns	1191	35.55. Colonnes
triggers	1192	35.56. Colonnes
udt_privileges	1193	35.57. Colonnes
usage_privileges	1194	35.58. Colonnes
user_defined_types	1194	35.59. Colonnes
user_mapping_options	1196	35.60. Colonnes
user_mappings	1196	35.61. 35.62. view_column_usage
Colonnes	1197	35.63. view_table_usage
Colonnes	1198	35.64. views
Colonnes	1198	36.1. Types
polymorphes	1207	36.2. Types C équivalents pour les types SQL
intégrés	1233	36.3. Stratégies d'arbre B
1271 36.4. Stratégies de hachage	1271	36.5. Stratégies d'arbre
R bidimensionnel GiST	1271	36.6. Stratégies ponctuelles SP-
GiST	1271	36.7. Stratégies de matrice
GIN	1272	36.8. Stratégies BRIN
Minmax	1272	

36.9. Fonctions de support de l'arbre B	1273
36.10. Fonctions de prise en charge du hachage	1273
36.11. Fonctions de support GiST	1273
36.12. Fonctions de support SP-GiST	1274
36.13. Fonctions de support GIN	1274
36.14. Fonctions de support BRIN	1275
38.1. Prise en charge du déclenchement d'événements par balise de commande	1307
41.1. Éléments de diagnostic disponibles	1364
41.2. Éléments de diagnostic d'erreur	1378
297. Politiques appliquées par type de commande	1779
298. Variables automatiques pgbench	2118
299. Opérateurs pgbench	2121
300. Fonctions pgbench	2123
51.1. Catalogues système	2308
51.2. Colonnes pg_aggregate	2310
51.3. Colonnes pg_am	2311
51.4. Colonnes pg_amop	2312
51.5. Colonnes pg_amprac	2313
51.6. Colonnes pg_attrdef	2313
51.7. Colonnes pg_attribute	2314
51.8. Colonnes pg_authid	2316
51.9. Colonnes pg_auth_members	2317
51.10. Colonnes pg_cast	2317
51.11. Colonnes pg_class	2318
51.12. Colonnes pg_collation	2321
51.13. pg_constraint Colonnes	2322
51.14. Colonnes pg_conversion	2323
51.15. Colonnes de la base de données pg	2324
51.16. Colonnes pg_db_role_setting	2325
51.17. Colonnes pg_default_acl	2326
51.18. Colonnes pg_depend	2326
51.19. pg_description Colonnes	2328
51.20. pg_enum Colonnes	2329
51.21. Colonnes pg_event_trigger	2329
51.22. Colonnes pg_extension	2330
51.23. Colonnes pg_foreign_data_wrapper	2330
51.24. Colonnes pg_foreign_server	2331
51.25. Colonnes de pg_foreign_table	2331
51.26. Colonnes pg_index	2332
51.27. pg_inherits Colonnes	2333
51.28. Colonnes pg_init_privs	2334
51.29. Colonnes pg_language	2334
51.30. pg_largeobject Colonnes	2335
51.31. Colonnes pg_largeobject_metadata	2336
51.32. pg_namespace Colonnes	2336
51.33. Colonnes pg_opclass	2336
51.34. pg_operator Colonnes	2337
51.35. Colonnes pg_opfamily	2338
51.36. pg_parameter_acl Colonnes	2338
51.37. pg_partitioned_table Colonnes	2339
51.38. pg_policy Colonnes	2339
51.39. Colonnes pg_proc	2340
51.40. pg_publication Colonnes	2342
51.41. pg_publication_namespace Colonnes	2343
51.42. pg_publication_rel Colonnes	2343
51.43. Colonnes pg_range	2344
51.44. Colonnes pg_replication_origin	2344
51.45. Colonnes pg_rewrite	2345

51.46. Colonnes pg_seclabel	2345
51.47. pg_sequence Colonnes	2346
51.48. Colonnes pg_shdepend	2347
51.49. pg_shdescription Colonnes	2348
51.50. pg_shseclabel Colonnes	2348
51.51. pg_statistic Colonnes	2349
51.52. Colonnes pg_statistic_ext	2350
51.53. Colonnes pg_statistic_ext_data	2351
51.54. Colonnes pg_subscription	2351
51.55. Colonnes pg_subscription_rel	2353
51.56. Colonnes de l'espace de tables pg	2353
51.57. pg_transform Colonnes	2353
51.58. pg_trigger Colonnes	2354
51.59. Colonnes pg_ts_config	2355
51.60. pg_ts_config_map Colonnes	2356
51.61. Colonnes pg_ts_dict	2356
51.62. pg_ts_parser Colonnes	2357
51.63. pg_ts_template Colonnes	2357
51.64. pg_type Colonnes	2358
51.65. Codes de catégorie de type	2361
51.66. Colonnes pg_user_mapping	2361
52.1. Vues système	2362
52.2. pg_available_extensions Colonnes	2363
52.3. pg_available_extension_versions Colonnes	2363
52.4. Colonnes pg_backend_memory_contexts	2364
52.5. Colonnes pg_config	2365
52.6. Colonnes pg_cursors	2365
52.7. pg_file_settings Colonnes	2366
52.8. Colonnes pg_group	2367
52.9. Colonnes pg_hba_file_rules	2367
52.10. pg_ident_file_mappings Colonnes	2368
52.11. Colonnes pg_indexes	2368
52.12. pg_locks Colonnes	2369
52.13. pg_matviews Colonnes	2372
52.14. pg_policies Colonnes	2372
52.15. pg_prepared_statements Colonnes	2373
52.16. pg_prepared_xacts Colonnes	2373
52.17. pg_publication_tables Colonnes	2374
52.18. Colonnes pg_replication_origin_status	2374
52.19. Colonnes pg_replication_slots	2375
52.20. Colonnes pg_roles	2377
52.21. Colonnes pg_rules	2377
52.22. pg_seclabels Colonnes	2378
52.23. pg_sequences Colonnes	2378
52.24. pg_settings Colonnes	2379
52.25. pg_shadow Colonnes	2381
52.26. pg_shmem_allocations Colonnes	2382
52.27. Colonnes pg_stats	2383
52.28. pg_stats_ext Colonnes	2384
52.29. Colonnes pg_stats_ext_exprs	2385
52.30. Colonnes pg_tables	2387
52.31. pg_timezone_abbrevs Colonnes	2387
52.32. pg_timezone_names Colonnes	2388
52.33. Colonnes pg_user	2388
52.34. Colonnes pg_user_mappings	2389
52.35. pg_views Colonnes	2389
52.36. Colonnes pg_wait_events	2390
64.1. Classes d'opérateurs GiST intégrées	2528

64.2. Classes d'opérateurs SP-GiST intégrées	2545
64.3. Classes d'opérateurs GIN intégrées	2558
64.4. Classes d'opérateurs BRIN intégrées	2565
64.5. Numéros de fonction et de support pour les classes d'opérateurs Minmax	2574
64.6. Numéros de fonction et de support pour les classes d'opérateurs d'inclusion	2574
64.7. Numéros de procédure et de support pour les classes d'opérateurs Bloom	2575
64.8. Numéros de procédure et de support pour les classes d'opérateurs minmax-multi	2576
65.1. Contenu de PGDATA	2579
65.2. Mise en page	2585
65.3. Mise en page des données d'en-tête de page	2586
65.4. Disposition des données d'en-tête de tuple Heap	2587
A.1. Codes d'erreur PostgreSQL	2619
B.1. Noms des mois	2630
B.2. Noms des jours de la semaine	2630
B.3. Modificateurs de champ Date/Heure	2630
C.1. Mots clés SQL	2636
F.1. Représentations externes du cube	2775
F.2. Opérateurs de cube	2775
F.3. Fonctions du cube	2776
F.4. Fonctions de distance terrestre basées sur le cube	2815
F.5. Opérateurs de distance terrestre ponctuels	2816
F.6. Opérateurs de magasin	2826
F.7. Fonctions hstore	2827
F.8. Fonctions intarray	2835
F.9. Opérateurs de tableau	2836
F.10. n'est pas Types de données	2839
F.11. n'est pas Fonctions	2840
F.12. Opérateurs d'arbres	2846
F.13. Fonctions ltree	2848
F.14. Colonnes pg_buffercache	2865
F.15. pg_buffercache_summary() Colonnes de sortie	2866
F.16. pg_buffercache_usage_counts() Colonnes de sortie	2866
F.17. Algorithmes pris en charge par crypt()	2870
F.18. Nombre d'itérations pour crypt()	2870
F.19. Vitesses des algorithmes de hachage	2871
F.20. pgrowlocks Colonnes de sortie	2883
F.21. pg_stat_statements Colonnes	2885
F.22. pg_stat_statements_info Colonnes	2889
F.23. Colonnes de sortie pgstattuple	2894
F.24. pgstattuple_approx Colonnes de sortie	2897
F.25. pg_trgm Fonctions	2901
F.26. pg_trgm Opérateurs	2902
F.27. seg Représentations externes	2925
F.28. Exemples d'entrées de segment valides	2925
F.29. Opérateurs Seg GiST	2925
F.30. Fonctions Sepgsql	2933
F.31. Fonctions tablefunc	2939
F.32. Paramètres de connexion	2946
F.33. Fonctions de génération d'UUID	2957
F.34. Fonctions renvoyant des constantes UUID	2958
F.35. Fonctions XML2	2959
F.36. Paramètres de la table xpath	2960
K.1. Limitations de PostgreSQL	2981

Liste d'exemples

8.1. Utilisation des types de caractères	156
8.2. Utilisation du type booléen	169
8.3. Utilisation des types de chaînes de bits	177
9.1. Feuille de style XSLT pour la conversion de la sortie SQL/XML en HTML 324	
10.1. Résolution de l'opérateur racine carrée	436
10.2. Résolution du type d'opérateur de concaténation de chaînes	437
10.3. Résolution des types d'opérateurs de valeur absolue et de négation	437
10.4. Type d'opérateur d'inclusion de tableau Résolution	438
10.5. Opérateur personnalisé sur un type de domaine	438
10.6. Résolution du type d'argument de la fonction d'arrondi	441
10.7. Résolution de la fonction variadique	441
10.8. Résolution du type de fonction de sous-chaîne	442
10.9. Conversion du type de stockage de caractères	443
10.10. Résolution de types avec des types sous-spécifiés dans une union	444
10.11. Résolution de type dans une union simple	444
10.12. Résolution de type dans une union transposée	445
10.13. Résolution de types dans une union imbriquée	445
11.1. Création d'un index partiel pour exclure les valeurs communes	454
11.2. Création d'un index partiel pour exclure les valeurs non pertinentes	455
11.3. Mise en place d'un index unique partiel	456
11.4. N'utilisez pas d'index partiels comme substitut au partitionnement	456
20.1. Exemple d'entrées dans pg_hba.conf	714
20.2. Exemple de fichier pg_ident.conf	718
32.1. Exemple de programme libpq 1	1019
32.2. Exemple de programme libpq 2	1021
32.3. Exemple de programme libpq 3	1024
33.1. Objets volumineux avec libpq : exemple de programme	1036
34.1. Exemple de programme SQLDA	1089
34.2. Programme ECPG : Accès aux objets volumineux	1103
40.1. Installation manuelle de PL/Perl	1345
41.1. Citation des valeurs dans les requêtes dynamiques	1362
41.2. Exceptions liées aux opérations UPDATE/INSERT	1377
41.3. Fonction de déclenchement PL/pgSQL	1391
41.4. Fonction de déclenchement PL/pgSQL pour l'audit	1392
41.5. Fonction de déclenchement de vue PL/pgSQL pour l'audit	1393
41.6. Fonction de déclenchement PL/pgSQL pour la gestion d'une table récapitulative	1394
41.7. Audit avec tableaux de transition	1396
41.8. Fonction de déclenchement d'événement PL/pgSQL	1398
41.9. Portage d'une fonction simple de PL/SQL vers PL/pgSQL	1406
41.10. Portage d'une fonction créant une autre fonction de PL/SQL vers PL/pgSQL	1407
41.11. Portage d'une procédure PL/SQL avec manipulation de chaînes et paramètres de sortie vers PL/pgSQL	1408
41.12. Portage d'une procédure PL/SQL vers PL/pgSQL	1410
F.1. Créer une table externe pour les journaux CSV PostgreSQL	2818
F.2. Créer une table étrangère avec une option sur une colonne	2819

Préface

Ce livre constitue la documentation officielle de PostgreSQL. Rédigé par les développeurs de PostgreSQL et d'autres bénévoles, en parallèle du développement du logiciel, il décrit toutes les fonctionnalités prises en charge officiellement par la version actuelle de PostgreSQL.

Pour rendre plus accessible la grande quantité d'informations sur PostgreSQL, ce livre est organisé en plusieurs parties. Chaque partie s'adresse à une catégorie d'utilisateurs différente, ou à des utilisateurs ayant différents niveaux d'expérience avec PostgreSQL :

- La première partie est une introduction informelle destinée aux nouveaux utilisateurs.

La deuxième partie décrit l'environnement du langage de requêtes SQL, notamment les types de données et les fonctions, ainsi que l'optimisation des performances au niveau utilisateur. Tout utilisateur de PostgreSQL devrait la lire.

La partie III décrit l'installation et l'administration du serveur. Toute personne exploitant un serveur PostgreSQL, que ce soit à titre privé ou pour le compte d'autrui, devrait lire cette partie.

- La partie IV décrit les interfaces de programmation des programmes clients PostgreSQL.

- La partie V contient des informations destinées aux utilisateurs avancés sur les capacités d'extensibilité du serveur.
Les sujets abordés incluent les types de données et les fonctions définis par l'utilisateur.

La partie VI contient des informations de référence sur les commandes SQL, les programmes clients et les programmes serveurs. Cette partie complète les autres parties grâce à des informations structurées, classées par commande ou programme.

- La partie VII contient diverses informations qui pourraient être utiles aux développeurs PostgreSQL.

1. Qu'est-ce que PostgreSQL ?

PostgreSQL est un système de gestion de base de données objet-relationnel (ORDBMS) basé sur POSTGRES, version 4.21 , développé au département d'informatique de l'université de Californie à Berkeley.

POSTGRES a été le pionnier de nombreux concepts qui ne sont devenus disponibles dans certains systèmes de bases de données commerciales que bien plus tard.

PostgreSQL est un descendant open source du code original de Berkeley. Il prend en charge une grande partie de la norme SQL et offre de nombreuses fonctionnalités modernes :

- Requêtes complexes •
- Clés étrangères •
- Déclencheurs
- Vues modifiables •
- Intégrité transactionnelle • Contrôle
- de concurrence multiversion

De plus, PostgreSQL peut être étendu par l'utilisateur de nombreuses manières, par exemple en ajoutant de nouveaux

- types de données • fonctions
- opérateurs •
- fonctions d'agrégation •
- méthodes d'indexation
- langages procéduraux

Grâce à sa licence libérale, PostgreSQL peut être utilisé, modifié et distribué gratuitement par quiconque, à des fins privées, commerciales ou académiques.

2. Un bref historique de PostgreSQL

¹ <https://dsf.berkeley.edu/postgres.html>

Le système de gestion de bases de données objet-relationnelles aujourd'hui connu sous le nom de PostgreSQL est dérivé de Le logiciel PostgreSQL a été développé à l'Université de Californie à Berkeley. Il bénéficie de plusieurs décennies de développement. PostgreSQL est aujourd'hui la base de données open source la plus avancée disponible sur le marché.

2.1. Le projet Berkeley POSTGRES

Le projet POSTGRES, dirigé par le professeur Michael Stonebraker, était financé par la Defense Advanced Research Projects Agency (DARPA), l'Army Research Office (ARO) et la National Science Center. La Fondation (NSF) et ESL, Inc. La mise en œuvre de POSTGRES a débuté en 1986. Les concepts initiaux du système ont été présentés dans [ston86], et la définition du modèle de données initial est apparue dans [rowe87]. La conception du système de règles à cette époque a été décrite dans [ston87a]. La justification et L'architecture du gestionnaire de stockage a été détaillée dans [ston87b].

Depuis, POSTGRES a connu plusieurs mises à jour majeures. Le premier système « de démonstration » est devenu Opérationnel en 1987 et présenté à la conférence ACM-SIGMOD de 1988. Version 1, décrite dans [ston90a] a été mis à la disposition de quelques utilisateurs externes en juin 1989. En réponse à une critique de la première règle Le système de règles ([ston89]) a été repensé ([ston90b]), et la version 2 a été publiée en juin. 1990 avec le nouveau système de règles. La version 3 est apparue en 1991 et a ajouté la prise en charge de plusieurs stockages. Des gestionnaires, un exécuteur de requêtes amélioré et un système de règles réécrit. Dans l'ensemble, les améliorations suivantes ont été apportées. Les versions jusqu'à Postgres95 (voir ci-dessous) étaient axées sur la portabilité et la fiabilité.

POSTGRES a été utilisé pour mettre en œuvre de nombreuses applications de recherche et de production différentes. Ces systèmes comprennent : un système d'analyse de données financières, un progiciel de surveillance des performances des moteurs à réaction, une base de données de suivi des astéroïdes, une base de données d'informations médicales et plusieurs systèmes d'information géographique. POSTGRES a également été utilisé comme outil pédagogique dans plusieurs universités. Enfin, Illustration Information Technologies (fusionnée ultérieurement avec Informix2) , qui appartient désormais à IBM3) a récupéré le code et l'a commercialisé. Fin 1992, POSTGRES est devenu le principal gestionnaire de données pour Sequoia. Projet de calcul scientifique de 2000 décrit dans [ston92].

La taille de la communauté d'utilisateurs externes a presque doublé au cours de l'année 1993. Cela devenait de plus en plus évident que la maintenance et le support du code prototype prenaient beaucoup de temps qui devrait ont été consacrés à la recherche sur les bases de données. Afin d'alléger cette charge de travail, le projet Berkeley POST-GRES a officiellement pris fin avec la version 4.2.

2.2. Postgres95

En 1994, Andrew Yu et Jolly Chen ont ajouté un interpréteur de langage SQL à POSTGRES. Sous une nouvelle direction Sous ce nom, Postgres95 a ensuite été publié sur le web pour trouver sa propre voie dans le monde en tant que descendant open-source du code original POSTGRES de Berkeley.

Le code de Postgres95 était entièrement écrit en C ANSI et sa taille a été réduite de 25 %. De nombreuses modifications internes ont amélioré les performances et la maintenabilité. La version 1.0.x de Postgres95 était environ 30 à 50 % plus rapide sur... Comparaison des performances de Wisconsin Benchmark avec POSTGRES, version 4.2. Outre les corrections de bogues, les nouveautés suivantes Les principales améliorations étaient :

Le langage de requête PostQUEL a été remplacé par SQL (implémenté côté serveur). (La bibliothèque d'interface libpq a été nommée d'après PostQUEL.) Les sous-requêtes n'étaient pas prises en charge avant PostgreSQL (voir ci-dessous), mais il était possible de les simuler dans Postgres 95 grâce à des fonctions SQL définies par l'utilisateur. Les fonctions d'agrégation ont été réimplémentées. La prise en charge de la clause GROUP BY a également été ajoutée.

- Un nouveau programme (psql) a été fourni pour les requêtes SQL interactives, utilisant GNU Readline. a largement remplacé l'ancien programme de surveillance.
- Une nouvelle bibliothèque frontale, libpq, prenait en charge les clients basés sur Tcl. Un exemple de shell, pgsqlsh, était fourni. a fourni de nouvelles commandes Tcl pour interfacer les programmes Tcl avec le serveur Postgres95.

² <https://www.ibm.com/analytics/informix>

³ <https://www.ibm.com/>

- L'interface des objets volumineux a été entièrement repensée. L'inversion des objets volumineux était le seul mécanisme pour le stockage de fichiers volumineux. (Le système de fichiers à inversion a été supprimé.)
- Le système de règles au niveau de l'instance a été supprimé. Les règles restent disponibles en tant que règles de réécriture.
- Un court tutoriel présentant les fonctionnalités SQL classiques ainsi que celles de Postgres95 a été distribué avec le code source.
- GNU make (au lieu de BSD make) a été utilisé pour la compilation. De plus, Postgres95 a pu être compilé avec un GCC non patché (l'alignement des données des doubles a été corrigé).

2.3. PostgreSQL

En 1996, il devint évident que le nom « Postgres95 » ne résisterait pas à l'épreuve du temps. Nous avons donc opté pour un nouveau nom, PostgreSQL, afin de refléter la relation entre la version originale de POSTGRES et les versions plus récentes dotées de la fonctionnalité SQL. Parallèlement, nous avons rétabli la numérotation des versions à partir de 6.0, reprenant ainsi la séquence initialement établie par le projet Berkeley POSTGRES.

Postgres est toujours considéré comme un nom de projet officiel, à la fois par tradition et parce que les gens trouvent plus facile de prononcer Postgres que PostgreSQL.

Lors du développement de Postgres95, l'accent a été mis sur l'identification et la compréhension des problèmes existants dans le code serveur. Avec PostgreSQL, l'accent est désormais mis sur l'amélioration des fonctionnalités et des capacités, même si les travaux se poursuivent dans tous les domaines.

Des détails sur ce qui s'est passé dans PostgreSQL depuis lors peuvent être trouvés dans l'annexe E.

3. Conventions

Les conventions suivantes sont utilisées dans le synopsis d'une commande : les crochets ([et]) indiquent des éléments facultatifs. Les accolades ({ et }) et les barres verticales (|) indiquent qu'une seule alternative doit être choisie. Les points de suspension (...) signifient que l'élément précédent peut être répété. Tous les autres symboles, y compris les parenthèses, doivent être interprétés littéralement.

Lorsque cela améliore la clarté, les commandes SQL sont précédées de l'invite => et les commandes shell de l'invite \$. En règle générale, les invites ne sont cependant pas affichées.

Un administrateur est généralement la personne chargée d'installer et d'exploiter le serveur. Un utilisateur peut être toute personne utilisant, ou souhaitant utiliser, une partie quelconque du système PostgreSQL. Ces termes ne doivent pas être interprétés de manière restrictive ; cet ouvrage ne repose pas sur des présupposés concernant les procédures d'administration système.

4. Informations complémentaires

Outre la documentation, c'est-à-dire ce livre, il existe d'autres ressources sur PostgreSQL :

Wiki

Le wiki4 de PostgreSQL contient la liste FAQ5 (Foire aux questions) du projet, la liste TODO6 et des informations détaillées sur de nombreux autres sujets.

Site web

Le site web de PostgreSQL7 contient des détails sur la dernière version et d'autres informations pour rendre votre travail ou vos jeux avec PostgreSQL plus productifs.

⁴ <https://wiki.postgresql.org>

⁵ https://wiki.postgresql.org/wiki/Frequently_Asked_Questions

⁶ <https://wiki.postgresql.org/wiki/TODO>

⁷ <https://www.postgresql.org>

Listes de diffusion

Les listes de diffusion sont un excellent moyen de poser vos questions, d'échanger avec d'autres utilisateurs et de contacter les développeurs. Consultez le site web de PostgreSQL pour plus d'informations.

Toi-même!

PostgreSQL est un projet open source. À ce titre, son support continu repose sur la communauté d'utilisateurs. Lorsque vous commencerez à utiliser PostgreSQL, vous aurez besoin d'aide, que ce soit via la documentation ou les listes de diffusion. N'hésitez pas à partager vos connaissances. Consultez les listes de diffusion et répondez aux questions. Si vous découvrez quelque chose qui n'est pas documenté, documentez-le et partagez-le. Si vous ajoutez des fonctionnalités au code, contribuez-y également.

5. Directives pour le signalement des bogues

Lorsque vous découvrez un bug dans PostgreSQL, nous souhaitons en être informés. Vos rapports de bugs contribuent grandement à améliorer la fiabilité de PostgreSQL, car même avec le plus grand soin, il est impossible de garantir que chaque composant de PostgreSQL fonctionnera sur toutes les plateformes et en toutes circonstances.

Les suggestions suivantes visent à vous aider à rédiger des rapports de bogues qui pourront être traités efficacement. Leur application n'est pas obligatoire, mais elle est généralement avantageuse pour tous.

Nous ne pouvons pas garantir la correction immédiate de tous les bugs. Si le bug est évident, critique ou affecte un grand nombre d'utilisateurs, il y a de fortes chances qu'il soit examiné. Il se peut également que nous vous demandions de mettre à jour votre application vers une version plus récente afin de vérifier si le bug persiste. Ou encore, nous pourrions décider que le bug ne peut être corrigé avant la finalisation d'une refonte majeure que nous prévoyons. Enfin, il se peut tout simplement que la correction soit trop complexe et que nous ayons d'autres priorités. Si vous avez besoin d'une assistance immédiate, envisagez de souscrire un contrat de support commercial.

5.1. Identification des bogues

Avant de signaler un bogue, veuillez lire et relire la documentation pour vérifier que vous pouvez bien effectuer l'action que vous souhaitez réaliser. Si la documentation n'est pas claire quant à la possibilité d'effectuer une action, veuillez également le signaler ; il s'agit d'un bogue. Si un programme se comporte différemment de ce qui est indiqué dans la documentation, il s'agit d'un bogue. Cela peut inclure, sans s'y limiter, les circonstances suivantes :

- Un programme se termine par un signal fatal ou un message d'erreur du système d'exploitation indiquant un problème au sein du programme.
(Un contre-exemple pourrait être un message « disque plein », car il vous incombe alors de résoudre le problème.)

- Un programme produit une sortie incorrecte pour toute entrée donnée.

- Un programme refuse d'accepter une entrée valide (telle que définie dans la documentation).

Un programme accepte des entrées invalides sans afficher d'avertissement ni de message d'erreur. Toutefois, gardez à l'esprit que votre conception d'une entrée invalide peut correspondre à notre conception d'une extension ou d'une compatibilité avec les pratiques traditionnelles.

- PostgreSQL ne parvient pas à compiler, à construire ou à s'installer conformément aux instructions sur les plateformes prises en charge.

Ici, le terme « programme » désigne tout fichier exécutable, et pas seulement le processus en arrière-plan.

Une lenteur ou une forte consommation de ressources ne constituent pas forcément un bug. Consultez la documentation ou demandez de l'aide sur une liste de diffusion pour optimiser vos applications. Le non-respect de la norme SQL n'est pas non plus nécessairement un bug, sauf si la conformité à la fonctionnalité concernée est explicitement revendiquée.

Avant de continuer, consultez la liste des tâches à faire et la FAQ pour vérifier si votre bogue est déjà connu.

Si vous ne parvenez pas à déchiffrer les informations de la liste des tâches à faire, veuillez signaler le problème. Nous pourrions au moins améliorer la clarté de cette liste.

5.2. Éléments à signaler

Le plus important concernant le signalement de bogues est de s'en tenir aux faits, et uniquement aux faits. Évitez les suppositions sur ce qui a mal fonctionné, sur le comportement apparent du programme ou sur la partie défectueuse. Si vous n'êtes pas familier avec son implémentation, vos suppositions risquent d'être erronées et ne nous seront d'aucune utilité. Même si vous l'êtes, les explications argumentées complètent les faits, mais ne les remplacent pas. Pour corriger un bogue, nous devons d'abord le constater par nous-mêmes. Signaler les faits bruts est relativement simple (un simple copier-coller depuis l'écran suffit généralement), mais trop souvent, des détails importants sont omis, car on pense qu'ils sont sans importance ou que le rapport sera compris de toute façon.

Chaque rapport de bogue doit contenir les éléments suivants :

- La séquence exacte des étapes, depuis le démarrage du programme, nécessaires pour reproduire le problème. Cette séquence doit être complète et autonome ; il ne suffit pas d'envoyer une simple instruction `SELECT` sans les instructions `CREATE TABLE` et `INSERT` préalables, si le résultat doit dépendre des données des tables. Nous n'avons pas le temps de reconstituer le schéma de votre base de données, et si nous devions créer nos propres données, nous risquerions de ne pas identifier le problème.

Le format idéal pour un cas de test relatif aux problèmes SQL est un fichier exécutable via l'interface `psql` qui reproduit le problème. (Assurez-vous que votre fichier de configuration `~/.psqlrc` est vide.)

Pour créer facilement ce fichier, utilisez `pg_dump` afin d'exporter les déclarations de tables et les données nécessaires à la configuration initiale, puis ajoutez la requête problématique. Il est conseillé de minimiser la taille de votre exemple, mais ce n'est pas indispensable. Si le bogue est reproductible, nous le trouverons de toute façon.

Si votre application utilise une autre interface client, comme PHP, veuillez isoler les requêtes problématiques. Nous ne pourrions probablement pas mettre en place de serveur web pour reproduire le problème. Dans tous les cas, veuillez fournir les fichiers d'entrée exacts ; ne supposez pas que le problème se produit avec des « fichiers volumineux » ou des « bases de données de taille moyenne », etc., car ces informations sont trop imprécises pour être utiles.

- Veuillez indiquer le résultat obtenu. Ne dites pas simplement que le programme « n'a pas fonctionné » ou « a planté ». Si un message d'erreur s'affiche, veuillez le fournir, même si vous ne le comprenez pas. Si le programme se termine par une erreur du système d'exploitation, veuillez préciser laquelle. Si rien ne se passe, veuillez également le signaler. Même si le résultat de votre test est un plantage du programme ou un comportement évident, il est possible qu'il ne se produise pas sur notre plateforme. Le plus simple est de copier la sortie du terminal, si possible.

Note

Si vous signalez un message d'erreur, veuillez obtenir la version la plus détaillée possible de ce message. Dans `psql`, utilisez la commande `\set VERBOSITY verbose` au préalable. Si vous extrayez le message du journal du serveur, définissez le paramètre d'exécution `log_error_verbosity` sur `verbose` afin que tous les détails soient consignés.

Note

En cas d'erreur fatale, le message d'erreur renvoyé par le client peut être incomplet. Veuillez également consulter les journaux du serveur de base de données. Si vous ne conservez pas ces journaux, il serait judicieux de commencer dès maintenant.

- Il est essentiel de préciser le résultat attendu. Si vous écrivez simplement « Cette commande me donne ce résultat. » ou « Ce n'est pas ce à quoi je m'attendais. », nous pourrions l'exécuter nous-mêmes, parcourir le résultat et penser qu'il est correct et conforme à nos attentes. Nous ne devrions pas avoir à perdre de temps à décrypter la sémantique exacte de vos commandes. Évitez surtout de dire simplement « Ce n'est pas ce que SQL attend/ce qu'Oracle fait. » Déterminer le comportement correct de SQL est une tâche complexe.

De plus, nous ne savons pas tous comment se comportent les autres bases de données relationnelles. (Si votre problème est un plantage de programme, vous pouvez évidemment omettre ce point.)

- Veuillez indiquer toutes les options de ligne de commande et autres options de démarrage, y compris les variables d'environnement et les fichiers de configuration que vous avez modifiés par rapport aux valeurs par défaut. Merci de fournir des informations précises. Si vous utilisez une distribution préconfigurée qui démarre le serveur de base de données au démarrage du système, vous devriez essayer de découvrir comment cela fonctionne.
- Toute action que vous avez effectuée différemment des instructions d'installation.
- La version de PostgreSQL. Vous pouvez utiliser la commande ``SELECT version();`` pour connaître la version du serveur auquel vous êtes connecté. La plupart des programmes exécutables prennent également en charge l'option ``--version`` ; par exemple, ``postgres --version`` et ``psql --version`` devraient fonctionner. Si cette fonction ou ces options n'existent pas, votre version est probablement trop ancienne pour justifier une mise à niveau. Si vous utilisez une version pré-packagée, comme un paquet RPM, veuillez le préciser, en incluant toute sous-version. S'il s'agit d'un instantané Git, veuillez le mentionner, en incluant le hachage du commit.

Si votre version est antérieure à la 17.10, nous vous recommanderons très probablement de la mettre à jour. Chaque nouvelle version apporte de nombreuses corrections de bogues et améliorations ; il est donc fort probable qu'un bogue rencontré dans une version antérieure de PostgreSQL ait déjà été résolu. Notre assistance pour les sites utilisant d'anciennes versions de PostgreSQL est limitée ; si vos besoins dépassent nos capacités, envisagez de souscrire un contrat d'assistance commerciale.

- Informations sur la plateforme. Cela inclut le nom et la version du noyau, la bibliothèque C, le processeur, les informations sur la mémoire, etc. Dans la plupart des cas, il suffit d'indiquer le fournisseur et la version, mais ne présumez pas que tout le monde connaisse la composition exacte de « Debian » ni que tous les utilisateurs travaillent sur une architecture x86_64. En cas de problèmes d'installation, des informations sur la chaîne d'outils présente sur votre machine (compilateur, make, etc.) sont également nécessaires.

N'ayez crainte si votre rapport de bogue est long. C'est tout à fait normal. Il vaut mieux tout signaler dès le départ plutôt que de devoir vous demander des précisions. Par ailleurs, si vos fichiers d'entrée sont volumineux, il est judicieux de demander au préalable si quelqu'un serait intéressé à les examiner. Voici un article [qui propose d'autres conseils pour signaler les bogues](#).

Ne perdez pas votre temps à chercher quelles modifications des données d'entrée font disparaître le problème. Cela ne vous aidera probablement pas à le résoudre. Si le bug ne peut être corrigé immédiatement, vous aurez encore le temps de trouver et de partager une solution de contournement. De même, ne perdez pas votre temps à deviner l'origine du bug. Nous la découvrirons bien assez tôt.

Lors de la rédaction d'un rapport de bogue, veuillez éviter toute confusion terminologique. Le logiciel est appelé « PostgreSQL », parfois simplement « Postgres ». Si vous faites référence au processus backend, veuillez le préciser ; ne dites pas simplement « PostgreSQL plante ». Le plantage d'un processus backend est très différent du plantage du processus parent « postgres ». Veuillez ne pas dire « le serveur a planté » lorsqu'un processus backend s'est arrêté, ni l'inverse. De plus, les programmes clients, tels que l'interface interactive « psql », sont totalement indépendants du backend. Veuillez indiquer clairement si le problème se situe côté client ou côté serveur.

5.3. Où signaler les bogues

En règle générale, veuillez envoyer les rapports de bogues à la liste de diffusion dédiée à l'adresse [<pgsql-bugs@lists.postgresql.org>](mailto:pgsql-bugs@lists.postgresql.org). Il vous est demandé d'utiliser un objet descriptif pour votre message électronique, pouvant éventuellement contenir des extraits du message d'erreur.

Une autre méthode consiste à remplir le formulaire de signalement de bogue disponible sur le site web du projet⁹. L'envoi d'un rapport de bogue de cette manière entraîne son envoi par courriel à la liste de diffusion [<pgsql-bugs@lists.postgresql.org>](mailto:pgsql-bugs@lists.postgresql.org).

⁸ <https://www.chiark.greenend.org.uk/~sgtatham/bugs.html>

⁹ <https://www.postgresql.org/>

Si votre rapport de bogue a des implications en matière de sécurité et que vous préférez qu'il ne devienne pas immédiatement visible dans les archives publiques, ne l'envoyez pas à `pgsql-bugs`. Les problèmes de sécurité peuvent être signalés de manière privée à `<security@postgresql.org>`.

N'envoyez pas de rapports de bogues aux listes de diffusion des utilisateurs, telles que `<pgsql-sql@lists.postgresql.org>` ou `<pgsql-general@lists.postgresql.org>`. Ces listes de diffusion sont destinées à...

répondre aux questions des utilisateurs, et leurs abonnés ne souhaitent généralement pas recevoir de rapports de bogues. Plus important encore, il est peu probable qu'ils les réparent.

De plus, merci de ne pas envoyer de rapports à la liste de diffusion des développeurs `<pgsql-hackers@lists.postgresql.org>`. Cette liste est dédiée aux discussions sur le développement de PostgreSQL, et il serait préférable que nous...

Nous pourrions séparer les rapports de bogues. Nous pourrions choisir d'aborder la discussion concernant votre rapport de bogue sur `pgsql-hackers`, si le problème nécessite un examen plus approfondi.

Si vous rencontrez un problème avec la documentation, le meilleur endroit pour le signaler est l'adresse e-mail dédiée à la documentation. Veuillez envoyer votre demande à `<pgsql-docs@lists.postgresql.org>`. Veuillez préciser quelle partie de la documentation vous pose problème.

Si votre bogue est un problème de portabilité sur une plateforme non prise en charge, envoyez un courriel à `<pgsql-hacker-s@lists.postgresql.org>`, afin que nous (et vous) puissions travailler sur le portage de PostgreSQL vers votre plateforme.

Note

En raison de la quantité regrettable de spams qui circulent, toutes les listes ci-dessus seront modérées.

Sauf si vous êtes abonné. Dans ce cas, la réception de l'e-mail peut prendre un certain temps.

Si vous souhaitez vous abonner aux listes, veuillez consulter le site <https://lists.postgresql.org/> pour obtenir des instructions.

Partie I. Tutoriel

Bienvenue dans ce tutoriel PostgreSQL. Il a pour but de vous familiariser avec PostgreSQL, les concepts des bases de données relationnelles et le langage SQL. Une connaissance générale de l'informatique est requise, mais aucune expérience particulière en Unix ou en programmation n'est nécessaire. Ce tutoriel vise à vous faire découvrir concrètement les aspects importants du système PostgreSQL. Il ne prétend pas être exhaustif.

Une fois ce tutoriel terminé, nous vous recommandons de consulter la partie II pour approfondir vos connaissances du langage SQL, ou la partie IV pour en savoir plus sur le développement d'applications avec PostgreSQL. Les personnes qui gèrent leur propre installation PostgreSQL devraient également consulter la partie III.

Table des matières

1. Pour commencer	3
1.1. Installation	3
1.2. Principes fondamentaux de l'architecture	3
1.3. Création d'une base de données	3
1.4. Accès à une base de données	5
2. Le langage SQL	7
2.1. Introduction	7
2.2. Concepts	7
2.3. Création d'une nouvelle table	7
2.4. Remplissage d'un tableau avec des lignes	8
2.5. Interroger une table	9
2.6. Jointures entre tables	11
2.7. Fonctions agrégées	13
2.8. Mises à jour	15
2.9. Suppressions	15
3. Fonctionnalités avancées	17
3.1. Introduction	17
3.2. Vues	17
3.3. Clés étrangères	17
3.4. Transactions	18
3.5. Fonctions de fenêtrage	20
3.6. Héritage	23
3.7. Conclusion	24

Chapitre 1. Pour commencer

1.1. Installation

Avant de pouvoir utiliser PostgreSQL, il est nécessaire de l'installer. Il est possible que PostgreSQL soit déjà installé sur votre système, soit parce qu'il est inclus dans votre distribution système, soit parce que votre administrateur système l'a installé. Dans ce cas, veuillez consulter la documentation de votre système d'exploitation ou contacter votre administrateur système pour obtenir des informations sur la procédure d'accès à PostgreSQL.

Si vous n'êtes pas certain que PostgreSQL soit déjà disponible ou qu'il convienne à votre projet expérimental, vous pouvez l'installer vous-même. C'est simple et cela peut constituer un bon exercice. PostgreSQL peut être installé par n'importe quel utilisateur sans privilèges ; aucun accès superutilisateur (root) n'est requis.

Si vous installez PostgreSQL vous-même, reportez-vous au chapitre 17 pour les instructions d'installation et revenez à ce guide une fois l'installation terminée. Suivez attentivement la section relative à la configuration des variables d'environnement.

Si votre administrateur système n'a pas configuré l'environnement par défaut, vous devrez peut-être effectuer des manipulations supplémentaires. Par exemple, si le serveur de base de données est une machine distante, vous devrez définir la variable d'environnement PGHOST avec le nom de cette machine. La variable d'environnement PGPORT devra peut-être également être définie. En résumé : si vous essayez de lancer une application et qu'elle indique qu'elle ne peut pas se connecter à la base de données, consultez votre administrateur système ou, si vous êtes vous-même administrateur, la documentation afin de vous assurer que votre environnement est correctement configuré. Si le paragraphe précédent n'est pas clair, veuillez lire la section suivante.

1.2. Principes fondamentaux de l'architecture

Avant de poursuivre, il est important de comprendre l'architecture système de base de PostgreSQL. Comprendre comment les différents composants de PostgreSQL interagissent permettra de mieux appréhender ce chapitre.

En termes de bases de données, PostgreSQL utilise un modèle client/serveur. Une session PostgreSQL se compose des processus (programmes) suivants qui coopèrent :

- Un processus serveur gère les fichiers de la base de données, accepte les connexions des applications clientes et effectue des opérations sur la base de données pour le compte de ces dernières. Le programme serveur de base de données s'appelle PostgreSQL.

L'application cliente (interface utilisateur) qui effectue des opérations sur la base de données. Les applications clientes peuvent être très diverses : un client peut être un outil en mode texte, une application graphique, un serveur web accédant à la base de données pour afficher des pages web, ou encore un outil spécialisé de maintenance de base de données. Certaines applications clientes sont fournies avec la distribution PostgreSQL ; la plupart sont développées par les utilisateurs.

Comme c'est souvent le cas pour les applications client/serveur, le client et le serveur peuvent se trouver sur des machines différentes. Dans ce cas, ils communiquent via une connexion réseau TCP/IP. Il est important d'en tenir compte, car les fichiers accessibles sur une machine cliente peuvent ne pas l'être (ou ne l'être que sous un nom différent) sur la machine du serveur de base de données.

Le serveur PostgreSQL peut gérer plusieurs connexions simultanées de clients. Pour ce faire, il crée un nouveau processus (« fork ») pour chaque connexion. Dès lors, le client et le nouveau processus serveur communiquent sans intervention du processus PostgreSQL d'origine. Ainsi, le processus serveur superviseur est toujours actif, en attente de connexions client, tandis que les processus client et serveur associés sont temporaires. (Tout ceci est bien sûr invisible pour l'utilisateur. Nous le mentionnons ici par souci d'exhaustivité.)

1.3. Création d'une base de données

Pour vérifier l'accès au serveur de base de données, commencez par tenter d'en créer une. Un serveur PostgreSQL en fonctionnement peut gérer plusieurs bases de données. Généralement, une base de données distincte est utilisée pour chaque projet ou chaque utilisateur.

Il est possible que l'administrateur de votre site ait déjà créé une base de données à votre disposition. Dans ce cas, vous pouvez ignorer cette étape et passer directement à la section suivante.

Pour créer une nouvelle base de données, nommée ici mydb, utilisez la commande suivante :

```
$ createdb mydb
```

Si cela ne produit aucune réponse, cette étape a réussi et vous pouvez passer au reste de cette section.

Si vous voyez un message similaire à :

```
createdb : commande introuvable
```

PostgreSQL n'a donc pas été installé correctement. Soit il n'a pas été installé du tout, soit le chemin de recherche de votre shell ne l'inclut pas. Essayez d'exécuter la commande avec un chemin absolu :

```
$ /usr/local/pgsql/bin/createdb mydb
```

Le chemin d'accès sur votre site peut être différent. Contactez votre administrateur système ou consultez les instructions d'installation pour corriger le problème.

Une autre réponse possible serait la suivante :

```
createdb : erreur : la connexion au serveur sur le socket « /tmp/.s.PGSQL.5432 » a échoué :  
Aucun fichier ou répertoire de ce type. Le serveur est-il en cours d'exécution localement et  
accepte-t-il les connexions ?  
cette prise ?
```

Cela signifie que le serveur n'a pas démarré ou qu'il n'est pas à l'écoute à l'endroit où createdb s'attend à le contacter. Veuillez vérifier les instructions d'installation ou contacter l'administrateur.

Une autre réponse possible serait la suivante :

```
createdb : erreur : la connexion au serveur sur le socket « /tmp/.s.PGSQL.5432 » a échoué :  
ERREUR : le rôle « joe » n'existe pas
```

Votre nom d'utilisateur est mentionné. Cela se produit si l'administrateur n'a pas créé de compte utilisateur PostgreSQL pour vous. (Les comptes utilisateurs PostgreSQL sont différents des comptes utilisateurs du système d'exploitation.) Si vous êtes l'administrateur, consultez le chapitre 21 pour obtenir de l'aide sur la création de comptes. Vous devrez vous connecter avec le compte utilisateur du système d'exploitation sous lequel PostgreSQL a été installé (généralement « postgres ») pour créer le premier compte utilisateur. Il est également possible que votre nom d'utilisateur PostgreSQL soit différent de votre nom d'utilisateur système ; dans ce cas, vous devez utiliser l'option `-U` ou définir la variable d'environnement `PGUSER` pour spécifier votre nom d'utilisateur PostgreSQL.

Si vous possédez un compte utilisateur mais que celui-ci ne dispose pas des privilèges requis pour créer une base de données, vous verrez le message suivant :

```
createdb : erreur : la création de la base de données a échoué : ERREUR : autorisation  
Création de base de données refusée
```

Tous les utilisateurs ne sont pas autorisés à créer de nouvelles bases de données. Si PostgreSQL refuse de créer des bases de données pour vous, l'administrateur du site doit vous accorder cette autorisation. Dans ce cas, veuillez contacter votre administrateur. Si vous avez installé PostgreSQL vous-même, connectez-vous pour suivre ce tutoriel avec le compte utilisateur utilisé pour démarrer le serveur.¹

Vous pouvez également créer des bases de données avec d'autres noms. PostgreSQL vous permet de créer autant de bases de données que vous le souhaitez sur un même site. Les noms de bases de données doivent commencer par une lettre et ne dépassent pas 63 octets. Il est pratique de créer une base de données portant le même nom que votre nom d'utilisateur. De nombreux outils utilisent ce nom par défaut, ce qui vous évite de le saisir. Pour créer cette base de données, il vous suffit de taper :

```
$ crééb
```

Si vous ne souhaitez plus utiliser votre base de données, vous pouvez la supprimer. Par exemple, si vous êtes le propriétaire (créateur) de la base de données mydb, vous pouvez la supprimer à l'aide de la commande suivante :

```
$ dropdb mydb
```

(Pour cette commande, le nom de la base de données ne correspond pas par défaut au nom du compte utilisateur. Vous devez toujours le spécifier.) Cette action supprime définitivement tous les fichiers associés à la base de données et est irréversible ; elle ne doit donc être effectuée qu'avec la plus grande prudence.

Vous trouverez plus d'informations sur createdb et dropdb dans les documentations respectives de createdb et dropdb.

1.4. Accès à une base de données

Une fois la base de données créée, vous pouvez y accéder par :

- Exécuter le programme de terminal interactif PostgreSQL, appelé psql, qui permet de saisir, modifier et exécuter des commandes SQL de manière interactive.
- Utiliser un outil d'interface graphique existant comme pgAdmin ou une suite bureautique compatible ODBC ou JDBC pour créer et manipuler une base de données. Ces possibilités ne sont pas abordées dans ce tutoriel.
- Écrire une application personnalisée, en utilisant l'une des nombreuses interfaces de langage disponibles. Ces possibilités...

Les caractéristiques sont abordées plus en détail dans la partie IV.

Vous souhaitez probablement lancer psql pour tester les exemples de ce tutoriel. Vous pouvez l'activer pour la base de données mydb en saisissant la commande :

```
$ psql mydb
```

Si vous ne spécifiez pas le nom de la base de données, il s'agira par défaut de votre nom d'utilisateur. Vous avez déjà découvert ce principe dans la section précédente avec la base de données créée.

Dans psql, le message suivant s'affichera :

```
psql (17.10)
```

```
Tapez « help » pour obtenir de l'aide.
```

```
ma base de données =>
```

La dernière ligne pourrait également être :

¹ Voici pourquoi cela fonctionne : les noms d'utilisateur PostgreSQL sont distincts des comptes utilisateurs du système d'exploitation. Lorsque vous vous connectez à une base de données, vous pouvez choisir le nom d'utilisateur PostgreSQL à utiliser ; sinon, le nom par défaut sera celui de votre compte utilisateur actuel. Il existe toujours un compte utilisateur PostgreSQL portant le même nom que l'utilisateur du système d'exploitation qui a démarré le serveur, et cet utilisateur dispose toujours des droits nécessaires pour créer des bases de données. Au lieu de vous connecter avec ce compte utilisateur, vous pouvez également spécifier l'option `-U` pour choisir un autre nom d'utilisateur PostgreSQL.

mydb=#

Cela signifie que vous êtes superutilisateur de la base de données, ce qui est très probablement le cas si vous avez installé vous-même l'instance PostgreSQL. Être superutilisateur signifie que vous n'êtes pas soumis aux contrôles d'accès. Pour les besoins de ce tutoriel, cela n'a pas d'importance.

Si vous rencontrez des problèmes au démarrage de psql, reportez-vous à la section précédente. Les diagnostics de createdb et de psql sont similaires ; si le premier a fonctionné, le second devrait également fonctionner.

La dernière ligne affichée par psql est l'invite de commande ; elle indique que psql est à l'écoute et que vous pouvez saisir des requêtes SQL dans un espace de travail géré par psql. Essayez ces commandes :

mydb=> SELECT version();

version

PostgreSQL 17.10 sur x86_64-pc-linux-gnu, compilé par gcc (Debian 4.9.2-10) 4.9.2, 64 bits (1 ligne)

mydb=> SELECT current_date; date

2016-01-07

(1 rangée)

mydb=> SELECT 2 + 2; ?colonne?

4

(1 rangée)

Le programme psql comporte plusieurs commandes internes qui ne sont pas des commandes SQL. Elles commencent par une barre oblique inverse, « \ ». Par exemple, vous pouvez obtenir de l'aide sur la syntaxe de diverses commandes SQL PostgreSQL en saisissant :

mydb=> \h

Pour quitter psql, tapez :

mydb=> \q

psql se fermera et vous ramènera à votre invite de commandes. (Pour accéder à d'autres commandes internes, saisissez \? à l'invite psql.) La documentation complète de psql décrit toutes ses fonctionnalités. Dans ce tutoriel, nous n'utiliserons pas explicitement ces fonctionnalités, mais vous pouvez les utiliser vous-même lorsque cela s'avère utile.

Chapitre 2. Le langage SQL

2.1. Introduction

Ce chapitre présente un aperçu de l'utilisation de SQL pour effectuer des opérations simples. Ce tutoriel n'est qu'une introduction et ne constitue en aucun cas un guide complet de SQL. De nombreux ouvrages traitent de SQL, notamment [melt93] et [date97]. Notez que certaines fonctionnalités du langage PostgreSQL sont des extensions de la norme.

Dans les exemples qui suivent, nous supposons que vous avez créé une base de données nommée mydb, comme décrit dans le chapitre précédent, et que vous avez pu démarrer psql.

Les exemples de ce manuel se trouvent également dans la distribution source de PostgreSQL, dans le répertoire src/tutorial/. (Les distributions binaires de PostgreSQL peuvent ne pas contenir ces fichiers.) Pour utiliser ces fichiers, placez-vous d'abord dans ce répertoire et exécutez la commande make :

```
$ cd .../src/tutorial $ make
```

Cela crée les scripts et compile les fichiers C contenant les fonctions et les types définis par l'utilisateur. Ensuite, pour démarrer le tutoriel, procédez comme suit :

```
$ psql -s mydb
```

```
...
```

```
mydb=> \i basics.sql
```

La commande `\i` permet de lire les commandes depuis le fichier spécifié. L'option `-s` de psql active le mode pas à pas, qui marque une pause avant l'envoi de chaque instruction au serveur. Les commandes utilisées dans cette section se trouvent dans le fichier `basics.sql`.

2.2. Concepts

PostgreSQL est un système de gestion de bases de données relationnelles (SGBDR). Autrement dit, il s'agit d'un système permettant de gérer des données stockées dans des relations. Une relation est, en termes mathématiques, une table. L'idée de stocker des données dans des tables est aujourd'hui si répandue qu'elle peut sembler évidente, mais il existe d'autres façons d'organiser les bases de données. Les fichiers et répertoires des systèmes d'exploitation de type Unix constituent un exemple de base de données hiérarchique. Plus récemment, on trouve les bases de données orientées objet.

Chaque table est un ensemble nommé de lignes. Chaque ligne d'une même table possède le même ensemble de colonnes nommées, et chaque colonne est d'un type de données spécifique. Bien que l'ordre des colonnes soit fixe dans chaque ligne, il est important de noter que SQL ne garantit en aucun cas l'ordre des lignes au sein de la table (même s'il est possible de les trier explicitement pour l'affichage).

Les tables sont regroupées en bases de données, et un ensemble de bases de données est géré par un seul serveur PostgreSQL. Une instance de serveur constitue un cluster de base de données.

2.3. Création d'une nouvelle table

Vous pouvez créer une nouvelle table en spécifiant son nom, ainsi que tous les noms de colonnes et leurs types :

```
CRÉER UN TABLEAU météo (
    ville                varchar(80),
    temp_lo              entier,          -- basse température
    temp_hi              entier,          -- haute température
    date                 réel,            -- précipitations
    prcp                 date
);
```

Vous pouvez saisir ceci dans psql en respectant les sauts de ligne. psql reconnaîtra que la commande n'est pas... terminé jusqu'au point-virgule.

Les espaces blancs (c'est-à-dire les espaces, les tabulations et les sauts de ligne) peuvent être utilisés librement dans les commandes SQL. Cela signifie que vous pouvez saisir la commande différemment de ce qui précède, ou même sur une seule ligne. Deux tirets (« -- ») Insérer des commentaires. Tout ce qui suit est ignoré jusqu'à la fin de la ligne. SQL est insensible à la casse pour les mots clés et les identificateurs, sauf lorsque ces derniers sont placés entre guillemets doubles pour préserver la casse. (non fait ci-dessus).

varchar(80) spécifie un type de données pouvant stocker des chaînes de caractères arbitraires jusqu'à 80 caractères. en longueur. `int` est le type entier classique. `real` est un type permettant de stocker des nombres à virgule flottante simple précision. Le terme « date » devrait être explicite. (Oui, la colonne de type date s'appelle également « date ».) Cela peut être pratique ou déroutant — à vous de choisir.

PostgreSQL prend en charge les types SQL standard int, smallint, real, double precision, char(N), varchar(N), date, heure, horodatage et intervalle, ainsi que d'autres types de Une utilité générale et un riche ensemble de types géométriques. PostgreSQL peut être personnalisé avec une grande flexibilité. un certain nombre de types de données définis par l'utilisateur. Par conséquent, les noms de types ne sont pas des mots clés dans la syntaxe, sauf lorsque cela est nécessaire pour prendre en charge des cas particuliers dans la norme SQL.

Le deuxième exemple consistera à stocker les villes et leur situation géographique associée :

```
CRÉER TABLE villes (
    nom                  varchar(80),
    emplacement          indiquer
);
```

Le type point est un exemple de type de données spécifique à PostgreSQL.

Enfin, il convient de mentionner que si vous n'avez plus besoin d'une table ou si vous souhaitez la recréer différemment vous pouvez le supprimer à l'aide de la commande suivante :

```
SUPPRIMER LA TABLE nom_table;
```

2.4. Remplissage d'un tableau avec des lignes

L'instruction INSERT est utilisée pour remplir une table avec des lignes :

```
INSÉRER DANS weather VALUES ('San Francisco', 46, 50, 0.25,
    '1994-11-27');
```

Notez que tous les types de données utilisent des formats d'entrée assez explicites. Les constantes qui ne sont pas de simples valeurs numériques Il faut généralement l'entourer de guillemets simples ('), comme dans l'exemple. Le type de date est en fait assez Il est flexible quant aux formats acceptés, mais pour ce tutoriel, nous nous en tiendrons au format sans ambiguïté présenté ici.

Le type point nécessite une paire de coordonnées en entrée, comme indiqué ici :

```
INSÉRER DANS les villes VALEURS ('San Francisco', '(-194.0, 53.0)');
```

La syntaxe utilisée jusqu'à présent vous oblige à vous souvenir de l'ordre des colonnes. Une syntaxe alternative permet vous devez lister explicitement les colonnes :

```
INSÉRER DANS weather (ville, temp_lo, temp_hi, prcp, date)
VALEURS ('San Francisco', 43, 57, 0,0, '1994-11-29');
```

Vous pouvez lister les colonnes dans un ordre différent si vous le souhaitez, ou même omettre certaines colonnes, par exemple si les précipitations sont inconnues :

```
INSÉRER DANS weather (date, city, temp_hi, temp_lo)
VALEURS ('1994-11-29', 'Hayward', 54, 37);
```

De nombreux développeurs estiment qu'il est préférable de lister explicitement les colonnes plutôt que de se fier implicitement à leur ordre.

Veuillez saisir toutes les commandes indiquées ci-dessus afin de disposer de données pour la suite sections.

Vous auriez également pu utiliser la commande COPY pour charger de grandes quantités de données à partir de fichiers texte. C'est généralement le cas. plus rapide car la commande COPY est optimisée pour cette application, mais offre moins de flexibilité que INSERER. Par exemple :

```
COPIER la météo DEPUIS '/home/user/weather.txt';
```

où le nom du fichier source doit être disponible sur la machine exécutant le processus backend,

Ce n'est pas le client qui est concerné, car le processus backend lit directement le fichier. Les données insérées ci-dessus concernent la météo.

Le tableau peut également être inséré à partir d'un fichier contenant (les valeurs sont séparées par une tabulation) :

San Francisco	46	50	0,25	27 novembre 1994
San Francisco 37	43	57	0,0	29/11/1994
Hayward	54	\N		29 novembre 1994

Vous pouvez en savoir plus sur la commande COPY dans la documentation COPY.

2.5. Interroger une table

Pour extraire des données d'une table, on interroge cette table. Une instruction SQL SELECT est utilisée à cet effet. L'instruction est divisée en une liste de sélection (la partie qui liste les colonnes à retourner), une liste de table (la partie qui liste les tables à partir desquelles extraire les données), et une qualification facultative (la partie qui précise les restrictions éventuelles). Par exemple, pour récupérer toutes les lignes de la table weather, tapez :

```
SÉLECTIONNER * DE la météo;
```

Ici, * est un raccourci pour « toutes les colonnes ». ¹ On obtiendrait donc le même résultat avec :

```
SÉLECTIONNER ville, temp_lo, temp_hi, prcp, date DE météo;
```

Le résultat devrait être :

ville	temp_lo	temp_hi	prcp	date
-----+-----+-----+-----				

¹ Bien que SELECT * soit utile pour les requêtes improvisées, son utilisation est généralement considérée comme une mauvaise pratique en production, car l'ajout d'une colonne à la table est problématique. changerait les résultats.

San Francisco	46	50 0,25 27/11/1994
San Francisco	43	57 0 29/11/1994
Hayward (3 rangées)	37	54 29/11/1994

Vous pouvez saisir des expressions, et pas seulement de simples références de colonnes, dans la liste de sélection. Par exemple, vous pouvez faire:

SÉLECTIONNER ville, (temp_hi+temp_lo)/2 AS temp_avg, date DE météo;

Cela devrait donner :

ville	température_moyenne	date
San Francisco	48	27/11/1994
San Francisco	50	29 novembre 1994
Hayward (3 rangées)	45	29 novembre 1994

Notez comment la clause AS est utilisée pour renommer la colonne de sortie. (La clause AS est facultative.)

Une requête peut être « qualifiée » en ajoutant une clause WHERE qui spécifie les lignes souhaitées.

La clause WHERE contient une expression booléenne (valeur de vérité), et seules les lignes pour lesquelles la valeur booléenne est vraie sont prises en compte.

Les expressions vraies sont renvoyées. Les opérateurs booléens usuels (ET, OU et NON) sont autorisés.

qualification. Par exemple, la requête suivante permet de récupérer la météo de San Francisco les jours de pluie :

SÉLECTIONNER * DE la météo

OÙ ville = 'San Francisco' ET prcp > 0,0 ;

Résultat:

ville	temp_lo temp_hi prcp	date
San Francisco (1 rangée)	46	50 0,25 27/11/1994

Vous pouvez demander que les résultats d'une requête soient renvoyés par ordre trié :

SÉLECTIONNER * DE la météo

TRIER PAR ville ;

ville	temp_lo temp_hi prcp	date
Hayward	37	54 0 29 novembre 1994
San Francisco	43	57 29/11/1994
San Francisco	46	50 0,25 27/11/1994

Dans cet exemple, l'ordre de tri n'est pas entièrement spécifié, et vous pourriez donc obtenir les lignes de San Francisco dans les deux cas, vous obtiendrez toujours les résultats indiqués ci-dessus.

SÉLECTIONNER * DE la météo

TRIER PAR ville, temp_lo;

Vous pouvez demander la suppression des lignes en double du résultat d'une requête :

SÉLECTIONNER UNE VILLE DISTINCTE
DE LA météo ;

```

      ville
-----
Hayward
San Francisco
(2 rangées)

```

La encore, l'ordre des lignes de résultats peut varier. Pour garantir des résultats cohérents, utilisez conjointement `DISTINCT` et `ORDER BY` : ²

SÉLECTIONNER UNE VILLE DISTINCTE
D'après la météo
TRIER PAR ville ;

2.6. Jointures entre tables

Jusqu'à présent, nos requêtes n'ont accédé qu'à une seule table à la fois. Il est possible d'accéder à plusieurs tables simultanément, ou d'accéder à la même table de manière à traiter plusieurs lignes en même temps. Les requêtes qui accèdent à plusieurs tables (ou à plusieurs instances de la même table) simultanément sont appelées requêtes de jointure. Elles combinent les lignes d'une table avec celles d'une autre, à l'aide d'une expression spécifiant les lignes à apparier. Par exemple, pour renvoyer tous les enregistrements météorologiques ainsi que la localisation de la ville associée, la base de données doit comparer la colonne « ville » de chaque ligne de la table « météo » avec la colonne « nom » de toutes les lignes de la table « villes », et sélectionner les paires de lignes où ces valeurs correspondent. Ceci serait réalisé par la requête suivante :

SÉLECTIONNER * DE weather JOIN cities ON city = name;

ville emplacement	temp_lo temp_hi prcp	date	nom	
San Francisco	46	50 0,25 27/11/1994	San Francisco (-194,53)	
San Francisco	43	57	0 29 novembre 1994	San Francisco (-194,53) (2 lignes)

Observez deux choses concernant l'ensemble de résultats :

- Aucun résultat n'est disponible pour la ville de Hayward. En effet, aucune entrée correspondante n'existe dans la table des villes ; la jointure ignore donc les lignes non appariées de la table météo. Nous verrons prochainement comment résoudre ce problème.
- Deux colonnes contiennent le nom de la ville. Ceci est normal car les listes de colonnes des tables « météo » et « villes » sont concaténées. En pratique, ce comportement est toutefois déconseillé ; il est donc préférable de lister explicitement les colonnes de sortie plutôt que d'utiliser *.

² Dans certains systèmes de bases de données, notamment les anciennes versions de PostgreSQL, l'implémentation de `DISTINCT` ordonne automatiquement les lignes, rendant ainsi `ORDER BY` inutile. Cependant, cela n'est pas exigé par la norme SQL, et la version actuelle de PostgreSQL ne garantit pas que `DISTINCT` entraîne l'ordonnement des lignes.

Il ne s'agit que d'un modèle conceptuel. La jointure est généralement effectuée de manière plus efficace que la comparaison de chaque paire de lignes possible, mais cela est invisible pour l'utilisateur.

```
SÉLECTIONNER ville, temp_lo, temp_hi, prcp, date, emplacement
DE LA MÉDIATURE JOINTURE villes SUR ville = nom;
```

Comme les colonnes avaient toutes des noms différents, l'analyseur a automatiquement déterminé à quelle table elles appartenaient. Si les deux tables contenaient des noms de colonnes identiques, il faudrait les préciser .
pour indiquer lequel vous vouliez dire, comme dans :

```
SÉLECTIONNER weather.city, weather.temp_lo, weather.temp_hi,
météo.prcp, météo.date, villes.localisation
DE LA RENOUVELLEMENT météo JOIN villes SUR météo.ville = villes.nom;
```

Il est généralement admis qu'il est de bonne pratique de qualifier tous les noms de colonnes dans une requête de jointure, afin que la requête ne génère pas d'erreurs. échouer si un nom de colonne en double est ajouté ultérieurement à l'une des tables.

Les requêtes de jointure du type vu jusqu'ici peuvent également être écrites sous cette forme :

```
SÉLECTIONNER *
  À partir de la météo, des villes
  OÙ ville = nom;
```

Cette syntaxe est antérieure à la syntaxe JOIN/ON, introduite dans SQL-92. Les tables sont simplement
L'expression de comparaison est ajoutée à la clause WHERE et figure dans la clause FROM. Les résultats sont les suivants :
L'ancienne syntaxe implicite et la nouvelle syntaxe explicite JOIN/ON sont identiques. Mais pour un lecteur de
La requête, grâce à sa syntaxe explicite, en facilite la compréhension : la condition de jointure est introduite.
par son propre mot-clé alors qu'auparavant la condition était intégrée à la clause WHERE.
avec d'autres conditions.

Nous allons maintenant déterminer comment récupérer les enregistrements de Hayward. Voici ce que nous attendons de la requête :
Il s'agit de parcourir le tableau météorologique et, pour chaque ligne, de trouver la ou les lignes de villes correspondantes. Si aucune correspondance n'est trouvée, une recherche est effectuée.
Lorsqu'une ligne est trouvée, nous souhaitons substituer des « valeurs vides » aux colonnes du tableau des villes.
Ce type de requête est appelé une jointure externe. (Les jointures que nous avons vues jusqu'à présent sont des jointures internes.) La commande
Cela ressemble à ceci :

```
SÉLECTIONNER *
  DE la météo GAUCHE EXTERNE JOINTURE villes SUR météo.ville =
  villes.nom;
```

ville emplacement	temp_lo temp_hi prcp	date	nom
Hayward	37 54	29 novembre 1994	
San Francisco 46		50 0,25 27/11/1994	San
Francisco (-194,53)			
San Francisco 43	57	0 29 novembre 1994	San
Francisco (-194,53)			

(3 rangées)

Cette requête est appelée jointure externe gauche car la table mentionnée à gauche de l'opérateur de jointure sera
chacune de ses lignes apparaît au moins une fois dans le résultat, tandis que le tableau de droite ne contiendra que celles-ci.
Afficher les lignes correspondant à une ligne du tableau de gauche. Lors de l'affichage d'une ligne du tableau de gauche pour laquelle il existe une correspondance.
Aucune correspondance dans la table de droite, des valeurs vides (null) sont substituées aux colonnes de la table de droite.

Exercice : Il existe également des jointures externes droites et des jointures externes complètes. Essayez de découvrir à quoi elles servent.

On peut également joindre une table à elle-même. On parle alors d'une auto-jointure. Par exemple, supposons que nous souhaitions... trouver tous les relevés météorologiques dont la température se situe dans la plage d'autres relevés météorologiques. Nous devons donc... Comparez les colonnes temp_lo et temp_hi de chaque ligne météo aux colonnes temp_lo et temp_hi de toutes les autres lignes météo. Pour ce faire, utilisez la requête suivante :

```
SÉLECTIONNER w1.city, w1.temp_lo AS low, w1.temp_hi AS high,
                w2.city, w2.temp_lo AS basse, w2.temp_hi AS haute
DE la météo w1 REJOINDRE la météo w2
SUR w1.temp_lo < w2.temp_lo ET w1.temp_hi > w2.temp_hi;
```

ville	bas	haut	ville	bas	haut
San Francisco	43	57	San Francisco	46	50
Hayward	37	54	San Francisco	46	50

(2 rangées)

Nous avons renommé ici le tableau météorologique w1 et w2 afin de pouvoir distinguer le côté gauche et le côté droit de la jointure. Vous pouvez également utiliser ces types d'alias dans d'autres requêtes pour gagner du temps de saisie, par exemple :

```
SÉLECTIONNER *
DE LA MOYENNE météo w JOINTURE villes c SUR w.ville = c.nom;
```

Vous rencontrerez ce style d'abréviation assez fréquemment.

2.7. Fonctions agrégées

Comme la plupart des autres bases de données relationnelles, PostgreSQL prend en charge les fonctions d'agrégation . Cette fonction calcule un résultat unique à partir de plusieurs lignes d'entrée. Par exemple, elle permet d'obtenir le nombre, la somme, la moyenne, le maximum et le minimum sur un ensemble de lignes.

À titre d'exemple, nous pouvons trouver la valeur minimale de température la plus élevée possible avec :

```
SÉLECTIONNER max(temp_lo) DE weather;
```

```
max
-----
46
(1 rangée)
```

Si nous voulions savoir dans quelle ville (ou quelles villes) cette lecture a eu lieu, nous pourrions essayer :

```
SÉLECTIONNER la ville DE la base de données météo OÙ temp_lo = max(temp_lo);
```

FAUX

mais cela ne fonctionnera pas car la valeur maximale agrégée ne peut pas être utilisée dans la clause WHERE. (Cette restriction existe car la clause WHERE détermine quelles lignes seront incluses dans le calcul agrégé ; Il est donc évident qu'elle doit être évaluée avant le calcul des fonctions agrégées. Cependant, comme c'est souvent le cas, Dans ce cas, la requête peut être reformulée pour obtenir le résultat souhaité, ici en utilisant une sous-requête :

```
SÉLECTIONNEZ une ville DANS LA MÉTÉO
OÙ temp_lo = (SÉLECTIONNER max(temp_lo) DE weather);
```

```

      ville
-----
San Francisco
(1 rangée)

```

Ceci est acceptable car la sous-requête est un calcul indépendant qui calcule son propre agrégat séparément de ce qui se passe dans la requête externe.

Les agrégats sont également très utiles en combinaison avec les clauses GROUP BY. Par exemple, on peut obtenir le nombre de relevés et la température minimale maximale observée dans chaque ville grâce à :

```

SÉLECTIONNER ville, nombre(*), max(temp_lo)
D'après la météo
GROUPE PAR ville ;

```

```

      ville          | nombre | max
-----+-----+----
Hayward |          1 | 37 2 | 46
San Francisco | (2
rangées)

```

ce qui nous donne une ligne de résultat par ville. Chaque résultat agrégé est calculé sur les lignes du tableau correspondant à cette ville. Nous pouvons filtrer ces lignes groupées à l'aide de HAVING :

```

SÉLECTIONNER ville, nombre(*), max(temp_lo)
D'après la météo
GROUPE PAR ville
AYANT max(temp_lo) < 40 ;

```

```

      ville | nombre | max
-----+-----+----
Hayward | (1          1 | 37
rangée)

```

ce qui nous donne les mêmes résultats uniquement pour les villes dont toutes les valeurs de temp_lo sont inférieures à 40. Enfin, si nous ne nous intéressons qu'aux villes dont le nom commence par « S », nous pourrions procéder comme suit :

```

SÉLECTIONNER ville, nombre(*), max(temp_lo)
D'après la météo
OÙ la ville COMME 'S%'
GROUPE PAR ville ;

```

```

      ville          | nombre | max
-----+-----+----
San Francisco | (1 rangée)          2 | 46

```

1 L'opérateur LIKE effectue une correspondance de modèles et est expliqué dans la section 9.7.

Il est important de comprendre l'interaction entre les agrégats et les clauses WHERE et HAVING de SQL. La différence fondamentale entre WHERE et HAVING est la suivante : WHERE sélectionne les lignes d'entrée avant le calcul des groupes et des agrégats (elle contrôle donc quelles lignes sont prises en compte dans le calcul des agrégats), tandis que HAVING sélectionne les lignes de groupe après le calcul des groupes et des agrégats. Ainsi, la clause WHERE...

La clause ne doit pas contenir de fonctions d'agrégation ; il est absurde d'essayer d'utiliser une fonction d'agrégation pour déterminer quelles lignes serviront d'entrées aux agrégats. Par ailleurs, la clause HAVING contient toujours fonctions agrégées. (À proprement parler, vous pouvez écrire une clause HAVING qui n'utilise pas Les agrégats, mais c'est rarement utile. La même condition pourrait être utilisée plus efficacement dans la clause WHERE. scène.)

Dans l'exemple précédent, nous pouvons appliquer la restriction du nom de la ville dans WHERE, puisqu'elle ne nécessite aucune agrégation. C'est plus efficace que d'ajouter la restriction à HAVING, car nous évitons d'effectuer le regroupement. et les calculs agrégés pour toutes les lignes qui échouent au test WHERE.

Une autre façon de sélectionner les lignes à inclure dans un calcul d'agrégation consiste à utiliser FILTER, qui est un option par agrégat :

```
SELECT city, count(*) FILTER (WHERE temp_lo < 45), max(temp_lo)
```

D'après la météo

GROUPE PAR ville ;

ville	nombre	max
Hayward	1	37
San Francisco (2	1	46

rangées)

FILTER est très similaire à WHERE, à ceci près qu'il supprime uniquement les lignes de l'entrée de la fonction d'agrégation à laquelle il est associé. Ici, l'agrégation COUNT ne compte que les lignes avec temp_lo. en dessous de 45 ; mais l'agrégat maximal est toujours appliqué à toutes les lignes, il trouve donc toujours la valeur de 46.

2.8. Mises à jour

Vous pouvez mettre à jour les lignes existantes à l'aide de la commande UPDATE. Supposons que vous découvriez la température Toutes les mesures sont erronées de 2 degrés après le 28 novembre. Vous pouvez corriger les données comme suit :

Mise à jour météo

```
Définir temp_hi = temp_hi - 2, temp_lo = temp_lo - 2
OÙ date > '1994-11-28';
```

Voici le nouvel état des données :

SÉLECTIONNER * DE la météo;

ville	temp_lo	temp_hi	prcp	date
San Francisco	46	50	0,25	27/11/1994
San Francisco	41	55	0	29/11/1994
Hayward (3 rangées)	35	52		29/11/1994

2.9. Suppressions

Il est possible de supprimer des lignes d'une table à l'aide de la commande DELETE. Supposons que vous ne soyez plus intéressé. en fonction de la météo à Hayward. Vous pouvez ensuite procéder comme suit pour supprimer ces lignes du tableau :

```
SUPPRIMER DE la table weather WHERE city = 'Hayward';
```

Tous les relevés météorologiques appartenant à Hayward ont été supprimés.

SÉLECTIONNER * DE la météo;

ville	temp_lo	temp_hi	prcp	date
San Francisco	46	50	0,25	27/11/1994
San Francisco	41	55	0	29/11/1994

(2 rangées)

Il convient de se méfier des déclarations de cette forme

SUPPRIMER DE nom_de_table;

Sans précision, la commande DELETE supprimera toutes les lignes de la table spécifiée, la laissant vide.
Le système ne demandera pas de confirmation avant de procéder !

Chapitre 3. Fonctionnalités avancées

3.1. Introduction

Dans le chapitre précédent, nous avons abordé les principes fondamentaux de l'utilisation de SQL pour stocker et accéder à vos données dans PostgreSQL. Nous allons maintenant examiner des fonctionnalités plus avancées de SQL qui simplifient la gestion et préviennent la perte ou la corruption de vos données. Enfin, nous étudierons quelques extensions de PostgreSQL.

Ce chapitre fera parfois référence à des exemples du chapitre 2 afin de les modifier ou de les améliorer ; il est donc conseillé d'avoir lu ce chapitre. Certains exemples de ce chapitre sont également disponibles dans le fichier `advanced.sql` du répertoire du tutoriel. Ce fichier contient aussi des exemples de données à charger, qui ne sont pas reproduits ici. (Pour savoir comment utiliser ce fichier, reportez-vous à la section 2.1.)

3.2. Vues

Reportez-vous aux requêtes de la section 2.6. Supposons que la liste combinée des relevés météorologiques et de la localisation des villes soit particulièrement utile à votre application, mais que vous ne souhaitiez pas saisir la requête à chaque fois. Vous pouvez créer une vue sur la requête, ce qui lui donne un nom auquel vous pouvez faire référence comme à une table ordinaire :

```
CRÉER UNE VUE myview COMME
    SÉLECTIONNER nom, temp_lo, temp_hi, prcp, date, emplacement
        À partir de la météo, des villes
        OÙ ville = nom;

SÉLECTIONNER * DE mavue;
```

L'utilisation judicieuse des vues est un aspect essentiel d'une bonne conception de base de données SQL. Les vues permettent d'encapsuler les détails de la structure de vos tables, susceptibles d'évoluer au fil de votre application, derrière des interfaces cohérentes.

Les vues peuvent être utilisées presque partout où un tableau réel peut être utilisé. Il est courant de superposer des vues.

3.3. Clés étrangères

Rappelez-vous les tables « météo » et « villes » du chapitre 2. Prenons l'exemple suivant : vous souhaitez empêcher l'insertion de nouvelles lignes dans la table « météo » sans entrée correspondante dans la table « villes ». C'est ce qu'on appelle le maintien de l'intégrité référentielle des données. Dans les systèmes de bases de données simples, cela se ferait (le cas échéant) en consultant d'abord la table « villes » pour vérifier l'existence d'un enregistrement correspondant, puis en insérant ou en rejetant les nouvelles données météorologiques. Cette approche présente plusieurs inconvénients et est très peu pratique ; c'est pourquoi PostgreSQL peut automatiser cette tâche.

La nouvelle déclaration des tables ressemblerait à ceci :

```
CRÉER TABLE villes (
    nom                clé primaire varchar(80),
    point de localisation
);

CRÉER TABLE weather ( ville varchar(80)
    références cities(name), temp_lo int,
```

```

temp_hi entier, réel,
date
prcp          date
);

```

Essayez maintenant d'insérer un enregistrement invalide :

```
INSÉRER DANS weather VALUES ('Berkeley', 45, 53, 0.0, '1994-11-28');
```

ERREUR : l'insertion ou la mise à jour dans la table « weather » viole la contrainte de clé étrangère « weather_city_fkey »

DÉTAIL : La clé (ville)=(Berkeley) n'est pas présente dans la table « villes ».

Le comportement des clés étrangères peut être finement paramétré pour votre application. Nous n'aborderons pas plus en détail cet exemple simple dans ce tutoriel ; veuillez consulter le chapitre 5 pour plus d'informations. Une utilisation correcte des clés étrangères améliorera considérablement la qualité de vos applications de base de données ; nous vous encourageons donc vivement à vous familiariser avec ce concept.

3.4. Transactions

Les transactions sont un concept fondamental de tous les systèmes de bases de données. L'essence même d'une transaction réside dans le regroupement de plusieurs étapes en une seule opération binaire. Les états intermédiaires entre les étapes sont invisibles pour les autres transactions simultanées. Ainsi, en cas d'échec empêchant la transaction de se terminer, aucune des étapes n'affecte la base de données.

Prenons l'exemple d'une base de données bancaire contenant les soldes des différents comptes clients, ainsi que le total des dépôts par agence. Supposons que nous souhaitions enregistrer un paiement de 100,00 \$ du compte d'Alice vers celui de Bob. En simplifiant à l'extrême, les commandes SQL correspondantes pourraient ressembler à ceci :

```

MISE À JOUR des comptes DÉFINIR le solde = solde - 100,00
    OÙ nom = 'Alice';
MISE À JOUR branches DÉFINIR solde = solde - 100,00
    OÙ nom = (SÉLECTIONNER nom_de_la_branche DE comptes OÙ nom =
    'Alice');
MISE À JOUR des comptes DÉFINIR le solde = solde + 100,00
    OÙ nom = 'Bob';
MISE À JOUR branches DÉFINIR solde = solde + 100,00
    OÙ nom = (SÉLECTIONNER nom_de_la_branche DE comptes OÙ nom =
    'Bob');

```

Les détails de ces commandes importent peu ici ; l'essentiel est que plusieurs mises à jour distinctes sont nécessaires pour réaliser cette opération relativement simple. Les responsables de notre banque voudront s'assurer que soit toutes ces mises à jour sont effectuées, soit aucune. Il serait inacceptable qu'une panne système entraîne le versement de 100 \$ à Bob sans que le compte d'Alice n'ait été débité. De même, Alice ne resterait pas longtemps satisfaite si son compte était débité sans que celui de Bob ne soit crédité. Nous avons besoin d'une garantie : si un problème survient en cours d'opération, aucune des étapes déjà exécutées ne sera prise en compte. Le regroupement des mises à jour dans une transaction nous apporte cette garantie. Une transaction est dite atomique : du point de vue des autres transactions, elle est soit entièrement réalisée, soit pas du tout.

Nous souhaitons également avoir la garantie qu'une fois une transaction terminée et validée par le système de base de données, elle soit enregistrée de manière permanente et ne soit pas perdue, même en cas de panne survenant peu après. Par exemple, si nous enregistrons un retrait d'espèces effectué par Bob, nous ne voulons pas risquer que le débit de son compte disparaisse suite à une panne juste après sa sortie de la banque. Une base de données transactionnelle garantit que toutes les modifications apportées par une transaction sont enregistrées de manière permanente (sur disque) avant que la transaction ne soit considérée comme terminée.

Une autre propriété importante des bases de données transactionnelles est étroitement liée à la notion de mise à jour atomique : lorsque plusieurs transactions s'exécutent simultanément, chacune ne doit pas pouvoir voir les modifications incomplètes effectuées par les autres. Par exemple, si une transaction calcule le total des soldes de toutes les agences, il serait incorrect qu'elle inclue le débit de l'agence d'Alice sans inclure le crédit de l'agence de Bob, et inversement. Les transactions doivent donc être de type « tout ou rien », non seulement en termes d'effet permanent sur la base de données, mais aussi en termes de visibilité au fur et à mesure de leur exécution. Les mises à jour effectuées par une transaction en cours sont invisibles aux autres transactions jusqu'à la fin de la transaction, moment auquel toutes les mises à jour deviennent visibles simultanément.

Dans PostgreSQL, une transaction est initialisée en encadrant les commandes SQL de la transaction par les instructions `BEGIN` et `COMMIT`. Ainsi, notre transaction bancaire ressemblerait à ceci :

```
COMMENCER;  
MISE À JOUR des comptes DÉFINIR le solde = solde - 100,00  
    OÙ nom = 'Alice';  
-- etc etc  
COMMETTRE;
```

Si, en cours de transaction, nous décidons de ne pas valider (par exemple, si nous constatons que le solde d'Alice est devenu négatif), nous pouvons utiliser la commande `ROLLBACK` au lieu de `COMMIT`, et toutes nos modifications effectuées jusqu'à présent seront annulées.

PostgreSQL considère chaque instruction SQL comme exécutée dans le cadre d'une transaction. Si vous n'utilisez pas l'instruction `BEGIN`, chaque instruction comporte implicitement un `BEGIN` et, en cas de succès, la requête est exécutée dans une transaction. L'instruction `COMMIT` l'entoure. Un groupe d'instructions entouré par `BEGIN` et `COMMIT` est parfois appelé bloc de transaction.

Note

Certaines bibliothèques clientes émettent automatiquement les commandes `BEGIN` et `COMMIT`, ce qui peut permettre d'obtenir l'effet de blocage des transactions sans intervention de votre part. Consultez la documentation de l'interface que vous utilisez.

Il est possible de contrôler les instructions d'une transaction de manière plus précise grâce aux points de sauvegarde. Ces points permettent d'annuler sélectivement certaines parties de la transaction tout en validant le reste. Après avoir défini un point de sauvegarde avec la commande `SAVEPOINT`, vous pouvez, si nécessaire, revenir à ce point avec `ROLLBACK TO`. Toutes les modifications apportées à la base de données de la transaction entre la définition du point de sauvegarde et le moment où vous y revenez sont annulées, mais les modifications antérieures à ce point sont conservées.

Après une restauration à un point de sauvegarde, celui-ci reste défini, vous pouvez donc y revenir plusieurs fois. À l'inverse, si vous êtes certain de ne plus avoir besoin de revenir à un point de sauvegarde particulier, vous pouvez le libérer afin de libérer des ressources système. Notez que la libération d'une sauvegarde ou la restauration d'un point de sauvegarde entraîne la libération automatique de toutes les sauvegardes postérieures.

Tout ceci se déroule à l'intérieur du bloc de transaction, donc rien n'est visible pour les autres sessions de base de données. Lorsque vous validez le bloc de transactions, les actions validées deviennent visibles comme une unité pour les autres sessions, tandis que les actions annulées ne deviennent jamais visibles.

En repensant à la base de données bancaires, supposons que nous débitons 100 \$ du compte d'Alice et créditions celui de Bob, pour ensuite nous apercevoir que nous aurions dû créditer celui de Wally. Nous pourrions gérer cela à l'aide de points de sauvegarde comme ceci :

```
COMMENCER;  
MISE À JOUR des comptes DÉFINIR le solde = solde - 100,00  
    OÙ nom = 'Alice';  
POINT DE SAUVEGARDE mon_point_de_sauvegarde ;
```

```

MISE À JOUR des comptes DÉFINIR le solde = solde + 100,00
    OÙ nom = 'Bob';
Oups... oubliez ça et utilisez le compte de Wally.
RESTAURATION À mon_point_de_sauvegarde ;
MISE À JOUR des comptes DÉFINIR le solde = solde + 100,00
    OÙ nom = 'Wally';
COMMETTRE;

```

Cet exemple est, bien sûr, simplifié à l'extrême, mais il est possible de contrôler de nombreuses opérations dans un bloc de transactions. grâce à l'utilisation de points de sauvegarde. De plus, la fonction ROLLBACK TO est le seul moyen de reprendre le contrôle d'une partie. Bloc de transaction mis en état d'abandon par le système en raison d'une erreur, sans possibilité de restauration. complètement et en recommençant.

3.5. Fonctions de fenêtrage

Une fonction de fenêtre effectue un calcul sur un ensemble de lignes de tableau qui sont liées d'une manière ou d'une autre à la valeur. ligne actuelle. Ceci est comparable au type de calcul qui peut être effectué avec une fonction d'agrégation. Cependant, contrairement aux fonctions d'agrégation classiques, les fonctions de fenêtrage ne regroupent pas les lignes en une seule ligne de sortie. Au contraire, chaque ligne conserve son identité propre. En coulisses, La fonction de fenêtre permet d'accéder à plus que la simple ligne actuelle du résultat de la requête.

Voici un exemple qui montre comment comparer le salaire de chaque employé avec le salaire moyen dans son secteur. ou son département :

```

SELECT depname, empno, salary, avg(salary) OVER (PARTITION BY
    nom_du_département) DE salaire_de_l'employé;

```

nom du département numéro d'employé salaire	moyenne
développer 11 5200 5020.0000000000000000	
développer 7 4200 5020.0000000000000000	
développer 9 4500 5020.0000000000000000	
développer 8 6000 5020.0000000000000000	
développer personnel 10 5200 5020.0000000000000000	
personnel ventes 5 3500 3700.0000000000000000	
ventes ventes (10	
lignes)	
3 4800 4866.6666666666666667	
1 5000 4866.6666666666666667	
4 4800 4866.6666666666666667	

Les trois premières colonnes de sortie proviennent directement de la table empsalary, et il y a une seule ligne de sortie pour chaque ligne du tableau. La quatrième colonne représente une moyenne calculée sur l'ensemble des lignes du tableau. qui ont la même valeur depname que la ligne actuelle. (Il s'agit en fait de la même fonction que la fonction d'agrégation de moyenne non-fenêtrée, mais la clause OVER fait en sorte qu'elle soit traitée comme une fonction de fenêtre et calculé sur toute la largeur de la fenêtre.)

Un appel de fonction de fenêtre contient toujours une clause OVER suivant directement l'appel de la fonction de fenêtre. nom et argument(s). C'est ce qui le distingue syntaxiquement d'une fonction normale ou d'une agrégation non-fenêtrée. La clause OVER détermine précisément comment les lignes de la requête sont réparties. traitement par la fonction de fenêtre. La clause PARTITION BY à l'intérieur de OVER divise les lignes en groupes, ou partitions, qui partagent les mêmes valeurs de l'expression PARTITION BY. Pour chaque ligne, La fonction de fenêtre est calculée sur les lignes appartenant à la même partition que la ligne actuelle.

Vous pouvez également contrôler l'ordre de traitement des lignes par les fonctions de fenêtre à l'aide de ORDER BY. dans OVER. (La clause ORDER BY de la fenêtre n'a même pas besoin de correspondre à l'ordre dans lequel les lignes sont (Résultat.) Voici un exemple :

```
SÉLECTIONNER nom_du_département, numéro_d'employé, salaire,
rang() SUR (PARTITION PAR nom_département ORDER PAR salaire DESC)
DE l'employésalaire;
```

```
nom du département | numéro d'employé | salaire | grade
-----+-----+-----+-----
développer |      8 | 6000 | 10 | 5200      1
développer |      | 11 | 5200 | 9 | 4500      2
développer |      | 7 | 4200 | 2 | 3900 |      2
développer |      5 | 3500 | 1 | 5000 |      4
développer | personnel 4 | 4800 | 3 | 4800 |      5
| personnel | ventes |      1
| |      2
| |      1
ventes      2
ventes      2
(10 lignes)
```

Comme illustré ici, la fonction de classement attribue un rang numérique à chaque valeur distincte de ORDER BY dans la partition de la ligne actuelle, en utilisant l'ordre défini par la clause ORDER BY. Le rang ne nécessite pas de valeur explicite paramètre, car son comportement est entièrement déterminé par la clause OVER.

Les lignes prises en compte par une fonction de fenêtre sont celles de la « table virtuelle » produite par la requête. Clause FROM filtrée par ses clauses WHERE, GROUP BY et HAVING, le cas échéant. Par exemple, une ligne Supprimée car elle ne répond pas à la condition WHERE et n'est pas prise en compte par les fonctions de fenêtre. Une requête peut contenir plusieurs fonctions de fenêtrage qui découpent les données de différentes manières en utilisant différents OVER les clauses, mais elles agissent toutes sur la même collection de lignes définie par cette table virtuelle.

Nous avons déjà vu que la clause ORDER BY peut être omise si l'ordre des lignes n'est pas important. C'est également le cas. Il est possible d'omettre PARTITION BY, auquel cas il n'y a qu'une seule partition contenant toutes les lignes.

Il existe un autre concept important associé aux fonctions de fenêtre : pour chaque ligne, il existe un ensemble de lignes à l'intérieur de sa partition appelée cadre de fenêtre. Certaines fonctions de fenêtre agissent uniquement sur les lignes du cadre de fenêtre. Le cadre de la fenêtre, et non la partition entière. Par défaut, si une clause ORDER BY est spécifiée, le cadre est utilisé. comprend toutes les lignes depuis le début de la partition jusqu'à la ligne actuelle, plus toutes les lignes suivantes. qui sont égales à la ligne actuelle selon la clause ORDER BY¹. Lorsque ORDER BY est omis, Le cadre par défaut comprend toutes les lignes de la partition. ¹ Voici un exemple utilisant la fonction somme :

```
SELECT salary, sum(salary) OVER () FROM empsalary;
```

```
salaire | somme
-----+-----
5200 | 47100
5000 | 47100
3500 | 47100
4800 | 47100
3900 | 47100
4200 | 47100
4500 | 47100
4800 | 47100
6000 | 47100
5200 | 47100
(10 lignes)
```

¹ Il existe d'autres façons de définir le cadre de la fenêtre, mais ce tutoriel ne les aborde pas. Consultez la section 4.2.8 pour plus de détails.

Ci-dessus, comme la clause OVER ne contient pas de clause ORDER BY, le cadre de la fenêtre est identique à la partition, qui, faute de PARTITION BY, correspond à la table entière ; autrement dit, chaque somme est calculée sur l'ensemble de la table, ce qui explique le même résultat pour chaque ligne de sortie. En revanche, si l'on ajoute une clause ORDER BY, les résultats sont très différents :

```
SELECT salary, sum(salary) OVER (ORDER BY salary) FROM emp;
```

```

      salaire | somme
-----+-----
      3500 | 3500 3900 |
      7400 | 4200 | 11600
      4500 | 16100 4800 |
     25700 | 4800 | 25700
      5000 | 30700 5200 |
     41100 | 5200 | 41100
      6000 | 47100 (10
      lignes)

```

Ici, la somme est calculée à partir du premier salaire (le plus bas) jusqu'au salaire actuel, en incluant les éventuels doublons de ce dernier (notez les résultats pour les salaires en double).

Les fonctions de fenêtrage sont autorisées uniquement dans la liste SELECT et la clause ORDER BY de la requête. Leur utilisation est interdite ailleurs, notamment dans les clauses GROUP BY, HAVING et WHERE. En effet, elles s'exécutent logiquement après le traitement de ces clauses. De plus, les fonctions de fenêtrage s'exécutent après les fonctions d'agrégation qui ne sont pas des fonctions de fenêtrage. Cela signifie qu'il est valide d'inclure un appel de fonction d'agrégation dans les arguments d'une fonction de fenêtrage, mais l'inverse n'est pas vrai.

S'il est nécessaire de filtrer ou de regrouper des lignes après l'exécution des calculs de fenêtrage, vous pouvez utiliser une sous-sélection. Par exemple :

```

SÉLECTIONNER depname, empno, salary, enroll_date
DEPUIS

      (SELECT depname, empno, salary, enroll_date, rank() OVER (PARTITION BY
        depname ORDER BY salary DESC, empno) AS pos

        DE l'employé
      ) AS ss
OÙ pos < 3 ;

```

La requête ci-dessus n'affiche que les lignes de la sous-requête dont le rang est inférieur à 3.

Lorsqu'une requête fait intervenir plusieurs fonctions de fenêtrage, il est possible de les spécifier chacune dans une clause OVER distincte, mais cette approche est redondante et source d'erreurs si l'on souhaite appliquer le même comportement de fenêtrage à plusieurs fonctions. Il est préférable de nommer chaque comportement de fenêtrage dans une clause WINDOW, puis de le référencer dans OVER. Par exemple :

```

SELECT sum(salary) OVER w, avg(salary) OVER w
      DE l'employé
      FENÊTRE w AS (PARTITION PAR nom_de_départEMENT ORDER PAR salaire DESC);

```

Vous trouverez plus de détails sur les fonctions de fenêtrage dans la section 4.2.8, la section 9.22, la section 7.2.5 et la page de référence SELECT.

3.6. Héritage

L'héritage est un concept issu des bases de données orientées objet. Il ouvre de nouvelles perspectives intéressantes en matière de conception de bases de données.

Créons deux tables : une table « villes » et une table « capitales ». Naturellement, les capitales sont aussi des villes ; il faut donc trouver un moyen de les indiquer implicitement dans la liste des villes. Avec un peu d'ingéniosité, on pourrait imaginer un système comme celui-ci :

```
CRÉER TABLE capitales (
    nom          texte,
    population réelle, altitude entière,
    char(2)      -- (en pieds)
    État
);

CRÉER TABLE non_capitals ( texte, population réelle, altitude
    nom          entière

                                -- (en pieds)
);

CRÉER UNE VUE des villes
    SÉLECTIONNER nom, population, altitude DE capitales
    UNION
    SÉLECTIONNER nom, population, altitude DE non_capitals;
```

Cela fonctionne correctement pour les requêtes, mais cela devient compliqué lorsqu'il faut mettre à jour plusieurs lignes, par exemple.

Une meilleure solution est la suivante :

```
CRÉER TABLE villes ( texte, population réelle,
    nom          altitude
    entière

                                -- (en pieds)
);

CRÉER TABLE capitales (
    État          char(2) UNIQUE NOT NULL
) HÉRITE (villes);
```

Dans ce cas, la ligne « capitales » hérite de toutes les colonnes (nom, population et altitude) de sa table parente « villes ». Le type de la colonne « nom » est « texte », un type natif de PostgreSQL pour les chaînes de caractères de longueur variable. La table « capitales » possède une colonne supplémentaire, « état », qui affiche l'abréviation de l'État. Dans PostgreSQL, une table peut hériter de zéro ou plusieurs autres tables.

Par exemple, la requête suivante permet de trouver les noms de toutes les villes, y compris les capitales d'État, situées à une altitude supérieure à 500 pieds :

```
SÉLECTIONNER nom, altitude
DES villes
OÙ l'altitude est supérieure à 500 ;
```

qui renvoie :

nom	altitude
-----+-----	
Las Vegas	2174
Mariposa	1953
Madison (3	845
rangées)	

En revanche, la requête suivante trouve toutes les villes qui ne sont pas des capitales d'État et qui sont situées à une altitude supérieure à 500 pieds :

```
SÉLECTIONNER nom, altitude
DES SEULEMENT des villes
OÙ l'altitude est supérieure à 500 ;
```

nom	altitude
-----+-----	
Las Vegas	2174
Mariposa (2	1953
rangées)	

Ici, le terme `ONLY` précédant « cities » indique que la requête doit être exécutée uniquement sur la table « cities », et non sur les tables situées en dessous de « cities » dans la hiérarchie d'héritage. La plupart des commandes que nous avons déjà abordées — `SELECT`, `UPDATE` et `DELETE` — prennent en charge cette notation `ONLY`.

Note

Bien que l'héritage soit souvent utile, son intégration avec les contraintes d'unicité et les clés étrangères en limite l'intérêt. Voir la section 5.11 pour plus de détails.

3.7. Conclusion

PostgreSQL possède de nombreuses fonctionnalités qui ne sont pas abordées dans cette introduction, destinée aux nouveaux utilisateurs de SQL. Ces fonctionnalités sont décrites plus en détail dans la suite de cet ouvrage.

Si vous souhaitez approfondir vos connaissances, veuillez consulter le site web de PostgreSQL² pour accéder aux liens suivants : plus de ressources.

² <https://www.postgresql.org>

Partie II. Le langage SQL

Cette section décrit l'utilisation du langage SQL dans PostgreSQL. Nous commençons par présenter la syntaxe générale de SQL, puis expliquons comment créer des tables, alimenter la base de données et l'interroger. La partie centrale liste les types de données et les fonctions disponibles pour les commandes SQL. Enfin, nous abordons plusieurs aspects importants pour l'optimisation d'une base de données.

L'information est organisée de manière à ce qu'un utilisateur novice puisse la suivre du début à la fin et acquérir une compréhension complète des sujets sans avoir à se référer trop souvent au texte. Les chapitres sont conçus pour être autonomes, permettant ainsi aux utilisateurs avancés de les lire individuellement à leur guise. L'information est présentée sous forme narrative, organisée en unités thématiques. Les lecteurs souhaitant une description complète d'une commande particulière sont invités à consulter la partie VI.

Les lecteurs doivent savoir se connecter à une base de données PostgreSQL et exécuter des commandes SQL. Ceux qui ne sont pas familiarisés avec ces notions sont invités à consulter d'abord la première partie. Les commandes SQL sont généralement saisies à l'aide du terminal interactif `psql` de PostgreSQL, mais d'autres programmes aux fonctionnalités similaires peuvent également être utilisés.

Table des matières

4. Syntaxe SQL	33
4.1. Structure lexicale	33
4.1.1. Identifiants et mots clés	33
4.1.2. Constantes	35
4.1.3. Opérateurs	40
4.1.4. Caractères spéciaux	40
4.1.5. Commentaires	41
4.1.6. Priorité des opérateurs	41
4.2. Expressions de valeurs	42
4.2.1. Références de colonnes	43
4.2.2. Paramètres de position	43
4.2.3. Indices	43
4.2.4. Sélection des champs	44
4.2.5. Appels d'opérateurs	44
4.2.6. Appels de fonctions	45
4.2.7. Expressions agrégées	45
4.2.8. Appels de fonctions de fenêtre	48
4.2.9. Moulages de types	50
4.2.10. Expressions de classement	51
4.2.11. Sous-requêtes scalaires	52
4.2.12. Constructeurs de tableaux	52
4.2.13. Constructeurs de lignes	54
4.2.14. Règles d'évaluation des expressions	55
4.3. Appel de fonctions	56
4.3.1. Utilisation de la notation positionnelle	57
4.3.2. Utilisation de la notation nommée	57
4.3.3. Utilisation de la notation mixte	58
5. Définition des données	59
5.1. Notions de base sur les tableaux	59
5.2. Valeurs par défaut	60
5.3. Colonnes d'identité	61
5.4. Colonnes générées	62
5.5. Contraintes	64
5.5.1. Vérifier les contraintes	64
5.5.2. Contraintes de non-nullité	66
5.5.3. Contraintes d'unicité	67
5.5.4. Clés primaires	68
5.5.5. Clés étrangères	69
5.5.6. Contraintes d'exclusion	72
5.6. Colonnes du système	73
5.7. Modification des tableaux	74
5.7.1. Ajout d'une colonne	74
5.7.2. Suppression d'une colonne	75
5.7.3. Ajout d'une contrainte	75
5.7.4. Suppression d'une contrainte	75
5.7.5. Modification de la valeur par défaut d'une colonne	75
5.7.6. Modification du type de données d'une colonne	76
5.7.7. Renommer une colonne	76
5.7.8. Renommer un tableau	76
5.8. Privilèges	76
5.9. Politiques de sécurité des lignes	81
5.10. Schémas	87
5.10.1. Création d'un schéma	88
5.10.2. Le schéma public	89
5.10.3. Le chemin de recherche du schéma	89

5.10.4. Schémas et privilèges	91
5.10.5. Schéma du catalogue système	91
5.10.6. Modèles d'utilisation	91
5.10.7. Portabilité	92
5.11. Héritage	92
5.11.1. Avertissements	95
5.12. Partitionnement des tableaux	96
5.12.1. Aperçu	96
5.12.2. Partitionnement déclaratif	97
5.12.3. Partitionnement par héritage	102
5.12.4. Élagage des partitions	106
5.12.5. Partitionnement et exclusion des contraintes	108
5.12.6. Bonnes pratiques pour le partitionnement déclaratif	108
5.13. Données étrangères	109
5.14. Autres objets de base de données	110
5.15. Suivi des dépendances	110
6. Manipulation des données	112
6.1. Insertion des données 112.....	
6.2. Mise à jour des données 113.....	
6.3. Suppression des données 114.....	
6.4. Retour des données des lignes modifiées	114
7. Requêtes	116
7.1. Aperçu	116
7.2. Expressions tabulaires	116
7.2.1. La clause FROM	117
7.2.2. La clause WHERE	125
7.2.3. Les clauses GROUP BY et HAVING	126
7.2.4. GROUPEMENTS, CUBE ET ROLLUP	129
7.2.5. Traitement de la fonction de fenêtre	132
7.3. Listes de sélection	132
7.3.1. Éléments de la liste de sélection	132
7.3.2. Étiquettes de colonnes	133
7.3.3. DISTINCT	133
7.4. Combinaison de requêtes (UNION, INTERSECT, EXCEPT)	134
7.5. Tri des lignes (ORDER BY)	135
7.6. LIMITE et DÉCALAGE	136
7.7. VALEURS Listes	136
7.8. Requêtes WITH (Expressions de table communes)	137
7.8.1. SÉLECTIONNER AVEC 138.....	
7.8.2. Requêtes récursives	138
7.8.3. Matérialisation des expressions de table communes	143
7.8.4. Instructions de modification des données dans WITH 144.....	
8. Types de données	147
8.1. Types numériques	148
8.1.1. Types d'entiers	149
8.1.2. Nombres de précision arbitraire	149
8.1.3. Types à virgule flottante	151
8.1.4. Types de numéros de série	153
8.2. Types monétaires	154
8.3. Types de caractères	154
8.4. Types de données binaires	157
8.4.1. Format hexadécimal de bytea	157
8.4.2. Format d'échappement des octets	157
8.5. Types de date/heure	159
8.5.1. Saisie de la date et de l'heure	160
8.5.2. Sortie Date/Heure	164
8.5.3. Fuseaux horaires	165
8.5.4. Entrée d'intervalle	166

8.5.5. Sortie d'intervalle	168	
8.6. Type booléen	168	
8.7. Types énumérés	169	
8.7.1. Déclaration des types énumérés	169	
8.7.2. Commande	170	
8.7.3. Type de sécurité	170	
8.7.4. Détails de mise en œuvre	171	
8.8. Types géométriques	171	
8.8.1. Points	172	
8.8.2. Lignes	172	
8.8.3. Segments de droite	172	
8.8.4. Boîtes	172	
8.8.5. Chemins	173	
8.8.6. Polygones	173	
8.8.7. Cercles	173	
8.9. Types d'adresses réseau	174	
8.9.1. inet	174	
8.9.2. cidr	174	
8.9.3. inet vs. cidr	175	
8.9.4. macaddr	175	
8.9.5. macaddr8	175	
8.10. Types de chaînes binaires	176	
8.11. Types de recherche textuelle	177	
8.11.1. tsvector	177	
8.11.2. tsquery	178	
8.12. Type d'UUID	180	
8.13. Type XML	180	
8.13.1. Création de valeurs XML	180	
8.13.2. Gestion de l'encodage	181	
8.13.3. Accès aux valeurs XML	182	
8.14. Types JSON	182	
8.14.1. Syntaxe d'entrée et de sortie JSON	184	
8.14.2. Conception de documents JSON	185	
8.14.3. jsonb Conteneur et existence	185	
8.14.4. Indexation jsonb	187	
8.14.5. Indices jsonb	189	
8.14.6. Transformations	191	
8.14.7. jsonpath Type	191	
8.15. Tableaux	192	
8.15.1. Déclaration des types de tableaux	193	
8.15.2. Saisie de la valeur du tableau	193	
8.15.3. Accès aux tableaux	195	
8.15.4. Modification des tableaux	197	
8.15.5. Recherche dans les tableaux	200	
8.15.6. Syntaxe d'entrée et de sortie des tableaux	201	
8.16. Types composites	202	
8.16.1. Déclaration des types composites	202	
8.16.2. Construction de valeurs composites	203	
8.16.3. Accès aux types composites	204	
8.16.4. Modification des types composites	204	
8.16.5. Utilisation des types composites dans les requêtes	205	
8.16.6. Syntaxe d'entrée et de sortie de type composite	207	
8.17. Types de gammes		208
8.17.1. Cuisinières encastrables et cuisinières multifonctions	209	
8.17.2. Exemples	209	
8.17.3. Limites inclusives et exclusives	209	
8.17.4. Plages infinies (non bornées)	210	
8.17.5. Plage d'entrée/sortie	210	

8.17.6. Construction de plages et de plages multiples	211
8.17.7. Types de plages discrètes	212
8.17.8. Définition de nouveaux types de plages	212
8.17.9. Indexation	213
8.17.10. Contraintes sur les plages	213
8.18. Types de domaines	214
8.19. Types d'identificateurs d'objets	215
8h20. Type pg_isn	217
8.21. Pseudo-types	218
9. Fonctions et opérateurs	220
9.1. Opérateurs logiques	220
9.2. Fonctions et opérateurs de comparaison	221
9.3. Fonctions et opérateurs mathématiques	225
9.4. Fonctions et opérateurs de chaînes de caractères	233
9.4.1. Format	241
9.5. Fonctions et opérateurs de chaînes binaires	243
9.6. Fonctions et opérateurs de chaînes binaires	247
9.7. Appariement de motifs	249
9.7.1. J'AIME 250.....	
9.7.2. SIMILAIRE AUX Expressions régulières	251
9.7.3. Expressions régulières POSIX	252
9.8. Fonctions de mise en forme des types de données	268
9.9. Fonctions et opérateurs de date/heure	276
9.9.1. EXTRAIT, partie_date	283
9.9.2. date_trunc	287
9.9.3. date_bin	288
9.9.4. À FUSEAU HORAIRE et À LOCAL	289
9.9.5. Date/Heure actuelle	291
9.9.6. Retarder l'exécution	292
9.10. Fonctions de support des énumérations	293
9.11. Fonctions et opérateurs géométriques	294
9.12. Fonctions et opérateurs d'adressage réseau	301
9.13. Fonctions et opérateurs de recherche de texte	304
9.14. Fonctions UUID	310
9.15. Fonctions XML	311
9.15.1. Production de contenu XML	311
9.15.2. Prédicats XML	315
9.15.3. Traitement XML	317
9.15.4. Correspondance des tables avec XML 322.....	
9.16. Fonctions et opérateurs JSON	325
9.16.1. Traitement et création de données JSON 9.16.2.	326
Le langage SQL/JSON Path	339
9.16.3. Fonctions de requête SQL/JSON	350
9.16.4. JSON_TABLE	353
9.17. Fonctions de manipulation de séquences	358
9.18. Expressions conditionnelles	360
9.18.1. AFFAIRE	360
9.18.2. COALESCE	362
9.18.3. NULLIF	362
9.18.4. PLUS GRAND et PLUS PETIT	362
9.19. Fonctions et opérateurs de tableaux	363
9.20. Fonctions et opérateurs de plage/multiplage	366
9.21. Fonctions agrégées	372
9.22. Fonctions de fenêtrage	380
9.23. Fonctions de support de fusion	381
9.24. Expressions de sous-requête	382
9.24.1. EXISTE	382
9.24.2. DANS 382.....	

9.24.3. NON INCLUS	383
DANS 9.24.4. TOUT/CERTAINS	383
9.24.5. TOUS	384
9.24.6. Comparaison en une seule rangée	384
9.25. Comparaisons de lignes et de tableaux	385
9.25.1. DANS	385
9.25.2. PAS DANS	385
9.25.3. N'IMPORTE QUEL/CERTAINS (tableau)	386
9.25.4. TOUS (tableau)	386
9.25.5. Comparaison des constructeurs de lignes	386
9.25.6. Comparaison des types composites	387
9.26. Fonctions de retour en ensemble	387
9.27. Fonctions et opérateurs d'information système	391
9.27.1. Fonctions d'information de session	391
9.27.2. Fonctions de consultation des privilèges d'accès	394
9.27.3. Fonctions d'interrogation de la visibilité du schéma	397
9.27.4. Fonctions d'information du catalogue système	398
9.27.5. Informations sur les objets et fonctions d'adressage	404
9.27.6. Fonctions d'information des commentaires	405
9.27.7. Fonctions de vérification de la validité des données	405
9.27.8. Fonctions d'identification de transaction et d'informations d'instantané	406
9.27.9. Fonctions d'information sur les transactions engagées	408
9.27.10. Fonctions de contrôle des données	409
9.27.11. Informations sur la version Fonctions	410
9.27.12. Fonctions de synthèse d'informations WAL	411
9.28. Fonctions d'administration système	412
9.28.1. Fonctions de configuration	412
9.28.2. Fonctions de signalisation du serveur	412
9.28.3. Fonctions de contrôle de sauvegarde	414
9.28.4. Fonctions de contrôle de récupération	416
9.28.5. Fonctions de synchronisation des instantanés	418
9.28.6. Fonctions de gestion de la réplication	418
9.28.7. Fonctions de gestion des objets de base de données	422
9.28.8. Fonctions de maintenance de l'index	425
9.28.9. Fonctions génériques d'accès aux fichiers	425
9.28.10. Fonctions de verrouillage consultatif	427
9.29. Fonctions de déclenchement	429
9.30. Fonctions de déclenchement d'événements	430
9.30.1. Capture des modifications à la fin de la commande	430
9.30.2. Traitement des objets supprimés par une commande DDL	431
9.30.3. Gestion d'un événement de réécriture de table	432
9.31. Fonctions d'information statistique	433
9.31.1. Inspection des listes MCV	433
10. Conversion de type	434
10.1. Aperçu	434
10.2. Opérateurs	435
10.3. Fonctions	439
10.4. Stockage de valeur	443
10.5. UNION, CASE et constructions connexes	444
10.6. SÉLECTIONNER les colonnes de sortie	445
11. Index	447
11.1. Introduction	447
11.2. Types d'index	448
11.2.1. Arbre B	448
11.2.2. Hachage	449
11.2.3. GiST	449
11.2.4. SP-GiST	449
11.2.5. GIN	449

11.2.6. BRIN	450
11.3. Index à plusieurs colonnes	450
11.4. Index et ORDER BY	451
11.5. Combinaison de plusieurs index	452
11.6. Index uniques	453
11.7. Indices sur les expressions	453
11.8. Index partiels	454
11.9. Numérisations d'index uniquement et index de couverture	457
11.10. Classes et familles d'opérateurs	459
11.11. Index et classements	461
11.12. Examen de l'utilisation de l'index	461
12. Recherche en texte intégral	463
12.1. Introduction	463
12.1.1. Qu'est-ce qu'un document ?	464
12.1.2. Appariement de texte de base	464
12.1.3. Configurations	466
12.2. Tables et index	467
12.2.1. Recherche dans une table	467
12.2.2. Création des index	468
12.3. Contrôle de la recherche textuelle	469
12.3.1. Analyse des documents	469
12.3.2. Analyse des requêtes	470
12.3.3. Classement des résultats de recherche	473
12.3.4. Résultats mis en évidence	475
12.4. Fonctionnalités supplémentaires	476
12.4.1. Manipulation de documents	476
12.4.2. Manipulation des requêtes	477
12.4.3. Déclencheurs de mises à jour automatiques	480
12.4.4. Collecte des statistiques documentaires	481
12.5. Analyseurs syntaxiques	482
12.6. Dictionnaires	484
12.6.1. Mots vides	485
12.6.2. Dictionnaire simple	485
12.6.3. Dictionnaire des synonymes	487
12.6.4. Dictionnaire des synonymes	488
12.6.5. Dictionnaire Ispell	491
12.6.6. Dictionnaire de la boule de neige	493
12.7. Exemple de configuration	493
12.8. Test et débogage de la recherche de texte	495
12.8.1. Tests de configuration	495
12.8.2. Tests de l'analyseur syntaxique	497
12.8.3. Tests de dictionnaire	498
12.9. Types d'index préférés pour la recherche textuelle	499
12.10. Support psql	500
12.11. Limitations	503
13. Contrôle de la concurrence	505
13.1. Introduction	505
13.2. Isolation des transactions	505
13.2.1. Niveau d'isolement engagé (lecture)	506
13.2.2. Niveau d'isolation de lecture répétable	508
13.2.3. Niveau d'isolation sérialisable	509
13.3. Verrouillage explicite	511
13.3.1. Verrous au niveau de la table	512
13.3.2. Verrous de rangée	514
13.3.3. Verrous au niveau de la page	515
13.3.4. Interblocages	515
13.3.5. Écluses consultatives	516
13.4. Contrôles de cohérence des données au niveau de l'application	517

13.4.1. Application de la cohérence avec les transactions sérialisables	517	
13.4.2. Application de la cohérence avec des verrous de blocage explicites	518	
13.5. Gestion des échecs de sérialisation	518	
13.6. Avertissements	519	
13.7. Verrouillage et indexation	519	
14. Conseils de performance	521	
14.1. Utilisation d'EXPLAIN	521	
14.1.1. EXPLICATION des notions de base	521	
14.1.2. EXPLIQUER ANALYSER	529	
14.1.3. Mises en garde	533	
14.2. Statistiques utilisées par le planificateur	534	
14.2.1. Statistiques à une seule colonne	534	
14.2.2. Statistiques étendues	536	
14.3. Contrôle du planificateur avec des clauses JOIN explicites	540	
14.4. Remplissage d'une base de données	542	
14.4.1. Désactiver la validation automatique	542	
14.4.2. Utiliser COPIE	542	
14.4.3. Suppression des index	542	
14.4.4. Suppression des contraintes de clé étrangère	543	
14.4.5. Augmenter maintenance_work_mem	543	
14.4.6. Augmenter max_wal_size	543	
14.4.7. Désactiver la réplication d'archivage et de flux WAL	543	
14.4.8. Exécuter ANALYZE ensuite	543	
14.4.9. Quelques remarques concernant pg_dump	544	
14.5. Réglages non durables	544	
15. Requête parallèle	546	
15.1. Fonctionnement des requêtes parallèles	546	
15.2. Quand peut-on utiliser les requêtes parallèles ?	547	
15.3. Plans parallèles	548	
15.3.1. Analyses parallèles	548	
15.3.2. Jointures parallèles	548	
15.3.3. Agrégation parallèle	549	
15.3.4. Annexe parallèle	549	
15.3.5. Conseils pour les plans parallèles	549	
15.4. Sécurité parallèle		550
15.4.1. Étiquetage parallèle des fonctions et des agrégats		550

Chapitre 4. Syntaxe SQL

Ce chapitre décrit la syntaxe du langage SQL. Il constitue la base de la compréhension des chapitres suivants, qui détailleront l'application des commandes SQL pour définir et modifier des données.

Nous conseillons également aux utilisateurs déjà familiarisés avec SQL de lire attentivement ce chapitre, car il contient plusieurs règles et concepts qui sont mis en œuvre de manière incohérente entre les bases de données SQL ou qui sont spécifiques à PostgreSQL.

4.1. Structure lexicale

L'entrée SQL consiste en une séquence de commandes. Une commande est composée d'une suite de jetons, terminée par un point-virgule (« ; »). La fin du flux d'entrée marque également la fin d'une commande. La validité des jetons dépend de la syntaxe de la commande.

Un jeton peut être un mot-clé, un identifiant, un identifiant entre guillemets, une valeur littérale (ou constante) ou un symbole de caractère spécial. Les jetons sont généralement séparés par des espaces (espace, tabulation, saut de ligne), sauf en cas d'ambiguïté (généralement uniquement si un caractère spécial est adjacent à un autre type de jeton).

Par exemple, voici une entrée SQL (syntaxiquement) valide :

```
SÉLECTIONNER * DE MA_TABLE;  
MISE À JOUR DE MA_TABLE DÉFINIR A = 5 ;  
INSÉRER DANS MA_TABLE LES VALEURS (3, 'salut');
```

Il s'agit d'une séquence de trois commandes, une par ligne (bien que cela ne soit pas obligatoire ; plusieurs commandes peuvent figurer sur une même ligne, et les commandes peuvent être utilement réparties sur plusieurs lignes).

De plus, les commentaires peuvent figurer dans les entrées SQL. Ce ne sont pas des jetons, mais plutôt des espaces.

La syntaxe SQL est assez incohérente quant à l'identification des commandes et des opérandes ou paramètres. Les premiers éléments constituent généralement le nom de la commande ; ainsi, dans l'exemple précédent, on parlerait habituellement de commandes « SELECT », « UPDATE » et « INSERT ». Cependant, la commande UPDATE requiert toujours un élément SET à une position précise, et cette variante particulière de INSERT exige également un élément VALUES pour être complète. Les règles de syntaxe exactes de chaque commande sont décrites dans la partie VI.

4.1.1. Identifiants et mots clés

Les jetons tels que SELECT, UPDATE ou VALUES dans l'exemple ci-dessus sont des exemples de mots clés, c'est-à-dire des mots ayant une signification fixe dans le langage SQL. Les jetons MY_TABLE et A sont des exemples d'identificateurs. Ils identifient les noms de tables, de colonnes ou d'autres objets de base de données, selon la commande dans laquelle ils sont utilisés. C'est pourquoi on les appelle parfois simplement « noms ». Les mots clés et les identificateurs ont la même structure lexicale ; par conséquent, il est impossible de savoir si un jeton est un identificateur ou un mot clé sans connaître le langage. Une liste complète des mots clés se trouve à l'annexe C.

Les identificateurs et mots clés SQL doivent commencer par une lettre (az, mais aussi les lettres diacritées et les lettres non latines) ou par un trait de soulignement (_). Les caractères suivants peuvent être des lettres, des traits de soulignement, des chiffres (0 à 9) ou des signes dollar (\$). Notez que les signes dollar ne sont pas autorisés dans les identificateurs, conformément à la norme SQL ; leur utilisation peut donc réduire la portabilité des applications. La norme SQL ne définit pas de mot clé contenant des chiffres ou commençant ou se terminant par un trait de soulignement ; les identificateurs de cette forme sont donc protégés contre d'éventuels conflits avec les futures extensions de la norme.

Le système n'utilise pas plus de NAMEDATALEN-1 octets pour un identifiant ; les noms plus longs peuvent être saisis dans les commandes, mais ils seront tronqués. Par défaut, NAMEDATALEN vaut 64, la longueur maximale d'un identifiant est donc de 63 octets. Si cette limite pose problème, il est possible de l'augmenter en modifiant la constante NAMEDATALEN dans le fichier `src/include/pg_config_manual.h`.

Les mots clés et les identifiants non entre guillemets ne sont pas sensibles à la casse. Par conséquent :

```
MISE À JOUR DE MA_TABLE DÉFINIR A = 5 ;
```

peut également s'écrire comme suit :

```
mettre à jour my_Table Définir a = 5;
```

Une convention souvent utilisée consiste à écrire les mots clés en majuscules et les noms en minuscules, par exemple :

```
MISE À JOUR my_table DÉFINIR a = 5 ;
```

Il existe un second type d'identificateur : l'identificateur délimité ou identificateur entre guillemets. Il est formé en encadrant une séquence de caractères quelconque par des guillemets doubles ("). Un identificateur délimité est toujours un identificateur, jamais un mot-clé. Ainsi, « select » peut être utilisé pour faire référence à une colonne ou une table nommée « select », tandis qu'un « select » non entre guillemets serait interprété comme un mot-clé et provoquerait donc une erreur d'analyse syntaxique lorsqu'il est utilisé à la place d'un nom de table ou de colonne. L'exemple peut être écrit avec des identificateurs entre guillemets comme suit :

```
MISE À JOUR "my_table" DÉFINIR "a" = 5;
```

Les identificateurs entre guillemets peuvent contenir n'importe quel caractère, à l'exception du caractère de code zéro. (Pour inclure un guillemet double, utilisez deux guillemets doubles.) Ceci permet de créer des noms de tables ou de colonnes qui seraient autrement impossibles, comme ceux contenant des espaces ou des esperluettes. La limite de longueur reste applicable.

L'utilisation de guillemets pour un identifiant le rend sensible à la casse, tandis que les noms non guillemets sont toujours convertis en minuscules. Par exemple, PostgreSQL considère les identifiants FOO, foo et "foo" comme identiques, mais « Foo » et « FOO » sont différents des trois premiers et entre eux. (La conversion en minuscules des noms non guillemets dans PostgreSQL est incompatible avec la norme SQL, qui exige que ces noms soient convertis en majuscules. Ainsi, foo devrait être équivalent à "FOO" et non à "foo" selon la norme. Pour développer des applications portables, il est conseillé d'utiliser systématiquement des guillemets pour un nom donné, ou de ne jamais les utiliser.)

Une variante des identificateurs entre guillemets permet d'inclure des caractères Unicode échappés, identifiés par leur point de code. Cette variante commence par U& (U majuscule ou minuscule suivi d'une esperluette) immédiatement avant le guillemet ouvrant, sans espace, par exemple U&"foo". (Notez que cela crée une ambiguïté avec l'opérateur &. Utilisez des espaces autour de l'opérateur pour éviter ce problème.) À l'intérieur des guillemets, les caractères Unicode peuvent être spécifiés sous forme échappée en écrivant une barre oblique inverse suivie du numéro de point de code hexadécimal à quatre chiffres, ou bien une barre oblique inverse suivie d'un signe plus suivi d'un numéro de point de code hexadécimal à six chiffres. Par exemple, l'identificateur « data » pourrait s'écrire :

```
U&"d\0061t\+000061"
```

L'exemple suivant, moins trivial, écrit le mot russe « slon » (éléphant) en caractères cyrilliques :

```
U&"\0441\043B\043E\043D"
```

Si un caractère d'échappement différent de la barre oblique inverse est souhaité, il peut être spécifié à l'aide de la clause UESCAPE après la chaîne, par exemple :

```
U&"d!0061t!+000061" UESCAPE ''
```

Le caractère d'échappement peut être n'importe quel caractère unique autre qu'un chiffre hexadécimal, le signe plus, une apostrophe, un guillemet double ou un espace. Notez que le caractère d'échappement est écrit entre guillemets simples, et non doubles, après UESCAPE.

Pour inclure littéralement le caractère d'échappement dans l'identificateur, écrivez-le deux fois.

On peut utiliser indifféremment le format d'échappement à 4 chiffres ou à 6 chiffres pour spécifier des paires de substitution UTF-16 afin de composer des caractères dont le point de code est supérieur à U+FFFF, bien que la disponibilité du format à 6 chiffres rende techniquement cela inutile. (Les paires de substitution ne sont pas stockées directement, mais combinées en un seul point de code.)

Si l'encodage du serveur n'est pas UTF-8, le point de code Unicode identifié par l'une de ces séquences d'échappement est converti en l'encodage réel du serveur ; une erreur est signalée si cela s'avère impossible.

4.1.2. Constantes

Il existe trois types de constantes implicitement typées dans PostgreSQL : les chaînes de caractères, les chaînes de bits et les nombres. Les constantes peuvent également être spécifiées avec des types explicites, ce qui permet une représentation plus précise et un traitement plus efficace par le système. Ces alternatives sont abordées dans les sous-sections suivantes.

4.1.2.1. Constantes de chaîne

En SQL, une constante de chaîne est une séquence arbitraire de caractères délimitée par des guillemets simples ('), par exemple « Ceci est une chaîne ». Pour inclure un guillemet simple dans une constante de chaîne, écrivez deux guillemets simples consécutifs, par exemple « Le cheval de Dianne ». Attention : il ne s'agit pas du même caractère que le guillemet double (").

Deux chaînes de caractères constantes, séparées uniquement par des espaces et au moins un saut de ligne, sont concaténées et traitées comme si elles n'avaient formé qu'une seule chaîne constante. Par exemple :

```
SÉLECTIONNER 'foo'  
'bar';
```

est équivalent à :

```
SÉLECTIONNER 'foobar';
```

mais:

```
SÉLECTIONNER 'foo'      'bar';
```

Cette syntaxe n'est pas valide. (Ce comportement quelque peu étrange est spécifié par SQL ; PostgreSQL suit la norme.)

4.1.2.2. Constantes de chaîne avec échappements de style C

PostgreSQL accepte également les constantes de chaîne d'échappement, qui constituent une extension de la norme SQL. Une constante de chaîne d'échappement est spécifiée en écrivant la lettre E (majuscule ou minuscule) juste avant l'apostrophe ouvrante, par exemple : E'foo'. (Lorsqu'une constante de chaîne d'échappement s'étend sur plusieurs lignes, écrivez E uniquement avant la première apostrophe ouvrante.) Dans une chaîne d'échappement, une barre oblique inverse (\) marque le début d'une séquence d'échappement de type C, où la combinaison de la barre oblique inverse et du ou des caractères suivants représente une valeur d'octet spéciale, comme illustré dans le tableau 4.1.

Tableau 4.1. Séquences d'échappement de la barre oblique inverse

Séquence d'échappement Backslash	Interprétation
<code>b</code>	retour arrière
<code>\f</code>	alimentation du formulaire
<code>\n</code>	nouvelle ligne
<code>\r</code>	retour chariot
<code>\t</code>	langnette
<code>\o, \oo, \ooo (o = 0–7) \xh, \xhh</code>	valeur d'octet octal
<code>(h = 0–9, A–F)</code>	valeur d'octet hexadécimale
<code>\xxxx, \Uxxxxxxxx (x = 0–9, A–F)</code>	Valeur de caractère Unicode hexadécimal 16 ou 32 bits ue

Tout autre caractère suivant une barre oblique inverse est interprété littéralement. Ainsi, pour inclure une barre oblique inverse, Écrivez deux barres obliques inverses (`\\`). De plus, une apostrophe peut être incluse dans une chaîne d'échappement en écrivant `\'`, en plus de la manière normale de `"`.

Il est de votre responsabilité de vous assurer que les séquences d'octets que vous créez, notamment lorsque vous utilisez les séquences d'échappement octales ou hexadécimales, composent des caractères valides dans l'encodage du jeu de caractères du serveur. Une alternative utile consiste à utiliser les séquences d'échappement Unicode ou la syntaxe alternative d'échappement Unicode, expliquée dans la section 4.1.2.3 ; Le serveur vérifiera que la conversion de caractères est possible.

Prudence

Si le paramètre de configuration `standard_conforming_strings` est désactivé, PostgreSQL reconnaît les séquences d'échappement avec barre oblique inverse dans les constantes de chaînes, qu'elles soient régulières ou non. Cependant, depuis PostgreSQL 9.1, ce paramètre est activé par défaut, ce qui signifie que les séquences d'échappement avec barre oblique inverse ne sont reconnues que dans les constantes de chaînes d'échappement. Ce comportement est plus conforme aux normes, mais peut entraîner des dysfonctionnements dans certaines applications qui s'appuient sur le comportement historique, selon lequel les échappements par barre oblique inverse ont toujours été reconnus. Une solution de contournement consiste à désactiver ce paramètre, mais il est préférable d'abandonner son utilisation. Échappement par barre oblique inverse. Si vous devez utiliser une barre oblique inverse pour représenter un caractère spécial, écrivez la constante de chaîne avec un `E`.

En plus de `standard_conforming_strings`, les paramètres de configuration `escape_string_warning` et `backslash_quote` régissent le traitement des barres obliques inverses dans les constantes de chaîne.

Le caractère de code zéro ne peut pas figurer dans une constante de chaîne.

4.1.2.3. Constantes de chaîne avec séquences d'échappement Unicode

PostgreSQL prend également en charge un autre type de syntaxe d'échappement pour les chaînes de caractères, permettant de spécifier des caractères arbitraires. Caractères Unicode par point de code. Une constante de chaîne d'échappement Unicode commence par `U` (majuscule ou minuscule). lettre majuscule `U` suivie d'une esperluette) immédiatement avant la citation ouvrante, sans aucun espace dans entre, par exemple, `U&'foo'`. (Notez que cela crée une ambiguïté avec l'opérateur `&`. Utilisez des espaces.) (Entourez l'opérateur pour éviter ce problème.) À l'intérieur des guillemets, des caractères Unicode peuvent être spécifiés. sous forme échappée en écrivant une barre oblique inverse suivie du numéro de point de code hexadécimal à quatre chiffres ou bien une barre oblique inverse suivie d'un signe plus, puis d'un code hexadécimal à six chiffres. nombre. Par exemple, la chaîne « données » pourrait s'écrire comme

```
U&d'0061t+000061'
```

L'exemple suivant, moins trivial, écrit le mot russe « slon » (éléphant) en caractères cyrilliques :

```
U&\0441\043B\043E\043D'
```

Si un caractère d'échappement différent de la barre oblique inverse est souhaité, il peut être spécifié à l'aide de la clause UESCAPE après la chaîne, par exemple :

```
U&'d\0061t\+000061' UESCAPE '!'
```

Le caractère d'échappement peut être n'importe quel caractère unique autre qu'un chiffre hexadécimal, le signe plus, une apostrophe, une guillemet double ou un espace.

Pour inclure littéralement le caractère d'échappement dans la chaîne, écrivez-le deux fois.

On peut utiliser indifféremment le format d'échappement à 4 chiffres ou à 6 chiffres pour spécifier des paires de substitution UTF-16 afin de composer des caractères dont le point de code est supérieur à U+FFFF, bien que la disponibilité du format à 6 chiffres rende techniquement cela inutile. (Les paires de substitution ne sont pas stockées directement, mais combinées en un seul point de code.)

Si l'encodage du serveur n'est pas UTF-8, le point de code Unicode identifié par l'une de ces séquences d'échappement est converti en l'encodage réel du serveur ; une erreur est signalée si cela s'avère impossible.

De plus, la syntaxe d'échappement Unicode pour les constantes de chaîne ne fonctionne que lorsque le paramètre de configuration `'standard_conforming_strings'` est activé. En effet, sans cette activation, cette syntaxe pourrait perturber les clients analysant les requêtes SQL et entraîner des injections SQL et autres failles de sécurité. Si ce paramètre est désactivé, cette syntaxe sera rejetée et un message d'erreur sera généré.

4.1.2.4. Constantes de chaîne entre guillemets dollar

Bien que la syntaxe standard pour spécifier les constantes de chaîne soit généralement pratique, elle peut être difficile à comprendre lorsque la chaîne souhaitée contient de nombreuses apostrophes, car chacune d'elles doit être doublée. Pour faciliter la lecture des requêtes dans de telles situations, PostgreSQL propose une autre méthode, appelée « utilisation des guillemets en forme de dollar », pour écrire les constantes de chaîne. Une constante de chaîne entre guillemets en forme de dollar se compose d'un signe dollar (\$), d'une « étiquette » facultative de zéro ou plusieurs caractères, d'un autre signe dollar, d'une séquence arbitraire de caractères constituant le contenu de la chaîne, d'un signe dollar, de l'étiquette initiale et d'un dernier signe dollar. Par exemple, voici deux façons différentes de spécifier la chaîne « le cheval de Dianne » en utilisant des guillemets avec le dollar :

```
$$Le cheval de Dianne$$
$SomeTag$Le cheval de Dianne$SomeTag$
```

Notez qu'à l'intérieur de la chaîne entre guillemets dollar, les guillemets simples peuvent être utilisés sans avoir besoin d'être échappés. En effet, aucun caractère à l'intérieur d'une chaîne de caractères entre guillemets n'est échappé : le contenu de la chaîne est toujours écrit littéralement. Les barres obliques inverses et les signes dollar ne sont pas spéciaux, sauf s'ils font partie d'une séquence correspondant à la balise ouvrante.

Il est possible d'imbriquer des constantes de chaîne entre guillemets en choisissant des balises différentes à chaque niveau d'imbrication. On l'utilise le plus souvent pour écrire des définitions de fonctions. Par exemple :

```
$fonction$
-- COMMENCER
    RETOUR ($1 ~ $q${\t\r\n\v\\}$q$);
FIN;
$fonction$
```

Ici, la séquence ``$q${\t\r\n\v\\}$q$`` représente une chaîne littérale entre guillemets dollar `'[{\t\r\n\v \\}'`, qui sera reconnue par PostgreSQL lors de l'exécution du corps de la fonction. Cependant, comme cette séquence ne correspond pas au délimiteur de guillemets dollar externe ``$fonction$``, elle n'est considérée, pour la chaîne externe, que comme un simple ajout de caractères à la constante.

L'étiquette, le cas échéant, d'une chaîne entre guillemets suit les mêmes règles qu'un identifiant sans guillemets, à ceci près qu'elle ne peut pas contenir de signe dollar. Les étiquettes sont sensibles à la casse : ``$tag$String content$tag$`` est correct, mais ``$TAG$String content$tag$`` ne l'est pas.

Une chaîne de caractères entre guillemets dollar qui suit un mot-clé ou un identifiant doit en être séparée par un espace ; sinon, le délimiteur de guillemets dollar serait considéré comme faisant partie de l'identifiant précédent.

L'utilisation des guillemets en forme de dollar n'est pas une norme SQL, mais elle s'avère souvent plus pratique que la syntaxe standard des guillemets simples pour écrire des chaînes de caractères complexes. Elle est particulièrement utile pour représenter des constantes de type chaîne à l'intérieur d'autres constantes, comme c'est souvent le cas dans les définitions de fonctions procédurales. Avec la syntaxe des guillemets simples, chaque barre oblique inverse de l'exemple ci-dessus devrait être écrite comme quatre barres obliques inverses, réduites à deux lors de l'analyse de la constante de type chaîne d'origine, puis à une seule lors de la réanalyse de la constante de type chaîne interne pendant l'exécution de la fonction.

4.1.2.5. Constantes de chaînes de bits

Les constantes binaires ressemblent aux constantes de chaîne classiques, à la différence qu'un B (majuscule ou minuscule) précède immédiatement le guillemet ouvrant (sans espace), par exemple : `B'1001'`. Seuls les caractères 0 et 1 sont autorisés dans les constantes binaires.

Il est également possible de spécifier les constantes binaires en notation hexadécimale, en utilisant un X (majuscule ou minuscule) en début de chaîne, par exemple : `X'1FF'`. Cette notation est équivalente à une constante binaire dont chaque chiffre hexadécimal est représenté par quatre bits.

Les deux formes de constantes binaires peuvent être étendues sur plusieurs lignes de la même manière que les constantes de chaînes classiques. L'utilisation de guillemets en forme de dollar est impossible dans une constante binaire.

4.1.2.6. Constantes numériques

Les constantes numériques sont acceptées sous les formes générales suivantes :

```
chiffres
chiffres[chiffres][e[+-]chiffres] [chiffres].chiffres[e[+-]chiffres]
chiffrese[+-]chiffres
```

où « digits » représente un ou plusieurs chiffres décimaux (de 0 à 9). Au moins un chiffre doit précéder ou suivre la virgule, le cas échéant. Au moins un chiffre doit suivre l'exposant (e), le cas échéant. La constante ne doit contenir ni espaces ni autres caractères, à l'exception des tirets bas, qui peuvent servir au regroupement visuel comme décrit ci-dessous. Notez que le signe plus ou moins initial n'est pas considéré comme faisant partie de la constante ; il s'agit d'un opérateur appliqué à celle-ci.

Voici quelques exemples de constantes numériques valides :

```
42
3,5
4.
0,001
5e2
1,925e-3
```

De plus, les constantes entières non décimales sont acceptées sous les formes suivantes :

```
0x chiffres
hexadécimaux 000
chiffres 0bb chiffres
```

où hexdigits représente un ou plusieurs chiffres hexadécimaux (0-9, AF), octdigits un ou plusieurs chiffres octaux (0-7) et bindigits un ou plusieurs chiffres binaires (0 ou 1). Les chiffres hexadécimaux et les préfixes de base peuvent être en majuscules ou en minuscules. Seuls les entiers peuvent avoir une représentation non décimale ; les nombres comportant une partie fractionnaire ne sont pas concernés.

Voici quelques exemples de constantes entières non décimales valides :

```
0b100101
0B10011001
0o273
0O755
0x42f
0XFFFF
```

Pour faciliter le regroupement visuel, des tirets bas peuvent être insérés entre les chiffres. Cela n'a aucune incidence sur la valeur de la constante. Par exemple :

```
1_500_000_000
0b10001000_00000000
0o_1_755
0xFFFF_FFFF
1.618_034
```

Les traits de soulignement ne sont pas autorisés au début ou à la fin d'une constante numérique ou d'un groupe de chiffres (c'est-à-dire immédiatement avant ou après la virgule décimale ou le marqueur d'exposant), et il n'est pas possible d'avoir plus d'un trait de soulignement consécutif.

Une constante numérique sans virgule ni exposant est initialement considérée comme de type entier si sa valeur tient dans un entier (32 bits) ; sinon, elle est considérée comme de type bigint si sa valeur tient dans un bigint (64 bits) ; sinon, elle est considérée comme de type numérique. Les constantes contenant une virgule et/ou un exposant sont toujours initialement considérées comme de type numérique.

Le type de données initialement attribué à une constante numérique n'est qu'un point de départ pour les algorithmes de résolution de type. Dans la plupart des cas, la constante sera automatiquement convertie au type le plus approprié en fonction du contexte. Si nécessaire, vous pouvez forcer l'interprétation d'une valeur numérique comme un type de données spécifique en effectuant un transtypage. Par exemple, vous pouvez forcer le traitement d'une valeur numérique comme un nombre réel (float4) en écrivant :

```
RÉEL '1.23' -- style de chaîne 1.23::RÉEL -- style
PostgreSQL (historique)
```

Il s'agit en fait de cas particuliers des notations de casting générales qui seront abordées ci-après.

4.1.2.7. Constantes d'autres types

Une constante de type arbitraire peut être saisie en utilisant l'une des notations suivantes :

```
type 'chaîne'
'chaîne'::type
CAST('chaîne' AS type)
```

Le texte de la constante de type chaîne est transmis à la routine de conversion d'entrée du type spécifié. Le résultat est une constante du type indiqué. La conversion de type explicite peut être omise s'il n'y a pas d'ambiguïté quant au type que doit avoir la constante (par exemple, lorsqu'elle est affectée directement à une colonne de table), auquel cas elle est automatiquement convertie.

La constante de chaîne peut être écrite en utilisant soit la notation SQL standard, soit les guillemets dollar.

Il est également possible de spécifier une conversion de type en utilisant une syntaxe de type fonction :

```
nom_de_type('chaîne')
```

mais tous les noms de types ne peuvent pas être utilisés de cette manière ; voir la section 4.2.9 pour plus de détails.

Les syntaxes '::', 'CAST()' et d'appel de fonction permettent également de spécifier des conversions de type dynamiques d'expressions arbitraires, comme expliqué dans la section 4.2.9. Afin d'éviter toute ambiguïté syntaxique, la syntaxe 'type 'string'' ne peut être utilisée que pour spécifier le type d'une constante littérale simple. De plus, cette syntaxe ne fonctionne pas avec les types tableaux ; utilisez '::' ou 'CAST()' pour spécifier le type d'une constante de tableau.

La syntaxe CAST() est conforme à SQL. La syntaxe du type « string » est une généralisation de la norme : SQL ne spécifie cette syntaxe que pour quelques types de données, mais PostgreSQL l'autorise pour tous les types.

La syntaxe avec :: correspond à l'usage historique de PostgreSQL, tout comme la syntaxe d'appel de fonction.

4.1.3. Opérateurs

Un nom d'opérateur est une séquence de jusqu'à NAMEDATALEN-1 (63 par défaut) caractères de la liste suivante :

```
* / < > = ~ ! @ # % ^ & | ` ' + - ?
```

Il existe toutefois quelques restrictions concernant les noms des opérateurs :

- -- et /* ne peuvent apparaître nulle part dans un nom d'opérateur, car ils seront considérés comme le début de un commentaire.
- Un nom d'opérateur à plusieurs caractères ne peut pas se terminer par + ou -, sauf s'il contient également au moins l'un de ces personnages :

```
~ ! @ # % ^ & | ` ' ?
```

Par exemple, @- est un nom d'opérateur autorisé, mais *- ne l'est pas. Cette restriction permet à PostgreSQL d'analyser les requêtes conformes à SQL sans exiger d'espaces entre les éléments.

Lorsque vous utilisez des noms d'opérateurs non standard SQL, vous devez généralement séparer les opérateurs adjacents par des espaces afin d'éviter toute ambiguïté. Par exemple, si vous avez défini un opérateur préfixe nommé @, vous ne pouvez pas écrire X*@Y ; vous devez écrire X* @Y pour que PostgreSQL le considère comme deux noms d'opérateurs distincts et non un seul.

4.1.4. Caractères spéciaux

Certains caractères non alphanumériques ont une signification particulière, différente de celle d'un opérateur. Pour plus de détails sur leur utilisation, veuillez consulter la description de l'élément syntaxique correspondant.

Cette section a pour seul but d'informer de l'existence de ces personnages et d'en résumer les objectifs.

- Le symbole dollar (\$) suivi de chiffres sert à représenter un paramètre positionnel dans le corps d'une définition de fonction ou d'une instruction préparée. Dans d'autres contextes, le symbole dollar peut faire partie d'un identifiant ou d'une constante de chaîne entre guillemets.

Les parenthèses (()) servent généralement à regrouper des expressions et à définir la priorité des opérations. Dans certains cas, elles sont obligatoires car elles font partie intégrante de la syntaxe d'une commande SQL particulière.

Les crochets ([]) servent à sélectionner les éléments d'un tableau. Voir la section 8.15 pour plus d'informations.
sur les tableaux.

- Les virgules (,) sont utilisées dans certaines constructions syntaxiques pour séparer les éléments d'une liste.
- Le point-virgule (;) termine une commande SQL. Il ne peut apparaître nulle part à l'intérieur d'une commande, sauf à l'intérieur d'une constante de chaîne ou d'un identificateur entre guillemets.
- Le deux-points (:) est utilisé pour sélectionner des « tranches » dans des tableaux. (Voir section 8.15.) Dans certains dialectes SQL (comme dans le SQL intégré), le deux-points est utilisé pour préfixer les noms de variables.
- L'astérisque (*) est utilisé dans certains contextes pour désigner tous les champs d'une ligne de tableau ou d'une valeur composite. Il revêt également une signification particulière lorsqu'il est utilisé comme argument d'une fonction d'agrégation, à savoir que L'agrégat ne requiert aucun paramètre explicite.
- Le point (.) est utilisé dans les constantes numériques et pour séparer les noms de schéma, de table et de colonne.

4.1.5. Commentaires

Un commentaire est une séquence de caractères commençant par des doubles tirets et se prolongeant jusqu'à la fin de la ligne, par exemple :

```
-- Ceci est un commentaire SQL standard
```

Il est également possible d'utiliser des commentaires de bloc de style C :

```
/* commentaire multiligne
 * avec imbrication : /* commentaire de bloc imbriqué */
 */
```

où le commentaire commence par /* et se poursuit jusqu'à l'occurrence correspondante de */. Ces commentaires de bloc s'imbriquent, comme spécifié dans la norme SQL mais contrairement au C, ce qui permet de commenter des blocs plus importants, de code pouvant contenir des commentaires de bloc existants.

Un commentaire est supprimé du flux d'entrée avant toute analyse syntaxique ultérieure et est effectivement remplacé par espace blanc.

4.1.6. Priorité de l'opérateur

Le tableau 4.2 présente la priorité et l'associativité des opérateurs dans PostgreSQL. La plupart des opérateurs ont la même priorité et sont associatifs à gauche. La priorité et l'associativité des opérateurs sont intégrées au système d'analyse syntaxique. Ajoutez des parenthèses si vous souhaitez analyser une expression contenant plusieurs opérateurs, d'une autre manière que celle qu'impliquent les règles de priorité.

Tableau 4.2. Priorité des opérateurs (de la plus élevée à la plus basse)

Opérateur/Élément	Associativité	Description
.	gauche	séparateur de noms de tableaux/colonnes
::	gauche	Typecast de style PostgreSQL
[]	gauche	sélection d'éléments de tableau
+ -	droite	unaire plus, unaire moins
COLLATIONNER	gauche	sélection de collation
À	gauche	À L'HEURE DU FUSEAU HORAIRE, À L'HEURE LOCALE
^	gauche	exponentiation
* / %	gauche	multiplication, division, modulo
+ -	gauche	addition, soustraction

Opérateur/Élément	Associativité	Description
(tout autre opérateur)	gauche	tous les autres opérateurs natifs et définis par l'utilisateur
ENTRE DANS COMME JE SUIS SIMILAIRE		confinement de la portée, appartenance à un ensemble, correspondance de chaînes
< > = <= >= <>		opérateurs de comparaison
IS ISNULL NOTNULL		EST VRAI, EST FAUX, EST NUL, EST DISTINCT DE, etc.
PAS	droite	négation logique
ET	gauche	conjonction logique
OU	gauche	disjonction logique

Notez que les règles de priorité des opérateurs s'appliquent également aux opérateurs définis par l'utilisateur qui portent le même nom. comme les opérateurs intégrés mentionnés ci-dessus. Par exemple, si vous définissez un opérateur « + » pour une fonction personnalisée. Le type de données aura la même priorité que l'opérateur intégré « + », quel que soit le comportement du vôtre.

Lorsqu'un nom d'opérateur qualifié par schéma est utilisé dans la syntaxe OPERATOR, comme par exemple dans :

```
SÉLECTIONNER 3 OPÉRATEUR(pg_catalog.+) 4;
```

La construction OPERATOR est considérée comme ayant la priorité par défaut indiquée dans le tableau 4.2 pour « tout autre opérateur ». Ceci est vrai quel que soit l'opérateur spécifique qui apparaît à l'intérieur de OPERATOR().

Note

Les versions de PostgreSQL antérieures à la 9.5 utilisaient des règles de priorité des opérateurs légèrement différentes. En particulier, les opérateurs <=, >= et <> étaient traités comme des opérateurs génériques ; les tests IS avaient une priorité plus élevée ; et les constructions NOT BETWEEN et apparentées se comportaient de manière incohérente, étant prises en compte dans certains cas. Dans certains cas, la priorité de NOT est donnée à NOT plutôt qu'à BETWEEN. Ces règles ont été modifiées pour une meilleure conformité à la norme SQL et une réduction de la confusion due à un traitement incohérent de constructions logiquement équivalentes. Dans la plupart des cas, ces changements n'entraîneront aucun changement de comportement, ou peut-être des erreurs de type « opérateur inexistant » qui peuvent être résolues en ajoutant des parenthèses. Il existe cependant des cas particuliers où le comportement d'une requête peut changer sans aucune analyse syntaxique. Une erreur a été signalée.

4.2. Expressions de valeurs

Les expressions de valeur sont utilisées dans divers contextes, comme par exemple dans la liste cible de la commande SELECT. comme nouvelles valeurs de colonnes dans les commandes INSERT ou UPDATE, ou dans les conditions de recherche de plusieurs commandes. Le résultat d'une expression de valeur est parfois appelé scalaire, pour le distinguer du résultat d'une table. expression (qui est un tableau). Les expressions de valeur sont donc également appelées expressions scalaires (ou même (des expressions simples)). La syntaxe des expressions permet de calculer des valeurs à partir de parties primitives en utilisant opérations arithmétiques, logiques, ensemblistes et autres.

Une expression de valeur est l'une des suivantes :

- Une valeur constante ou littérale
- Référence à une colonne
- Une référence à un paramètre positionnel, dans le corps d'une définition de fonction ou d'une instruction préparée
- Une expression sous indice

- Une expression de sélection de champ
- Un appel d'opérateur
- Un appel de fonction
- Une expression agrégée
- Un appel de fonction de fenêtre
- Un type cast
- Une expression de collation
- Une sous-requête scalaire
- Un constructeur de tableau
- Un constructeur de lignes
- Une autre expression de valeur entre parenthèses (utilisée pour regrouper des sous-expressions et modifier la priorité)

Outre cette liste, il existe un certain nombre de constructions qui peuvent être classées comme des expressions, mais qui ne suivent aucune règle de syntaxe générale. Elles ont généralement la sémantique d'une fonction ou d'un opérateur et sont expliquées dans la section appropriée du chapitre 9. La clause IS NULL en est un exemple.

Nous avons déjà abordé les constantes dans la section 4.1.2. Les sections suivantes traitent des options restantes.

4.2.1. Références de colonnes

Une colonne peut être référencée sous la forme :

nom de colonne de corrélation

La corrélation est le nom d'une table (éventuellement accompagné d'un nom de schéma), ou un alias d'une table définie par une clause FROM. Le nom de la corrélation et le point de séparation peuvent être omis si le nom de la colonne est unique dans toutes les tables utilisées par la requête. (Voir également le chapitre 7.)

4.2.2. Paramètres de position

Une référence de paramètre positionnel sert à indiquer une valeur fournie en dehors d'une instruction SQL. Les paramètres sont utilisés dans les définitions de fonctions SQL et dans les requêtes préparées. Certaines bibliothèques clientes permettent également de spécifier des valeurs de données séparément de la chaîne de commande SQL ; dans ce cas, les paramètres servent à faire référence à ces valeurs de données externes. La forme d'une référence de paramètre est la suivante :

\$numéro

Par exemple, considérons la définition d'une fonction, dept, comme suit :

```
CRÉER UNE FONCTION dept(texte) RETOURNE dept
AS $$ SELECT * FROM dept WHERE name = $1 $$
LANGAGE SQL ;
```

Ici, \$1 fait référence à la valeur du premier argument de la fonction chaque fois que celle-ci est appelée.

4.2.3. Indices

Si une expression renvoie une valeur de type tableau, alors un élément spécifique de cette valeur peut être extrait en écrivant

```
expression[indice]
```

ou plusieurs éléments adjacents (une « tranche de tableau ») peuvent être extraits en écrivant

```
expression[indice_inférieur:indice_supérieur]
```

(Ici, les crochets [] doivent être interprétés littéralement.) Chaque indice est lui-même une expression, qui sera arrondie à la valeur entière la plus proche.

En général, l'expression matricielle doit être placée entre parenthèses, mais celles-ci peuvent être omises lorsque l'expression à indexer est simplement une référence de colonne ou un paramètre positionnel. De plus, plusieurs indices peuvent être concaténés lorsque le tableau d'origine est multidimensionnel. Par exemple :

```
mytable.arraycolumn[4]  
mytable.two_d_column[17][34] $1[10:42]
```

```
(arrayfunction(a,b))[42]
```

Les parenthèses dans le dernier exemple sont obligatoires. Voir la section 8.15 pour plus d'informations sur les tableaux.

4.2.4. Sélection du champ

Si une expression renvoie une valeur de type composite (type ligne), alors un champ spécifique de la ligne peut être extrait en écrivant

```
expression.nom_de_champ
```

En général, l'expression de ligne doit être placée entre parenthèses, mais celles-ci peuvent être omises lorsque l'expression à sélectionner est simplement une référence de table ou un paramètre positionnel. Par exemple :

```
mytable.mycolumn  
$1.somecolumn  
(rowfunction(a,b)).col3
```

(Ainsi, une référence de colonne qualifiée n'est en réalité qu'un cas particulier de la syntaxe de sélection de champ.) Un cas particulier important consiste à extraire un champ d'une colonne de table de type composite :

```
(compositecol).somefield  
(mytable.compositecol).somefield
```

Les parenthèses sont nécessaires ici pour indiquer que compositecol est un nom de colonne et non un nom de table, ou que mytable est un nom de table et non un nom de schéma dans le second cas.

Vous pouvez demander tous les champs d'une valeur composite en écrivant .* :

```
(compositecol).*
```

Cette notation se comporte différemment selon le contexte ; voir la section 8.16.5 pour plus de détails.

4.2.5. Appels d'opérateurs

Il existe deux syntaxes possibles pour l'appel d'un opérateur :

expression opérateur expression (opérateur infixe binaire)
 expression d'opérateur (opérateur préfixe unaire)

où le jeton opérateur suit les règles de syntaxe de la section 4.1.3, ou est l'un des mots clés ET, OU, et NON, ou est un nom d'opérateur qualifié sous la forme :

OPÉRATEUR(schéma.nom_opérateur)

Les opérateurs spécifiques existants et leur nature unaire ou binaire dépendent des opérateurs disponibles. ont été définies par le système ou l'utilisateur. Le chapitre 9 décrit les opérateurs intégrés.

4.2.6. Appels de fonction

La syntaxe d'un appel de fonction est le nom de la fonction (éventuellement qualifié par un nom de schéma), suivi de sa liste d'arguments entre parenthèses :

nom_fonction ([expression [, expression ...]])

Par exemple, le code suivant calcule la racine carrée de 2 :

```
racine_carrée(2)
```

La liste des fonctions intégrées se trouve au chapitre 9. L'utilisateur peut ajouter d'autres fonctions.

Lors de l'exécution de requêtes dans une base de données où certains utilisateurs se méfient les uns des autres, respectez les précautions de sécurité. à partir de la section 10.3 lors de l'écriture des appels de fonction.

Il est possible d'associer des noms aux arguments. Voir la section 4.3 pour plus de détails.

Note

Une fonction qui prend un seul argument de type composite peut être appelée, de manière optionnelle, en utilisant la syntaxe de sélection de champ, et inversement, la sélection de champ peut être écrite dans un style fonctionnel. Autrement dit Les notations `col(table)` et `table.col` sont interchangeables. Ce comportement n'est pas conforme à la norme SQL, mais il est proposé par PostgreSQL car il permet l'utilisation de fonctions pour simuler des « champs calculés ». Pour plus d'informations, consultez la section 8.16.5.

4.2.7. Expressions agrégées

Une expression agrégée représente l'application d'une fonction d'agrégation sur les lignes sélectionnées par une requête. Une fonction d'agrégation réduit plusieurs entrées à une seule valeur de sortie, telle que la somme ou moyenne des entrées. La syntaxe d'une expression d'agrégation est l'une des suivantes :

```
nom_agrégat (expression [ , ... ] [ order_by_clause ] ) [ FILTER
  ( WHERE filter_clause ) aggregate_name ]
(ALL expression [ , ... ] [ order_by_clause ] )
[ FILTER ( OÙ clause_filtre ) nom_agrégat (DISTINCT
expression [ , ... ] [ order_by_clause ] )
[ FILTER ( OÙ clause_filtre ) nom_agrégat ( * ) ]
nom_agrégat ( ( clause_trier ) [ FILTER ( OÙ clause_filtre ) [ expression [ , ... ] ] ) AU SEIN DU GROUPE
[ FILTER ( OÙ clause_filtre ) ]
```

où `aggregate_name` est un agrégat préalablement défini (éventuellement qualifié par un nom de schéma) et `expression` est une expression de valeur quelconque qui ne contient pas elle-même une expression d'agrégat ou un appel de fonction de fenêtre. Les clauses optionnelles `order_by_clause` et `filter_clause` sont décrites ci-dessous.

La première forme d'expression d'agrégation appelle l'agrégation une fois pour chaque ligne d'entrée. La deuxième forme est identique à la première, puisque `ALL` est la valeur par défaut. La troisième forme appelle l'agrégation une fois pour chaque valeur distincte de l'expression (ou ensemble de valeurs distinctes, dans le cas d'expressions multiples) trouvée dans les lignes d'entrée. La quatrième forme appelle également l'agrégation une fois pour chaque ligne d'entrée ; comme aucune valeur d'entrée particulière n'est spécifiée, elle est généralement utile uniquement pour la fonction d'agrégation `count(*)`. La dernière forme est utilisée avec les fonctions d'agrégation d'ensembles ordonnés, décrites ci-dessous.

La plupart des fonctions d'agrégation ignorent les valeurs nulles en entrée ; les lignes où une ou plusieurs expressions renvoient une valeur nulle sont donc ignorées. Ce comportement est généralement admis pour toutes les fonctions d'agrégation intégrées, sauf indication contraire.

Par exemple, `count(*)` renvoie le nombre total de lignes d'entrée ; `count(f1)` renvoie le nombre de lignes d'entrée dans lesquelles `f1` n'est pas nul, puisque `count` ignore les valeurs nulles ; et `count(distinct f1)` renvoie le nombre de valeurs distinctes non nulles de `f1`.

En règle générale, les lignes d'entrée sont transmises à la fonction d'agrégation dans un ordre indéterminé. Dans de nombreux cas, cela n'a pas d'importance ; par exemple, la fonction `min` produit le même résultat quel que soit l'ordre dans lequel les lignes d'entrée lui sont transmises. Cependant, certaines fonctions d'agrégation (telles que `array_agg` et `string_agg`) produisent des résultats qui dépendent de l'ordre des lignes d'entrée. Lors de l'utilisation d'une telle agrégation, la clause optionnelle `order_by_clause` permet de spécifier l'ordre souhaité. La clause `order_by_clause` a la même syntaxe que celle d'une clause `ORDER BY` au niveau de la requête, comme décrit dans la section 7.5, à la différence que ses expressions sont toujours des expressions et ne peuvent pas être des noms de colonnes de sortie ni des nombres. Par exemple :

```
AVEC vals (v) COMME ( VALEURS (1),(3),(4),(3),(2) )
SELECT array_agg(v ORDER BY v DESC) FROM vals; array_agg
```

```
-----
{4,3,3,2,1}
```

Étant donné que `jsonb` ne conserve que la dernière clé correspondante, l'ordre de ses clés peut être significatif :

```
AVEC vals (k, v) AS ( VALUES ('key0','1'), ('key1','3'), ('key1','2') )
```

```
SELECT jsonb_object_agg(k, v ORDER BY v) FROM vals;
-----
jsonb_object_agg
```

```
{"key0": "1", "key1": "3"}
```

Lorsqu'on utilise des fonctions d'agrégation à plusieurs arguments, il faut noter que la clause `ORDER BY` doit être placée après tous les arguments d'agrégation. Par exemple :

```
SELECT string_agg(a, ',' ORDER BY a) FROM table;
```

pas ceci :

```
SELECT string_agg(a ORDER BY a, ',') FROM table; -- incorrect
```

Cette dernière est syntaxiquement valide, mais elle représente un appel à une fonction d'agrégation à un seul argument avec deux clés `ORDER BY` (la seconde étant plutôt inutile puisqu'il s'agit d'une constante).

Si `DISTINCT` est spécifié avec une clause `ORDER BY`, les expressions `ORDER BY` ne peuvent faire référence qu'aux colonnes de la liste `DISTINCT`. Par exemple :

```

AVEC vals (v) COMME ( VALEURS (1),(3),(4),(3),(2) )
SELECT array_agg(DISTINCT v ORDER BY v DESC) FROM vals;
-----
tableau_agg
{4,3,2,1}

```

L'utilisation de la clause ORDER BY dans la liste des arguments classiques d'une fonction d'agrégation, comme décrit précédemment, permet de trier les lignes d'entrée pour les agrégats généraux et statistiques, pour lesquels le tri est optionnel. Il existe une sous-classe de fonctions d'agrégation appelées agrégats d'ensembles ordonnés, pour lesquels une clause ORDER BY est requise, généralement parce que le calcul de l'agrégation n'est pertinent que pour un ordre spécifique de ses lignes d'entrée. Les calculs de rang et de percentile sont des exemples typiques d'agrégats d'ensembles ordonnés. Pour un agrégat d'ensembles ordonnés, la clause ORDER BY est placée à l'intérieur de WITHIN GROUP (...), comme illustré dans la syntaxe alternative finale ci-dessus. Les expressions de la clause ORDER BY sont évaluées une fois par ligne d'entrée, comme les arguments classiques d'une fonction d'agrégation, triées selon les exigences de la clause ORDER BY, puis transmises à la fonction d'agrégation en tant qu'arguments d'entrée (Ceci diffère du cas d'une clause ORDER BY autre que WITHIN GROUP, qui n'est pas traitée comme un ou plusieurs arguments de la fonction d'agrégation.) Les expressions d'arguments précédant WITHIN GROUP, le cas échéant, sont appelées arguments directs afin de les distinguer des arguments agrégés listés dans la clause ORDER BY. Contrairement aux arguments d'agrégation classiques, les arguments directs ne sont évalués qu'une seule fois par appel d'agrégation, et non une fois par ligne d'entrée. Cela signifie qu'ils ne peuvent contenir de variables que si ces variables sont regroupées par GROUP BY ; cette restriction est identique à celle appliquée si les arguments directs n'étaient pas inclus dans une expression d'agrégation. Les arguments directs sont généralement utilisés pour des éléments tels que les fractions de percentile, qui n'ont de sens que sous forme de valeur unique par calcul d'agrégation. La liste des arguments directs peut être vide ; dans ce cas, écrivez simplement () et non (*). (PostgreSQL accepte les deux orthographes, mais seule la première est conforme à la norme SQL.)

Voici un exemple d'appel d'agrégation d'ensemble ordonné :

```

SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY income) FROM
ménages;
-----
percentile_cont
50489

```

qui obtient la valeur du 50e percentile, ou médiane, de la colonne des revenus des ménages du tableau.

Ici, 0,5 est un argument direct ; il serait absurde que la fraction de percentile soit une valeur variable à travers les rangées.

Si l'option FILTER est spécifiée, seules les lignes d'entrée pour lesquelles la clause filter_clause est évaluée à vrai sont transmises à la fonction d'agrégation ; les autres lignes sont ignorées. Par exemple :

```

SÉLECTIONNER
    count(*) AS non filtré, count(*) FILTER
    (WHERE i < 5) AS filtré
FROM generate_series(1,10) AS s(i); non filtré | filtré
-----+-----
          10 |          4
(1 rangée)

```

Les fonctions d'agrégation prédéfinies sont décrites dans la section 9.21. D'autres fonctions d'agrégation peuvent être ajoutées par l'utilisateur.

Une expression agrégée ne peut apparaître que dans la liste de résultats ou dans la clause HAVING d'une commande SELECT. Son utilisation est interdite dans d'autres clauses, telles que WHERE, car ces clauses sont évaluées logiquement avant que les résultats des agrégats ne soient formés.

Lorsqu'une expression d'agrégation apparaît dans une sous-requête (voir sections 4.2.11 et 9.24), elle est normalement évaluée sur les lignes de la sous-requête. Toutefois, une exception se produit si les arguments de l'agrégation (et la clause `filter\_clause`, le cas échéant) ne contiennent que des variables de niveau externe : l'agrégation est alors rattachée au niveau externe le plus proche et évaluée sur les lignes de cette sous-requête. L'expression d'agrégation, dans son ensemble, constitue alors une référence externe pour la sous-requête dans laquelle elle apparaît et se comporte comme une constante lors de toute évaluation de cette sous-requête. La restriction concernant sa présence uniquement dans la liste des résultats ou la clause `HAVING` s'applique au niveau de requête auquel appartient l'agrégation.

4.2.8. Appels de fonctions de fenêtre

Un appel de fonction de fenêtre représente l'application d'une fonction d'agrégation à une partie des lignes sélectionnées par une requête. Contrairement aux appels d'agrégation classiques, il n'est pas lié au regroupement des lignes sélectionnées en une seule ligne de sortie : chaque ligne reste distincte dans le résultat de la requête. Cependant, la fonction de fenêtre a accès à toutes les lignes qui feraient partie du groupe de la ligne courante, conformément à la spécification de regroupement (liste PARTITION BY) de l'appel de fonction de fenêtre. La syntaxe d'un appel de fonction de fenêtre est l'une des suivantes :

```
nom_fonction ([expression [, expression ... ]]) [ FILTRE ] SUR nom_fenêtre nom_fonction
  ( WHERE filter_clause )                ([expression [, expression ... ]])
[ FILTRE ]
  ( WHERE filter_clause ) nom_fonction  ] SUR ( définition_fenêtre )
( * )                                [ FILTRER ( OÙ clause_filtre ) ]
  SUR nom_fenêtre
nom_fonction ( * )                    [ FILTRER ( OÙ clause_filtre ) ] SUR
  ( définition_fenêtre )
```

où window_definition a la syntaxe

```
[ nom_fenêtre_existant ]
[ PARTITION PAR expression [, ...] ]
[ ORDER BY expression [ ASC | DESC | opérateur USING ] ] [ NULLS
  { PREMIER | NOM } ] [, ...] [ clause_cadre ]
```

La clause de cadre optionnelle peut être l'une des suivantes :

```
{ PLAGE | LIGNES | GROUPES } début_cadre [ exclusion_cadre ]
{ PLAGE | LIGNES | GROUPES } ENTRE frame_start ET frame_end [ exclusion_de_cadre ]
```

où frame_start et frame_end peuvent être l'un des suivants :

```
ILLIMITÉ PRÉCÉDENT
décaler PRÉCÉDENT
LIGNE ACTUELLE
décalage SUIVANT
SUIVI ILLIMITÉ
```

et frame_exclusion peut être l'un des

```
EXCLURE LA LIGNE ACTUELLE
EXCLURE GROUPE
EXCLURE LES ÉGALITÉS
N'EXCLURE AUCUN AUTRE
```

Ici, « expression » représente toute expression de valeur qui ne contient pas elle-même d'appels de fonctions de fenêtre.

`window_name` fait référence à une spécification de fenêtre nommée définie dans la clause `WINDOW` de la requête. Il est également possible de spécifier une définition de fenêtre complète entre parenthèses, en utilisant la même syntaxe que pour la définition d'une fenêtre nommée dans la clause `WINDOW` ; consultez la page de référence `SELECT` pour plus de détails. Il convient de noter que `OVER wname` n'est pas exactement équivalent à `OVER (wname ...)` ; cette dernière formulation implique la copie et la modification de la définition de la fenêtre et sera rejetée si la spécification de la fenêtre référencée inclut une clause `frame`.

La clause `PARTITION BY` regroupe les lignes de la requête en partitions, lesquelles sont traitées séparément par la fonction de fenêtre. Son fonctionnement est similaire à celui d'une clause `GROUP BY` au niveau de la requête, à ceci près que ses expressions sont toujours des expressions et ne peuvent pas être des noms ou des numéros de colonnes de sortie. Sans `PARTITION BY`, toutes les lignes produites par la requête sont traitées comme une seule partition. La clause `ORDER BY` détermine l'ordre dans lequel les lignes d'une partition sont traitées par la fonction de fenêtre. Son fonctionnement est similaire à celui d'une clause `ORDER BY` au niveau de la requête, mais elle ne peut pas non plus utiliser de noms ou de numéros de colonnes de sortie. Sans `ORDER BY`, les lignes sont traitées dans un ordre indéterminé.

La clause `frame_clause` spécifie l'ensemble des lignes constituant le cadre de la fenêtre, qui est un sous-ensemble de la partition actuelle, pour les fonctions de fenêtre qui agissent sur le cadre au lieu de la partition entière.

L'ensemble des lignes du cadre peut varier selon la ligne courante. Le cadre peut être spécifié en mode `PLAGE`, `LIGNES` ou `GROUPE` ; dans chaque cas, il s'étend de `frame_start` à `frame_end`. Si `frame_end` est omis, la fin est par défaut la `LIGNE COURANTE`.

Un `frame_start` de `UNBOUNDED PRECEDING` signifie que la trame commence par la première ligne de la partition, et de même un `frame_end` de `UNBOUNDED FOLLOWING` signifie que la trame se termine par la dernière ligne de la partition.

En mode `RANGE` ou `GROUPE`, un `frame_start` de `CURRENT ROW` signifie que la fenêtre commence à la première ligne homologue de la ligne courante (une ligne que la clause `ORDER BY` de la fenêtre considère comme équivalente à la ligne courante), tandis qu'un `frame_end` de `CURRENT ROW` signifie que la fenêtre se termine à la dernière ligne homologue de la ligne courante. En mode `ROWS`, `CURRENT ROW` désigne simplement la ligne courante.

Dans les options de trame « décalage `PRECEDING` » et « décalage `FOLLOWING` », le décalage doit être une expression ne contenant aucune variable, fonction d'agrégation ni fonction de fenêtre. La signification du décalage dépend du mode de trame :

- En mode `ROWS`, le décalage doit donner un entier non nul et non négatif, et l'option signifie que le cadre commence ou se termine le nombre de lignes spécifié avant ou après la ligne actuelle.
- En mode `GROUPE`, le décalage doit à nouveau donner un entier non nul et non négatif, et l'option signifie que le cadre commence ou se termine le nombre spécifié de groupes homologues avant ou après le groupe homologue de la ligne actuelle, où un groupe homologue est un ensemble de lignes équivalentes dans l'ordre `ORDER BY`. (Une clause `ORDER BY` doit figurer dans la définition de la fenêtre pour utiliser le mode `GROUPE`.)

En mode `RANGE`, ces options exigent que la clause `ORDER BY` spécifie une seule colonne. Le décalage indique la différence maximale entre la valeur de cette colonne dans la ligne courante et sa valeur dans les lignes précédentes ou suivantes du cadre. Le type de données de l'expression de décalage varie selon le type de données de la colonne de tri. Pour les colonnes de tri numériques, il est généralement du même type que la colonne de tri, mais pour les colonnes de tri de type date/heure, il s'agit d'un intervalle.

Par exemple, si la colonne de tri est de type date ou horodatage, on peut écrire `RANGE BETWEEN '1 day' PRECEDING AND '10 days' FOLLOWING`. Le décalage doit toujours être non nul et non négatif, bien que la signification de « non négatif » dépende de son type de données.

Dans tous les cas, la distance jusqu'au bout du cadre est limitée par la distance jusqu'au bout de la partition, de sorte que pour les lignes proches des extrémités de la partition, le cadre peut contenir moins de lignes qu'ailleurs.

Notez qu'en modes `ROWS` et `GROUPE`, `0 PRECEDING` et `0 FOLLOWING` sont équivalents à `CURRENT ROW`. Cela est généralement également vrai en mode `RANGE`, pour une signification de « zéro » adaptée au type de données.

L'option `frame_exclusion` permet d'exclure du cadre les lignes adjacentes à la ligne courante, même si elles seraient incluses selon les options `frame_start` et `frame_end`. `EXCLUDE CURRENT ROW` exclut la ligne courante du cadre. `EXCLUDE GROUP` exclut la ligne courante et ses lignes homologues. `EXCLUDE TIES` exclut toutes les lignes homologues de la ligne courante, mais pas la ligne courante elle-même. `EXCLUDE NO OTHERS` spécifie simplement le comportement par défaut : ne pas exclure la ligne courante ni ses lignes homologues.

L'option de cadrage par défaut est `RANGE UNBOUNDED PRECEDING`, ce qui correspond à `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Avec `ORDER BY`, le cadrage inclut toutes les lignes depuis le début de la partition jusqu'à la dernière ligne homologue de la ligne courante, selon la clause `ORDER BY`. Sans `ORDER BY`, toutes les lignes de la partition sont incluses dans le cadrage, car chaque ligne devient homologue de la ligne courante.

Les restrictions sont les suivantes : `frame_start` ne peut pas être `UNBOUNDED FOLLOWING`, `frame_end` ne peut pas être `UNBOUNDED PRECEDING`, et l'option `frame_end` ne peut pas apparaître avant l'option `frame_start` dans la liste des options `frame_start` et `frame_end` ci-dessus ; par exemple, `RANGE BETWEEN CURRENT ROW AND offset PRECEDING` n'est pas autorisé. En revanche, `ROWS BETWEEN 7 PRECEDING AND 8 PRECEDING` est autorisé, même s'il ne sélectionnerait jamais de lignes.

Si l'option `FILTER` est spécifiée, seules les lignes d'entrée pour lesquelles la clause `filter_clause` est vraie sont transmises à la fonction de fenêtre ; les autres lignes sont ignorées. Seules les fonctions de fenêtre agrégées acceptent une clause `FILTER`.

Les fonctions de fenêtrage intégrées sont décrites dans le tableau 9.65. L'utilisateur peut ajouter d'autres fonctions de fenêtrage. De plus, tout agrégat général ou statistique, intégré ou défini par l'utilisateur, peut servir de fonction de fenêtrage. (Les agrégats d'ensembles ordonnés et d'ensembles hypothétiques ne peuvent actuellement pas être utilisés comme fonctions de fenêtrage.)

La syntaxe utilisant `*` permet d'appeler des fonctions d'agrégation sans paramètre comme des fonctions de fenêtre, par exemple `count(*) OVER (PARTITION BY x ORDER BY y)`. L'astérisque (`*`) n'est généralement pas utilisé pour les fonctions spécifiques aux fenêtres. Ces dernières n'acceptent pas les clauses `DISTINCT` ou `ORDER BY` dans leur liste d'arguments.

Les appels de fonctions de fenêtre sont autorisés uniquement dans la liste `SELECT` et la clause `ORDER BY` de la requête.

Vous trouverez plus d'informations sur les fonctions de fenêtre dans la section 3.5, la section 9.22 et la section 7.2.5.

4.2.9. Casting de types

Une conversion de type spécifie une conversion d'un type de données à un autre. PostgreSQL accepte deux syntaxes équivalentes pour les conversions de type :

`CAST ( expression AS type ) expression::type`

La syntaxe `CAST` est conforme à SQL ; la syntaxe avec `::` correspond à l'usage historique de PostgreSQL.

Lorsqu'une conversion de type est appliquée à une expression de valeur d'un type connu, elle représente une conversion de type à l'exécution. La conversion réussira uniquement si une opération de conversion de type appropriée a été définie. Notez que cela diffère légèrement de l'utilisation des conversions avec des constantes, comme illustré dans la section 4.1.2.7. Une conversion appliquée à une chaîne littérale non modifiée représente l'affectation initiale d'un type à une valeur constante littérale ; elle réussira donc pour tout type (si le contenu de la chaîne littérale correspond à la syntaxe d'entrée acceptable pour le type de données).

Un transtypage explicite peut généralement être omis s'il n'y a pas d'ambiguïté quant au type que doit produire une expression de valeur (par exemple, lorsqu'elle est affectée à une colonne de table) ; le système effectuera automatiquement un transtypage dans ce cas. Toutefois, le transtypage automatique n'est effectué que pour les conversions marquées « OK ».

Cette fonction s'applique implicitement dans les catalogues système. Les autres conversions doivent être effectuées avec une syntaxe de conversion explicite. Cette restriction vise à empêcher l'application silencieuse de conversions inattendues.

Il est également possible de spécifier une conversion de type en utilisant une syntaxe de type fonction :

`nom_de_type (expression)`

Cependant, cela ne fonctionne que pour les types dont le nom est également valide comme nom de fonction. Par exemple, la double précision ne peut pas être utilisée de cette manière, contrairement à son équivalent `float8`. De même, les noms `interval`, `time` et `timestamp` ne peuvent être utilisés ainsi que s'ils sont placés entre guillemets doubles, en raison de conflits syntaxiques. Par conséquent, l'utilisation de cette syntaxe de conversion de type fonction engendre des incohérences et est probablement à éviter.

Note

La syntaxe ressemblant à une fonction correspond en réalité à un simple appel de fonction. Lorsqu'une des deux syntaxes de conversion standard est utilisée pour effectuer une conversion à l'exécution, elle invoque en interne une fonction enregistrée pour réaliser cette conversion. Par convention, ces fonctions de conversion portent le même nom que leur type de sortie ; ainsi, la « syntaxe ressemblant à une fonction » n'est rien d'autre qu'un appel direct à la fonction de conversion sous-jacente. Il est évident qu'une application portable ne devrait pas s'appuyer sur ce type de syntaxe. Pour plus de détails, consultez `CREATE CAST`.

4.2.10. Expressions de classement

La clause `COLLATE` remplace le classement d'une expression. Elle est ajoutée à l'expression à laquelle elle s'applique :

`expr COLLATE collation`

où le classement est un identifiant pouvant être qualifié par un schéma. La clause `COLLATE` impose une contrainte plus stricte que les opérateurs ; les parenthèses peuvent être utilisées si nécessaire.

Si aucun classement n'est explicitement spécifié, le système de base de données dérive un classement à partir des colonnes impliquées dans l'expression, ou il utilise par défaut le classement par défaut de la base de données si aucune colonne n'est impliquée dans l'expression.

Les deux utilisations courantes de la clause `COLLATE` consistent à remplacer l'ordre de tri dans une clause `ORDER BY`, par exemple :

`SÉLECTIONNER a, b, c DE tbl OÙ ... ORDREZ PAR COLLATE "C" ;`

et en remplaçant le classement d'un appel de fonction ou d'opérateur qui produit des résultats sensibles aux paramètres régionaux, par exemple :

`SELECT * FROM tbl WHERE a > 'foo' COLLATE "C";`

Notez que dans ce dernier cas, la clause `COLLATE` est associée à un argument d'entrée de l'opérateur concerné. L'argument auquel elle est associée (de l'opérateur ou de la fonction) est indifférent, car le classement appliqué par l'opérateur ou la fonction est déterminé en considérant tous les arguments ; une clause `COLLATE` explicite prévaudra donc sur le classement de tous les autres arguments.

(Il est toutefois incorrect d'associer des clauses `COLLATE` non concordantes à plusieurs arguments. Pour plus de détails, consultez la section 23.2.) Ainsi, cela donne le même résultat que l'exemple précédent :

```
SELECT * FROM tbl WHERE a COLLATE "C" > 'foo';
```

Mais ceci est une erreur :

```
SELECT * FROM tbl WHERE (a > 'foo') COLLATE "C";
```

car il tente d'appliquer un classement au résultat de l'opérateur >, qui est de type de données booléen non classable.

4.2.11. Sous-requêtes scalaires

Une sous-requête scalaire est une requête SELECT classique entre parenthèses qui renvoie une seule ligne et une seule colonne. (Pour plus d'informations sur l'écriture de requêtes, consultez le chapitre 7.) La requête SELECT est exécutée et la valeur unique renvoyée est utilisée dans l'expression de valeur qui l'entoure. Il est incorrect d'utiliser une requête renvoyant plusieurs lignes ou plusieurs colonnes comme sous-requête scalaire. (Toutefois, si lors d'une exécution particulière, la sous-requête ne renvoie aucune ligne, aucune erreur n'est commise ; le résultat scalaire est considéré comme nul.) La sous-requête peut faire référence à des variables de la requête qui l'entoure, lesquelles se comportent comme des constantes lors de chaque évaluation de la sous-requête. Voir également la section 9.24 pour d'autres expressions impliquant des sous-requêtes.

Par exemple, la formule suivante permet de trouver la ville ayant la plus grande population dans chaque État :

```
SÉLECTIONNER nom, (SÉLECTIONNER max(pop) DE villes OÙ villes.état =
états.nom)
DES états;
```

4.2.12. Constructeurs de tableaux

Un constructeur de tableau est une expression qui construit un tableau à partir des valeurs de ses éléments. Un constructeur de tableau simple se compose du mot-clé ARRAY, d'une accolade ouvrante [, d'une liste d'expressions (séparées par des virgules) pour les valeurs des éléments du tableau, et enfin d'une accolade fermante]. Par exemple :

```
SÉLECTIONNER TABLEAU[1,2,3+4];
-----
tableau
{1,2,7} (1
ligne)
```

Par défaut, le type des éléments du tableau est le type commun des expressions membres, déterminé selon les mêmes règles que pour les constructions UNION ou CASE (voir section 10.5). Vous pouvez modifier ce comportement en convertissant explicitement le constructeur du tableau vers le type souhaité, par exemple :

```
SÉLECTIONNER TABLEAU[1,2,22.7]::entier[];
-----
tableau
{1,2,23} (1
ligne)
```

Cela revient à convertir chaque expression individuellement dans le type d'élément du tableau. Pour plus d'informations sur la conversion de type, consultez la section 4.2.9.

Il est possible de créer des tableaux multidimensionnels en imbriquant des constructeurs de tableaux. Dans les constructeurs internes, le mot-clé ARRAY peut être omis. Par exemple, les constructions suivantes produisent le même résultat :

```
SÉLECTIONNER TABLEAU[TABLEAU[1,2], TABLEAU[3,4]];
```

```

      tableau
-----
      {{1,2},{3,4}}
      (1 rangée)

```

```
SÉLECTIONNER TABLEAU[[1,2],[3,4]];
```

```

      tableau
-----
      {{1,2},{3,4}}
      (1 rangée)

```

Les tableaux multidimensionnels étant nécessairement rectangulaires, les constructeurs internes de même niveau doivent produire des sous-tableaux de dimensions identiques. Toute conversion de type appliquée au constructeur ARRAY externe se propage automatiquement à tous les constructeurs internes.

Les éléments d'un constructeur de tableau multidimensionnel peuvent être n'importe quel élément produisant un tableau du type approprié, et pas seulement une sous-structure ARRAY. Par exemple :

```
CRÉER TABLE arr(f1 int[], f2 int[]);
```

```
INSÉRER DANS arr VALEURS (ARRAY[[1,2],[3,4]], ARRAY[[5,6],[7,8]]);
```

```
SÉLECTIONNER TABLEAU[f1, f2, '{{9,10},{11,12}}'::int[]] DE arr;
```

```

                                tableau
-----
      {{{1,2},{3,4}},{5,6},{7,8}},{9,10},{11,12}}}
      (1 rangée)

```

Vous pouvez créer un tableau vide, mais comme il est impossible d'avoir un tableau sans type, vous devez explicitement convertir votre tableau vide vers le type souhaité. Par exemple :

```
SÉLECTIONNER TABLEAU[]::entier[];
```

```

      tableau
-----
      {}
      (1 ligne)

```

Il est également possible de construire un tableau à partir des résultats d'une sous-requête. Dans ce cas, le constructeur de tableau s'écrit avec le mot-clé ARRAY suivi d'une sous-requête entre parenthèses (et non entre crochets). Par exemple :

```
SÉLECTIONNER TABLEAU(SÉLECTIONNER oid DE pg_proc OÙ pronomme COMME 'bytea%');
```

```

                                tableau
-----
      {2011,1954,1948,1952,1951,1244,1950,2005,1949,1953,2006,31,2412}
      (1 rangée)

```

```
SELECT ARRAY(SELECT ARRAY[i, i*2] FROM generate_series(1,5) AS
a(i));
```

```

                                tableau
-----
      {{1,2},{2,4},{3,6},{4,8},{5,10}}
      (1 rangée)

```

La sous-requête doit renvoyer une seule colonne. Si la colonne de sortie de la sous-requête n'est pas de type tableau, le tableau unidimensionnel résultant contiendra un élément pour chaque ligne du résultat de la sous-requête, avec un

Le type des éléments doit correspondre à celui de la colonne de sortie de la sous-requête. Si cette colonne est de type tableau, le résultat sera un tableau du même type, mais de dimension supérieure ; dans ce cas, toutes les lignes de la sous-requête doivent produire des tableaux de dimension identique, sinon le résultat ne serait pas rectangulaire.

Les indices d'un tableau créé avec la fonction ARRAY commencent toujours par un. Pour plus d'informations sur les tableaux, consultez la section 8.15.

4.2.13. Constructeurs de lignes

Un constructeur de ligne est une expression qui construit une valeur de ligne (également appelée valeur composite) à partir des valeurs de ses champs membres. Un constructeur de ligne se compose du mot-clé ROW, d'une parenthèse ouvrante, de zéro ou plusieurs expressions (séparées par des virgules) pour les valeurs des champs de la ligne, et enfin d'une parenthèse fermante. Par exemple :

```
SÉLECTIONNER LIGNE(1,2.5,'ceci est un test');
```

Le mot-clé ROW est facultatif lorsqu'il y a plus d'une expression dans la liste.

Un constructeur de ligne peut inclure la syntaxe valeur_ligne.*, qui sera développée en une liste des éléments de la valeur de la ligne, comme c'est le cas lorsque la syntaxe .* est utilisée au niveau supérieur d'une liste SELECT (voir section 8.16.5). Par exemple, si la table t possède les colonnes f1 et f2, celles-ci sont identiques :

```
SÉLECTIONNER LIGNE(t.*, 42) DE t;
SÉLECTIONNER LIGNE(t.f1, t.f2, 42) DE t;
```

Note

Avant PostgreSQL 8.2, la syntaxe `.\*` n'était pas interprétée dans les constructeurs de lignes. Ainsi, l'instruction `ROW(t.\*, 42)` créait une ligne à deux champs dont le premier était une autre valeur de ligne. Le nouveau comportement est généralement plus pratique. Si vous souhaitez conserver l'ancien comportement concernant les valeurs de lignes imbriquées, indiquez la valeur de la ligne interne sans `.\*`, par exemple : `ROW(t, 42)`.

Par défaut, la valeur créée par une expression ROW est d'un type d'enregistrement anonyme. Si nécessaire, elle peut être convertie en un type composite nommé : soit le type de ligne d'une table, soit un type composite créé avec CREATE TYPE AS. Une conversion explicite peut être nécessaire pour éviter toute ambiguïté. Par exemple :

```
CRÉER TABLE mytable(f1 int, f2 float, f3 text);
```

```
CRÉER UNE FONCTION getf1(mytable) RETOURNE int AS 'SELECT $1.f1'
  LANGUAGE SQL ;
```

```
-- Aucun cast nécessaire car il n'existe qu'un seul getf1() SELECT getf1(ROW(1,2.5,'ceci est
un test'));
```

```
getf1
-----
      1
(1 rangée)
```

```
CRÉER LE TYPE myrowtype COMME (f1 entier, f2 texte, f3 numérique);
```

```
CRÉER UNE FONCTION getf1(myrowtype) RETOURNE int AS 'SELECT $1.f1'
  LANGUAGE SQL ;
```

```
-- Il nous faut maintenant une conversion de type pour indiquer quelle fonction appeler :
```

```
SELECT getf1(ROW(1,2.5,'this is a test')); ERREUR : la fonction
getf1(record) n'est pas unique
```

```
SELECT getf1(ROW(1,2.5,'ceci est un test')::mytable);
  getf1
-----
      1
(1 rangée)
```

```
SELECT getf1(CAST(ROW(11,'ceci est un test',2.5) AS myrowtype));
  getf1
-----
     11
(1 rangée)
```

Les constructeurs de lignes permettent de créer des valeurs composites destinées à être stockées dans une colonne de type composite ou à être transmises à une fonction acceptant un paramètre composite. Il est également possible de tester les lignes à l'aide des opérateurs de comparaison standard décrits dans la section 9.2, de comparer une ligne à une autre comme décrit dans la section 9.25 et de les utiliser avec des sous-requêtes, comme expliqué dans la section 9.24.

4.2.14. Règles d'évaluation des expressions

L'ordre d'évaluation des sous-expressions n'est pas défini. En particulier, les entrées d'un opérateur ou d'une fonction ne sont pas nécessairement évaluées de gauche à droite ni dans aucun autre ordre fixe.

De plus, si le résultat d'une expression peut être déterminé en évaluant seulement certaines parties de celle-ci, d'autres sous-expressions pourraient ne pas être évaluées du tout. Par exemple, si l'on écrivait :

```
SÉLECTIONNER vrai OU somefunc();
```

alors la fonction somefunc() ne serait (probablement) pas appelée du tout. Il en serait de même si l'on écrivait :

```
SÉLECTIONNER somefunc() OU vrai;
```

Notez que cela n'est pas la même chose que le « court-circuitage » de gauche à droite des opérateurs booléens que l'on trouve dans certains langages de programmation.

Par conséquent, il est imprudent d'utiliser des fonctions à effets de bord dans des expressions complexes. Il est particulièrement dangereux de se fier aux effets de bord ou à l'ordre d'évaluation dans les clauses WHERE et HAVING, car ces clauses sont largement réorganisées lors de l'élaboration du plan d'exécution. Les expressions booléennes (combinaisons AND/OR/NOT) dans ces clauses peuvent être réorganisées de toutes les manières permises par les lois de l'algèbre de Boole.

Lorsqu'il est essentiel de contraindre l'ordre d'évaluation, une instruction CASE (voir section 9.18) peut être utilisée. Par exemple, voici une méthode peu fiable pour tenter d'éviter une division par zéro dans une clause WHERE :

```
SÉLECTIONNER ... OÙ x > 0 ET y/x > 1,5 ;
```

Mais ceci est sans danger :

```
SÉLECTIONNER ... WHERE CASE WHEN x > 0 THEN y/x > 1.5 ELSE false END;
```

L'utilisation d'une instruction CASE de cette manière compromettra les tentatives d'optimisation ; il convient donc de n'y recourir qu'en cas de nécessité. (Dans cet exemple précis, il serait préférable de contourner le problème en écrivant plutôt $y > 1,5 * x$.)

L'instruction CASE n'est cependant pas une solution miracle à ces problèmes. Une des limites de la technique illustrée ci-dessus est qu'elle n'empêche pas l'évaluation anticipée des sous-expressions constantes. Comme décrit dans la section 36.7, les fonctions et les opérateurs marqués IMMUTABLE peuvent être évalués lors de la planification de la requête plutôt que lors de son exécution. Ainsi, par exemple :

```
SÉLECTIONNER CASE WHEN x > 0 THEN x ELSE 1/0 END FROM tab;
```

cela risque d'entraîner une erreur de division par zéro, car le planificateur tente de simplifier la sous-expression constante, même si chaque ligne du tableau a $x > 0$, de sorte que la branche ELSE ne serait jamais exécutée.

Bien que cet exemple puisse paraître anodin, des cas similaires, n'impliquant pas explicitement de constantes, peuvent se présenter dans les requêtes exécutées au sein de fonctions. En effet, les valeurs des arguments de fonction et des variables locales peuvent être utilisées comme constantes dans les requêtes à des fins de planification. Dans les fonctions PL/pgSQL, par exemple, l'utilisation d'une instruction IF-THEN-ELSE pour protéger un calcul risqué est bien plus sûre que son simple placement dans une expression CASE.

Une autre limitation du même ordre est qu'une instruction CASE ne peut empêcher l'évaluation d'une expression d'agrégation qu'elle contient, car les expressions d'agrégation sont calculées avant les autres expressions d'une liste SELECT ou d'une clause HAVING. Par exemple, la requête suivante peut provoquer une erreur de division par zéro malgré une protection apparente contre ce type d'erreur :

```
SELECT CASE WHEN min(employés) > 0
           ALORS moyenne(dépenses / employés)
        FIN
      DES départements;
```

Les fonctions d'agrégation min() et avg() sont calculées simultanément sur toutes les lignes d'entrée. Par conséquent, si une ligne contient la valeur zéro pour le nombre d'employés, une erreur de division par zéro se produira avant même que le résultat de min() puisse être testé. Il est donc recommandé d'utiliser une clause WHERE ou FILTER afin d'empêcher que les lignes problématiques ne soient traitées par une fonction d'agrégation.

4.3. Appel de fonctions

PostgreSQL permet d'appeler les fonctions à paramètres nommés en utilisant soit la notation positionnelle, soit la notation nommée. Cette dernière est particulièrement utile pour les fonctions comportant un grand nombre de paramètres, car elle rend les associations entre les paramètres et leurs arguments effectifs plus explicites et fiables.

En notation positionnelle, l'appel d'une fonction s'écrit avec les valeurs de ses arguments dans le même ordre que celui défini dans la déclaration de la fonction. En notation nommée, les arguments correspondent aux paramètres de la fonction par leur nom et peuvent être écrits dans n'importe quel ordre. Pour chaque notation, il convient également de tenir compte de l'influence des types d'arguments de la fonction, comme indiqué dans la section 10.3.

Dans les deux notations, les paramètres dont les valeurs par défaut sont définies dans la déclaration de la fonction n'ont pas besoin d'être précisés lors de l'appel. Ceci est particulièrement utile dans la notation nommée, car toute combinaison de paramètres peut être omise ; tandis que dans la notation positionnelle, les paramètres ne peuvent être omis que de droite à gauche.

PostgreSQL prend également en charge la notation mixte, qui combine notation positionnelle et notation nommée. Dans ce cas, les paramètres positionnels sont écrits en premier, suivis des paramètres nommés.

Les exemples suivants illustreront l'utilisation des trois notations, à l'aide de la définition de fonction suivante :

```
CRÉER UNE FONCTION concat_lower_or_upper(a texte, b texte, uppercase
    booléen DEFAULT false)
RETOURNE le texte
```

COMME

\$\$

SELECT CASE

QUAND \$3 ALORS MAJUSCULES(\$1 || ' ' || \$2)

SINON INFÉRIEUR(1 \$ || ' ' || 2 \$)

FIN;

\$\$

LANGAGE SQL IMMUTABLE STRICT ;

La fonction `concat_lower_or_upper` possède deux paramètres obligatoires, `a` et `b`. Elle comporte également un paramètre optionnel, `uppercase`, dont la valeur par défaut est `false`. Les valeurs de `a` et `b` seront concaténées et converties en majuscules ou en minuscules selon la valeur du paramètre `uppercase`. Les autres détails de la définition de cette fonction ne sont pas pertinents ici (voir le chapitre 36 pour plus d'informations).

4.3.1. Utilisation de la notation positionnelle

La notation positionnelle est le mécanisme traditionnel pour passer des arguments aux fonctions dans PostgreSQL. Voici un exemple :

```
SELECT concat_lower_or_upper('Hello', 'World', true); concat_lower_or_upper
```

BONJOUR LE MONDE

(1 rangée)

Tous les arguments sont spécifiés dans l'ordre. Le résultat est en majuscules puisque l'option « majuscules » est activée. Un autre exemple est :

```
SELECT concat_lower_or_upper('Hello', 'World'); concat_lower_or_upper
```

Bonjour le monde

(1 rangée)

Ici, le paramètre de casse supérieure est omis ; il prend donc sa valeur par défaut « false », ce qui produit un résultat en minuscules. En notation positionnelle, les arguments peuvent être omis de droite à gauche, pourvu qu'ils aient une valeur par défaut.

4.3.2. Utilisation de la notation nommée

En notation nommée, le nom de chaque argument est spécifié à l'aide de `=>` pour le séparer de l'expression de l'argument. Par exemple :

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World'); concat_lower_or_upper
```

bonjour monde (1

ligne)

Là encore, l'argument `uppercase` a été omis ; sa valeur est donc implicitement définie sur `false`. L'un des avantages de la notation nommée est que les arguments peuvent être spécifiés dans n'importe quel ordre, par exemple :

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World', uppercase
=> vrai);
concaténer_inférieur_ou_supérieur
```

```

-----
BONJOUR LE MONDE
(1 rangée)

SELECT concat_lower_or_upper(a => 'Bonjour', uppercase => true, b =>
'Monde');
concat_minuscule_ou_majuscule
-----
BONJOUR LE MONDE
(1 rangée)

```

Une syntaxe plus ancienne basée sur ":" est prise en charge pour assurer la compatibilité ascendante :

```

SELECT concat_lower_or_upper(a := 'Bonjour', uppercase := true, b :=
'Monde');
concat_minuscule_ou_majuscule
-----
BONJOUR LE MONDE
(1 rangée)

```

4.3.3. Utilisation de la notation mixte

La notation mixte combine la notation positionnelle et la notation nommée. Cependant, comme indiqué précédemment, les arguments nommés ne peuvent pas précéder les arguments positionnels. Par exemple :

```

SELECT concat_lower_or_upper('Hello', 'World', uppercase => true); concat_lower_or_upper
-----
BONJOUR LE MONDE
(1 rangée)

```

Dans la requête ci-dessus, les arguments a et b sont spécifiés par leur position, tandis que la casse supérieure est indiquée par son nom. Dans cet exemple, cela n'apporte que peu d'éléments supplémentaires, hormis la documentation. Pour une fonction plus complexe comportant de nombreux paramètres avec des valeurs par défaut, l'utilisation d'une notation nommée ou mixte permet de gagner un temps précieux et de réduire les risques d'erreur.

Note

Les notations d'appel nommées et mixtes ne peuvent actuellement pas être utilisées lors de l'appel d'une fonction d'agrégation (mais elles fonctionnent lorsqu'une fonction d'agrégation est utilisée comme fonction de fenêtre).

Chapitre 5. Définition des données

Ce chapitre explique comment créer les structures de base de données qui hébergeront vos données. Dans une base de données relationnelle, les données brutes sont stockées dans des tables. La majeure partie de ce chapitre est donc consacrée à la création et à la modification des tables, ainsi qu'aux fonctionnalités permettant de contrôler les données qui y sont stockées. Nous aborderons ensuite l'organisation des tables en schémas et l'attribution de privilèges aux tables. Enfin, nous évoquerons brièvement d'autres fonctionnalités influençant le stockage des données, telles que l'héritage, le partitionnement des tables, les vues, les fonctions et les déclencheurs.

5.1. Notions de base sur les tables

Dans une base de données relationnelle, une table ressemble beaucoup à un tableau papier : elle se compose de lignes et de colonnes. Le nombre et l'ordre des colonnes sont fixes, et chaque colonne possède un nom. Le nombre de lignes est variable ; il reflète la quantité de données stockées à un instant donné. SQL ne garantit pas l'ordre des lignes dans une table. Lors de la lecture d'une table, les lignes apparaissent dans un ordre aléatoire, sauf si un tri est explicitement demandé. Ce point est abordé au chapitre 7. De plus, SQL n'attribue pas d'identifiants uniques aux lignes ; il est donc possible d'avoir plusieurs lignes parfaitement identiques dans une même table. Ceci est une conséquence du modèle mathématique sous-jacent à SQL, mais est généralement indésirable. Nous verrons plus loin dans ce chapitre comment gérer ce problème.

Chaque colonne possède un type de données. Ce type de données limite l'ensemble des valeurs possibles pouvant être attribuées à une colonne et définit la sémantique des données stockées dans la colonne afin qu'elles puissent être utilisées pour des calculs. Par exemple, une colonne de type numérique n'acceptera pas de chaînes de caractères arbitraires, et les données qu'elle contient pourront être utilisées pour des calculs mathématiques. En revanche, une colonne de type chaîne de caractères acceptera presque tous les types de données, mais ne se prête pas aux calculs mathématiques, même si d'autres opérations comme la concaténation de chaînes sont possibles.

PostgreSQL propose un ensemble important de types de données intégrés adaptés à de nombreuses applications. Les utilisateurs peuvent également définir leurs propres types de données. La plupart des types de données intégrés ont des noms et une sémantique explicites ; nous renvoyons donc à une explication détaillée au chapitre 8. Parmi les types de données fréquemment utilisés, on trouve : integer pour les nombres entiers, numeric pour les nombres éventuellement fractionnaires, text pour les chaînes de caractères, date pour les dates, time pour les heures et timestamp pour les valeurs contenant à la fois la date et l'heure.

Pour créer une table, vous utilisez la commande CREATE TABLE. Cette commande vous permet de spécifier au moins un nom pour la nouvelle table, les noms des colonnes et le type de données de chaque colonne. Par exemple :

```
CRÉER TABLE ma_première_table
( première_colonne texte,
  deuxième_colonne entier
);
```

Ceci crée une table nommée my_first_table avec deux colonnes. La première colonne, nommée first_column, est de type texte ; la seconde, nommée second_column, est de type entier. Les noms de la table et des colonnes respectent la syntaxe des identificateurs expliquée dans la section 4.1.1. Les noms de type sont généralement aussi des identificateurs, mais il existe des exceptions. Notez que la liste des colonnes est séparée par des virgules et encadrée par des parenthèses.

Bien sûr, l'exemple précédent était très artificiel. Normalement, on nomme les tables et les colonnes de manière à indiquer le type de données qu'elles contiennent. Prenons donc un exemple plus réaliste :

```
CRÉER TABLE produits ( numéro_produit
entier, nom texte, prix numérique
```

);

(Le type numérique peut stocker des composantes fractionnaires, comme c'est généralement le cas pour les montants monétaires.)

Conseil

Lorsqu'on crée de nombreuses tables interdépendantes, il est judicieux d'adopter une convention de nommage cohérente pour les tables et les colonnes. Par exemple, on peut choisir d'utiliser des noms au singulier ou au pluriel, les deux options ayant leurs préférences respectives.

Le nombre de colonnes qu'une table peut contenir est limité. Selon leur type, il se situe entre 250 et 1 600. Cependant, il est très rare de définir une table avec un nombre de colonnes aussi élevé, ce qui constitue souvent une conception discutable.

Si vous n'avez plus besoin d'une table, vous pouvez la supprimer à l'aide de la commande DROP TABLE. Par exemple :

```
SUPPRIMER LA TABLE my_first_table;  
Produits DROP TABLE ;
```

Tenter de supprimer une table inexistante est une erreur. Cependant, il est courant dans les scripts SQL de tenter systématiquement de supprimer chaque table avant de la créer, en ignorant les messages d'erreur, afin que le script fonctionne que la table existe ou non. (Si vous le souhaitez, vous pouvez utiliser la variante DROP TABLE IF EXISTS pour éviter les messages d'erreur, mais ce n'est pas une pratique courante en SQL.)

Si vous devez modifier un tableau existant, consultez la section 5.7 plus loin dans ce chapitre.

Grâce aux outils présentés jusqu'ici, vous pouvez créer des tables entièrement fonctionnelles. La suite de ce chapitre explique comment ajouter des fonctionnalités à la définition de la table afin de garantir l'intégrité des données, leur sécurité et leur facilité d'utilisation. Si vous êtes impatient de remplir vos tableaux de données dès maintenant, vous pouvez passer directement au chapitre 6 et lire le reste de ce chapitre plus tard.

5.2. Valeurs par défaut

Il est possible d'attribuer une valeur par défaut à une colonne. Lors de la création d'une nouvelle ligne, si certaines colonnes ne sont pas renseignées, elles seront automatiquement initialisées avec leurs valeurs par défaut respectives. Une commande de manipulation de données peut également demander explicitement qu'une colonne soit initialisée avec sa valeur par défaut, sans qu'il soit nécessaire de connaître cette valeur. (Pour plus de détails sur les commandes de manipulation de données, voir le chapitre 6.)

Si aucune valeur par défaut n'est explicitement déclarée, la valeur par défaut est nulle. Cela se justifie généralement car une valeur nulle peut être considérée comme représentant des données inconnues.

Dans la définition d'une table, les valeurs par défaut sont indiquées après le type de données de la colonne. Par exemple :

```
CRÉER TABLE produits ( numéro_produit  
    entier, nom texte, prix numérique  
    DEFAULT 9.99  
);
```

La valeur par défaut peut être une expression, qui sera évaluée lors de l'insertion de cette valeur (et non lors de la création de la table). Par exemple, une colonne d'horodatage peut avoir pour valeur par défaut CURRENT_TIMESTAMP, afin qu'elle corresponde à l'heure d'insertion de la ligne. Autre exemple courant : la génération d'un numéro de série pour chaque ligne. Dans PostgreSQL, cela se fait généralement de la manière suivante :

```
CRÉER DES PRODUITS DE TABLE (
    product_no entier DEFAULT nextval('products_product_no_seq'),
    ...
);
```

où la fonction `nextval()` fournit des valeurs successives à partir d'un objet séquence (voir section 9.17). Cette configuration est suffisamment courante pour qu'il existe un raccourci spécifique pour la désigner :

```
CRÉER TABLE produits ( numéro_produit
    SERIAL,
    ...
);
```

L'abréviation `SERIAL` est discutée plus en détail dans la section 8.1.4.

5.3. Colonnes d'identité

Une colonne d'identité est une colonne spéciale générée automatiquement à partir d'une séquence implicite. Elle peut servir à générer des valeurs clés.

Pour créer une colonne d'identité, utilisez la clause `GENERATED ... AS IDENTITY` dans `CREATE TABLE`, par exemple :

```
CRÉER LA TABLE personnes (
    id bigint GÉNÉRÉ TOUJOURS COMME IDENTITÉ,
    ...,
);
```

ou bien

```
CRÉER LA TABLE personnes (
    id bigint GÉNÉRÉ PAR DÉFAUT COMME IDENTITÉ,
    ...,
);
```

Consultez la fonction `CREATE TABLE` pour plus de détails.

Si une commande `INSERT` est exécutée sur la table contenant la colonne d'identité et qu'aucune valeur n'est explicitement spécifiée pour cette colonne, une valeur générée par la séquence implicite est insérée. Par exemple, avec les définitions ci-dessus et en supposant la présence de colonnes supplémentaires appropriées, écrire

```
INSÉRER DANS personnes (nom, adresse) VALEURS ('A', 'foo');
INSÉRER DANS personnes (nom, adresse) VALEURS ('B', 'bar');
```

générerait des valeurs pour la colonne `id` en commençant par 1 et donnerait les données de tableau suivantes :

identifiant	nom	adresse
1	A	foo
2	B	bar

Alternativement, le mot-clé `DEFAULT` peut être spécifié à la place d'une valeur pour demander explicitement la valeur générée par la séquence, comme

```
INSÉRER DANS personnes (id, nom, adresse) VALEURS (DEFAULT, 'C',  
'baz');
```

De même, le mot-clé `DEFAULT` peut être utilisé dans les commandes `UPDATE`.

Ainsi, à bien des égards, une colonne d'identité se comporte comme une colonne avec une valeur par défaut.

Les clauses `ALWAYS` et `BY DEFAULT` dans la définition de colonne déterminent la manière dont les valeurs explicitement spécifiées par l'utilisateur sont gérées dans les commandes `INSERT` et `UPDATE`. Dans une commande `INSERT`, si `ALWAYS` est sélectionné, une valeur spécifiée par l'utilisateur n'est acceptée que si l'instruction `INSERT` spécifie `OVERRIDING SYSTEM VALUE`. Si `BY DEFAULT` est sélectionné, la valeur spécifiée par l'utilisateur est prioritaire. Ainsi, l'utilisation de `BY DEFAULT` produit un comportement plus proche des valeurs par défaut, où la valeur par défaut peut être remplacée par une valeur explicite, tandis que `ALWAYS` offre une meilleure protection contre l'insertion accidentelle d'une valeur explicite.

Le type de données d'une colonne d'identité doit être l'un des types de données pris en charge par les séquences (voir `CREATE SEQUENCE`). Les propriétés de la séquence associée peuvent être spécifiées lors de la création d'une colonne d'identité (voir `CREATE TABLE`) ou modifiées ultérieurement (voir `ALTER TABLE`).

Une colonne d'identité est automatiquement marquée comme `NOT NULL`. Cependant, elle ne garantit pas l'unicité des valeurs. (Une séquence renvoie normalement des valeurs uniques, mais elle peut être réinitialisée ou des valeurs peuvent être insérées manuellement dans la colonne d'identité, comme expliqué précédemment.) L'unicité doit être assurée par une contrainte `PRIMARY KEY` ou `UNIQUE`.

Dans les hiérarchies d'héritage de tables, les colonnes d'identité et leurs propriétés d'une table enfant sont indépendantes de celles de ses tables parentes. Une table enfant n'hérite pas automatiquement des colonnes d'identité ni de leurs propriétés de la table parente. Lors d'une opération `INSERT` ou `UPDATE`, une colonne est traitée comme une colonne d'identité si elle l'est dans la table spécifiée dans l'instruction, et les propriétés d'identité correspondantes sont appliquées.

Les partitions héritent des colonnes d'identité de la table partitionnée. Elles ne peuvent pas avoir leurs propres colonnes d'identité. Les propriétés d'une colonne d'identité donnée sont cohérentes entre toutes les partitions de la hiérarchie de partitionnement.

5.4. Colonnes générées

Une colonne générée est une colonne spéciale toujours calculée à partir d'autres colonnes. Elle est donc à une colonne ce qu'une vue est à une table. Il existe deux types de colonnes générées : stockées et virtuelles. Une colonne générée stockée est calculée lors de son écriture (insertion ou mise à jour) et occupe de l'espace mémoire comme une colonne classique. Une colonne générée virtuelle n'occupe aucun espace mémoire et est calculée lors de sa lecture. Ainsi, une colonne générée virtuelle est similaire à une vue et une colonne générée stockée est similaire à une vue matérialisée (à la différence qu'elle est toujours mise à jour automatiquement). PostgreSQL n'implémente actuellement que les colonnes générées stockées.

Pour créer une colonne générée, utilisez la clause `GENERATED ALWAYS AS` dans `CREATE TABLE`, par exemple :

```
CRÉER LA TABLE personnes (  
    ...,  
    hauteur_cm numérique,  
    hauteur_in numérique GÉNÉRÉ TOUJOURS COMME (hauteur_cm / 2,54) STOCKÉ  
);
```

Le mot-clé `STORED` doit être spécifié pour choisir le type de stockage de la colonne générée. Consultez la documentation de `CREATE TABLE` pour plus de détails.

Il est impossible d'écrire directement dans une colonne générée. Dans les commandes `INSERT` ou `UPDATE`, aucune valeur ne peut être spécifiée pour une colonne générée, mais le mot-clé `DEFAULT` peut être utilisé.

Considérez les différences entre une colonne avec une valeur par défaut et une colonne générée. La valeur par défaut est évaluée une seule fois, lors de l'insertion initiale de la ligne, si aucune autre valeur n'a été fournie ; une colonne générée est mise à jour à chaque modification de la ligne et ne peut être redéfinie. Une valeur par défaut ne peut pas faire référence à d'autres colonnes de la table ; une expression de génération le ferait généralement. Une valeur par défaut peut utiliser des fonctions volatiles, comme `random()`, ou des fonctions faisant référence à l'heure actuelle ; cela n'est pas autorisé pour les colonnes générées.

Plusieurs restrictions s'appliquent à la définition des colonnes générées et des tables contenant des colonnes générées :

- L'expression de génération ne peut utiliser que des fonctions immuables et ne peut en aucun cas utiliser de sous-requêtes ni faire référence à autre chose que la ligne actuelle.
- Une expression de génération ne peut pas faire référence à une autre colonne générée.
- Une expression de génération ne peut pas faire référence à une colonne système, à l'exception de tableoid.
- Une colonne générée ne peut pas avoir de valeur par défaut ni de définition d'identité.
- Une colonne générée ne peut pas faire partie d'une clé de partition.
- Les tables étrangères peuvent contenir des colonnes générées. Voir `CREATE FOREIGN TABLE` pour plus de détails.
- Pour l'héritage et le partage :

Si une colonne parente est une colonne générée, sa colonne enfant doit également l'être ; toutefois, l'expression de génération de la colonne enfant peut être différente. L'expression de génération appliquée lors de l'insertion ou de la mise à jour d'une ligne est celle associée à la table dans laquelle la ligne est physiquement insérée. (Ce comportement diffère de celui des valeurs par défaut des colonnes : dans ce cas, c'est la valeur par défaut associée à la table spécifiée dans la requête qui est appliquée.)

- Si une colonne parente n'est pas une colonne générée, sa colonne enfant ne doit pas l'être non plus.

Pour les tables héritées, si vous définissez une colonne enfant sans clause `GENERATED` dans `... INHERITS`, sa clause `CRÉER UNE TABLE` `GENERATED` sera automatiquement copiée de la table parente. `ALTER TABLE INHERIT` exige que les colonnes parente et enfant aient déjà le même statut de génération, mais n'exige pas que leurs expressions de génération correspondent.

De même, pour les tables partitionnées, si vous définissez une colonne enfant sans clause `GENERATED` dans `CREATE TABLE ... PARTITION OF`, sa clause `GENERATED` sera automatiquement copiée de la table parente. `ALTER TABLE ... ATTACH PARTITION` exigera que les colonnes parente et enfant aient déjà le même statut de génération, mais n'exigera pas que leurs expressions de génération correspondent.

En cas d'héritage multiple, si une colonne parente est une colonne générée, toutes les colonnes parentes doivent l'être également. Si elles n'ont pas toutes la même expression de génération, l'expression souhaitée pour la colonne enfant doit être spécifiée explicitement.

Des considérations supplémentaires s'appliquent à l'utilisation des colonnes générées.

Les colonnes générées conservent des privilèges d'accès distincts de ceux de leurs colonnes de base sous-jacentes. Il est donc possible de configurer le système de sorte qu'un rôle particulier puisse lire les données d'une colonne générée sans pouvoir accéder aux données des colonnes de base sous-jacentes.

Les colonnes générées sont, en principe, mises à jour après l'exécution des déclencheurs `BEFORE`. Par conséquent, les modifications apportées aux colonnes de base dans un déclencheur `BEFORE` seront répercutées dans les colonnes générées. En revanche, l'accès aux colonnes générées dans les déclencheurs `BEFORE` est interdit.

- Les colonnes générées sont ignorées pour la réplication logique et ne peuvent pas être spécifiées dans un `CREATE PUBLISTE des colonnes LICATION`.

5.5. Contraintes

Les types de données permettent de limiter le type de données pouvant être stockées dans une table. Cependant, pour de nombreuses applications, la contrainte qu'ils imposent est trop générale. Par exemple, une colonne contenant le prix d'un produit ne devrait probablement accepter que des valeurs positives. Or, il n'existe pas de type de données standard qui n'accepte que des nombres positifs. Autre problème : il peut être nécessaire de contraindre les données d'une colonne par rapport à d'autres colonnes ou lignes. Par exemple, dans une table contenant des informations sur les produits, il ne devrait y avoir qu'une seule ligne par numéro de produit.

À cette fin, SQL vous permet de définir des contraintes sur les colonnes et les tables. Ces contraintes vous offrent un contrôle précis sur les données de vos tables. Si un utilisateur tente d'enregistrer des données dans une colonne qui enfreint une contrainte, une erreur est générée, même si la valeur provient de la définition de valeur par défaut.

5.5.1. Vérifier les contraintes

Une contrainte de vérification est le type de contrainte le plus générique. Elle permet de spécifier que la valeur d'une certaine colonne doit satisfaire une expression booléenne (valeur de vérité). Par exemple, pour exiger des prix de produits positifs, vous pouvez utiliser :

```
CRÉER TABLE produits ( product_no entier,  
                        name texte, price numérique VÉRIFIER  
                        (price > 0)  
  
);
```

Comme vous le voyez, la définition de la contrainte vient après le type de données, tout comme les définitions des valeurs par défaut. Les valeurs par défaut et les contraintes peuvent être listées dans n'importe quel ordre. Une contrainte de vérification se compose du mot-clé CHECK suivi d'une expression entre parenthèses. Cette expression doit concerner la colonne concernée, sinon la contrainte serait incohérente.

Vous pouvez également attribuer un nom distinct à la contrainte. Cela permet de clarifier les messages d'erreur et de faire référence à la contrainte lorsque vous devez la modifier. La syntaxe est la suivante :

```
CRÉER TABLE produits ( product_no entier,  
                        name texte, price numérique  
                        CONTRAINTE  
                        prix_positif VÉRIFIER (prix > 0)  
  
);
```

Pour spécifier une contrainte nommée, utilisez le mot-clé CONSTRAINT suivi d'un identifiant, puis de la définition de la contrainte. (Si vous ne spécifiez pas de nom de contrainte de cette manière, le système en choisira un automatiquement.)

Une contrainte de vérification peut également porter sur plusieurs colonnes. Supposons que vous stockiez un prix normal et un prix réduit, et que vous souhaitiez vous assurer que le prix réduit est inférieur au prix normal :

```
CRÉER TABLE produits ( numéro_produit  
                        entier, nom texte, prix numérique  
                        CHECK (prix > 0),  
                        prix_réduit numérique CHECK (prix_réduit > 0),  
  
                        VÉRIFIER (prix > prix_réduit)  
  
);
```

Les deux premières contraintes devraient vous sembler familières. La troisième utilise une nouvelle syntaxe. Elle n'est pas associée à une colonne particulière, mais apparaît comme un élément distinct dans la liste des colonnes séparées par des virgules. Les définitions de colonnes et celles de ces contraintes peuvent être listées dans un ordre aléatoire.

On dit que les deux premières contraintes sont des contraintes de colonne, tandis que la troisième est une contrainte de table, car elle est définie indépendamment de toute colonne. Les contraintes de colonne peuvent également être écrites comme des contraintes de table, mais l'inverse n'est pas nécessairement possible, puisqu'une contrainte de colonne est censée ne concerner que la colonne à laquelle elle est associée. (PostgreSQL n'impose pas cette règle, mais il est recommandé de la respecter pour que vos définitions de table soient compatibles avec d'autres systèmes de bases de données.) L'exemple ci-dessus pourrait également s'écrire :

```
CRÉER TABLE produits ( numéro_produit
    entier, nom texte, prix numérique,

    VÉRIFIER (prix > 0), prix_remise
    numérique,
    VÉRIFIER (prix_remise > 0),
    VÉRIFIER (prix > prix_réduit)
);
```

ou même :

```
CRÉER TABLE produits ( numéro_produit
    entier, nom texte, prix numérique
    VÉRIFIER (prix >
    0), prix_remise numérique,

    VÉRIFIER (prix_remise > 0 ET prix > prix_remise)
);
```

C'est une question de goût.

Il est possible d'attribuer des noms aux contraintes de table de la même manière qu'aux contraintes de colonne :

```
CRÉER TABLE produits ( numéro_produit
    entier, nom texte, prix numérique,

    VÉRIFIER (prix > 0), prix_remise
    numérique,
    VÉRIFIER (prix_remise > 0),
    CONTRAINTE valid_discount VÉRIFICATION (prix > prix_remise)
);
```

Il convient de noter qu'une contrainte de vérification est satisfaite si l'expression de vérification est vraie ou nulle. La plupart des expressions étant nulles si l'un de leurs opérandes est nul, elles ne permettent pas d'empêcher la présence de valeurs nulles dans les colonnes concernées. Pour garantir qu'une colonne ne contienne pas de valeurs nulles, on peut utiliser la contrainte NOT NULL décrite dans la section suivante.

Note

PostgreSQL ne prend pas en charge les contraintes CHECK qui font référence à des données de table autres que la ligne nouvelle ou mise à jour vérifiée. Bien qu'une contrainte CHECK qui enfreint cette règle puisse sembler fonctionner lors de tests simples, elle ne peut garantir que la base de données n'atteindra pas un état dans lequel

La condition de contrainte est fautive (en raison de modifications ultérieures des autres lignes concernées). Cela entraînerait l'échec d'une sauvegarde et d'une restauration de la base de données. La restauration pourrait échouer même si l'état complet de la base de données est conforme à la contrainte, car les lignes ne sont pas chargées dans un ordre respectant la contrainte. Dans la mesure du possible, utilisez des contraintes UNIQUE, EXCLUDE ou FOREIGN KEY pour exprimer les restrictions entre lignes et entre tables.

Si vous souhaitez effectuer une vérification ponctuelle par rapport aux autres lignes lors de l'insertion d'une ligne, plutôt qu'une garantie de cohérence maintenue en permanence, un déclencheur personnalisé peut être utilisé à cette fin. (Cette approche évite le problème de sauvegarde/restoration car pg_dump ne réinstalle les déclencheurs qu'après la restauration des données, de sorte que la vérification ne sera pas effectuée lors d'une sauvegarde/restoration.)

Note

PostgreSQL considère que les conditions des contraintes CHECK sont immuables, c'est-à-dire qu'elles renverront toujours le même résultat pour une même ligne d'entrée. C'est cette hypothèse qui justifie l'examen des contraintes CHECK uniquement lors de l'insertion ou de la mise à jour de lignes, et non à d'autres moments. (L'avertissement ci-dessus concernant l'interdiction de référencer des données d'autres tables est en réalité un cas particulier de cette restriction.)

Une méthode courante pour contourner cette contrainte consiste à référencer une fonction définie par l'utilisateur dans une expression CHECK, puis à modifier le comportement de cette fonction. PostgreSQL ne l'interdit pas, mais il ne détectera pas les lignes de la table qui ne respectent plus la contrainte CHECK. Cela entraînerait l'échec d'une sauvegarde et d'une restauration ultérieures de la base de données. La méthode recommandée pour gérer une telle modification est de supprimer la contrainte (à l'aide de ALTER TABLE), d'ajuster la définition de la fonction, puis de rajouter la contrainte, ce qui permettra de la vérifier à nouveau pour chaque ligne de la table.

5.5.2. Contraintes de non-nullité

Une contrainte non nulle indique simplement qu'une colonne ne doit pas prendre la valeur nulle. Exemple de syntaxe :

```
CRÉER TABLE produits ( product_no entier
                        NON NULL, name texte NON NULL, price numérique

);
```

Une contrainte NOT NULL est toujours écrite comme une contrainte de colonne. Elle est fonctionnellement équivalente à une contrainte CHECK (nom_colonne IS NOT NULL), mais sous PostgreSQL, la création explicite d'une contrainte NOT NULL est plus efficace. L'inconvénient est qu'il est impossible de nommer explicitement les contraintes NOT NULL ainsi créées.

Bien sûr, une colonne peut avoir plusieurs contraintes. Il suffit de les écrire les unes après les autres :

```
CRÉER TABLE produits ( product_no entier
                        NON NULL, name texte NON NULL, price numérique
                        NON NULL CHECK (price > 0)

);
```

L'ordre n'a pas d'importance. Il ne détermine pas nécessairement l'ordre dans lequel les contraintes sont vérifiées.

La contrainte NOT NULL a son inverse : la contrainte NULL. Cela ne signifie pas que la colonne doit nécessairement être nulle, ce qui serait évidemment inutile. Il s'agit simplement d'un comportement par défaut, la colonne pouvant contenir des valeurs nulles. La contrainte NULL n'est pas présente dans la norme SQL et ne doit pas être utilisée dans les applications portables. (Elle a été ajoutée à PostgreSQL uniquement pour assurer la compatibilité avec d'autres bases de données.)

(systèmes.) Certains utilisateurs l'apprécient cependant car il permet de basculer facilement la contrainte dans un fichier de script. Par exemple, vous pourriez commencer par :

```
CRÉER DES PRODUITS DE TABLE (  
    product_no entier NULL, name texte NULL,  
    price numérique NULL  
  
);
```

Insérez ensuite le mot-clé NOT à l'endroit souhaité.

Conseil

Dans la plupart des conceptions de bases de données, la majorité des colonnes doivent être marquées comme non nulles.

5.5.3. Contraintes d'unicité

Les contraintes d'unicité garantissent que les données contenues dans une colonne, ou un groupe de colonnes, sont uniques parmi toutes les lignes du tableau. La syntaxe est la suivante :

```
CRÉER TABLE produits ( product_no entier  
    UNIQUE, nom texte, prix numérique  
  
);
```

lorsqu'elle est écrite sous forme de contrainte de colonne, et :

```
CRÉER TABLE produits ( numéro_produit  
    entier, nom texte, prix numérique,  
  
    UNIQUE (numéro_de_produit)  
);
```

lorsqu'elle est écrite sous forme de contrainte de table.

Pour définir une contrainte d'unicité pour un groupe de colonnes, écrivez-la sous forme de contrainte de table, les noms des colonnes étant séparés par des virgules :

```
CRÉER UNE TABLE exemple ( a entier,  
    b entier, c entier,  
  
    UNIQUE (a, c)  
);
```

Cela précise que la combinaison des valeurs dans les colonnes indiquées est unique dans l'ensemble du tableau, bien que chacune des colonnes ne soit pas nécessairement (et ne l'est généralement pas) unique.

Vous pouvez attribuer votre propre nom à une contrainte unique, de la manière habituelle :

```
CRÉER DES PRODUITS DE TABLE (  

```

```
product_no entier CONSTRAINT doit_être_différent UNIQUE, nom texte, prix numérique
```

```
);
```

L'ajout d'une contrainte d'unicité crée automatiquement un index B-tree unique sur la colonne ou le groupe de colonnes spécifié dans la contrainte. Une restriction d'unicité ne portant que sur certaines lignes ne peut être formulée comme une contrainte d'unicité, mais il est possible de l'appliquer en créant un index partiel unique.

En général, une contrainte d'unicité est violée s'il existe plusieurs lignes dans la table où les valeurs de toutes les colonnes incluses dans la contrainte sont égales. Par défaut, deux valeurs nulles ne sont pas considérées comme égales lors de cette comparaison. Cela signifie que même en présence d'une contrainte d'unicité, il est possible de stocker des lignes dupliquées contenant une valeur nulle dans au moins une des colonnes contraintes. Ce comportement peut être modifié en ajoutant la clause `NULLS NOT DISTINCT`.

```
CRÉER DES PRODUITS DE TABLE (
    product_no entier UNIQUE NULLS NOT DISTINCT, nom texte, prix numérique
```

```
);
```

ou

```
CRÉER TABLE produits ( numéro_produit
    entier, nom texte, prix numérique,
```

```
    UNIQUE NULLS NOT DISTINCT (product_no)
);
```

Le comportement par défaut peut être spécifié explicitement à l'aide de `NULLS DISTINCT`. Le traitement par défaut des valeurs nulles dans les contraintes d'unicité est défini par l'implémentation selon la norme SQL ; d'autres implémentations peuvent avoir un comportement différent. Soyez donc vigilant lors du développement d'applications destinées à être portables.

5.5.4. Clés primaires

Une contrainte de clé primaire indique qu'une colonne, ou un groupe de colonnes, peut servir d'identifiant unique pour les lignes de la table. Cela implique que les valeurs soient à la fois uniques et non nulles. Ainsi, les deux définitions de table suivantes acceptent les mêmes données :

```
CRÉER DES PRODUITS DE TABLE (
    product_no entier UNIQUE NOT NULL, nom texte, prix
    numérique
```

```
);
```

```
CRÉER TABLE produits ( product_no entier
    CLÉ PRIMAIRE, nom texte, prix numérique
```

```
);
```

Les clés primaires peuvent s'étendre sur plusieurs colonnes ; la syntaxe est similaire à celle des contraintes d'unicité :

```
CRÉER UNE TABLE exemple ( a entier,  
                           b entier, c entier,
```

```
                           CLÉ PRIMAIRE (a, c)  
);
```

L'ajout d'une clé primaire créera automatiquement un index B-tree unique sur la colonne ou le groupe de colonnes listé dans la clé primaire, et forcera la ou les colonnes à être marquées comme NOT NULL.

Une table ne peut avoir qu'une seule clé primaire. (Il peut exister un nombre quelconque de contraintes d'unicité et de non-nullité, qui sont fonctionnellement presque identiques, mais une seule peut être identifiée comme clé primaire.) La théorie des bases de données relationnelles stipule que chaque table doit posséder une clé primaire. PostgreSQL n'impose pas cette règle, mais il est généralement préférable de la respecter.

Les clés primaires sont utiles à la fois pour la documentation et pour les applications clientes. Par exemple, une application graphique permettant de modifier les valeurs des lignes a probablement besoin de connaître la clé primaire d'une table pour identifier chaque ligne de manière unique. De plus, le système de base de données utilise une clé primaire de différentes manières, si elle a été déclarée ; par exemple, la clé primaire définit la ou les colonnes cibles par défaut pour les clés étrangères référençant la table.

5.5.5. Clés étrangères

Une contrainte de clé étrangère stipule que les valeurs d'une colonne (ou d'un groupe de colonnes) doivent correspondre aux valeurs figurant dans une ligne d'une autre table. On dit que cela garantit l'intégrité référentielle entre deux tables liées.

Supposons que vous ayez le tableau des produits que nous avons déjà utilisé à plusieurs reprises :

```
CRÉER TABLE produits ( product_no entier  
                        CLÉ PRIMAIRE, nom texte, prix numérique  
  
);
```

Supposons également que vous ayez une table stockant les commandes de ces produits. Nous voulons nous assurer que la table des commandes ne contienne que les commandes de produits existants. Nous définissons donc une contrainte de clé étrangère dans la table des commandes qui fait référence à la table des produits :

```
CRÉER LA TABLE commandes (  
    order_id entier CLÉ PRIMAIRE, product_no entier  
    RÉFÉRENCES produits (product_no), quantity entier  
  
);
```

Il est désormais impossible de créer des commandes avec des entrées product_no non NULL qui n'apparaissent pas dans la table des produits.

Dans ce cas, on dit que la table des commandes est la table de référence et la table des produits la table référencée . De même, il existe des colonnes de référence et des colonnes référencées.

Vous pouvez également abréger la commande ci-dessus comme suit :

```
CRÉER LA TABLE commandes (  
    order_id entier CLÉ PRIMAIRE,
```

```
product_no entier RÉFÉRENCES produits, quantité entier
```

```
);
```

car, en l'absence de liste de colonnes, la clé primaire de la table référencée est utilisée comme colonne(s) référencée(s).

Vous pouvez attribuer un nom personnalisé à une contrainte de clé étrangère, de la manière habituelle.

Une clé étrangère peut également contraindre et référencer un groupe de colonnes. Comme d'habitude, elle doit alors être écrite sous forme de contrainte de table. Voici un exemple de syntaxe simplifiée :

```
CRÉER TABLE t1 ( a entier CLÉ
```

```
    PRIMAIRE, b entier, c entier,
```

```
    CLÉ ÉTRANGÈRE (b, c) RÉFÈRE autre_table (c1, c2) );
```

Bien entendu, le nombre et le type des colonnes contraintes doivent correspondre au nombre et au type des colonnes référencées.

Il est parfois utile que l'« autre table » d'une contrainte de clé étrangère soit la même table ; on parle alors de clé étrangère autoréférentielle . Par exemple, si vous souhaitez que les lignes d'une table représentent les nœuds d'une structure arborescente, vous pouvez écrire :

```
CRÉER TABLE arbre (
```

```
    node_id entier CLÉ PRIMAIRE, parent_id entier
```

```
    RÉFÉRENCES arbre, nom texte,
```

```
    ...
```

```
);
```

Un nœud de niveau supérieur aurait un parent_id NULL, tandis que les entrées parent_id non NULL seraient contraintes de référencer des lignes valides de la table.

Une table peut comporter plusieurs contraintes de clé étrangère. Ceci permet de gérer les relations plusieurs-à-plusieurs entre les tables. Supposons que vous ayez des tables concernant les produits et les commandes, et que vous souhaitiez désormais autoriser une commande à contenir plusieurs produits (ce que la structure précédente ne permettait pas). Vous pourriez alors utiliser la structure de table suivante :

```
CRÉER TABLE produits ( product_no entier
```

```
    CLÉ PRIMAIRE, nom texte, prix numérique
```

```
);
```

```
CRÉER LA TABLE commandes (
```

```
    order_id entier CLÉ PRIMAIRE, shipping_address texte,
```

```
    ...
```

```
);
```

```
CRÉER TABLE order_items ( product_no entier
```

```
    RÉFÉRENCES produits, order_id entier RÉFÉRENCES commandes,
```

```

quantité entière,
CLÉ PRIMAIRE (numéro_produit, identifiant_commande)
);

```

Notez que la clé primaire chevauche les clés étrangères de la dernière table.

Nous savons que les clés étrangères empêchent la création de commandes sans produit associé. Mais que se passe-t-il si un produit est supprimé après la création d'une commande qui le référence ? SQL permet également de gérer ce cas. De manière intuitive, plusieurs options s'offrent à nous :

- Empêcher la suppression d'un produit référencé •
- Supprimer également les commandes
- Autre chose ?

Pour illustrer cela, appliquons la politique suivante à l'exemple de relation plusieurs-à-plusieurs ci-dessus : lorsqu'une personne souhaite supprimer un produit encore référencé par une commande (via `order\_items`), cette opération est interdite. Si une personne supprime une commande, les articles de cette commande sont également supprimés.

```

CRÉER TABLE produits ( product_no entier
CLÉ PRIMAIRE, nom texte, prix numérique

```

```
);
```

```

CRÉER LA TABLE commandes (
order_id entier CLÉ PRIMAIRE, shipping_address
texte,
...
);

```

```

CRÉER LA TABLE order_items ( product_no
entier RÉFÉRENCES produits ON DELETE RESTRICT, order_id entier RÉFÉRENCES orders ON
DELETE CASCADE, quantity entier,

```

```

CLÉ PRIMAIRE (numéro_produit, identifiant_commande)
);

```

Les suppressions restreintes et en cascade sont les deux options les plus courantes. RESTRICT empêche la suppression d'une ligne référencée. NO ACTION signifie que si des lignes référençant une ligne existent encore lors de la vérification de la contrainte, une erreur est générée ; il s'agit du comportement par défaut si vous ne spécifiez rien. (La principale différence entre ces deux options est que NO ACTION permet de différer la vérification à une étape ultérieure de la transaction, contrairement à RESTRICT.) CASCADE spécifie que lorsqu'une ligne référencée est supprimée, les lignes qui la référencent doivent également être automatiquement supprimées. Deux autres options sont disponibles : SET NULL et SET DEFAULT. Celles-ci attribuent respectivement la valeur NULL ou la valeur par défaut aux colonnes référençant la ligne référençante lors de sa suppression. Notez que ces options ne vous dispensent pas du respect des contraintes. Par exemple, si une action spécifie SET DEFAULT mais que la valeur par défaut ne satisfait pas la contrainte de clé étrangère, l'opération échouera.

Le choix approprié de l'action ON DELETE dépend du type d'objets représentés par les tables liées. Si la table référençant un élément qui fait partie intégrante de celui représenté par la table référencée et qui ne peut exister indépendamment, l'action CASCADE peut convenir. Si les deux tables représentent des objets indépendants, les actions RESTRICT ou NO ACTION sont plus appropriées ; une application souhaitant supprimer les deux objets devra alors le préciser et exécuter deux commandes DELETE. Dans l'exemple ci-dessus, les articles d'une commande en font partie, et il est pratique qu'ils soient supprimés automatiquement lors de la suppression de la commande. Cependant, les produits et les commandes sont différents ; par conséquent, faire en sorte que la suppression d'un produit entraîne automatiquement la suppression de certains articles de la commande pourrait s'avérer problématique. Les actions SET NULL ou SET DEFAULT peuvent convenir en présence d'une clé étrangère.

La relation représente une information facultative. Par exemple, si la table des produits contient une référence à un chef de produit et que cette entrée est supprimée, il peut être utile de définir le chef de produit du produit sur null ou une valeur par défaut.

Les actions SET NULL et SET DEFAULT peuvent prendre une liste de colonnes pour spécifier les colonnes à définir. Normalement, toutes les colonnes de la contrainte de clé étrangère sont définies ; la définition d'un sous-ensemble seulement est utile dans certains cas particuliers. Prenons l'exemple suivant :

```
CRÉER LA TABLE locataires ( locataire_id
    entier CLÉ PRIMAIRE
);

CRÉER LA TABLE utilisateurs (
    tenant_id entier RÉFÉRENCES locataires ON DELETE CASCADE, user_id entier NON NULL,

    CLÉ PRIMAIRE (tenant_id, user_id)
);

CRÉER TABLE posts ( tenant_id
    entier RÉFÉRENCES tenants ON DELETE CASCADE, post_id entier NON NULL, author_id entier,

    CLÉ PRIMAIRE (tenant_id, post_id),
    CLÉ ÉTRANGÈRE (tenant_id, author_id) RÉFÈRE users ON DELETE
    DÉFINIR NULL (author_id) );
```

Sans la spécification de la colonne, la clé étrangère définirait également la colonne tenant_id sur null, mais cette colonne est toujours requise dans le cadre de la clé primaire.

De même que pour ON DELETE, ON UPDATE est déclenché lorsqu'une colonne référencée est modifiée (mise à jour). Les actions possibles sont identiques, à ceci près que les listes de colonnes ne peuvent pas être spécifiées pour SET NULL et SET DEFAULT. Dans ce cas, CASCADE signifie que les valeurs mises à jour de la ou des colonnes référencées doivent être copiées dans la ou les lignes correspondantes.

Normalement, une ligne référençant une autre ligne n'est pas tenue de respecter la contrainte de clé étrangère si l'une de ses colonnes référençantes est nulle. Si l'option MATCH FULL est ajoutée à la déclaration de clé étrangère, une ligne référençant une autre ligne n'échappe à la contrainte que si toutes ses colonnes référençantes sont nulles (un mélange de valeurs nulles et non nulles entraînera donc systématiquement le non-respect de la contrainte MATCH FULL). Si vous ne souhaitez pas que les lignes référençant une autre ligne puissent échapper à la contrainte de clé étrangère, déclarez la ou les colonnes référençantes comme NOT NULL.

Une clé étrangère doit référencer des colonnes qui sont soit une clé primaire, soit une contrainte d'unicité, soit des colonnes d'un index non partiel unique. Cela signifie que les colonnes référencées sont toujours indexées afin de permettre des recherches efficaces pour déterminer si une ligne référençant une correspondance existe. Étant donné qu'une suppression d'une ligne de la table référencée ou une mise à jour d'une colonne référencée nécessite une analyse de cette table pour trouver les lignes correspondant à l'ancienne valeur, il est souvent judicieux d'indexer également les colonnes référençantes. Comme cela n'est pas toujours nécessaire et qu'il existe de nombreuses méthodes d'indexation, la déclaration d'une contrainte de clé étrangère ne crée pas automatiquement d'index sur les colonnes référençantes.

Vous trouverez plus d'informations sur la mise à jour et la suppression des données au chapitre 6. Consultez également la description de la syntaxe des contraintes de clé étrangère dans la documentation de référence de CREATE TABLE.

5.5.6. Contraintes d'exclusion

Les contraintes d'exclusion garantissent que si deux lignes quelconques sont comparées sur les colonnes ou expressions spécifiées à l'aide des opérateurs spécifiés, au moins une de ces comparaisons renverra faux ou nul. La syntaxe est la suivante :

```
CRÉER UN TABLEAU cercles (
    cercle c,
    EXCLURE L'UTILISATION DE gist (c AVEC &&)
);
```

Voir aussi CRÉER UNE TABLE ... CONTRAINTE ... EXCLURE pour plus de détails.

L'ajout d'une contrainte d'exclusion créera automatiquement un index du type spécifié dans la déclaration de contrainte.

5.6. Colonnes du système

Chaque table comporte plusieurs colonnes système définies implicitement par le système. Par conséquent, ces noms ne peuvent pas être utilisés pour nommer des colonnes définies par l'utilisateur. (Notez que ces restrictions sont indépendantes du fait que le nom soit un mot-clé ou non ; l'utilisation de guillemets ne permet pas de contourner ces restrictions.)

Vous n'avez pas vraiment à vous préoccuper de ces colonnes ; sachez simplement qu'elles existent.

tableoid

L'OID de la table contenant cette ligne. Cette colonne est particulièrement utile pour les requêtes portant sur des tables partitionnées (voir section 5.12) ou des hiérarchies d'héritage (voir section 5.11), car sans elle, il est difficile de déterminer la table d'origine d'une ligne. L'OID de la table peut être combiné avec la colonne oid de pg_class pour obtenir le nom de la table.

xmin

L'identifiant (ID de transaction) de la transaction d'insertion pour cette version de ligne. (Une version de ligne correspond à un état individuel d'une ligne ; chaque mise à jour d'une ligne crée une nouvelle version de ligne pour la même ligne logique.)

cmin

L'identificateur de commande (commençant à zéro) au sein de la transaction d'insertion.

xmax

L'identifiant (ID de transaction) de la transaction de suppression, ou zéro pour une version de ligne non supprimée. Cette colonne peut contenir une valeur différente de zéro dans une version de ligne visible. Cela indique généralement que la transaction de suppression n'a pas encore été validée ou qu'une tentative de suppression a été annulée.

cmax

L'identifiant de la commande au sein de la transaction de suppression, ou zéro.

ctid

L'emplacement physique de la version de la ligne dans sa table. Notez que, bien que le ctid permette de localiser rapidement la version de la ligne, le ctid d'une ligne change si celle-ci est mise à jour ou déplacée par VACUUM FULL. Par conséquent, le ctid ne doit pas être utilisé comme identifiant de ligne. Une clé primaire doit être utilisée pour identifier les lignes logiques.

Les identifiants de transaction sont également codés sur 32 bits. Dans une base de données à longue durée de vie, il est possible que les identifiants de transaction soient réinitialisés. Ce problème n'est pas critique si les procédures de maintenance appropriées sont mises en œuvre ; voir le chapitre 24 pour plus de détails. Il est toutefois imprudent de se fier à l'unicité des identifiants de transaction sur le long terme (plus d'un milliard de transactions).

Les identificateurs de commande sont également codés sur 32 bits. Cela impose une limite stricte de 2^{32} (4 milliards) de commandes SQL par transaction. En pratique, cette limite ne pose pas de problème : elle concerne le nombre de commandes SQL, et non le nombre de lignes traitées. De plus, seules les commandes modifiant effectivement le contenu de la base de données consomment un identificateur.

5.7. Modification des tableaux

Lorsque vous créez une table et que vous vous rendez compte d'une erreur, ou que les besoins de l'application évoluent, vous pouvez la supprimer et la recréer. Cependant, cette solution n'est pas pratique si la table contient déjà des données ou si elle est référencée par d'autres objets de la base de données (par exemple, une contrainte de clé étrangère). C'est pourquoi PostgreSQL propose un ensemble de commandes permettant de modifier les tables existantes. Il est important de noter que cette modification est conceptuellement différente de la modification des données contenues dans la table : il s'agit ici de modifier la définition, ou la structure, de la table.

Tu peux:

- Ajouter des colonnes
- Supprimer des colonnes
- Ajouter des contraintes
- Supprimer les contraintes
- Modifier les valeurs par défaut •
- Modifier les types de données des colonnes • Renommer les colonnes
- Renommer les tables

Toutes ces actions sont effectuées à l'aide de la commande ALTER TABLE, dont la page de référence contient des détails supplémentaires par rapport à ceux présentés ici.

5.7.1. Ajout d'une colonne

Pour ajouter une colonne, utilisez une commande comme :

```
ALTER TABLE produits ADD COLUMN description text;
```

La nouvelle colonne est initialement remplie avec la valeur par défaut indiquée (null si aucune clause DEFAULT n'est spécifiée).

Conseil

À partir de PostgreSQL 11, l'ajout d'une colonne avec une valeur par défaut constante ne signifie plus que chaque ligne de la table doit être mise à jour lors de l'exécution de l'instruction ALTER TABLE.

Au lieu de cela, la valeur par défaut sera renvoyée lors du prochain accès à la ligne et appliquée lors de la réécriture de la table, ce qui rend la commande ALTER TABLE très rapide, même sur les grandes tables.

Toutefois, si la valeur par défaut est volatile (par exemple, `clock_timestamp()`), chaque ligne devra être mise à jour avec la valeur calculée lors de l'exécution de ALTER TABLE. Pour éviter une opération de mise à jour potentiellement longue, notamment si vous prévoyez de toute façon de remplir la colonne principalement avec des valeurs non par défaut, il peut être préférable d'ajouter la colonne sans valeur par défaut, d'insérer les valeurs correctes à l'aide de UPDATE, puis d'ajouter la valeur par défaut souhaitée comme décrit ci-dessous.

Vous pouvez également définir simultanément des contraintes sur la colonne, en utilisant la syntaxe habituelle :

```
MODIFIER LE TABLEAU produits AJOUTER UNE COLONNE texte de description VÉRIFIER (description  
<> ");
```

En fait, toutes les options qui peuvent être appliquées à la description d'une colonne dans CREATE TABLE peuvent être utilisées ici. Gardez toutefois à l'esprit que la valeur par défaut doit respecter les contraintes données, sinon l'ajout échouera.

Vous pouvez également ajouter des contraintes ultérieurement (voir ci-dessous) après avoir correctement rempli la nouvelle colonne.

5.7.2. Suppression d'une colonne

Pour supprimer une colonne, utilisez une commande comme :

```
MODIFIER TABLE produits DROP COLUMN description ;
```

Les données contenues dans la colonne disparaissent. Les contraintes de table liées à cette colonne sont également supprimées. Toutefois, si la colonne est référencée par une contrainte de clé étrangère d'une autre table, PostgreSQL ne supprimera pas silencieusement cette contrainte. Vous pouvez autoriser la suppression de toutes les dépendances de la colonne en ajoutant CASCADE :

```
ALTER TABLE produits DROP COLUMN description CASCADE ;
```

Voir la section 5.15 pour une description du mécanisme général sous-jacent.

5.7.3. Ajout d'une contrainte

Pour ajouter une contrainte, on utilise la syntaxe des contraintes de table. Par exemple :

```
ALTER TABLE produits ADD CHECK (nom <> "");  
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);  
ALTER TABLE produits ADD FOREIGN KEY (product_group_id) REFERENCES product_groups;
```

Pour ajouter une contrainte non nulle, qui ne peut pas être écrite comme une contrainte de table, utilisez la syntaxe suivante :

```
ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

La contrainte sera vérifiée immédiatement ; les données du tableau doivent donc satisfaire à la contrainte avant de pouvoir être ajoutées.

5.7.4. Suppression d'une contrainte

Pour supprimer une contrainte, vous devez connaître son nom. Si vous lui avez attribué un nom, c'est simple. Sinon, le système lui a attribué un nom généré, que vous devez retrouver. La commande `psql \d nom-de-la-table` peut être utile ; d'autres interfaces peuvent également permettre d'examiner les détails de la table. La commande est alors :

```
ALTER TABLE products DROP CONSTRAINT some_name;
```

Comme pour la suppression d'une colonne, vous devez ajouter CASCADE si vous souhaitez supprimer une contrainte dont dépend une autre. Par exemple, une contrainte de clé étrangère peut dépendre d'une contrainte d'unicité ou de clé primaire sur la ou les colonnes référencées.

Cela fonctionne de la même manière pour tous les types de contraintes, à l'exception des contraintes non nulles. Pour supprimer une contrainte non nulle : utiliser:

```
ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

(Rappelons que les contraintes non nulles n'ont pas de nom.)

5.7.5. Modification de la valeur par défaut d'une colonne

Pour définir une nouvelle valeur par défaut pour une colonne, utilisez une commande comme :

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;
```

Notez que cela n'affecte pas les lignes existantes dans la table, cela modifie simplement la valeur par défaut pour les futures commandes INSERT.

Pour supprimer toute valeur par défaut, utilisez :

```
ALTER TABLE produits ALTER COLUMN prix DROP DEFAULT ;
```

Cela revient en fait à définir la valeur par défaut à null. Par conséquent, il n'est pas erroné de ne pas définir de valeur par défaut lorsqu'elle n'a pas été définie, car la valeur par défaut est implicitement null.

5.7.6. Modification du type de données d'une colonne

Pour convertir une colonne en un autre type de données, utilisez une commande comme :

```
ALTER TABLE produits ALTER COLUMN prix TYPE numérique(10,2);
```

Cette opération ne réussira que si chaque entrée existante dans la colonne peut être convertie au nouveau type par une conversion implicite. Si une conversion plus complexe est nécessaire, vous pouvez ajouter une clause USING qui spécifie comment calculer les nouvelles valeurs à partir des anciennes.

PostgreSQL tentera de convertir la valeur par défaut de la colonne (le cas échéant) vers le nouveau type, ainsi que toutes les contraintes qui la concernent. Cependant, ces conversions peuvent échouer ou produire des résultats inattendus. Il est souvent préférable de supprimer toutes les contraintes sur la colonne avant de modifier son type, puis de les rétablir ultérieurement en les modifiant.

5.7.7. Renommer une colonne

Pour renommer une colonne :

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

5.7.8. Renommer une table

Pour renommer une table :

```
ALTER TABLE produits RENAME TO articles;
```

5.8. Privilèges

Lorsqu'un objet est créé, un propriétaire lui est attribué. Ce propriétaire est généralement le rôle qui a exécuté l'instruction de création. Pour la plupart des objets, initialement, seul le propriétaire (ou un superutilisateur) peut interagir avec eux. Pour permettre à d'autres rôles de les utiliser, des privilèges doivent leur être accordés.

Il existe différents types de privilèges : SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER, CREATE, CONNECT, TEMPORARY, EXECUTE, USAGE, SET, ALTER SYSTEM et MAINTAIN. Les privilèges applicables à un objet donné varient selon son type (table, fonction, etc.). Vous trouverez plus de détails sur la signification de ces privilèges ci-dessous. Les sections et chapitres suivants vous montreront également comment les utiliser.

Le droit de modifier ou de détruire un objet est inhérent à la qualité de propriétaire et ne peut être octroyé ou révoqué de façon autonome. (Toutefois, comme tous les privilèges, ce droit peut être transmis par héritage aux membres de la famille du propriétaire ; voir la section 21.3.)

Un objet peut être attribué à un nouveau propriétaire à l'aide d'une commande ALTER du type approprié à l'objet, par exemple :

```
ALTER TABLE nom_table PROPRIÉTAIRE EN nouveau_propriétaire;
```

Les superutilisateurs peuvent toujours le faire ; les rôles ordinaires ne peuvent le faire que s'ils sont à la fois le propriétaire actuel de l'objet (ou s'ils héritent des privilèges du rôle propriétaire) et capables de définir le rôle propriétaire.

Tous les droits de propriété de l'ancien propriétaire sont transférés au nouveau propriétaire en même temps que le droit de propriété.

Pour attribuer des privilèges, on utilise la commande GRANT. Par exemple, si « joe » est un rôle existant et « accounts » une table existante, le privilège de mise à jour de la table peut être accordé avec :

```
ACCORDER LA MISE À JOUR DES COMPTES À joe ;
```

L'indication ALL au lieu d'un privilège spécifique octroie tous les privilèges pertinents pour le type d'objet.

Le rôle spécial PUBLIC permet d'octroyer un privilège à tous les utilisateurs du système. De plus, des rôles de groupe peuvent être créés pour faciliter la gestion des privilèges lorsqu'une base de données compte de nombreux utilisateurs ; pour plus de détails, consultez le chapitre 21.

Pour révoquer un privilège précédemment accordé, utilisez la commande REVOKE, dont le nom est tout à fait approprié :

```
RÉVOQUER TOUS LES COMPTES PUBLICS ;
```

En règle générale, seul le propriétaire de l'objet (ou un superutilisateur) peut accorder ou révoquer des privilèges sur cet objet. Toutefois, il est possible d'accorder un privilège « avec option d'octroi », ce qui permet au bénéficiaire de l'octroyer à son tour à d'autres personnes. Si l'option d'octroi est révoquée ultérieurement, tous ceux qui ont reçu le privilège de ce bénéficiaire (directement ou par chaîne d'octrois) perdront ce privilège. Pour plus de détails, consultez les pages de référence relatives à l'octroi et à la révocation.

Le propriétaire d'un objet peut choisir de révoquer ses propres privilèges ordinaires, par exemple pour rendre une table accessible en lecture seule à lui-même comme aux autres. Cependant, les propriétaires sont toujours considérés comme détenant toutes les options d'octroi de privilèges ; ils peuvent donc toujours réattribuer leurs propres privilèges.

Les privilèges disponibles sont :

SÉLECTIONNER

Autorise les requêtes SELECT sur n'importe quelle colonne, ou sur une ou plusieurs colonnes spécifiques, d'une table, d'une vue, d'une vue matérialisée ou d'un autre objet de type table. Autorise également l'utilisation de COPY TO. Ce privilège est également requis pour référencer les valeurs de colonnes existantes dans les opérations UPDATE, DELETE ou MERGE. Pour les séquences, ce privilège autorise également l'utilisation de la fonction curval. Pour les objets volumineux, ce privilège autorise la lecture de l'objet.

INSÉRER

Permet l'insertion d'une nouvelle ligne dans une table, une vue, etc. Cette autorisation peut être accordée sur une ou plusieurs colonnes spécifiques, auquel cas seules ces colonnes peuvent être affectées à la commande INSERT (les autres colonnes recevront alors des valeurs par défaut). Permet également l'utilisation de COPY FROM.

MISE À JOUR

Permet la mise à jour de n'importe quelle colonne, ou de colonnes spécifiques, d'une table, d'une vue, etc. (En pratique, toute commande UPDATE non triviale nécessitera également le privilège SELECT, car elle doit faire référence aux colonnes de la table pour déterminer les lignes à mettre à jour et/ou pour calculer les nouvelles valeurs des colonnes.)

Les instructions `SELECT ... FOR UPDATE` et `SELECT ... FOR SHARE` requièrent également ce privilège sur au moins une colonne, en plus du privilège `SELECT`. Pour les séquences, ce privilège autorise l'utilisation des fonctions `nextval` et `setval`. Pour les objets volumineux, ce privilège permet d'écrire ou de tronquer l'objet.

SUPPRIMER

Permet de supprimer une ligne d'une table, d'une vue, etc. (En pratique, toute commande `DELETE` non triviale nécessitera également le privilège `SELECT`, car elle doit faire référence aux colonnes de la table pour déterminer les lignes à supprimer.)

TRONQUER

Permet la fonction `TRUNCATE` sur un tableau.

RÉFÉRENCES

Permet la création d'une contrainte de clé étrangère faisant référence à une table, ou à une ou plusieurs colonnes spécifiques d'une table.

DÉCLENCHEMENT

Permet la création d'un déclencheur sur une table, une vue, etc.

CRÉER

Pour les bases de données, cela permet de créer de nouveaux schémas et publications au sein de la base de données, et d'installer des extensions de confiance dans celle-ci.

Pour les schémas, cette fonctionnalité permet la création de nouveaux objets. Pour renommer un objet existant, vous devez en être le propriétaire et disposer du privilège correspondant pour le schéma qui le contient.

Pour les espaces de tables, il permet de créer des tables, des index et des fichiers temporaires au sein de l'espace de tables, et permet de créer des bases de données qui utilisent cet espace de tables comme espace de tables par défaut.

Notez que la révocation de ce privilège ne modifiera ni l'existence ni l'emplacement des objets existants.

CONNECTER

Autorise le titulaire de l'autorisation à se connecter à la base de données. Ce privilège est vérifié au démarrage de la connexion (en plus de vérifier les restrictions imposées par `pg_hba.conf`).

TEMPORAIRE

Permet la création de tables temporaires lors de l'utilisation de la base de données.

EXÉCUTER

Permet d'appeler une fonction ou une procédure, y compris d'utiliser les opérateurs implémentés au-dessus de cette fonction. Il s'agit du seul type de privilège applicable aux fonctions et aux procédures.

USAGE

Pour les langages procéduraux, permet d'utiliser le langage pour la création de fonctions dans ce langage. C'est le seul type de privilège applicable aux langages procéduraux.

Pour les schémas, cette autorisation permet d'accéder aux objets qu'ils contiennent (sous réserve du respect des privilèges propres à chaque objet). Concrètement, elle permet au titulaire de l'autorisation de consulter les objets au sein du schéma. Sans cette autorisation, il est toujours possible de voir les noms des objets, par exemple en interrogeant les catalogues système. De plus, après la révocation de cette autorisation, des sessions existantes peuvent contenir des instructions ayant précédemment effectué cette recherche ; cette méthode n'est donc pas totalement sécurisée pour empêcher l'accès aux objets.

Pour les séquences, permet l'utilisation des fonctions curval et nextval.

Pour les types et les domaines, permet d'utiliser le type ou le domaine dans la création de tables, de fonctions et autres objets de schéma. (Notez que ce privilège ne contrôle pas toute « utilisation » du type, comme par exemple valeurs du type apparaissant dans les requêtes. Cela empêche uniquement la création d'objets qui dépendent de le type. L'objectif principal de ce privilège est de contrôler quels utilisateurs peuvent créer des dépendances. sur un type, ce qui pourrait empêcher le propriétaire de modifier le type ultérieurement.)

Pour les wrappers de données externes, permet la création de nouveaux serveurs utilisant le wrapper de données externes.

Pour les serveurs externes, permet la création de tables externes utilisant le serveur. Les bénéficiaires peuvent également créer : modifier ou supprimer leurs propres mappages d'utilisateurs associés à ce serveur.

ENSEMBLE

Permet de définir une nouvelle valeur pour un paramètre de configuration du serveur au cours de la session actuelle. (Tandis que Ce privilège peut être accordé sur n'importe quel paramètre ; il est dénué de sens sauf pour les paramètres qui seraient (Normalement, les privilèges de superutilisateur sont requis pour configurer.)

ALTER SYSTEM

Permet de configurer un paramètre de configuration du serveur avec une nouvelle valeur à l'aide de la commande ALTER. Commande SYSTÈME.

MAINTENIR

Permet d'effectuer les opérations suivantes : aspiration, analyse, regroupement, actualisation de la vue matérialisée, réindexation et VERROUILLER LA TABLE sur une relation.

Les privilèges requis par les autres commandes sont indiqués sur la page de référence de la commande respective.

PostgreSQL accorde par défaut des privilèges PUBLIC sur certains types d'objets lorsque ces objets sont créé. Aucun privilège n'est accordé par défaut à PUBLIC sur les tables, les colonnes de table, les séquences, les ressources étrangères Enveloppes de données, serveurs externes, objets volumineux, schémas, espaces de tables ou paramètres de configuration. Pour les autres types d'objets, les privilèges par défaut accordés à PUBLIC sont les suivants : privilèges CONNECT et TEMPORARY (création de tables temporaires) pour les bases de données ; privilège EXECUTE pour les fonctions et procédures ; et privilège USAGE pour les langues et les types de données (y compris les domaines). Le propriétaire de l'objet peut, bien sûr, révoquer les privilèges par défaut et ceux expressément accordés. (Pour une sécurité maximale, émettre la REVOKE dans la même transaction qui crée l'objet ; il n'y a alors aucune fenêtre dans laquelle une autre L'utilisateur peut utiliser l'objet. De plus, ces paramètres de privilèges par défaut peuvent être modifiés à l'aide de la commande ALTER. Commande PRIVILÈGES PAR DÉFAUT.

Le tableau 5.1 présente les abréviations à une lettre utilisées pour ces types de privilèges dans les listes de contrôle d'accès (ACL). valeurs de la liste de contrôle). Vous verrez ces lettres dans le résultat des commandes psql listées ci-dessous, ou lors de l'examen des colonnes ACL des catalogues système.

Tableau 5.1. Abréviations des privilèges ACL

Privilège	Abréviation	Types d'objets applicables
SÉLECTIONNER	r (« lire »)	OBJET VOLUMINEUX, SÉQUENCE, TABLEAU (et objets de type tableau), colonne de tableau
INSÉRER	un (« ajouter »)	TABLEAU, colonne du tableau
MISE À JOUR	w (« écrire »)	OBJET DE GRANDE TAILLE, SÉQUENCE, TABLEAU, tableau colonne
SUPPRIMER	d	TABLEAU
TRONQUER	D	TABLEAU
RÉFÉRENCES	x	TABLEAU, colonne du tableau
DÉCLENCHEMENT	t	TABLEAU

Privilège	Abréviation	Types d'objets applicables
CRÉER	C	BASE DE DONNÉES, SCHÉMA, ESPACE DE TABLES
CONNECTER	c	BASE DE DONNÉES
TEMPORAIRE	T	BASE DE DONNÉES
EXÉCUTER	X	FONCTION, PROCÉDURE
USAGE	U	DOMAINE, ENVELOPEUR DE DONNÉES ÉTRANGÈRES, SERVEUR ÉTRANGER, LANGUE, SCHÉMA, SÉQUENCE, TYPE
ENSEMBLE	s	PARAMÈTRE
ALTER SYSTEM	UN	PARAMÈTRE
MAINTENIR	m	TABLEAU

Le tableau 5.2 récapitule les privilèges disponibles pour chaque type d'objet SQL, en utilisant les abréviations suivantes : comme indiqué ci-dessus. Il présente également la commande `psql` qui peut être utilisée pour examiner les paramètres de privilèges pour chaque type d'objet.

Tableau 5.2. Récapitulatif des privilèges d'accès

Type d'objet	Tous les privilèges	Public par défaut privilèges	Commande psql
BASE DE DONNÉES	CTc	Tc	\l
DOMAINE	U	U	\dD+
FONCTION ou PROCÉDURE	X	X	\df+
ENVELOPEUR DE DONNÉES ÉTRANGÈRES	U	aucun	rosée+
SERVEUR ÉTRANGER	U	aucun	\des+
LANGUE	U	U	\dL+
OBJET DE GRANDE TAILLE	rw	aucun	\dl+
PARAMÈTRE	sA	aucun	\dconfig+
SCHÉMA	UC	aucun	\dn+
SÉQUENCE	rwU	aucun	\dp
TABLEAU (et objets de type tableau)	arwdDxtm	aucun	\dp
Colonne du tableau	arwx	aucun	\dp
ESPACE DE TABLE	C	aucun	\db+
TAPER	U	U	dT+

Les privilèges accordés à un objet particulier sont affichés sous forme de liste d'acilitem entrées, chacune ayant le format suivant :

bénéficiaire=abréviation-privilège[*].../concedant

Chaque acilitem énumère toutes les permissions accordées à un bénéficiaire par un donateur particulier.

Les privilèges spécifiques sont représentés par des abréviations d'une lettre tirées du tableau 5.1, avec un astérisque ajouté si le

Le privilège a été accordé avec l'option `grant`. Par exemple, `calvin=r*w/hobbes` spécifie que le rôle

Calvin possède le privilège `SELECT (r)` avec l'option d'octroi (*) ainsi que le privilège non octroyable

`MISE À JOUR (w)`, toutes deux accordées par le rôle Hobbes. Si Calvin possède également certains privilèges sur le même

Un objet octroyé par un autre donateur apparaîtrait comme une entrée d'acilitem distincte. Une entrée vide

Le champ bénéficiaire dans un acilitem signifie `PUBLIC`.

Par exemple, supposons que l'utilisateur `miriam` crée la table `mytable` et effectue les opérations suivantes :

```
GRANT SELECT ON mytable TO PUBLIC;
ACORDER LES PROCÈS DE SELECT, UPDATE, INSERT SUR mytable À admin;
GRANT SELECT (col1), UPDATE (col1) ON mytable TO miriam_rw;
```

La commande \dp de psql afficherait alors :

=> \dp matable

Schéma	Nom	Type	Privilèges d'accès	Colonnes
public	mytable	table	miriam=arwdDxtm/miriam+ col1:	
			+	
			=r/miriam	+
			miriam_rw=rw/miriam	
			admin=arw/miriam	

(1 rangée)

Si la colonne « Privilèges d'accès » est vide pour un objet donné, cela signifie que l'objet possède des privilèges par défaut. (c'est-à-dire que son entrée de privilèges dans le catalogue système correspondant est nulle). Les privilèges par défaut incluent toujours tous les privilèges pour le propriétaire, et peuvent inclure certains privilèges pour le PUBLIC en fonction du type d'objet, Comme expliqué ci-dessus, la première instruction GRANT ou REVOKE sur un objet instanciera les privilèges par défaut. (produisant, par exemple, miriam=arwdDxtm/miriam) puis les modifier conformément à la requête spécifiée. De même, les entrées ne sont affichées dans « Privilèges de colonne » que pour les colonnes dont les privilèges ne sont pas ceux par défaut. (Remarque : à cette fin, « privilèges par défaut » désigne toujours les privilèges par défaut intégrés pour la colonne.) Type de l'objet. Un objet dont les privilèges ont été modifiés par une commande ALTER DEFAULT PRIVILEGES. La commande sera toujours affichée avec une entrée de privilège explicite qui inclut les effets de ALTER.

Notez que les options d'autorisation implicites du propriétaire ne sont pas indiquées dans l'affichage des privilèges d'accès. Un astérisque (*) sera n'apparaissent que lorsque des options d'attribution ont été explicitement accordées à une personne.

La colonne « Privilèges d'accès » affiche (aucun) lorsque l'entrée des privilèges de l'objet n'est pas nulle, mais vide. Cela signifie qu'aucun privilège n'est accordé, même au propriétaire de l'objet — une situation rare. (Le propriétaire dispose toujours d'options d'octroi implicites dans ce cas, et pourrait donc octroyer à nouveau ses propres privilèges ; mais Elle n'en a pas pour le moment.

5.9. Politiques de sécurité des lignes

En plus du système de privilèges standard SQL disponible via GRANT, les tables peuvent avoir des lignes des politiques de sécurité qui limitent, pour chaque utilisateur, les lignes pouvant être renvoyées par des requêtes normales ou inséré, mis à jour ou supprimé par des commandes de modification de données. Cette fonctionnalité est également connue sous le nom de modification au niveau des lignes. Sécurité. Par défaut, les tables ne possèdent aucune politique, de sorte que si un utilisateur dispose de privilèges d'accès à une table Selon le système de privilèges SQL, toutes les lignes qu'il contient sont également disponibles pour les requêtes et les mises à jour.

Lorsque la sécurité des lignes est activée sur une table (avec ALTER TABLE ... ACTIVER LA SÉCURITÉ AU NIVEAU DES LIGNES TY), tout accès normal à la table pour la sélection ou la modification des lignes doit être autorisé par une ligne. politique de sécurité. (Toutefois, le propriétaire de la table n'est généralement pas soumis aux politiques de sécurité des lignes.) Si non Une politique existe pour la table ; une politique de refus par défaut est utilisée, ce qui signifie qu'aucune ligne n'est visible ni ne peut être affichée. Les opérations qui s'appliquent à l'ensemble du tableau, telles que TRUNCATE et REFERENCES, ne sont pas modifiées.

sous réserve de la sécurité des rangées.

Les politiques de sécurité des lignes peuvent être spécifiques aux commandes, aux rôles ou aux deux. Une politique peut être spécifiée. S'appliquer à TOUTES les commandes, ou à SELECT, INSERT, UPDATE ou DELETE. Plusieurs rôles peuvent être attribués. une politique donnée est attribuée, et les règles normales d'appartenance aux rôles et d'héritage s'appliquent.

Pour spécifier les lignes visibles ou modifiables selon une politique, une expression renvoyant un résultat booléen est requise. Cette expression sera évaluée pour chaque ligne avant toute condition ou fonction issue de la requête utilisateur. (Seules les fonctions étanches, qui garantissent l'absence de fuite d'informations, font exception à cette règle ; l'optimiseur peut choisir d'appliquer ces fonctions avant le contrôle de sécurité des lignes.) Les lignes pour lesquelles l'expression ne renvoie pas « vrai » ne seront pas traitées.

Des expressions distinctes peuvent être spécifiées pour contrôler indépendamment les lignes visibles et celles qui peuvent être modifiées. Les expressions de stratégie sont exécutées dans le cadre de la requête et avec les privilèges de l'utilisateur qui l'exécute, même si des fonctions de définition de sécurité peuvent être utilisées pour accéder à des données inaccessibles à cet utilisateur.

Les superutilisateurs et les rôles disposant de l'attribut BYPASSRLS contournent systématiquement le système de sécurité des lignes lors de l'accès à une table. Les propriétaires de table contournent généralement également la sécurité des lignes, mais ils peuvent choisir de s'y soumettre à l'aide de la commande ALTER TABLE ... FORCE ROW LEVEL SECURITY.

L'activation et la désactivation de la sécurité des lignes, ainsi que l'ajout de politiques à une table, relèvent toujours du seul privilège du propriétaire de la table.

Les politiques sont créées à l'aide de la commande CREATE POLICY, modifiées à l'aide de la commande ALTER POLICY et supprimées à l'aide de la commande DROP POLICY. Pour activer ou désactiver la sécurité des lignes d'une table donnée, utilisez la commande ALTER TABLE.

Chaque règle possède un nom et plusieurs règles peuvent être définies pour une même table. Les règles étant spécifiques à chaque table, chaque règle d'une table doit avoir un nom unique. Différentes tables peuvent avoir des règles portant le même nom.

Lorsqu'une requête comporte plusieurs règles, celles-ci sont combinées à l'aide de l'opérateur OU (pour les règles permissives, qui sont les règles par défaut) ou de l'opérateur ET (pour les règles restrictives). Ce principe est similaire à celui qui stipule qu'un rôle donné possède les privilèges de tous les rôles dont il est membre. La différence entre les règles permissives et restrictives est abordée plus en détail ci-dessous.

À titre d'exemple simple, voici comment créer une stratégie sur la relation de compte pour autoriser uniquement les membres du rôle de gestionnaire à accéder aux lignes, et uniquement aux lignes de leurs comptes :

```
CRÉER LA TABLE comptes (manager text, company text, contact_email
    texte);
```

```
ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;
```

```
CRÉER UNE POLITIQUE account_managers SUR accounts POUR managers
    UTILISATION (gestionnaire = utilisateur_actuel);
```

La politique ci-dessus fournit implicitement une clause WITH CHECK identique à sa clause USING, de sorte que la contrainte s'applique à la fois aux lignes sélectionnées par une commande (un gestionnaire ne peut donc pas SELECT, UPDATE ou DELETE des lignes existantes appartenant à un autre gestionnaire) et aux lignes modifiées par une commande (les lignes appartenant à un autre gestionnaire ne peuvent donc pas être créées via INSERT ou UPDATE).

Si aucun rôle n'est spécifié ou si le nom d'utilisateur spécial PUBLIC est utilisé, la stratégie s'applique à tous les utilisateurs du système. Pour permettre à chaque utilisateur d'accéder uniquement à sa propre ligne dans une table d'utilisateurs, une stratégie simple peut être utilisée :

```
CRÉER UNE POLITIQUE user_policy SUR utilisateurs
    UTILISATION (nom_utilisateur = utilisateur_actuel);
```

Cela fonctionne de la même manière que dans l'exemple précédent.

Pour appliquer une politique différente aux lignes ajoutées au tableau par rapport aux lignes visibles, plusieurs politiques peuvent être combinées. Cette paire de politiques permettrait à tous les utilisateurs de consulter toutes les lignes du tableau des utilisateurs, mais de modifier uniquement les leurs.

```

CRÉER UNE POLITIQUE user_sel_policy SUR les utilisateurs
POUR SÉLECTIONNER
    UTILISATION (vrai) ;
CRÉER UNE POLITIQUE user_mod_policy SUR utilisateurs
    UTILISATION (nom_utilisateur = utilisateur_actuel);

```

Dans une commande SELECT, ces deux règles sont combinées par l'opérateur OU, ce qui permet de sélectionner toutes les lignes. Dans les autres types de commandes, seule la seconde règle s'applique, et le résultat est identique.

La sécurité des lignes peut également être désactivée à l'aide de la commande ALTER TABLE. La désactivation de la sécurité des lignes ne supprime pas les stratégies définies sur la table ; elles sont simplement ignorées. Toutes les lignes de la table sont alors visibles et modifiables, sous réserve du système de privilèges SQL standard.

Voici un exemple plus détaillé de l'utilisation de cette fonctionnalité en environnement de production. La table passwd émule un fichier de mots de passe Unix :

```

-- Exemple simple basé sur un fichier de mot de passe
CRÉER LA TABLE passwd ( nom_utilisateur
    pwhash uid                                texte UNIQUE NON NULL, texte, int CLÉ
                                           PRIMAIRE, int NON NULL, texte
    gid                                       NON NULL, texte, texte,
    nom_réel                                texte NON NULL,
    téléphone_domicile
    informations_supplémentaires répertoire_personnel NON NULL
);

CRÉER LE RÔLE admin ; -- Administrateur
CRÉER LE RÔLE bob;      -- Utilisateur normal
CRÉER LE RÔLE alice; -- Utilisateur normal

-- Remplir le tableau
INSÉRER DANS passwd VALEURS
    ('admin','xxx',0,0,'Admin','111-222-3333',null,'/root','/bin/
tired');
INSÉRER DANS passwd VALEURS
    ('bob','xxx',1,1,'Bob','123-456-7890',null,'/home/bob','/bin/
zsh');
INSÉRER DANS passwd VALEURS
    ('alice','xxx',2,1,'Alice','098-765-4321',null,'/home/alice','/ bin/zsh');

-- Veuillez à activer la sécurité au niveau des lignes sur la table
ALTER TABLE passwd ENABLE ROW LEVEL SECURITY;

-- Créer des politiques
-- L'administrateur peut voir toutes les lignes et en ajouter d'autres.
CRÉER UNE POLITIQUE admin_all SUR passwd POUR admin EN UTILISANT (true) AVEC VÉRIFICATION
    (vrai);
-- Les utilisateurs normaux peuvent voir toutes les lignes
CRÉER UNE POLITIQUE all_view SUR passwd POUR LA SÉLECTION EN UTILISANT (true);
-- Les utilisateurs normaux peuvent mettre à jour leurs propres enregistrements, mais -- limitent
les shells qu'un utilisateur normal est autorisé à définir
CRÉER UNE POLITIQUE user_mod ON passwd POUR LA MISE À JOUR

```

```

UTILISATION (utilisateur_actuel = nom_utilisateur)
AVEC CHÈQUE (
    utilisateur_actuel = nom_utilisateur ET
    shell IN ('/bin/bash','/bin/sh','/bin/dash','/bin/zsh','/bin/
tcsh')
);

-- Accorder à l'administrateur tous les droits normaux
GRANT SELECT, INSERT, UPDATE, DELETE ON passwd TO admin;
-- Les utilisateurs n'ont qu'un accès limité aux colonnes publiques
ACCORDER SÉLECTION
    (nom_utilisateur, uid, gid, nom_réel, numéro_de_téléphone_domicile, informations_supplémentaires,
    répertoire personnel, shell)
    ON passwd AU public;
-- Autoriser les utilisateurs à mettre à jour certaines colonnes
MISE À JOUR DES SUBVENTIONS
    (mot_de_passe, nom_réel, numéro_de_téléphone_domicile, informations_supplémentaires, shell)
    ON passwd AU public;

```

Comme pour tout paramètre de sécurité, il est important de tester et de s'assurer que le système se comporte comme prévu. L'exemple ci-dessus démontre que le système d'autorisation fonctionne correctement.

-- L'administrateur peut consulter toutes les lignes et tous les champs

postgres=> définir le rôle administrateur ;

ENSEMBLE

postgres=> table passwd;

```

nom_utilisateur | mot_de_passe | uid | gid | nom_réel | numéro_de_téléphone_domicile |
informations supplémentaires | répertoire personnel | shell
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

administrateur | xxx | / | 0 | 0 | Admin | /bin/ | 111-222-3333 |
racine dash
bob | xxx | / | 1 | 1 | Bob | 123-456-7890 |
home/bob | /bin/zsh
Alice | xxx | / | 2 | 1 | Alice | 098-765-4321 |
home/alice | /bin/zsh

```

(3 rangées)

-- Tester ce qu'Alice est capable de faire

postgres=> définir le rôle alice;

ENSEMBLE

postgres=> table passwd;

ERREUR : autorisation refusée pour le mot de passe de la table

postgres=> sélectionner

```

nom_utilisateur, vrai_nom, téléphone_fixe, informations_supplémentaires, répertoire_personnel, shell depuis
mot de passe;

```

```

nom_utilisateur | nom_réel | téléphone_domicile | informations_supplémentaires | répertoire_personnel |
coquille
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

+-----+

```

```

administrateur | Admin | / | 111-222-3333 | /racine
bin/dash
bob | Bob | / | 123-456-7890 | /home/bob
bin/zsh
Alice | Alice | /bin/ | 098-765-4321 | /home/alice
zsh

```

(3 rangées)

```

postgres=> update passwd set user_name = 'joe';
ERREUR : accès refusé pour le mot de passe de la table
-- Alice est autorisée à changer son propre vrai nom, mais pas les autres
postgres=> update passwd set real_name = 'Alice Doe';
MISE À JOUR 1

postgres=> update passwd set real_name = 'John Doe' where user_name
= 'admin';
MISE À JOUR 0

postgres=> update passwd set shell = '/bin/xx';
ERREUR : la nouvelle ligne viole l'option WITH CHECK pour « passwd »
postgres=> supprimer de
passwd ;
ERREUR : permission refusée pour la table passwd
postgres=> insert into
passwd (user_name) values ('xxx');
ERREUR : accès refusé pour le mot de passe de la table
-- Alice peut changer son propre mot de passe ; RLS empêche silencieusement la mise à jour des autres
lignes
postgres=> update passwd
set pwhash = 'abc' ;
MISE À JOUR 1

```

Toutes les politiques définies jusqu'à présent sont des politiques permissives, ce qui signifie que lorsqu'elles sont appliquées plusieurs fois, elles sont combinées à l'aide de l'opérateur booléen « OU ». Bien que les politiques permissives puissent être conçues pour n'autoriser l'accès aux lignes que dans les cas prévus, il peut être plus simple de les combiner avec des politiques restrictives (auxquelles les enregistrements doivent répondre et qui sont combinées à l'aide de l'opérateur booléen « ET »). En reprenant l'exemple précédent, nous ajoutons une politique restrictive exigeant que l'administrateur se connecte via un socket Unix local pour accéder aux enregistrements de la table passwd :

```

CRÉER UNE POLITIQUE admin_local_only SUR passwd COMME RESTRICTIVE À admin EN UTILISANT
(pg_catalog.inet_client_addr() EST NULL);

```

On constate alors qu'un administrateur se connectant via un réseau ne verra aucun enregistrement, en raison de la politique restrictive :

=> SÉLECTIONNER l'utilisateur

```

actuel; utilisateur actuel
-----

```

```

administrateur

```

(1 rangée)

=> sélectionner inet_client_addr(); inet_client_addr

```

-----
127.0.0.1

```

(1 rangée)

=> TABLE passwd;

```

nom_utilisateur | mot_de_passe | uid | gid | nom_réal | téléphone_domicile | infos_supplémentaires |
répertoire_personnel | shell
-----+-----+-----+-----+-----+-----+-----+-----+-----

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+

```

(0 lignes)

=> MISE À JOUR du mot de passe : pwhash = NULL ;

MISE À JOUR 0

Les contrôles d'intégrité référentielle, tels que les contraintes d'unicité ou de clé primaire et les références de clés étrangères, contournent toujours la sécurité des lignes afin de garantir l'intégrité des données. Il convient d'être vigilant lors de l'élaboration des schémas et des politiques au niveau des lignes afin d'éviter les fuites d'informations par « canal caché » via ces contrôles d'intégrité référentielle.

Dans certains contextes, il est crucial de s'assurer que la sécurité des lignes n'est pas appliquée. Par exemple, lors d'une sauvegarde, il serait catastrophique que la sécurité des lignes entraîne l'omission silencieuse de certaines lignes. Dans ce cas, vous pouvez désactiver le paramètre de configuration `row\_security`. Cela ne désactive pas la sécurité des lignes en soi ; cela génère simplement une erreur si les résultats d'une requête sont filtrés par une politique. La cause de l'erreur peut alors être analysée et corrigée.

Dans les exemples ci-dessus, les expressions de stratégie ne prennent en compte que les valeurs actuelles de la ligne consultée ou mise à jour. Il s'agit du cas le plus simple et le plus performant ; dans la mesure du possible, il est préférable de concevoir les applications de sécurité des lignes de cette manière. S'il est nécessaire de consulter d'autres lignes ou d'autres tables pour prendre une décision de stratégie, cela peut être réalisé à l'aide de sous-requêtes SELECT ou de fonctions contenant des requêtes SELECT dans les expressions de stratégie. Attention toutefois : de tels accès peuvent créer des conditions de concurrence susceptibles d'entraîner des fuites d'informations si l'on n'y prend pas garde. À titre d'exemple, considérons la structure de table suivante :

```
-- définition des groupes privilégiés
CRÉER TABLE groupes (group_id int CLÉ PRIMAIRE, group_name texte NON NULL);

INSÉRER DANS groupes VALEURS (1, 'faible'),
(2, 'moyen'), (5,
'élevé');

GRANT ALL ON groups TO alice; -- alice est l'administratrice
GRANT SELECT ON groups TO public;

-- définition des niveaux de privilèges des utilisateurs
CRÉER TABLE utilisateurs (nom_utilisateur texte CLÉ PRIMAIRE,
group_id int NOT NULL REFERENCES groupes);

INSÉRER DANS users VALUES
(« alice », 5), (« bob »,
2), (« mallory », 2);

ACCORDER TOUS LES UTILISATEURS À alice;
GRANT SELECT ON users TO public;

-- tableau contenant les informations à protéger
CRÉER TABLE information (texte d'information,
group_id int NOT NULL REFERENCES groupes);

INSÉRER DANS information VALEURS
(« à peine secret », 1), (« légèrement
secret », 2), (« très secret », 5);

ALTER TABLE information ENABLE ROW LEVEL SECURITY;

-- une ligne doit être visible et modifiable par les utilisateurs dont la sécurité est contrôlée.
group_id est --
supérieur ou égal à l'id de groupe de la ligne
CRÉER UNE POLITIQUE fp_s SUR information POUR SÉLECTIONNER
```

```
UTILISER (group_id <= (SELECT group_id FROM users WHERE user_name = current_user));
```

CRÉER UNE POLITIQUE fp_u SUR les informations POUR MISE À JOUR

```
UTILISER (group_id <= (SELECT group_id FROM users WHERE user_name = current_user));
```

-- nous ne comptons que sur la sécurité au niveau des lignes (RLS) pour protéger la table d'informations
ACCORDER TOUTES LES INFORMATIONS AU PUBLIC ;

Supposons maintenant qu'Alice souhaite modifier les informations « légèrement secrètes », mais qu'elle décide qu'il ne faut pas faire confiance à Mal-lory concernant le nouveau contenu de cette ligne, alors elle fait :

COMMENCER;

```
MISE À JOUR utilisateurs SET group_id = 1 WHERE user_name = 'mallory';
```

```
MISE À JOUR des informations SET info = 'secret de Mallory' WHERE group_id  
= 2;
```

COMMETTRE;

Cela semble sûr ; il n'y a aucune fenêtre temporelle pendant laquelle Mallory devrait pouvoir voir la chaîne « secret de Mallory ». Cependant, il existe une condition de concurrence. Si Mallory effectue simultanément, par exemple,

```
SELECT * FROM information WHERE group_id = 2 FOR UPDATE;
```

et sa transaction étant en mode READ COMMITTED, il lui est possible de voir « secret de Mallory ».

Cela se produit si sa transaction atteint la ligne d'information juste après celle d'Alice. Le système se bloque en attendant la validation de la transaction d'Alice, puis récupère le contenu mis à jour de la ligne grâce à la clause FOR UPDATE. Cependant, il ne récupère pas de ligne mise à jour pour la sous-requête SELECT implicite sur la table « users », car cette sous-requête ne comportait pas de clause FOR UPDATE ; la ligne « users » est alors lue avec l'instantané pris au début de la requête. Par conséquent, l'expression de stratégie teste l'ancienne valeur du niveau de privilège de Mallory et lui permet de consulter la ligne mise à jour.

Il existe plusieurs solutions à ce problème. Une solution simple consiste à utiliser `SELECT ... FOR SHARE` dans les sous-requêtes `SELECT` des politiques de sécurité des lignes. Cependant, cela nécessite d'accorder le privilège UPDATE sur la table référencée (ici, `users`) aux utilisateurs concernés, ce qui peut s'avérer problématique. (Une autre politique de sécurité des lignes pourrait être appliquée pour les empêcher d'exercer ce privilège ; ou la sous-requête `SELECT` pourrait être intégrée à une fonction de définition de sécurité.) De plus, une utilisation intensive et simultanée des verrous de partage de lignes sur la table référencée pourrait engendrer des problèmes de performance, notamment si ses mises à jour sont fréquentes. Une autre solution, pratique si les mises à jour de la table référencée sont peu fréquentes, consiste à acquérir un verrou `ACCESS EXCLUSIVE` sur la table lors de sa mise à jour, afin qu'aucune transaction simultanée ne puisse accéder aux anciennes valeurs de lignes. On peut également attendre la fin de toutes les transactions simultanées après la validation d'une mise à jour de la table référencée et avant d'effectuer des modifications dépendant de la nouvelle situation de sécurité.

Pour plus de détails, consultez CREATE POLICY et ALTER TABLE.

5.10. Schémas

Un cluster de bases de données PostgreSQL contient une ou plusieurs bases de données nommées. Les rôles et certains autres types d'objets sont partagés entre les différentes bases de données du cluster. Une connexion client au serveur ne peut accéder aux données que d'une seule base de données : celle spécifiée dans la requête de connexion.

Note

Les utilisateurs d'un cluster n'ont pas nécessairement le privilège d'accéder à toutes les bases de données du cluster. Le partage des noms de rôles signifie qu'il ne peut pas y avoir de rôles différents nommés, par exemple, Joe dans deux rôles différents.

bases de données du même cluster ; mais le système peut être configuré pour permettre à Joe d'accéder uniquement à certaines bases de données.

Une base de données contient un ou plusieurs schémas nommés, qui contiennent eux-mêmes des tables. Les schémas contiennent également d'autres types d'objets nommés, tels que des types de données, des fonctions et des opérateurs. Au sein d'un même schéma, deux objets du même type ne peuvent pas porter le même nom. De plus, les tables, les séquences, les index, les vues, les vues matérialisées et les tables externes partagent le même espace de noms ; par conséquent, un index et une table doivent avoir des noms différents s'ils appartiennent au même schéma. Un même nom d'objet peut être utilisé dans différents schémas sans conflit ; par exemple, les schémas « schéma1 » et « monschéma » peuvent tous deux contenir des tables nommées « matable ». Contrairement aux bases de données, les schémas ne sont pas strictement séparés : un utilisateur peut accéder aux objets de n'importe quel schéma de la base de données à laquelle il est connecté, s'il dispose des privilèges nécessaires.

Il existe plusieurs raisons pour lesquelles on pourrait vouloir utiliser des schémas :

- Pour permettre à de nombreux utilisateurs d'utiliser une même base de données sans interférer les uns avec les autres.
- Organiser les objets de la base de données en groupes logiques afin de les rendre plus faciles à gérer.
- Les applications tierces peuvent être placées dans des schémas distincts afin d'éviter les conflits de noms d'autres objets.

Les schémas sont analogues aux répertoires au niveau du système d'exploitation, à ceci près que les schémas ne peuvent pas être imbriqués.

5.10.1. Création d'un schéma

Pour créer un schéma, utilisez la commande CREATE SCHEMA. Donnez-lui le nom de votre choix.

Par exemple:

```
CRÉER SCHÉMA myschema;
```

Pour créer des objets dans un schéma ou y accéder, écrivez un nom qualifié composé du nom du schéma et du nom de la table, séparés par un point :

schéma.table

Cela fonctionne partout où un nom de table est attendu, y compris les commandes de modification de table et les commandes d'accès aux données abordées dans les chapitres suivants. (Par souci de concision, nous ne parlerons que de tables, mais les mêmes principes s'appliquent à d'autres types d'objets nommés, tels que les types et les fonctions.)

En fait, la syntaxe encore plus générale

table de schéma de base de données

Cela peut également être utilisé, mais actuellement, ce n'est qu'une formalité pour se conformer à la norme SQL. Si vous indiquez un nom de base de données, il doit être identique à celui de la base de données à laquelle vous vous connectez.

Pour créer une table dans le nouveau schéma, utilisez :

```
CRÉER LA TABLE myschema.mytable (  
...
```

);

Pour supprimer un schéma s'il est vide (tous les objets qu'il contient ont été supprimés), utilisez :

```
ABANDONNER SCHÉMA myschema;
```

Pour supprimer un schéma incluant tous les objets qu'il contient, utilisez :

```
DROP SCHEMA myschema CASCADE;
```

Voir la section 5.15 pour une description du mécanisme général sous-jacent.

Il est souvent utile de créer un schéma appartenant à un tiers (car c'est l'un des moyens de limiter les activités de vos utilisateurs à des espaces de noms bien définis). La syntaxe correspondante est la suivante :

```
CRÉER UN SCHÉMA nom_schéma AUTORISATION nom_utilisateur ;
```

Vous pouvez même omettre le nom du schéma ; dans ce cas, il sera identique au nom d'utilisateur. Consultez la section 5.10.6 pour en savoir plus sur l'utilité de cette fonctionnalité.

Les noms de schémas commençant par `pg_` sont réservés au système et ne peuvent pas être créés par les utilisateurs.

5.10.2. Le schéma public

Dans les sections précédentes, nous avons créé des tables sans spécifier de nom de schéma. Par défaut, ces tables (et autres objets) sont automatiquement placées dans un schéma nommé « public ». Chaque nouvelle base de données contient un tel schéma. Ainsi, les opérations suivantes sont équivalentes :

```
CRÉER TABLE produits ( ... );
```

et:

```
CRÉER TABLE public.products ( ... );
```

5.10.3. Le chemin de recherche du schéma

Les noms qualifiés sont fastidieux à rédiger, et il est souvent préférable de ne pas intégrer un nom de schéma particulier dans les applications. C'est pourquoi les tables sont souvent désignées par des noms non qualifiés, composés uniquement de leur nom. Le système détermine la table concernée en suivant un chemin de recherche, c'est-à-dire une liste de schémas à parcourir. La première table correspondante dans ce chemin est considérée comme la table recherchée.

Si aucune correspondance n'est trouvée dans le chemin de recherche, une erreur est signalée, même si des noms de tables correspondants existent dans d'autres schémas de la base de données.

La possibilité de créer des objets portant le même nom dans différents schémas complique l'écriture de requêtes référençant systématiquement les mêmes objets. Elle offre également aux utilisateurs la possibilité de modifier, intentionnellement ou non, le comportement des requêtes d'autres utilisateurs. En raison de la fréquence des noms non qualifiés dans les requêtes et de leur utilisation au sein de PostgreSQL, l'ajout d'un schéma à `'search_path'` revient à faire confiance à tous les utilisateurs disposant du privilège `CREATE` sur ce schéma. Lors de l'exécution d'une requête classique, un utilisateur malveillant capable de créer des objets dans un schéma de votre chemin de recherche peut en prendre le contrôle et exécuter des fonctions SQL arbitraires comme si vous les aviez exécutées.

Le premier schéma mentionné dans le chemin de recherche est appelé schéma courant. Outre le fait qu'il s'agit du premier schéma recherché, c'est également celui dans lequel les nouvelles tables seront créées si la commande CREATE TABLE ne spécifie pas de nom de schéma.

Pour afficher le chemin de recherche actuel, utilisez la commande suivante :

AFFICHER le chemin de recherche ;

Dans la configuration par défaut, cela renvoie :

```
-----
chemin_de_recherche
```

```
"$user", public
```

Le premier élément spécifie qu'un schéma portant le même nom que l'utilisateur actuel doit être recherché.

Si aucun schéma de ce type n'existe, l'entrée est ignorée. Le deuxième élément fait référence au schéma public que nous avons déjà vu.

Le premier schéma rencontré dans le chemin de recherche est l'emplacement par défaut pour la création de nouveaux objets. C'est pourquoi, par défaut, les objets sont créés dans le schéma public. Lorsqu'un objet est référencé dans un autre contexte sans spécification de schéma (modification de table, modification de données ou commandes de requête), le chemin de recherche est parcouru jusqu'à ce qu'un objet correspondant soit trouvé. Par conséquent, dans la configuration par défaut, tout accès non qualifié ne peut concerner que le schéma public.

Pour ajouter notre nouveau schéma au chemin d'accès, nous utilisons :

```
DÉFINIR search_path SUR myschema,public;
```

(Nous omettons ici \$user car nous n'en avons pas besoin immédiatement.) Nous pouvons ensuite accéder à la table sans qualification de schéma :

```
SUPPRIMER LA TABLE mytable;
```

De plus, étant donné que myschema est le premier élément du chemin, les nouveaux objets y seraient créés par défaut.

Nous aurions également pu écrire :

```
DÉFINIR search_path SUR myschema ;
```

Nous n'avons alors plus accès au schéma public sans autorisation explicite. Ce schéma public n'a rien de particulier, si ce n'est qu'il existe par défaut. Il peut également être supprimé.

Voir également la section 9.27 pour d'autres façons de manipuler le chemin de recherche du schéma.

Le chemin de recherche fonctionne de la même manière pour les noms de types de données, les noms de fonctions et les noms d'opérateurs que pour les noms de tables. Les noms de types de données et de fonctions peuvent être qualifiés exactement de la même manière que les noms de tables. Si vous devez écrire un nom d'opérateur qualifié dans une expression, une disposition particulière s'applique : vous devez écrire

```
OPÉRATEUR(schéma.opérateur)
```

Cela est nécessaire pour éviter toute ambiguïté syntaxique. Par exemple :

SÉLECTIONNER 3 OPÉRATEUR(pg_catalog.*) 4;

En pratique, on s'appuie généralement sur le chemin de recherche des opérateurs, afin de ne pas avoir à écrire quelque chose d'aussi laid.

5.10.4. Schémas et privilèges

Par défaut, les utilisateurs ne peuvent accéder aux objets des schémas dont ils ne sont pas propriétaires. Pour ce faire, le propriétaire du schéma doit accorder le privilège USAGE sur ce schéma. Par défaut, ce privilège est public et accessible à tous. Pour permettre aux utilisateurs d'utiliser les objets d'un schéma, des privilèges supplémentaires peuvent être nécessaires, selon l'objet concerné.

Un utilisateur peut également être autorisé à créer des objets dans le schéma d'un autre utilisateur. Pour ce faire, le privilège CREATE doit lui être accordé sur le schéma. Dans les bases de données mises à niveau depuis PostgreSQL 14 ou une version antérieure, ce privilège est public pour tous les utilisateurs. Certains cas d'utilisation nécessitent la révocation de ce privilège.

RÉVOQUER CRÉER SUR SCHÉMA public À PARTIR DE PUBLIC ;

(Le premier terme « public » désigne le schéma, le second « public » signifie « chaque utilisateur ». Dans le premier sens, il s'agit d'un identifiant, dans le second, d'un mot-clé ; d'où la différence de majuscules ; voir les consignes de la section 4.1.1.)

5.10.5. Schéma du catalogue système

Outre les schémas publics et ceux créés par l'utilisateur, chaque base de données contient un schéma pg_catalog, qui regroupe les tables système ainsi que tous les types de données, fonctions et opérateurs intégrés. pg_catalog fait toujours partie intégrante du chemin de recherche. S'il n'est pas explicitement mentionné dans ce chemin, il est implicitement pris en compte avant les autres schémas. Ceci garantit que les noms intégrés seront toujours trouvés. Toutefois, vous pouvez placer explicitement pg_catalog à la fin de votre chemin de recherche si vous préférez que les noms définis par l'utilisateur prévalent sur les noms intégrés.

Les noms des tables système commençant par « pg_ », il est préférable de les éviter afin de prévenir tout conflit si une version ultérieure définit une table système portant le même nom que la vôtre. (Avec le chemin de recherche par défaut, une référence non qualifiée à votre nom de table serait alors résolue comme la table système.) Les tables système continueront de suivre la convention des noms commençant par « pg_ », afin d'éviter tout conflit avec les noms de tables utilisateur non qualifiés, tant que les utilisateurs n'utilisent pas le préfixe « pg_ ».

5.10.6. Modèles d'utilisation

Les schémas permettent d'organiser vos données de multiples façons. Un modèle d'utilisation de schéma sécurisé empêche les utilisateurs non autorisés de modifier le comportement des requêtes d'autres utilisateurs. Lorsqu'une base de données n'utilise pas un tel modèle, les utilisateurs souhaitant l'interroger en toute sécurité doivent prendre des mesures de protection au début de chaque session. Concrètement, ils doivent initialiser le paramètre `search\_path` à une chaîne vide ou supprimer de ce paramètre les schémas accessibles en écriture aux utilisateurs non administrateurs. La configuration par défaut prend en charge plusieurs modèles d'utilisation :

- Limitez l'accès des utilisateurs ordinaires aux schémas privés. Pour ce faire, assurez-vous d'abord qu'aucun schéma ne dispose de privilèges CREATE publics. Ensuite, pour chaque utilisateur devant créer des objets non temporaires, créez un schéma portant son nom, par exemple : CREATE SCHEMA alice AUTHORIZATION alice. (Rappelons que le chemin de recherche par défaut commence par \$user, qui correspond au nom d'utilisateur. Par conséquent, si chaque utilisateur dispose d'un schéma distinct, il accède par défaut à ses propres schémas.) Ce modèle garantit une utilisation sécurisée des schémas, sauf si un utilisateur non fiable est propriétaire de la base de données ou dispose des privilèges ADMIN OPTION pour un rôle approprié ; dans ce cas, aucune méthode d'utilisation sécurisée des schémas n'est applicable.

Dans PostgreSQL 15 et versions ultérieures, la configuration par défaut prend en charge ce modèle d'utilisation. Dans les versions antérieures, ou lors de l'utilisation d'une base de données mise à niveau depuis une version antérieure, vous devrez supprimer le privilège `CREATE public` du schéma public (exécutez la commande `REVOKE CREATE ON SCHEMA public FROM PUBLIC`). Ensuite, il est recommandé d'auditer le schéma public afin d'identifier les objets portant le même nom que ceux du schéma `pg_catalog`.

Supprimez le schéma public du chemin de recherche par défaut en modifiant le fichier `postgresql.conf` ou en exécutant la commande `ALTER ROLE ALL SET search_path = "$user"`. Ensuite, accordez les privilèges nécessaires à la création d'objets dans le schéma public. Seuls les noms qualifiés permettront de sélectionner les objets du schéma public. Bien que les références qualifiées aux tables soient acceptables, les appels aux fonctions du schéma public seront non sécurisés. Si vous créez des fonctions ou des extensions dans le schéma public, utilisez plutôt le premier modèle. Sinon, comme le premier modèle, cette méthode est sécurisée sauf si un utilisateur non fiable est le propriétaire de la base de données ou dispose des privilèges `ADMIN OPTION` pour un rôle approprié.

- Conservez le chemin de recherche par défaut et accordez les privilèges de création dans le schéma public. Tous les utilisateurs accèdent implicitement à ce schéma. Cela simule une situation où les schémas ne sont pas disponibles, assurant une transition en douceur depuis un environnement sans schéma. Cependant, cette approche n'est jamais sécurisée. Elle n'est acceptable que pour une base de données mono-utilisateur ou un petit nombre d'utilisateurs se faisant mutuellement confiance. Dans les bases de données mises à niveau depuis PostgreSQL 14 ou une version antérieure, il s'agit du comportement par défaut.

Pour chaque modèle, afin d'installer des applications partagées (tables à usage général, fonctionnalités supplémentaires fournies par des tiers, etc.), placez-les dans des schémas distincts. N'oubliez pas d'accorder les privilèges nécessaires pour permettre aux autres utilisateurs d'y accéder. Ces derniers pourront ensuite référencer ces objets supplémentaires en qualifiant leur nom avec le nom du schéma, ou bien ajouter les schémas supplémentaires à leur chemin de recherche, selon leur préférence.

5.10.7. Portabilité

Dans la norme SQL, la notion d'objets appartenant à différents utilisateurs au sein d'un même schéma n'existe pas. De plus, certaines implémentations interdisent la création de schémas portant un nom différent de celui de leur propriétaire. En réalité, les concepts de schéma et d'utilisateur sont quasiment équivalents dans un système de base de données qui n'implémente que la prise en charge de base des schémas spécifiée dans la norme. Par conséquent, de nombreux utilisateurs considèrent que les noms qualifiés se composent en fait de `nom_utilisateur.nom_table`. C'est ainsi que PostgreSQL se comportera concrètement si vous créez un schéma par utilisateur pour chaque utilisateur.

De plus, la norme SQL ne prévoit pas la notion de schéma public. Pour une conformité maximale à la norme, il est déconseillé d'utiliser un schéma public.

Bien entendu, certains systèmes de bases de données SQL peuvent ne pas implémenter de schémas, ou se contenter d'offrir une prise en charge des espaces de noms en autorisant un accès inter-bases de données (éventuellement limité). Si vous devez utiliser ces systèmes, la meilleure solution pour une portabilité maximale est de ne pas utiliser de schémas.

5.11. Héritage

PostgreSQL implémente l'héritage de tables, ce qui peut s'avérer un outil utile pour les concepteurs de bases de données. (SQL:1999 et versions ultérieures définissent une fonctionnalité d'héritage de type, qui diffère à bien des égards des fonctionnalités décrites ici.)

Prenons un exemple : supposons que nous souhaitions créer un modèle de données pour les villes. Chaque État compte plusieurs villes, mais une seule capitale. Nous voulons pouvoir retrouver rapidement la capitale de n'importe quel État. Pour cela, nous pouvons créer deux tables : une pour les capitales d'État et une autre pour les villes qui ne sont pas capitales. Cependant, comment faire pour obtenir des données sur une ville, qu'elle soit capitale ou non ? L'héritage permet de résoudre ce problème. Nous définissons la table des capitales de manière à ce qu'elle hérite de la table des villes.

```
CRÉER TABLE villes (
    nom          texte,
    altitude de la flotter,
    population    int      -- en pieds
);
```

```
CRÉER TABLE capitales (
    état char(2)
) HÉRITE (villes);
```

Dans ce cas, la table des capitales hérite de toutes les colonnes de sa table parente, les villes. Capitales des États
Elles comportent également une colonne supplémentaire, « état », qui indique leur état.

Dans PostgreSQL, une table peut hériter de zéro ou plusieurs autres tables, et une requête peut faire référence à l'une ou l'autre de ces tables. toutes les lignes d'une table ou toutes les lignes d'une table plus toutes ses tables descendantes. Ce dernier comportement est le par défaut. Par exemple, la requête suivante trouve les noms de toutes les villes, y compris les capitales d'État, qui sont situés à une altitude supérieure à 500 pieds :

```
SÉLECTIONNER nom, altitude
DES villes
OÙ l'altitude est supérieure à 500 ;
```

Avec les données d'exemple du tutoriel PostgreSQL (voir section 2.1), on obtient :

nom	altitude
Las Vegas	2174
Mariposa	1953
Madison	845

En revanche, la requête suivante trouve toutes les villes qui ne sont pas des capitales d'État et qui sont situées à une altitude supérieure à 500 pieds :

```
SÉLECTIONNER nom, altitude
DES SEULEMENT des villes
OÙ l'altitude est supérieure à 500 ;
```

nom	altitude
Las Vegas	2174
Mariposa	1953

Ici, le mot-clé ONLY indique que la requête doit s'appliquer uniquement aux villes, et non à toutes les tables sous les villes dans la hiérarchie d'héritage. Bon nombre des commandes que nous avons déjà abordées. — SELECT, UPDATE et DELETE — prennent en charge le mot-clé ONLY.

Vous pouvez également écrire le nom de la table avec un astérisque final pour indiquer explicitement que les tables descendantes sont comprises:

```
SÉLECTIONNER nom, altitude
DES villes*
OÙ l'altitude est supérieure à 500 ;
```

Il n'est pas nécessaire d'écrire *, car ce comportement est toujours celui par défaut. Cependant, cette syntaxe reste valable. Prise en charge pour assurer la compatibilité avec les versions antérieures où la valeur par défaut pouvait être modifiée.

Dans certains cas, vous souhaitez peut-être savoir de quelle table provient une ligne particulière. Chaque table possède une colonne système appelée `tableoid` qui vous indique la table d'origine :

```
SÉLECTIONNER c.tableoid, c.name, c.elevation DE cities c OÙ c.elevation > 500;
```

qui renvoie :

tableoid nom	altitude
139793 Las Vegas	2174
139793 Mariposa	1953
139798 Madison	845

(Si vous essayez de reproduire cet exemple, vous obtiendrez probablement des OID numériques différents.) En effectuant une jointure avec `pg_class`, vous pouvez voir les noms de tables réels :

```
SELECT p.relname, c.name, c.elevation FROM cities c, pg_class p
WHERE c.elevation > 500 AND c.tableoid =
p.oid;
```

qui renvoie :

nom de famille nom	altitude
villes Las Vegas villes Mariposa	2174
capitales Madison	1953
	845

Une autre façon d'obtenir le même résultat consiste à utiliser le type alias `regclass`, qui affichera l'OID de la table de manière symbolique :

```
SELECT c.tableoid::regclass, c.name, c.elevation FROM cities c
```

```
OÙ c.élévation > 500 ;
```

L'héritage ne propage pas automatiquement les données des commandes `INSERT` ou `COPY` aux autres tables de la hiérarchie d'héritage. Dans notre exemple, l'instruction `INSERT` suivante échouera :

```
INSÉRER DANS les villes (nom, population, altitude, état)
VALEURS ("Albany", NULL, NULL, "NY") ;
```

On pourrait espérer que les données soient acheminées vers la table des capitales, mais ce n'est pas le cas : la commande `INSERT` insère toujours les données dans la table spécifiée. Dans certains cas, il est possible de rediriger l'insertion à l'aide d'une règle (voir chapitre 39). Cependant, cela ne résout pas le problème ici, car la table des villes ne contient pas la colonne « état », et la commande sera donc rejetée avant même que la règle puisse être appliquée.

Toutes les contraintes de vérification et les contraintes de non-nullité d'une table parente sont automatiquement héritées par ses tables enfants, sauf indication contraire explicite avec des clauses `NO INHERIT`. Les autres types de contraintes (contraintes d'unicité, de clé primaire et de clé étrangère) ne sont pas hérités.

Une table peut hériter de plusieurs tables parentes ; dans ce cas, elle possède l'union des colonnes définies par les tables parentes. Les colonnes déclarées dans la définition de la table enfant sont ajoutées à cet ensemble.

Si un même nom de colonne apparaît dans plusieurs tables parentes, ou à la fois dans une table parente et dans la définition de la table enfant, ces colonnes sont fusionnées afin qu'il n'y ait qu'une seule colonne portant ce nom dans la table enfant. Pour être fusionnées, les colonnes doivent avoir le même type de données ; dans le cas contraire, une erreur est générée. Les contraintes de vérification héréditaires et les contraintes de non-nullité sont fusionnées de la même manière. Ainsi, par exemple, une colonne fusionnée sera marquée comme non-nullité si l'une des définitions de colonne dont elle provient est marquée comme non-nullité. Les contraintes de vérification sont fusionnées si elles portent le même nom ; la fusion échoue si leurs conditions diffèrent.

L'héritage de tables est généralement établi lors de la création de la table enfant, à l'aide de la clause `INHERITS` de l'instruction `CREATE TABLE`. Il est également possible d'ajouter une nouvelle relation parent-enfant à une table déjà définie de manière compatible, en utilisant la variante `INHERIT` de `ALTER TABLE`. Pour ce faire, la nouvelle table enfant doit déjà comporter des colonnes ayant les mêmes noms et types que celles de la table parente. Elle doit également comporter des contraintes de vérification ayant les mêmes noms et expressions que celles de la table parente. De même, un lien d'héritage peut être supprimé d'une table enfant à l'aide de la variante `NO INHERIT` de `ALTER TABLE`. L'ajout et la suppression dynamiques de liens d'héritage de cette manière peuvent s'avérer utiles lorsque la relation d'héritage est utilisée pour le partitionnement de tables (voir section 5.12).

Une méthode pratique pour créer une table compatible qui deviendra ultérieurement une table enfant consiste à utiliser la clause `LIKE` dans l'instruction `CREATE TABLE`. Cela crée une nouvelle table avec les mêmes colonnes que la table source.

S'il existe des contraintes `CHECK` définies sur la table source, l'option `INCLUDING CONSTRAINTS` de `LIKE` doit être spécifiée, car le nouvel enfant doit avoir des contraintes correspondant à celles du parent pour être considéré comme compatible.

Il est impossible de supprimer une table parente tant qu'au moins une de ses tables enfants existe. De même, les colonnes et les contraintes de vérification des tables enfants héritées d'une table parente ne peuvent être ni supprimées ni modifiées. Pour supprimer une table et toutes ses tables descendantes, une solution simple consiste à supprimer la table parente à l'aide de l'option `CASCADE` (voir section 5.15).

La commande `ALTER TABLE` propage les modifications apportées aux définitions de données des colonnes et aux contraintes de vérification dans la hiérarchie d'héritage. Il est possible de supprimer des colonnes dont dépendent d'autres tables uniquement avec l'option `CASCADE`. `ALTER TABLE` applique les mêmes règles de fusion et de rejet des colonnes dupliquées que `CREATE TABLE`.

Les requêtes héritées effectuent des contrôles d'autorisation d'accès uniquement sur la table parente. Ainsi, par exemple, accorder l'autorisation `UPDATE` sur la table « `citys` » implique également l'autorisation de mettre à jour les lignes de la table « `capitals` » lorsqu'elles sont consultées via « `citys` ». Cela donne l'impression que les données se trouvent (aussi) dans la table parente. Cependant, la table « `capitals` » ne pourrait pas être mise à jour directement sans une autorisation supplémentaire. De même, les politiques de sécurité des lignes de la table parente (voir section 5.9) sont appliquées aux lignes provenant des tables enfants lors d'une requête héritée. Les politiques d'une table enfant, le cas échéant, ne sont appliquées que si cette table est explicitement nommée dans la requête ; dans ce cas, toutes les politiques associées à ses tables parentes sont ignorées.

Les tables externes (voir section 5.13) peuvent également faire partie des hiérarchies d'héritage, en tant que tables parentes ou enfants, au même titre que les tables classiques. Si une table externe fait partie d'une hiérarchie d'héritage, les opérations non prises en charge par cette table externe ne le sont pas non plus pour l'ensemble de la hiérarchie.

5.11.1. Mises en garde

Notez que toutes les commandes SQL ne sont pas compatibles avec les hiérarchies d'héritage. Les commandes d'interrogation, de modification de données ou de modification de schéma (par exemple, `SELECT`, `UPDATE`, `DELETE`, la plupart des variantes de `ALTER TABLE`, mais pas `INSERT` ni `ALTER TABLE ... RENAME`) incluent généralement les tables enfants par défaut et permettent d'utiliser la notation `ONLY` pour les exclure. Les commandes de maintenance et d'optimisation de bases de données (par exemple, `REINDEX`, `VACUUM`) fonctionnent généralement uniquement sur des tables physiques individuelles et ne prennent pas en charge la récursivité sur les hiérarchies d'héritage. Le comportement de chaque commande est documenté dans sa page de référence (Commandes SQL).

Une limitation importante de la fonctionnalité d'héritage est que les index (y compris les contraintes d'unicité) et les contraintes de clé étrangère ne s'appliquent qu'aux tables individuelles, et non à leurs enfants. Ceci est valable aussi bien du côté référençant que du côté référencé d'une contrainte de clé étrangère. Ainsi, dans l'exemple ci-dessus :

Si nous déclarions ``citys.name`` comme UNIQUE ou comme CLÉ PRIMAIRE, cela n'empêcherait pas la table ``capitals`` de contenir des lignes avec des noms identiques à ceux de la table ``citys``. Ces lignes dupliquées apparaîtraient par défaut dans les requêtes portant sur ``citys``. En fait, par défaut, la table ``capitals`` ne serait soumise à aucune contrainte d'unicité et pourrait donc contenir plusieurs lignes portant le même nom. Il serait possible d'ajouter une contrainte d'unicité à ``capitals``, mais cela n'empêcherait pas les doublons par rapport à ``citys``.

De même, si l'on spécifie que `cities.name` fait référence à une autre table, cette contrainte ne se propage pas automatiquement à `capitals`. Dans ce cas, il est possible de contourner ce problème en ajoutant manuellement la même contrainte REFERENCES à `capitals`.

Spécifier que la colonne REFERENCES `cities(name)` d'une autre table permettrait à cette dernière de contenir des noms de villes, mais pas de capitales. Il n'existe pas de solution de contournement satisfaisante dans ce cas.

Certaines fonctionnalités non implémentées pour les hiérarchies d'héritage le sont pour le partitionnement déclaratif. Il convient d'être très prudent avant de décider si le partitionnement avec héritage existant est utile à votre application.

5.12. Partitionnement des tables

PostgreSQL prend en charge le partitionnement de base des tables. Cette section explique pourquoi et comment implémenter le partitionnement dans la conception de votre base de données.

5.12.1. Vue d'ensemble

Le partitionnement consiste à diviser ce qui constitue logiquement une seule grande table en parties physiques plus petites. Le partitionnement présente plusieurs avantages :

- Les performances des requêtes peuvent être considérablement améliorées dans certaines situations, notamment lorsque la plupart des lignes fréquemment consultées de la table se trouvent dans une seule partition ou dans un petit nombre de partitions.
Le partitionnement remplace efficacement les niveaux supérieurs de l'arbre des index, ce qui augmente la probabilité que les parties les plus utilisées des index tiennent en mémoire.
- Lorsque des requêtes ou des mises à jour accèdent à un pourcentage important d'une seule partition, les performances peuvent être améliorées en utilisant une analyse séquentielle de cette partition au lieu d'utiliser un index, ce qui nécessiterait des lectures à accès aléatoire dispersées dans toute la table.

Les opérations de chargement et de suppression en masse peuvent être réalisées en ajoutant ou en supprimant des partitions, à condition que le modèle d'utilisation soit pris en compte lors de la conception du partitionnement. La suppression d'une partition individuelle à l'aide de DROP TABLE ou d'ALTER TABLE DETACH PARTITION est beaucoup plus rapide qu'une opération en masse. Ces commandes évitent également totalement la surcharge liée à l'exécution de VACUUM induite par une suppression en masse.

- Les données rarement utilisées peuvent être transférées vers des supports de stockage moins chers et plus lents.

Ces avantages ne sont généralement intéressants que pour les tables de taille très importante. Le seuil précis à partir duquel le partitionnement devient bénéfique dépend de l'application, mais on considère généralement que la taille de la table doit dépasser la mémoire physique du serveur de base de données.

PostgreSQL offre une prise en charge intégrée des formes de partitionnement suivantes :

Partitionnement de la plage

Le tableau est partitionné en « plages » définies par une ou plusieurs colonnes clés, sans chevauchement entre les plages de valeurs attribuées aux différentes partitions. Par exemple, on peut partitionner par plages de dates ou par plages d'identifiants d'objets métier spécifiques. Les limites de chaque plage sont comprises entre 10 et 20. Par exemple, si la plage d'une partition va de 1 à 10 et celle de la partition suivante de 10 à 20, la valeur 10 appartient à la seconde partition et non à la première.

Partitionnement de liste

Le tableau est partitionné en listant explicitement quelle(s) valeur(s) clé(s) apparaît(nt) dans chaque partition.

Partitionnement par hachage

La table est partitionnée en spécifiant un module et un reste pour chaque partition. Chaque partition contient les lignes pour lesquelles la valeur de hachage de la clé de partition, divisée par le module spécifié, produit le reste spécifié.

Si votre application nécessite d'utiliser d'autres formes de partitionnement non mentionnées ci-dessus, des méthodes alternatives telles que l'héritage et les vues UNION ALL peuvent être employées. Ces méthodes offrent une grande flexibilité, mais ne bénéficient pas des mêmes avantages en termes de performances que le partitionnement déclaratif intégré.

5.12.2. Partitionnement déclaratif

PostgreSQL permet de déclarer qu'une table est divisée en partitions. La table ainsi divisée est appelée table partitionnée. La déclaration comprend la méthode de partitionnement décrite précédemment, ainsi qu'une liste de colonnes ou d'expressions servant de clé de partition.

La table partitionnée est une table « virtuelle » sans espace de stockage propre. Ce stockage est assuré par les partitions, qui sont des tables classiques associées à la table partitionnée. Chaque partition stocke un sous-ensemble des données, défini par ses limites. Toutes les lignes insérées dans une table partitionnée sont acheminées vers la partition appropriée en fonction des valeurs de la ou des colonnes de clé de partition.

La mise à jour de la clé de partition d'une ligne entraînera son déplacement vers une autre partition si elle ne respecte plus les limites de sa partition d'origine.

Les partitions peuvent être définies comme des tables partitionnées, ce qui permet de créer des sous-partitionnements. Bien que toutes les partitions doivent posséder les mêmes colonnes que leur table parente, elles peuvent avoir leurs propres index, contraintes et valeurs par défaut, distincts de ceux des autres partitions. Consultez la commande CREATE TABLE pour plus d'informations sur la création de tables partitionnées et de partitions.

Il n'est pas possible de convertir une table classique en table partitionnée, ni l'inverse. En revanche, il est possible d'ajouter une table classique ou partitionnée existante comme partition d'une table partitionnée, ou de supprimer une partition d'une table partitionnée pour la transformer en table autonome. Cette opération peut simplifier et accélérer de nombreuses opérations de maintenance. Consultez la documentation de la commande ALTER TABLE pour en savoir plus sur les sous-commandes ATTACH PARTITION et DETACH PARTITION.

Les partitions peuvent également être des tables externes, mais une grande prudence est de mise car il incombe alors à l'utilisateur de s'assurer que le contenu de la table externe respecte la règle de partitionnement. D'autres restrictions s'appliquent également. Consultez la documentation de CREATE FOREIGN TABLE pour plus d'informations.

5.12.2.1. Exemple

Supposons que nous construisions une base de données pour une grande entreprise de crèmes glacées. L'entreprise mesure quotidiennement les températures maximales ainsi que les ventes de crèmes glacées dans chaque région. Idéalement, nous souhaitons un tableau comme celui-ci :

```
CRÉER TABLE mesure ( int non nul, date non nul, int, int
    ville_id date de
    journalisation
    température de
    pointe unités de vente
);
```

Nous savons que la plupart des requêtes n'accéderont qu'aux données de la semaine, du mois ou du trimestre précédents, car cette table servira principalement à générer des rapports en ligne pour la direction. Afin de réduire la quantité de données anciennes à stocker, nous avons décidé de ne conserver que les données des trois dernières années.

Chaque mois, nous supprimerons les données du mois le plus ancien. Dans ce cas, le partitionnement nous permettra de répondre à tous nos besoins concernant le tableau des mesures.

Pour utiliser le partitionnement déclaratif dans ce cas, suivez les étapes suivantes :

1. Créez la table de mesure en tant que table partitionnée en spécifiant la clause `PARTITION BY`, qui comprend la méthode de partitionnement (`RANGE` dans ce cas) et la liste des colonnes à utiliser comme clé de partition.

```
CRÉER TABLE mesure ( int non nul, date non nul, int, int
    ville_id date de
    journalisation
    température de
    pointe unités de vente
```

1. Partitionnement par plage (`logdate`) ; 2. Création des partitions. La définition de chaque partition doit spécifier des limites correspondant à la méthode de partitionnement et à la clé de partition du parent. Attention : si les limites de la nouvelle partition chevauchent celles d'une ou plusieurs partitions existantes, une erreur se produira.

Les partitions ainsi créées sont à tous égards des tables PostgreSQL normales (ou, éventuellement, des tables externes). Il est possible de spécifier un espace de tables et des paramètres de stockage pour chaque partition séparément.

Dans notre exemple, chaque partition doit contenir un mois de données, afin de respecter l'exigence de suppression des données d'un mois à la fois. Les commandes pourraient donc ressembler à ceci :

```
CRÉER LA TABLE measurement_y2006m02 PARTITION DE measurement
    POUR LES VALEURS DE ('2006-02-01') À ('2006-03-01');
```

```
CRÉER LA TABLE measurement_y2006m03 PARTITION DE measurement
    POUR LES VALEURS DE ('2006-03-01') À ('2006-04-01');
```

...

```
CRÉER LA TABLE measurement_y2007m11 PARTITION DE measurement
    POUR LES VALEURS DE ('2007-11-01') À ('2007-12-01');
```

```
CRÉER LA TABLE measurement_y2007m12 PARTITION DE measurement
    POUR LES VALEURS DE ('2007-12-01') À ('2008-01-01')
    TABLESPACE fasttablespace;
```

```
CRÉER LA TABLE measurement_y2008m01 PARTITION DE measurement
    POUR LES VALEURS DE ('2008-01-01') À ('2008-02-01')
    AVEC (travailleurs_parallèles = 4)
    TABLESPACE fasttablespace;
```

(Rappelons que les partitions adjacentes peuvent partager une valeur limite, puisque les limites supérieures d'un intervalle sont traitées comme des limites exclusives.)

Si vous souhaitez implémenter un sous-partitionnement, spécifiez à nouveau la clause `PARTITION BY` dans les commandes utilisées pour créer les partitions individuelles, par exemple :

```
CRÉER LA TABLE measurement_y2006m02 PARTITION DE measurement
    POUR LES VALEURS DE ('2006-02-01') À ('2006-03-01')
    PARTITION PAR PLAGE (température de pointe) ;
```

Après la création des partitions de `measurement_y2006m02`, toutes les données insérées dans `measurement` et mappées sur `measurement_y2006m02` (ou les données insérées directement dans `measurement_y2006m02`, ce qui est autorisé si sa contrainte de partitionnement est respectée) seront ensuite traitées.

Les données sont redirigées vers l'une de ses partitions en fonction de la colonne `peaktemp`. La clé de partition spécifiée peut être identique à celle de la partition parente. Toutefois, il convient d'être vigilant lors de la définition des limites d'une sous-partition afin que l'ensemble des données qu'elle accepte ne soit qu'un sous-ensemble de ce que les limites de la partition parente autorisent ; le système ne vérifie pas si c'est effectivement le cas.

L'insertion de données dans la table parente qui ne correspondent à aucune des partitions existantes provoquera une erreur ; une partition appropriée doit être ajoutée manuellement.

Il n'est pas nécessaire de créer manuellement les contraintes de table décrivant les conditions limites des partitions. Ces contraintes seront créées automatiquement.

3. Créez un index sur la ou les colonnes clés, ainsi que tout autre index nécessaire, sur la table partitionnée. (L'index clé n'est pas strictement indispensable, mais il est généralement utile.) Cela crée automatiquement un index correspondant sur chaque partition, et toute partition créée ou ajoutée ultérieurement disposera également de cet index. Un index ou une contrainte d'unicité déclarée sur une table partitionnée est « virtuel » au même titre que la table partitionnée elle-même : les données sont stockées dans les index enfants des tables de partition individuelles.

CRÉER UN INDEX SUR la mesure (`logdate`);

4. Assurez-vous que le paramètre de configuration `enable_partition_pruning` n'est pas désactivé dans la version post-installation. `gresql.conf`. Si c'est le cas, les requêtes ne seront pas optimisées comme prévu.

Dans l'exemple ci-dessus, nous créerions une nouvelle partition chaque mois ; il serait donc judicieux d'écrire un script qui génère automatiquement le DDL requis.

5.12.2.2. Maintenance des partitions

En règle générale, l'ensemble des partitions défini lors de la création initiale d'une table n'est pas figé. Il est fréquent de vouloir supprimer les partitions contenant d'anciennes données et d'en ajouter régulièrement de nouvelles pour les nouvelles données. L'un des principaux avantages du partitionnement est précisément qu'il permet d'effectuer cette tâche, autrement fastidieuse, de manière quasi instantanée en manipulant la structure des partitions, plutôt qu'en déplaçant physiquement d'importantes quantités de données.

L'option la plus simple pour supprimer les anciennes données consiste à supprimer la partition qui n'est plus nécessaire :

```
SUPPRIMER LA TABLE measurement_y2006m02;
```

Cette méthode permet de supprimer très rapidement des millions d'enregistrements, car elle n'a pas besoin de les supprimer individuellement. Notez toutefois que la commande ci-dessus nécessite l'acquisition d'un verrou d'accès exclusif sur la table parente.

Une autre option, souvent préférable, consiste à supprimer la partition de la table partitionnée tout en conservant l'accès à celle-ci en tant que table à part entière. Cela se présente sous deux formes :

```
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02;
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02
    EN MÊME TEMPS;
```

Ces commandes permettent d'effectuer des opérations supplémentaires sur les données avant leur suppression. Par exemple, il est souvent judicieux de sauvegarder les données à l'aide de `COPY`, `pg_dump` ou d'outils similaires. Il peut également être utile de regrouper les données dans des formats plus petits, d'effectuer d'autres manipulations de données ou de générer des rapports. La première forme de la commande requiert un verrou `ACCESS EXCLUSIVE` sur la table parente. L'ajout du qualificateur `CONCURRENTLY`, comme dans la seconde forme, permet à l'opération de détachement de ne nécessiter qu'un verrou `SHARE UPDATE EXCLUSIVE` sur la table parente ; toutefois, consultez `ALTER TABLE ... DETACH PARTITION` pour plus de détails sur les restrictions.

De même, nous pouvons ajouter une nouvelle partition pour gérer de nouvelles données. Nous pouvons créer une partition vide dans la table partitionnée, tout comme les partitions d'origine ont été créées ci-dessus :

```
CRÉER LA TABLE measurement_y2008m02 PARTITION DE measurement
POUR LES VALEURS DE ('2008-02-01') À ('2008-03-01')
TABLESPACE fasttablespace;
```

Au lieu de créer une nouvelle partition, il est parfois plus pratique de créer une table distincte de la structure de partition et de l'y attacher ultérieurement. Cela permet de charger, vérifier et transformer les nouvelles données avant leur insertion dans la table partitionnée. De plus, l'opération ATTACH PARTITION ne requiert qu'un verrou SHARE UPDATE EXCLUSIVE sur la table partitionnée, contrairement au verrou ACCESS EXCLUSIVE nécessaire pour CREATE TABLE ... PARTITION OF. Elle est donc plus compatible avec les opérations simultanées sur la table partitionnée ; consultez ALTER TABLE ATTACH PARTITION pour plus de détails. L'option CREATE TABLE LIKE peut s'avérer utile pour éviter de répéter inutilement la définition de la table parente ; par exemple :

```
CRÉER LA TABLE measurement_y2008m02
(COMME MESURE, Y COMPRIS LES VALEURS PAR DÉFAUT ET LES CONTRAINTES)
TABLESPACE fasttablespace;
```

```
ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
VÉRIFIER (logdate >= DATE '2008-02-01' ET logdate < DATE
'2008-03-01');
```

```
\copy measurement_y2008m02 from 'measurement_y2008m02' -- éventuellement d'autres travaux
de préparation des données
```

```
ALTER TABLE measurement ATTACH PARTITION measurement_y2008m02
POUR LES VALEURS DE ('2008-02-01') À ('2008-03-01');
```

Notez que lors de l'exécution de la commande ATTACH PARTITION, la table est analysée afin de valider la contrainte de partition, un verrou ACCESS EXCLUSIVE étant maintenu sur cette partition. Comme indiqué précédemment, il est recommandé d'éviter cette analyse en créant une contrainte CHECK correspondant à la contrainte de partition attendue sur la table avant de l'attacher. Une fois l'opération ATTACH PARTITION terminée, il est recommandé de supprimer la contrainte CHECK, devenue inutile. Si la table attachée est elle-même partitionnée, chacune de ses sous-partitions est verrouillée et analysée de manière récursive jusqu'à ce qu'une contrainte CHECK appropriée soit rencontrée ou que les partitions feuilles soient atteintes.

De même, si la table partitionnée possède une partition DEFAULT, il est recommandé de créer une contrainte CHECK excluant la contrainte de la partition à attacher. À défaut, la partition DEFAULT sera analysée afin de vérifier qu'elle ne contient aucun enregistrement devant se trouver dans la partition à attacher. Cette opération sera effectuée avec un verrou ACCESS EXCLUSIVE sur la partition DEFAULT. Si la partition DEFAULT est elle-même une table partitionnée, chacune de ses partitions sera vérifiée récursivement de la même manière que la table à attacher, comme indiqué précédemment.

Comme mentionné précédemment, il est possible de créer des index sur des tables partitionnées afin qu'ils soient appliqués automatiquement à l'ensemble de la hiérarchie. Ceci est très pratique car non seulement toutes les partitions existantes seront indexées, mais également les futures partitions. Cependant, la création d'index sur des tables partitionnées présente une limitation : l'utilisation du qualificateur CONCURRENTLY est impossible, ce qui peut entraîner des temps de verrouillage longs. Pour éviter cela, vous pouvez utiliser CREATE INDEX ON ONLY sur la table partitionnée. Cette commande crée le nouvel index en le marquant comme invalide, empêchant ainsi son application automatique aux partitions existantes. Vous pouvez ensuite créer des index individuellement sur chaque partition en utilisant CONCURRENTLY et les rattacher à l'index de la table parente à l'aide de ALTER INDEX ... ATTACH PARTITION. Une fois les index de toutes les partitions rattachés à l'index parent, ce dernier sera automatiquement validé. Exemple :

```
CRÉER UN INDEX measurement_usls_idx SUR UNIQUEMENT measurement (unitsales);
```

```
CRÉER UN INDEX EN CONCURRENCE measurement_usls_200602_idx
```

```

ON mesure_y2006m02 (unités vendues);
ALTER INDEX measurement_usls_idx
ATTACHER LA PARTITION measurement_usls_200602_idx;
...

```

Cette technique peut également être utilisée avec les contraintes UNIQUE et PRIMARY KEY ; les index sont créés implicitement lors de la création de la contrainte. Exemple :

```

ALTER TABLE ONLY measurement ADD UNIQUE (city_id, logdate);

ALTER TABLE measurement_y2006m02 ADD UNIQUE (city_id, logdate);
ALTER INDEX measurement_city_id_logdate_key
ATTACHER LA PARTITION measurement_y2006m02_city_id_logdate_key;
...

```

5.12.2.3. Limitations

Les limitations suivantes s'appliquent aux tables partitionnées :

Pour créer une contrainte d'unicité ou de clé primaire sur une table partitionnée, les clés de partition ne doivent contenir aucune expression ni appel de fonction, et les colonnes de la contrainte doivent inclure toutes les colonnes de la clé de partition. Cette limitation s'explique par le fait que les index individuels composant la contrainte ne peuvent garantir l'unicité qu'au sein de leurs propres partitions ; par conséquent, la structure de partition elle-même doit garantir l'absence de doublons entre les différentes partitions.

De même, une contrainte d'exclusion doit inclure toutes les colonnes de la clé de partition. De plus, elle doit comparer l'égalité de ces colonnes (et non utiliser l'opérateur &&). Cette limitation découle de l'impossibilité d'appliquer des restrictions inter-partitions. La contrainte peut inclure des colonnes supplémentaires ne faisant pas partie de la clé de partition et les comparer avec les opérateurs de votre choix.

- Les déclencheurs BEFORE ROW sur INSERT ne peuvent pas modifier la partition de destination finale d'une instruction.
nouvelle rangée.

- Il est interdit de mélanger relations temporaires et permanentes dans la même arborescence de partitions. Par conséquent, si la table partitionnée est permanente, ses partitions doivent l'être également, et inversement. Lors de l'utilisation de relations temporaires, tous les membres de l'arborescence de partitions doivent appartenir à la même session.

Les partitions individuelles sont liées à leur table partitionnée par héritage, en interne. Cependant, il n'est pas possible d'utiliser toutes les fonctionnalités génériques de l'héritage avec les tables partitionnées de manière déclarative ni avec leurs partitions, comme expliqué ci-dessous. Notamment, une partition ne peut avoir d'autre parent que la table partitionnée dont elle fait partie, et une table ne peut hériter à la fois d'une table partitionnée et d'une table classique. Cela signifie que les tables partitionnées et leurs partitions ne partagent jamais de hiérarchie d'héritage avec les tables classiques.

Étant donné qu'une hiérarchie de partitions constituée de la table partitionnée et de ses partitions constitue toujours une hiérarchie d'héritage, tableoid et toutes les règles normales d'héritage s'appliquent comme décrit dans la section 5.11, à quelques exceptions près :

Les partitions ne peuvent pas contenir de colonnes absentes de la table parente. Il est impossible de spécifier des colonnes lors de la création de partitions avec CREATE TABLE, ni d'en ajouter ultérieurement avec ALTER TABLE. Les tables peuvent être ajoutées comme partition avec ALTER TABLE ATTACH PARTITION ... uniquement si leurs colonnes correspondent exactement à celles de la table parente.

Les contraintes CHECK et NOT NULL d'une table partitionnée sont toujours héritées par toutes ses partitions. Il est interdit de créer des contraintes CHECK marquées NO INHERIT sur les tables partitionnées. Vous ne pouvez pas supprimer une contrainte NOT NULL sur une colonne d'une partition si la même contrainte est présente dans la table parente.

L'utilisation de ONLY pour ajouter ou supprimer une contrainte uniquement sur la table partitionnée est prise en charge tant qu'il n'existe aucune partition. Dès lors que des partitions sont présentes, l'utilisation de ONLY générera une erreur pour toute contrainte autre que UNIQUE et PRIMARY KEY. Il est possible d'ajouter des contraintes sur les partitions elles-mêmes et de les supprimer (si elles ne sont pas présentes dans la table parente).

- Comme une table partitionnée ne contient aucune donnée en elle-même, les tentatives d'utilisation de TRUNCATE ONLY sur une partition échouent. Le tableau modifié renverra toujours une erreur.

5.12.3. Partitionnement par héritage

Bien que le partitionnement déclaratif intégré convienne à la plupart des cas d'utilisation courants, une approche plus flexible peut s'avérer utile dans certaines circonstances. Le partitionnement peut être implémenté à l'aide de l'héritage de tables, ce qui permet plusieurs fonctionnalités non prises en charge par le partitionnement déclaratif, telles que :

- Pour le partitionnement déclaratif, les partitions doivent avoir exactement le même ensemble de colonnes que la table partitionnée, tandis qu'avec l'héritage de table, les tables enfants peuvent avoir des colonnes supplémentaires absentes de la table parente.
- L'héritage de table permet l'héritage multiple.

Le partitionnement déclaratif ne prend en charge que le partitionnement par plage, par liste et par hachage, tandis que l'héritage de tables permet de diviser les données selon les préférences de l'utilisateur. (Notez toutefois que si l'exclusion de contraintes ne permet pas d'élaguer efficacement les tables enfants, les performances des requêtes peuvent être médiocres.)

5.12.3.1. Exemple

Cet exemple construit une structure de partitionnement équivalente à l'exemple de partitionnement déclaratif ci-dessus. Suivez les étapes suivantes :

1. Créez la table « racine », dont hériteront toutes les tables « enfants ». Cette table sera vide. Ne définissez aucune contrainte de vérification sur cette table, sauf si vous souhaitez qu'elles s'appliquent à toutes les tables enfants. Il est également inutile d'y définir des index ou des contraintes d'unicité. Dans notre exemple, la table racine est la table de mesures telle que définie initialement :

```
CRÉER TABLE mesure ( int non nul, date non nul, int, int
    ville_id date de
    journalisation
    température de
    pointe unités de vente
);
```

2. Créez plusieurs tables « enfants » qui héritent chacune de la table racine. Normalement, ces tables n'ajoutent aucune colonne à l'ensemble hérité de la racine. Tout comme pour le partitionnement déclaratif, ces tables sont des tables PostgreSQL (ou tables externes) tout à fait classiques.

```
CRÉER TABLE measurement_y2006m02 () HÉRITE (measurement);
CRÉER TABLE measurement_y2006m03 () HÉRITE (measurement);
...
```

```
CREATE TABLE measurement_y2007m11 () INHERITS (measurement); CREATE TABLE
measurement_y2007m12 () INHERITS (measurement); CREATE TABLE measurement_y2008m01 ()
INHERITS (measurement);
```

3. Ajoutez des contraintes de table non chevauchantes aux tables enfants pour définir les valeurs de clé autorisées dans chacune.

Des exemples typiques seraient :

```
VÉRIFIER ( x = 1 )
VÉRIFIER (comté DANS ( 'Oxfordshire', 'Buckinghamshire',
    'Warwickshire' ))
VÉRIFIER ( outletID >= 100 ET outletID < 200 )
```

Assurez-vous que les contraintes garantissent l'absence de chevauchement entre les valeurs clés autorisées dans les différentes tables enfants. Une erreur fréquente consiste à définir des contraintes de plage comme :

```
VÉRIFIER (ID de point de vente ENTRE 100 ET 200)
```

```
VÉRIFIER (ID de point de vente ENTRE 200 ET 300)
```

Ceci est incorrect car il n'est pas clair à quelle table enfant appartient la valeur clé 200. Il convient plutôt de définir les plages comme suit :

```
CRÉER TABLE mesure_y2006m02 (
    VÉRIFIER ( logdate >= DATE '2006-02-01' ET logdate < DATE '2006-03-01' )
```

```
) HÉRITE (mesure);
```

```
CRÉER TABLE mesure_y2006m03 (
    VÉRIFIER ( logdate >= DATE '2006-03-01' ET logdate < DATE '2006-04-01' )
```

```
) HÉRITE (mesure);
```

...

```
CRÉER TABLE mesure_y2007m11 (
    VÉRIFIER ( logdate >= DATE '2007-11-01' ET logdate < DATE '2007-12-01' )
```

```
) HÉRITE (mesure);
```

```
CRÉER TABLE mesure_y2007m12 (
    VÉRIFIER ( logdate >= DATE '2007-12-01' ET logdate < DATE '2008-01-01' )
```

```
) HÉRITE (mesure);
```

```
CRÉER TABLE mesure_y2008m01 (
    VÉRIFIER ( logdate >= DATE '2008-01-01' ET logdate < DATE '2008-02-01' )
```

```
) HÉRITE (mesure);
```

4. Pour chaque table enfant, créez un index sur la ou les colonnes clés, ainsi que tous les autres index nécessaires. vouloir.

```
CRÉER UN INDEX measurement_y2006m02_logdate SUR measurement_y2006m02
(logdate);
```

```
CRÉER UN INDEX measurement_y2006m03_logdate SUR measurement_y2006m03
(logdate);
```

```
CRÉER UN INDEX measurement_y2007m11_logdate SUR measurement_y2007m11
(logdate);
```

```
CRÉER UN INDEX measurement_y2007m12_logdate SUR measurement_y2007m12
(logdate);
```

```
CRÉER UN INDEX measurement_y2008m01_logdate SUR measurement_y2008m01
(logdate);
```

5. Nous souhaitons que notre application puisse exécuter la commande INSERT INTO measurement... et que les données soient redirigées vers la table enfant appropriée. Pour ce faire, nous pouvons associer une fonction de déclenchement adaptée à la table racine. Si les données ne doivent être ajoutées qu'à la dernière table enfant, une fonction de déclenchement très simple suffit :

```
CRÉER OU REMPLACER LA FONCTION measurement_insert_trigger()
DÉCLENCHÉUR DE RETOUR EN $$
```

COMMENCER

```
INSÉRER DANS measurement_y2008m01 VALUES (NEW.*); RETOURNER NULL;
```

FIN; \$

\$

LANGAGE plpgsql;

Après avoir créé la fonction, nous créons un déclencheur qui appelle la fonction de déclenchement :

CRÉER UN DÉCLENCHEUR insérer_déclencheur_de_mesure

AVANT L'INSERTION SUR la mesure

POUR CHAQUE LIGNE EXÉCUTER LA FONCTION measurement_insert_trigger();

Nous devons redéfinir la fonction de déclenchement chaque mois afin qu'elle insère toujours les données dans la table enfant courante. La définition du déclencheur, quant à elle, n'a pas besoin d'être mise à jour.

Nous pourrions vouloir insérer des données et laisser le serveur localiser automatiquement la table enfant dans laquelle la ligne doit être ajoutée. Nous pourrions le faire avec une fonction de déclenchement plus complexe, par exemple :

CRÉER OU REMPLACER LA FONCTION measurement_insert_trigger()

DÉCLENCHEUR DE RETOUR EN \$\$

COMMENCER

```
SI ( NEW.logdate >= DATE '2006-02-01' ET
```

```
NEW.logdate < DATE '2006-03-01' ) ALORS
```

```
INSÉRER DANS measurement_y2006m02 VALUES (NOUVEAU.*);
```

```
SINON SI ( NEW.logdate >= DATE '2006-03-01' ET
```

```
NEW.logdate < DATE '2006-04-01' ) ALORS
```

```
INSÉRER DANS measurement_y2006m03 VALUES (NOUVEAU.*);
```

```
...
```

```
SINON SI ( NEW.logdate >= DATE '2008-01-01' ET
```

```
NEW.logdate < DATE '2008-02-01' ) ALORS
```

```
INSÉRER DANS measurement_y2008m01 VALUES (NEW.*); SINON
```

```
LEVER UNE EXCEPTION 'Date hors plage. Corrigez la fonction
```

```
measurement_insert_trigger() !'; FIN SI; RETOURNER NULL;
```

FIN; \$

\$

LANGAGE plpgsql;

La définition du déclencheur reste inchangée. Notez que chaque test IF doit correspondre exactement à la contrainte CHECK de sa table enfant.

Bien que cette fonction soit plus complexe que dans le cas d'un seul mois, elle n'a pas besoin d'être mise à jour aussi souvent, car les branches peuvent être ajoutées à l'avance.

Note

En pratique, il peut être préférable de vérifier d'abord le plus récent enfant, si la plupart des insertions concernent cet enfant. Par souci de simplicité, nous avons présenté les tests du déclencheur dans le même ordre que dans les autres parties de cet exemple.

Une autre approche pour rediriger les insertions vers la table enfant appropriée consiste à définir des règles, plutôt qu'un déclencheur, sur la table racine. Par exemple :

```
CRÉER UNE RÈGLE measurement_insert_y2006m02 COMME
INSERER À LA MESURE OÙ
    ( logdate >= DATE '2006-02-01' ET logdate < DATE '2006-03-01' )
```

FAITES À LA PLACE

```
    INSÉRER DANS measurement_y2006m02 VALUES (NOUVEAU.*);
```

...

```
CRÉER UNE RÈGLE measurement_insert_y2008m01 COMME
INSERER À LA MESURE OÙ
    ( logdate >= DATE '2008-01-01' ET logdate < DATE '2008-02-01' )
```

FAITES À LA PLACE

```
    INSÉRER DANS measurement_y2008m01 VALUES (NOUVEAU.*);
```

Une règle engendre une surcharge nettement supérieure à celle d'un déclencheur, mais cette surcharge n'est supportée qu'une seule fois par requête et non une seule fois par ligne. Cette méthode peut donc s'avérer avantageuse pour les insertions en masse. Dans la plupart des cas, cependant, la méthode par déclencheur offrira de meilleures performances.

Notez que la commande COPY ignore les règles. Si vous souhaitez l'utiliser pour insérer des données, vous devrez les copier dans la table enfant appropriée et non directement dans la table racine. La commande COPY déclenche des événements ; vous pouvez donc l'utiliser normalement avec cette approche.

Un autre inconvénient de l'approche par règles est qu'il n'existe aucun moyen simple de provoquer une erreur si l'ensemble de règles ne couvre pas la date d'insertion ; les données seront alors insérées silencieusement dans la table racine.

- Assurez-vous que le paramètre de configuration `constraint_exclusion` n'est pas désactivé dans `post-gresql.conf` ; sinon, des tables enfants pourraient être accédées inutilement.

Comme on peut le constater, une hiérarchie de tables complexe peut nécessiter une quantité importante de DDL. Dans l'exemple ci-dessus, nous créerions une nouvelle table enfant chaque mois ; il serait donc judicieux d'écrire un script qui génère automatiquement le DDL nécessaire.

5.12.3.2. Maintenance pour le partage d'héritage

Pour supprimer rapidement les anciennes données, il suffit de supprimer la table enfant devenue inutile :

```
SUPPRIMER LA TABLE measurement_y2006m02;
```

Pour supprimer la table enfant de la table de hiérarchie d'héritage tout en conservant l'accès à celle-ci en tant que table à part entière :

```
ALTER TABLE measurement_y2006m02 NO INHERIT measurement;
```

Pour ajouter une nouvelle table enfant afin de gérer de nouvelles données, créez une table enfant vide, de la même manière que les tables enfants d'origine ont été créées ci-dessus :

```
CRÉER TABLE mesure_y2008m02 (
    VÉRIFIER ( logdate >= DATE '2008-02-01' ET logdate < DATE '2008-03-01' )
```

```
) HÉRITE (mesure);
```

Il est également possible de créer et de remplir la nouvelle table enfant avant de l'ajouter à la hiérarchie des tables. Cela permettrait de charger, de vérifier et de transformer les données avant de les rendre accessibles aux requêtes sur la table parente.

```
CRÉER LA TABLE measurement_y2008m02
```

```
(COMME MESURE Y COMPRIS LES VALEURS PAR DÉFAUT Y COMPRIS LES CONTRAINTES) ;
ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
  VÉRIFIER (logdate >= DATE '2008-02-01' ET logdate < DATE
    '2008-03-01');
\copy measurement_y2008m02 from 'measurement_y2008m02' -- éventuellement d'autres travaux
de préparation des données
ALTER TABLE measurement_y2008m02 INHERIT measurement;
```

5.12.3.3. Mises en garde

Les mises en garde suivantes s'appliquent au partitionnement implémenté à l'aide de l'héritage :

Il n'existe aucun moyen automatique de vérifier que toutes les contraintes CHECK sont mutuellement exclusives. Il est plus sûr de créer un code qui génère des tables enfants et crée et/ou modifie les objets associés plutôt que de les écrire manuellement.

- Les index et les contraintes de clé étrangère s'appliquent aux tables individuelles et non à leurs tables enfants héritées, par conséquent Il y a toutefois quelques mises en garde à prendre en compte.

Les schémas présentés ici supposent que les valeurs des colonnes clés d'une ligne ne changent jamais, ou du moins pas suffisamment pour nécessiter son déplacement vers une autre partition. Une mise à jour tentant de le faire échouera en raison des contraintes de vérification. Si vous devez gérer de tels cas, vous pouvez ajouter des déclencheurs de mise à jour appropriés aux tables enfants, mais cela complexifie considérablement la gestion de la structure.

- Si vous utilisez des commandes VACUUM ou ANALYZE manuelles, n'oubliez pas que vous devez les exécuter sur chaque table enfant individuellement. Une commande comme :

```
ANALYSER la mesure ;
```

ne traitera que la table racine.

- Les instructions INSERT avec clauses ON CONFLICT sont peu susceptibles de fonctionner comme prévu, car l'action ON CONFLICT n'est entreprise qu'en cas de violations uniques sur la relation cible spécifiée, et non sur ses relations enfants.

Des déclencheurs ou des règles seront nécessaires pour acheminer les lignes vers la table enfant souhaitée, sauf si l'application connaît explicitement le schéma de partitionnement. La mise en œuvre de ces déclencheurs peut s'avérer complexe et sera beaucoup plus lente que le routage des tuples effectué en interne par le partitionnement déclaratif.

5.12.4. Élagage par partition

L'élagage des partitions est une technique d'optimisation des requêtes qui améliore les performances des tables partitionnées de manière déclarative. Par exemple :

```
DÉFINIR enable_partition_pruning = activé ; -- la valeur par défaut
SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
```

Sans élagage des partitions, la requête ci-dessus analyserait chacune des partitions de la table de mesures. Avec l'élagage des partitions activé, le planificateur examine la définition de chaque partition et vérifie qu'il est inutile de l'analyser, car elle ne contient aucune ligne correspondant à la clause WHERE de la requête. Le cas échéant, il exclut (élague) la partition du plan d'exécution.

La commande EXPLAIN, associée au paramètre de configuration enable_partition_pruning, permet de comparer un plan avec et sans élagage des partitions. Voici un exemple de plan non optimisé pour ce type de configuration de table :

```
DÉFINIR enable_partition_pruning = désactivé ;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE
'2008-01-01';
```

PLAN DE REQUÊTE

```
-----
-----
Agrégat (coût=188,76..188,77 lignes=1 largeur=8)
  -> Ajouter (coût=0.00..181.05 lignes=3085 largeur=0)
    -> Analyse séquentielle sur measurement_y2006m02 (coût=0.00..33.12 lignes=617 largeur=0)
      Filtre : (logdate >= '2008-01-01'::date)
    -> Analyse séquentielle sur measurement_y2006m03 (coût=0.00..33.12 lignes=617 largeur=0)
      Filtre : (logdate >= '2008-01-01'::date)
  ...
    -> Analyse séquentielle sur measurement_y2007m11 (coût=0.00..33.12 lignes=617 largeur=0)
      Filtre : (logdate >= '2008-01-01'::date)
    -> Analyse séquentielle sur measurement_y2007m12 (coût=0.00..33.12 lignes=617 largeur=0)
      Filtre : (logdate >= '2008-01-01'::date)
    -> Analyse séquentielle sur measurement_y2008m01 (coût=0.00..33.12 lignes=617 largeur=0)
      Filtre : (logdate >= '2008-01-01'::date)
```

Certaines partitions, voire toutes, peuvent utiliser des analyses d'index au lieu d'analyses séquentielles complètes de la table. L'important est qu'il est inutile d'analyser les partitions les plus anciennes pour répondre à cette requête. En activant l'élégage des partitions, on obtient un plan d'exécution nettement plus performant qui fournit la même réponse.

```
DÉFINIR enable_partition_pruning = activé ;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE
'2008-01-01';
```

PLAN DE REQUÊTE

```
-----
-----
Agrégat (coût=37,75..37,76 lignes=1 largeur=8)
  -> Analyse séquentielle sur measurement_y2008m01 (coût=0.00..33.12 lignes=617 largeur=0)
    Filtre : (logdate >= '2008-01-01'::date)
```

Il est important de noter que l'élégage des partitions est uniquement déterminé par les contraintes implicites des clés de partition, et non par la présence d'index. Par conséquent, il n'est pas nécessaire de définir des index sur les colonnes clés. La nécessité de créer un index pour une partition donnée dépend de la taille des requêtes qui analysent cette partition : une grande partie ou une petite. Un index sera utile dans le second cas, mais pas dans le premier.

L'élégage des partitions peut être effectué non seulement lors de la planification d'une requête, mais aussi lors de son exécution. Ceci est utile car cela permet d'élaguer davantage de partitions lorsque les clauses contiennent des expressions dont les valeurs sont inconnues lors de la planification de la requête, par exemple, des paramètres définis dans une instruction PREPARE, utilisant une valeur obtenue d'une sous-requête ou utilisant une valeur paramétrée dans une jointure par boucle imbriquée. L'élégage des partitions lors de l'exécution peut être effectué à l'un des moments suivants :

- Lors de l'initialisation du plan de requête, l'élégage des partitions peut être effectué pour les valeurs de paramètres connues durant cette phase. Les partitions élaguées à cette étape n'apparaissent pas dans les résultats EXPLAIN et EXPLAIN ANALYZE de la requête. Le nombre de partitions supprimées est visible dans la propriété « Sous-plans supprimés » du résultat EXPLAIN. Attention : les partitions supprimées lors de cette étape restent verrouillées au début de l'exécution.

- Lors de l'exécution du plan de requête, l'élagage des partitions peut être effectué afin de supprimer celles dont les valeurs ne sont connues que pendant l'exécution. Il s'agit notamment des valeurs issues des sous-requêtes et des paramètres d'exécution, comme ceux des jointures par boucles imbriquées paramétrées. La valeur de ces paramètres pouvant varier, l'élagage est effectué dès qu'un paramètre d'exécution utilisé par cette opération change. Pour déterminer si des partitions ont été élaguées durant cette phase, il est nécessaire d'examiner attentivement la propriété « loops » dans la sortie de EXPLAIN ANALYZE. Les sous-plans correspondant à différentes partitions peuvent présenter des valeurs différentes selon le nombre d'élagages effectués. Certains peuvent apparaître comme « jamais exécutés » s'ils ont été élagués systématiquement.

L'élagage des partitions peut être désactivé à l'aide du paramètre `enable_partition_pruning`.

5.12.5. Partitionnement et exclusion des contraintes

L'exclusion de contraintes est une technique d'optimisation de requêtes similaire à l'élagage de partitions. Bien qu'elle soit principalement utilisée pour le partitionnement implémenté à l'aide de la méthode d'héritage traditionnelle, elle peut servir à d'autres fins, notamment avec le partitionnement déclaratif.

L'exclusion de contraintes fonctionne de manière très similaire à l'élagage de partitions, à ceci près qu'elle utilise les contraintes CHECK de chaque table (d'où son nom), tandis que l'élagage de partitions utilise les limites de partition, qui n'existent que dans le cas d'un partitionnement déclaratif. Autre différence : l'exclusion de contraintes s'applique uniquement lors de la planification ; aucune tentative de suppression de partitions n'est effectuée lors de l'exécution.

Le fait que l'exclusion de contraintes utilise des contraintes CHECK, ce qui la rend lente par rapport à l'élagage de partitions, peut parfois être utilisé comme un avantage : étant donné que des contraintes peuvent être définies même sur des tables partitionnées de manière déclarative, en plus de leurs limites de partition internes, l'exclusion de contraintes peut permettre d'éliminer des partitions supplémentaires du plan de requête.

Le paramètre par défaut (et recommandé) de `constraint_exclusion` n'est ni activé ni désactivé, mais un paramètre intermédiaire appelé `partition`. Ce paramètre limite l'application de la technique aux seules requêtes susceptibles d'opérer sur des tables partitionnées par héritage. L'activation de `constraint_exclusion` force le planificateur à examiner les contraintes CHECK dans toutes les requêtes, même les plus simples qui n'en tireraient probablement aucun avantage.

Les mises en garde suivantes s'appliquent à l'exclusion des contraintes :

- L'exclusion des contraintes n'est appliquée que lors de la planification des requêtes, contrairement à l'élagage des partitions, qui peut également être appliqué lors de l'exécution de la requête.

L'exclusion de contraintes ne fonctionne que lorsque la clause WHERE de la requête contient des constantes (ou des paramètres externes). Par exemple, une comparaison avec une fonction non immuable telle que `CUR-RENT_TIMESTAMP` ne peut pas être optimisée, car le planificateur ne peut pas savoir dans quelle table enfant la valeur de la fonction sera stockée lors de l'exécution.

- Veillez à la simplicité des contraintes de partitionnement, sans quoi le planificateur risque de ne pas pouvoir démontrer qu'il n'est pas nécessaire de visiter les tables enfants. Utilisez des conditions d'égalité simples pour le partitionnement par liste, ou des tests d'intervalle simples pour le partitionnement par intervalle, comme illustré dans les exemples précédents. En règle générale, les contraintes de partitionnement ne doivent contenir que des comparaisons des colonnes de partitionnement à des constantes, à l'aide d'opérateurs indexables par arbre B, car seules les colonnes indexables par arbre B sont autorisées dans la clé de partitionnement.

Lors de l'exclusion de contraintes, toutes les contraintes de toutes les tables enfants de la table parente sont examinées. Par conséquent, un grand nombre de tables enfants risque d'augmenter considérablement le temps de planification des requêtes. Le partitionnement traditionnel basé sur l'héritage fonctionne donc correctement avec une centaine de tables enfants maximum ; il est déconseillé d'en utiliser plusieurs milliers.

5.12.6. Bonnes pratiques pour le partitionnement déclaratif

Le choix du mode de partitionnement d'une table doit être effectué avec soin, car une mauvaise conception peut avoir un impact négatif sur les performances de la planification et de l'exécution des requêtes.

L'une des décisions de conception les plus cruciales concerne le choix de la ou des colonnes de partitionnement. Souvent, il est préférable de partitionner selon la ou les colonnes les plus fréquemment utilisées dans les clauses WHERE des requêtes exécutées sur la table partitionnée. Les clauses WHERE compatibles avec les contraintes de partitionnement permettent d'éliminer les partitions inutiles. Toutefois, les exigences liées à la clé primaire ou à une contrainte UNIQUE peuvent vous contraindre à faire d'autres choix. La suppression des données indésirables est également un facteur important à prendre en compte lors de la planification de votre stratégie de partitionnement. Une partition entière peut être détachée rapidement ; il peut donc être judicieux de concevoir la stratégie de partitionnement de manière à ce que toutes les données à supprimer simultanément se trouvent dans une seule partition.

Choisir le nombre cible de partitions pour diviser la table est une décision cruciale. Un nombre insuffisant de partitions peut entraîner des index trop volumineux et une mauvaise localité des données, ce qui risque de réduire le taux d'accès au cache. Cependant, un nombre excessif de partitions peut également poser problème. Il peut se traduire par des temps de planification des requêtes plus longs et une consommation de mémoire plus élevée, tant lors de la planification que de l'exécution, comme expliqué ci-dessous. Lors du choix du partitionnement, il est important d'anticiper les évolutions futures. Par exemple, si vous optez pour une partition par client et que vous avez actuellement peu de grands clients, il est essentiel d'envisager les conséquences d'une augmentation du nombre de petits clients dans quelques années. Dans ce cas, il peut être préférable de partitionner par hachage et de choisir un nombre raisonnable de partitions plutôt que de partitionner par liste en espérant que le nombre de clients ne dépasse pas les limites pratiques du partitionnement.

Le sous-partitionnement peut s'avérer utile pour subdiviser les partitions susceptibles de devenir plus volumineuses que d'autres. Une autre option consiste à utiliser le partitionnement par plage avec plusieurs colonnes dans la clé de partition. Ces deux méthodes peuvent facilement engendrer un nombre excessif de partitions ; il est donc conseillé de faire preuve de modération.

Il est important de prendre en compte la surcharge liée au partitionnement lors de la planification et de l'exécution des requêtes. Le planificateur de requêtes gère généralement correctement les hiérarchies de partitions comportant jusqu'à quelques milliers de partitions, à condition que les requêtes classiques lui permettent d'éliminer la quasi-totalité des partitions. Les temps de planification s'allongent et la consommation de mémoire augmente lorsque davantage de partitions subsistent après l'élimination des partitions par le planificateur. Un autre motif de préoccupation lié à un grand nombre de partitions est l'augmentation potentielle de la consommation de mémoire du serveur au fil du temps, notamment si de nombreuses sessions accèdent à un grand nombre de partitions. En effet, chaque partition nécessite le chargement de ses métadonnées dans la mémoire locale de chaque session y accédant.

Pour les charges de travail de type entrepôt de données, il peut être judicieux d'utiliser un plus grand nombre de partitions que pour une charge de travail de type OLTP. Généralement, dans les entrepôts de données, le temps de planification des requêtes est moins critique, car la majeure partie du temps de traitement est consacrée à l'exécution des requêtes. Quel que soit le type de charge de travail, il est important de prendre les bonnes décisions dès le départ, car le repartitionnement de grandes quantités de données peut s'avérer extrêmement long. Les simulations de la charge de travail prévue sont souvent utiles pour optimiser la stratégie de partitionnement. Il ne faut jamais supposer que plus de partitions sont meilleures que moins, ni l'inverse.

récioproquement.

5.13. Données étrangères

PostgreSQL implémente certaines parties de la spécification SQL/MED, ce qui vous permet d'accéder à des données externes à PostgreSQL via des requêtes SQL classiques. Ces données sont appelées données externes. (Attention à ne pas confondre ce terme avec les clés étrangères, qui sont un type de contrainte au sein de la base de données.)

L'accès aux données externes se fait via une interface de données externes. Cette interface est une bibliothèque qui communique avec une source de données externe, masquant les détails de la connexion et de l'extraction des données. Certaines interfaces sont disponibles sous forme de modules complémentaires (voir l'annexe F). D'autres peuvent être trouvées dans des produits tiers. Si aucune interface existante ne répond à vos besoins, vous pouvez développer la vôtre (voir le chapitre 57).

Pour accéder à des données externes, vous devez créer un objet serveur externe , qui définit la connexion à une source de données externe particulière selon les options utilisées par son wrapper de données externes. Ensuite, vous devez créer une ou plusieurs tables externes, qui définissent la structure de la source distante.

Les données d'une table externe peuvent être utilisées dans les requêtes comme celles d'une table classique, mais cette table n'est pas stockée sur le serveur PostgreSQL. Lorsqu'elle est utilisée, PostgreSQL demande au gestionnaire de données externe de récupérer les données depuis la source externe, ou de leur transmettre des données dans le cas des commandes de mise à jour.

L'accès aux données distantes peut nécessiter une authentification auprès de la source de données externe. Ces informations peuvent être fournies par un mappage utilisateur, qui peut inclure des données supplémentaires telles que les noms d'utilisateur et les mots de passe en fonction du rôle PostgreSQL actuel.

Pour plus d'informations, consultez CREATE FOREIGN DATA WRAPPER, CREATE SERVER, CREATE USER MAPPING, CREATE FOREIGN TABLE et IMPORT FOREIGN SCHEMA.

5.14. Autres objets de base de données

Les tables sont les objets centraux d'une structure de base de données relationnelle, car elles contiennent vos données. Cependant, elles ne sont pas les seuls objets présents dans une base de données. De nombreux autres types d'objets peuvent être créés pour rendre l'utilisation et la gestion des données plus efficaces et plus pratiques. Ils ne sont pas abordés dans ce chapitre, mais nous vous en présentons une liste ci-dessous afin que vous ayez une idée des possibilités :

- Vues
- Fonctions, procédures et opérateurs
- Types de données et domaines
- Déclencheurs et règles de réécriture

Des informations détaillées sur ces sujets figurent dans la partie V.

5.15. Suivi des dépendances

Lorsque vous créez des structures de bases de données complexes comprenant de nombreuses tables avec des contraintes de clés étrangères, des vues, des déclencheurs, des fonctions, etc., vous créez implicitement un réseau de dépendances entre les objets. Par exemple, une table avec une contrainte de clé étrangère dépend de la table qu'elle référence.

Pour garantir l'intégrité de la structure de la base de données, PostgreSQL empêche la suppression d'objets dont d'autres dépendent. Par exemple, toute tentative de suppression de la table `products` (vue dans la section 5.5.5), dont dépend la table `orders`, générerait un message d'erreur similaire à celui-ci :

Produits DROP TABLE ;

ERREUR : impossible de supprimer les produits de table car d'autres objets en dépendent.
il

DÉTAIL : la contrainte orders_product_no_fkey sur la table orders dépend
produits de table

ASTUCE : Utilisez DROP ... CASCADE pour supprimer également les objets dépendants.

Le message d'erreur contient une indication utile : si vous ne souhaitez pas supprimer individuellement tous les objets dépendants, vous pouvez exécuter :

Produits DROP TABLE CASCADE ;

et tous les objets dépendants seront supprimés, ainsi que tous les objets qui dépendent d'eux, de manière récursive. Dans ce cas, la table des commandes n'est pas supprimée ; seule la contrainte de clé étrangère l'est. L'opération s'arrête là car aucune donnée ne dépend de cette contrainte. (Pour plus d'informations, consultez la commande DROP ...) CASCADE fera l'affaire, exécutez DROP sans CASCADE et lisez la sortie DETAIL.)

Presque toutes les commandes DROP de PostgreSQL permettent de spécifier CASCADE. Bien entendu, la nature des dépendances possibles varie selon le type d'objet. Vous pouvez également utiliser RESTRICT au lieu de CASCADE pour obtenir le comportement par défaut, qui consiste à empêcher la suppression des objets dont dépendent d'autres objets.

Note

Selon la norme SQL, la spécification de RESTRICT ou CASCADE est requise dans une commande DROP. Aucun système de base de données n'applique cette règle, mais le comportement par défaut (RESTRICT ou CASCADE) varie d'un système à l'autre.

Si une commande DROP liste plusieurs objets, l'option CASCADE n'est requise que s'il existe des dépendances en dehors du groupe spécifié. Par exemple, avec la commande DROP TABLE tab1, tab2, l'existence d'une clé étrangère référençant tab1 depuis tab2 n'implique pas nécessairement que CASCADE soit nécessaire pour que l'opération réussisse.

Pour une fonction ou une procédure définie par l'utilisateur dont le corps est défini comme une chaîne de caractères littérale, PostgreSQL suit les dépendances associées aux propriétés visibles de la fonction, telles que les types de ses arguments et de son résultat, mais pas les dépendances qui ne peuvent être connues qu'en examinant le corps de la fonction. Prenons l'exemple suivant :

```
CRÉER LE TYPE arc-en-ciel COMME ÉNUMÉRATION ('rouge', 'orange', 'jaune',  
                                              'vert', 'bleu', 'violet');
```

```
CRÉER TABLE my_colors (couleur arc-en-ciel, texte de la note);
```

```
CRÉER UNE FONCTION get_color_note (arc-en-ciel) RETOURNE text AS  
  'SELECT note FROM my_colors WHERE color = $1'  
  LANGUAGE SQL ;
```

(Voir la section 36.5 pour une explication des fonctions du langage SQL.) PostgreSQL détecte que la fonction `get_color_note` dépend du type `rainbow` : supprimer ce type entraînerait la suppression de la fonction, car le type de son argument ne serait plus défini. Cependant, PostgreSQL ne considère pas que `get_color_note` dépende de la table `my_colors` et ne la supprimera donc pas si cette table est supprimée. Bien que cette approche présente des inconvénients, elle offre également des avantages. La fonction reste valide, dans une certaine mesure, même si la table est manquante ; son exécution provoquerait une erreur. La création d'une nouvelle table portant le même nom permettrait à la fonction de fonctionner à nouveau.

En revanche, pour une fonction ou une procédure SQL dont le corps est écrit selon les conventions SQL, ce corps est analysé lors de la définition de la fonction et toutes les dépendances reconnues par l'analyseur syntaxique sont enregistrées. Ainsi, si nous écrivons la fonction ci-dessus comme

```
CRÉER UNE FONCTION get_color_note (arc-en-ciel) RETOURNE texte  
DÉBUT ATOMIQUE  
  SÉLECTIONNER note DE my_colors OÙ couleur = $1;  
FIN;
```

La dépendance de la fonction vis-à-vis de la table `my_colors` sera alors connue et appliquée par DROP.

Chapitre 6. Manipulation des données

Le chapitre précédent expliquait comment créer des tables et autres structures pour stocker vos données. Il est maintenant temps de remplir ces tables. Ce chapitre décrit comment insérer, mettre à jour et supprimer des données. Le chapitre suivant expliquera enfin comment récupérer vos données perdues depuis longtemps dans la base de données.

6.1. Insertion des données

Lorsqu'une table est créée, elle est vide. La première étape pour qu'une base de données soit exploitable consiste à y insérer des données. Les données sont insérées ligne par ligne. Il est possible d'insérer plusieurs lignes en une seule commande, mais uniquement des lignes complètes. Même si vous ne connaissez que certaines valeurs de colonnes, une ligne complète doit être créée.

Pour créer une nouvelle ligne, utilisez la commande INSERT. Cette commande requiert le nom de la table et les valeurs des colonnes. Par exemple, prenons la table « produits » du chapitre 5 :

```
CRÉER TABLE produits ( numéro_produit
                        entier, nom texte, prix numérique
                        );
```

Voici un exemple de commande pour insérer une ligne :

```
INSÉRER DANS produits VALEURS (1, 'Fromage', 9.99);
```

Les valeurs des données sont listées dans l'ordre où apparaissent les colonnes dans le tableau, séparées par des virgules. Généralement, les valeurs des données seront des valeurs littérales (constantes), mais les expressions scalaires sont également autorisées.

La syntaxe ci-dessus présente l'inconvénient de nécessiter la connaissance de l'ordre des colonnes dans le tableau. Pour éviter cela, vous pouvez également lister explicitement les colonnes. Par exemple, les deux commandes suivantes ont le même effet que la précédente :

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Fromage',
9.99);
INSERT INTO products (name, price, product_no) VALUES ('Cheese',
9.99, 1);
```

De nombreux utilisateurs considèrent qu'il est judicieux de toujours indiquer les noms des colonnes.

Si vous ne disposez pas de valeurs pour toutes les colonnes, vous pouvez en omettre certaines. Dans ce cas, les colonnes seront remplies avec leurs valeurs par défaut. Par exemple :

```
INSÉRER DANS produits (numéro_produit, nom) VALEURS (1, 'Fromage');
INSÉRER DANS produits VALEURS (1, 'Fromage');
```

La seconde forme est une extension PostgreSQL. Elle remplit les colonnes à partir de la gauche avec autant de valeurs fournies, les autres étant initialisées par défaut.

Pour plus de clarté, vous pouvez également demander explicitement des valeurs par défaut, pour des colonnes individuelles ou pour la ligne entière :

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Fromage',
DÉFAUT);
```

INSÉRER DANS produits VALEURS PAR DÉFAUT ;

Vous pouvez insérer plusieurs lignes en une seule commande :

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Fromage', 9.99), (2, 'Pain', 1.99), (3, 'Lait', 2.99);
```

Il est également possible d'insérer le résultat d'une requête (qui peut ne contenir aucune ligne, une seule ligne ou plusieurs lignes) :

```
INSÉRER DANS produits (numéro_produit, nom, prix)
SÉLECTIONNER product_no, name, price FROM new_products
OÙ date_de_sortie = 'aujourd'hui';
```

Cela permet d'exploiter pleinement la puissance du mécanisme de requête SQL (chapitre 7) pour calculer les lignes à insérer.

Conseil

Lors de l'insertion simultanée d'un grand nombre de données, privilégiez la commande COPY. Bien que moins flexible que la commande INSERT, elle est plus performante. Consultez la section 14.4 pour plus d'informations sur l'amélioration des performances lors du chargement en masse.

6.2. Mise à jour des données

La modification des données déjà présentes dans la base de données est appelée mise à jour. Vous pouvez mettre à jour des lignes individuelles, toutes les lignes d'une table ou un sous-ensemble de lignes. Chaque colonne peut être mise à jour séparément ; les autres colonnes restent inchangées.

Pour mettre à jour les lignes existantes, utilisez la commande UPDATE. Celle-ci nécessite trois informations :

1. Le nom de la table et de la colonne à mettre à jour. 2. La nouvelle valeur de la colonne. 3. La ou les lignes à mettre à jour.

Rappelons-nous du chapitre 5 que SQL ne fournit généralement pas d'identifiant unique pour les lignes. Il n'est donc pas toujours possible de spécifier directement la ligne à mettre à jour. On définit plutôt les conditions que doit remplir une ligne pour être mise à jour. Seule la présence d'une clé primaire dans la table (qu'elle soit déclarée ou non) permet de cibler avec certitude les lignes individuelles en choisissant une condition correspondant à cette clé primaire. Les outils d'accès graphique aux bases de données exploitent ce principe pour permettre la mise à jour individuelle des lignes.

Par exemple, cette commande met à jour le prix de tous les produits dont le prix est de 5 à 10 :

```
MISE À JOUR des produits DÉFINIR prix = 10 OÙ prix = 5 ;
```

Cela peut entraîner la mise à jour de zéro, une ou plusieurs lignes. Tenter une mise à jour qui ne correspond à aucune ligne n'est pas une erreur.

Examinons cette commande en détail. Elle commence par le mot-clé UPDATE, suivi du nom de la table. Comme d'habitude, le nom de la table peut être qualifié par le schéma ; sinon, il est recherché dans le chemin d'accès. Vient ensuite le mot-clé SET, suivi du nom de la colonne, du signe égal et de la nouvelle valeur de la colonne. Cette nouvelle valeur peut être n'importe quelle expression scalaire, et pas seulement une constante. Par exemple, pour augmenter le prix de tous les produits de 10 %, vous pouvez utiliser :

MISE À JOUR des produits DÉFINIR prix = prix * 1,10 ;

Comme vous le voyez, l'expression de la nouvelle valeur peut faire référence à la ou aux valeurs existantes dans la ligne. Nous avons également omis la clause WHERE. Si elle est omise, toutes les lignes du tableau sont mises à jour. Si elle est présente, seules les lignes correspondant à la condition WHERE sont mises à jour. Notez que le signe égal dans la clause SET correspond à une affectation, tandis que celui de la clause WHERE correspond à une comparaison ; cela ne crée cependant aucune ambiguïté. Bien entendu, la condition WHERE n'est pas nécessairement un test d'égalité. De nombreux autres opérateurs sont disponibles (voir le chapitre 9). L'expression doit néanmoins renvoyer un résultat booléen.

Vous pouvez mettre à jour plusieurs colonnes dans une commande UPDATE en indiquant plusieurs affectations dans la clause SET. Par exemple :

MISE À JOUR mytable DÉFINIR a = 5, b = 3, c = 1 OÙ a > 0 ;

6.3. Suppression des données

Nous avons jusqu'à présent expliqué comment ajouter et modifier des données dans les tables. Il nous reste à voir comment supprimer les données devenues inutiles. De même que l'ajout de données ne peut se faire que par lignes entières, la suppression d'une table ne peut se faire que par lignes entières. Dans la section précédente, nous avons expliqué que SQL ne permet pas d'accéder directement à des lignes individuelles. Par conséquent, la suppression de lignes ne peut se faire qu'en spécifiant des conditions auxquelles les lignes à supprimer doivent correspondre. Si la table possède une clé primaire, vous pouvez spécifier la ligne exacte. Vous pouvez également supprimer des groupes de lignes correspondant à une condition, ou supprimer toutes les lignes de la table en une seule fois.

La commande DELETE permet de supprimer des lignes ; sa syntaxe est très similaire à celle de la commande UPDATE. Par exemple, pour supprimer toutes les lignes du tableau des produits dont le prix est de 10, utilisez :

SUPPRIMER DE la base de données produits OÙ prix = 10 ;

Si vous écrivez simplement :

SUPPRIMER DES produits ;

Toutes les lignes du tableau seront alors supprimées ! Attention programmeur !

6.4. Renvoi des données des lignes modifiées

Il est parfois utile de récupérer des données à partir de lignes modifiées pendant leur manipulation. Les commandes INSERT, UPDATE, DELETE et MERGE disposent toutes d'une clause RETURNING optionnelle permettant cela. L'utilisation de RETURNING évite d'effectuer une requête de base de données supplémentaire pour collecter les données et s'avère particulièrement précieuse lorsqu'il serait autrement difficile d'identifier avec certitude les lignes modifiées.

Le contenu autorisé d'une clause RETURNING est identique à la liste de résultats d'une commande SELECT (voir section 7.3). Il peut contenir les noms des colonnes de la table cible ou des expressions de valeur utilisant ces colonnes. L'utilisation de la syntaxe abrégée RETURNING * permet de sélectionner toutes les colonnes de la table cible, dans l'ordre.

Lors d'une insertion, les données renvoyées par la fonction RETURNING correspondent à la ligne telle qu'elle a été insérée. Ceci est peu utile pour les insertions simples, car cela ne ferait que répéter les données fournies par le client. En revanche, cela peut s'avérer très pratique lorsqu'on utilise des valeurs par défaut calculées. Par exemple, lorsqu'une colonne de type sériel est utilisée pour fournir des identifiants uniques, RETURNING peut renvoyer l'identifiant attribué à une nouvelle ligne.

CRÉER LA TABLE utilisateurs (prénom texte, nom texte, id numéro de série
clé primaire);

```
INSÉRER DANS users (firstname, lastname) VALUES ('Joe', 'Cool')
RETOURNANT l'identifiant ;
```

La clause RETURNING est également très utile avec INSERT ... SELECT.

Lors d'une mise à jour (UPDATE), les données renvoyées correspondent au nouveau contenu de la ligne modifiée. Par exemple :

```
MISE À JOUR des produits DÉFINIR le prix = prix * 1,10
WHERE prix <= 99,99 RETOURNER
nom, prix AS nouveau_prix;
```

Lors d'une suppression, les données renvoyées correspondent au contenu de la ligne supprimée. Par exemple :

```
SUPPRIMER DES produits
OÙ obsolescence_date = 'aujourd'hui'
RETOUR *;
```

Dans une opération MERGE, les données renvoyées par RETURNING correspondent au contenu de la ligne source auquel s'ajoute le contenu de la ligne cible insérée, mise à jour ou supprimée. Comme il est fréquent que la source et la cible partagent de nombreuses colonnes, spécifier RETURNING * peut entraîner la duplication de nombreuses colonnes. Il est donc souvent plus judicieux de préciser la valeur renvoyée afin de ne conserver que la ligne source ou la ligne cible. Par exemple :

```
FUSIONNER DANS les produits p UTILISANT nouveaux_produits n SUR p.product_no = n.product_no LORSQUE
NON CORRESPONDANT
ALORS INSÉRER LES VALEURS (n.product_no, n.name, n.price)

LORSQUE CORRESPONDANCE, METTRE À JOUR DÉFINIR nom = n.nom, prix = n.prix RETOURNER p.*;
```

Si des déclencheurs sont présents sur la table cible (chapitre 37), les données renvoyées par RETURNING correspondent à la ligne modifiée par ces déclencheurs. L'inspection des colonnes calculées par les déclencheurs constitue donc un autre cas d'utilisation courant de RETURNING.

Chapitre 7. Requêtes

Les chapitres précédents expliquaient comment créer des tables, comment les remplir de données et comment manipuler ces données. Nous allons maintenant aborder la récupération des données depuis la base de données.

7.1. Vue d'ensemble

Le processus d'extraction de données d'une base de données, ou la commande permettant de récupérer des données, est appelé une requête. En SQL, la commande SELECT est utilisée pour spécifier les requêtes. La syntaxe générale de la commande SELECT est la suivante :

```
[AVEC with_queries] SELECT select_list FROM table_expression [sort_specification]
```

Les sections suivantes décrivent en détail la liste de sélection, l'expression de table et la spécification de tri. Les requêtes WITH sont traitées en dernier, car il s'agit d'une fonctionnalité avancée.

Une requête simple se présente sous la forme suivante :

```
SÉLECTIONNER * DE table1;
```

En supposant l'existence d'une table nommée table1, cette commande récupère toutes les lignes et toutes les colonnes définies par l'utilisateur de table1. (La méthode de récupération dépend de l'application cliente. Par exemple, le programme psql affichera une table en ASCII à l'écran, tandis que les bibliothèques clientes proposeront des fonctions permettant d'extraire des valeurs individuelles du résultat de la requête.) La spécification de la liste SELECT * désigne toutes les colonnes fournies par l'expression de table. Une liste SELECT peut également sélectionner un sous-ensemble des colonnes disponibles ou effectuer des calculs à partir de ces colonnes. Par exemple, si table1 possède des colonnes nommées a, b et c (et éventuellement d'autres), vous pouvez exécuter la requête suivante :

```
SÉLECTIONNEZ a, b + c DE table1;
```

(en supposant que b et c soient de type numérique). Voir la section 7.3 pour plus de détails.

FROM table1 est une expression de table simple : elle ne lit qu'une seule table. En général, les expressions de table peuvent être des constructions complexes composées de tables de base, de jointures et de sous-requêtes. Mais vous pouvez aussi omettre complètement l'expression de table et utiliser la commande SELECT comme une calculatrice.

```
SÉLECTIONNER 3 * 4 ;
```

Ceci est plus utile si les expressions de la liste de sélection renvoient des résultats différents. Par exemple, vous pourriez appeler une fonction de cette manière :

```
SÉLECTIONNER aléatoire();
```

7.2. Expressions tabulaires

Une expression de table permet de calculer une table. Elle contient une clause FROM, éventuellement suivie des clauses WHERE, GROUP BY et HAVING. Les expressions de table simples font référence à une table sur disque, appelée table de base, tandis que les expressions plus complexes permettent de modifier ou de combiner des tables de base de différentes manières.

Les clauses facultatives WHERE, GROUP BY et HAVING dans l'expression de table spécifient un enchaînement de transformations successives appliquées à la table dérivée de la clause FROM. Toutes ces transformations

Les opérations produisent une table virtuelle qui fournit les lignes transmises à la liste de sélection pour calculer les lignes de sortie de la requête.

7.2.1. La clause FROM

La clause FROM permet de dériver une table à partir d'une ou plusieurs autres tables données dans une liste de références de tables séparées par des virgules.

DE référence_table [, référence_table [, ...]]

Une référence de table peut être un nom de table (éventuellement qualifié par schéma), ou une table dérivée telle qu'une sous-requête, une jointure ou une combinaison complexe de ces éléments. Si plusieurs références de table sont listées dans la clause FROM, les tables sont jointes de manière croisée (c'est-à-dire que le produit cartésien de leurs lignes est formé ; voir ci-dessous). Le résultat de la liste FROM est une table virtuelle intermédiaire qui peut ensuite être transformée par les clauses WHERE, GROUP BY et HAVING, et qui constitue finalement le résultat de l'expression de table globale.

Lorsqu'une référence de table désigne une table parente d'une hiérarchie d'héritage, elle renvoie les lignes non seulement de cette table, mais aussi de toutes ses tables descendantes, sauf si le mot-clé ONLY précède le nom de la table. Toutefois, seules les colonnes présentes dans la table désignée sont renvoyées ; les colonnes ajoutées dans les sous-tables sont ignorées.

Au lieu d'écrire ONLY avant le nom de la table, vous pouvez écrire * après pour indiquer explicitement que les tables descendantes sont incluses. Cette syntaxe n'est plus vraiment utile, car la recherche dans les tables descendantes est désormais le comportement par défaut. Elle est toutefois conservée pour assurer la compatibilité avec les versions antérieures.

7.2.1.1. Tables jointes

Une table jointe est une table dérivée de deux autres tables (réelles ou dérivées) selon les règles du type de jointure spécifique. Les jointures internes, externes et croisées sont disponibles. La syntaxe générale d'une table jointe est la suivante :

T1 type_de_jointure T2 [condition_de_jointure]

Les jointures de tous types peuvent être enchaînées ou imbriquées : T1 et T2 peuvent être jointes, ou les deux. Les parenthèses permettent de contrôler l'ordre des clauses JOIN. Sans parenthèses, les clauses JOIN s'imbriquent de gauche à droite.

Types de jointure

Jointure croisée

Jonction transversale T1 T2

Pour toute combinaison possible de lignes de T1 et T2 (c'est-à-dire un produit cartésien), la table résultante contiendra une ligne composée de toutes les colonnes de T1 suivies de toutes les colonnes de T2. Si les tables ont respectivement N et M lignes, la table résultante aura N * M lignes.

L'instruction FROM T1 CROSS JOIN T2 est équivalente à FROM T1 INNER JOIN T2 ON TRUE (voir ci-dessous). Elle est également équivalente à FROM T1, T2.

Note

Cette dernière équivalence n'est pas tout à fait valable lorsqu'il y a plus de deux tables, car l'opérateur JOIN est plus contraignant que la virgule. Par exemple, FROM T1 CROSS JOIN T2 INNER JOIN T3 ON condition n'est pas équivalent à FROM T1, T2 INNER JOIN.

La condition T3 est activée car elle peut faire référence à T1 dans le premier cas, mais pas dans le second.

Adhésions qualifiées

T1 { [INTÉRIEUR] | { GAUCHE | DROITE | COMPLET } [EXTÉRIEUR] } REJOINDRE T2

expression booléenne ON

T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2 USING (join column list)

T1 NATUREL { [INTÉRIEUR] | { GAUCHE | DROITE | COMPLET } [EXTÉRIEUR] } REJOINDRE T2

Les termes INNER et OUTER sont facultatifs dans tous les formulaires. INNER est la valeur par défaut ; LEFT, RIGHT et FULL impliquent une jointure externe.

La condition de jointure est spécifiée dans la clause ON ou USING, ou implicitement par le mot NATURAL.

La condition de jointure détermine quelles lignes des deux tables sources sont considérées comme « correspondantes », comme expliqué en détail ci-dessous.

Les types possibles de jointure qualifiée sont :

Jonction intérieure

Pour chaque ligne R1 de T1, la table jointe a une ligne pour chaque ligne de T2 qui satisfait la condition de jointure avec R1.

Jonction extérieure gauche

On effectue d'abord une jointure interne. Ensuite, pour chaque ligne de T1 ne satisfaisant aucune condition de jointure avec une ligne de T2, une ligne de jointure est ajoutée avec des valeurs nulles dans les colonnes de T2. Ainsi, la table résultante contient toujours au moins une ligne pour chaque ligne de T1.

Jonction extérieure droite

On effectue d'abord une jointure interne. Ensuite, pour chaque ligne de T2 ne satisfaisant aucune condition de jointure avec une ligne de T1, une ligne de jointure est ajoutée avec des valeurs nulles dans les colonnes de T1. C'est l'inverse d'une jointure externe gauche : la table résultante contiendra toujours une ligne pour chaque ligne de T2.

Jonction extérieure complète

Tout d'abord, une jointure interne est effectuée. Ensuite, pour chaque ligne de T1 ne satisfaisant aucune condition de jointure avec une ligne de T2, une ligne jointe est ajoutée avec des valeurs nulles dans les colonnes de T2. De même, pour chaque ligne de T2 ne satisfaisant aucune condition de jointure avec une ligne de T1, une ligne jointe avec des valeurs nulles dans les colonnes de T1 est ajoutée.

La clause ON est la condition de jointure la plus générale : elle prend une expression booléenne du même type que celle utilisée dans une clause WHERE. Deux lignes de T1 et T2 correspondent si l'expression ON est vraie.

La clause USING est une syntaxe abrégée permettant d'exploiter le cas particulier où les deux côtés de la jointure utilisent le même nom pour la ou les colonnes de jointure. Elle prend une liste de noms de colonnes partagés, séparés par des virgules, et crée une condition de jointure incluant une comparaison d'égalité pour chaque colonne. Par exemple, la jointure de T1 et T2 avec USING (a, b) produit la condition de jointure ON T1.a = T2.a AND T1.b = T2.b.

De plus, la fonction JOIN USING supprime les colonnes redondantes : il est inutile d'afficher les deux colonnes correspondantes, puisqu'elles ont nécessairement la même valeur. Alors que JOIN ON affiche toutes les colonnes de T1 suivies de toutes les colonnes de T2, JOIN USING produit une colonne pour chaque paire de colonnes listées (dans l'ordre indiqué), suivie des colonnes restantes de T1, puis des colonnes restantes de T2.

Enfin, NATURAL est une forme abrégée de USING : elle forme une liste USING composée de toutes les colonnes noms qui apparaissent dans les deux tables d'entrée. Comme pour USING, ces colonnes n'apparaissent qu'une seule fois dans la Tableau de sortie. S'il n'y a pas de noms de colonnes communs, NATURAL JOIN se comporte comme CROSS.

REJOINDRE.

Note

L'utilisation de `USING` est relativement à l'abri des modifications de colonnes dans les relations jointes, car seules les colonnes listées sont concernées.

Les colonnes sont combinées. L'option NATURAL est considérablement plus risquée car toute modification du schéma peut entraîner des problèmes.

L'ajout d'une nouvelle colonne correspondante à l'une ou l'autre relation entraînera la jointure.
pour combiner également cette nouvelle colonne.

Pour résumer, supposons que nous ayons des tables t1 :

numéro	nom
1	un
2	b
3	c

et t2 :

nombre	valeur
1	xxx
3	yyy
5	zzz

Nous obtenons alors les résultats suivants pour les différentes jointures :

=> SÉLECTIONNER * DE t1 JOINTURE CROISÉE t2;

numéro	nom	numéro	valeur
1	a	1	xxx
a	1	3	yyy
2	b	5	zzz
b	2	1	xxx
c	3	3	yyy
3	c	5	zzz
lignes)		1	xxx
		3	yyy
		5	zzz

=> SELECT * FROM t1 INNER JOIN t2 ON t1.num = t2.num;

numéro	nom	numéro	valeur
1	a	3	xxx
c	(2	3	yyy
lignes)			

=> SELECT * FROM t1 INNER JOIN t2 USING (num);

numéro	nom	valeur
1	un	xxx

```

      3 | c (2      | yyy
lignes)

```

=> SÉLECTIONNER * DE t1 JOINTURE INTÉRIEURE NATURELLE t2;

```

numéro | nom | valeur
-----+-----+-----
      1 | a 3 |      | xxx
      c (2      | yyy
lignes)

```

=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num;

```

numéro | nom | numéro | valeur
-----+-----+-----+-----
      1 | a 2 |      | 1 | xxx
      b 3 | c      |      |
(3 lignes)      | 3 | yyy

```

=> SELECT * FROM t1 LEFT JOIN t2 USING (num);

```

numéro | nom | valeur
-----+-----+-----
      1 | a 2 |      | xxx
      b 3 | c      |
(3 lignes)      | yyy

```

=> SELECT * FROM t1 RIGHT JOIN t2 ON t1.num = t2.num;

```

numéro | nom | numéro | valeur
-----+-----+-----+-----
      1 | a 3 |      | 1 | xxx
      c | (3      |      | 3 | yyy
      |      |      | 5 | zzz
lignes)

```

=> SELECT * FROM t1 FULL JOIN t2 ON t1.num = t2.num;

```

numéro | nom | numéro | valeur
-----+-----+-----+-----
      1 | a 2 |      | 1 | xxx
      b 3 | c |      |
(4 lignes)      | 3 | yyy
      |      |      | 5 | zzz

```

La condition de jointure spécifiée avec ON peut également contenir des conditions qui ne sont pas directement liées à la jointure. Cela peut s'avérer utile pour certaines requêtes, mais il convient d'y réfléchir attentivement. Par exemple :

=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num AND t2.value =

```

'xxx';
numéro | nom | numéro | valeur
-----+-----+-----+-----
      1 | a 2 |      | 1 | xxx
      b 3 | c |      |
(3 lignes)      |

```

Notez que le fait de placer la restriction dans la clause WHERE produit un résultat différent :

```
=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num WHERE t2.value
= 'xxx'; num |
nom | num | valeur
-----+-----+-----
1 | a (1      |      1 | xxx
ligne)
```

En effet, une restriction placée dans la clause ON est traitée avant la jointure, tandis qu'une restriction placée dans la clause WHERE est traitée après. Cela n'a pas d'importance pour les jointures internes, mais c'est crucial pour les jointures externes.

7.2.1.2. Alias des tables et des colonnes

On peut attribuer un nom temporaire aux tables et aux références de tables complexes afin de faire référence à la table dérivée dans le reste de la requête. On appelle cela un alias de table.

Pour créer un alias de table, écrivez

```
FROM table_reference AS alias
```

ou

```
DE L'alias de référence de table
```

Le mot-clé AS est un bruit optionnel. alias peut être n'importe quel identifiant.

L'utilisation typique des alias de table consiste à attribuer des identifiants courts aux noms de tables longs afin de faciliter la lecture des clauses de jointure. Par exemple :

```
SELECT * FROM some_very_long_table_name s JOIN another_fairly_long_name
a ON s.id = a.num;
```

L'alias devient le nouveau nom de la référence à la table pour la requête en cours ; il est interdit de faire référence à la table par son nom d'origine ailleurs dans la requête. Par conséquent, cette requête n'est pas valide :

```
SELECT * FROM my_table AS m WHERE my_table.a > 5; -- faux
```

Les alias de table servent principalement à simplifier la notation, mais il est nécessaire de les utiliser lors de la jointure d'une table avec elle-même, par exemple :

```
SELECT * FROM people AS mère JOIN people AS enfant ON mère.id = enfant.mère_id;
```

Les parenthèses servent à lever les ambiguïtés. Dans l'exemple suivant, la première instruction attribue l'alias « b » à la seconde instance de « my_table », tandis que la seconde l'attribue au résultat de la jointure :

```
SELECT * FROM my_table AS a CROSS JOIN my_table AS b ...
SELECT * FROM (my_table AS a CROSS JOIN my_table) AS b ...
```

Une autre forme d'aliasage de table consiste à attribuer des noms temporaires aux colonnes de la table, ainsi qu'à la table elle-même :

```
DE table_reference [AS] alias ( colonne1 [, colonne2 [, ...]] )
```

Si le nombre d'alias de colonnes spécifiés est inférieur au nombre de colonnes de la table, les colonnes restantes ne sont pas renommées. Cette syntaxe est particulièrement utile pour les auto-jointures ou les sous-requêtes.

Lorsqu'un alias est appliqué au résultat d'une clause JOIN, cet alias masque le ou les noms d'origine au sein de la clause JOIN. Par exemple :

```
SELECT a.* FROM my_table AS a JOIN your_table AS b ON ...
```

est une requête SQL valide, mais :

```
SELECT a.* FROM (my_table AS a JOIN your_table AS b ON ...) AS c
```

n'est pas valide ; l'alias de table a n'est pas visible en dehors de l'alias c.

7.2.1.3. Sous-requêtes

Les sous-requêtes spécifiant une table dérivée doivent être placées entre parenthèses. Elles peuvent se voir attribuer un alias de table et, éventuellement, des alias de colonne (comme dans la section 7.2.1.2). Par exemple :

```
FROM (SELECT * FROM table1) AS alias_name
```

Cet exemple est équivalent à FROM table1 AS alias_name. Des cas plus complexes, qui ne peuvent se réduire à une simple jointure, apparaissent lorsque la sous-requête implique un regroupement ou une agrégation.

Une sous-requête peut également être une liste de VALEURS :

```
DE (VALEURS ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))
  AS noms(prénom, nom)
```

Là encore, l'utilisation d'un alias de table est facultative. Attribuer des alias aux colonnes de la liste VALUES est également facultatif, mais recommandé. Pour plus d'informations, consultez la section 7.7.

Conformément à la norme SQL, un alias de table doit être fourni pour une sous-requête. PostgreSQL autorise l'omission de l'alias et du nom d'utilisateur (AS), mais il est recommandé d'en définir un dans le code SQL susceptible d'être porté sur un autre système.

7.2.1.4. Fonctions de table

Les fonctions de table produisent un ensemble de lignes, composées soit de types de données de base (types scalaires), soit de types de données composites (lignes de table). Elles s'utilisent comme une table, une vue ou une sous-requête dans la clause FROM d'une requête. Les colonnes renvoyées par les fonctions de table peuvent être incluses dans les clauses SELECT, JOIN ou WHERE de la même manière que les colonnes d'une table, d'une vue ou d'une sous-requête.

Les fonctions de table peuvent également être combinées à l'aide de la syntaxe ROWS FROM, les résultats étant renvoyés dans des colonnes parallèles ; le nombre de lignes de résultat dans ce cas est celui du résultat de fonction le plus important, les résultats plus petits étant complétés par des valeurs nulles pour correspondre.

```
appel_fonction [AVEC ORDINALITÉ] [[AS] alias_table [(alias_colonne
  [, ... ])] ]
LIGNES DE( appel_fonction [, ... ] ) [AVEC ORDINALITÉ]
  [[AS] table_alias [(column_alias [, ... ])] ]
```

Si la clause WITH ORDINALITY est spécifiée, une colonne supplémentaire de type bigint sera ajoutée aux colonnes de résultats de la fonction. Cette colonne numérote les lignes du jeu de résultats, à partir de 1. (Il s'agit d'une généralisation de la syntaxe standard SQL pour UNNEST...) AVEC ORDINALITÉ.)

Par défaut, la colonne ordinale s'appelle ordinalité, mais un autre nom de colonne peut lui être attribué à l'aide d'une clause AS.

La fonction de table spéciale UNNEST peut être appelée avec un nombre quelconque de paramètres de tableau, et elle renvoie un nombre correspondant de colonnes, comme si UNNEST (Section 9.19) avait été appelée sur chaque paramètre séparément et combinée à l'aide de la construction ROWS FROM.

```
DÉMONTAGE( expression_tableau [, ... ] ) [AVEC ORDINALITÉ]
  [[AS] table_alias [(column_alias [, ... ])]]
```

Si aucun alias de table n'est spécifié, le nom de la fonction est utilisé comme nom de table ; dans le cas d'une construction ROWS FROM(), c'est le nom de la première fonction qui est utilisé.

Si aucun alias de colonne n'est fourni, le nom de la colonne pour une fonction renvoyant un type de données de base est identique au nom de la fonction. Pour une fonction renvoyant un type composite, les colonnes de résultat prennent les noms des attributs individuels du type.

Quelques exemples :

```
CRÉER TABLE foo (fooid int, foosubid int, fooname texte);
```

```
CRÉER UNE FONCTION getfoo(int) RETOURNE SETOF foo AS $$
  SÉLECTIONNER * DE foo OÙ fooid = $1;
$$ LANGUAGE SQL;
```

```
SÉLECTIONNER * DE getfoo(1) AS t1;
```

```
SELECT * FROM foo
  OÙ foosubid DANS (
    SELECT foosubid FROM
      getfoo(foo.fooid) z WHERE z.fooid = foo.fooid
  );
```

```
CRÉER UNE VUE vw_getfoo COMME SÉLECTIONNER * DE getfoo(1);
```

```
SÉLECTIONNER * DE vw_getfoo;
```

Dans certains cas, il est utile de définir des fonctions de table qui peuvent renvoyer différents ensembles de colonnes selon leur mode d'appel. Pour ce faire, la fonction de table peut être déclarée comme renvoyant le pseudo-type « enregistrement » sans paramètre de sortie. Lorsqu'une telle fonction est utilisée dans une requête, la structure de ligne attendue doit être spécifiée dans la requête elle-même, afin que le système puisse l'analyser et la planifier.

Cette syntaxe ressemble à ceci :

```
appel_fonction [AS] alias (définition_colonne [, ... ]) appel_fonction AS [alias] (définition_colonne [, ... ])
```

```
LIGNES DE ( ... appel_fonction AS (définition_colonne [, ... ])
  [, ... ] )
```

Lorsqu'on n'utilise pas la syntaxe ROWS FROM(), la liste de définitions de colonnes remplace la liste d'alias de colonnes qui pourrait être associée à l'élément FROM ; les noms dans les définitions de colonnes servent d'alias de colonnes. Avec la syntaxe ROWS FROM(), une liste de définitions de colonnes peut être associée à chaque fonction membre séparément ; ou, s'il n'y a qu'une seule fonction membre et aucune clause WITH ORDINALITY, une liste de définitions de colonnes peut être utilisée à la place d'une liste d'alias de colonnes après ROWS FROM().

Prenons cet exemple :

```

SÉLECTIONNER *
FROM dblink('dbname=mydb', 'SELECT proname, prosrc FROM
pg_proc')
AS t1 (nom proname, texte prosrc)
OÙ pronom COMME 'bytea%';

```

La fonction dblink (qui fait partie du module dblink) exécute une requête distante. Elle est déclarée comme renvoyant un enregistrement, car elle peut être utilisée pour tout type de requête. L'ensemble des colonnes doit être spécifié dans la requête appelante afin que l'analyseur sache, par exemple, à quoi le caractère * doit être remplacé.

Cet exemple utilise ROWS FROM :

```

SÉLECTIONNER *
À PARTIR DES LIGNES DE
(
    json_to_recordset('["a":40,"b":"foo"], {"a":"100","b":"bar"}')
    AS (a ENTIER, b TEXTE), générer_série(1,
    3)
) AS x (p, q, s)
TRIER PAR p;

p | q | s
-----+-----+---
40 | foo | 1 100 | barre
| 2 | 3
|

```

Cette fonction combine deux fonctions en une seule clause FROM. La fonction `json_to_recordset()` doit renvoyer deux colonnes : une première de type entier et une seconde de type texte. Le résultat de `generate_series()` est utilisé directement. La clause ORDER BY trie les valeurs des colonnes par ordre croissant d'entiers.

7.2.1.5. Sous-requêtes LATÉRALES

Les sous-requêtes figurant dans la clause FROM peuvent être précédées du mot-clé LATERAL. Cela leur permet de faire référence aux colonnes fournies par les éléments FROM précédents. (Sans LATERAL, chaque sous-requête est évaluée indépendamment et ne peut donc pas faire référence à un autre élément FROM.)

Les fonctions de table apparaissant dans FROM peuvent également être précédées du mot clé LATERAL, mais pour les fonctions, ce mot clé est facultatif ; les arguments de la fonction peuvent dans tous les cas contenir des références aux colonnes fournies par les éléments FROM précédents.

Un élément LATERAL peut apparaître au niveau supérieur de la liste FROM ou dans une arborescence JOIN. Dans ce dernier cas, il peut également faire référence à tout élément situé à gauche d'une JOIN dont il est à droite.

Lorsqu'un élément FROM contient des références croisées LATERAL, l'évaluation se déroule comme suit : pour chaque ligne de l'élément FROM fournissant la ou les colonnes référencées, ou pour chaque ensemble de lignes de plusieurs éléments FROM fournissant ces colonnes, l'élément LATERAL est évalué à l'aide des valeurs des colonnes de cette ligne ou de cet ensemble de lignes. Les lignes résultantes sont ensuite jointes, comme d'habitude, aux lignes à partir desquelles elles ont été calculées. Cette opération est répétée pour chaque ligne ou ensemble de lignes de la ou des tables sources des colonnes.

Un exemple trivial de LATERAL est

```
SELECT * FROM foo, LATERAL (SELECT * FROM bar WHERE bar.id = foo.bar_id) ss;
```

Ce n'est pas particulièrement utile car cela donne exactement le même résultat que la méthode plus conventionnelle.

```
SELECT * FROM foo, bar WHERE bar.id = foo.bar_id;
```

La fonction LATERAL est particulièrement utile lorsque la colonne référencée est nécessaire au calcul des lignes à joindre. Une application courante consiste à fournir une valeur d'argument à une fonction renvoyant un ensemble. Par exemple, si la fonction `vertices(polygon)` renvoie l'ensemble des sommets d'un polygone, on pourrait identifier les sommets proches des polygones stockés dans une table grâce à :

```
SÉLECTIONNER p1.id, p2.id, v1, v2 DE polygones
p1, polygones p2,
    LATERAL sommets(p1.poly) v1, LATERAL
    sommets(p2.poly) v2
OÙ (v1 <-> v2) < 10 ET p1.id != p2.id;
```

Cette requête pourrait également être écrite

```
SELECT p1.id, p2.id, v1, v2 FROM polygones p1
CROSS JOIN LATERAL vertices(p1.poly) v1, polygones p2 CROSS JOIN LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

ou selon plusieurs autres formulations équivalentes. (Comme déjà mentionné, le mot-clé LATERAL est superflu dans cet exemple, mais nous l'utilisons par souci de clarté.)

Il est souvent particulièrement pratique d'utiliser une jointure externe gauche (LEFT JOIN) avec une sous-requête latérale (LATERAL), afin que les lignes sources apparaissent dans le résultat même si la sous-requête latérale ne renvoie aucune ligne pour ces lignes. Par exemple, si la fonction `get_prod-uct_names()` renvoie les noms des produits fabriqués par un fabricant, mais que certains fabricants de notre table ne produisent actuellement aucun produit, nous pouvons identifier lesquels comme suit :

```
SÉLECTIONNER m.nom
FROM manufacturers m LEFT JOIN LATERAL get_product_names(m.id)
    pname ON vrai
OÙ pname EST NULL ;
```

7.2.2. La clause WHERE

La syntaxe de la clause WHERE est

WHERE condition de recherche

où `search_condition` est une expression de valeur quelconque (voir section 4.2) qui renvoie une valeur de type booléen.

Une fois la clause FROM traitée, chaque ligne de la table virtuelle dérivée est comparée à la condition de recherche. Si la condition est vraie, la ligne est conservée dans la table de sortie ; sinon (c'est-à-dire si le résultat est faux ou nul), elle est supprimée. La condition de recherche fait généralement référence à au moins une colonne de la table générée par la clause FROM ; cela n'est pas obligatoire, mais sans cela, la clause WHERE serait pratiquement inutile.

Note

La condition de jointure d'une jointure interne peut être écrite soit dans la clause WHERE, soit dans la clause JOIN. Par exemple, ces expressions de table sont équivalentes :

```
DE a, b OÙ a.id = b.id ET b.val > 5
```

et:

```
FROM a INNER JOIN b ON (a.id = b.id) WHERE b.val > 5
```

ou peut-être même :

```
FROM a NATURAL JOIN b WHERE b.val > 5
```

Le choix entre ces deux syntaxes est principalement une question de style. La syntaxe JOIN dans la clause FROM est probablement moins portable vers d'autres systèmes de gestion de bases de données SQL, même si elle fait partie de la norme SQL. Pour les jointures externes, il n'y a pas d'autre choix : elles doivent impérativement être effectuées dans la clause FROM. La clause ON ou USING d'une jointure externe n'est pas équivalente à une condition WHERE, car elle entraîne l'ajout de lignes (pour les lignes d'entrée non appariées) ainsi que la suppression de lignes dans le résultat final.

Voici quelques exemples de clauses WHERE :

```
SELECT ... FROM fdt WHERE c1 > 5
```

```
SÉLECTIONNER ... DE fdt OÙ c1 DANS (1, 2, 3)
```

```
SÉLECTIONNER ... DE fdt OÙ c1 DANS (SÉLECTIONNER c1 DE t2)
```

```
SÉLECTIONNER ... DE fdt OÙ c1 DANS (SÉLECTIONNER c3 DE t2 OÙ c2 =  
    fdt.c1 + 10)
```

```
SELECT ... FROM fdt WHERE c1 BETWEEN (SELECT c3 FROM t2 WHERE c2 = fdt.c1 + 10) AND 100
```

```
SÉLECTIONNER ... DE fdt OÙ EXISTE (SÉLECTIONNER c1 DE t2 OÙ c2 >  
    fdt.c1)
```

fdt est la table dérivée dans la clause FROM. Les lignes ne correspondant pas à la condition de recherche de la clause WHERE sont éliminées de fdt. Notez l'utilisation de sous-requêtes scalaires comme expressions de valeur. À l'instar de toute autre requête, les sous-requêtes peuvent employer des expressions de table complexes. Notez également comment fdt est référencée dans les sous-requêtes. Qualifier c1 comme fdt.c1 n'est nécessaire que si c1 est également le nom d'une colonne de la table d'entrée dérivée de la sous-requête. Toutefois, cette qualification apporte de la clarté, même lorsqu'elle n'est pas indispensable. Cet exemple illustre comment la portée du nommage des colonnes d'une requête externe s'étend à ses sous-requêtes.

7.2.3. Les clauses GROUP BY et HAVING

Après avoir passé le filtre WHERE, la table d'entrée dérivée peut être soumise à un regroupement, à l'aide de la clause GROUP BY, et à l'élimination des lignes de groupe à l'aide de la clause HAVING.

```
SÉLECTIONNER liste_sélection  
DEPUIS ...  
[OÙ ...]  
GROUP BY référence_colonne_de_groupage  
[, référence_colonne_de_groupement]...
```

La clause GROUP BY permet de regrouper les lignes d'un tableau qui présentent les mêmes valeurs dans toutes les colonnes listées. L'ordre des colonnes n'a pas d'importance. Son effet est de :

Regroupez les lignes ayant des valeurs communes en une seule ligne représentant toutes les lignes du groupe. Ceci permet d'éliminer les redondances dans les résultats et/ou de calculer des agrégats applicables à ces groupes. Par exemple :

=> SÉLECTIONNER * DE test1; x | y

```
---+---
a | 3 c |
2 b | 5 a
| 1 (4
lignes)
```

=> SÉLECTIONNER x DE test1 GROUPE PAR x;

```
x
---
un
b
c
(3 rangées)
```

Dans la deuxième requête, nous n'aurions pas pu écrire `SELECT * FROM test1 GROUP BY x`, car la colonne y ne contient aucune valeur unique pouvant être associée à chaque groupe. Les colonnes de regroupement peuvent être utilisées dans la clause `SELECT` puisqu'elles contiennent chacune une valeur unique.

En général, lorsqu'un tableau est groupé, les colonnes non incluses dans la clause `GROUP BY` ne peuvent être référencées que dans des expressions d'agrégation. Voici un exemple d'expression d'agrégation :

=> SÉLECTIONNER x, somme(y) DE test1 GROUPE PAR x;

```
x | somme
---+-----
a | b      4
| c |      5
(3      2
lignes)
```

La fonction `sum` est une fonction d'agrégation qui calcule une valeur unique pour l'ensemble du groupe. Vous trouverez plus d'informations sur les fonctions d'agrégation disponibles dans la section 9.21.

Conseil

Le regroupement sans expressions d'agrégation calcule l'ensemble des valeurs distinctes d'une colonne. On peut également obtenir le même résultat avec la clause `DISTINCT` (voir section 7.3.3).

Voici un autre exemple : il calcule le total des ventes pour chaque produit (plutôt que le total des ventes de tous les produits) :

```
SELECT product_id, p.name, (sum(s.units) * p.price) AS sales FROM products p LEFT JOIN sales s USING
(product_id)
GROUP BY product_id, p.name, p.price;
```

Dans cet exemple, les colonnes `product_id`, `p.name` et `p.price` doivent figurer dans la clause `GROUP BY` car elles sont référencées dans la liste de sélection de la requête (voir ci-dessous). La colonne `s.units` n'a pas besoin d'être dans la liste `GROUP BY` car elle est uniquement utilisée dans une expression d'agrégation (`sum(...)`).

qui représente les ventes d'un produit. Pour chaque produit, la requête renvoie une ligne récapitulative de toutes ses ventes.

Si la table des produits est configurée de sorte que, par exemple, `product_id` soit la clé primaire, il suffirait alors de regrouper par `product_id` dans l'exemple ci-dessus, puisque le nom et le prix dépendraient fonctionnellement de l'identifiant du produit, et il n'y aurait donc aucune ambiguïté quant à la valeur de nom et de prix à renvoyer pour chaque groupe d'identifiants de produits.

En SQL strict, la clause `GROUP BY` ne permet de regrouper que par colonnes de la table source. PostgreSQL étend cette limitation pour autoriser également le regroupement par colonnes de la liste de sélection. Le regroupement par expressions de valeur, et non par simples noms de colonnes, est également possible.

Si une table a été groupée à l'aide de `GROUP BY`, mais que seuls certains groupes vous intéressent, la clause `HAVING` peut être utilisée, de la même manière qu'une clause `WHERE`, pour exclure des groupes du résultat. La syntaxe est la suivante :

```
SELECT select_list FROM ... [WHERE ...] GROUP BY HAVING boolean_expression ...
```

Les expressions de la clause `HAVING` peuvent faire référence à la fois à des expressions groupées et à des expressions non groupées (qui impliquent nécessairement une fonction d'agrégation).

Exemple:

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING sum(y) > 3;
```

```
x | somme
---+-----
a |      4
b |      5
(2 lignes)
```

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING x < 'c';
```

```
x | somme
---+-----
a |      4
b |      5
(2 lignes)
```

Voici à nouveau un exemple plus réaliste :

```
SELECT product_id, p.name, (sum(s.units) * (p.price - p.cost)) AS
profit
FROM products p LEFT JOIN sales s USING (product_id)
WHERE s.date > CURRENT_DATE - INTERVAL '4 semaines'
GROUPER PAR product_id, p.name, p.price, p.cost HAVING sum(p.price *
s.units) > 5000;
```

Dans l'exemple ci-dessus, la clause `WHERE` sélectionne les lignes selon une colonne non groupée (l'expression n'est valable que pour les ventes des quatre dernières semaines), tandis que la clause `HAVING` limite le résultat aux groupes dont le chiffre d'affaires brut total est supérieur à 5 000. Notez que les expressions d'agrégation ne doivent pas nécessairement être identiques dans toutes les parties de la requête.

Si une requête contient des appels de fonctions d'agrégation, mais aucune clause `GROUP BY`, le regroupement a tout de même lieu : le résultat est une seule ligne de groupe (ou peut-être aucune ligne du tout, si la ligne unique est ensuite éliminée par `HAVING`). Il en va de même si elle contient une clause `HAVING`, même sans appel de fonction d'agrégation ni clause `GROUP BY`.

7.2.4. GROUPEMENTS, CUBE et ROLLUP

Des opérations de regroupement plus complexes que celles décrites précédemment sont possibles grâce au concept d'ensembles de regroupement. Les données sélectionnées par les clauses FROM et WHERE sont regroupées séparément selon chaque ensemble de regroupement spécifié. Les agrégats sont calculés pour chaque groupe, comme pour les clauses GROUP BY simples, puis les résultats sont renvoyés. Par exemple :

```
=> SÉLECTIONNER * DE items_sold; marque | taille |
      ventes
```

```
-----+-----+-----
      Foo | L           | 10 |
      Foo | M           | 20 | 15
      Bar | M           | 5  |
      Barre | L (4
rangées)
```

```
=> SÉLECTIONNER marque, taille, somme(ventes) DE articles_vendus GROUPER PAR GROUPEMENT
      ENSEMBLES ((marque), (taille), ()); marque |
      taille | somme
```

```
-----+-----+-----
      Foo |           | 30 |
      Bar |           | 20 | 15
           | L       | 35 | 50
           | M
```

```
| (5 lignes)
```

Chaque sous-liste d'ensembles de regroupement peut spécifier zéro ou plusieurs colonnes ou expressions et est interprétée de la même manière que si elle figurait directement dans la clause GROUP BY. Un ensemble de regroupement vide signifie que toutes les lignes sont agrégées en un seul groupe (qui est affiché même si aucune ligne n'était présente en entrée), comme décrit précédemment pour les fonctions d'agrégation sans clause GROUP BY.

Dans les lignes de résultats des ensembles de regroupement où ces colonnes sont absentes, les références aux colonnes ou expressions de regroupement sont remplacées par des valeurs nulles. Pour déterminer le regroupement correspondant à une ligne de résultat donnée, consultez le tableau 9.64.

Une notation abrégée est fournie pour spécifier deux types courants d'ensembles de regroupement. Une clause de la forme

```
ROLLUP ( e1, e2, e3, ... )
```

représente la liste d'expressions donnée et tous les préfixes de la liste, y compris la liste vide ; elle est donc équivalente à

```
GROUPEMENT
      D'ENSEMBLES ( ( e1, e2, e3, ... ),
      ...
      ( e1, e2 ), ( e1 ),
      ( )
    )
```

Cette méthode est couramment utilisée pour l'analyse de données hiérarchiques ; par exemple, le salaire total par département, division et le total à l'échelle de l'entreprise.

Une clause de la forme

CUBE (e1, e2, ...)

représente la liste donnée et tous ses sous-ensembles possibles (c'est-à-dire l'ensemble des parties).

CUBE (a, b, c)

est équivalent à

```

GROUPEMENT D'ENSEMBLES (
    ( a, b, c ),
    ( a, b ),
    ( a, c ),
    ( un ),
    ( b, c ),
    ( ),      b
    ( c ),
    ( )
)

```

Les éléments individuels d'une clause CUBE ou ROLLUP peuvent être soit des expressions individuelles, soit des sous-listes. des éléments entre parenthèses. Dans ce dernier cas, les sous-listes sont traitées comme des unités uniques à ces fins de la génération des ensembles de regroupement individuels. Par exemple :

CUBE ((a, b), (c, d))

est équivalent à

```

GROUPEMENT D'ENSEMBLES (
    ( a, b, c, d ),
    ( a, b ),
    ( c, d ),
    ( )
)

```

et

ROLLUP (a, (b, c), d)

est équivalent à

```

GROUPEMENT D'ENSEMBLES (
    ( a, b, c, d ),
    ( a, b, c ),
    ( un ),
    ( )
)

```

Les constructions CUBE et ROLLUP peuvent être utilisées soit directement dans la clause GROUP BY, soit de manière imbriquée. à l'intérieur d'une clause GROUPING SETS. Si une clause GROUPING SETS est imbriquée dans une autre, la L'effet est le même que si tous les éléments de la proposition interne avaient été écrits directement dans la proposition externe.

Si plusieurs éléments de regroupement sont spécifiés dans une seule clause GROUP BY, alors la liste finale des éléments de regroupement Un ensemble est le produit cartésien de ses éléments constitutifs. Par exemple :

GROUPER PAR a, CUBE (b, c), ENSEMBLES DE GROUPEMENT ((d), (e))

est équivalent à

GROUPER PAR ENSEMBLES DE

GROUPEMENT ((a, b, c, d), (a, b, c, e), (a, b,
d), (a, c, d), (a, (a, b, e), (a, c,
d), e), (a, e)

)

Lorsqu'on spécifie plusieurs éléments de regroupement, l'ensemble final de groupes de regroupement peut contenir des doublons. Par exemple :

GROUP BY ROLLUP (a, b), ROLLUP (a, c)

est équivalent à

GROUPER PAR ENSEMBLES DE GROUPEMENT (

(a, b, c), (a, b),
(a, b), (a,
c), (a), (a),
(a, c), (a),
()

)

Si ces doublons sont indésirables, ils peuvent être supprimés directement à l'aide de la clause DISTINCT dans la clause GROUP BY. Par conséquent :

GROUP BY DISTINCT ROLLUP (a, b), ROLLUP (a, c)

est équivalent à

GROUPER PAR ENSEMBLES DE GROUPEMENT (

(a, b, c), (a, b),
(a, c), (a),
()

)

Cela diffère de l'utilisation de SELECT DISTINCT, car les lignes de résultat peuvent toujours contenir des doublons. Si l'une des colonnes non groupées contient NULL, cette valeur sera identique à la valeur NULL utilisée lorsque cette même colonne est groupée.

Note

La construction (a, b) est normalement reconnue dans les expressions comme un constructeur de ligne. Dans la clause GROUP BY, cela ne s'applique pas aux niveaux supérieurs des expressions, et (a, b) est analysée.

sous forme de liste d'expressions, comme décrit ci-dessus. Si, pour une raison quelconque, vous avez besoin d'un constructeur de ligne dans une expression de regroupement, utilisez ROW(a, b).

7.2.5. Traitement de la fonction de fenêtre

Si la requête contient des fonctions de fenêtre (voir sections 3.5, 9.22 et 4.2.8), ces fonctions sont évaluées après tout regroupement, agrégation ou filtrage HAVING. Autrement dit, si la requête utilise des agrégats, GROUP BY ou HAVING, les lignes examinées par les fonctions de fenêtre sont les lignes regroupées et non les lignes de la table d'origine issues des clauses FROM/WHERE.

Lorsqu'on utilise plusieurs fonctions de fenêtrage, celles qui possèdent des clauses PARTITION BY et ORDER BY équivalentes dans leur définition de fenêtre sont assurées de traiter les lignes d'entrée dans le même ordre, même si la clause ORDER BY ne détermine pas cet ordre de manière unique. Cependant, aucune garantie n'est donnée quant au comportement des fonctions ayant des spécifications PARTITION BY ou ORDER BY différentes. (Dans ce cas, un tri est généralement nécessaire entre les évaluations des fonctions de fenêtrage, et ce tri ne garantit pas la préservation de l'ordre des lignes que sa clause ORDER BY considère comme équivalentes.)

Actuellement, les fonctions de fenêtrage requièrent toujours des données pré-triées ; par conséquent, le résultat de la requête sera trié selon l'une ou l'autre des clauses PARTITION BY/ORDER BY de ces fonctions. Il est toutefois déconseillé de s'y fier. Utilisez une clause ORDER BY explicite au niveau supérieur si vous souhaitez garantir un tri précis des résultats.

7.3. Sélectionner les listes

Comme indiqué dans la section précédente, l'expression de table de la commande SELECT construit une table virtuelle intermédiaire en combinant éventuellement des tables et des vues, en supprimant des lignes, en effectuant des regroupements, etc. Cette table est ensuite transmise au traitement par la liste de sélection. Cette dernière détermine les colonnes de la table intermédiaire qui seront effectivement affichées.

7.3.1. Éléments de la liste de sélection

Le type de liste de sélection le plus simple est *, qui renvoie toutes les colonnes produites par l'expression de table. Autrement, une liste de sélection est une liste d'expressions de valeurs séparées par des virgules (comme défini dans la section 4.2). Par exemple, il pourrait s'agir d'une liste de noms de colonnes :

SÉLECTIONNEZ a, b, c DE ...

Les noms de colonnes a, b et c correspondent soit aux noms réels des colonnes des tables référencées dans la clause FROM, soit aux alias qui leur sont attribués, comme expliqué dans la section 7.2.1.2. L'espace de noms disponible dans la liste SELECT est le même que dans la clause WHERE, sauf si un regroupement est utilisé, auquel cas il est le même que dans la clause HAVING.

Si plusieurs tables possèdent une colonne portant le même nom, le nom de la table doit également être indiqué, comme dans l'exemple suivant :

SÉLECTIONNER tbl1.a, tbl2.a, tbl1.b DE ...

Lorsqu'on travaille avec plusieurs tables, il peut également être utile de demander toutes les colonnes d'une table particulière :

SÉLECTIONNER tbl1.*, tbl2.a DE ...

Voir la section 8.16.5 pour plus d'informations sur la notation table_name.*.

Si une expression de valeur arbitraire est utilisée dans la liste de sélection, elle ajoute conceptuellement une nouvelle colonne virtuelle à la table renvoyée. L'expression de valeur est évaluée une fois pour chaque ligne de résultat, avec les valeurs de la ligne.

Les expressions de la liste SELECT ne font pas nécessairement référence à des colonnes de la table définie dans la clause FROM ; il peut s'agir, par exemple, d'expressions arithmétiques constantes.

7.3.2. Étiquettes de colonnes

Les éléments de la liste de sélection peuvent se voir attribuer des noms en vue d'un traitement ultérieur, par exemple pour être utilisés dans une clause ORDER BY ou pour être affichés par l'application cliente. Par exemple :

SÉLECTIONNEZ a AS valeur, b + c AS somme DE ...

Si aucun nom de colonne de sortie n'est spécifié via AS, le système attribue un nom par défaut. Pour les références de colonnes simples, il s'agit du nom de la colonne référencée. Pour les appels de fonction, il s'agit du nom de la fonction. Pour les expressions complexes, le système génère un nom générique.

Le mot-clé AS est généralement facultatif, mais dans certains cas où le nom de colonne souhaité correspond à un mot-clé PostgreSQL, il est impératif d'écrire AS ou d'encadrer le nom de colonne de guillemets doubles afin d'éviter toute ambiguïté. (L'annexe C indique les mots-clés qui requièrent l'utilisation de AS comme étiquette de colonne.) Par exemple, FROM est un de ces mots-clés ; par conséquent, la syntaxe suivante ne fonctionnera pas :

SELECT a FROM, b + c AS somme FROM ...

mais l'un ou l'autre le fait :

SELECT a AS from, b + c AS sum FROM ...

SELECT a "from", b + c AS sum FROM ...

Pour une sécurité maximale contre d'éventuels ajouts futurs de mots clés, il est recommandé d'écrire systématiquement AS ou de mettre entre guillemets le nom de la colonne de sortie.

Note

La dénomination des colonnes de sortie diffère ici de celle utilisée dans la clause FROM (voir section 7.2.1.2). Il est possible de renommer deux fois la même colonne, mais seul le nom indiqué dans la liste SELECT sera conservé.

7.3.3. DISTINCT

Une fois la liste de sélection traitée, le tableau de résultats peut être soumis, en option, à l'élimination des lignes en double. Le mot-clé DISTINCT est placé directement après SELECT pour spécifier cette opération.

SÉLECTIONNER DISTINCT select_list ...

(Au lieu de DISTINCT, le mot-clé ALL peut être utilisé pour spécifier le comportement par défaut consistant à conserver toutes les lignes.)

Deux lignes sont considérées comme distinctes si elles diffèrent par au moins une valeur de colonne. Les valeurs nulles sont considérées comme égales dans cette comparaison.

Alternativement, une expression arbitraire peut déterminer quelles lignes doivent être considérées comme distinctes :

```
SELECT DISTINCT ON (expression [, expression ...]) select_list ...
```

Ici, l'expression représente une valeur arbitraire évaluée pour chaque ligne. Les lignes pour lesquelles toutes les expressions sont égales sont considérées comme des doublons, et seule la première ligne de l'ensemble est conservée dans le résultat. Notez que la « première ligne » d'un ensemble est imprévisible, sauf si la requête est triée sur suffisamment de colonnes pour garantir un ordre unique des lignes arrivant au filtre DISTINCT. (Le traitement DISTINCT ON intervient après le tri ORDER BY.)

La clause DISTINCT ON ne fait pas partie de la norme SQL et est parfois considérée comme une mauvaise pratique en raison du caractère potentiellement indéterminé de ses résultats. En utilisant judicieusement GROUP BY et les sous-requêtes dans FROM, on peut éviter cette construction, mais elle reste souvent l'alternative la plus pratique.

7.4. Combinaison de requêtes (UNION, INTERSECT, EXCEPT)

Les résultats de deux requêtes peuvent être combinés à l'aide des opérations ensemblistes union, intersection et différence. La syntaxe est

```
requête1 UNION [TOUS] requête2 requête1  
INTERSECT [TOUS] requête2 requête1 EXCEPT  
[TOUS] requête2
```

où query1 et query2 sont des requêtes qui peuvent utiliser n'importe laquelle des fonctionnalités abordées jusqu'à présent.

L'opérateur UNION ajoute le résultat de la requête 2 à celui de la requête 1 (sans garantie que l'ordre de retour des lignes soit respecté). De plus, il élimine les doublons, comme l'opérateur DISTINCT, sauf si UNION ALL est utilisé.

INTERSECT renvoie toutes les lignes qui figurent à la fois dans le résultat de la requête 1 et dans le résultat de la requête 2. Les lignes en double sont éliminées sauf si l'option INTERSECT ALL est utilisée.

La clause EXCEPT renvoie toutes les lignes présentes dans le résultat de la requête 1 mais absentes du résultat de la requête 2. (On parle parfois de différence entre deux requêtes.) Là encore, les doublons sont éliminés, sauf si l'option EXCEPT ALL est utilisée.

Pour calculer l'union, l'intersection ou la différence de deux requêtes, les deux requêtes doivent être « compatibles pour l'union », ce qui signifie qu'elles renvoient le même nombre de colonnes et que les colonnes correspondantes ont des types de données compatibles, comme décrit dans la section 10.5.

Les opérations ensemblistes peuvent être combinées, par exemple

```
requête 1 UNION requête 2 SAUF requête 3
```

ce qui équivaut à

```
(requête 1 UNION requête 2) SAUF requête 3
```

Comme illustré ici, vous pouvez utiliser des parenthèses pour contrôler l'ordre d'évaluation. Sans parenthèses, UNION et EXCEPT s'associent de gauche à droite, mais INTERSECT établit une liaison plus forte que ces deux opérateurs.

```
requête 1 UNION requête 2 INTERSECT requête 3
```

moyens

requête1 UNION (requête2 INTERSECT requête3)

Vous pouvez également encadrer une requête individuelle de parenthèses. Ceci est important si la requête doit utiliser l'une des clauses abordées dans les sections suivantes, comme LIMIT. Sans parenthèses, vous obtiendrez une erreur de syntaxe, ou bien la clause sera interprétée comme s'appliquant à la sortie de l'opération ensembliste plutôt qu'à l'une de ses entrées. Par exemple :

```
SELECT a FROM b UNION SELECT x FROM y LIMIT 10
```

est accepté, mais cela signifie

```
(SELECT a FROM b UNION SELECT x FROM y) LIMIT 10
```

pas

```
SELECT a FROM b UNION (SELECT x FROM y LIMIT 10)
```

7.5. Tri des lignes (ORDER BY)

Une fois la table de résultats générée par une requête (après traitement de la liste de sélection), il est possible de la trier. Si aucun tri n'est effectué, les lignes seront renvoyées dans un ordre aléatoire. Cet ordre dépendra des types de plans d'analyse et de jointure, ainsi que de l'ordre des données sur le disque, mais il ne doit pas être considéré comme acquis. Un ordre de sortie particulier n'est garanti que si l'étape de tri est explicitement activée.

La clause ORDER BY spécifie l'ordre de tri :

```
SÉLECTIONNER liste_sélection
  À PARTIR DE l'expression_table
  TRIER PAR expression_sort1 [ASC | DESC] [NULLS { PREMIER | NOM }]
    [, expression_sort_2 [ASC | DESC] [NULLS { PREMIER | NOM }] ...]
```

L'expression de tri peut être n'importe quelle expression valide dans la liste de sélection de la requête. Par exemple :

```
SÉLECTIONNER a, b DE table1 TRIER PAR a + b, c;
```

Lorsqu'on spécifie plusieurs expressions, les dernières valeurs servent à trier les lignes égales selon les premières. Chaque expression peut être suivie des mots-clés optionnels ASC ou DESC pour définir le sens du tri (croissant ou décroissant). Le tri ASC est le tri par défaut. Le tri croissant place les valeurs les plus petites en premier, « plus petites » étant défini par l'opérateur <. De même, le tri décroissant est déterminé par l'opérateur >.

Les options NULLS FIRST et NULLS LAST permettent de déterminer si les valeurs nulles apparaissent avant ou après les valeurs non nulles dans l'ordre de tri. Par défaut, les valeurs nulles sont triées comme si elles étaient supérieures à toute valeur non nulle ; autrement dit, NULLS FIRST est l'option par défaut pour un tri DESC, et NULLS LAST dans le cas contraire.

Notez que les options de tri sont considérées indépendamment pour chaque colonne de tri. Par exemple, ORDER BY x, y DESC signifie ORDER BY x ASC, y DESC, ce qui est différent de ORDER BY x DESC, y DESC.

¹ En réalité, PostgreSQL utilise la classe d'opérateurs B-tree par défaut pour le type de données de l'expression afin de déterminer l'ordre de tri pour ASC et DESC.

Par convention, les types de données sont configurés de sorte que les opérateurs < et > correspondent à cet ordre de tri, mais le concepteur d'un type de données défini par l'utilisateur peut choisir de faire quelque chose de différent.

Une expression de tri peut également être l'étiquette ou le numéro d'une colonne de sortie, comme dans :

```
SÉLECTIONNER a + b AS somme, c DE table1 TRIER PAR somme;
SÉLECTIONNER a, max(b) DE table1 GROUPER PAR a TRIER PAR 1;
```

Ces deux méthodes de tri s'effectuent selon la première colonne de sortie. Notez qu'un nom de colonne de sortie doit être utilisé seul, c'est-à-dire qu'il ne peut pas être inclus dans une expression ; par exemple, ceci est incorrect :

```
SÉLECTIONNER a + b AS somme, c DE table1 TRIER PAR somme + c;
faux
```

Cette restriction vise à réduire l'ambiguïté. Une ambiguïté subsiste toutefois si un critère ORDER BY est un nom simple pouvant correspondre soit au nom d'une colonne de sortie, soit à celui d'une colonne de l'expression de table. Dans ce cas, c'est la colonne de sortie qui est utilisée. Cela ne prêterait à confusion que si vous utilisiez AS pour renommer une colonne de sortie afin qu'elle corresponde au nom d'une autre colonne de la table.

La clause ORDER BY peut être appliquée au résultat d'une combinaison UNION, INTERSECT ou EXCEPT, mais dans ce cas, elle ne peut trier que par noms ou numéros de colonnes de sortie, et non par expressions.

7.6. LIMITE ET COMPENSATION

Les fonctions LIMIT et OFFSET vous permettent de récupérer uniquement une partie des lignes générées par le reste de la requête :

```
SELECT select_list FROM
    table_expression [ ORDER BY ... ]

    [ LIMIT { nombre | TOUS } ]
    [ DÉCALAGE début ]
```

Si un nombre limite est spécifié, le nombre de lignes retournées ne dépassera pas cette valeur (voire sera inférieur si la requête elle-même produit moins de lignes). LIMIT ALL est équivalent à l'omission de la clause LIMIT, de même que LIMIT avec un argument NULL.

La fonction OFFSET indique d'ignorer le nombre de lignes spécifié avant de commencer à renvoyer les lignes. OFFSET 0 est équivalent à l'omission de la clause OFFSET, de même que OFFSET avec un argument NULL.

Si les valeurs OFFSET et LIMIT apparaissent toutes les deux, les lignes OFFSET sont ignorées avant de commencer à compter les lignes LIMIT renvoyées.

Lorsqu'on utilise LIMIT, il est important d'utiliser une clause ORDER BY afin de contraindre les lignes de résultat à un ordre unique. Sinon, le résultat de la requête sera un sous-ensemble imprévisible des lignes. Par exemple, si vous demandez les lignes 10 à 20, dans quel ordre ? L'ordre est inconnu, sauf si vous avez spécifié ORDER BY.

L'optimiseur de requêtes tient compte de LIMIT lors de la génération des plans d'exécution. Par conséquent, vous obtiendrez probablement des plans différents (entraînant des ordres de lignes différents) selon les valeurs de LIMIT et OFFSET. Ainsi, l'utilisation de valeurs LIMIT/OFFSET différentes pour sélectionner différents sous-ensembles de résultats de requête produira des résultats incohérents, sauf si vous imposez un ordre de résultats prévisible avec ORDER BY. Il ne s'agit pas d'un bug ; c'est une conséquence inhérente au fait que SQL ne garantit pas l'ordre des résultats d'une requête, sauf si ORDER BY est utilisé pour contraindre cet ordre.

Les lignes ignorées par une clause OFFSET doivent tout de même être calculées sur le serveur ; par conséquent, une valeur OFFSET importante peut s'avérer inefficace.

7.7. VALEURS Listes

VALUES permet de générer une « table constante » utilisable dans une requête sans avoir à créer et remplir une table sur disque. La syntaxe est la suivante :

VALEURS (expression [, ...]) [, ...]

Chaque liste d'expressions entre parenthèses génère une ligne dans le tableau. Toutes les listes doivent comporter le même nombre d'éléments (c'est-à-dire le nombre de colonnes du tableau), et les éléments correspondants de chaque liste doivent avoir des types de données compatibles. Le type de données attribué à chaque colonne du résultat est déterminé selon les mêmes règles que pour l'opérateur UNION (voir section 10.5).

Par exemple :

VALEURS (1, 'un'), (2, 'deux'), (3, 'trois');

renverra un tableau de deux colonnes et trois lignes. Cela équivaut concrètement à :

```
SELECT 1 AS colonne1, 'un' AS colonne2
UNION TOUS
SÉLECTIONNER 2, 'deux'
UNION TOUS
SÉLECTIONNER 3, 'trois';
```

Par défaut, PostgreSQL attribue les noms column1, column2, etc. aux colonnes d'une table VALUES. Ces noms de colonnes ne sont pas spécifiés par la norme SQL et varient selon les systèmes de bases de données ; il est donc généralement préférable de les remplacer par défaut par une liste d'alias de table, comme ceci :

```
=> SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t
      (nombre, lettre);
nombre | lettre
-----+-----
      1 | un 2 |
      2 | deux 3 |
      3 | trois (3 rangées)
```

Du point de vue syntaxique, VALUES suivi de listes d'expressions est considéré comme équivalent à :

```
SELECT select_list FROM table_expression
```

et peut apparaître partout où une instruction SELECT peut apparaître. Par exemple, vous pouvez l'utiliser dans une UNION ou lui associer une clause de tri (ORDER BY, LIMIT et/ou OFFSET). VALUES est le plus souvent utilisé comme source de données dans une commande INSERT, puis comme sous-requête.

Pour plus d'informations, consultez la section VALEURS.

7.8. Requêtes WITH (Expressions de tables communes)

L'opérateur WITH permet d'insérer des instructions auxiliaires dans une requête plus complexe. Ces instructions, souvent appelées expressions de table communes (CTE), définissent des tables temporaires qui n'existent que pour une seule requête. Chaque instruction auxiliaire d'une clause WITH peut être une instruction SELECT, INSERT, UPDATE, DELETE ou MERGE ; la clause WITH elle-même est associée à une instruction principale qui peut également être une instruction SELECT, INSERT, UPDATE, DELETE ou MERGE.

7.8.1. SÉLECTIONNER DANS AVEC

L'intérêt principal de SELECT dans WITH est de décomposer les requêtes complexes en parties plus simples. Par exemple :

```
AVEC les ventes régionales COMME (
    SÉLECTIONNER la région, SOMME(montant) AS ventes_totales
    À PARTIR DES commandes
    GROUP BY région),
top_regions AS (
    SÉLECTIONNER la région
    À PARTIR des ventes régionales
    OÙ total_ventes > (SÉLECTIONNER SOMME(total_ventes)/10 DE ventes_régionales) )
```

```
SÉLECTIONNER la
    région, le produit,
    SOMME(quantité) AS unités_produit,
    SOMME(montant) AS ventes_produits
À PARTIR DES commandes
OÙ région DANS (SÉLECTIONNER région DE top_regions)
GROUPE PAR région, produit ;
```

Cette requête affiche le total des ventes par produit uniquement dans les régions les plus performantes. La clause WITH définit deux instructions auxiliaires : regional_sales et top_regions. Le résultat de regional_sales est utilisé dans top_regions, et celui de top_regions dans la requête SELECT principale. Cet exemple aurait pu être écrit sans WITH, mais cela aurait nécessité deux niveaux de sous-requêtes SELECT imbriquées. Cette version est plus lisible.

7.8.2. Requêtes récursives

Le modificateur optionnel RECURSIVE transforme WITH, d'une simple commodité syntaxique, en une fonctionnalité permettant de réaliser des opérations impossibles en SQL standard. Grâce à RECURSIVE, une requête WITH peut faire référence à son propre résultat. Voici un exemple très simple : cette requête calcule la somme des entiers de 1 à 100 :

```
AVEC RÉCURSIF t(n) COMME (
    VALEURS (1)
    UNION TOUS
    SÉLECTIONNER n+1 DE t OÙ n < 100
)
SÉLECTIONNER la somme(n) DE t;
```

La forme générale d'une requête WITH récursive est toujours la suivante : un terme non récursif, suivi de UNION (ou UNION ALL), puis d'un terme récursif, seul ce dernier pouvant contenir une référence au résultat de la requête. Une telle requête s'exécute comme suit :

Évaluation récursive des requêtes

1. Évaluer le terme non récursif. Pour UNION (mais pas UNION ALL), supprimer les lignes en double.
Incluez toutes les lignes restantes dans le résultat de la requête récursive et placez-les également dans une table de travail temporaire.
2. Tant que la table de travail n'est pas vide, répétez ces étapes :
 - a. Évaluez le terme récursif en remplaçant la référence récursive par le contenu actuel de la table de travail. Pour UNION (mais pas UNION ALL), supprimez les lignes en double et

Supprimer les lignes qui font doublon avec une ligne de résultat précédente. Inclure toutes les lignes restantes dans le résultat de la requête récursive et les placer également dans une table intermédiaire temporaire.

- b. Remplacez le contenu du tableau de travail par le contenu du tableau intermédiaire, puis vider le tableau intermédiaire.

Note

Bien que RECURSIVE permette de spécifier des requêtes de manière récursive, ces requêtes sont évaluées en interne de manière itérative.

Dans l'exemple ci-dessus, la table de travail ne contient qu'une seule ligne à chaque étape et prend successivement les valeurs de 1 à 100. À la 100e étape, aucun résultat n'est affiché en raison de la clause WHERE, et la requête s'arrête donc.

Les requêtes récursives sont généralement utilisées pour traiter des données hiérarchiques ou arborescentes. Un exemple utile est cette requête permettant de trouver toutes les sous-parties directes et indirectes d'un produit, à partir d'un tableau présentant uniquement les éléments inclus :

```

AVEC RÉCURSIONNEL inclus_parties(sous_partie, partie, quantité) COMME (
    SELECT sub_part, part, quantity FROM parts WHERE part = 'our_product'

    UNION TOUS
    SELECT p.sub_part, p.part, p.quantity * pr.quantity FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part

)
SELECT sub_part, SUM(quantité) as quantité_totale
DE included_parts
GROUPE PAR sous-partie

```

7.8.2.1. Ordre de recherche

Lors du parcours d'un arbre à l'aide d'une requête récursive, il peut être utile de trier les résultats selon un parcours en profondeur ou en largeur. Pour ce faire, on peut calculer une colonne de tri en plus des autres colonnes de données, puis l'utiliser pour trier les résultats à la fin. Il est important de noter que cela ne détermine pas l'ordre dans lequel la requête parcourt les lignes ; cet ordre dépend, comme toujours en SQL, de l'implémentation. Cette approche offre simplement un moyen pratique de trier les résultats a posteriori.

Pour créer un parcours en profondeur, nous calculons pour chaque ligne de résultat un tableau des lignes déjà visitées. Par exemple, considérons la requête suivante qui effectue une recherche dans une arborescence de table à l'aide d'un champ de lien :

```

AVEC RECHERCHE_arbre_réursive(id, lien, données) AS ( SÉLECTIONNER t.id, t.lien,
    t.données
    DE l'arbre t
    UNION TOUS
    SELECT t.id, t.link, t.data FROM tree t, search_tree
    st WHERE t.id = st.link

)
SÉLECTIONNER * DE search_tree;

```

Pour ajouter des informations sur le parcours en profondeur, vous pouvez écrire ceci :

```

AVEC RECURSIVE search_tree(id, link, data, path) AS ( SELECT t.id, t.link, t.data, ARRAY[t.id]

    DE l'arbre t
  UNION TOUS
    SELECT t.id, t.link, t.data, path || t.id FROM tree t, search_tree st WHERE t.id =
    st.link

)

SÉLECTIONNER * DE search_tree TRIER PAR chemin;

```

Dans le cas général où plusieurs champs doivent être utilisés pour identifier une ligne, utilisez un tableau de lignes.
Par exemple, si nous devons suivre les champs f1 et f2 :

```

AVEC recherche_arbre_réursive(id, lien, données, chemin) COMME (
    SÉLECTIONNER t.id, t.link, t.data, ARRAY[ROW(t.f1, t.f2)]
  DE l'arbre t
  UNION TOUS
    SÉLECTIONNER t.id, t.link, t.data, path || LIGNE(t.f1, t.f2)
  DE l'arbre t, recherche_arbre st OÙ t.id = st.link

)

SÉLECTIONNER * DE search_tree TRIER PAR chemin;

```

Conseil

Omettez la syntaxe ROW() dans le cas courant où un seul champ doit être suivi. Cela permet d'utiliser un tableau simple plutôt qu'un tableau composite, ce qui améliore l'efficacité.

Pour créer un tri en largeur, vous pouvez ajouter une colonne qui suit la profondeur de la recherche, par exemple :

```

AVEC RECURSIVE search_tree(id, link, data, depth) AS ( SELECT t.id, t.link, t.data, 0

    DE l'arbre t
  UNION TOUS
    SELECT t.id, t.link, t.data, depth + 1 FROM tree t, search_tree st WHERE
    t.id = st.link

)

SÉLECTIONNER * DE search_tree TRIER PAR profondeur;

```

Pour obtenir un tri stable, ajoutez des colonnes de données comme colonnes de tri secondaires.

Conseil

L'algorithme d'évaluation récursive des requêtes produit son résultat selon un ordre de recherche en largeur. Toutefois, il s'agit d'un détail d'implémentation et il est peut-être imprudent de s'y fier. L'ordre des lignes au sein de chaque niveau est assurément indéfini ; un ordre explicite pourrait donc s'avérer nécessaire.

Il existe une syntaxe intégrée pour calculer une colonne de tri en profondeur ou en largeur. Par exemple :

```

AVEC recherche_arbre_réursive(id, lien, données) COMME (

```

```

        SÉLECTIONNER t.id, t.link, t.data
    DE l'arbre t
    UNION TOUS
        SELECT t.id, t.link, t.data FROM tree t, search_tree
        st WHERE t.id = st.link

```

) RECHERCHE EN PROFONDEUR D'ABORD PAR id DÉFINIR la colonne de commande

SÉLECTIONNER * DE search_tree TRIER PAR ordercol;

```

    AVEC RECHERCHE_arbre_réursive(id, lien, données) AS ( SÉLECTIONNER t.id, t.lien,
        t.données
    DE l'arbre t
    UNION TOUS
        SELECT t.id, t.link, t.data FROM tree t, search_tree
        st WHERE t.id = st.link

```

) RECHERCHE EN LARGE D'ABORD PAR id DÉFINIR la colonne d'ordre

SÉLECTIONNER * DE search_tree TRIER PAR ordercol;

Cette syntaxe est développée en interne pour se rapprocher des formes manuscrites ci-dessus. La clause SEARCH spécifie le type de recherche (en profondeur ou en largeur), la liste des colonnes à suivre pour le tri et le nom de la colonne contenant les données de résultat utilisables pour le tri. Cette colonne sera automatiquement ajoutée aux lignes de sortie de l'expression de table commune (CTE).

7.8.2.2. Détection de cycle

Lorsqu'on travaille avec des requêtes récursives, il est important de s'assurer que la partie récursive de la requête ne renvoie aucun tuple, sous peine de provoquer une boucle infinie. Parfois, l'utilisation de UNION au lieu de UNION ALL permet d'y parvenir en ignorant les lignes qui dupliquent les résultats précédents. Cependant, il arrive souvent qu'une boucle ne comporte pas de lignes de résultat parfaitement identiques : il peut être nécessaire de vérifier un ou quelques champs seulement pour déterminer si le même point a déjà été atteint. La méthode standard pour gérer ces situations consiste à calculer un tableau des valeurs déjà visitées. Prenons l'exemple de la requête suivante, qui effectue une recherche dans un graphe tabulaire à l'aide d'un champ de lien :

```

    AVEC recherche_graphe_réursive(id, lien, données, profondeur) COMME (
        SELECT g.id, g.link, g.data, 0 FROM graph g UNION ALL

        SELECT g.id, g.link, g.data, sg.depth + 1 FROM graph g, search_graph sg
        WHERE g.id = sg.link

    )
    SÉLECTIONNER * DE search_graph;

```

Cette requête bouclera si les relations de liens contiennent des cycles. Comme nous avons besoin d'un résultat détaillé, remplacer simplement UNION ALL par UNION ne suffirait pas à éliminer la boucle. Il nous faut donc identifier si nous atteignons à nouveau la même ligne en suivant un chemin de liens particulier. Nous ajoutons deux colonnes, is_cycle et path, à la requête sujette aux boucles :

```

    AVEC RECHERCHE RÉCURSIVE_graphe(id, lien, données, profondeur, est_cycle, chemin)
    AS
        ( SELECT g.id, g.link, g.data, 0, false, ARRAY[g.id]

        À PARTIR DU GRAPHIQUE
        g UNION TOUT

        SÉLECTIONNER g.id, g.link, g.data, sg.depth + 1,

```

```

        g.id = ANY(path), path || g.id
    FROM graph g,
    search_graph sg WHERE g.id = sg.link AND NOT
    is_cycle
)
SÉLECTIONNER * DE search_graph;

```

Outre la prévention des cycles, la valeur du tableau est souvent utile en soi, car elle représente le « chemin » emprunté pour atteindre une ligne particulière.

Dans le cas général où plusieurs champs doivent être vérifiés pour détecter un cycle, utilisez un tableau de lignes. Par exemple, si nous devons comparer les champs f1 et f2 :

```

AVEC RECHERCHE RÉCURSIVE_graphe(id, lien, données, profondeur, est_cycle, chemin)
AS
    ( SELECT g.id, g.link, g.data, 0, false, ARRAY[ROW(g.f1,
        g.f2)]

        D'après le graphique g
    UNION TOUS
        SÉLECTIONNER g.id, g.link, g.data, sg.depth + 1,
            LIGNE(g.f1, g.f2) = N'IMPORTE QUELLE(chemin),
            chemin || LIGNE(g.f1, g.f2)
        À partir du graphe g, recherche_graphe sg OÙ g.id =
            sg.link ET NON is_cycle
    )
SÉLECTIONNER * DE search_graph;

```

Conseil

Omettez la syntaxe ROW() dans le cas courant où il suffit de vérifier un seul champ pour détecter un cycle. Cela permet d'utiliser un tableau simple plutôt qu'un tableau composite, ce qui améliore l'efficacité.

Il existe une syntaxe intégrée pour simplifier la détection des cycles. La requête ci-dessus peut également s'écrire ainsi :

```

AVEC recherche_graphe RÉCURSIVE(id, lien, données, profondeur) COMME (
    SELECT g.id, g.link, g.data, 1 FROM graph g UNION ALL

    SELECT g.id, g.link, g.data, sg.depth + 1 FROM graph g, search_graph sg
    WHERE g.id = sg.link ) CYCLE id SET is_cycle
    USING path SELECT * FROM
search_graph;

```

Elle sera ensuite réécrite en interne sous la forme ci-dessus. La clause CYCLE spécifie d'abord la liste des colonnes à suivre pour la détection de cycles, puis le nom d'une colonne indiquant si un cycle a été détecté, et enfin le nom d'une autre colonne permettant de suivre le chemin. Les colonnes « cycle » et « chemin » seront automatiquement ajoutées aux lignes de sortie de l'expression de table commune (CTE).

Conseil

La colonne de chemin cyclique est calculée de la même manière que la colonne de tri en profondeur présentée dans la section précédente. Une requête peut comporter à la fois une clause SEARCH et une clause CYCLE, mais une clause de tri en profondeur ne peut pas être utilisée.

Spécifier à la fois la recherche et la détection de cycles engendrerait des calculs redondants ; il est donc plus efficace d'utiliser uniquement la clause CYCLE et de trier par la colonne de chemin. Si un parcours en largeur est souhaité, spécifier à la fois SEARCH et CYCLE peut s'avérer utile.

Une astuce utile pour tester des requêtes lorsque vous n'êtes pas certain qu'elles puissent boucler consiste à insérer une clause LIMIT dans la requête parente. Par exemple, cette requête bouclerait indéfiniment sans la clause LIMIT :

```
AVEC RÉCURSIF t(n) COMME (
  SÉLECTIONNER 1
  UNION TOUS
  SÉLECTIONNER n+1 DE t
)
SÉLECTIONNER n DE t LIMITE 100;
```

Cela fonctionne car l'implémentation de PostgreSQL n'évalue que les lignes d'une requête WITH effectivement récupérées par la requête parente. Il est déconseillé d'utiliser cette astuce en production, car d'autres systèmes peuvent se comporter différemment. De plus, cela ne fonctionnera généralement pas si la requête externe trie les résultats de la requête récursive ou les joint à une autre table, car dans ce cas, elle tentera généralement de récupérer l'intégralité du résultat de la requête WITH.

7.8.3. Matérialisation des expressions de table communes

Une propriété utile des requêtes WITH est qu'elles ne sont normalement évaluées qu'une seule fois par exécution de la requête parente, même si elles sont référencées plusieurs fois par la requête parente ou par des requêtes WITH sœurs. Ainsi, les calculs complexes nécessaires à plusieurs endroits peuvent être placés dans une requête WITH afin d'éviter les redondances. Une autre application possible consiste à empêcher les évaluations multiples indésirables de fonctions à effets de bord. Cependant, l'optimiseur ne peut pas propager les restrictions de la requête parente à une requête WITH référencée à plusieurs reprises, car cela pourrait affecter toutes les utilisations de la sortie de la requête WITH alors qu'une seule devrait en affecter une. La requête WITH référencée à plusieurs reprises sera évaluée telle quelle, sans suppression des lignes que la requête parente pourrait ignorer ultérieurement. (Cependant, comme mentionné précédemment, l'évaluation peut s'arrêter prématurément si la ou les références à la requête ne nécessitent qu'un nombre limité de lignes.)

Toutefois, si une requête WITH est non récursive et sans effet de bord (c'est-à-dire une requête SELECT ne contenant aucune fonction volatile), elle peut être intégrée à la requête parente, permettant ainsi une optimisation conjointe des deux niveaux de requête. Par défaut, cela se produit si la requête parente référence la requête WITH une seule fois, mais pas si elle la référence plusieurs fois. Vous pouvez modifier ce comportement en spécifiant MATERIALIZED pour forcer le calcul séparé de la requête WITH, ou en spécifiant NOT MATERIALIZED pour forcer son intégration à la requête parente. Cette dernière option risque d'entraîner un calcul en double de la requête WITH, mais peut tout de même générer des économies si chaque utilisation de la requête WITH ne nécessite qu'une petite partie de son résultat total.

Un exemple simple de ces règles est

```
AVEC w COMME (
  SÉLECTIONNER * DE big_table
)
SÉLECTIONNER * DE w OÙ clé = 123;
```

Cette requête WITH sera supprimée, produisant le même plan d'exécution que

```
SELECT * FROM big_table WHERE key = 123;
```

En particulier, s'il existe un index sur la clé, il sera probablement utilisé pour récupérer uniquement les lignes dont la clé est égale à 123. Par ailleurs, dans

```

AVEC w COMME (
    SÉLECTIONNER * DE big_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref WHERE w2.key = 123;

```

La requête WITH sera matérialisée, produisant une copie temporaire de big_table qui sera ensuite jointe à elle-même, sans bénéficier d'aucun index. Cette requête sera exécutée beaucoup plus efficacement si elle est écrite comme suit :

```

AVEC w COMME NON MATÉRIALISÉ (
    SÉLECTIONNER * DE big_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref WHERE w2.key = 123;

```

afin que les restrictions de la requête parente puissent être appliquées directement aux analyses de big_table.

Un exemple où l'état NON MATÉRIALISÉ pourrait être indésirable est

```

AVEC w COMME (
    SELECT key, very_expensive_function(val) as f FROM some_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.f = w2.f;

```

Ici, la matérialisation de la requête WITH garantit que very_expensive_function n'est évaluée qu'une seule fois par ligne de table, et non deux fois.

Les exemples ci-dessus montrent uniquement l'utilisation de WITH avec SELECT, mais il peut être utilisé de la même manière avec INSERT, UPDATE, DELETE ou MERGE. Dans chaque cas, il crée une ou plusieurs tables temporaires qui peuvent être référencées dans la commande principale.

7.8.4. Instructions de modification de données dans WITH

Vous pouvez utiliser des instructions de modification de données (INSERT, UPDATE, DELETE ou MERGE) dans WITH. Cela vous permet d'effectuer plusieurs opérations différentes dans une même requête. Voici un exemple :

```

AVEC lignes_déplacées COMME (
    SUPPRIMER DES produits
    OÙ
        "date" >= '2010-10-01' ET
        "date" < '2010-11-01'
    RETOUR *
)
INSERT INTO products_log
SÉLECTIONNER * DE lignes déplacées;

```

Cette requête déplace les lignes de la table 'products' vers 'products_log'. L'instruction 'DELETE' dans 'WITH' supprime les lignes spécifiées de 'products', en retournant leur contenu grâce à sa clause 'RETURNING'; puis la requête principale lit ce résultat et l'insère dans 'products_log'.

Un point important de l'exemple ci-dessus est que la clause WITH est rattachée à l'instruction INSERT, et non à la sous-requête SELECT qu'elle contient. Ceci est nécessaire car les instructions modifiant les données ne sont autorisées que dans les clauses WITH rattachées à l'instruction principale. Cependant, les règles de visibilité habituelles de WITH s'appliquent; il est donc possible de faire référence au résultat de l'instruction WITH depuis la sous-requête SELECT.

Les instructions de modification de données dans WITH comportent généralement une clause RETURNING (voir section 6.4), comme illustré dans l'exemple ci-dessus. C'est le résultat de la clause RETURNING, et non la table cible de l'instruction de modification, qui constitue la table temporaire accessible par la suite de la requête. Si une instruction de modification de données dans WITH ne comporte pas de clause RETURNING, elle ne crée aucune table temporaire et ne peut être référencée dans la suite de la requête. Cette instruction sera néanmoins exécutée. Voici un exemple peu pertinent :

```
AVEC t COMME (
    SUPPRIMER DE foo
)
SUPPRIMER DE la barre ;
```

Cet exemple supprimerait toutes les lignes des tables foo et bar. Le nombre de lignes affectées communiqué au client ne concernerait que les lignes supprimées de la table bar.

Les auto-références récursives dans les instructions de modification de données ne sont pas autorisées. Dans certains cas, il est possible de contourner cette limitation en faisant référence au résultat d'une instruction WITH récursive, par exemple :

```
AVEC RÉCURSIONNEL inclus_parties(sous_partie, partie) COMME (
    SELECT sub_part, part FROM parts WHERE part = 'our_product'
    UNION TOUS
    SELECT p.sub_part, p.part FROM included_parts
    pr, parts p WHERE p.part = pr.sub_part
)
SUPPRIMER DES parties
    OÙ partie DANS (SÉLECTIONNER partie DE parties_incluses);
```

Cette requête permettrait de supprimer toutes les sous-parties directes et indirectes d'un produit.

Les instructions modifiant les données dans WITH sont exécutées une seule fois, et toujours jusqu'à leur terme, que la requête principale lise ou non la totalité (ou une partie) de leur résultat. Notez que cela diffère de la règle applicable aux instructions SELECT dans WITH : comme indiqué dans la section précédente, l'exécution d'une instruction SELECT ne se poursuit que jusqu'à ce que la requête principale exige son résultat.

Les sous-instructions de WITH sont exécutées simultanément entre elles et avec la requête principale.

Par conséquent, lors de l'utilisation d'instructions de modification de données dans WITH, l'ordre d'exécution des mises à jour spécifiées est imprévisible. Toutes les instructions sont exécutées avec le même instantané (voir chapitre 13), elles ne peuvent donc pas observer les effets des unes sur les autres sur les tables cibles. Ceci atténue les effets de l'imprévisibilité de l'ordre réel des mises à jour de lignes et signifie que le retour de données est le seul moyen de communiquer les modifications entre les différentes sous-instructions WITH et la requête principale. Un exemple de ceci est dans

```
AVEC t COMME (
    MISE À JOUR des produits DÉFINIR le prix = prix * 1,05 RETOUR *
)
SÉLECTIONNER * DE produits;
```

La requête SELECT externe renverrait les prix d'origine avant l'exécution de la requête UPDATE, tandis que dans

```
AVEC t COMME (
    MISE À JOUR des produits DÉFINIR le prix = prix * 1,05 RETOUR *
)
SÉLECTIONNER * DE t;
```

La requête SELECT externe renverrait les données mises à jour.

La mise à jour de la même ligne à deux reprises dans une même instruction n'est pas prise en charge. Une seule modification est effectuée, mais il est difficile (voire impossible) de prédire avec certitude laquelle. Ceci s'applique également à la suppression d'une ligne déjà mise à jour dans la même instruction : seule la mise à jour est conservée.

Il est donc généralement conseillé d'éviter de modifier deux fois la même ligne dans une même instruction. En particulier, évitez d'utiliser des sous-instructions WITH qui pourraient affecter les mêmes lignes que celles modifiées par l'instruction principale ou une sous-instruction sœur. Les effets d'une telle instruction seraient imprévisibles.

Actuellement, toute table utilisée comme cible d'une instruction de modification de données dans WITH ne doit pas comporter de règle conditionnelle, ni de règle ALSO, ni de règle INSTEAD qui s'étend à plusieurs instructions.

Chapitre 8. Types de données

PostgreSQL propose un large éventail de types de données natifs. Les utilisateurs peuvent ajouter de nouveaux types à PostgreSQL à l'aide de la commande `CREATE TYPE`.

Le tableau 8.1 présente tous les types de données intégrés à usage général. La plupart des noms alternatifs listés dans la colonne « Aliases » correspond aux noms utilisés en interne par PostgreSQL pour des raisons historiques. De plus, Certains types utilisés en interne ou obsolètes sont disponibles, mais ne sont pas répertoriés ici.

Tableau 8.1. Types de données

Nom	Alias	Description
bigint	int8	entier signé de huit octets
série à gros chiffres	série8	entier auto-incrémenté de huit octets
bit [(n)]		chaîne de bits de longueur fixe
bit variable [(n)]	varbit [(n)]	chaîne de bits de longueur variable
booléen	booléen	booléen logique (vrai/faux)
boîte		boîte rectangulaire sur un plan
bytea		données binaires (« tableau d'octets »)
caractère [(n)]	char [(n)]	chaîne de caractères de longueur fixe
caractère variable [(n)] varchar	[(n)]	chaîne de caractères de longueur variable
cidre		Adresse réseau IPv4 ou IPv6
cercle		cercle sur un plan
date		date du calendrier (année, mois, jour)
double précision	float, float8	nombre à virgule flottante double précision (8 octets)
inet		Adresse hôte IPv4 ou IPv6
entier	int, int4	entier signé de quatre octets
intervalle [champs] [(p)]		période
json		données textuelles JSON
jsonb		Données JSON binaires, décomposées
doubler		ligne infinie sur un plan
lseg		segment de droite sur un plan
adresse MAC		Adresse MAC (Media Access Control)
macaddr8		Adresse MAC (Media Access Control) (Format EUI-64)
argent		montant en devises
numérique [(p, s)]	décimal [(p, s)]	valeur numérique exacte de précision sélectionnable
chemin		chemin géométrique sur un plan
pg_lsn		Numéro de séquence du journal PostgreSQL
pg_snapshot		instantané de l'identifiant de transaction au niveau utilisateur
indiquer		point géométrique sur un plan

Nom	Alias	Description
polygone		chemin géométrique fermé sur un plan
réel	flotter4	nombre à virgule flottante simple précision (4 octets)
smallint	int2	entier signé de deux octets
petite série	série2	entier auto-incrémenté de deux octets
en série	série4	entier auto-incrémenté de quatre octets
texte		chaîne de caractères de longueur variable
heure [(p)] fuseau [sans horaire]		heure de la journée (sans fuseau horaire)
temps [(p)] avec le temps zone	timetz	heure de la journée, fuseau horaire inclus
horodatage [(p)] fuseau [avec- horaire de sortie]		date et heure (sans fuseau horaire)
horodatage [(p)] avec fuseau horaire	timestampz date et heure, y compris le fuseau horaire	
tsquery		requête de recherche de texte
tsvector		document de recherche de texte
txid_snapshot		instantané de l'identifiant de transaction au niveau utilisateur (obsolète ; voir pg_snapshot)
uuid		identifiant unique universel
XML		Données XML

Compatibilité

Les types suivants (ou leurs variantes orthographiques) sont spécifiés par SQL : bigint, bit, bit varying, boolean, char, character varying, character, varchar, date, double precision, integer, interval, numeric, decimal, real, smallint, heure (avec ou sans fuseau horaire), horodatage (avec ou sans fuseau horaire), xml.

Chaque type de données possède une représentation externe déterminée par ses fonctions d'entrée et de sortie. Les types intégrés ont des formats externes évidents. Cependant, plusieurs types sont soit spécifiques à PostgreSQL, comme des tracés géométriques, ou peuvent présenter plusieurs formats possibles, tels que les types date et heure. Les fonctions d'entrée et de sortie ne sont pas inversibles, c'est-à-dire que le résultat d'une fonction de sortie peut perdre en précision, par rapport à l'entrée originale.

8.1. Types numériques

Les types numériques comprennent les entiers de deux, quatre et huit octets, les nombres à virgule flottante de quatre et huit octets, et les décimaux à précision sélectionnable. Le tableau 8.2 répertorie les types disponibles.

Tableau 8.2. Types numériques

Nom	Description de la	taille de stockage	Gamme
smallint	2 octets	choix typique d'entiers	-32768 à +32767
entier	4 octets	à petite plage pour l'entier -2147483648 à +2147483647	
bigint	8 octets	entier à grande plage	-9223372036854775808 à +9223372036854775807

Nom	Description de la	taille de stockage	Gamme
décimal	variable	précision spécifiée par l'utilisateur, exact	jusqu'à 131072 chiffres avant la virgule décimale; jusqu'à 16383 chiffres après la virgule décimale
numérique	variable	précision spécifiée par l'utilisateur, exact	jusqu'à 131072 chiffres avant la virgule décimale; jusqu'à 16383 chiffres après la virgule décimale
réel	4 octets	précision variable, précision inexacte	à 6 décimales
double précision 8 octets		précision variable, précision inexacte	de 15 chiffres décimaux
petite série	2 octets	petit entier auto-incrémenté	1 à 32767
en série	4 octets	entier auto-incrémenté de 1 à 2147483647	
série à gros chiffres	8 octets	grand entier auto-incrémenté	1 à 9223372036854775807

La syntaxe des constantes pour les types numériques est décrite dans la section 4.1.2. Les types numériques possèdent un Ensemble complet des opérateurs et fonctions arithmétiques correspondants. Voir le chapitre 9 pour plus d'informations. Les sections suivantes décrivent les différents types en détail.

8.1.1. Types d'entiers

Les types smallint, integer et bigint stockent des nombres entiers, c'est-à-dire des nombres sans partie fractionnaire, de différentes plages. Toute tentative de stockage de valeurs en dehors de la plage autorisée entraînera une erreur.

Le type entier est le choix le plus courant, car il offre le meilleur compromis entre plage de valeurs, taille de stockage et performances. Le type smallint n'est généralement utilisé que lorsque l'espace disque est limité. Le type bigint Ce type est conçu pour être utilisé lorsque la plage du type entier est insuffisante.

SQL spécifie uniquement les types entiers integer (ou int), smallint et bigint. Les noms de types int2, int4 et int8 sont des extensions, qui sont également utilisées par certains autres systèmes de bases de données SQL.

8.1.2. Nombres de précision arbitraire

Le type numérique permet de stocker des nombres comportant un très grand nombre de chiffres. Il est particulièrement recommandé pour stocker des montants monétaires et autres quantités nécessitant une grande précision. Calculs avec Les valeurs numériques donnent des résultats exacts lorsque cela est possible (addition, soustraction, multiplication, etc.). Cependant, les calculs sur les valeurs numériques sont beaucoup plus lents que ceux effectués sur les entiers ou les nombres à virgule flottante décrits dans la section suivante.

Nous utilisons ci-dessous les termes suivants : La précision d'un nombre est le nombre total de chiffres significatifs. dans la partie entière, c'est-à-dire le nombre de chiffres de part et d'autre de la virgule décimale. L' échelle d'un Le chiffre numérique correspond au nombre de chiffres décimaux dans la partie fractionnaire, à droite de la virgule. Le nombre 23,5141 a une précision de 6 et une échelle de 4. On peut considérer que les entiers ont une échelle de zéro.

Il est possible de configurer à la fois la précision maximale et l'échelle maximale d'une colonne numérique. Déclarez une colonne de type numérique en utilisant la syntaxe suivante :

NUMÉRIQUE (précision, échelle)

La précision doit être positive, tandis que l'échelle peut être positive ou négative (voir ci-dessous). Autre possibilité :

NUMÉRIQUE (précision)

sélectionne une échelle de 0. Spécification :

NUMÉRIQUE

L'absence de précision et d'échelle crée une colonne « numérique non contrainte » pouvant contenir des valeurs numériques de longueur quelconque, dans la limite des capacités de l'implémentation. Ce type de colonne ne convertit pas les valeurs saisies à une échelle particulière, contrairement aux colonnes numériques avec une échelle définie. (La norme SQL impose une échelle par défaut de 0, soit une conversion à la précision entière. Nous estimons cette contrainte peu utile. Pour une meilleure portabilité, spécifiez toujours explicitement la précision et l'échelle.)

Note

La précision maximale qui peut être explicitement spécifiée dans une déclaration de type numérique est de 1000. Une colonne numérique non contrainte est soumise aux limites décrites dans le tableau 8.2.

Si l'échelle d'une valeur à stocker est supérieure à l'échelle déclarée de la colonne, le système arrondit la valeur au nombre de décimales spécifié. Ensuite, si le nombre de chiffres après la virgule dépasse la précision déclarée moins l'échelle déclarée, une erreur est générée. Par exemple, une colonne déclarée comme

NUMÉRIQUE(3, 1)

arrondira les valeurs à une décimale et pourra stocker des valeurs comprises entre -99,9 et 99,9 inclus.

À partir de PostgreSQL 15, il est possible de déclarer une colonne numérique avec une échelle négative. Les valeurs seront alors arrondies à gauche de la virgule décimale. La précision représente toujours le nombre maximal de chiffres non arrondis. Ainsi, une colonne déclarée comme

NUMÉRIQUE(2, -3)

arrondira les valeurs au millier le plus proche et pourra stocker des valeurs comprises entre -99000 et 99000 inclus.

Il est également possible de déclarer une échelle supérieure à la précision déclarée. Une telle colonne ne peut contenir que des valeurs fractionnaires et le nombre de zéros situés juste après la virgule doit être au moins égal à l'échelle déclarée moins la précision déclarée. Par exemple, une colonne déclarée comme

NUMÉRIQUE(3, 5)

arrondira les valeurs à 5 décimales et pourra stocker des valeurs comprises entre -0,00999 et 0,00999 inclus.

Note

PostgreSQL autorise toute valeur d'échelle comprise entre -1000 et 1000 pour la déclaration d'un type numérique. Cependant, la norme SQL exige que l'échelle soit comprise entre 0 et la précision. L'utilisation d'échelles en dehors de cette plage peut entraîner des problèmes de portabilité avec d'autres systèmes de bases de données.

Les valeurs numériques sont stockées physiquement sans zéros supplémentaires en début ni en fin. Par conséquent, la précision et l'échelle déclarées d'une colonne sont des valeurs maximales, et non des allocations fixes. (En ce sens, le type numérique est plus proche de varchar(n) que de char(n).) L'espace de stockage requis est de deux octets pour chaque groupe de quatre chiffres décimaux, plus trois à huit octets d'espace supplémentaire.

Outre les valeurs numériques ordinaires, le type numérique possède plusieurs valeurs spéciales :

Infini
-Infini
NaN

Ces symboles sont adaptés de la norme IEEE 754 et représentent respectivement « l'infini », « moins l'infini » et « non un nombre ». Lorsque vous écrivez ces valeurs comme constantes dans une commande SQL, vous devez les spécifier en encadrant les guillemets, par exemple `UPDATE table SET x = '-Infinity'`. Lors de la saisie, ces

Les chaînes de caractères sont reconnues sans tenir compte de la casse. Les valeurs infinies peuvent également s'écrire : `inf` et `-inf`.

Les valeurs infinies se comportent comme prévu mathématiquement. Par exemple, l'infini plus n'importe quelle valeur finie La valeur est égale à l'infini, de même que l'infini plus l'infini ; mais l'infini moins l'infini renvoie NaN (not a number), car il n'a pas d'interprétation bien définie. Notez qu'une infinité ne peut que être stocké dans une colonne numérique non contrainte, car il dépasse théoriquement toute précision finie limite.

La valeur NaN (not a number) est utilisée pour représenter des résultats de calcul indéfinis. En général, toute Une opération avec une entrée NaN renvoie également NaN. La seule exception concerne les autres entrées de l'opération. sont telles que le même résultat serait obtenu si le NaN était remplacé par n'importe quel nombre fini ou infini. valeur numérique ; ensuite, cette valeur de sortie est également utilisée pour NaN. (Un exemple de ce principe est que NaN élevé à la puissance zéro, cela donne un.)

Note

Dans la plupart des implémentations du concept « pas un nombre », NaN n'est pas considéré comme égal à quoi que ce soit. autre valeur numérique (y compris NaN). Afin de permettre le tri et l'utilisation des valeurs numériques Dans les index arborescents, PostgreSQL traite les valeurs NaN comme égales et supérieures à toutes les valeurs non-NaN. valeurs.

Les types décimal et numérique sont équivalents. Ces deux types font partie de la norme SQL.

Lors de l'arrondi des valeurs, le type numérique arrondit les égalités à l'opposé de zéro, tandis que (sur la plupart des machines) le Les types de nombres à précision réelle et double arrondissent les égalités au nombre pair le plus proche. Par exemple :

```
SÉLECTIONNER x,
  arrondi(x::numérique) AS num_arrondi,
  arrondi(x::double précision) AS dbl_round
FROM generate_series(-3.5, 3.5, 1) as x;
 x      | num_round | dbl_round
-----+-----+-----
-3,5 | -2,5      | -4 | -3      | -4
| -1,5 |          | -2 | -2      | -2
-0,5 | 0,5       | -1 | 1       | -2
| 1,5 | 2,5       | 2 | 3       | -0
| 3,5 |          | 4 |         | 0
(8                                           2
lignes)                                     2
                                           4
```

8.1.3. Types à virgule flottante

Les types de données `real` et `double precision` sont des types numériques inexacts, à précision variable.

Sur toutes les plateformes actuellement prises en charge, ces types sont des implémentations de la norme IEEE 754 pour le binaire.

Arithmétique à virgule flottante (précision simple et double, respectivement), dans la mesure où le processeur sous-jacent, le système d'exploitation et le compilateur la prennent en charge.

« Inexact » signifie que certaines valeurs ne peuvent pas être converties exactement au format interne et sont stockées sous forme d'approximations, de sorte que le stockage et la récupération d'une valeur peuvent présenter de légères différences. La gestion de ces erreurs et leur propagation dans les calculs constituent un domaine entier des mathématiques et de l'informatique et ne seront pas abordées ici, à l'exception des points suivants :

- Si vous avez besoin d'un stockage et de calculs précis (par exemple pour des montants monétaires), utilisez le format numérique. tapez plutôt.
- Si vous souhaitez effectuer des calculs complexes avec ces types pour des raisons importantes, notamment si vous vous appuyez sur un certain comportement dans les cas limites (infini, sous-dépassement), vous devez évaluer soigneusement l'implémentation.
- Comparer deux valeurs à virgule flottante pour vérifier leur égalité ne fonctionne pas toujours comme prévu.

Sur toutes les plateformes actuellement prises en charge, le type réel a une plage de valeurs allant d'environ 1E-37 à 1E+37 avec une précision d'au moins 6 chiffres décimaux. Le type double précision a une plage de valeurs allant d'environ 1E-307 à 1E+308 avec une précision d'au moins 15 chiffres. Les valeurs trop grandes ou trop petites provoqueront une erreur. Un arrondi peut être effectué si la précision d'un nombre saisi est trop élevée. Les nombres trop proches de zéro et qui ne peuvent être représentés comme distincts de zéro provoqueront une erreur de dépassement de capacité négatif.

Par défaut, les valeurs à virgule flottante sont affichées sous forme de texte dans leur représentation décimale précise la plus courte ; la valeur décimale produite est plus proche de la valeur binaire réelle stockée que toute autre valeur représentable avec la même précision binaire. (Toutefois, la valeur de sortie n'est actuellement jamais exactement à mi-chemin entre deux valeurs représentables, afin d'éviter un bogue répandu où les routines d'entrée ne respectent pas correctement la règle d'arrondi à l'entier le plus proche.) Cette valeur utilisera au maximum 17 chiffres décimaux significatifs pour les valeurs float8 et au maximum 9 chiffres pour les valeurs float4.

Note

Ce format de sortie court et précis est beaucoup plus rapide à générer que le format arrondi historique.

Pour assurer la compatibilité avec les résultats générés par les anciennes versions de PostgreSQL et permettre une réduction de la précision de l'affichage, le paramètre `extra_float_digits` permet d'afficher les résultats sous forme décimale arrondie. La valeur 0 rétablit le comportement par défaut, à savoir un arrondi à 6 chiffres après la virgule (pour le type `float4`) ou 15 (pour le type `float8`). Une valeur négative réduit davantage le nombre de chiffres après la virgule ; par exemple, -2 arrondit l'affichage à 4 ou 13 chiffres après la virgule, respectivement.

Toute valeur de `extra_float_digits` supérieure à 0 sélectionne le format le plus court et précis.

Note

Les applications nécessitant des valeurs précises devaient traditionnellement définir `extra_float_digits` à 3 pour les obtenir. Pour une compatibilité maximale entre les versions, il est recommandé de conserver cette configuration.

Outre les valeurs numériques ordinaires, les types à virgule flottante possèdent plusieurs valeurs spéciales :

Infini
-Infini
NaN

Ces valeurs représentent respectivement les valeurs spéciales « infini », « moins l'infini » et « non-un nombre » de la norme IEEE 754. Lorsque vous écrivez ces valeurs comme constantes dans une commande SQL, vous devez les encadrer de guillemets.

Par exemple, la commande `UPDATE table SET x = 'Infinity'` permet de saisir ces chaînes de caractères sans tenir compte de la casse. La valeur infinie peut également s'écrire `inf` ou `-inf`.

Note

La norme IEEE 754 stipule que NaN ne doit pas être comparé à une autre valeur à virgule flottante (y compris NaN). Afin de permettre le tri et l'utilisation des valeurs à virgule flottante dans les index arborescents, PostgreSQL considère les valeurs NaN comme égales et supérieures à toutes les autres valeurs.

PostgreSQL prend également en charge les notations SQL standard `float` et `float(p)` pour spécifier les types numériques approximatifs. Ici, `p` spécifie la précision minimale acceptable en chiffres binaires. PostgreSQL interprète `float(1)` à `float(24)` comme sélectionnant le type réel, tandis que `float(25)` à `float(53)` sélectionne la double précision. Les valeurs de `p` en dehors de la plage autorisée génèrent une erreur. `float` sans précision spécifiée est interprété comme une double précision.

8.1.4. Types de numéros de série

Note

Cette section décrit une méthode spécifique à PostgreSQL pour créer une colonne à incrémentation automatique. Une autre méthode consiste à utiliser la fonctionnalité de colonne d'identité standard de SQL, décrite à la section 5.3.

Les types de données `smallserial`, `serial` et `bigserial` ne sont pas de véritables types, mais simplement une commodité de notation pour la création de colonnes d'identifiants uniques (semblable à la propriété `AUTO_INCREMENT` prise en charge par certaines autres bases de données). Dans l'implémentation actuelle, il faut spécifier :

```
CRÉER LA TABLE nom_table ( nom_colonne SERIAL
```

```
);
```

est équivalent à spécifier :

```
CRÉER UNE SÉQUENCE nom_table_nom_colonne_seq AS entier;
```

```
CRÉER UNE TABLE nom_table ( nom_colonne
```

```
entier NOT NULL DEFAULT
```

```
nextval('tablename_colname_seq') ); ALTER SEQUENCE
```

```
tablename_colname_seq OWNED BY tablename.colname;
```

Nous avons donc créé une colonne d'entiers et configuré ses valeurs par défaut pour qu'elles soient générées par une séquence. Une contrainte `NOT NULL` est appliquée afin d'empêcher l'insertion de valeurs nulles.

(Dans la plupart des cas, il est également conseillé d'ajouter une contrainte `UNIQUE` ou `PRIMARY KEY` pour empêcher l'insertion accidentelle de valeurs en double, mais cela n'est pas automatique.) Enfin, la séquence est marquée comme « appartenant à » la colonne, de sorte qu'elle sera supprimée si la colonne ou la table est supprimée.

Note

Étant donné que les types `smallserial`, `serial` et `bigserial` utilisent des séquences, des « trous » ou des lacunes peuvent apparaître dans la séquence de valeurs de la colonne, même si aucune ligne n'est supprimée. Une valeur allouée à partir de la séquence est considérée comme « utilisée » même si aucune ligne contenant cette valeur n'est insérée avec succès dans la colonne de la table. Cela peut se produire, par exemple, si la transaction d'insertion est annulée. Pour plus de détails, consultez la fonction `nextval()` dans la section 9.17.

Pour insérer la valeur suivante de la séquence dans la colonne séquentielle, indiquez que cette colonne doit conserver sa valeur par défaut. Vous pouvez le faire soit en excluant la colonne de la liste des colonnes dans l'instruction INSERT, soit en utilisant le mot-clé DEFAULT.

Les types `serial` et `serial4` sont équivalents : ils créent tous deux des colonnes d'entiers. Les types `bigserial` et `serial8` fonctionnent de la même manière, à ceci près qu'ils créent une colonne de type `bigint`. Il est recommandé d'utiliser `bigserial` si vous prévoyez d'utiliser plus de 231 identifiants au cours de la durée de vie de la table. Les types `smallserial` et `serial2` fonctionnent également de la même manière, à ceci près qu'ils créent une colonne de type `smallint`.

La séquence créée pour une colonne séquentielle est automatiquement supprimée lorsque la colonne propriétaire est supprimée. Vous pouvez supprimer la séquence sans supprimer la colonne, mais cela entraînera la suppression de l'expression par défaut de la colonne.

8.2. Types monétaires

Le type monétaire stocke un montant avec une précision fractionnaire fixe (voir tableau 8.3). Cette précision est déterminée par le paramètre `lc_monetary` de la base de données. La plage indiquée dans le tableau suppose la présence de deux décimales. Les entrées sont acceptées dans divers formats, notamment les entiers et les nombres à virgule flottante, ainsi que le format monétaire standard, tel que « 1 000,00 \$ ». La sortie est généralement au format monétaire standard, mais peut varier selon les paramètres régionaux.

Tableau 8.3. Types monétaires

Nom	Taille de stockage : 8 octets	Gamme
argent	montant en devises	-92233720368547758,08 à +92233720368547758.07

Étant donné que le format de données de ce type dépend des paramètres régionaux, il est possible que le chargement de données monétaires dans une base de données dont la variable `lc_monetary` est différente ne fonctionne pas. Pour éviter tout problème, avant de restaurer une sauvegarde dans une nouvelle base de données, assurez-vous que la variable `lc_monetary` possède la même valeur, ou une valeur équivalente, que dans la base de données d'origine.

Les valeurs des types de données `numeric`, `int` et `bigint` peuvent être converties en monnaie. La conversion à partir des types de données `real` et `double precision` peut être effectuée en effectuant d'abord une conversion en `numeric`, par exemple :

```
SÉLECTIONNER '12.34'::float8::numeric::money;
```

Toutefois, cette pratique est déconseillée. Il est préférable de ne pas utiliser de nombres à virgule flottante pour les transactions monétaires en raison des risques d'erreurs d'arrondi.

Une valeur monétaire peut être convertie en valeur numérique sans perte de précision. La conversion vers d'autres types peut potentiellement entraîner une perte de précision et doit également être effectuée en deux étapes :

```
SÉLECTIONNER '52093.89'::argent::numérique::float8;
```

La division d'une valeur monétaire par une valeur entière s'effectue en tronquant la partie décimale vers zéro. Pour obtenir un résultat arrondi, divisez par une valeur à virgule flottante, ou convertissez la valeur monétaire en numérique avant la division, puis reconvertissez-la en valeur monétaire après. (Cette dernière méthode est préférable pour éviter toute perte de précision.) Lorsqu'une valeur monétaire est divisée par une autre, le résultat est en double précision (c'est-à-dire un nombre pur, et non une somme d'argent) ; les unités monétaires s'annulent lors de la division.

8.3. Types de caractères

Tableau 8.4. Types de caractères

Nom	Description
<code>character varying(n)</code> , <code>varchar(n)</code>	longueur variable avec limite
<code>caractère(n)</code> , <code>char(n)</code> , <code>bpchar(n)</code>	longueur fixe, rembourrage
<code>bpchar</code>	vierge, longueur variable illimitée, découpe vierge
<code>texte</code>	longueur variable illimitée

Le tableau 8.4 présente les types de caractères à usage général disponibles dans PostgreSQL.

SQL définit deux types de caractères principaux : `character varying(n)` et `character(n)`, où `n` est un entier positif. Ces deux types peuvent stocker des chaînes de caractères d'une longueur maximale de `n` caractères (et non octets). Toute tentative de stockage d'une chaîne plus longue dans une colonne de ces types entraînera une erreur, sauf si les caractères excédentaires sont tous des espaces ; dans ce cas, la chaîne sera tronquée à la longueur maximale. (Cette exception, quelque peu surprenante, est exigée par la norme SQL.) Cependant, si une valeur est explicitement convertie en `character varying(n)` ou `character(n)`, une valeur trop longue sera tronquée à `n` caractères sans générer d'erreur. (Ceci est également exigé par la norme SQL.) Si la chaîne à stocker est plus courte que la longueur déclarée, les valeurs de type `character` seront complétées par des espaces ; les valeurs de type `character varying` stockeront simplement la chaîne la plus courte.

De plus, PostgreSQL propose le type `text`, qui stocke des chaînes de caractères de toute longueur. Bien que ce type ne fasse pas partie de la norme SQL, plusieurs autres systèmes de gestion de bases de données SQL le prennent également en charge. `text` est le type de données de chaîne natif de PostgreSQL ; la plupart des fonctions intégrées manipulant des chaînes sont déclarées comme acceptant ou renvoyant du `text` et non du `character varying`. Dans de nombreux cas, le type `character varying` se comporte comme un domaine différent de celui du `text`.

Le nom de type `varchar` est un alias pour `character varying`, tandis que `bpchar` (avec le spécificateur de longueur) et `char` sont des alias pour `character`. Les alias `varchar` et `char` sont définis dans la norme SQL ; `bpchar` est une extension PostgreSQL.

Si elle est spécifiée, la longueur `n` doit être supérieure à zéro et ne peut excéder 10 485 760. Si le type `character varying` (ou `varchar`) est utilisé sans spécification de longueur, il accepte les chaînes de toute longueur. Si le type `bpchar` n'a pas de spécification de longueur, il accepte également les chaînes de toute longueur, mais les espaces de fin sont sans signification. Si le type `character` (ou `char`) n'a pas de spécification, il est équivalent à `character(1)`.

Les valeurs de type caractère sont physiquement complétées par des espaces jusqu'à la largeur spécifiée `n`, et sont stockées et affichées ainsi. Cependant, les espaces de fin sont considérés comme sémantiquement insignifiants et ignorés lors de la comparaison de deux valeurs de type caractère. Dans les classements où les espaces sont significatifs, ce comportement peut produire des résultats inattendus ; par exemple, `SELECT 'a'::CHAR(2) collate "C" < 'E'a'n'::CHAR(2)` renvoie `true`, même si la locale C considère un espace comme supérieur à un saut de ligne. Les espaces de fin sont supprimés lors de la conversion d'une valeur de type caractère vers un autre type de chaîne. Notez que les espaces de fin sont sémantiquement significatifs dans les valeurs de type caractère variable et texte, ainsi que lors de l'utilisation de la correspondance de modèles (LIKE et expressions régulières).

Les caractères pouvant être stockés dans ces types de données dépendent du jeu de caractères de la base de données, sélectionné lors de sa création. Quel que soit le jeu de caractères choisi, le caractère de code zéro (parfois appelé NUL) ne peut pas être stocké. Pour plus d'informations, consultez la section 23.3.

L'espace de stockage requis pour une chaîne courte (jusqu'à 126 octets) est de 1 octet plus la chaîne elle-même, incluant les espaces de remplissage pour les caractères. Les chaînes plus longues nécessitent 4 octets supplémentaires au lieu de 1. Le système compresse automatiquement les chaînes longues, ce qui peut réduire l'espace disque requis. Les valeurs très longues sont également stockées dans des tables d'arrière-plan afin de ne pas perturber l'accès rapide aux valeurs des colonnes plus courtes. Dans tous les cas, la plus longue chaîne de caractères pouvant être stockée est d'environ 1 Go. (La valeur maximale autorisée pour `n` dans la déclaration du type de données est inférieure. Il serait inutile de la modifier car, avec les encodages de caractères multioctets, le nombre de caractères et d'octets peut être très différent. Si vous souhaitez stocker de longues chaînes sans limite supérieure spécifique, utilisez un type texte ou un type caractère variable sans spécificateur de longueur, plutôt que de définir une limite arbitraire.)

Conseil

Il n'y a aucune différence de performance entre ces trois types, hormis une capacité de stockage accrue.

L'espace est nécessaire lors de l'utilisation du type avec remplissage blanc, et quelques cycles CPU supplémentaires sont requis pour vérifier la longueur.

lors du stockage dans une colonne de longueur limitée. Bien que la méthode `character(n)` présente des avantages en

termes de performances dans certains autres systèmes de bases de données, PostgreSQL ne possède pas cet avantage ; en fait

La fonction `character(n)` est généralement la plus lente des trois en raison de ses coûts de stockage supplémentaires.

Dans la plupart des situations, il convient d'utiliser plutôt du texte ou des caractères variables.

Reportez-vous à la section 4.1.2.1 pour obtenir des informations sur la syntaxe des littéraux de chaîne et au chapitre 9 pour obtenir des informations sur les opérateurs et fonctions disponibles.

Exemple 8.1. Utilisation des types de caractères

```
CRÉER TABLE test1 (un caractère(4));
INSÉRER DANS test1 VALEURS ('ok');
SELECT a, char_length(a) FROM test1; --
```

un	longueur_char
ok	2

```
CRÉER TABLE test2 (b varchar(5));
INSÉRER DANS test2 VALEURS ('ok');
INSÉRER DANS test2 VALEURS ('bon ');
INSÉRER DANS test2 VALEURS ('trop long');
ERREUR : valeur trop longue pour le type caractère variable(5)
INSERT INTO test2 VALUES ('trop long'::varchar(5)); -- explicite
troncature
SÉLECTIONNER b, char_length(b) DE test2;
```

b	longueur_char
d'accord	2
bien trop	5
	5

 La fonction `char_length` est abordée dans la section 9.4.

PostgreSQL propose deux autres types de caractères de longueur fixe, présentés dans le tableau 8.5. Ceux-ci ne sont pas destinés à un usage général, uniquement pour les catalogues système internes. Le type de nom est utilisé pour stocker des identifiants. Sa longueur est actuellement définie à 64 octets (63 caractères utilisables plus le terminateur). mais doit être référencée à l'aide de la constante `NAMEDATALEN` dans le code source C. La longueur est définie à au moment de la compilation (et est donc ajustable pour des usages spécifiques) ; la longueur maximale par défaut peut changer dans une prochaine version. Le type « char » (notez les guillemets) diffère de `char(1)` en ce qu'il utilise uniquement un octet de mémoire et ne peut donc stocker qu'un seul caractère ASCII. Il est utilisé dans le système de catalogues en tant que type d'énumération simpliste.

Tableau 8.5. Types de caractères spéciaux

Nom	Taille du stockage	Description
"carboniser"	1 octet	type interne mono-octet

Nom	Taille du stockage	Description
nom	64 octets	type interne pour les noms d'objets

8.4. Types de données binaires

Le type de données `bytea` permet le stockage de chaînes binaires ; voir le tableau 8.6.

Tableau 8.6. Types de données binaires

Nom	Taille du stockage	Description
<code>bytea</code>	1 ou 4 octets plus la chaîne binaire proprement dite	chaîne binaire de longueur variable

Une chaîne binaire est une séquence d'octets. Elle se distingue des chaînes de caractères de deux manières. Premièrement, les chaînes binaires autorisent le stockage d'octets de valeur zéro et d'autres octets « non imprimables » (généralement, les octets hors de l'intervalle décimal de 32 à 126). Les chaînes de caractères interdisent les octets nuls, ainsi que toute autre valeur d'octet et séquence d'octets invalide selon l'encodage de caractères sélectionné par la base de données. Deuxièmement, les opérations sur les chaînes binaires traitent les octets eux-mêmes, tandis que le traitement des chaînes de caractères dépend des paramètres régionaux. En résumé, les chaînes binaires conviennent au stockage de données brutes, tandis que les chaînes de caractères conviennent au stockage de texte.

Le type `bytea` prend en charge deux formats d'entrée et de sortie : le format hexadécimal et le format d'échappement historique de PostgreSQL. Ces deux formats sont toujours acceptés en entrée. Le format de sortie dépend du paramètre de configuration `bytea_output` ; le format hexadécimal est utilisé par défaut. (Notez que le format hexadécimal a été introduit dans PostgreSQL 9.0 ; les versions antérieures et certains outils ne le prennent pas en charge.)

La norme SQL définit un type de chaîne binaire différent, appelé BLOB ou BINARY LARGE OBJECT.

Le format d'entrée est différent de celui de `bytea`, mais les fonctions et opérateurs fournis sont pour la plupart les mêmes. même.

8.4.1. Format hexadécimal de `bytea`

Le format « hex » encode les données binaires sous forme de 2 chiffres hexadécimaux par octet, le quartet le plus significatif en premier. La chaîne entière est précédée de la séquence `\x` (pour la distinguer du format d'échappement). Dans certains contextes, il peut être nécessaire d'échapper la barre oblique inverse initiale en la doublant (voir section 4.1.2.1). En entrée, les chiffres hexadécimaux peuvent être en majuscules ou en minuscules, et les espaces sont autorisés entre les paires de chiffres (mais pas à l'intérieur d'une paire de chiffres ni dans la séquence `\x` initiale). Le format hexadécimal est compatible avec un large éventail d'applications et de protocoles externes, et sa conversion est généralement plus rapide que celle du format d'échappement ; son utilisation est donc préférable.

Exemple:

```
DÉFINIR bytea_output = 'hex';
```

```
SELECT '\xDEADBEEF'::bytea; bytea
```

```
-----
\xdeadbeef
```

8.4.2. Format d'échappement `bytea`

Le format « escape » est le format PostgreSQL traditionnel pour le type `bytea`. Il consiste à représenter une chaîne binaire comme une séquence de caractères ASCII, en convertissant les octets ne pouvant être représentés par un caractère ASCII en séquences d'échappement spéciales. Si, du point de vue de l'application, représenter les octets par des caractères est pertinent, cette représentation peut s'avérer pratique.

Mais en pratique, c'est généralement source de confusion car cela brouille la distinction entre les chaînes binaires et Les chaînes de caractères, ainsi que le mécanisme d'échappement particulier qui a été choisi, sont quelque peu difficiles à manier. Par conséquent, il est probablement préférable d'éviter ce format pour la plupart des nouvelles applications.

Lors de la saisie de valeurs bytea au format d'échappement, les octets de certaines valeurs doivent être échappés, tandis que tous les autres Les valeurs d'octet peuvent être échappées. En général, pour échapper un octet, convertissez-le en sa valeur octale à trois chiffres et le faire précéder d'une barre oblique inverse. La barre oblique inverse elle-même (valeur décimale octet 92) peut également être représentée par double barre oblique inverse. Le tableau 8.7 indique les caractères à échapper et donne l'alternative. séquences d'échappement, le cas échéant.

Tableau 8.7. Octets échappés littéraux bytea

Octet décimal Valeur	Description	Entrée échappée Représentation	Exemple	Représentation hexadécimale
0	octet zéro	'\000'	'\000'::octet \x00	
39	citation simple	'"' ou '\047'	'"::bytea	\x27
92	barre oblique inverse	'\' ou '\134'	'\'::bytea	\x5c
de 0 à 31 et 127 jusqu'à 255	« non imprimable » octets	'\xxx' (octal valeur)	'\001'::bytea \x01	

L'obligation d'échapper les octets non imprimables varie selon les paramètres régionaux. Dans certains cas Vous pouvez vous en tirer sans les laisser s'échapper.

La raison pour laquelle les guillemets simples doivent être doublés, comme indiqué dans le tableau 8.7, est que cela est vrai pour toute chaîne de caractères. littéral dans une commande SQL. L'analyseur de chaînes littérales génériques consomme les guillemets simples les plus externes. et réduit toute paire de guillemets simples à un seul caractère de données. La fonction d'entrée bytea ne voit que cela. une simple apostrophe, qu'elle traite comme un caractère de données ordinaire. Cependant, la fonction d'entrée bytea traite les barres obliques inverses sont considérées comme spéciales, et les autres comportements présentés dans le tableau 8.7 sont implémentés par cette fonction.

Dans certains contextes, les barres obliques inverses doivent être doublées par rapport à ce qui est indiqué ci-dessus, car le générique L'analyseur de chaînes littérales réduira également les paires de barres obliques inverses à un seul caractère de données ; voir la section 4.1.2.1.

Par défaut, les octets de Bytea sont affichés au format hexadécimal. Si vous modifiez l'option 'bytea_output' en 'escape', les octets « non imprimables » sont convertis en leur valeur octale équivalente à trois chiffres et précédés d'une barre oblique inverse. La plupart des octets « imprimables » sont affichés selon leur représentation standard dans le jeu de caractères du client, par exemple :

```
DÉFINIR bytea_output = 'escape';
```

```
SELECT 'abc \153\154\155 \052\251\124'::bytea;
```

```
      bytea
-----
abc klm *1251T
```

L'octet dont la valeur décimale est 92 (antislash) est doublé dans le résultat. Voir le tableau 8.8 pour plus de détails.

Tableau 8.8. Octets échappés de la sortie bytea

Octet décimal Valeur	Description	Sortie échappée Représentation	Exemple	Résultat de sortie
92	barre oblique inverse	\\	'\134'::octet \\	
de 0 à 31 et 127 jusqu'à 255	« non imprimable » octets	\xxx (valeur octale)	'\001'::byte \001	
32 à 126	octets « imprimables »	caractère client représentation ensembliste	'\176'::octet ~	

Selon l'interface PostgreSQL que vous utilisez, vous pourriez avoir des opérations supplémentaires à effectuer concernant l'échappement et le déséchappement des chaînes d'octets. Par exemple, vous devrez peut-être également échapper les sauts de ligne et les retours chariot si votre interface les traduit automatiquement.

8.5. Types de date/heure

PostgreSQL prend en charge l'ensemble des types de date et d'heure SQL, présentés dans le tableau 8.9. Les opérations disponibles sur ces types de données sont décrites dans la section 9.9. Les dates sont comptées selon le calendrier grégorien, même pour les années antérieures à l'introduction de ce calendrier (voir la section B.6 pour plus d'informations).

Tableau 8.9. Types de date/heure

Nom	Taille de stockage	Description	Valeur faible	8 octets	Résolution haute	valeur
horodatage [(p)] [sans heure zone]		la date et l'heure (sans fuseau horaire)	4713 av. J.-C.		294276 ap. J.-C.	1 microseconde
horodatage [(p)] avec l'heure zone	8 octets	la date et l'heure, avec l'heure zone	4713 av. J.-C.		294276 ap. J.-C.	1 microseconde
date	4 octets	date (sans heure)	4713 av. J.-C.		5874897 après J.-C.	1 jour
temps [(p)] [sans temps zone]	8 octets	heure de la journée (sans date)	00:00:00		24:00:00	1 microsecond
temps [(p)] avec le temps zone	12 octets	heure de la journée (sans date), avec fuseau horaire	00:00:00+1559 24:00:00-1559		1 microsecond	
intervalle [champs] [(p)]	16 octets	intervalle de temps	178000000 années		178000000 années	1 microsecond

Note

La norme SQL exige que l'écriture d'un horodatage seul soit équivalente à celle d'un horodatage sans fuseau horaire, et PostgreSQL respecte ce comportement. `timestampz` est accepté comme abréviation pour `timestamp` avec fuseau horaire ; il s'agit d'une extension PostgreSQL.

Les types `time`, `timestamp` et `interval` acceptent une valeur de précision optionnelle `p` qui spécifie le nombre de chiffres après la virgule conservés dans le champ des secondes. Par défaut, aucune limite de précision n'est explicitement définie. La plage de valeurs autorisée pour p est de 0 à 6.

Le type intervalle possède une option supplémentaire qui permet de restreindre l'ensemble des champs stockés en saisissant l'une de ces phrases :

ANNÉE
MOIS
JOUR

HEURE
 MINUTE
 DEUXIÈME
 DE L'ANNÉE AU MOIS
 JOUR À HEURE
 JOUR À MINUTE
 JOUR À SECONDE
 HEURE À MINUTE
 HEURE À SECONDE
 MINUTE À SECONDE

Notez que si les champs et p sont tous deux spécifiés, les champs doivent inclure SECOND, car la précision s'applique uniquement aux secondes.

Le type `time with time zone` est défini par la norme SQL, mais cette définition présente des propriétés qui rendent son utilité discutable. Dans la plupart des cas, une combinaison de `date`, `time`, `timestamp` sans fuseau horaire et `timestamp` avec fuseau horaire devrait suffire.

Gamme de fonctionnalités de date/heure requises par toute application.

8.5.1. Saisie de la date et de l'heure

La saisie de la date et de l'heure est acceptée dans la quasi-totalité des formats courants, notamment ISO 8601, compatible SQL, PostgreSQL traditionnel, etc. Pour certains formats, l'ordre du jour, du mois et de l'année dans la date est important. Les données d'entrée sont ambiguës et il est possible de spécifier l'ordre attendu de ces champs. Définissez le paramètre `DateStyle` doit être défini sur MDY pour sélectionner l'interprétation mois-jour-année, et sur DMY pour sélectionner l'interprétation jour-mois-année, ou YMD pour sélectionner l'interprétation année-mois-jour.

PostgreSQL offre une plus grande flexibilité dans la gestion des dates et heures que ne l'exige la norme SQL. Consultez l'annexe B pour connaître les règles d'analyse exactes des dates et heures et les champs de texte reconnus, notamment les mois, jours de la semaine et fuseaux horaires.

N'oubliez pas que toute valeur littérale de date ou d'heure doit être placée entre guillemets simples, comme les chaînes de caractères. Pour plus d'informations, veuillez consulter la section 4.1.2.7. SQL requiert la syntaxe suivante.

`type [ (p) ] 'valeur'`

où p est une spécification de précision optionnelle indiquant le nombre de chiffres fractionnaires dans les secondes champ. La précision peut être spécifiée pour les types heure, horodatage et intervalle, et peut varier de 0 à 6. Si aucune précision n'est spécifiée dans une spécification de constante, la précision par défaut est celle de la valeur littérale. valeur (mais pas plus de 6 chiffres).

8.5.1.1. Dates

Le tableau 8.10 présente quelques exemples de valeurs possibles pour le type de date.

Tableau 8.10. Saisie des dates

Exemple	Description
1999-01-08	ISO 8601 ; 8 janvier dans n'importe quel mode (format recommandé)
8 janvier 1999	sans ambiguïté dans tous les modes de saisie de date
1/8/1999	8 janvier en mode MDY ; 1er août en mode DMY
18/01/1999	18 janvier en mode MDY ; rejeté dans les autres modes
01/02/03	2 janvier 2003 en mode MDY ; 1er février 2003 en mode DMY ; 3 février 2001 en mode AMD
8 janvier 1999	8 janvier dans n'importe quel mode

Exemple	Description
8 janvier 1999	8 janvier dans n'importe quel mode
08 janvier 1999	8 janvier dans n'importe quel mode
99-janv.-08	8 janvier en mode AAMD, sinon erreur
08-janv.-99	8 janvier, sauf erreur en mode AMD
08-janv.-99	8 janvier, sauf erreur en mode AMD
19990108	ISO 8601 ; 8 janvier 1999 dans n'importe quel mode
990108	ISO 8601 ; 8 janvier 1999 dans n'importe quel mode
1999.008	année et jour de l'année
J2451187	date julienne
8 janvier 99 av. J.-C.	an 99 avant J.-C.

8.5.1.2. Temps

Les types d'heure sont l'heure [(p)] sans fuseau horaire et l'heure [(p)] avec Fuseau horaire. L'heure seule est équivalente à l'heure sans fuseau horaire.

Les données valides pour ces types de saisie consistent en une heure de la journée suivie, éventuellement, d'un fuseau horaire. (Voir tableau 8.11) et le tableau 8.12.) Si un fuseau horaire est spécifié dans l'entrée pour l'heure sans fuseau horaire, il est ignoré. Ignoré. Vous pouvez également spécifier une date, mais elle sera ignorée, sauf si vous utilisez un nom de fuseau horaire. qui implique une règle relative à l'heure d'été, comme America/New_York. Dans ce cas, il s'agit de préciser la date. Il est nécessaire de le faire afin de déterminer si l'heure normale ou l'heure d'été s'applique. Le décalage horaire est enregistré dans l'horodatage avec la valeur du fuseau horaire et est affiché tel qu'il est stocké ; il est non ajusté au fuseau horaire actif.

Tableau 8.11. Saisie du temps

Exemple	Description
04:05:06.789	ISO 8601
04:05:06	ISO 8601
04:05	ISO 8601
040506	ISO 8601
04h05	Identique à 04h05 ; le matin n'a aucune incidence valeur
16h05	identique à 16:05 ; l'heure saisie doit être inférieure ou égale à 12
04:05:06.789-8	ISO 8601, avec fuseau horaire UTC désactivé ensemble
04:05:06-08:00	ISO 8601, avec fuseau horaire décalé par rapport à UTC
04:05-08:00	ISO 8601, avec fuseau horaire UTC désactivé ensemble
040506-08	ISO 8601, avec fuseau horaire décalé par rapport à UTC
040506+0730	ISO 8601, avec heures fractionnaires fuseau horaire en décalage UTC
040506+07:30:00	Décalage UTC spécifié en secondes (et non autorisé dans la norme ISO 8601)
04:05:06 PST	fuseau horaire spécifié par abréviation

Exemple	Description
12 avril 2003 04:05:06 Amérique/New_York	fuseau horaire spécifié par nom complet

Tableau 8.12. Saisie du fuseau horaire

Exemple	Description
TVP	Abréviation (pour l'heure normale du Pacifique)
Amérique/New_York	Nom complet du fuseau horaire
PST8PDT	Spécification de fuseau horaire de style POSIX
-8:00:00	Décalage UTC pour l'heure du Pacifique
-8:00	Décalage UTC pour PST (format étendu ISO 8601)
-800	Décalage UTC pour PST (format de base ISO 8601)
-8	Décalage UTC pour PST (format de base ISO 8601)
zoulou	Abréviation militaire de UTC
Z	Forme abrégée de zoulou (également dans la norme ISO 8601)

Reportez-vous à la section 8.5.3 pour plus d'informations sur la manière de spécifier les fuseaux horaires.

8.5.1.3. Horodatage

Les données d'entrée valides pour les types d'horodatage consistent en la concaténation d'une date et d'une heure, suivie de un fuseau horaire facultatif, suivi de l'abréviation AD ou BC facultative. (Il est également possible d'indiquer AD/BC avant le fuseau horaire.) (fuseau horaire, mais ce n'est pas l'ordre privilégié.) Ainsi :

08/01/1999 04:05:06

et:

08/01/1999 04:05:06 -8:00

sont des valeurs valides, conformes à la norme ISO 8601. De plus, le format courant est le suivant :

8 janvier 1999, 04:05:06 PST

est pris en charge.

La norme SQL fait la distinction entre un horodatage sans fuseau horaire et un horodatage avec fuseau horaire.

Les caractères de fuseau horaire sont indiqués par la présence d'un symbole « + » ou « - » et d'un décalage horaire après l'heure. Par conséquent, selon la norme,

HORODATAGE '2004-10-19 10:23:54'

est un horodatage sans fuseau horaire, tandis que

HORODATAGE '2004-10-19 10:23:54+02'

Il s'agit d'un horodatage avec fuseau horaire. PostgreSQL n'examine jamais le contenu d'une chaîne littérale. avant de déterminer son type, et traitera donc les deux éléments ci-dessus comme des horodatages sans Fuseau horaire. Pour qu'une valeur littérale soit traitée comme un horodatage avec fuseau horaire, attribuez-lui le fuseau horaire. type explicite correct :

HORODATAGE AVEC FUSEAU HORAIRE '2004-10-19 10:23:54+02'

Dans une valeur identifiée comme un horodatage sans fuseau horaire, PostgreSQL

ignore silencieusement toute indication de fuseau horaire. Autrement dit, la valeur résultante est dérivée des champs de date/heure. dans la chaîne d'entrée, et n'est pas ajustée en fonction du fuseau horaire.

Pour les horodatages avec fuseau horaire, une chaîne d'entrée incluant un fuseau horaire explicite est requise. sera convertie en UTC (Temps Universel Coordonné) en utilisant le décalage approprié pour cette heure fuseau horaire. Si aucun fuseau horaire n'est spécifié dans la chaîne de caractères saisie, le fuseau horaire indiqué sera utilisé par défaut. L'heure est définie par le paramètre de fuseau horaire du système et convertie en UTC en utilisant le décalage horaire. fuseau horaire. Dans les deux cas, la valeur est stockée en interne au format UTC, et l'heure initialement indiquée ou supposée. La zone n'est pas conservée.

Lorsqu'un horodatage avec fuseau horaire est affiché, il est toujours converti de UTC vers le fuseau horaire. Le fuseau horaire actuel est affiché comme l'heure locale de ce fuseau. Pour afficher l'heure dans un autre fuseau horaire, cliquez ici. zone, soit changez de fuseau horaire, soit utilisez la construction AT TIME ZONE (voir section 9.9.4).

Conversions entre horodatage sans fuseau horaire et horodatage avec fuseau horaire

On suppose généralement que l'horodatage sans fuseau horaire doit être pris ou donné comme fuseau horaire : heure locale. Un fuseau horaire différent peut être spécifié pour la conversion en utilisant AT TIME. ZONE.

8.5.1.4. Valeurs spéciales

PostgreSQL prend en charge plusieurs valeurs d'entrée de date/heure spéciales pour plus de commodité, comme indiqué dans le tableau 8.13. Les valeurs infini et -infini sont spécialement représentées au sein du système et seront

affichés sans modification ; mais les autres ne sont que des abréviations qui seront converties en caractères ordinaires.

Les valeurs de date et d'heure sont converties en une heure précise lors de la lecture. (En particulier, « now » et les chaînes de caractères associées sont converties en une heure précise.)

(valeur dès qu'elles sont lues.) Toutes ces valeurs doivent être placées entre guillemets simples lorsqu'elles sont utilisées. comme constantes dans les commandes SQL.

Tableau 8.13. Saisie spéciale de date/heure

Chaîne d'entrée	Types valides	Description
époque	date, horodatage	1970-01-01 00:00:00+00 (Unix (temps système zéro))
infini	date, horodatage, intervalle	postérieur à tous les autres horodatages
-infini	date, horodatage, intervalle	plus tôt que jamais auparavant timbres
maintenant	date, heure, horodatage de l'heure de début de la transaction en cours	
aujourd'hui	date, horodatage	minuit (00:00) aujourd'hui
demain	date, horodatage	minuit (00:00) demain
hier	date, horodatage	minuit (00:00) hier
allballs	temps	00:00:00.00 UTC

Les fonctions compatibles SQL suivantes peuvent également être utilisées pour obtenir la valeur de l'heure actuelle pour le type de données correspondant : CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, LOCALTIME, LOCALTIMESTAMP. (Voir section 9.9.5.) Notez qu'il s'agit de fonctions SQL et qu'elles ne sont pas reconnues.

dans les chaînes de données d'entrée.

Prudence

Bien que les chaînes de caractères « now », « today », « tomorrow » et « yesterday » soient parfaitement utilisables dans les commandes SQL interactives, elles peuvent avoir un comportement surprenant lorsque la commande est enregistrée.

exécutés ultérieurement, par exemple dans les requêtes préparées, les vues et les définitions de fonctions. La chaîne peut être convertie en une valeur temporelle spécifique qui continue d'être utilisée longtemps après être devenue obsolète. Utilisez plutôt une fonction SQL dans ce genre de contexte. Par exemple, `CURRENT_DATE + 1` est plus sûr que `'demain'::date`.

8.5.2. Sortie date/heure

Le format de sortie des types date/heure peut être défini sur l'un des quatre styles suivants : ISO 8601, SQL (Ingres), Le format de date standard est le format POSTGRES (format Unix) ou allemand. Le format par défaut est le format ISO. (SQL) La norme exige l'utilisation du format ISO 8601. Le nom du format de sortie « SQL » est historique. accident.) Le tableau 8.14 présente des exemples de chaque style de sortie. La sortie des types date et heure est En général, seule la date ou l'heure est conservée, conformément aux exemples donnés. Cependant, les POSTGRES Ce style génère des valeurs de date uniquement au format ISO.

Tableau 8.14. Styles d'affichage de la date et de l'heure

Spécifications de style	Description	Exemple
ISO	Norme ISO 8601, SQL	17/12/1997 07:37:16-08
SQL	style traditionnel	17/12/1997 07:37:16.00 PST
Postgres	style original	Mercredi 17 décembre 1997 à 07:37:16 PST
Allemand	style régional	17/12/1997 07:37:16.00 PST

Note

La norme ISO 8601 spécifie l'utilisation de la lettre T majuscule pour séparer la date et l'heure. PostgreSQL Il accepte ce format en entrée, mais en sortie, il utilise un espace au lieu de T, comme indiqué ci-dessus. est pour des raisons de lisibilité et de cohérence avec la RFC 3339 ainsi qu'avec certains autres systèmes de bases de données.

Dans les styles SQL et POSTGRES, le jour précède le mois si l'ordre des champs est spécifié au format JJ/MM/AAAA ; sinon, le mois précède le jour. (Voir la section 8.5.1 pour savoir comment ce paramètre influe également sur l'interprétation des valeurs saisies.) Le tableau 8.15 présente des exemples.

Tableau 8.15. Conventions d'ordre des dates

format de date	Paramètres de saisie	Ordre	Exemple de sortie
SQL, DMY	jour/mois/année	mois/	17/12/1997 15:37:16.00 CET
SQL, MDY	jour/année		17/12/1997 07:37:16.00 PST
Postgres, DMY	jour/mois/année		Mercredi 17 décembre 1997 à 07:37:16 PST

Selon la norme ISO, le fuseau horaire est toujours indiqué par un décalage numérique signé par rapport à UTC, avec un signe positif. Panneau utilisé pour les zones à l'est de Greenwich. Le décalage sera indiqué par hh (heures seulement) s'il s'agit d'un entier. Nombre d'heures, sinon au format hh:mm s'il s'agit d'un nombre entier de minutes, sinon au format hh:mm:ss. (Le troisième cas) Ceci est impossible avec les normes de fuseaux horaires modernes, mais peut apparaître lors de la manipulation d'horodatages. (qui sont antérieurs à l'adoption des fuseaux horaires standardisés.) Dans les autres formats de date, le fuseau horaire est affiché sous forme d'abréviation alphabétique si elle est couramment utilisée dans la zone actuelle. Sinon, elle apparaît sous forme de Décalage numérique signé au format de base ISO 8601 (hh ou hhmm).

Le style de date/heure peut être sélectionné par l'utilisateur à l'aide de la commande `SET datestyle`, `DateStyle` paramètre dans le fichier de configuration `postgresql.conf`, ou la variable d'environnement `PGDATESTYLE` sur le serveur ou le client.

¹ <https://datatracker.ietf.org/doc/html/rfc3339>

La fonction de formatage `to_char` (voir section 9.8) est également disponible comme moyen plus flexible de formater la sortie date/heure.

8.5.3. Fuseaux horaires

Les fuseaux horaires, et les conventions relatives aux fuseaux horaires, sont influencés par des décisions politiques, et pas seulement par la géométrie terrestre.

Les fuseaux horaires se sont quelque peu standardisés dans le monde au cours du XXe siècle, mais restent sujets à des modifications arbitraires, notamment en ce qui concerne l'heure d'été. PostgreSQL utilise la base de données IANA (Olson), largement répandue, pour obtenir des informations sur l'historique des fuseaux horaires. Pour les dates futures, on suppose que les dernières règles connues pour un fuseau horaire donné continueront d'être appliquées indéfiniment.

PostgreSQL s'efforce d'être compatible avec les définitions standard SQL pour une utilisation courante. Cependant, la norme SQL présente un mélange étrange de types et de fonctionnalités de date et d'heure. Deux problèmes évidents se posent :

Bien que le type `date` ne puisse pas être associé à un fuseau horaire, le type `heure` le peut. Dans la pratique, les fuseaux horaires n'ont guère de sens s'ils ne sont pas associés à une date et à une heure, car le décalage horaire peut varier au cours de l'année en fonction de l'heure d'été.

- Le fuseau horaire par défaut est défini comme un décalage numérique constant par rapport à UTC. Il est donc impossible de tenir compte de l'heure d'été lors de calculs de dates et d'heures entre les périodes d'heure d'été et d'hiver.

Pour pallier ces difficultés, nous recommandons d'utiliser des types `date/heure` contenant à la fois la date et l'heure lors de la gestion des fuseaux horaires. Nous déconseillons l'utilisation du type « `time` » avec fuseau horaire (bien qu'il soit pris en charge par PostgreSQL pour les applications existantes et pour la conformité à la norme SQL).

PostgreSQL utilise votre fuseau horaire local pour tout type contenant uniquement une date ou une heure.

Toutes les dates et heures prenant en compte le fuseau horaire sont stockées en interne en UTC. Elles sont converties en heure locale dans le fuseau horaire spécifié par le paramètre de configuration `TimeZone` avant d'être affichées au client.

PostgreSQL vous permet de spécifier les fuseaux horaires de trois manières différentes :

- Un nom de fuseau horaire complet, par exemple `America/New_York`. Les noms de fuseaux horaires reconnus sont listés dans la vue `pg_timezone_names` (voir section 52.32). PostgreSQL utilise les données de fuseaux horaires IANA, largement répandues, à cette fin ; par conséquent, ces mêmes noms de fuseaux horaires sont également reconnus par d'autres logiciels.
- Une abréviation de fuseau horaire, par exemple `PST`. Cette spécification définit simplement un décalage par rapport à l'UTC, contrairement aux noms complets de fuseaux horaires qui peuvent également inclure des règles de passage à l'heure d'été. Les abréviations reconnues sont listées dans la vue `pg_timezone_abbrevs` (voir section 52.31). Vous ne pouvez pas définir les paramètres de configuration `TimeZone` ou `log_timezone` avec une abréviation de fuseau horaire, mais vous pouvez utiliser des abréviations dans les valeurs d'entrée de date/heure et avec l'opérateur `AT TIME ZONE`.
- Outre les noms et abréviations de fuseaux horaires, PostgreSQL accepte les spécifications de fuseaux horaires de type `POSIX`, comme décrit dans la section B.5. Cette option n'est généralement pas préférable à l'utilisation d'un fuseau horaire nommé, mais elle peut s'avérer nécessaire si aucune entrée de fuseau horaire IANA appropriée n'est disponible.

En résumé, voici la différence entre les abréviations et les noms complets : les abréviations indiquent un décalage horaire précis par rapport à l'UTC, tandis que de nombreux noms complets impliquent une règle d'heure d'été locale et peuvent donc avoir deux décalages horaires UTC. Par exemple, `2014-06-04 12:00 America/New_York` correspond à midi, heure locale à New York, qui, à cette date, était l'heure d'été de l'Est (UTC-4). Ainsi, `2014-06-04 12:00 EDT` désigne ce même instant. En revanche, `2014-06-04 12:00 EST` désigne midi, heure normale de l'Est (UTC-5), que l'heure d'été soit en vigueur ou non à cette date.

Pour compliquer les choses, certaines juridictions ont utilisé la même abréviation de fuseau horaire pour désigner différents décalages UTC à différentes époques ; par exemple, à Moscou, `MSK` a signifié UTC+3 certaines années et UTC+4 d'autres. PostgreSQL interprète ces abréviations selon leur signification (ou leur signification la plus récente) à la date spécifiée ; mais, comme pour l'exemple `EST` ci-dessus, cela ne correspond pas nécessairement à l'heure civile locale à cette date.

Dans tous les cas, les noms et abréviations des fuseaux horaires sont reconnus sans distinction de casse. (Il s'agit d'un changement) (à partir des versions de PostgreSQL antérieures à la version 8.2, qui étaient sensibles à la casse dans certains contextes mais pas dans d'autres.)

Ni les noms ni les abréviations des fuseaux horaires ne sont intégrés en dur au serveur ; ils sont obtenus à partir de fichiers de configuration stockés sous `.../share/timezone/` et `.../share/timezonesets/` de le répertoire d'installation (voir section B.4).

Le paramètre de configuration `TimeZone` peut être défini dans le fichier `postgresql.conf`, ou dans n'importe quel autre fichier de configuration. D'autres méthodes standard sont décrites au chapitre 19. Il existe également des méthodes spécifiques pour le paramétrer :

- La commande SQL `SET TIME ZONE` définit le fuseau horaire de la session. Il s'agit d'une alternative orthographe de `SET TIMEZONE TO` avec une syntaxe plus compatible avec les spécifications SQL.
- La variable d'environnement `PGTZ` est utilisée par les clients `libpq` pour envoyer une commande `SET TIME ZONE` au serveur lors de la connexion.

8.5.4. Entrée d'intervalle

Les valeurs d'intervalle peuvent être écrites en utilisant la syntaxe détaillée suivante :

`[@] unité de quantité [unité de quantité...] [direction]`

où la quantité est un nombre (éventuellement signé) ; l'unité est la microseconde, la milliseconde, la seconde, la minute, l'heure, le jour, la semaine, le mois, l'année, la décennie, le siècle, le millénaire, ou leurs abréviations ou pluriels ; la direction peut être « il y a » ou vide. Le symbole arobase (@) est facultatif.

Les quantités des différentes unités sont additionnées implicitement en tenant compte des signes.

tous les champs. Cette syntaxe est également utilisée pour l'affichage par intervalles, si `IntervalStyle` est défini sur `postgres_verbose`.

Les quantités de jours, d'heures, de minutes et de secondes peuvent être spécifiées sans indication explicite d'unité.

Par exemple, « 1 12:59:10 » se lit de la même manière que « 1 jour 12 heures 59 minutes 10 secondes ». De plus,

On peut spécifier une combinaison d'années et de mois avec un tiret ; par exemple, « 200-10 » se lit « 200-10 ».

Identique à « 200 ans et 10 mois ». (Ces formes abrégées sont en fait les seules autorisées par la norme SQL, et sont utilisés pour la sortie lorsque `IntervalStyle` est défini sur `sql_standard`.)

Les valeurs d'intervalle peuvent également être écrites sous forme d'intervalles de temps ISO 8601, en utilisant soit le « format avec désignateurs » de la section 4.4.3.2 de la norme, soit le « format alternatif » de la section 4.4.3.3.

Les désignateurs ressemblent à ceci :

`P` unité de quantité [unité de quantité ...] [`T` [unité de quantité ...]]

La chaîne doit commencer par un `P` et peut inclure un `T` qui introduit les unités de temps de la journée. Les options disponibles

Les abréviations des unités sont données dans le tableau 8.16. Les unités peuvent être omises et peuvent être spécifiées dans n'importe quel ordre. mais les unités inférieures à un jour doivent apparaître après `T`. En particulier, la signification de `M` dépend de si c'est avant ou après `T`.

Tableau 8.16. Abréviations des unités d'intervalle ISO 8601

Abréviation	Signification
Y	Années
M	Mois (dans la partie date)
W	Semaines
D	Jours
H	Heures
M	Minutes (dans la partie temps)
S	Secondes

Dans un autre format :

P [années-mois-jours]

[T heures:minutes:secondes]

La chaîne doit commencer par P, et un T sépare la date et l'heure de l'intervalle. Les valeurs sont exprimées sous forme de nombres, comme les dates au format ISO 8601.

Lors de la définition d'une constante d'intervalle avec une spécification de champs, ou lors de l'affectation d'une chaîne de caractères à une colonne d'intervalle définie avec une spécification de champs, l'interprétation des quantités non marquées dépend des champs. Par exemple, INTERVAL '1' YEAR est interprété comme 1 an, tandis que INTER-VAL '1' signifie 1 seconde. De plus, les valeurs de champ situées à droite du champ le moins significatif autorisé par la spécification de champs sont ignorées. Par exemple, écrire INTERVAL '1 jour 2:03:04'

L'option HEURE EN MINUTE supprime le champ des secondes, mais pas celui du jour.

Selon la norme SQL, tous les champs d'une valeur d'intervalle doivent avoir le même signe. Un signe négatif initial s'applique donc à tous les champs ; par exemple, le signe négatif dans l'intervalle littéral « -1 2:03:04 » s'applique à la fois aux jours et aux parties heure/minute/seconde. PostgreSQL autorise des signes différents pour les champs et, traditionnellement, considère chaque champ de la représentation textuelle comme ayant un signe indépendant. Ainsi, dans cet exemple, la partie heure/minute/seconde est considérée comme positive. Si 'IntervalStyle' est défini sur 'sql_standard', un signe initial est considéré comme s'appliquant à tous les champs (mais uniquement si aucun autre signe n'apparaît). Sinon, l'interprétation traditionnelle de PostgreSQL est utilisée. Pour éviter toute ambiguïté, il est recommandé d'ajouter un signe explicite à chaque champ si l'un d'eux est négatif.

En interne, les valeurs d'intervalle sont stockées sous forme de trois champs entiers : mois, jours et microsecondes. Ces champs sont conservés séparément car le nombre de jours dans un mois varie, et une journée peut compter 23 ou 25 heures en cas de passage à l'heure d'été. Une chaîne de caractères représentant un intervalle et utilisant d'autres unités est normalisée dans ce format, puis reconstruite de manière standardisée pour l'affichage, par exemple :

```
SELECT '2 ans 15 mois 100 semaines 99 heures 123456789 millisecondes'::interval; intervalle
```

```
-----
3 ans 3 mois 700 jours 133:17:36.789
```

Ici, les semaines, considérées comme « 7 jours », ont été conservées séparément, tandis que les unités de temps plus petites et plus grandes ont été combinées et normalisées.

Les valeurs des champs de saisie peuvent comporter des parties fractionnaires, par exemple « 1,5 semaine » ou « 01:02:03,45 ». Cependant, comme la fonction 'interval' ne stocke en interne que des champs entiers, les valeurs fractionnaires doivent être converties en unités plus petites. Les parties fractionnaires d'unités supérieures à un mois sont arrondies à l'entier le plus proche, par exemple « 1,5 an » devient « 1 an et 6 mois ». Les parties fractionnaires de semaines et de jours sont converties en un nombre entier de jours et de microsecondes, en considérant 30 jours par mois et 24 heures par jour, par exemple « 1,75 mois » devient 1 mois, 22 jours et 12:00:00. Seules les secondes seront affichées sous forme fractionnaire.

Le tableau 8.17 présente quelques exemples d'entrées d'intervalle valides.

Tableau 8.17. Entrée par intervalle

Exemple	Description
1-2	Format standard SQL : 1 an et 2 mois
3 4:05:06	Format standard SQL : 3 jours 4 heures 5 minutes 6 secondes
1 an, 2 mois, 3 jours, 4 heures, 5 minutes et 6 secondes	Format Postgres traditionnel : 1 an 2 mois 3 jours 4 heures 5 minutes 6 secondes
P1Y2M3DT4H5M6S	ISO 8601 « Format avec désignateurs » : même signification que ci-dessus

Exemple	Description
P0001-02-03T04:05:06	ISO 8601 « format alternatif » : même signification que au-dessus de

8.5.5. Sortie par intervalle

Comme expliqué précédemment, PostgreSQL stocke les valeurs d'intervalle sous forme de mois, de jours et de microsecondes. Pour l'affichage, le champ « mois » est converti en années et mois en divisant par 12. Le champ « jours » est Affiché tel quel. Le champ des microsecondes est converti en heures, minutes, secondes et fractions de seconde. Ainsi, les mois, les minutes et les secondes ne seront jamais affichés comme dépassant les plages 0–11, 0–59 et 0–59 respectivement, tandis que les champs affichés pour les années, les jours et les heures peuvent être assez longs. (Les fonctions `justi-fy_days` et `justify_hours` peuvent être utilisées s'il est souhaitable de transposer les jours longs ou (valeurs des heures dans le champ supérieur suivant.)

Le format de sortie du type intervalle peut être défini sur l'un des quatre styles suivants : `sql_standard`, `postgres`, `postgres_verbose` ou `iso_8601`, à l'aide de la commande `SET intervalstyle`.

Le format par défaut est celui de PostgreSQL. Le tableau 8.18 présente des exemples pour chaque style de sortie.

Le style `sql_standard` produit une sortie conforme à la spécification de la norme SQL pour chaînes littérales d'intervalle, si la valeur de l'intervalle respecte les restrictions de la norme (soit année-mois seulement, soit (uniquement de jour, sans mélange de composantes positives et négatives). Sinon, le résultat ressemble à ceci : une chaîne littérale standard année-mois suivie d'une chaîne littérale jour, avec des signes explicites ajoutés à lever l'ambiguïté des intervalles à signes mixtes.

Le résultat obtenu avec le style `postgres` correspond à celui des versions de PostgreSQL antérieures à la version 8.4.

Le paramètre `DateStyle` a été défini sur ISO.

Le résultat obtenu avec le style `postgres_verbose` correspond à celui des versions de PostgreSQL antérieures à 8.4 lorsque le paramètre `DateStyle` était défini sur une sortie non ISO.

Le format de sortie de l'ISO 8601 correspond au « format avec désignateurs » décrit dans la section 4.4.3.2. de la norme ISO 8601.

Tableau 8.18. Exemples de style de sortie par intervalles

Spécifications de style	Intervalle année-mois, intervalle	jour-heure, intervalle mixte	
<code>sql_standard</code>	1-2	3 4:05:06	-1-2 +3 -4:05:06
PostgreSQL	1 an et 2 mois	3 jours 04:05:06	-1 an -2 mois +3 jours -04:05:06
<code>postgres_verbose @ 1 an 2 mois</code>		@ 3 jours 4 heures 5 minutes 6 secondes	@ 1 an 2 mois -3 jours 4 heures 5 minutes 6 # y a quelques secondes
<code>iso_8601</code>	P1Y2M	P3DT4H5M6S	P-1Y-2M3D T-4H-5M-6S

8.6. Type booléen

PostgreSQL fournit le type SQL standard booléen ; voir le tableau 8.19. Le type booléen peut avoir plusieurs états : « vrai », « faux » et un troisième état, « inconnu », qui est représenté par la valeur nulle SQL.

Tableau 8.19. Type de données booléennes

Nom	Taille de stockage :	Description
booléen	1 octet	état de vrai ou faux

Les constantes booléennes peuvent être représentées dans les requêtes SQL par les mots clés SQL `TRUE`, `FALSE` et `NULL`.

La fonction d'entrée de type de données pour le type booléen accepte les représentations de chaîne suivantes pour l'état « vrai » :

```
vrai
Oui
sur
1
```

et ces représentations de l'état « faux » :

```
FAUX
Non
désactivé
0
```

Les préfixes uniques de ces chaînes sont également acceptés, par exemple t ou n. Les espaces blancs en début ou en fin de chaîne sont ignorés, et la casse n'a pas d'importance.

La fonction de sortie de type de données pour le type booléen émet toujours soit t soit f, comme le montre l'exemple 8.2.

Exemple 8.2. Utilisation du type booléen

```
CRÉER TABLE test1 (a booléen, b texte);
INSÉRER DANS test1 VALEURS (TRUE, 'sic est');
INSERT INTO test1 VALUES (FALSE, 'non est');
SÉLECTIONNER * DE test1;
```

```
un |      b
---+-----
t | sic est f | non est
```

```
SÉLECTIONNER * DE test1 OÙ a;
```

```
un |      b
---+-----
t | sic est
```

Les mots clés TRUE et FALSE constituent la méthode privilégiée (conforme à SQL) pour écrire des constantes booléennes dans les requêtes SQL. Toutefois, vous pouvez également utiliser leur représentation sous forme de chaîne de caractères en suivant la syntaxe générique des constantes littérales décrite dans la section 4.1.2.7, par exemple 'yes'::boolean.

Notez que l'analyseur syntaxique comprend automatiquement que TRUE et FALSE sont de type booléen, mais ce n'est pas le cas pour NULL, car ce dernier peut être de n'importe quel type. Dans certains contextes, il peut donc être nécessaire de convertir explicitement NULL en booléen, par exemple avec NULL::boolean. Inversement, cette conversion peut être omise pour une chaîne littérale de type booléen lorsque l'analyseur syntaxique peut déduire que la valeur littérale est nécessairement de type booléen.

8.7. Types énumérés

Les types énumérés (enum) sont des types de données qui constituent un ensemble statique et ordonné de valeurs. Ils sont équivalents aux types enum pris en charge par de nombreux langages de programmation. Par exemple, les jours de la semaine ou un ensemble de valeurs d'état pour une donnée constituent un type enum.

8.7.1. Déclaration des types énumérés

Les types énumérés sont créés à l'aide de la commande CREATE TYPE, par exemple :

```
CRÉER LE TYPE mood AS ENUM ('sad', 'ok', 'happy');
```

Une fois créé, le type énuméré peut être utilisé dans les définitions de tables et de fonctions comme n'importe quel autre type :

```
CRÉER LE TYPE mood AS ENUM ('sad', 'ok', 'happy');
CRÉER TABLE personne (
    nom texte,
    humeur_actuelle mood
);
INSÉRER DANS personne VALEURS ('Moe', 'heureux');
SELECT * FROM person WHERE current_mood = 'happy';
    nom | humeur_actuelle
-----+-----
    Moe | heureux (1 ligne)
```

8.7.2. Commande

L'ordre des valeurs dans un type énuméré correspond à l'ordre dans lequel elles ont été listées lors de la création du type. Tous les opérateurs de comparaison standard et les fonctions d'agrégation associées sont pris en charge pour les énumérations. Par exemple :

```
INSÉRER DANS personne VALEURS ('Larry', 'triste');
INSÉRER DANS personne VALEURS ('Curly', 'ok');
SELECT * FROM person WHERE current_mood > 'sad'; nom | current_mood
-----+-----
    Moe | heureux
    Bouclé | ok (2
lignes)

SELECT * FROM person WHERE current_mood > 'sad' ORDER BY current_mood; name | current_mood
-----+-----
    Bouclé | ok
    Moe | heureux (2 lignes)

SÉLECTIONNER le nom
DE la personne
OÙ humeur_actuelle = (SÉLECTIONNER MIN(humeur_actuelle) DE personne);
    nom
-----
    Larry (1
rangée)
```

8.7.3. Type de sécurité

Chaque type de données énuméré est distinct et ne peut être comparé à un autre type énuméré. Voir cet exemple :

```
CRÉER LE TYPE bonheur COMME ÉNUMÉRATION ('heureux', 'très heureux', 'extatique');
CRÉER TABLE holidays ( num_weeks
    entier, happiness happiness
);
```

```

INSÉRER DANS holidays(num_weeks,happiness) VALUES (4, 'happy');
INSÉRER DANS holidays(num_weeks,happiness) VALUES (6, 'très heureux');
INSÉRER DANS holidays(num_weeks,happiness) VALUES (8, 'ecstatic');
INSÉRER DANS holidays(num_weeks,happiness) VALUES (2, 'sad');
ERREUR : valeur d'entrée invalide pour l'énumération happiness : « sad »

SELECT person.name, holidays.num_weeks FROM person, holidays
    OÙ personne.humeur_actuelle = vacances.bonheur;
ERREUR : l'opérateur n'existe pas : mood = happiness

```

Si vous avez vraiment besoin de faire quelque chose comme ça, vous pouvez soit écrire un opérateur personnalisé, soit ajouter une instruction explicite. conversions à votre requête :

```

SELECT person.name, holidays.num_weeks FROM person, holidays
    OÙ personne.humeur_actuelle::texte = vacances.bonheur::texte;
nom | nombre_semaines
-----+-----
Moe | (1                4
ligne)

```

8.7.4. Détails de mise en œuvre

Les étiquettes d'énumération sont sensibles à la casse ; ainsi, « happy » n'est pas la même chose que « HAPPY ». Espaces blancs dans les étiquettes est également important.

Bien que les types énumérés soient principalement destinés à des ensembles statiques de valeurs, il est possible d'en ajouter de nouvelles. Les valeurs peuvent être ajoutées à un type énuméré existant, et les valeurs peuvent être renommées (voir ALTER TYPE). Les valeurs existantes ne peuvent pas être modifiées. ne peuvent pas être supprimés d'un type énuméré, ni l'ordre de tri de ces valeurs ne peut être modifié, sauf en les supprimant. et en recréant le type énuméré.

Une valeur d'énumération occupe quatre octets sur le disque. La longueur de l'étiquette textuelle d'une valeur d'énumération est limitée par le paramètre NAMEDATALEN compilé dans PostgreSQL ; dans les versions standard, cela signifie au maximum 63 octets.

Les traductions des valeurs d'énumération internes en étiquettes textuelles sont conservées dans le catalogue système pg_enum. Consulter directement ce catalogue peut s'avérer utile.

8.8. Types géométriques

Les types de données géométriques représentent des objets spatiaux bidimensionnels. Le tableau 8.20 présente les types géométriques. disponible dans PostgreSQL.

Tableau 8.20. Types géométriques

Nom	Taille de stockage :	Description	Représentation
indiquer	16 octets	Point sur un plan	(x,y)
doubler	24 octets	Ligne infinie	{ABC}
lseg	32 octets	Segment de droite fini	[(x1,y1),(x2,y2)]
boîte	32 octets	boîte rectangulaire	(x1,y1),(x2,y2)
chemin	16+16n octets	Chemin fermé (similaire à un polygone)	((x1,y1),...)
chemin	16+16n octets	Chemin ouvert	[(x1,y1),...]
polygone	40+16n octets	Polygone (similaire à un chemin fermé)	((x1,y1),...)
cercle	24 octets	Cercle	<(x,y),r> (centre point et rayon)

Dans tous ces types, les coordonnées individuelles sont stockées sous forme de nombres à double précision (float8).

Un ensemble complet de fonctions et d'opérateurs est disponible pour effectuer diverses opérations géométriques telles que :
La mise à l'échelle, la translation, la rotation et la détermination des intersections sont expliquées dans la section 9.11.

8.8.1. Points

Les points sont les éléments de base bidimensionnels des types géométriques. Valeurs du type point sont spécifiées en utilisant l'une des syntaxes suivantes :

```
( x , y )
x , y
```

où x et y sont les coordonnées respectives, sous forme de nombres à virgule flottante.

Les points sont affichés en utilisant la première syntaxe.

8.8.2. Lignes

Les droites sont représentées par l'équation linéaire $Ax + By + C = 0$, où A et B ne sont pas simultanément nuls. Valeurs
Les entrées et sorties de type ligne se présentent sous la forme suivante :

```
{ A, B, C }
```

Vous pouvez également utiliser l'un des formulaires suivants pour la saisie :

```
( x1 , y1 ) , ( x2 , y2 )
(( ( x1 , y1 ) , ( x2 , y2 ) , y2 ) )
( x1 , y1 ) , y1 , ( x2 , y2 )
x1 , x2 , y2
```

où (x1,y1) et (x2,y2) sont deux points différents sur la ligne.

8.8.3. Segments de droite

Les segments de droite sont représentés par des paires de points qui en sont les extrémités. Valeurs de type
Les segments logiques (lseg) sont spécifiés à l'aide de l'une des syntaxes suivantes :

```
( x1 , y1 ) , ( x2 , y2 )
(( ( x1 , y1 ) , ( x2 , y2 ) , y2 ) )
( x1 x1 , y1 ) , ( x2 x2 , y2 )
x1 , x2 , y2
```

où (x1,y1) et (x2,y2) sont les extrémités du segment de droite.

Les segments de ligne sont affichés en utilisant la première syntaxe.

8.8.4. Boîtes

Les boîtes sont représentées par des paires de points correspondant aux coins opposés de la boîte. Les valeurs de type boîte sont
spécifiées en utilisant l'une des syntaxes suivantes :

```
( ( x1 , y1 ) , ( x2 , y2 ) )
( x1 , y1 ) , y1 , ( x2 , y2 )
x1 , x2 , y2
```

où (x_1, y_1) et (x_2, y_2) sont deux coins opposés quelconques de la boîte.

Les boîtes sont affichées en utilisant la deuxième syntaxe.

Deux coins opposés quelconques peuvent être fournis en entrée, mais les valeurs seront réorganisées si nécessaire pour le stockage. les coins supérieur droit et inférieur gauche, dans cet ordre.

8.8.5. Chemins

Les chemins sont représentés par des listes de points connectés. Les chemins peuvent être ouverts, c'est-à-dire que le premier et le dernier point de la liste sont des points connectés. Les listes sont considérées comme non connectées, ou fermées, tandis que les premier et dernier points sont considérés comme connectés.

Les valeurs de type chemin sont spécifiées à l'aide de l'une des syntaxes suivantes :

```
[ ( x1 , y1 ) , , ... , ( xn , yn ) , ]
[ ( x1 , y1 ) , , ... , ( xn , yn ) , ) ]
( x1 , y1 ) , y1 , , ... , ( xn
( x1 x1 y1 , ... , xn , yn )
, ... , xn , yn
```

où les points représentent les extrémités des segments de droite constituant le chemin. Crochets ([]) indique un chemin ouvert, tandis que les parenthèses ([]) indiquent un chemin fermé. Lorsque les parenthèses extérieures sont omises, car dans les syntaxes troisième à cinquième, un chemin fermé est supposé.

Les chemins sont affichés en utilisant la première ou la deuxième syntaxe, selon le cas.

8.8.6. Polygones

Les polygones sont représentés par des listes de points (les sommets du polygone). Les polygones sont très similaires aux chemins fermés ; la différence sémantique essentielle est qu'un polygone est considéré comme incluant la zone à l'intérieur, alors qu'il n'y a pas de chemin.

Une différence importante d'implémentation entre les polygones et les chemins réside dans la représentation stockée. Un polygone inclut son plus petit rectangle englobant. Cela accélère certaines opérations de recherche, bien que le calcul de la boîte englobante ajoute une surcharge lors de la construction de nouveaux polygones.

Les valeurs de type polygone sont spécifiées à l'aide de l'une des syntaxes suivantes :

```
( ( x1 , y1 ) , , ... , ( xn , yn ) , )
( x1 , y1 ) , y1 , , ... , ( xn , yn )
( x1 , y1 , ... , xn , yn )
x1 , ... , xn , yn
```

où les points sont les extrémités des segments de droite constituant la frontière du polygone.

Les polygones sont affichés en utilisant la première syntaxe.

8.8.7. Cercles

Les cercles sont représentés par un centre et un rayon. Les valeurs de type cercle sont spécifiées à l'aide de n'importe quel des syntaxes suivantes :

```
< ( x ( ( x , y ) , , r >
( x , y ) , y ) , r )
, r
x , y , r
```

où (x,y) est le point central et r est le rayon du cercle.

Les cercles sont affichés en utilisant la première syntaxe.

8.9. Types d'adresses réseau

PostgreSQL propose des types de données pour stocker les adresses IPv4, IPv6 et MAC, comme indiqué dans le tableau 8.21. Il est préférable d'utiliser ces types plutôt que des types texte brut pour stocker les adresses réseau, car ces types proposent une vérification des erreurs de saisie et des opérateurs et fonctions spécialisés (voir section 9.12).

Tableau 8.21. Types d'adresses réseau

Nom	Taille du stockage	Description
cidre	7 ou 19 octets	Réseaux IPv4 et IPv6
inet	7 ou 19 octets	Hôtes et réseaux IPv4 et IPv6
adresse MAC	6 octets	adresses MAC
macaddr8	8 octets	Adresses MAC (format EUI-64)

Lors du tri des types de données inet ou cidr, les adresses IPv4 seront toujours triées avant les adresses IPv6, y compris les adresses IPv4 encapsulées ou mappées à des adresses IPv6, telles que ::10.2.3.4 ou ::ffff:10.4.3.2.

8.9.1. inet

Le type inet contient une adresse hôte IPv4 ou IPv6, et éventuellement son sous-réseau, le tout dans un seul champ. est représenté par le nombre de bits d'adresse réseau présents dans l'adresse de l'hôte (le « masque de sous-réseau »). Si le masque de sous-réseau est de 32 et que l'adresse est de type IPv4, la valeur n'indique pas un sous-réseau, mais seulement un hôte unique. En IPv6, la longueur d'une adresse est de 128 bits, ce qui permet de spécifier une adresse hôte unique. Notez que si vous souhaitez Pour n'accepter que les réseaux, vous devez utiliser le type cidr plutôt que inet.

Le format d'entrée pour ce type est adresse/y, où adresse est une adresse IPv4 ou IPv6 et y est le nombre de bits du masque de sous-réseau. Si la partie /y est omise, le masque de sous-réseau est considéré comme étant de 32 bits pour IPv4, ou 128 pour IPv6, la valeur ne représentant donc qu'un seul hôte. À l'affichage, la partie /y est omise. si le masque de sous-réseau spécifie un seul hôte.

8.9.2. cidr

Le type CIDR contient une spécification de réseau IPv4 ou IPv6. Les formats d'entrée et de sortie sont conformes à la norme Classless. Conventions de routage des domaines Internet. Le format de spécification des réseaux est adresse/y où L'adresse est l'adresse la plus basse du réseau représentée sous forme d'adresse IPv4 ou IPv6, et y est le nombre de bits dans le masque de sous-réseau. Si y est omis, il est calculé à l'aide d'hypothèses issues de l'ancien réseau classifié. système de numérotation, sauf qu'il sera au moins suffisamment grand pour inclure tous les octets écrits dans le Saisie. Spécifier une adresse réseau dont les bits situés à droite du masque de sous-réseau spécifié sont incorrects constitue une erreur.

Le tableau 8.22 présente quelques exemples.

Tableau 8.22. Exemples d'entrée de type cidr

Entrée cidr	Sortie cidr	abréviation(cidr)
192.168.100.128/25	192.168.100.128/25	192.168.100.128/25
192.168/24	192.168.0.0/24	192.168.0/24
192.168/25	192.168.0.0/25	192.168.0.0/25
192.168.1	192.168.1.0/24	192.168.1/24
192,168	192.168.0.0/24	192.168.0/24

Entrée cidr	Sortie cidr	abréviation(cidr)
128.1	128.1.0.0/16	128,1/16
128	128.0.0.0/16	128,0/16
128.1.2	128.1.2.0/24	128.1.2/24
10.1.2	10.1.2.0/24	10.1.2/24
10.1	10.1.0.0/16	10.1/16
10	10.0.0.0/8	10/8
10.1.2.3/32	10.1.2.3/32	10.1.2.3/32
2001:4f8:3:ba::/64	2001:4f8:3:ba::/64	2001:4f8:3:ba/64
2001:4f8:3:ba:2e0:81f-f:fe22:d1f1/128	2001:4f8:3:ba:2e0:81f-f:fe22:d1f1/128	2001:4f8:3:ba:2e0:81f-f:fe22:d1f1/128
::ffff:1.2.3.0/120	::ffff:1.2.3.0/120	::ffff:1.2.3/120
::ffff:1.2.3.0/128	::ffff:1.2.3.0/128	::ffff:1.2.3.0/128

8.9.3. inet vs. cidr

La différence essentielle entre les types de données `inet` et `cidr` est que `inet` accepte des valeurs non nulles bits à droite du masque de sous-réseau, contrairement à `cidr`. Par exemple, `192.168.0.1/24` est valide pour `inet` mais pas pour `cidr`.

Conseil

Si le format de sortie des valeurs `inet` ou `cidr` ne vous convient pas, essayez les fonctions `host`, `text`, et `abréviation`.

8.9.4. macaddr

Le type `macaddr` stocke les adresses MAC, connues par exemple à partir des adresses matérielles des cartes Ethernet. (bien que les adresses MAC soient également utilisées à d'autres fins). Les entrées sont acceptées dans les formats suivants :

```
'08:00:2b:01:02:03'
'08-00-2b-01-02-03'
'08002b:010203'
'08002b-010203'
'0800.2b01.0203'
'0800-2b01-0203'
'08002b010203'
```

Ces exemples spécifient tous la même adresse. Les chiffres de a à 10 sont acceptés en majuscules et en minuscules. f. Le résultat se présente toujours sous la première forme affichée.

La norme IEEE 802-2001 spécifie la deuxième forme présentée (avec des tirets) comme la forme canonique pour Les adresses MAC, et spécifient la première forme (avec deux-points) telle qu'utilisée avec la notation inversée des bits, le bit de poids fort en premier, de sorte que `08-00-2b-01-02-03` = `10:00:D4:80:40:C0`. Cette convention est largement ignorée de nos jours. et cela ne concerne que les protocoles réseau obsolètes (comme Token Ring). PostgreSQL n'en fait aucun cas. dispositions pour l'inversion des bits ; tous les formats acceptés utilisent l'ordre LSB canonique.

Les cinq autres formats d'entrée ne font partie d'aucune norme.

8.9.5. macaddr8

Le type `macaddr8` stocke les adresses MAC au format EUI-64, utilisé notamment pour les adresses matérielles des cartes Ethernet (bien que les adresses MAC servent également à d'autres fins). Ce type accepte les adresses MAC de 6 et 8 octets et les stocke au format 8 octets. Les adresses MAC au format 6 octets seront stockées au format 8 octets, les 4e et 5e octets étant respectivement définis sur FF et FE. Notez que l'IPv6 utilise un format EUI-64 modifié : le 7e bit doit être positionné à 1 après la conversion depuis EUI-48. La fonction `macaddr8_set7bit` permet d'effectuer cette modification. De manière générale, toute entrée composée de paires de chiffres hexadécimaux (sur les limites des octets), séparées éventuellement par `:`, `-` ou `.`, est acceptée. Le nombre de chiffres hexadécimaux doit être de 16 (8 octets) ou de 12 (6 octets). Les espaces en début et en fin de chaîne sont ignorés.

Voici des exemples de formats d'entrée acceptés :

```
'08:00:2b:01:02:03:04:05'
'08-00-2b-01-02-03-04-05'
'08002b:0102030405'
'08002b-0102030405'
'0800.2b01.0203.0405'
'0800-2b01-0203-0405'
'08002b01:02030405'
'08002b0102030405'
```

Ces exemples spécifient tous la même adresse. Les chiffres de a à f sont acceptés en majuscules et en minuscules. Le résultat est toujours affiché sous la première forme indiquée.

Les six derniers formats d'entrée présentés ci-dessus ne font partie d'aucune norme.

Pour convertir une adresse MAC traditionnelle de 48 bits au format EUI-48 au format EUI-64 modifié afin de l'inclure comme partie hôte d'une adresse IPv6, utilisez `macaddr8_set7bit` comme indiqué :

```
SELECT macaddr8_set7bit('08:00:2b:01:02:03');
```

```
-----
macaddr8_set7bit
0a:00:2b:ff:fe:01:02:03
(1 rangée)
```

8.10. Types de chaînes de bits

Les chaînes de bits sont des chaînes de 1 et de 0. Elles peuvent servir à stocker ou à visualiser des masques de bits. Il existe deux types de bits SQL : `bit(n)` et `bit varying(n)`, où `n` est un entier positif.

Les données de type `bit` doivent correspondre exactement à la longueur `n` ; toute tentative de stockage de chaînes de bits plus courtes ou plus longues constitue une erreur. Les données de type `bit varying` ont une longueur variable jusqu'à la longueur maximale `n` ; les chaînes plus longues seront rejetées. Écrire un bit sans spécifier de longueur équivaut à `bit(1)`, tandis qu'un bit variable sans spécification de longueur signifie une longueur illimitée.

Note

Si l'on convertit explicitement une chaîne de bits en `bit(n)`, elle sera tronquée ou complétée par des zéros à droite pour ne comporter que `n` bits, sans générer d'erreur. De même, si l'on convertit explicitement une chaîne de bits en `bit varying(n)`, elle sera tronquée à droite si sa longueur dépasse `n` bits.

Pour plus d'informations sur la syntaxe des constantes de chaînes binaires, veuillez consulter la section 4.1.2.5. Des opérateurs logiques binaires et des fonctions de manipulation de chaînes sont disponibles ; voir la section 9.6.

Exemple 8.3. Utilisation des types de chaînes de bits

```
CRÉER TABLE test (a BIT(3), b BIT VARIABLE(5));
INSÉRER DANS test VALEURS (B'101', B'00');
INSÉRER DANS test VALEURS (B'10', B'101');
```

ERREUR : la longueur de la chaîne binaire 2 ne correspond pas au type bit(3)

```
INSÉRER DANS test VALEURS (B'10'::bit(3), B'101');
SÉLECTIONNER * DE test;
```

```

a | b
---+---
101 | 00 100 |
101
```

Une valeur de chaîne de bits nécessite 1 octet pour chaque groupe de 8 bits, plus 5 ou 8 octets de surcharge selon la longueur de la chaîne (mais les valeurs longues peuvent être compressées ou déplacées hors ligne, comme expliqué dans la section 8.3 pour les chaînes de caractères).

8.11. Types de recherche textuelle

PostgreSQL propose deux types de données conçus pour la recherche plein texte, qui consiste à parcourir un ensemble de documents en langage naturel afin de trouver ceux qui correspondent le mieux à une requête. Le type `tsvector` représente un document sous une forme optimisée pour la recherche textuelle ; le type `tsquery` représente de la même manière une requête textuelle. Le chapitre 12 explique en détail cette fonctionnalité, et la section 9.13 résume les fonctions et opérateurs associés.

8.11.1. tsvector

Une valeur `tsvector` est une liste triée de lexèmes distincts, c'est-à-dire des mots normalisés afin de fusionner les différentes variantes d'un même mot (voir le chapitre 12 pour plus de détails). Le tri et l'élimination des doublons sont effectués automatiquement lors de la saisie, comme illustré dans cet exemple :

```
SÉLECTIONNER 'un gros chat assis sur un tapis et mangeant un gros rat'::tsvector;
-----
          tsvector
-----
'un' 'et' 'mangé' 'chat' 'gras' 'tapis' 'sur' 'rat' 'assit'
```

Pour représenter les lexèmes contenant des espaces ou de la ponctuation, entourez-les de guillemets :

```
SÉLECTIONNER $$le lexème '          ' contient des espaces$$::tsvector;
-----
          tsvector
-----
' ' 'contient' 'lexème' 'espaces' 'le'
```

(Dans cet exemple et le suivant, nous utilisons des chaînes de caractères entre guillemets dollar pour éviter toute confusion liée à l'utilisation de guillemets doubles à l'intérieur des chaînes.) Les guillemets et les barres obliques inverses inclus doivent être doublés :

```
SELECT $$le lexème 'Joe' contient une citation$$::tsvector;
-----
          tsvector
-----
« Joe » « a » « contient » « lexème » « citation » « le »
```

Il est possible d'associer des positions numériques aux lexèmes :

```
SÉLECTIONNER 'a:1 gras:2 chat:3 sat:4 sur:5 a:6 tapis:7 et:8 mangé:9 a:10
gras:11 rat:12':tsvector;
-----
                                     tsvector
-----

'a':1,6,10 'and':8 'ate':9 'cat':3 'fat':2,11 'mat':7 'on':5 'rat':12 'sat':4
```

La position indique généralement l'emplacement du mot source dans le document. Cette information peut servir au classement par proximité. Les valeurs de position sont comprises entre 1 et 16 383 ; les valeurs supérieures sont automatiquement fixées à 16 383. Les positions identiques pour un même lexème sont ignorées.

Les lexèmes qui possèdent une position peuvent être étiquetés avec un poids, qui peut être A, B, C ou D. D est la valeur par défaut et n'est donc pas affichée dans le résultat :

```
SELECT 'a:1A fat:2B,4C cat:5D':tsvector; tsvector
-----

'a':1A 'chat':5 'gros':2B,4C
```

On utilise généralement des pondérations pour refléter la structure du document, par exemple en distinguant les mots du titre de ceux du corps du texte. Les fonctions de classement pour la recherche textuelle peuvent attribuer différentes priorités aux différents marqueurs de pondération.

Il est important de comprendre que le type tsvector lui-même n'effectue aucune normalisation des mots ; il suppose que les mots qui lui sont fournis sont normalisés de manière appropriée pour l'application. Par exemple,

```
SÉLECTIONNER 'Les gros rats':tsvector;
-----
                                     tsvector
-----

« Gros » « Rats » « Le »
```

Pour la plupart des applications de recherche de texte en anglais, les mots ci-dessus seraient considérés comme non normalisés, mais tsvector n'en tient pas compte. Le texte brut du document doit généralement être traité par to_tsvector afin de normaliser les mots de manière appropriée pour la recherche.

```
SELECT to_tsvector('english', 'The Fat Rats'); to_tsvector
-----

'gros':2 'rat':3
```

Pour plus de détails, veuillez consulter le chapitre 12.

8.11.2. tsquery

Une valeur tsquery stocke les lexèmes à rechercher et peut les combiner à l'aide des opérateurs booléens & (ET), | (OU) et ! (NON), ainsi que de l'opérateur de recherche de phrases <-> (SUIVI DE). Il existe également une variante <N> de l'opérateur SUIVI DE, où N est une constante entière spécifiant la distance entre les deux lexèmes recherchés. <-> est équivalent à <1>.

Les parenthèses permettent de grouper ces opérateurs. En leur absence, ! (NON) est l'opérateur le plus contraignant, suivi de <-> (SUIVI DE), puis de & (ET), et enfin de | (OU), l'opérateur le moins contraignant.

Voici quelques exemples :

```
SELECT 'fat & rat'::tsquery; tsquery
```

```
-----
« gros » et « rat »
```

```
SELECT 'fat & (rat | chat)'::tsquery;
-----
tsquery
```

```
'gros' & ( 'rat' | 'chat' )
```

```
SELECT 'fat & rat & ! chat'::tsquery;
-----
tsquery
```

```
'gros' & 'rat' & '!chat'
```

En option, les lexèmes d'une requête tsquery peuvent être étiquetés avec une ou plusieurs lettres de pondération, ce qui les limite à ne correspondre qu'aux lexèmes tsvector ayant l'une de ces pondérations :

```
SELECT 'fat:ab & cat'::tsquery; tsquery
```

```
-----
'gros':AB et 'chat'
```

De plus, les lexèmes d'une requête tsquery peuvent être étiquetés avec * pour spécifier une correspondance de préfixe :

```
SELECT 'super:*'::tsquery; tsquery
```

```
-----
'super':*
```

Cette requête correspondra à tout mot d'un tsvector commençant par « super ».

Les règles de citation des lexèmes sont identiques à celles décrites précédemment pour les lexèmes dans tsvector ; et, comme pour tsvector, toute normalisation nécessaire des mots doit être effectuée avant la conversion au type tsquery. La fonction to_tsquery est pratique pour effectuer cette normalisation.

```
SELECT to_tsquery('Fat:ab & Cats'); to_tsquery
```

```
-----
'gros':AB et 'chat'
```

Notez que la fonction to_tsquery traitera les préfixes de la même manière que les autres mots, ce qui signifie que cette comparaison renvoie vrai :

```
SELECT to_tsvector( 'postgraduate' ) @@ to_tsquery( 'postgres:*' );
?colonne?
```

```
-----
t
```

car postgres est transformé en postgr :

```
SELECT to_tsvector( 'postgraduate' ), to_tsquery( 'postgres:*' );
vers_tsvector | vers_tsquery
```

```
-----+-----
'postgradu':1 | 'postgr':*
```

qui correspondra à la forme lemmatisée de post-diplôme.

8.12. Type d'UUID

Le type de données uuid stocke les identifiants uniques universels (UUID) tels que définis par la RFC 4122, ², ISO/ la norme IEC 9834-8:2005 et les normes associées. (Certains systèmes désignent ce type de données comme un identifiant unique global, ou GUID.) Cet identifiant est une chaîne de 128 bits générée par un algorithme conçu pour qu'il soit extrêmement improbable qu'un même identifiant soit généré par une autre entité utilisant le même algorithme. Par conséquent, pour les systèmes distribués, ces identifiants offrent une meilleure garantie d'unicité que les générateurs de séquences, qui ne sont uniques qu'au sein d'une même base de données.

Un UUID s'écrit sous la forme d'une séquence de chiffres hexadécimaux minuscules, regroupés en plusieurs groupes séparés par des tirets : un groupe de 8 chiffres, suivi de trois groupes de 4 chiffres, puis d'un groupe de 12 chiffres, soit un total de 32 chiffres représentant les 128 bits. Voici un exemple d'UUID sous cette forme standard :

```
a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
```

PostgreSQL accepte également les formats alternatifs suivants : utilisation de chiffres en majuscules, format standard entre accolades, omission de certains ou de tous les tirets, ajout d'un tiret après chaque groupe de quatre chiffres. Exemples :

```
A0EEBC99-9C0B-4EF8-BB6D-6BB9BD380A11 {a0eebc99-9c0b-4ef8-
bb6d-6bb9bd380a11} a0eebc999c0b4ef8bb6d6bb9bd380a11 a0ee-
bc99-9c0b-4ef8-bb6d-6bb9-bd38-0a11 {a0eebc99-9c0b4ef8-
bb6d6bb9-bd380a11}
```

Le résultat est toujours au format standard.

Consultez la section 9.14 pour savoir comment générer un UUID dans PostgreSQL.

8.13. Type XML

Le type de données XML permet de stocker des données XML. Son avantage par rapport au stockage de données XML dans un champ texte réside dans la vérification de la validité des valeurs saisies et dans la disponibilité de fonctions permettant d'effectuer des opérations sécurisées sur ce type ; voir la section 9.15. L'utilisation de ce type de données requiert une installation compilée avec l'option `--with-libxml`.

Le type XML peut stocker des « documents » bien formés, tels que définis par la norme XML, ainsi que des fragments de « contenu », définis par référence au « nœud de document »³, plus permissif, du modèle de données XQuery et XPath. En résumé, cela signifie que les fragments de contenu peuvent comporter plusieurs éléments de premier niveau ou nœuds de caractères. L'expression `xmlvalue IS DOCUMENT` permet de déterminer si une valeur XML donnée correspond à un document complet ou à un simple fragment de contenu.

Les limites et les notes de compatibilité du type de données xml se trouvent dans la section D.3.

8.13.1. Création de valeurs XML

Pour produire une valeur de type xml à partir de données de type caractère, utilisez la fonction `xmlparse` :

² <https://datatracker.ietf.org/doc/html/rfc4122>

³ <https://www.w3.org/TR/2010/REC-xpath-datamodel-20101214/#DocumentNode>

XMLPARSE({ DOCUMENT | CONTENU } valeur)

Exemples :

XMLPARSE (DOCUMENT '<?xml version="1.0"?><book><title>Manuel</ title><chapter>...</chapter></book>')

XMLPARSE (CONTENU 'abc<foo>bar</foo><bar>foo</bar>')

Bien que ce soit la seule façon de convertir des chaînes de caractères en valeurs XML conformément à la norme SQL, les syntaxes spécifiques à PostgreSQL :

```
xml '<foo>bar</foo>' '<foo>bar</
foo>':::xml
```

peut également être utilisé.

Le type XML ne valide pas les valeurs d'entrée par rapport à une déclaration de type de document (DTD), même lorsque la valeur d'entrée spécifie une DTD. Il n'existe actuellement aucune prise en charge intégrée pour la validation par rapport à d'autres langages de schémas XML tels que XML Schema.

L'opération inverse, qui produit une valeur de chaîne de caractères à partir de XML, utilise la fonction xmlserial-ize :

XMLSERIALIZE ({ DOCUMENT | CONTENU } valeur AS type [[NON]
[INDENT])

Le type peut être de type caractère, caractère variable ou texte (ou un alias pour l'un de ces types). Conformément à la norme SQL, il s'agit de la seule méthode de conversion entre les types XML et caractère, mais PostgreSQL permet également d'effectuer un cast simple.

L'option INDENT permet d'afficher le résultat de manière formatée, tandis que l'option NO INDENT (valeur par défaut) affiche simplement la chaîne d'entrée originale. La conversion vers un type caractère produit également la chaîne originale.

Lorsqu'une valeur de chaîne de caractères est convertie vers ou depuis le type xml sans passer par XMLPARSE ou XMLSERIALIZE, respectivement, le choix de DOCUMENT ou CONTENT est déterminé par le paramètre de configuration de session « option XML », qui peut être défini à l'aide de la commande standard :

DÉFINIR L'OPTION XML { DOCUMENT | CONTENU };

ou une syntaxe plus proche de celle de PostgreSQL

DÉFINIR xmloption SUR { DOCUMENT | CONTENU };

La valeur par défaut est CONTENT, donc toutes les formes de données XML sont autorisées.

8.13.2. Gestion de l'encodage

Il convient d'être prudent lors de la gestion de plusieurs encodages de caractères côté client, serveur et dans les données XML qui y transitent. En mode texte (mode normal) pour envoyer des requêtes au serveur et leurs résultats au client, PostgreSQL convertit toutes les données de caractères échangées entre le client et le serveur, et inversement, vers l'encodage de leur destinataire respectif (voir section 23.3). Cela inclut les représentations sous forme de chaînes de caractères des valeurs XML, comme dans les exemples précédents. En principe, les déclarations d'encodage contenues dans les données XML peuvent devenir invalides lors de la conversion des données entre le client et le serveur, car la déclaration d'encodage intégrée reste inchangée. Pour pallier ce problème, les déclarations d'encodage des chaînes de caractères présentées comme entrée pour le type XML sont ignorées, et le contenu est considéré comme non modifiable.

Les données XML doivent être encodées selon l'encodage du serveur. Par conséquent, pour un traitement correct, les chaînes de caractères XML doivent être envoyées par le client dans l'encodage client actuel. Il incombe au client de convertir les documents dans cet encodage avant de les envoyer au serveur, ou d'ajuster son encodage en conséquence. À la sortie, les valeurs de type XML ne comportent pas de déclaration d'encodage ; les clients doivent donc considérer que toutes les données sont encodées selon l'encodage client actuel.

Lorsqu'on utilise le mode binaire pour transmettre les paramètres de requête au serveur et renvoyer les résultats au client, aucune conversion d'encodage n'est effectuée. La situation est donc différente. Dans ce cas, une déclaration d'encodage dans les données XML est prise en compte. Si elle est absente, les données sont supposées être encodées en UTF-8 (conformément à la norme XML ; PostgreSQL ne prend pas en charge l'UTF-16). À l'affichage, les données contiennent une déclaration d'encodage spécifiant l'encodage du client, sauf si celui-ci est UTF-8, auquel cas cette déclaration est omise.

Il va de soi que le traitement des données XML avec PostgreSQL sera plus fiable et plus efficace si l'encodage des données XML, l'encodage côté client et l'encodage côté serveur sont identiques. Étant donné que les données XML sont traitées en interne en UTF-8, les calculs seront optimaux si l'encodage côté serveur est également UTF-8.

Prudence

Certaines fonctions XML peuvent ne pas fonctionner du tout avec des données non ASCII lorsque l'encodage du serveur n'est pas UTF-8. Ce problème est connu pour les fonctions `xmltable()` et `xpath()` en particulier.

8.13.3. Accès aux valeurs XML

Le type de données XML est particulier car il ne propose aucun opérateur de comparaison. En effet, il n'existe pas d'algorithme de comparaison bien défini et universellement applicable aux données XML. De ce fait, il est impossible de récupérer des lignes en comparant une colonne XML à une valeur de recherche. Les valeurs XML doivent donc généralement être accompagnées d'un champ clé distinct, tel qu'un identifiant. Une solution alternative pour comparer des valeurs XML consiste à les convertir au préalable en chaînes de caractères, mais il convient de noter que la comparaison de chaînes de caractères n'a que peu de rapport avec une méthode de comparaison XML réellement efficace.

Comme il n'existe pas d'opérateurs de comparaison pour le type de données XML, il est impossible de créer un index directement sur une colonne de ce type. Pour effectuer des recherches rapides dans des données XML, des solutions de contournement sont possibles : convertir l'expression en une chaîne de caractères et l'indexer, ou indexer une expression XPath. Bien entendu, la requête devra être adaptée pour effectuer la recherche à l'aide de l'expression indexée.

La fonctionnalité de recherche textuelle de PostgreSQL peut également être utilisée pour accélérer les recherches dans l'ensemble des documents XML. Cependant, la prise en charge du prétraitement nécessaire n'est pas encore disponible dans la distribution PostgreSQL.

8.14. Types JSON

Les types de données JSON servent à stocker des données JSON (JavaScript Object Notation), comme spécifié dans la RFC 7159 . Ces données peuvent également être stockées sous forme de texte, mais les types de données JSON présentent l'avantage de garantir la validité de chaque valeur stockée selon les règles JSON. Diverses fonctions et opérateurs spécifiques à JSON sont également disponibles pour les données stockées dans ces types ; voir la section 9.16.

PostgreSQL propose deux types de données JSON : `json` et `jsonb`. Pour optimiser les requêtes sur ces types de données, PostgreSQL fournit également le type de données `jsonpath`, décrit dans la section 8.14.7.

Les types de données JSON et JSONB acceptent des ensembles de valeurs d'entrée quasi identiques. La principale différence pratique réside dans l'efficacité. Le type JSON stocke une copie exacte du texte d'entrée, que les fonctions de traitement doivent réanalyser à chaque exécution ; tandis que les données JSONB sont stockées dans un format binaire décomposé, ce qui ralentit légèrement la saisie en raison de la surcharge de conversion supplémentaire, mais la rend nettement plus rapide.

⁴ <https://datatracker.ietf.org/doc/html/rfc7159>

Le traitement est simple, car aucune réanalyse n'est nécessaire. JSONB prend également en charge l'indexation, ce qui peut constituer un avantage considérable.

Le type JSON stocke une copie exacte du texte d'entrée, préservant ainsi les espaces blancs sémantiquement insignifiants entre les jetons, ainsi que l'ordre des clés au sein des objets JSON. De plus, si un objet JSON contient la même clé plusieurs fois, toutes les paires clé/valeur sont conservées (les fonctions de traitement considèrent la dernière valeur comme étant la plus pertinente). À l'inverse, le type JSONB ne préserve ni les espaces blancs, ni l'ordre des clés des objets, ni les clés d'objets dupliquées. Si des clés dupliquées sont spécifiées dans l'entrée, seule la dernière valeur est conservée.

En général, la plupart des applications devraient préférer stocker les données JSON au format jsonb, sauf en cas de besoins très spécifiques, tels que des hypothèses héritées concernant l'ordre des clés d'objet.

La RFC 7159 spécifie que les chaînes JSON doivent être encodées en UTF-8. Par conséquent, les types JSON ne peuvent pas se conformer strictement à la spécification JSON à moins que l'encodage de la base de données ne soit en UTF-8. Toute tentative d'inclusion directe de caractères non représentables dans l'encodage de la base de données échouera ; en revanche, les caractères représentables dans l'encodage de la base de données mais pas en UTF-8 seront autorisés.

La RFC 7159 autorise les chaînes JSON à contenir des séquences d'échappement Unicode, notées `\uXXXX`. Dans la fonction d'entrée du type json, les séquences d'échappement Unicode sont autorisées quel que soit l'encodage de la base de données et leur validité syntaxique est uniquement vérifiée (c'est-à-dire que quatre chiffres hexadécimaux suivent `\u`). En revanche, la fonction d'entrée du type jsonb est plus stricte : elle interdit les séquences d'échappement Unicode pour les caractères non représentables dans l'encodage de la base de données. Le type jsonb rejette également `\u0000` (car ce caractère n'est pas représentable dans le type text de PostgreSQL) et exige que toute utilisation de paires de substitution Unicode pour désigner des caractères hors du plan multilingue de base Unicode soit correcte. Les séquences d'échappement Unicode valides sont converties en un seul caractère pour le stockage ; cela inclut la réduction des paires de substitution en un seul caractère.

Note

De nombreuses fonctions de traitement JSON décrites dans la section 9.16 convertissent les séquences d'échappement Unicode en caractères ordinaires et génèrent donc les mêmes types d'erreurs que celles décrites précédemment, même si leur entrée est de type JSON et non JSONB. Le fait que la fonction d'entrée JSON n'effectue pas ces vérifications peut être considéré comme un vestige du passé, bien que cela permette un stockage simple (sans traitement) des séquences d'échappement Unicode JSON dans un encodage de base de données qui ne prend pas en charge les caractères représentés.

Lors de la conversion d'entrées JSON textuelles en jsonb, les types primitifs décrits par la RFC 7159 sont effectivement mappés sur les types natifs de PostgreSQL, comme illustré dans le tableau 8.23. Par conséquent, certaines contraintes mineures supplémentaires s'appliquent aux données jsonb valides. Ces contraintes ne concernent ni le type json, ni le JSON en général, et correspondent aux limites de représentation du type de données sous-jacent. Notamment, jsonb rejette les nombres hors de la plage du type de données numériques de PostgreSQL, contrairement à json. Ces restrictions, définies par l'implémentation, sont autorisées par la RFC 7159.

Toutefois, en pratique, de tels problèmes sont bien plus susceptibles de se produire dans d'autres implémentations, car il est courant de représenter le type primitif numérique de JSON en virgule flottante double précision IEEE 754 (ce que la RFC 7159 prévoit et autorise explicitement). Lors de l'utilisation de JSON comme format d'échange avec de tels systèmes, il convient de prendre en compte le risque de perte de précision numérique par rapport aux données initialement stockées par PostgreSQL.

À l'inverse, comme indiqué dans le tableau, il existe quelques restrictions mineures sur le format d'entrée des types primitifs JSON qui ne s'appliquent pas aux types PostgreSQL correspondants.

Tableau 8.23. Types primitifs JSON et types PostgreSQL correspondants

Type primitif JSON	Type PostgreSQL	Notes
chaîne	texte	Le caractère <code>\u0000</code> est interdit, de même que les séquences d'échappement Unicode représentant des caractères non disponibles dans l'encodage de la base de données.

Type primitif JSON	Type PostgreSQL	Notes
nombre	numérique	Les valeurs NaN et infinies ne sont pas autorisées.
booléen	booléen	Seules les orthographes « vrai » et « faux » en minuscules sont acceptées.
nul	(aucun)	SQL NULL est un concept différent

8.14.1. Syntaxe d'entrée et de sortie JSON

La syntaxe d'entrée/sortie pour les types de données JSON est conforme à la RFC 7159.

Voici toutes les expressions JSON (ou JSONB) valides :

-- Valeur scalaire/primitive simple
 -- Les valeurs primitives peuvent être des nombres, des chaînes de caractères entre guillemets, vrai, faux ou nul

```
SÉLECTIONNER '5'::json;
```

-- Tableau de zéro ou plusieurs éléments (les éléments ne sont pas nécessairement identiques).
 taper)

```
SELECT '[1, 2, "foo", null]'::json;
```

-- Objet contenant des paires de clés et de valeurs -- Notez que les clés de l'objet doivent toujours être des chaînes entre guillemets
 SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::json;

-- Les tableaux et les objets peuvent être imbriqués de manière arbitraire

```
SELECT '{"foo": [true, "bar"], "tags": {"a": 1, "b": null}}'::json;
```

Comme indiqué précédemment, lorsqu'une valeur JSON est saisie puis affichée sans traitement supplémentaire, json renvoie le texte original, tandis que jsonb ne conserve pas les détails sémantiquement insignifiants tels que les espaces. Par exemple, veuillez noter les différences suivantes :

```
SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::json;
                                     json
```

```

{"bar": "baz", "balance": 7.77, "active": false}
(1 rangée)
```

```
SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::jsonb;
                                     jsonb
```

```

{"bar": "baz", "active": false, "balance": 7.77}
(1 rangée)
```

Un détail sémantiquement insignifiant qu'il convient de noter est que, dans jsonb, les nombres seront affichés conformément au comportement du type numérique sous-jacent. En pratique, cela signifie que les nombres saisis avec la notation E seront affichés sans cette notation, par exemple :

```
SELECT '{"reading": 1.230e-5}'::json, '{"reading":
```

```
1.230e-5}'::jsonb; json jsonb -----
```

```

+-----+
{"reading": 1.230e-5} | {"reading": 0.00001230}
```

```

{"reading": 1.230e-5} | {"reading": 0.00001230}
(1 rangée)
```

Cependant, jsonb conservera les zéros fractionnaires de fin, comme on le voit dans cet exemple, même si ceux-ci sont sémantiquement insignifiants pour des usages tels que les vérifications d'égalité.

Pour la liste des fonctions et opérateurs intégrés disponibles pour la construction et le traitement des valeurs JSON, voir la section 9.16.

8.14.2. Conception de documents JSON

La représentation des données au format JSON offre une flexibilité bien supérieure au modèle relationnel traditionnel, un atout majeur dans les environnements aux exigences évolutives. Ces deux approches peuvent parfaitement coexister et se compléter au sein d'une même application. Toutefois, même pour les applications exigeant une flexibilité maximale, il est recommandé que les documents JSON possèdent une structure relativement fixe. Cette structure n'est généralement pas imposée (bien qu'il soit possible d'imposer certaines règles métier de manière déclarative), mais une structure prévisible facilite la rédaction de requêtes permettant de synthétiser efficacement un ensemble de « documents » (données) dans un tableau.

Les données JSON sont soumises aux mêmes considérations de contrôle de concurrence que tout autre type de données lorsqu'elles sont stockées dans une table. Bien que le stockage de documents volumineux soit envisageable, il convient de noter que toute mise à jour entraîne l'acquisition d'un verrou au niveau de la ligne entière. Il est donc recommandé de limiter la taille des documents JSON afin de réduire les conflits de verrouillage entre les transactions de mise à jour. Idéalement, chaque document JSON devrait représenter une donnée atomique qui, selon les règles métier, ne peut raisonnablement être subdivisée en données plus petites susceptibles d'être modifiées indépendamment.

8.14.3. Conteneur et existence de jsonb

Le test d'inclusion est une fonctionnalité importante de jsonb. Il n'existe pas d'ensemble de fonctionnalités équivalent pour le type json. Ce test vérifie si un document jsonb en contient un autre.

Ces exemples renvoient la valeur « vrai », sauf indication contraire :

```
-- Les valeurs scalaires/primitives simples contiennent uniquement des valeurs identiques :
SELECT "'foo'::jsonb @> "'foo'::jsonb;
```

```
-- Le tableau situé à droite est contenu dans celui situé à gauche.
gauche:
SELECT '[1, 2, 3]::jsonb @> '[1, 3]::jsonb;
```

```
L'ordre des éléments du tableau n'est pas significatif, donc ceci l'est aussi
vrai:
SELECT '[1, 2, 3]::jsonb @> '[3, 1]::jsonb;
```

```
-- Les éléments dupliqués dans le tableau n'ont pas d'importance non plus :
SELECT '[1, 2, 3]::jsonb @> '[1, 2, 2]::jsonb;
```

```
-- L'objet avec une seule paire à droite est contenu -- dans l'objet à gauche :
SELECT '{"product":
"PostgreSQL", "version": 9.4, "jsonb": true}'::jsonb @> '{"version":
9.4}'::jsonb;
```

```
-- Le tableau situé à droite n'est pas considéré comme contenu dans
le
-- tableau à gauche, même si un tableau similaire est imbriqué à l'intérieur
il:
SELECT '[1, 2, [1, 3]]::jsonb @> '[1, 3]::jsonb; -- renvoie faux
```

```
Mais grâce à une couche d'imbrication, il est contenu :
SELECT '[1, 2, [1, 3]]::jsonb @> '[[1, 3]]::jsonb;
```

```
-- De même, aucun confinement n'est signalé ici :
SELECT '{"foo": {"bar": "baz"}}::jsonb @> '{"bar": "baz"}::jsonb; -- renvoie faux
```

```
-- Contient une clé de niveau supérieur et un objet vide :
SELECT '{"foo": {"bar": "baz"}}::jsonb @> '{"foo": {}}::jsonb;
```

Le principe général est que l'objet contenu doit correspondre à l'objet conteneur quant à sa structure et à son contenu, éventuellement après suppression des éléments de tableau ou des paires clé/valeur non concordantes. Il est important de noter que l'ordre des éléments du tableau n'a pas d'importance lors d'une comparaison de contenu, et que les éléments dupliqués ne sont comptabilisés qu'une seule fois.

Par exception au principe général selon lequel les structures doivent correspondre, un tableau peut contenir une valeur primitive :

```
-- Ce tableau contient la valeur de chaîne primitive :
SELECT '{"foo", "bar"}::jsonb @> "bar"::jsonb;
```

```
-- Cette exception n'est pas réciproque -- un non-confinement est signalé
ici:
SELECT "bar"::jsonb @> '{"bar"}::jsonb; -- renvoie false
```

jsonb possède également un opérateur d'existence, une variante de l'opérateur de contenance : il vérifie si une chaîne de caractères (donnée comme valeur textuelle) apparaît comme clé d'objet ou élément de tableau au niveau supérieur de la valeur jsonb. Ces exemples renvoient vrai, sauf indication contraire :

```
-- La chaîne existe en tant qu'élément de tableau :
SELECT '{"foo", "bar", "baz"}::jsonb ? 'bar';
```

```
-- La chaîne existe en tant que clé d'objet :
SELECT '{"foo": "bar"}::jsonb ? 'foo';
```

```
-- Les valeurs d'objet ne sont pas prises en compte :
SELECT '{"foo": "bar"}::jsonb ? 'bar'; -- renvoie false
```

```
-- Comme pour la contenance, l'existence doit correspondre au niveau supérieur : SELECT '{"foo":
{"bar": "baz"}}::jsonb ? 'bar'; -- renvoie false
```

Une chaîne de caractères est considérée comme existante si elle correspond à un type JSON primitif.
chaîne :

```
SELECT "'foo'::jsonb ? 'foo';
```

Les objets JSON sont mieux adaptés que les tableaux pour tester l'inclusion ou l'existence lorsqu'il y a de nombreuses clés ou éléments impliqués, car contrairement aux tableaux, ils sont optimisés en interne pour la recherche et n'ont pas besoin d'être recherchés linéairement.

Conseil

Du fait de l'imbrication des éléments JSON, une requête appropriée peut s'affranchir de la sélection explicite des sous-objets. Par exemple, supposons une colonne « doc » contenant des objets au niveau supérieur, la plupart de ces objets possédant des champs « tags » qui contiennent des tableaux de sous-objets. Cette requête trouvera les entrées où apparaissent à la fois des sous-objets contenant « term:"paris » et « term:"food », tout en ignorant les clés de ce type en dehors du tableau « tags ».

```
SELECT doc->'site_name' FROM websites
WHERE doc @> '{"tags":{"term":"paris"}, {"term":"food"}}';
```

On pourrait obtenir le même résultat avec, par exemple,

```
SELECT doc->'site_name' FROM websites
WHERE doc->'tags' @> [{"term":"paris"}, {"term":"food"}];
```

mais cette approche est moins flexible, et souvent moins efficace également.

En revanche, l'opérateur d'existence JSON n'est pas imbriqué : il ne recherchera la clé ou l'élément de tableau spécifié qu'au niveau supérieur de la valeur JSON.

Les différents opérateurs de confinement et d'existence, ainsi que tous les autres opérateurs et fonctions JSON, sont documentés dans la section 9.16.

8.14.4. Indexation jsonb

Les index GIN permettent de rechercher efficacement des clés ou des paires clé/valeur dans un grand nombre de documents JSONB (données). Deux « classes d'opérateurs » GIN sont disponibles, offrant différents compromis entre performance et flexibilité.

La classe d'opérateur GIN par défaut pour jsonb prend en charge les requêtes avec les opérateurs d'existence de clé ?, ?| et ? &, l'opérateur d'inclusion @> et les opérateurs de correspondance jsonpath @? et @@ (Pour plus de détails sur la sémantique implémentée par ces opérateurs, voir le tableau 9.46.) Voici un exemple de création d'un index avec cette classe d'opérateur :

```
CRÉER UN INDEX idxgin SUR api EN UTILISANT GIN (jdoc);
```

La classe d'opérateur GIN non standard jsonb_path_ops ne prend pas en charge les opérateurs key-exists, mais elle prend en charge @>, @? et @@. Voici un exemple de création d'index avec cette classe d'opérateur :

```
CRÉER UN INDEX idxginp SUR api EN UTILISANT GIN (jdoc jsonb_path_ops);
```

Prenons l'exemple d'une table stockant des documents JSON récupérés d'un service web tiers, avec une définition de schéma documentée. Un document typique est :

```
{
  "guid": "9c36adc1-7fb5-4d5b-83b4-90356a46061a", "name": "Angela Barton",
  "is_active": true, "company": "MagnaFone",
  "address": "178 Howard Place,
  Gulf, Washington, 702", "registered":
  "2009-11-07T08:53:22 +08:00", "latitude": 19.793713, "longitude": 86.513373, "tags": [ "enim",
  "aliquip", "qui"
}
```

Nous stockons ces documents dans une table nommée api, dans une colonne jsonb nommée jdoc. Si un index GIN est créé sur cette colonne, des requêtes comme la suivante peuvent utiliser cet index :

```
-- Rechercher les documents dans lesquels la clé « company » a la valeur « Magnafone »
```

```
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @> '{"company": "MagnaFone"}';
```

Cependant, l'index ne pouvait pas être utilisé pour des requêtes comme la suivante, car bien que l'opérateur ? soit indexable, il n'est pas appliqué directement à la colonne indexée jdoc :

```
-- Rechercher les documents dans lesquels la clé « tags » contient une clé ou un tableau
   élément "qui"
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc -> 'tags' ?
   'qui';
```

Néanmoins, grâce à une utilisation appropriée des index d'expression, la requête ci-dessus peut exploiter un index. Si l'interrogation d'éléments spécifiques au sein de la clé « tags » est fréquente, la définition d'un index de ce type peut s'avérer judicieuse :

```
CRÉER UN INDEX idxgintags SUR api EN UTILISANT GIN ((jdoc -> 'tags'));
```

Désormais, la clause WHERE jdoc -> 'tags' ? 'qui' sera interprétée comme l'application de l'opérateur indexable ? à l'expression indexée jdoc -> 'tags'. (Pour plus d'informations sur les index d'expressions, consultez la section 11.7.)

Une autre approche pour interroger des données consiste à exploiter la notion de contenance, par exemple :

```
-- Trouver les documents dans lesquels la clé « tags » contient un élément de tableau
   "qui"
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @> '{"tags":
   ["qui"]}';
```

Un index GIN simple sur la colonne jdoc peut prendre en charge cette requête. Cependant, il est important de noter qu'un tel index stockera des copies de chaque clé et valeur de la colonne jdoc, tandis que l'index d'expression de l'exemple précédent ne stocke que les données associées à la clé tags. Bien que l'approche par index simple soit beaucoup plus flexible (puisque'elle prend en charge les requêtes sur n'importe quelle clé), les index d'expression ciblés sont généralement plus petits et plus rapides à parcourir qu'un index simple.

Les index GIN prennent également en charge les opérateurs @? et @@@, qui permettent la correspondance avec les chemins JSON. Exemples sont

```
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @? '$.tags[*]' ? (@ == "qui");
```

```
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @@@ '$.tags[*]
   == "qui";
```

Pour ces opérateurs, un index GIN extrait les clauses de la forme `accessors\_chain == constante` du modèle `jsonpath` et effectue la recherche d'index en fonction des clés et des valeurs mentionnées dans ces clauses. La chaîne d'accesseurs peut inclure les accesseurs `.key`, `[\*]` et `[index]`. La classe d'opérateur `jsonb\_ops` prend également en charge les accesseurs `.\*` et `.\*.\*`, contrairement à la classe d'opérateur `jsonb\_path\_ops`.

Bien que la classe d'opérateurs `jsonb\_path\_ops` ne prenne en charge que les requêtes avec les opérateurs `@>`, `@?` et `@@`, elle offre des performances nettement supérieures à celles de la classe d'opérateurs par défaut `jsonb\_ops`. Un index `jsonb\_path\_ops` est généralement beaucoup plus petit qu'un index `jsonb\_ops` portant sur les mêmes données, et la spécificité des recherches est accrue, notamment lorsque les requêtes contiennent des clés fréquemment présentes dans les données. Par conséquent, les opérations de recherche sont généralement plus performantes qu'avec la classe d'opérateurs par défaut.

La différence technique entre un index GIN jsonb_ops et un index GIN jsonb_path_ops réside dans le fait que le premier crée des éléments d'index indépendants pour chaque clé et valeur des données, tandis que le second crée des éléments d'index indépendants pour chaque paire clé-valeur des données.

indexer uniquement les éléments pour chaque valeur des données.⁵ En résumé, chaque élément d'index `jsonb_path_ops` est un hachage de la valeur et de la ou des clés y menant. Par exemple, pour indexer `{"foo": {"bar": "baz"}}`, un seul élément d'index est créé, intégrant les trois valeurs (foo, bar et baz) dans le hachage. Ainsi, une requête de présence recherchant cette structure aboutit à une recherche d'index extrêmement spécifique ; or, il est impossible de savoir si foo apparaît comme clé. En revanche, un index `jsonb_ops` crée trois éléments d'index représentant séparément foo, bar et baz. La requête de présence recherche alors les lignes contenant ces trois éléments. Bien que les index GIN puissent effectuer une telle recherche ET de manière relativement efficace, elle reste moins spécifique et plus lente que la recherche `jsonb_path_ops` équivalente, surtout si un très grand nombre de lignes contiennent l'un des trois éléments d'index.

L'un des inconvénients de l'approche `jsonb_path_ops` est qu'elle ne crée aucune entrée d'index pour les structures JSON ne contenant aucune valeur, telles que `{"a": {}}`. Si une recherche de documents contenant une telle structure est demandée, elle nécessitera une analyse complète de l'index, ce qui est très lent. `jsonb_path_ops` est donc mal adaptée aux applications qui effectuent fréquemment ce type de recherches.

`jsonb` prend également en charge les index btree et de hachage. Ces derniers ne sont généralement utiles que lorsqu'il est important de vérifier l'égalité de documents JSON complets. L'ordre des données `jsonb` dans l'arbre btree est rarement d'un grand intérêt, mais par souci d'exhaustivité, il est important de le mentionner :

Objet > Tableau > Booléen > Nombre > Chaîne > null

Objet avec n paires > objet avec n - 1 paires

Tableau à n éléments > tableau à n - 1 éléments

À l'exception du fait que (pour des raisons historiques) un tableau de premier niveau vide est trié comme inférieur à null. Les objets ayant un nombre égal de paires sont comparés dans l'ordre suivant :

clé-1, valeur-1, clé-2 ...

Notez que les clés d'objet sont comparées dans leur ordre de stockage ; en particulier, comme les clés les plus courtes sont stockées avant les clés les plus longues, cela peut donner des résultats contre-intuitifs, tels que :

```
{ "aa": 1, "c": 1 } > { "b": 1, "d": 1 }
```

De même, les tableaux comportant un nombre égal d'éléments sont comparés dans l'ordre suivant :

élément 1, élément 2 ...

Les valeurs JSON primitives sont comparées selon les mêmes règles de comparaison que pour le type de données PostgreSQL sous-jacent. Les chaînes de caractères sont comparées selon le classement par défaut de la base de données.

8.14.5. Sous-indexation jsonb

Le type de données `jsonb` prend en charge les expressions d'indexation de type tableau pour extraire et modifier des éléments. Les valeurs imbriquées peuvent être indiquées par l'enchaînement d'expressions d'indexation, selon les mêmes règles que l'argument `path` de la fonction `jsonb_set`. Si une valeur `jsonb` est un tableau, les indices numériques commencent à zéro et les entiers négatifs sont incrémentés à partir du dernier élément du tableau. Les expressions de découpage ne sont pas prises en charge. Le résultat d'une expression d'indexation est toujours de type `jsonb`.

Les instructions UPDATE peuvent utiliser l'indexation dans la clause SET pour modifier les valeurs `jsonb`. Les chemins d'indexation doivent être parcourables pour toutes les valeurs concernées, dans la mesure où elles existent. Par exemple, le chemin `val['a']['b']['c']` peut être parcouru jusqu'à c si chaque val, `val['a']` et `val['a']['b']`

⁵ À cette fin, le terme « valeur » inclut les éléments de tableau, bien que la terminologie JSON considère parfois les éléments de tableau comme distincts des valeurs au sein des objets.

est un objet. Si `val[a]` ou `val[a][b]` n'est pas défini, il sera créé comme un objet vide et rempli ultérieurement. Cependant, si `val` lui-même ou l'une des valeurs intermédiaires est définie comme une valeur non-objet (chaîne de caractères, nombre ou valeur `null` de JSONB), le parcours est impossible ; une erreur est alors levée et la transaction est annulée.

Exemple de syntaxe d'indexation :

```
-- Extraire la valeur de l'objet par clé
SELECT ('{"a": 1}::jsonb')[a];

-- Extraire la valeur d'un objet imbriqué par chemin de clé
SELECT ('{"a": {"b": {"c": 1}}}::jsonb')[a][b][c];

-- Extraire un élément du tableau par son index
SÉLECTIONNER ('[1, "2", null]::jsonb')[1];

-- Mettre à jour la valeur de l'objet par clé. Notez les guillemets autour de '1' :
    assigné
-- la valeur doit également être de type jsonb
MISE À JOUR table_name SET jsonb_field['key'] = '1';

-- Cela générera une erreur si le champ jsonb_field['a'][b] d'un enregistrement
    est quelque chose
— autre qu'un objet. Par exemple, la valeur '{"a": 1}' a un
    valeur numérique
-- de la touche 'a'.
MISE À JOUR table_name SET jsonb_field['a'][b][c] = '1';

Filtrer les enregistrements à l'aide d'une clause WHERE avec index. Puisque
    résultat de
-- L'indexation est de type jsonb, la valeur à laquelle nous la comparons doit également
    être jsonb.
-- Les guillemets doubles font également de « value » une chaîne jsonb valide.
SELECT * FROM nom_table WHERE jsonb_field['key'] = "value";
```

L'affectation de jsonb par indexation gère certains cas particuliers différemment de jsonb_set. Lorsqu'une valeur jsonb source est NULL, l'affectation par indexation se déroule comme s'il s'agissait d'une valeur JSON vide du type (objet ou tableau) sous-entendu par la clé d'indexation :

```
-- Là où jsonb_field était NULL, il vaut maintenant '{"a": 1}'
MISE À JOUR table_name SET jsonb_field['a'] = '1';

-- Là où jsonb_field était NULL, il est maintenant [1]
MISE À JOUR table_name SET jsonb_field[0] = '1';
```

Si un index est spécifié pour un tableau contenant trop peu d'éléments, des éléments NULL seront ajoutés jusqu'à ce que l'index soit accessible et que la valeur puisse être définie.

```
-- Là où jsonb_field était [], il est maintenant [null, null, 2] ; -- là où jsonb_field était [0], il est maintenant
[0, null, 2]
MISE À JOUR table_name SET jsonb_field[2] = '2';
```

Une valeur jsonb acceptera des affectations à des chemins d'indices inexistants tant que le dernier élément existant à parcourir est un objet ou un tableau, comme l'indique l'indice correspondant (l'élément indiqué par le dernier indice du chemin n'est pas parcouru et peut être de n'importe quel type). Tableau imbriqué et

Des structures d'objets seront créées et, dans le premier cas, complétées par des valeurs nulles, comme spécifié par le chemin d'index, jusqu'à ce que la valeur assignée puisse être placée.

```
-- Là où jsonb_field était {}, il est maintenant {"a": [{"b": 1}]}
MISE À JOUR table_name SET jsonb_field['a'][0]['b'] = '1';

-- Là où jsonb_field était [], il est maintenant [null, {"a": 1}]
MISE À JOUR table_name SET jsonb_field[1]['a'] = '1';
```

8.14.6. Transformations

Des extensions supplémentaires sont disponibles qui implémentent des transformations pour le type jsonb pour différents langages procéduraux.

Les extensions pour PL/Perl s'appellent jsonb_plperl et jsonb_plperlU. Elles permettent de convertir les valeurs jsonb en tableaux, tables de hachage et scalaires Perl, selon le cas.

L'extension pour PL/Python s'appelle jsonb_plpython3u. Si vous l'utilisez, les valeurs jsonb sont converties en dictionnaires, listes et scalaires Python, selon le cas.

Parmi ces extensions, jsonb_plperl est considérée comme « fiable », c'est-à-dire qu'elle peut être installée par des utilisateurs non-administrateurs disposant du privilège CREATE sur la base de données courante. Les autres nécessitent des privilèges d'administrateur pour être installées.

8.14.7. Type jsonpath

Le type 'jsonpath' implémente la prise en charge du langage de chemin SQL/JSON dans PostgreSQL pour interroger efficacement les données JSON. Il fournit une représentation binaire de l'expression de chemin SQL/JSON analysée, qui spécifie les éléments à extraire des données JSON par le moteur de chemin pour un traitement ultérieur avec les fonctions de requête SQL/JSON.

La sémantique des prédicats et opérateurs de chemin SQL/JSON suit généralement celle de SQL. Parallèlement, afin de faciliter la manipulation des données JSON, la syntaxe des chemins SQL/JSON utilise certaines conventions JavaScript :

- Le point (.) est utilisé pour l'accès des membres.
- Les crochets ([]) sont utilisés pour accéder aux tableaux.
- Les tableaux SQL/JSON sont indexés à partir de 0, contrairement aux tableaux SQL classiques qui commencent à 1.

Les littéraux numériques dans les expressions de chemin SQL/JSON suivent les règles JavaScript, qui diffèrent légèrement de celles de SQL et de JSON. Par exemple, SQL/JSON autorise '.1' et '1.', invalides en JSON. Les littéraux entiers non décimaux et le séparateur de type underscore sont pris en charge : '1_000_000', '0x1EEE_FFFF', '0o273', '0b100101'. Dans SQL/JSON (et en JavaScript, mais pas en SQL), aucun séparateur de type underscore ne doit suivre immédiatement le préfixe de base.

Dans une requête SQL, une expression de chemin est généralement écrite sous forme de chaîne de caractères littérale. Elle doit donc être placée entre guillemets simples, et tout guillemet simple présent dans la valeur doit être doublé (voir section 4.1.2.1). Certaines expressions de chemin nécessitent des chaînes de caractères littérales. Ces chaînes imbriquées suivent les conventions JavaScript/ECMAScript : elles doivent être entourées de guillemets doubles, et des barres obliques inverses peuvent y être utilisées pour représenter des caractères difficiles à saisir. En particulier, pour écrire un guillemet double dans une chaîne de caractères, on utilise \" , et pour écrire une barre oblique inverse, on utilise \\. Parmi les autres séquences de barres obliques inverses spéciales, on trouve celles reconnues dans les chaînes JavaScript : \b, \f, \n, \r, \t, \v pour divers caractères de contrôle ASCII, \xNN pour un code de caractère écrit avec seulement deux chiffres hexadécimaux, \uNNNN pour un caractère Unicode identifié par son point de code à 4 chiffres hexadécimaux, et \u{N...} pour un point de code de caractère Unicode écrit avec 1 à 6 chiffres hexadécimaux.

Une expression de chemin est constituée d'une séquence d'éléments de chemin, qui peuvent être de l'un des types suivants :

- Littéraux de chemin des types primitifs JSON : texte Unicode, numérique, vrai, faux ou nul.
- Variables de chemin répertoriées dans le tableau 8.24.
- Les opérateurs d'accès sont répertoriés dans le tableau 8.25.
- les opérateurs et méthodes jsonpath listés dans la section 9.16.2.3.
- Les parenthèses, qui peuvent être utilisées pour fournir des expressions de filtre ou définir l'ordre d'évaluation du chemin.

Pour plus de détails sur l'utilisation des expressions jsonpath avec les fonctions de requête SQL/JSON, voir la section 9.16.2.

Tableau 8.24. Variables jsonpath

Variable	Description
\$	Une variable représentant la valeur JSON interrogée (l'élément de contexte).
\$varname	Une variable nommée. Sa valeur peut être définie par les paramètres vars de plusieurs fonctions de traitement JSON ; voir le tableau 9.49 pour plus de détails.
@	Une variable représentant le résultat de l'évaluation du chemin dans les expressions de filtre.

Tableau 8.25. Accesseurs jsonpath

Opérateur d'accès	Description
.clé ."\$varname"	Accesseur de membre qui renvoie un membre d'objet avec la clé spécifiée. Si le nom de la clé correspond à une variable nommée commençant par avec \$ ou ne respecte pas les règles JavaScript pour un identifiant, doit être placée entre guillemets doubles pour en faire une chaîne littérale.
.*	Accesseur de membre générique qui renvoie les valeurs de tous les membres situé au niveau supérieur de l'objet actuel.
.**	Accesseur de membre générique récursif qui traite tous les niveaux de La hiérarchie JSON de l'objet courant et renvoie toutes les valeurs des membres, quel que soit leur niveau d'imbrication. Il s'agit d'une requête PostgreSQL extension de la norme SQL/JSON.
.**{niveau} .**{start_level à fin_niveau}	Comme .**, mais sélectionne uniquement les niveaux spécifiés de la hiérarchie JSON. Les niveaux d'imbrication sont spécifiés par des entiers. Le niveau zéro correspond à l'objet courant. Pour accéder au niveau d'imbrication le plus bas, Vous pouvez utiliser le dernier mot-clé. Il s'agit d'une extension PostgreSQL de la norme SQL/JSON.
[indice, ...]	Accesseur d'élément de tableau. L'indice peut être donné sous deux formes : index ou start_index à end_index. La première forme renvoie un seul élément du tableau par son index. La deuxième forme renvoie une tranche de tableau par la plage d'indices, incluant les éléments qui correspondent aux indices de début et de fin fournis. L'index spécifié peut être un entier ou une expression renvoyant une seule valeur numérique, qui est automatiquement convertie en Entier. L'indice zéro correspond au premier élément du tableau. peut également être utilisé pour désigner le dernier élément du tableau. ce qui est utile pour manipuler des tableaux de longueur inconnue.
[*]	Accesseur d'éléments de tableau générique qui renvoie tous les éléments du tableau.

8.15. Tableaux

PostgreSQL permet de définir les colonnes d'une table comme des tableaux multidimensionnels de longueur variable. Il est possible de créer des tableaux de n'importe quel type de base intégré ou défini par l'utilisateur : type énuméré, type composite, type intervalle ou domaine.

8.15.1. Déclaration des types de tableaux

Pour illustrer l'utilisation des types de tableaux, nous créons le tableau suivant :

```
CRÉER TABLE sal_emp ( texte, payer_par_trimestre
    nom                entier[],
    horaire texte[]
);
```

Comme indiqué, un type de données de type tableau est nommé en ajoutant des crochets ([]) au nom du type de données des éléments du tableau. La commande ci-dessus créera une table nommée sal_emp avec une colonne de type texte (nom), un tableau unidimensionnel de type entier (pay_by_quarter) représentant le salaire trimestriel de l'employé, et un tableau bidimensionnel de type texte (schedule) représentant son horaire hebdomadaire.

La syntaxe de CREATE TABLE permet de spécifier la taille exacte des tableaux, par exemple :

```
CRÉER LA TABLE tictactoe (
    carrés entier[3][3]
);
```

Cependant, l'implémentation actuelle ignore toute limite de taille de tableau fournie, c'est-à-dire que le comportement est le même que pour les tableaux de longueur non spécifiée.

L'implémentation actuelle n'impose pas non plus le nombre de dimensions déclaré. Les tableaux d'un même type d'élément sont tous considérés comme étant du même type, indépendamment de leur taille ou du nombre de dimensions. Par conséquent, déclarer la taille du tableau ou le nombre de dimensions dans CREATE TABLE relève uniquement de la documentation ; cela n'a aucune incidence sur le comportement à l'exécution.

Une syntaxe alternative, conforme à la norme SQL grâce à l'utilisation du mot-clé ARRAY, peut être employée pour les tableaux unidimensionnels. pay_by_quarter aurait pu être défini comme suit :

```
payer_par_trimestre tableau entier [4],
```

Ou, si aucune taille de tableau ne doit être spécifiée :

```
pay_by_quarter tableau d'entiers,
```

Comme auparavant, PostgreSQL n'impose toutefois aucune restriction de taille.

8.15.2. Saisie de valeurs de tableau

Pour écrire une valeur de tableau sous forme de constante littérale, placez les valeurs des éléments entre accolades et séparez-les par des virgules. (Si vous connaissez le C, cela ressemble à la syntaxe C d'initialisation des structures.) Vous pouvez mettre des guillemets autour de n'importe quelle valeur d'élément, et vous devez le faire si elle contient des virgules ou des accolades. (Plus de détails ci-dessous.) Ainsi, le format général d'une constante de tableau est le suivant :

```
'{ val1 délim val2 délim ... }'
```

où `délim` est le caractère délimiteur du type, tel qu'enregistré dans son entrée `pg\_type`. Parmi les types de données standard fournis dans la distribution PostgreSQL, tous utilisent une virgule (,), à l'exception du type `box`.

qui utilise un point-virgule (;). Chaque valeur est soit une constante du type d'élément du tableau, soit un sous-tableau. Voici un exemple de constante de tableau :

```
'{{1,2,3},{4,5,6},{7,8,9}}'
```

Cette constante est un tableau bidimensionnel 3x3 composé de trois sous-tableaux d'entiers.

Pour affecter la valeur NULL à un élément d'un tableau, saisissez NULL comme valeur de cet élément. (Toute variante de NULL, en majuscules ou en minuscules, convient.) Si vous souhaitez obtenir la valeur de chaîne « NULL », vous devez l'entourer de guillemets doubles.

(Ces types de constantes de tableau ne sont en réalité qu'un cas particulier des constantes de type générique abordées dans la section 4.1.2.7. La constante est initialement traitée comme une chaîne de caractères et transmise à la routine de conversion d'entrée du tableau. Une spécification de type explicite peut être nécessaire.)

Nous pouvons maintenant afficher quelques instructions INSERT :

```
INSERT INTO sal_emp
  VALEURS ('Facture',
    '{10000, 10000, 10000, 10000}', '{"réunion", "déjeuner"}',
    '{"formation", "présentation"}');
```

```
INSERT INTO sal_emp
  VALEURS ('Carol', '{20000,
    25000, 25000, 25000}', '{"petit-déjeuner", "consultation"}',
    '{"réunion", "déjeuner"}');
```

Le résultat des deux insertions précédentes ressemble à ceci :

```
SELECT * FROM sal_emp; nom |
payer_par_trimestre      |
-----+-----
+-----+-----
Facture | {10000,10000,10000,10000} | {{réunion,déjeuner},
{formation,présentation}}
Carol   | {20000,25000,25000,25000} | {{petit-déjeuner,consultation}, {réunion,déjeuner}} (2 lignes)
```

Les tableaux multidimensionnels doivent avoir des dimensions identiques. Une incohérence provoque une erreur, par exemple :

```
INSERT INTO sal_emp
  VALEURS ('Facture',
    '{10000, 10000, 10000, 10000}', '{"réunion", "déjeuner"}',
    '{"réunion"}');
ERREUR : littéral de tableau mal formé : '{"meeting", "lunch"}, {"meeting"}'
```

DÉTAIL : Les tableaux multidimensionnels doivent comporter des sous-tableaux correspondants dimensions.

La syntaxe du constructeur ARRAY peut également être utilisée :

```
INSERT INTO sal_emp
  VALEURS ('Facture',
```

```
TABLEAU[10000, 10000, 10000, 10000],
TABLEAU[['réunion', 'déjeuner'], ['formation', 'présentation']]);
```

```
INSERT INTO sal_emp
  VALEURS ('Carol',
  TABLEAU[20000, 25000, 25000, 25000],
  TABLEAU[['petit-déjeuner', 'consultation'], ['réunion', 'déjeuner']]);
```

Notez que les éléments du tableau sont des constantes ou expressions SQL ordinaires ; par exemple, les chaînes de caractères sont entre guillemets simples, contrairement aux littéraux de tableau qui sont entre guillemets doubles. La syntaxe du constructeur ARRAY est décrite plus en détail dans la section 4.2.12.

8.15.3. Accès aux tableaux

Nous pouvons maintenant exécuter quelques requêtes sur la table. Nous allons d'abord montrer comment accéder à un seul élément d'un tableau. Cette requête permet de récupérer les noms des employés dont la rémunération a changé au cours du deuxième trimestre :

```
SÉLECTIONNER nom DE sal_emp OÙ pay_by_quarter[1] <> pay_by_quarter[2];
```

```
nom
-----
Carol
(1 rangée)
```

Les indices des tableaux sont écrits entre crochets. Par défaut, PostgreSQL utilise une convention de numérotation à partir de 1 pour les tableaux, c'est-à-dire qu'un tableau de n éléments commence par tableau[1] et se termine par tableau[n].

Cette requête récupère le salaire du troisième trimestre de tous les employés :

```
SÉLECTIONNER pay_by_quarter[3] DE sal_emp;
```

```
payer par trimestre
-----
                10000
                25000
(2 rangées)
```

Nous pouvons également accéder à des portions rectangulaires arbitraires d'un tableau, ou sous-tableaux. Une portion de tableau est désignée par la notation borne inférieure:borne supérieure pour une ou plusieurs dimensions du tableau. Par exemple, cette requête récupère le premier élément de l'agenda de Bill pour les deux premiers jours de la semaine :

```
SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';
```

```
calendrier
-----
{{réunion},{formation}} (1 ligne)
```

Si une dimension quelconque est écrite sous forme de tranche, c'est-à-dire qu'elle contient un deux-points, alors toutes les dimensions sont traitées comme des tranches. Toute dimension ne comportant qu'un seul nombre (sans deux-points) est interprétée comme allant de 1 au nombre spécifié. Par exemple, [2] est interprété comme [1:2], comme dans cet exemple :

```
SELECT schedule[1:2][2] FROM sal_emp WHERE name = 'Bill';
```

calendrier

```

{{réunion,déjeuner},{formation,présentation}}
(1 rangée)

```

Pour éviter toute confusion avec le cas non-tranché, il est préférable d'utiliser la syntaxe de tranche pour toutes les dimensions, par exemple [1:2] [1:1], et non [2][1:1].

Il est possible d'omettre la borne inférieure et/ou la borne supérieure d'un spécificateur de tranche ; la borne manquante est remplacée par la limite inférieure ou supérieure des indices du tableau. Par exemple :

```
SELECT schedule[:2][2:] FROM sal_emp WHERE name = 'Bill';
```

calendrier

```

{{déjeuner},{présentation}} (1 ligne)

```

```
SELECT schedule[:,1:1] FROM sal_emp WHERE name = 'Bill';
```

calendrier

```

{{réunion},{formation}} (1 ligne)

```

Une expression d'indice de tableau renvoie null si le tableau lui-même ou l'un de ses indices est nul. De même, null est renvoyé si un indice est hors des limites du tableau (ce cas ne génère pas d'erreur). Par exemple, si le tableau `schedule` a actuellement les dimensions `[1:3][1:2]`, alors référencer `schedule[3][3]` renvoie null. De même, une référence à un tableau avec un nombre incorrect d'indices renvoie null plutôt qu'une erreur.

Une expression de découpage de tableau renvoie également null si le tableau lui-même ou l'une de ses expressions d'indice est nulle. Cependant, dans d'autres cas, comme la sélection d'une tranche de tableau entièrement située en dehors des limites du tableau courant, une expression de découpage renvoie un tableau vide (de dimension zéro) au lieu de null. (Ce comportement diffère de celui des expressions classiques et est dû à des raisons historiques.) Si la tranche demandée chevauche partiellement les limites du tableau, elle est réduite silencieusement à la seule région de chevauchement au lieu de renvoyer null.

Les dimensions actuelles de n'importe quelle valeur de tableau peuvent être récupérées avec la fonction `array_dims` :

```
SELECT array_dims(schedule) FROM sal_emp WHERE name = 'Carol';
```

dimensions du tableau

```

[1:2][1:2] (1
ligne)

```

La fonction `array_dims` renvoie un résultat textuel, pratique pour la lecture humaine mais potentiellement moins adapté aux programmes. Les dimensions peuvent également être obtenues avec `array_upper` et `array_lower`, qui renvoient respectivement les bornes supérieure et inférieure d'une dimension de tableau spécifiée.

```
SELECT array_upper(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

tableau supérieur

```

2

```

```

(1 rangée)

```

`array_length` renverra la longueur d'une dimension de tableau spécifiée :

```
SELECT array_length(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

```
longueur_du_tableau
```

```
2
```

```
(1 rangée)
```

La cardinalité renvoie le nombre total d'éléments d'un tableau, toutes dimensions confondues. Elle correspond en fait au nombre de lignes qu'un appel à la fonction ``unnest`` produirait.

```
SELECT cardinality(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
cardinalité
```

```
4
```

```
(1 rangée)
```

8.15.4. Modification des tableaux

Une valeur de tableau peut être entièrement remplacée :

```
MISE À JOUR sal_emp DÉFINIR pay_by_quarter = '{25000,25000,27000,27000}'
OÙ nom = 'Carol';
```

ou en utilisant la syntaxe d'expression ARRAY :

```
MISE À JOUR sal_emp DÉFINIR pay_by_quarter = ARRAY[25000,25000,27000,27000]
OÙ nom = 'Carol';
```

Un tableau peut également être mis à jour élément par élément :

```
MISE À JOUR sal_emp DÉFINIR pay_by_quarter[4] = 15000
OÙ nom = 'Bill';
```

ou mis à jour dans une tranche :

```
MISE À JOUR sal_emp DÉFINIR pay_by_quarter[1:2] = '{27000,27000}'
OÙ nom = 'Carol';
```

Les syntaxes de découpage avec borne inférieure et/ou borne supérieure omises peuvent également être utilisées, mais uniquement lors de la mise à jour d'une valeur de tableau qui n'est ni NULL ni de dimension zéro (sinon, il n'y a pas de limite d'indice existante à substituer).

La valeur d'un tableau stocké peut être augmentée en affectant des valeurs à des éléments absents. Les positions intermédiaires entre les éléments précédemment présents et les nouveaux éléments seront remplies par des valeurs nulles. Par exemple, si le tableau ``myarray`` contient actuellement 4 éléments, il en contiendra six après une mise à jour affectant ``myarray[6]`` ; ``myarray[5]`` contiendra alors la valeur nulle. Actuellement, cette méthode d'augmentation de valeur n'est autorisée que pour les tableaux unidimensionnels, et non pour les tableaux multidimensionnels.

L'affectation par indice permet de créer des tableaux dont les indices ne commencent pas à 1. Par exemple, on peut affecter ``myarray[-2:7]`` pour créer un tableau dont les indices vont de -2 à 7.

De nouvelles valeurs de tableau peuvent également être construites à l'aide de l'opérateur de concaténation, `||` :

```
SÉLECTIONNER TABLEAU[1,2] || TABLEAU[3,4];
?colonne?
```

```
-----
{1,2,3,4} (1
ligne)
```

```
SÉLECTIONNER TABLEAU[5,6] || TABLEAU[[1,2],[3,4]];
?colonne?
```

```
-----
{{5,6},{1,2},{3,4}}
(1 rangée)
```

L'opérateur de concaténation permet d'ajouter un élément au début ou à la fin d'un tableau unidimensionnel. Il accepte également deux tableaux N-dimensionnels, ou un tableau N-dimensionnel et un tableau N+1-dimensionnel.

Lorsqu'un élément est ajouté au début ou à la fin d'un tableau unidimensionnel, le résultat est un tableau dont l'indice minimal est identique à celui de l'opérande. Par exemple :

```
SELECT array_dims(1 || '[0:1]={2,3}':int[]); array_dims
```

```
-----
[0:2] (1
ligne)
```

```
SELECT array_dims(ARRAY[1,2] || 3); array_dims
```

```
-----
[1:3] (1
ligne)
```

Lorsqu'on concatène deux tableaux de même dimension, le résultat conserve l'indice minimal de la dimension externe du tableau de gauche. Il s'agit d'un tableau composé de chaque élément du tableau de gauche suivi de chaque élément du tableau de droite. Par exemple :

```
SELECT array_dims(ARRAY[1,2] || ARRAY[3,4,5]); array_dims
```

```
-----
[1:5] (1
ligne)
```

```
SELECT array_dims(ARRAY[[1,2],[3,4]] || ARRAY[[5,6],[7,8],[9,0]]); array_dims
```

```
-----
[1:5][1:2] (1
ligne)
```

Lorsqu'un tableau à N dimensions est inséré au début ou à la fin d'un tableau à N+1 dimensions, le résultat est analogue au cas des tableaux d'éléments décrit précédemment. Chaque sous-tableau à N dimensions correspond essentiellement à un élément de la dimension externe du tableau à N+1 dimensions. Par exemple :

```
SELECT array_dims(ARRAY[1,2] || ARRAY[[3,4],[5,6]]); array_dims
```

```
-----
[1:3][1:2]
```

(1 rangée)

On peut également construire un tableau à l'aide des fonctions `array\_prepend`, `array\_append` ou `array\_cat`. Les deux premières ne prennent en charge que les tableaux unidimensionnels, tandis que `array\_cat` prend en charge les tableaux multidimensionnels. Exemples :

```
SELECT array_prepend(1, ARRAY[2,3]); array_prepend
```

```
{1,2,3}
(1 rangée)
```

```
SELECT array_append(ARRAY[1,2], 3); array_append
```

```
{1,2,3}
(1 rangée)
```

```
SELECT array_cat(ARRAY[1,2], ARRAY[3,4]); array_cat
```

```
{1,2,3,4} (1
ligne)
```

```
SELECT array_cat(ARRAY[[1,2],[3,4]], ARRAY[5,6]); array_cat
```

```
{{1,2},{3,4},{5,6}}
(1 rangée)
```

```
SELECT array_cat(ARRAY[5,6], ARRAY[[1,2],[3,4]]); array_cat
```

```
{{5,6},{1,2},{3,4}}
```

Dans les cas simples, l'opérateur de concaténation décrit ci-dessus est préférable à l'utilisation directe de ces fonctions. Cependant, comme l'opérateur de concaténation est surchargé pour couvrir les trois cas, il existe des situations où l'utilisation de l'une des fonctions est utile pour éviter toute ambiguïté. Par exemple :

```
SELECT ARRAY[1, 2] || '{3, 4}'; -- la valeur littérale non typée est interprétée comme
un tableau ?
colonne ?
```

```
{1,2,3,4}
```

```
SÉLECTIONNER TABLEAU[1, 2] || '7';
```

-- celui-ci aussi

```
ERREUR : littéral de tableau mal formé : "7"
```

```
SÉLECTIONNER TABLEAU[1, 2] || NULL ;
```

-- il en va de même pour un objet non décoré

```
NUL
?colonne?
```

```
{1,2} (1
ligne)
```

```
SELECT array_append(ARRAY[1, 2], NULL);
censé
```

-- cela aurait pu être

ajouter au tableau

{1,2,NULL}

Dans les exemples précédents, l'analyseur syntaxique voit un tableau d'entiers d'un côté de l'opérateur de concaténation et une constante de type indéterminé de l'autre. Pour déterminer le type de la constante, il suppose qu'elle est du même type que l'autre entrée de l'opérateur, ici un tableau d'entiers. L'opérateur de concaténation est donc interprété comme représentant ``array_cat`` et non ``array_append``. Si cette interprétation est erronée, on peut la corriger en convertissant la constante vers le type des éléments du tableau ; toutefois, l'utilisation explicite de ``array_append`` est souvent préférable.

8.15.5. Recherche dans les tableaux

Pour rechercher une valeur dans un tableau, il faut vérifier chaque valeur. Cela peut se faire manuellement si l'on connaît la taille du tableau. Par exemple :

```
SELECT * FROM sal_emp WHERE pay_by_quarter[1] = 10000 OR pay_by_quarter[2] = 10000
                        OR pay_by_quarter[3] = 10000 OR
                        pay_by_quarter[4] = 10000;
```

Cependant, cette méthode devient rapidement fastidieuse pour les grands tableaux et s'avère inutile si la taille du tableau est inconnue. Une méthode alternative est décrite dans la section 9.25. La requête ci-dessus pourrait être remplacée par :

```
SELECT * FROM sal_emp WHERE 10000 = ANY (pay_by_quarter);
```

De plus, vous pouvez trouver les lignes où le tableau a toutes les valeurs égales à 10000 avec :

```
SELECT * FROM sal_emp WHERE 10000 = ALL (pay_by_quarter);
```

Il est également possible d'utiliser la fonction `generate_subscripts`. Par exemple :

```
SÉLECTIONNER * DE
    (SÉLECTIONNER payer_par_trimestre,
     générer_sous-titres(payer_par_trimestre, 1) AS s
    DE sal_emp) AS foo
OÙ payer_par_trimestre[s] = 10000;
```

Cette fonction est décrite dans le tableau 9.68.

Vous pouvez également effectuer une recherche dans un tableau à l'aide de l'opérateur `&&`, qui vérifie si l'opérande de gauche chevauche l'opérande de droite. Par exemple :

```
SÉLECTIONNER * DE sal_emp OÙ pay_by_quarter && ARRAY[10000];
```

Cet opérateur et d'autres opérateurs de tableaux sont décrits plus en détail dans la section 9.19. Il peut être accéléré par un index approprié, comme décrit dans la section 11.2.

Vous pouvez également rechercher des valeurs spécifiques dans un tableau à l'aide des fonctions ``array_position`` et ``array_positions``. La première renvoie l'indice de la première occurrence d'une valeur dans le tableau ; la seconde renvoie un tableau contenant les indices de toutes les occurrences de la valeur dans le tableau. Par exemple :

```
SÉLECTIONNER
    position_tableau(ARRAY['sun','mon','mar','wed','thu','ven','sam'], 'mon');
```

```

position_tableau
-----
                2
(1 rangée)

SELECT array_positions(ARRAY[1, 4, 3, 1, 3, 4, 2, 1], 1); array_positions
-----
{1,4,8}
(1 rangée)

```

Conseil

Les tableaux ne sont pas des ensembles ; la recherche d'éléments spécifiques d'un tableau peut être le signe d'une mauvaise conception de la base de données. Envisagez d'utiliser un tableau distinct, avec une ligne pour chaque élément qui serait un élément de tableau. La recherche sera ainsi facilitée et le système sera probablement plus performant pour un grand nombre d'éléments.

8.15.6. Syntaxe d'entrée et de sortie des tableaux

La représentation textuelle externe d'une valeur de tableau se compose d'éléments interprétés selon les règles de conversion d'entrée/sortie du type d'élément du tableau, ainsi que d'une décoration indiquant la structure du tableau. La décoration consiste en des accolades ({ et }) autour de la valeur du tableau, ainsi qu'en des caractères de délimitation entre les éléments adjacents. Le caractère de délimitation est généralement une virgule (,), mais peut être différent : il est déterminé par le paramètre `typdelim` du type d'élément du tableau. Parmi les types de données standard fournis par la distribution PostgreSQL, tous utilisent une virgule, à l'exception du type `box`, qui utilise un point-virgule (;). Dans un tableau multidimensionnel, chaque dimension (ligne, plan, cube, etc.) possède son propre niveau d'accolades, et les délimiteurs doivent être placés entre les entités adjacentes de même niveau, elles aussi délimitées par des accolades.

La routine de sortie de tableau placera des guillemets doubles autour des valeurs des éléments si celles-ci sont des chaînes vides, contiennent des accolades, des caractères de délimitation, des guillemets doubles, des barres obliques inverses ou des espaces, ou correspondent au mot NULL. Les guillemets doubles et les barres obliques inverses inclus dans les valeurs des éléments seront échappés par des barres obliques inverses. Pour les types de données numériques, on peut supposer sans risque que les guillemets doubles n'apparaîtront jamais, mais pour les types de données textuelles, il faut s'attendre à gérer la présence ou l'absence de guillemets.

Par défaut, l'indice de la limite inférieure des dimensions d'un tableau est fixé à un. Pour représenter des tableaux avec d'autres limites inférieures, les plages d'indices peuvent être spécifiées explicitement avant l'écriture du contenu du tableau. Cette syntaxe consiste à encadrer les limites inférieure et supérieure de chaque dimension par des crochets ([]), séparés par un deux-points (:). Le signe égal (=) suit cette syntaxe. Par exemple :

```

SÉLECTIONNER f1[1[-2][3] AS e1, f1[1[-1][5] AS e2
FROM (SELECT '[1:-2:-1][3:5]={{{1,2,3},{4,5,6}}}'::int[] AS f1)
AS ss;

```

```

e1 | e2
----+----
 1 | 6 (1
rangée)

```

La routine de sortie du tableau inclura les dimensions explicites dans son résultat uniquement lorsqu'il existe une ou plusieurs bornes inférieures différentes de une.

Si la valeur écrite pour un élément est NULL (quelle que soit la casse), l'élément est considéré comme NULL.

La présence de guillemets ou de barres obliques inverses désactive cette fonctionnalité et autorise la saisie de la valeur littérale « NULL ». De plus, pour assurer la compatibilité avec les versions de PostgreSQL antérieures à la version 8.2, le paramètre de configuration `array\_nulls` peut être désactivé afin d'empêcher la reconnaissance de NULL comme une valeur nulle.

Comme indiqué précédemment, lors de l'écriture d'une valeur de tableau, vous pouvez utiliser des guillemets doubles autour de chaque élément individuel. élément de tableau. Vous devez le faire si la valeur de l'élément risque autrement de perturber l'analyseur de valeurs de tableau. Par exemple, les éléments contenant des accolades, des virgules (ou le caractère délimiteur du type de données), des guillemets doubles, des barres obliques inverses ou des espaces en début ou en fin de chaîne doivent être placés entre guillemets doubles. Les chaînes vides et Les chaînes de caractères correspondant au mot NULL doivent également être placées entre guillemets. Pour insérer un guillemet double ou une barre oblique inverse dans une chaîne de caractères entre guillemets :

Pour indiquer la valeur d'un élément du tableau, faites-la précéder d'une barre oblique inverse. Vous pouvez également éviter les guillemets et utiliser l'échappement par barre oblique inverse pour protéger tous les caractères de données qui seraient autrement interprétés comme de la syntaxe de tableau.

Vous pouvez ajouter un espace avant une accolade ouvrante ou après une accolade fermante. Vous pouvez également ajouter un espace avant ou après chaque chaîne de caractères d'un élément individuel. Dans tous ces cas, les espaces seront ignorés. Cependant, espaces blancs à l'intérieur d'éléments entre guillemets doubles, ou entourés de part et d'autre par des caractères non blancs un élément n'est pas ignoré.

Conseil

La syntaxe du constructeur ARRAY (voir section 4.2.12) est souvent plus facile à utiliser que la syntaxe littérale des tableaux lors de l'écriture de valeurs de tableau dans des commandes SQL. Dans ARRAY, chaque élément Les valeurs sont écrites de la même manière que si elles ne faisaient pas partie d'un tableau.

8.16. Types composites

Un type composite représente la structure d'une ligne ou d'un enregistrement ; il s'agit essentiellement d'une liste de noms de champs. et leurs types de données. PostgreSQL permet d'utiliser les types composites de nombreuses manières similaires à celles utilisées pour les types composites. On peut utiliser des types simples. Par exemple, une colonne d'une table peut être déclarée comme étant de type composite.

8.16.1. Déclaration des types composites

Voici deux exemples simples de définition de types composites :

```
CRÉER TYPE complexe AS (
    r          double précision,
    je         double précision
);
```

```
CRÉER TYPE article_inventaire AS (
    nom        texte,
    fournisseur_id prix    entier,
                    numérique
);
```

La syntaxe est comparable à CREATE TABLE, sauf que seuls les noms et les types de champs peuvent être spécifiés ; Aucune contrainte (telle que NOT NULL) ne peut être incluse actuellement. Notez que le mot-clé AS est essentiel ; Sans cela, le système interprétera cela comme une commande CREATE TYPE différente, et vous...

Je rencontre des erreurs de syntaxe étranges.

Une fois les types définis, nous pouvons les utiliser pour créer des tables :

```
CRÉER TABLE sur_main (
    article inventaire_article,
    compter          entier
);
```

```
INSÉRER DANS on_hand VALUES (LIGNE('dés flous', 42, 1.99), 1000);
```

ou fonctions :

```
CRÉER LA FONCTION price_extension(article_inventaire, entier) RETOURNE
numérique
```

```
AS 'SELECT $1.price * $2' LANGUAGE SQL;
```

```
SELECT price_extension(item, 10) FROM on_hand;
```

Chaque fois que vous créez une table, un type composite est également créé automatiquement, portant le même nom que le tableau, pour représenter le type de ligne du tableau. Par exemple, si nous avions dit :

```
CRÉER TABLE article_inventaire (
    nom                texte,
    fournisseur_id prix  fournisseurs de références entières,
                        VÉRIFICATION numérique (prix > 0)
);
```

Le même type composite `inventory_item` présenté ci-dessus serait alors créé en tant que sous-produit et pourrait être utilisé de la même manière. Notez cependant une restriction importante de l'implémentation actuelle : comme aucune contrainte n'est associée à un type composite, les contraintes indiquées dans le tableau La définition ne s'applique pas aux valeurs du type composite en dehors du tableau. (Pour contourner ce problème, créez un domaine sur le type composite et appliquez les contraintes souhaitées en tant que contraintes CHECK.) domaine.)

8.16.2. Construction de valeurs composites

Pour écrire une valeur composite sous forme de constante littérale, placez les valeurs des champs entre parenthèses et séparez-les. Séparez-les par des virgules. Vous pouvez encadrer n'importe quelle valeur de champ par des guillemets doubles, et vous devez le faire si elle contient Virgules ou parenthèses. (Plus de détails ci-dessous.) Ainsi, la forme générale d'une constante composée est le suivant :

```
'( val1      ,  val2      , ... )'
```

Voici un exemple :

```
'("dés flous",42,1.99)'
```

ce qui serait une valeur valide du type `inventory_item` défini ci-dessus. Pour qu'un champ soit NULL, n'écrivez aucun caractère à sa position dans la liste. Par exemple, cette constante spécifie une valeur NULL. troisième champ :

```
'("dés flous",42,)'
```

Si vous souhaitez une chaîne vide plutôt que NULL, utilisez des guillemets doubles :

```
'( "",42,)'
```

Ici, le premier champ est une chaîne vide non nulle, le troisième est nul.

(Ces constantes ne sont en réalité qu'un cas particulier des constantes de type générique abordées dans la section 4.1.2.7. La constante est initialement traitée comme une chaîne de caractères et transmise à la routine de conversion d'entrée de type composite. Une spécification de type explicite peut être nécessaire pour indiquer le type à convertir.) constante à.)

La syntaxe des expressions ROW permet également de construire des valeurs composites. Dans la plupart des cas, elle est nettement plus simple à utiliser que la syntaxe littérale de chaîne, car il n'est pas nécessaire de gérer plusieurs niveaux de guillemets. Nous avons déjà utilisé cette méthode ci-dessus :

```
LIGNE('dés flous', 42, 1,99)
LIGNE("", 42, NULL)
```

Le mot-clé ROW est en réalité facultatif tant que l'expression comporte plus d'un champ ; on peut donc simplifier les expressions comme suit :

```
(« dés flous », 42, 1,99)
("", 42, NULL)
```

La syntaxe de l'expression ROW est discutée plus en détail dans la section 4.2.13.

8.16.3. Accès aux types composites

Pour accéder à un champ d'une colonne composite, on saisit un point suivi du nom du champ, comme pour sélectionner un champ dans une table. La ressemblance est telle qu'il est souvent nécessaire d'utiliser des parenthèses pour éviter toute confusion avec l'analyseur syntaxique. Par exemple, pour sélectionner des sous-champs de notre table d'exemple « on_hand », on pourrait utiliser une syntaxe similaire à celle-ci :

```
SÉLECTIONNER item.name DE on_hand OÙ item.price > 9.99;
```

Cela ne fonctionnera pas car, conformément aux règles de syntaxe SQL, « name item » est interprété comme un nom de table et non comme un nom de colonne de « on_hand ». Vous devez l'écrire comme ceci :

```
SÉLECTIONNER (article).nom DE en stock OÙ (article).prix > 9,99 ;
```

ou si vous devez également utiliser le nom de la table (par exemple dans une requête multitable), comme ceci :

```
SÉLECTIONNER (on_hand.item).name DE on_hand OÙ (on_hand.item).price
> 9,99 ;
```

L'objet entre parenthèses est désormais correctement interprété comme une référence à la colonne de l'élément, et le sous-champ peut alors être sélectionné à partir de celui-ci.

Des problèmes de syntaxe similaires se posent chaque fois que vous sélectionnez un champ dans une valeur composite. Par exemple, pour sélectionner un seul champ du résultat d'une fonction qui renvoie une valeur composite, vous devrez écrire quelque chose comme :

```
SÉLECTIONNER (my_func(...)).field DE ...
```

Sans les parenthèses supplémentaires, cela générera une erreur de syntaxe.

Le nom du champ spécial * signifie « tous les champs », comme expliqué plus en détail dans la section 8.16.5.

8.16.4. Modification des types composites

Voici quelques exemples de la syntaxe correcte pour insérer et mettre à jour des colonnes composites. Tout d'abord, l'insertion ou la mise à jour d'une colonne entière :

```
INSÉRER DANS mytab (complex_col) VALEURS((1.1,2.2));
```

MISE À JOUR mytab SET complex_col = ROW(1.1,2.2) WHERE ...;

Le premier exemple omet ROW, le second l'utilise ; nous aurions pu procéder de l'une ou l'autre manière.

Nous pouvons mettre à jour un sous-champ individuel d'une colonne composite :

MISE À JOUR mytab DÉFINIR complex_col.r = (complex_col).r + 1 OÙ ...;

Notez ici que nous n'avons pas besoin (et en effet, nous ne pouvons pas) de mettre des parenthèses autour du nom de la colonne qui apparaît juste après SET, mais que nous avons besoin de parenthèses lorsque nous faisons référence à la même colonne dans l'expression à droite du signe égal.

Et nous pouvons également spécifier des sous-champs comme cibles pour INSERT :

INSÉRER DANS mytab (complex_col.r, complex_col.i) VALEURS(1.1, 2.2);

Si nous n'avons pas fourni de valeurs pour tous les sous-champs de la colonne, les sous-champs restants auraient été remplis de valeurs nulles.

8.16.5. Utilisation des types composites dans les requêtes

Il existe diverses règles de syntaxe et comportements spécifiques associés aux types composites dans les requêtes. Ces règles offrent des raccourcis utiles, mais peuvent prêter à confusion si l'on ignore leur logique sous-jacente.

Dans PostgreSQL, une référence au nom (ou à l'alias) d'une table dans une requête fait en réalité référence à la valeur composite de la ligne courante de cette table. Par exemple, si nous avons une table `inventory\_item` comme indiqué ci-dessus, nous pourrions écrire :

SÉLECTIONNER c DE l'article d'inventaire c;

Cette requête produit une seule colonne à valeurs composites, nous pourrions donc obtenir un résultat comme celui-ci :

```

      c
-----
("dés flous",42,1,99) (1 ligne)

```

Notez cependant que les noms simples sont mis en correspondance avec les noms de colonnes avant les noms de tables ; cet exemple ne fonctionne donc que parce qu'il n'existe aucune colonne nommée c dans les tables de la requête.

La syntaxe classique de nom de colonne qualifié `table\_name.column\_name` peut être interprétée comme l'application d'une sélection de champ à la valeur composite de la ligne courante du tableau. (Pour des raisons d'efficacité, ce n'est pas ainsi que cela est implémenté en réalité.)

Lorsque nous écrivons

SÉLECTIONNER c.* DE l'article d'inventaire c;

Ensuite, conformément à la norme SQL, nous devrions obtenir le contenu de la table décomposé en colonnes distinctes :

```

      nom          | identifiant_fournisseur | prix -----+-----
+-----

```

dés flous | (1 rangée)

42 | 1,99

comme si la requête était

```
SÉLECTIONNER c.name, c.supplier_id, c.price DE inventory_item c;
```

PostgreSQL appliquera ce comportement d'expansion à toute expression à valeur composite. Cependant, comme indiqué ci-dessus, il est nécessaire d'encadrer de parenthèses la valeur à laquelle `.\*` est appliqué lorsqu'il ne s'agit pas d'un simple nom de table. Par exemple, si `myfunc()` est une fonction renvoyant un type composite avec les colonnes `a`, `b` et `c`, alors ces deux requêtes auront le même résultat :

```
SELECT (myfunc(x)).* FROM some_table; SELECT
(myfunc(x)).a, (myfunc(x)).b, (myfunc(x)).c FROM some_table;
```

Conseil

PostgreSQL gère l'expansion des colonnes en transformant concrètement la première forme en la seconde. Dans cet exemple, la fonction `myfunc()` serait appelée trois fois par ligne, quelle que soit la syntaxe. S'il s'agit d'une fonction coûteuse, vous pouvez éviter cela en utilisant une requête comme celle-ci :

```
SELECT m.* FROM some_table, LATERAL myfunc(x) AS m;
```

L'utilisation d'une clause FROM LATERAL pour la fonction évite son exécution multiple par ligne. `m.\*` est toujours développé en `ma`, `mb`, `mc`, mais ces variables ne sont plus que des références au résultat de la clause FROM. (Le mot-clé LATERAL est facultatif ici, mais nous l'indiquons pour préciser que la fonction récupère `x` depuis `some\_table`.)

La syntaxe `composite\_value.\*` entraîne une expansion de colonne de ce type lorsqu'elle apparaît au niveau supérieur d'une liste de résultats SELECT, d'une liste RETURNING dans INSERT/UPDATE/DELETE/MERGE, d'une clause VAL-UES ou d'un constructeur de ligne. Dans tous les autres contextes (y compris lorsqu'elle est imbriquée dans l'une de ces constructions), l'ajout de `.\*` à une valeur composite ne modifie pas cette valeur, car il signifie « toutes les colonnes » et la même valeur composite est donc produite à nouveau. Par exemple, si `somefunc()` accepte un argument de type `composite\_value`, ces requêtes sont identiques :

```
SELECT somefunc(c.*) FROM inventory_item c; SELECT somefunc(c)
FROM inventory_item c;
```

Dans les deux cas, la ligne courante de `inventory\_item` est transmise à la fonction comme argument unique de type composite. Bien que `.\*` soit sans effet dans ces cas, son utilisation est recommandée car elle indique clairement qu'une valeur composite est attendue. En particulier, l'analyseur syntaxique considérera `c` dans `c.\*` comme un nom de table ou un alias, et non comme un nom de colonne, ce qui évite toute ambiguïté. Sans `.\*`, il est impossible de déterminer si `c` désigne un nom de table ou de colonne, et l'interprétation en nom de colonne sera privilégiée s'il existe une colonne nommée `c`.

Un autre exemple illustrant ces concepts est que toutes ces requêtes signifient la même chose :

```
SELECT * FROM inventory_item c ORDER BY c; SELECT * FROM
inventory_item c ORDER BY c.*; SELECT * FROM inventory_item c ORDER
BY ROW(c.*);
```

Toutes ces clauses ORDER BY spécifient la valeur composite de la ligne, ce qui a pour effet de trier les lignes selon les règles décrites dans la section 9.25.6. Cependant, si `inventory\_item` contenait une colonne

Si la colonne est nommée « c », le premier cas serait différent des autres, car il s'agirait de trier uniquement selon cette colonne. Compte tenu des noms de colonnes précédemment indiqués, ces requêtes sont également équivalentes à celles ci-dessus :

```
SÉLECTIONNER * DE inventory_item c TRIER PAR LIGNE(c.name, c.supplier_id,
c.prix);
SELECT * FROM inventory_item c ORDER BY (c.name, c.supplier_id,
c.prix);
```

(Le dernier cas utilise un constructeur de ligne avec le mot-clé ROW omis.)

Une autre particularité syntaxique des valeurs composites est la possibilité d'utiliser la notation fonctionnelle pour extraire un champ d'une valeur composite. En d'autres termes, les notations `field(table)` et `table.field` sont interchangeables. Par exemple, ces requêtes sont équivalentes :

```
SELECT c.name FROM inventory_item c WHERE c.price > 1000; SELECT name(c) FROM
inventory_item c WHERE price(c) > 1000;
```

De plus, si nous avons une fonction qui accepte un seul argument de type composite, nous pouvons l'appeler avec l'une ou l'autre notation. Ces requêtes sont toutes équivalentes :

```
SELECT somefunc(c) FROM inventory_item c; SELECT somefunc(c.*)
FROM inventory_item c; SELECT c.somefunc FROM inventory_item c;
```

Cette équivalence entre la notation fonctionnelle et la notation de champ permet d'utiliser des fonctions sur des types composites pour implémenter des « champs calculés ». Une application utilisant la dernière requête ci-dessus n'aurait pas besoin de savoir directement que `somefunc` n'est pas une colonne réelle de la table.

Conseil

En raison de ce comportement, il est déconseillé de nommer une fonction prenant un seul argument de type composite de la même manière qu'un champ de ce type. En cas d'ambiguïté, l'interprétation par nom de champ sera privilégiée si la syntaxe utilisée est celle du nom de champ, tandis que l'interprétation par fonction sera privilégiée si la syntaxe d'appel de fonction est utilisée. Cependant, les versions de PostgreSQL antérieures à la version 11 privilégiaient systématiquement l'interprétation par nom de champ, sauf si la syntaxe de l'appel exigeait qu'il s'agisse d'un appel de fonction. Une solution pour forcer l'interprétation par fonction dans les versions antérieures consiste à qualifier le nom de la fonction par le schéma, c'est-à-dire à écrire `schema.func(compositevalue)`.

8.16.6. Syntaxe d'entrée et de sortie de type composite

La représentation textuelle externe d'une valeur composite comprend des éléments interprétés selon les règles de conversion d'entrée/sortie propres à chaque type de champ, ainsi qu'une mise en forme indiquant la structure composite. Cette mise en forme consiste en des parenthèses ((et)) autour de la valeur et en des virgules (,) entre les éléments adjacents. Les espaces en dehors des parenthèses sont ignorés, mais à l'intérieur, ils sont considérés comme faisant partie de la valeur du champ et leur importance dépend des règles de conversion d'entrée du type de données du champ. Par exemple :

```
'( 42)'
```

Les espaces seront ignorés si le type de champ est un entier, mais pas s'il s'agit de texte.

Comme indiqué précédemment, lors de la saisie d'une valeur composite, vous pouvez encadrer de guillemets doubles n'importe quelle valeur de champ individuelle. Vous devez le faire si la valeur du champ risque de créer une confusion avec la valeur composite.

Analyseur syntaxique. En particulier, les champs contenant des parenthèses, des virgules, des guillemets doubles ou des barres obliques inverses doivent être placés entre guillemets doubles. Pour insérer un guillemet double ou une barre oblique inverse dans la valeur d'un champ composite entre guillemets, faites-le précéder d'une barre oblique inverse. (De plus, une paire de guillemets doubles à l'intérieur d'une valeur de champ entre guillemets doubles est interprétée comme un seul guillemet double, de la même manière que pour les guillemets simples dans les chaînes littérales SQL.) Vous pouvez également éviter les guillemets et utiliser l'échappement par barre oblique inverse pour protéger tous les caractères de données qui seraient autrement interprétés comme de la syntaxe composite.

Une valeur de champ complètement vide (aucun caractère entre les virgules ou les parenthèses) représente une valeur NULL. Pour saisir une valeur qui est une chaîne vide plutôt qu'une valeur NULL, écrivez "".

La routine de sortie composite placera des guillemets doubles autour des valeurs de champ si celles-ci sont des chaînes vides ou contiennent des parenthèses, des virgules, des guillemets, des barres obliques inverses ou des espaces. (Ceci n'est pas indispensable, mais améliore la lisibilité.) Les guillemets et les barres obliques inverses inclus dans les valeurs de champ seront doublés.

Note

N'oubliez pas que ce que vous écrivez dans une commande SQL sera d'abord interprété comme une chaîne littérale, puis comme une valeur composite. Cela double le nombre de barres obliques inverses nécessaires (en supposant l'utilisation de la syntaxe d'échappement des chaînes). Par exemple, pour insérer un champ texte contenant un guillemet et une barre oblique inverse dans une valeur composite, vous devrez écrire :

```
INSÉRER ... VALEURS ('"\'");
```

Le processeur de chaînes littérales supprime un niveau de barres obliques inverses, de sorte que la chaîne reçue par l'analyseur de valeurs composites ressemble à ("\'"). La chaîne fournie à la routine d'entrée du type de données texte devient alors ". (Si nous utilisons un type de données dont la routine d'entrée traite également les barres obliques inverses de manière particulière, comme le type bytea par exemple, nous pourrions avoir besoin de jusqu'à huit barres obliques inverses dans la commande pour en insérer une seule dans le champ composite stocké.) L'utilisation de guillemets en forme de dollar (voir section 4.1.2.4) permet d'éviter de doubler les barres obliques inverses.

Conseil

La syntaxe du constructeur ROW est généralement plus simple à utiliser que la syntaxe des littéraux composites pour écrire des valeurs composites dans les commandes SQL. En ROW, les valeurs des champs individuels sont écrites de la même manière que lorsqu'elles ne font pas partie d'un composite.

8.17. Types de gammes

Les types de plage représentent un intervalle de valeurs d'un type d'élément (appelé sous-type de la plage). Par exemple, des plages d'horodatage peuvent servir à représenter les plages horaires de réservation d'une salle de réunion. Dans ce cas, le type de données est `tsrange` (abréviation de « intervalle d'horodatage »), et `timestamp` est le sous-type. Ce dernier doit avoir un ordre total afin de bien définir si les valeurs de l'élément se situent à l'intérieur, avant ou après l'intervalle de valeurs.

Les types de plages sont utiles car ils permettent de représenter plusieurs valeurs d'éléments dans une seule plage et d'exprimer clairement des concepts tels que le chevauchement des plages. L'utilisation des plages de dates et d'heures à des fins de planification en est l'exemple le plus évident ; mais les plages de prix, les plages de mesure d'un instrument, etc., peuvent également s'avérer utiles.

À chaque type d'intervalle correspond un type d'intervalle multiple. Un intervalle multiple est une liste ordonnée d'intervalles non contigus, non vides et non nuls. La plupart des opérateurs d'intervalle fonctionnent également avec les intervalles multiples et disposent de quelques fonctions spécifiques.

8.17.1. Types de cuisinières encastrables et de cuisinières multi-cuisinières

PostgreSQL intègre les types de plages suivants :

- `int4range` — Plage d'entiers, `int4multirange` — Plage multiple correspondante
- `int8range` — Plage de bigint, `int8multirange` — Plage de multiint correspondante
- `numrange` — Plage de valeurs numériques, `nummultirange` — Plage multiple correspondante
- `tsrange` — Plage d'horodatages sans fuseau horaire, `tsmultirange` — correspondant Multirange
- `tstzrange` — Plage d'horodatage avec fuseau horaire, `tstzmultirange` — Plage multiple correspondante
- `daterange` — Plage de dates, `datemultirange` — Plage multiple correspondante

De plus, vous pouvez définir vos propres types de plages ; consultez `CREATE TYPE` pour plus d'informations.

8.17.2. Exemples

```
CRÉER UNE TABLE réservation (chambre int, pendant tsrange);
INSÉRER DANS réservation VALUES
    (1108, '[2010-01-01 14:30, 2010-01-01 15:30)');
```

```
-- Confinement
```

```
SELECT int4range(10, 20) @> 3;
```

```
-- Chevauchements
```

```
SELECT numrange(11.1, 22.2) && numrange(20.0, 30.0);
```

```
-- Extraire la limite supérieure
```

```
SÉLECTIONNER upper(int8range(15, 25));
```

```
-- Calculer l'intersection
```

```
SELECT int4range(10, 20) * int4range(15, 25);
```

```
-- Le stand est-il vide ?
```

```
SELECT isempty(numrange(1, 5));
```

Voir les tableaux 9.56 et 9.58 pour des listes complètes des opérateurs et des fonctions sur les types de plage.

8.17.3. Limites inclusives et exclusives

Tout intervalle non vide possède deux bornes : une borne inférieure et une borne supérieure. Tous les points compris entre ces bornes font partie de l'intervalle. Une borne inclusive signifie que le point de la borne est inclus dans l'intervalle, tandis qu'une borne exclusive signifie que le point de la borne n'en est pas inclus.

Dans la représentation textuelle d'un intervalle, une borne inférieure inclusive est représentée par « [» tandis qu'une borne inférieure exclusive est représentée par « (». De même, une borne supérieure inclusive est représentée par «] », tandis qu'une borne supérieure exclusive est représentée par «) ». (Voir la section 8.17.5 pour plus de détails.)

Les fonctions `lower_inc` et `upper_inc` testent respectivement l'inclusivité des bornes inférieure et supérieure d'une valeur de plage.

8.17.4. Plages infinies (non bornées)

La borne inférieure d'un intervalle peut être omise ; ainsi, toutes les valeurs inférieures à la borne supérieure sont incluses dans l'intervalle, par exemple : `(,3)`. De même, si la borne supérieure de l'intervalle est omise, toutes les valeurs supérieures à la borne inférieure sont incluses. Si les bornes inférieure et supérieure sont toutes deux omises, toutes les valeurs du type d'élément sont considérées comme appartenant à l'intervalle. Spécifier une borne manquante comme inclusive est automatiquement converti en exclusive, par exemple : `[,]` devient `(,)`. Ces valeurs manquantes peuvent être interprétées comme des valeurs $\pm\infty$, mais il s'agit de valeurs particulières du type « intervalle », considérées comme étant au-delà des valeurs $\pm\infty$ de tout élément de ce type.

Les types d'éléments qui prennent en charge la notion d'« infini » peuvent l'utiliser comme valeur limite explicite. Par exemple, avec les plages d'horodatage, `[aujourd'hui,infini]` exclut la valeur d'horodatage spéciale infini, tandis que `[aujourd'hui,infini]` l'inclut, de même que `[aujourd'hui,)` et `[aujourd'hui,]`.

Les fonctions `lower_inf` et `upper_inf` testent respectivement les limites inférieures et supérieures infinies d'un intervalle.

8.17.5. Plage d'entrée/sortie

La valeur saisie pour une plage de valeurs doit suivre l'un des modèles suivants :

(borne inférieure, borne supérieure) (borne inférieure, borne supérieure) [borne inférieure, borne supérieure) [borne inférieure, borne supérieure] vide

Les parenthèses ou les crochets indiquent si les bornes inférieure et supérieure sont exclusives ou incluses, comme décrit précédemment. Notez que le motif final est vide, ce qui représente un intervalle vide (un intervalle ne contenant aucun point).

La borne inférieure peut être une chaîne de caractères valide pour le sous-type, ou une valeur vide pour indiquer l'absence de borne inférieure. De même, la borne supérieure peut être une chaîne de caractères valide pour le sous-type, ou une valeur vide pour indiquer l'absence de borne supérieure.

Chaque valeur à encadrer peut être placée entre guillemets doubles (« " »). Ceci est nécessaire si la valeur contient des parenthèses, des crochets, des virgules, des guillemets ou des barres obliques inverses, car ces caractères seraient sinon interprétés comme faisant partie de la syntaxe d'intervalle. Pour insérer un guillemet ou une barre oblique inverse dans une valeur à encadrer, faites-le précéder d'une barre oblique inverse. (De plus, une paire de guillemets à l'intérieur d'une valeur à encadrer est interprétée comme un seul guillemet, de la même manière que pour les guillemets simples dans les chaînes littérales SQL.) Vous pouvez également éviter les guillemets et utiliser l'échappement par barre oblique inverse pour protéger tous les caractères de données qui seraient autrement interprétés comme faisant partie de la syntaxe d'intervalle. Enfin, pour écrire une valeur à encadrer qui est une chaîne vide, écrivez « " », car une valeur vide représente une limite infinie.

Les espaces sont autorisés avant et après la valeur de l'intervalle, mais tout espace entre parenthèses ou crochets est considéré comme faisant partie de la limite inférieure ou supérieure. (Selon le type d'élément, cela peut être significatif ou non.)

Note

Ces règles sont très similaires à celles qui régissent l'écriture des valeurs de champs dans les littéraux de type composite. Voir la section 8.16.6 pour plus de détails.

Exemples :

-- comprend 3, n'inclut pas 7 et comprend tous les points dans
entre

SÉLECTIONNER '[3,7)]:int4range;

-- n'inclut ni 3 ni 7, mais inclut tous les points de
entre

SÉLECTIONNER '(3,7)':int4range;

-- comprend uniquement le point 4

SÉLECTIONNER '[4,4]':int4range;

-- ne contient aucun point (et sera normalisé à « vide »)

SÉLECTIONNER '[4,4)':int4range;

Les données d'entrée d'un multirange sont des accolades ({ et }) contenant zéro ou plusieurs intervalles valides, séparés par des virgules. Les espaces sont autorisés autour des accolades et des virgules. Cette syntaxe rappelle celle des tableaux, bien que les multiranges soient beaucoup plus simples : ils n'ont qu'une seule dimension et il n'est pas nécessaire d'encadrer leur contenu par des guillemets. (Les limites de leurs intervalles peuvent toutefois être encadrées par des guillemets, comme indiqué ci-dessus.)

Exemples :

SÉLECTIONNER '{}':int4multirange;

SELECT '{{[3,7)}':int4multirange;

SELECT '{{[3,7), [8,9)}':int4multirange;

8.17.6. Construction de plages et de plages multiples

Chaque type d'intervalle possède une fonction constructeur portant le même nom. L'utilisation de cette fonction est souvent plus pratique que la définition d'une constante littérale d'intervalle, car elle évite d'avoir à encadrer les valeurs limites par des guillemets supplémentaires. La fonction constructeur accepte deux ou trois arguments. La forme à deux arguments construit un intervalle au format standard (borne inférieure incluse, borne supérieure exclue), tandis que la forme à trois arguments construit un intervalle dont les limites sont définies par le troisième argument. Ce dernier doit être l'une des chaînes de caractères « () », « [] », « [] » ou « [] ». Par exemple :

-- La forme complète est : limite inférieure, limite supérieure et argument de texte
indiquant --

l'inclusivité/l'exclusivité des limites.

SÉLECTIONNER numrange(1.0, 14.0, '[)');

-- Si le troisième argument est omis, '[)' est supposé.

SÉLECTIONNER numrange(1.0, 14.0);

-- Bien que '[)' soit spécifié ici, la valeur affichée sera
converti en

— forme canonique, puisque int8range est un type de plage discret (voir
ci-dessous).

SELECT int8range(1, 14, '[)');

-- L'utilisation de NULL pour l'une ou l'autre des bornes entraîne une plage non bornée sur
de ce côté.

SÉLECTIONNER numrange(NULL, 2.2);

Chaque type de plage possède également un constructeur multiplage portant le même nom. Ce constructeur accepte zéro ou plusieurs arguments, qui sont tous des plages du type approprié. Par exemple :

```
SELECT nummultirange(); SELECT
nummultirange(numrange(1.0, 14.0)); SELECT nummultirange(numrange(1.0,
14.0), numrange(20.0, 25.0));
```

8.17.7. Types de plages discrètes

Une plage discrète est une plage dont le type d'élément possède un « intervalle » bien défini, comme un entier ou une date. Dans ces cas, deux éléments peuvent être considérés comme adjacents lorsqu'il n'existe aucune valeur valide entre eux. Cela contraste avec les plages continues, où il est toujours (ou presque toujours) possible d'identifier d'autres valeurs d'éléments entre deux valeurs données. Par exemple, une plage sur le type numérique est continue, de même qu'une plage sur l'horodatage. (Bien que l'horodatage ait une précision limitée et puisse donc théoriquement être traité comme discret, il est préférable de le considérer comme continu, car la taille du pas n'est généralement pas pertinente.)

Une autre façon d'appréhender un type intervalle discret est de considérer qu'à chaque valeur d'élément correspond une valeur « suivante » ou « précédente ». Grâce à cela, il est possible de convertir entre une représentation inclusive et une représentation exclusive des limites d'un intervalle, en choisissant la valeur d'élément suivante ou précédente à la place de celle initialement fournie. Par exemple, dans un intervalle d'entiers, [4,8] et (3,9) désignent le même ensemble de valeurs ; ce qui ne serait pas le cas pour un intervalle numérique.

Un type d'intervalle discret doit posséder une fonction de canonisation qui prenne en compte le pas souhaité pour le type d'élément. Cette fonction de canonisation a pour rôle de convertir les valeurs équivalentes du type d'intervalle afin qu'elles aient des représentations identiques, notamment des bornes inclusives ou exclusives cohérentes.

Si aucune fonction de canonisation n'est spécifiée, les plages de valeurs ayant un formatage différent seront toujours considérées comme inégales, même si elles représentent en réalité le même ensemble de valeurs.

Les types de plages intégrés `int4range`, `int8range` et `daterange` utilisent tous une forme canonique qui inclut la borne inférieure et exclut la borne supérieure ; c'est-à-dire `[]`. Les types de plages définis par l'utilisateur peuvent toutefois utiliser d'autres conventions.

8.17.8. Définition de nouveaux types de gammes

Les utilisateurs peuvent définir leurs propres types d'intervalles. La raison la plus courante est d'utiliser des intervalles sur des sous-types non disponibles parmi les types d'intervalles intégrés. Par exemple, pour définir un nouveau type d'intervalle de sous-type `float8` :

```
CRÉER LE TYPE floatrange AS RANGE ( subtype = float8,
    subtype_diff = float8mi
);
```

```
SÉLECTIONNER '[1.234, 5.678]':floatrange;
```

Étant donné que `float8` n'a pas de « pas » significatif, nous ne définissons pas de fonction de canonisation dans cet exemple.

Lorsque vous définissez votre propre plage, vous obtenez automatiquement un type multiplage correspondant.

Définir votre propre type de plage vous permet également de spécifier une classe d'opérateurs d'arbre B de sous-type différent ou un classement différent à utiliser, afin de modifier l'ordre de tri qui détermine quelles valeurs appartiennent à une plage donnée.

Si le sous-type est considéré comme ayant des valeurs discrètes plutôt que continues, la commande `CREATE TYPE` doit spécifier une fonction canonique. Cette fonction prend en entrée une plage de valeurs et doit renvoyer une plage équivalente, dont les bornes et le format peuvent différer. La sortie canonique de deux plages représentant le même ensemble de valeurs (par exemple, les plages d'entiers [1, 7] et [1, 8]) doit être identique. Le choix de la représentation canonique importe peu, pourvu que deux valeurs équivalentes avec des formats différents soient toujours associées à la même valeur avec le même format. Outre l'ajustement du format des bornes inclusives/exclusives, une

La fonction de canonisation peut arrondir les valeurs limites si l'intervalle souhaité dépasse la capacité de stockage du sous-type. Par exemple, un type de plage sur un horodatage peut être défini avec un intervalle d'une heure ; dans ce cas, la fonction de canonisation devra arrondir les bornes qui ne sont pas des multiples d'une heure, ou générer une erreur.

De plus, tout type de plage destiné à être utilisé avec les index GiST ou SP-GiST doit définir une fonction de différence de sous-type, ou `subtype_diff`. (L'index fonctionnera même sans `subtype_diff`, mais il sera probablement beaucoup moins efficace qu'avec une fonction de différence.) La fonction de différence de sous-type prend deux valeurs d'entrée du sous-type et renvoie leur différence (X moins Y) sous forme de nombre à virgule flottante 8 bits (float8). Dans notre exemple, la fonction `float8mi`, qui sous-tend l'opérateur de soustraction standard `float8`, peut être utilisée ; mais pour tout autre sous-type, une conversion de type sera nécessaire. Il faudra peut-être aussi réfléchir à la manière de représenter les différences sous forme numérique. Dans la mesure du possible, la fonction `subtype_diff` doit respecter l'ordre de tri implicite de la classe d'opérateurs et du classement sélectionnés ; autrement dit, son résultat doit être positif lorsque son premier argument est supérieur à son second, conformément à l'ordre de tri.

Voici un exemple moins simplifié de la fonction `subtype_diff` :

```
CRÉER UNE FONCTION time_subtype_diff(x temps, y temps) RETOURNE float8 AS
'SELECT EXTRACT(EPOCH FROM (x - y))' LANGUAGE sql STRICT IMMUTABLE;
```

```
CRÉER LE TYPE timerange AS RANGE ( subtype = time,
    subtype_diff =
    time_subtype_diff
);
```

```
SÉLECTIONNER '[11:10, 23:00]::timerange;
```

Consultez la section `CRÉER UN TYPE` pour plus d'informations sur la création de types de plage.

8.17.9. Indexation

Il est possible de créer des index GiST et SP-GiST pour les colonnes de type plage. Les index GiST peuvent également être créés pour les colonnes de type multiplage. Par exemple, pour créer un index GiST :

```
CRÉER UN INDEX reservation_idx SUR reservation UTILISANT GIST (pendant);
```

Un index GiST ou SP-GiST sur les plages peut accélérer les requêtes utilisant les opérateurs de plage suivants : `=`, `&&`, `<@`, `@>`, `<<`, `>>`, `-|`, `&<` et `&>`. Un index GiST sur les plages multiples peut accélérer les requêtes utilisant le même ensemble d'opérateurs. Un index GiST sur les plages et un index GiST sur les plages multiples peuvent également accélérer les requêtes utilisant les opérateurs de conversion de type plage vers plage multiple et de plage multiple vers plage suivants : `&&`, `<@`, `@>`, `<<`, `>>`, `-|`, `&<` et `&>`. Voir le tableau 9.56 pour plus d'informations.

De plus, il est possible de créer des index B-tree et de hachage pour les colonnes de type plage. Pour ces types d'index, l'égalité est la seule opération de plage réellement utile. Un ordre de tri B-tree est défini pour les valeurs de plage, avec les opérateurs `<` et `>` correspondants, mais cet ordre est assez arbitraire et rarement pertinent en pratique. La prise en charge des index B-tree et de hachage pour les types plage vise principalement à permettre le tri et le hachage internes aux requêtes, plutôt que la création d'index à proprement parler.

8.17.10. Contraintes sur les plages

Bien que la contrainte `UNIQUE` soit naturelle pour les valeurs scalaires, elle est généralement inadaptée aux types de plages. Une contrainte d'exclusion est souvent plus appropriée (voir `CREATE TABLE ... CONSTRAINT ... EXCLUDE`). Les contraintes d'exclusion permettent de spécifier des contraintes telles que « non-chevauchement » sur un type de plage. Par exemple :

```
CRÉER UNE RÉSERVATION DE TABLE (
    pendant tsrange,
    EXCLUEZ L'UTILISATION DE GIST (pendant WITH &&)
);
```

Cette contrainte empêchera la présence simultanée de valeurs identiques dans le tableau :

```
INSÉRER DANS reservation VALUES
    ('[2010-01-01 11:30, 2010-01-01 15:00]');
INSÉRER 0 1
```

```
INSÉRER DANS reservation VALUES
    ('[2010-01-01 14:45, 2010-01-01 15:45]');
ERREUR : une valeur de clé conflictuelle viole la contrainte d'exclusion
    "réservation_pendant_excl"
DÉTAIL : Clé (pendant)=(["2010-01-01 14:45:00","2010-01-01
    15:45:00"]) conflits
avec clé existante (pendant)=(["2010-01-01 11:30:00","2010-01-01
    15:00:00"]).
```

Vous pouvez utiliser l'extension `btree_gist` pour définir des contraintes d'exclusion sur des types de données scalaires simples, qui peuvent ensuite être combinées avec des exclusions de plages pour une flexibilité maximale. Par exemple, après l'installation de `btree_gist`, la contrainte suivante rejettera les plages qui se chevauchent uniquement si les numéros de salle de réunion sont égaux :

```
CRÉER L'EXTENSION btree_gist;
CRÉER LA TABLE room_reservation (
    texte de la pièce,
    pendant tsrange,
    EXCLURE L'UTILISATION DE GIST (pièce AVEC =, pendant AVEC &&)
);
```

```
INSÉRER DANS room_reservation VALUES
    ('123A', '[01/01/2010 14h00, 01/01/2010 15h00]');
INSÉRER 0 1
```

```
INSÉRER DANS room_reservation VALUES
    ("123A", "[2010-01-01 14:30, 2010-01-01 15:30]");
ERREUR : une valeur de clé conflictuelle viole la contrainte d'exclusion
    "room_reservation_room_pendant_excl"
DÉTAIL : Clé (pièce, pendant)=(123A, ["2010-01-01 14:30:00","2010-01-01 15:30:00"])
    conflits
avec clé existante (chambre, pendant)=(123A, ["2010-01-01 14:00:00","2010-01-01 15:00:00"]).
```

```
INSÉRER DANS room_reservation VALUES
    (« 123B », « [2010-01-01 14:30, 2010-01-01 15:30] »)
INSÉRER 0 1
```

8.18. Types de domaines

Un domaine est un type de données défini par l'utilisateur et basé sur un autre type sous-jacent. Il peut comporter des contraintes limitant ses valeurs valides à un sous-ensemble de celles autorisées par le type sous-jacent. Sinon, son comportement est identique à celui du type sous-jacent : tout opérateur ou fonction applicable au type sous-jacent fonctionne également sur le type du domaine. Le type sous-jacent peut être n'importe quel type de base intégré ou défini par l'utilisateur, un type énuméré, un type tableau, un type composite, un type intervalle ou un autre domaine.

Par exemple, nous pourrions créer un domaine sur les entiers qui n'accepte que les entiers positifs :

```
CRÉER UN DOMAINE posint AS entier VÉRIFIER (VALEUR > 0);
CRÉER TABLE mytable (id position);
INSERT INTO mytable VALUES(1); -- fonctionne
INSERT INTO mytable VALUES(-1); -- échec
```

Lorsqu'un opérateur ou une fonction du type sous-jacent est appliqué à une valeur du domaine, ce dernier est automatiquement converti vers le type sous-jacent. Ainsi, par exemple, le résultat de `mytable.id - 1` est considéré comme étant de type `integer` et non `posint`. On pourrait écrire `(mytable.id - 1)::posint` pour reconvertir le résultat en `posint`, ce qui entraînerait une nouvelle vérification des contraintes du domaine. Dans ce cas, une erreur serait générée si l'expression avait été appliquée à la valeur `1` de l'identifiant. Il est possible d'affecter une valeur du type sous-jacent à un champ ou une variable du type du domaine sans conversion explicite, mais les contraintes du domaine seront alors vérifiées.

Pour plus d'informations, consultez la section [CRÉER UN DOMAINE](#).

8.19. Types d'identificateurs d'objets

Les identificateurs d'objets (OID) sont utilisés en interne par PostgreSQL comme clés primaires pour diverses tables système. Le type `oid` représente un identificateur d'objet. Il existe également plusieurs alias pour `oid`, chacun nommé `regsomething`. Le tableau 8.26 en donne un aperçu.

Le type `oid` est actuellement implémenté comme un entier non signé de quatre octets. Par conséquent, sa taille est insuffisante pour garantir l'unicité à l'échelle de la base de données dans les grandes bases de données, voire même dans les grandes tables individuelles.

Le type `oid` lui-même ne possède que peu d'opérations comparables. Il peut toutefois être converti en entier, puis manipulé à l'aide des opérateurs entiers standard. (Attention à la confusion possible entre nombres signés et non signés lors de cette opération.)

Les types alias `OID` ne possèdent aucune opération propre, à l'exception de routines d'entrée et de sortie spécialisées. Ces routines peuvent accepter et afficher les noms symboliques des objets système, plutôt que leur valeur numérique brute, contrairement au type `oid`. Les alias permettent une recherche simplifiée des valeurs `OID` des objets. Par exemple, pour examiner les lignes `pg_attribute` relatives à la table `mytable`, on peut écrire :

```
SELECT * FROM pg_attribute WHERE attrelid = 'mytable'::regclass;
```

plutôt que :

```
SÉLECTIONNER * DE pg_attribute
OÙ attrelid = (SÉLECTIONNER oid DE pg_class OÙ relname = 'mytable');
```

Bien que cela ne paraisse pas si mal en soi, c'est encore trop simplifié. Une sous-requête bien plus complexe serait nécessaire pour sélectionner l'`OID` approprié s'il existe plusieurs tables nommées « `mytable` » dans différents schémas. Le convertisseur d'entrée `regclass` gère la recherche de table en fonction du chemin du schéma, et effectue donc automatiquement la opération adéquate. De même, la conversion de l'`OID` d'une table en `regclass` est pratique pour l'affichage symbolique d'un `OID` numérique.

Tableau 8.26. Types d'identificateurs d'objets

Nom	Références	Description	Exemple de valeur
<code>oid</code>	à l'origine <code>regoid</code>	identifiant d'objet numérique	564182

Nom	Références	Description	Exemple de valeur
classe régulière	classe pg	nom de relation	pg_type
régularisation	pg_collation	nom de classement	"POSIX"
regconfig	pg_ts_config	configuration de recherche textuelle	Anglais
dictionnaire régional	pg_ts_dict	dictionnaire de recherche textuelle simple	
espace de noms de registre	espace de noms pg	nom de l'espace de noms	catalogue pg
régulateur	opérateur pg	nom de l'opérateur	+
régulateur	opérateur pg	opérateur avec argument types	*(entier, entier) ou - (AUCUN, entier)
regproc	pg_proc	nom de la fonction	somme
procédure régulière	pg_proc	fonction avec argument types	somme(int4)
regrole	pg_authid	nom du rôle	forgeron
regtype	pg_type	nom du type de données	entier

Tous les types d'alias OID pour les objets regroupés par espace de noms acceptent les noms qualifiés par schéma. et affichera les noms qualifiés par schéma dans la sortie si l'objet est introuvable dans le schéma actuel chemin de recherche sans qualification. Par exemple, myschema.mytable est une entrée acceptable pour regclass (si une telle table existe). Cette valeur peut être affichée sous le nom myschema.mytable, ou simplement mytable, en fonction du chemin de recherche actuel. Les types d'alias regproc et regoper n'acceptent que des noms d'entrée uniques (non surchargés), leur utilité est donc limitée ; pour la plupart des cas Les termes regprocedure ou regoperator sont plus appropriés. Pour regoperator, les opérateurs unaires sont identifiés en écrivant NONE pour l'opérande inutilisé.

Les fonctions de saisie pour ces types autorisent les espaces entre les jetons et replient les lettres majuscules. en minuscules, sauf à l'intérieur des guillemets doubles ; ceci est fait pour rendre les règles de syntaxe similaires à la manière Les noms des objets sont écrits en SQL. En revanche, les fonctions de sortie utiliseront des guillemets doubles si nécessaire. pour que la sortie soit un identifiant SQL valide. Par exemple, l'OID d'une fonction nommée Foo (avec La lettre F majuscule prenant deux arguments entiers peut être saisie comme "Foo (int, entier) '::regprocedure. Le résultat serait de la forme "Foo"(entier,entier). Le nom de la fonction et les noms des types d'arguments pourraient également être qualifiés par un schéma.

De nombreuses fonctions intégrées de PostgreSQL acceptent l'OID d'une table, ou d'un autre type d'objet de base de données, et Par commodité, ils sont déclarés comme prenant regclass (ou le type alias OID approprié). Cela signifie Vous n'avez pas besoin de rechercher manuellement l'OID de l'objet, vous pouvez simplement saisir son nom sous forme de chaîne de caractères. Par exemple, la fonction nextval(regclass) prend l'OID d'une relation de séquence, vous pouvez donc l'appeler

Cela ressemble à ceci :

nextval('foo')	fonctionne sur la séquence foo
nextval('FOO')	Identique à ci-dessus
nextval('"Foo"')	fonctionne sur la séquence Foo
nextval('myschema.foo')	fonctionne sur myschema.foo
nextval('"myschema".foo') idem comme ci-dessus	
nextval('foo')	recherche le chemin de recherche pour foo

Note

Lorsque vous écrivez l'argument d'une telle fonction sous forme de chaîne littérale sans ornement, il devient une constante de type regclass (ou du type approprié). Comme il s'agit simplement d'un OID, permettra de suivre l'objet initialement identifié malgré tout changement ultérieur de nom, de schéma, etc.

Ce comportement de « liaison anticipée » est généralement souhaitable pour les références d'objets dans les valeurs par défaut des colonnes et

vues. Mais il peut arriver que vous souhaitiez une « liaison tardive », où la référence à l'objet est résolue à l'exécution. Pour obtenir ce comportement, forcez le stockage de la constante en tant que constante textuelle plutôt qu'en tant que regclass :

```
nextval('foo'::text)          foo est recherché lors de l'exécution
```

La fonction `to_regclass()` et ses fonctions apparentées peuvent également être utilisées pour effectuer des recherches à l'exécution. Voir le tableau 9.74.

Un autre exemple pratique d'utilisation de `regclass` consiste à rechercher l'OID d'une table listée dans les vues `information_schema`, qui ne fournissent pas directement ces OID. On pourrait par exemple souhaiter appeler la fonction `pg_relation_size()`, qui requiert l'OID de la table. En tenant compte des règles ci-dessus, la méthode correcte pour ce faire est la suivante :

```
SELECT table_schema, table_name,
       pg_relation_size((quote_ident(table_schema) || '.' || quote_ident(table_name))::regclass)
```

À PARTIR DE `information_schema.tables`
OÙ ...

La fonction `quote_ident()` se chargera d'encadrer les identifiants de guillemets doubles lorsque cela est nécessaire. L'opération semble plus simple.

```
SELECT pg_relation_size(table_name)
FROM information_schema.tables WHERE
...
```

Cette méthode n'est pas recommandée, car elle échouera pour les tables situées en dehors de votre chemin de recherche ou dont les noms nécessitent des guillemets.

Une propriété supplémentaire de la plupart des alias OID est la création de dépendances. Si une constante de l'un de ces types apparaît dans une expression stockée (comme l'expression par défaut d'une colonne ou une vue), elle crée une dépendance sur l'objet référencé. Par exemple, si une colonne possède une expression par défaut ``nextval('my_seq'::regclass)``, PostgreSQL comprend que cette expression dépend de la séquence ``my_seq``, et le système n'autorisera donc pas la suppression de la séquence sans supprimer au préalable l'expression par défaut. L'alternative ``nextval('my_seq'::text)`` ne crée pas de dépendance. (Le type ``regole`` fait exception à cette règle : les constantes de ce type ne sont pas autorisées dans les expressions stockées.)

Un autre type d'identifiant utilisé par le système est l'identifiant `xid`, ou identifiant de transaction (abrégé `xact`). Il s'agit du type de données des colonnes système `xmin` et `xmax`. Les identifiants de transaction sont des valeurs de 32 bits. Dans certains contextes, une variante 64 bits, `xid8`, est utilisée. Contrairement aux valeurs `xid`, les valeurs `xid8` augmentent de manière strictement monotone et ne peuvent pas être réutilisées au cours de la durée de vie d'un cluster de base de données. Voir la section 66.1 pour plus de détails.

Un troisième type d'identificateur utilisé par le système est le `cid`, ou identificateur de commande. Il s'agit du type de données des colonnes système `cmin` et `cmax`. Les identificateurs de commande sont également des valeurs de 32 bits.

Un dernier type d'identifiant utilisé par le système est le `tid`, ou identifiant de tuple (identifiant de ligne). Il s'agit du type de données de la colonne système `ctid`. Un identifiant de tuple est une paire (numéro de bloc, index du tuple dans le bloc) qui identifie l'emplacement physique de la ligne dans sa table.

(Les colonnes du système sont expliquées plus en détail dans la section 5.6.)

8.20. Type `pg_lsn`

Le type de données `pg_lsn` peut être utilisé pour stocker des données LSN (numéro de séquence de journalisation) qui sont un pointeur vers un emplacement dans le WAL. Ce type représente un type `XLogRecPtr` et un type système interne de PostgreSQL.

En interne, un LSN est un entier de 64 bits représentant une position d'octet dans le flux de journalisation anticipée. est imprimé sous la forme de deux nombres hexadécimaux de 8 chiffres maximum chacun, séparés par une barre oblique ; par exemple, `16/B374D848`. Le type `pg_lsn` prend en charge les opérateurs de comparaison standard, tels que `=` et `>`. Deux Les LSN peuvent être soustraits à l'aide de l'opérateur `-` ; le résultat correspond au nombre d'octets séparant ces emplacements de journalisation anticipée. De même, le nombre d'octets peut être ajouté ou soustrait d'un LSN à l'aide de l'opérateur `+`. Les opérateurs `+(pg_lsn,numeric)` et `-(pg_lsn,numeric)` sont utilisés respectivement. Notez que le LSN calculé doit être compris dans la plage du type `pg_lsn`, c'est-à-dire entre `0/0` et `FFFFFFFF/FFFFFFFF`.

8.21. Pseudo-types

Le système de types de PostgreSQL contient un certain nombre d'entrées à usage spécifique qui sont collectivement appelées pseudo-types. Un pseudo-type ne peut pas être utilisé comme type de données de colonne, mais il peut être utilisé pour déclarer un Type d'argument ou de résultat d'une fonction. Chaque pseudo-type disponible est utile dans les situations où un Le comportement de cette fonction ne se limite pas à la simple prise ou au renvoi d'une valeur de données SQL spécifiques. type. Le tableau 8.27 répertorie les pseudo-types existants.

Tableau 8.27. Pseudo-types

Nom	Description
<code>*</code>	Indique qu'une fonction accepte tout type de données d'entrée.
<code>tout élément</code>	Indique qu'une fonction accepte tout type de données (voir section 36.2.5).
<code>tout tableau</code>	Indique qu'une fonction accepte tout type de données de tableau (voir Section 36.2.5).
<code>anyonarray</code>	Indique qu'une fonction accepte tout type de données autre qu'un tableau (voir section 36.2.5).
<code>anyenum</code>	Indique qu'une fonction accepte tout type de données énuméré (voir Article 36.2.5 et article 8.7).
<code>toute gamme</code>	Indique qu'une fonction accepte tout type de données de plage (voir Article 36.2.5 et article 8.17).
<code>toute gamme multiple</code>	Indique qu'une fonction accepte tout type de données multirange (voir section 36.2.5 et section 8.17).
<code>compatible</code>	Indique qu'une fonction accepte tout type de données, avec conversion automatique des arguments multiples en un type de données commun. type (voir section 36.2.5).
<code>anycompatiblearray</code>	Indique qu'une fonction accepte tout type de données de tableau, avec promotion automatique de plusieurs arguments vers un argument commun type de données (voir section 36.2.5).
<code>tout non-tableau compatible</code>	Indique qu'une fonction accepte tout type de données autre qu'un tableau. avec la promotion automatique de plusieurs arguments vers un type de données commun (voir section 36.2.5).
<code>toute plage compatible</code>	Indique qu'une fonction accepte tout type de données de plage, avec promotion automatique de plusieurs arguments vers un argument commun type de données (voir section 36.2.5 et section 8.17).
<code>toute gamme multi-compatible</code>	Indique qu'une fonction accepte tout type de données multirange. avec la promotion automatique de plusieurs arguments vers un type de données commun (voir section 36.2.5 et section 8.17).
<code>chaîne c</code>	Indique qu'une fonction accepte ou renvoie une chaîne C terminée par un caractère nul.

Nom	Description
interne	Indique qu'une fonction accepte ou renvoie une valeur interne au serveur. type de données.
gestionnaire_de_langue	Un gestionnaire d'appels de langage procédural est déclaré pour retourner <code>gestionnaire_de_langue</code> .
gestionnaire fdw	Un gestionnaire d'encapsulation de données étrangères est déclaré pour renvoyer <code>fd-w_handler</code> .
table_am_handler	Un gestionnaire de méthode d'accès à une table est déclaré pour renvoyer <code>table_am_handler</code> .
gestionnaire_index_am	Un gestionnaire de méthode d'accès à l'index est déclaré pour retourner dans <code>gestionnaire_dex_am</code> .
gestionnaire tsm	Un gestionnaire de méthode tablesample est déclaré pour renvoyer <code>gestionnaire_tsm</code> .
enregistrer	Identifie une fonction prenant ou renvoyant une ligne non spécifiée <code>taper</code> .
déclenchement	Une fonction de déclenchement est déclarée pour renvoyer un déclencheur.
déclencheur d'événement	Une fonction de déclenchement d'événement est déclarée pour renvoyer <code>even-t_trigger</code> .
pg_ddl_command	Identifie une représentation des commandes DDL disponible pour les déclencheurs d'événements.
vide	Indique qu'une fonction ne renvoie aucune valeur.
inconnu	Identifie un type non encore résolu, par exemple, d'un objet non décoré chaîne littérale.

Les fonctions codées en C (qu'elles soient intégrées ou chargées dynamiquement) peuvent être déclarées pour accepter ou renvoyer n'importe quel type. de ces pseudo-types. Il incombe à l'auteur de la fonction de s'assurer que celle-ci se comportera en toute sécurité. lorsqu'un pseudo-type est utilisé comme type d'argument.

Les fonctions codées dans des langages procéduraux ne peuvent utiliser de pseudo-types que dans les limites autorisées par leur langage d'implémentation. Actuellement, la plupart des langages procéduraux interdisent l'utilisation d'un pseudo-type comme argument. type, et autoriser uniquement void et record comme type de résultat (plus trigger ou event_trigger lorsque la fonction est utilisée comme déclencheur ou déclencheur d'événement). Certaines prennent également en charge les fonctions polymorphes utilisant la pseudotypes polymorphes, qui sont présentés ci-dessus et discutés en détail dans la section 36.2.5.

Le pseudo-type interne est utilisé pour déclarer des fonctions destinées à être appelées uniquement en interne. par le système de base de données, et non par un appel direct dans une requête SQL. Si une fonction possède au moins une Si l'argument est de type interne, il ne peut pas être appelé depuis SQL. Pour préserver la sécurité des types de ceci Il est important de respecter cette règle de codage : ne créez aucune fonction déclarée comme devant retourner une valeur. interne sauf si elle comporte au moins un argument interne.

Chapitre 9. Fonctions et opérateurs

PostgreSQL fournit un grand nombre de fonctions et d'opérateurs pour les types de données intégrés. Ce chapitre décrit la plupart d'entre elles, bien que des fonctions spécifiques supplémentaires apparaissent dans les sections pertinentes de le manuel. Les utilisateurs peuvent également définir leurs propres fonctions et opérateurs, comme décrit dans la partie V. Le `psql` Les commandes `\df` et `\do` peuvent être utilisées pour lister respectivement toutes les fonctions et tous les opérateurs disponibles.

La notation utilisée tout au long de ce chapitre pour décrire les types de données des arguments et des résultats d'une fonction L'opérateur « ou » fonctionne ainsi :

répéter (texte, entier) → texte

qui indique que la fonction `repeat` prend un argument de texte et un argument entier et renvoie un résultat de Saisissez du texte. La flèche vers la droite sert également à indiquer le résultat d'un exemple, comme ceci :

répéter('Pg', 4) → PgPgPgPg

Si la portabilité vous préoccupe, notez que la plupart des fonctions et opérateurs décrits

Dans ce chapitre, à l'exception des opérateurs arithmétiques et de comparaison les plus triviaux et de quelques autres, nous n'aborderons pas les opérations arithmétiques et de comparaison les plus élémentaires.

Les fonctions explicitement marquées ne sont pas spécifiées par la norme SQL. Certaines de ces fonctionnalités étendues sont présentes dans d'autres systèmes de gestion de bases de données SQL, et dans de nombreux cas, ces fonctionnalités sont... compatible et cohérente entre les différentes implémentations.

9.1. Opérateurs logiques

Les opérateurs logiques habituels sont disponibles :

booléen ET booléen → booléen

booléen OU booléen → booléen

NON booléen → booléen

SQL utilise un système logique à trois valeurs : vrai, faux et nul, qui représente « inconnu ». Consultez les tables de vérité suivantes :

a	b	a ET b	a OU b
VRAI	VRAI	VRAI	VRAI
VRAI	FAUX	FAUX	VRAI
VRAI	NUL	NUL	VRAI
FAUX	FAUX	FAUX	FAUX
FAUX	NUL	FAUX	NUL
NUL	NUL	NUL	NUL

a	PAS a
VRAI	FAUX
FAUX	VRAI
NUL	NUL

Les opérateurs ET et OU sont commutatifs, c'est-à-dire que l'on peut inverser les opérandes de gauche et de droite sans les inverser. ce qui affecte le résultat. (Toutefois, il n'est pas garanti que l'opérande de gauche soit évalué avant celui de droite.) opérande. Voir la section 4.2.14 pour plus d'informations sur l'ordre d'évaluation des sous-expressions.

9.2. Fonctions et opérateurs de comparaison

Les opérateurs de comparaison habituels sont disponibles, comme indiqué dans le tableau 9.1.

Tableau 9.1. Opérateurs de comparaison

Opérateur	Description
type de données < type de données → booléen	Moins que
type de données > type de données → booléen	Supérieur à
type de données <= type de données → booléen	Inférieur ou égal à
type de données >= type de données → booléen	Supérieur ou égal à
type de données = type de données → booléen	Égal
type de données <> type de données → booléen	Non égal
type de données → booléen	Non égal

Note

L'opérateur <> est la notation SQL standard pour « différent de ». != est un alias, converti en <> très tôt lors de l'analyse syntaxique. Par conséquent, il est impossible d'implémenter des opérateurs != et <> ayant des comportements différents.

Ces opérateurs de comparaison sont disponibles pour tous les types de données intégrés possédant un ordre naturel, notamment les types numériques, chaînes de caractères et dates/heures. De plus, les tableaux, les types composites et les intervalles peuvent être comparés si leurs types de données constitutifs sont comparables.

Il est généralement possible de comparer des valeurs de types de données apparentés ; par exemple, 'integer > bigint' fonctionne. Dans certains cas, ce type de comparaison est directement implémenté par des opérateurs de comparaison « inter-types », mais si aucun opérateur de ce type n'est disponible, l'analyseur syntaxique convertit le type le moins général vers le type le plus général et applique l'opérateur de comparaison de ce dernier.

Comme indiqué ci-dessus, tous les opérateurs de comparaison sont des opérateurs binaires qui renvoient des valeurs de type booléen. Ainsi, les expressions comme `1 < 2 < 3` ne sont pas valides (car l'opérateur < ne permet pas de comparer une valeur booléenne à 3). Utilisez les prédicats BETWEEN ci-dessous pour effectuer des tests d'intervalle.

Il existe également des prédicats de comparaison, comme indiqué dans le tableau 9.2. Ceux-ci se comportent comme des opérateurs, mais ont une syntaxe spéciale imposée par la norme SQL.

Tableau 9.2. Prédicats de comparaison

Prédicat
Description Exemple(s)
type de données ENTRE type de données ET type de données → booléen Entre (y compris les extrémités de la plage). <code>2 ENTRE 1 ET 3 → t</code> <code>2 ENTRE 3 ET 1 → f</code>
type de données NOT BETWEEN type de données AND type de données → booléen Pas entre (la négation de BETWEEN). <code>2 NON COMPTE ENTRE 1 ET 3 → f</code>

Prédicat
Description Exemple(s)
type de données ENTRE SYMÉTRIQUE type de données ET type de données → booléen Entre, après avoir trié les deux valeurs d'extrémité. 2 ENTRE SYMÉTRIQUE 3 ET 1 → t
type de données NOT BETWEEN SYMMETRIC type de données ET type de données → booléen Pas entre, après avoir trié les deux valeurs d'extrémité. 2 NON ENTRE SYMÉTRIQUE 3 ET 1 → f
datatype IS DISTINCT FROM datatype → boolean Not equal, treating null as a comparable value. 1 EST DISTINCT DE NULL → t (plutôt que NULL) NULL EST DISTINCT DE NULL → f (plutôt que NULL)
datatype IS NOT DISTINCT FROM datatype → boolean Equal, traitant null comme une valeur comparable. 1 N'EST PAS DISTINCT DE NULL → f (plutôt que NULL) NULL N'EST PAS DISTINCT DE NULL → t (plutôt que NULL)
type de données IS NULL → booléen Teste si la valeur est nulle. 1,5 EST NUL → f
type de données N'EST PAS NULL → booléen Vérifier si la valeur n'est pas nulle. 'null' N'EST PAS NUL → t
type de données ISNULL → booléen Teste si la valeur est nulle (syntaxe non standard).
type de données NOTNULL → booléen Teste si la valeur n'est pas nulle (syntaxe non standard).
booléen IS TRUE → Testez si l'expression booléenne renvoie vrai. vrai EST VRAI → t NULL::boolean EST VRAI → f (plutôt que NULL)
booléen N'EST PAS VRAI → booléen Vérifier si l'expression booléenne renvoie faux ou inconnu. vrai N'EST PAS VRAI → f NULL::boolean N'EST PAS VRAI → t (plutôt que NULL)
booléen EST FAUX → Testez si l'expression booléenne renvoie faux. vrai EST FAUX → f NULL::boolean EST FAUX → f (plutôt que NULL)
booléen N'EST PAS FAUX → booléen Vérifier si l'expression booléenne renvoie vrai ou inconnu. VRAI N'EST PAS FAUX → t NULL::boolean N'EST PAS FAUX → t (plutôt que NULL)

Prédicat
Description Exemple(s)
booléen EST INCONNU → booléen Vérifier si l'expression booléenne renvoie une valeur inconnue. vrai EST INCONNU → f NULL::boolean EST INCONNU → t (plutôt que NULL)
booléen IS NOT UNKNOWN → Testez si l'expression booléenne renvoie vrai ou faux. vrai N'EST PAS INCONNU → t NULL::boolean N'EST PAS INCONNU → f (plutôt que NULL)

Le prédicat BETWEEN simplifie les tests de plage :

a ENTRE x ET y

est équivalent à

a >= x ET a <= y

Notez que BETWEEN considère les valeurs extrêmes comme incluses dans l'intervalle. BETWEEN SYMMETRIC est similaire à BETWEEN, à ceci près que l'argument à gauche de AND n'a pas besoin d'être inférieur ou égal à celui de droite. Dans le cas contraire, ces deux arguments sont automatiquement inversés, garantissant ainsi un intervalle non vide.

Les différentes variantes de BETWEEN sont implémentées à l'aide des opérateurs de comparaison ordinaires et fonctionneront donc pour tout type de données pouvant être comparé.

Note

L'utilisation de l'opérateur ET dans la syntaxe BETWEEN crée une ambiguïté quant à son utilisation en tant qu'opérateur logique. Pour résoudre ce problème, seuls certains types d'expressions sont autorisés comme second argument d'une clause BETWEEN. Si vous devez écrire une sous-expression plus complexe dans BETWEEN, placez-la entre parenthèses.

Les opérateurs de comparaison ordinaires renvoient null (signifiant « inconnu »), et non vrai ou faux, lorsqu'une des entrées est nulle. Par exemple, 7 = NULL renvoie null, de même que 7 <> NULL. Lorsque ce comportement ne convient pas, utilisez les prédicats IS [NOT] DISTINCT FROM.

a est distinct de b
 a n'est pas distinct de b

Pour les entrées non nulles, IS DISTINCT FROM se comporte comme l'opérateur <>. Cependant, si les deux entrées sont nulles, il renvoie faux, et si une seule entrée est nulle, il renvoie vrai. De même, IS NOT DISTINCT FROM est identique à = pour les entrées non nulles, mais il renvoie vrai lorsque les deux entrées sont nulles, et faux lorsqu'une seule entrée est nulle. Ainsi, ces prédicats se comportent comme si la valeur nulle était une donnée normale, et non une valeur « inconnue ».

Pour vérifier si une valeur est nulle ou non, utilisez les prédicats suivants :

L'expression est nulle.

L'expression n'est pas nulle.

ou les prédicats équivalents, mais non standard :

expression ISNULL expression
NOTNULL

N'écrivez pas l'expression = NULL car NULL n'est pas « égal à » NULL. (La valeur nulle représente une valeur inconnue, et on ne sait pas si deux valeurs inconnues sont égales.)

Conseil

Certaines applications peuvent s'attendre à ce que l'expression `x = NULL` renvoie `true` si l'expression est évaluée à `null`. Il est fortement recommandé de modifier ces applications afin qu'elles soient conformes à la norme SQL. Si cela s'avère impossible, la variable de configuration `transform\_null\_equals` est disponible. Si elle est activée, PostgreSQL convertira les clauses `x = NULL` en `x IS NULL`.

Si l'expression est de type ligne, alors IS NULL est vrai lorsque l'expression elle-même est nulle ou lorsque tous les champs de la ligne sont nuls, tandis que IS NOT NULL est vrai lorsque l'expression elle-même est non nulle et que tous les champs de la ligne sont non nuls. De ce fait, IS NULL et IS NOT NULL ne renvoient pas toujours des résultats inverses pour les expressions de type ligne ; en particulier, une expression de type ligne contenant à la fois des champs nuls et non nuls renverra faux pour les deux tests. Par exemple :

```
SELECT ROW(1,2.5,'ceci est un test') = ROW(1, 3, 'pas le même');
```

```
SELECT ROW(table.*) IS NULL FROM table; -- détecter toutes les lignes nulles
```

```
SELECT ROW(table.*) IS NOT NULL FROM table; -- détecter toutes les valeurs non nulles
```

rangées

```
SELECT NOT(ROW(table.*) IS NOT NULL) FROM TABLE; -- détecter au moins
```

une valeur nulle dans les lignes

Dans certains cas, il peut être préférable d'écrire ROW IS DISTINCT FROM NULL ou ROW IS NOT DISTINCT FROM NULL, ce qui vérifiera simplement si la valeur globale de la ligne est nulle sans effectuer de tests supplémentaires sur les champs de la ligne.

Les valeurs booléennes peuvent également être testées à l'aide des prédicats.

L'expression booléenne est VRAIE L'expression
booléenne n'est PAS VRAIE L'expression booléenne
est FAUSSE L'expression booléenne n'est PAS
FAUSSE L'expression booléenne est INCONNUE
L'expression booléenne n'est PAS INCONNUE

Ces fonctions renverront toujours vrai ou faux, jamais une valeur nulle, même si l'opérande est nul. Une entrée nulle est interprétée comme la valeur logique « inconnu ». Notez que IS UNKNOWN et IS NOT UNKNOWN sont respectivement équivalents à IS NULL et IS NOT NULL, à ceci près que l'expression d'entrée doit être de type booléen.

Certaines fonctions liées à la comparaison sont également disponibles, comme indiqué dans le tableau 9.3.

Tableau 9.3. Fonctions de comparaison

Fonction
Description Exemple(s)
<code>num_nonnulls(VARIADIC "any")</code> → entier Renvoie le nombre d'arguments non nuls. <code>num_nonnulls(1, NULL, 2)</code> <code>→ 2</code>
<code>num_nulls(VARIADIC "any")</code> → entier Renvoie le nombre d'arguments nuls. <code>num_nulls(1, NULL, 2)</code> → 1

9.3. Fonctions et opérateurs mathématiques

Des opérateurs mathématiques sont disponibles pour de nombreux types PostgreSQL. Pour les types sans conventions mathématiques standard (par exemple, les types date/heure), leur comportement est décrit dans les sections suivantes.

Le tableau 9.4 présente les opérateurs mathématiques disponibles pour les types numériques standard. Sauf indication contraire, les opérateurs indiqués comme acceptant un type numérique sont disponibles pour tous les types : `smallint`, `integer`, `bigint`, `numeric`, `real` et `double precision`. Les opérateurs indiqués comme acceptant un type intégral sont disponibles pour les types `smallint`, `integer` et `bigint`.

Sauf indication contraire, chaque forme d'un opérateur renvoie le même type de données que son ou ses arguments. Les appels impliquant plusieurs types de données d'arguments, tels que entier + numérique, sont résolus en utilisant le type apparaissant plus tard dans ces listes.

Tableau 9.4. Opérateurs mathématiques

Opérateur
Description Exemple(s)
<code>type_numérique + type_numérique</code> → <code>type_numérique</code> Ajout <code>2 + 3</code> → 5
<code>+ type_numérique</code> → <code>type_numérique</code> Unaire plus (pas d'opération) <code>+ 3,5</code> → 3,5
<code>type_numérique - type_numérique</code> → <code>type_numérique</code> Soustraction <code>2 - 3</code> → -1
<code>- type_numérique</code> → <code>type_numérique</code> Négation <code>- (-4)</code> → 4
<code>type_numérique * type_numérique</code> → <code>type_numérique</code> Multiplication <code>2 * 3</code> → 6
<code>type_numérique / type_numérique</code> → <code>type_numérique</code> Division (pour les types entiers, la division tronque le résultat vers zéro) <code>5,0 / 2</code> → 2,5000000000000000 <code>5 / 2</code> → 2

Opérateur
Description Exemple(s)
$(-5) / 2 \rightarrow -2$
$\text{type_numérique} \% \text{type_numérique} \rightarrow \text{type_numérique}$ Modulo (reste) ; disponible pour smallint, integer, bigint et numeric $5 \% 4 \rightarrow 1$
$\text{numérique} ^ \text{numérique} \rightarrow \text{numérique}$ $\text{double précision} ^ \text{double précision} \rightarrow \text{double précision}$ Exponentiation $2 ^ 3 \rightarrow 8$ Contrairement à la pratique mathématique habituelle, les occurrences multiples de ^ seront associées de gauche à droite par défaut : $2 ^ 3 ^ 3 \rightarrow 512$ $2 ^ (3 ^ 3) \rightarrow 134217728$
$\sqrt{\text{double précision}} \rightarrow \text{racine carrée double précision}$ $\sqrt{25,0} \rightarrow 5$
$\sqrt[3]{\text{double précision}} \rightarrow \text{double précision}$ Racine du cube $\sqrt[3]{64,0} \rightarrow 4$
$@ \text{type_numérique} \rightarrow \text{type_numérique}$ Valeur absolue $@ -5,0 \rightarrow 5,0$
$\text{type_intégral} \& \text{type_intégral} \rightarrow \text{type_intégral}$ ET bit à bit $91 \& 15 \rightarrow 11$
$\text{type_intégral} \text{type_intégral} \rightarrow \text{type_intégral}$ OU bit à bit $32 3 \rightarrow 35$
$\text{type_intégral} \# \text{type_intégral} \rightarrow \text{type_intégral}$ OU exclusif bit à bit $17 \# 5 \rightarrow 20$
$\sim \text{type_intégral} \rightarrow \text{type_intégral}$ NON bit à bit $\sim 1 \rightarrow -2$
$\text{type_intégral} \ll \text{entier} \rightarrow \text{type_intégral}$ Décalage bit à gauche $1 \ll 4 \rightarrow 16$
$\text{type_intégral} \gg \text{entier} \rightarrow \text{type_intégral}$ Décalage bit à bit vers la droite $8 \gg 2 \rightarrow 2$

Le tableau 9.5 présente les fonctions mathématiques disponibles. Nombre d'entre elles sont proposées sous plusieurs formes, avec différents types d'arguments. Sauf indication contraire, toute forme donnée d'une fonction renvoie le résultat.

Les données doivent être du même type que leurs arguments ; les cas de types différents sont traités de la même manière que pour les opérateurs. Les fonctions manipulant des données en double précision sont généralement implémentées dans la bibliothèque C du système hôte ; leur précision et leur comportement dans les cas limites peuvent donc varier selon le système hôte.

Tableau 9.5. Fonctions mathématiques

Fonction
Description Exemple(s)
<code>abs (type_numérique) → type_numérique</code> Valeur absolue <code>abs(-17,4) → 17,4</code>
<code>cbrt (double précision) → double précision</code> Racine du cube <code>cbrt(64.0) → 4</code>
<code>plafond (numérique) → numérique</code> <code>plafond (double précision) → double précision</code> Entier le plus proche supérieur ou égal à l'argument <code>plafond(42.2) → 43</code> <code>plafond(-42,8) → -42</code>
<code>plafond (numérique) → numérique</code> <code>plafond (double précision) → double précision</code> Entier supérieur ou égal à l'argument le plus proche (équivalent à <code>plafond</code>) <code>plafond(95,3) → 96</code>
<code>degrés (double précision) → double précision</code> Convertit les radians en degrés : <code>degrés(0,5) → 28,64788975654116</code>
<code>div (y numérique, x numérique) → numérique</code> Quotient entier de y/x (tronqué vers zéro) <code>div(9, 4) → 2</code>
<code>erf (double précision) → double précision</code> Fonction d'erreur <code>erf(1.0) → 0,8427007929497149</code>
<code>erfc (double précision) → double précision</code> Fonction d'erreur complémentaire (1 - erf(x), sans perte de précision pour les grandes entrées) <code>erfc(1.0) → 0,15729920705028513</code>
<code>exp (numérique) → numérique</code> <code>exp (double précision) → double précision</code> Exponentielle (e élevé à la puissance donnée) <code>exp(1,0) → 2,7182818284590452</code>
<code>factorielle (bigint) → numérique</code> Factorielle <code>factorielle(5) → 120</code>
<code>étage (numérique) → numérique</code>

Fonction
Description Exemple(s)
partie entière (double précision) → double précision Entier le plus proche inférieur ou égal à l'argument étage(42,8) → 42 étage(-42,8) → -43
pgcd(type_numérique, type_numérique) → type_numérique Plus grand commun diviseur (le plus grand nombre positif qui divise les deux entrées sans reste) ; renvoie 0 si les deux entrées sont nulles ; disponible pour les types entier, bigint et numérique. pgcd(1071, 462) → 21
lcm (type_numérique, type_numérique) → type_numérique Plus petit commun multiple (le plus petit nombre strictement positif qui soit un multiple entier des deux entrées) ; renvoie 0 si l'une des entrées est nulle ; disponible pour les types entier, bigint et numérique. lcm(1071, 462) → 23562
ln (numérique) → numérique ln (double précision) → double précision Logarithme népérien ln(2,0) → 0,6931471805599453
log (numérique) → numérique logarithme (double précision) → double précision logarithme en base 10 log(100) → 2
log10 (numérique) → numérique log10 (double précision) → double précision Logarithme en base 10 (identique à log) log10(1000) → 3
log (b numérique, x numérique) → numérique Logarithme de x en base b log(2.0, 64.0) → 6.000000000000000
min_scale (numérique) → entier Échelle minimale (nombre de décimales fractionnaires) nécessaire pour représenter précisément la valeur fournie min_scale(8.4100) → 2
mod (y type_numérique, x type_numérique) → type_numérique Reste de y/x ; disponible pour smallint, integer, bigint et numeric mod(9, 4) → 1
pi () → double précision Valeur approximative de π pi() → 3,141592653589793
puissance (a numérique, b numérique) → numérique puissance (a double précision, b double précision) → double précision

Fonction Description Exemple(s) a	
élevé à la puissance b puissance(9, 3) → 729	
radians (double précision) → double précision Convertit les degrés en radians radians(45.0) → 0.7853981633974483	
arrondi (numérique) → numérique arrondir (double précision) → double précision Arrondit à l'entier le plus proche. Pour les nombres numériques, en cas d'égalité, on arrondit à l'entier le plus proche. En double précision, le comportement de départage dépend de la plateforme, mais « arrondir au nombre pair le plus proche » est la règle la plus courante. arrondi(42,4) → 42	
arrondir (v numérique, s entier) → numérique Arrondit v à s décimales. En cas d'égalité, on arrondit à l'entier le plus proche de zéro. round(42.4382, 2) → 42.44 arrondi(1234,56, -1) → 1230	
échelle (numérique) → entier Échelle de l'argument (nombre de décimales de la partie fractionnaire) échelle(8,4100) → 4	
signe (numérique) → numérique signe (double précision) → double précision Signe de l'argument (-1, 0 ou +1) signe(-8,4) → -1	
racine carrée (numérique) → numérique racine carrée (double précision) → double précision Racine carrée $\sqrt{2}$ → 1,4142135623730951	
trim_scale (numérique) → numérique Réduit l'échelle de la valeur (nombre de décimales fractionnaires) en supprimant les zéros de fin. œufs trim_scale(8.4100) → 8.41	
tronquer (numérique) → numérique trunc (double précision) → double précision Tronque à l'entier (vers zéro) tronquer(42.8) → 42 troncature(-42,8) → -42	
tronquer (v numérique, s entier) → numérique Tronque v à s décimales tronquer(42.4382, 2) → 42.43	
largeur_bucket (opérande numérique, valeur numérique basse, valeur numérique haute, nombre entier) → entier	

Fonction
Description Exemple(s)
<code>width_bucket</code>(opérande double précision, valeur basse double précision, valeur haute double précision, nombre entier) → entier. Renvoie le numéro du compartiment dans lequel se trouve l'opérande dans un histogramme comportant nombre de compartiments de largeur égale, couvrant l'intervalle de valeurs basses à hautes. Les compartiments ont des bornes inférieures inclusives et des bornes supérieures exclusives. Renvoie 0 pour une entrée inférieure à basse, ou nombre + 1 pour une entrée supérieure ou égale à haute. Si basse > haute, le comportement est inversé : le compartiment 1 est alors celui juste en dessous de basse, et les bornes inclusives sont alors du côté supérieur. <code>width_bucket(5.35, 0.024, 10.06, 5) → 3 ; width_bucket(9, 10, 0, 10) → 2</code>
<code>largeur_bucket</code> (opérande anycompatible, seuils anycompatiblearray) → entier Renvoie le numéro du compartiment auquel appartient l'opérande, étant donné un tableau listant les bornes inférieures incluses des compartiments. Renvoie 0 pour une entrée inférieure à la première borne inférieure. L'opérande et les éléments du tableau peuvent être de n'importe quel type possédant des opérateurs de comparaison standard. Le tableau des seuils doit être trié par ordre décroissant, sous peine d'obtenir des résultats inattendus. <code>largeur_bucket(maintenant(), tableau['hier', 'aujourd'hui', 'demain-ligne']::timestampz[]) → 2</code>

Le tableau 9.6 présente des fonctions permettant de générer des nombres aléatoires.

Tableau 9.6. Fonctions aléatoires

Fonction
Description Exemple(s)
<code>random()</code> → double précision Renvoie une valeur aléatoire dans la plage $0.0 \leq x < 1.0$ <code>aléatoire()</code> → 0,897124072839091
<code>aléatoire</code> (entier min, entier max) → entier <code>random</code>(min bigint, max bigint) → bigint <code>random</code>(min numeric, max numeric) → numeric Renvoie une valeur aléatoire dans l'intervalle min ≤ x ≤ max. Pour le type numérique, le résultat aura le même nombre de décimales fractionnaires que min ou max, selon celui qui en possède le plus. plus. <code>random(1, 10) → 7</code> <code>aléatoire (-0,499, 0,499) → 0,347</code>
<code>random_normal</code>([moyenne double précision[, écart-type double précision]]) → double précision Renvoie une valeur aléatoire issue d'une distribution normale avec les paramètres donnés ; la moyenne est par défaut égale à 0,0 et l'écart type à 1,0. <code>random_normal(0.0, 1.0) → 0.051285419</code>
<code>setseed</code> (double précision) → void Définit la valeur initiale pour les appels suivants à <code>random()</code> et <code>random_normal()</code> ; l'argument doit être compris entre -1.0 et 1.0 inclus. <code>setseed(0.12345)</code>

Les fonctions `random()` et `random_normal()` listées dans le Tableau 9.6 utilisent un générateur de nombres pseudo-aléatoires déterministe. Ce générateur est rapide, mais inadapté aux applications cryptographiques ; le module `pgcrypt-to` propose une alternative plus sécurisée. Si `setseed()` est appelé, la série de résultats des appels suivants à ces fonctions dans la session en cours peut être reproduite en réappelant `setseed()` avec le même argument. Sans appel préalable à `setseed()` dans la même session, le premier appel à l'une de ces fonctions génère une graine à partir d'une source de bits aléatoires dépendant de la plateforme.

Le tableau 9.7 présente les fonctions trigonométriques disponibles. Chacune de ces fonctions existe en deux variantes : l'une mesurant les angles en radians et l'autre en degrés.

Tableau 9.7. Fonctions trigonométriques

Fonction
Description Exemple(s)
<code>acos</code> (double précision) → double précision Cosinus inverse, résultat en radians <code>acos(1) → 0</code>
<code>acosd</code> (double précision) → double précision Cosinus inverse, résultat en degrés <code>acosd(0.5) → 60</code>
<code>asin</code> (double précision) → double précision Sinus inverse, résultat en radians <code>asin(1) → 1.5707963267948966</code>
<code>asind</code> (double précision) → double précision Sinus inverse, résultat en degrés <code>asind(0,5) → 30</code>
<code>atan</code> (double précision) → tangente inverse en double précision, résultat en radians <code>atan(1) →</code> <code>0,7853981633974483</code>
<code>atand</code> (double précision) → double précision Inverse tangente, résultat en degrés <code>atand(1) → 45</code>
<code>atan2</code> (y double précision, x double précision) → double précision L'arc tangente de y/x donne le résultat en radians <code>atan2(1, 0) → 1,5707963267948966</code>
<code>atan2d</code> (y double précision, x double précision) → double précision L'arc tangente de y/x donne un résultat en degrés <code>atan2d(1, 0) → 90</code>
<code>cos</code> (double précision) → double précision Cosinus, argument en radians <code>cos(0) → 1</code>
<code>cosd</code> (double précision) → double précision Cosinus, argument en degrés <code>cosd(60) → 0,5</code>
<code>cot</code> (double précision) → double précision Cotangente, argument en radians

Fonction
Description Exemple(s)
$\cot(0,5) \rightarrow 1,830487721712452$
$\cotd$ (double précision) $\rightarrow$ double précision Cotangente, argument en degrés $\cotd(45) \rightarrow 1$
$\sinus$ (double précision) $\rightarrow$ double précision Sinus, argument en radians $\sin(1) \rightarrow 0,8414709848078965$
$\sind$ (double précision) $\rightarrow$ sinus en double précision, argument en degrés $\sind(30) \rightarrow 0,5$
$\tan$ (double précision) $\rightarrow$ double précision Tangente, argument en radians $\tan(1) \rightarrow 1,5574077246549023$
$\tand$ (double précision) $\rightarrow$ double précision Tangente, argument en degrés $\tand(45) \rightarrow 1$

Note

Une autre façon de travailler avec des angles mesurés en degrés consiste à utiliser les fonctions de transformation d'unité `radians()` et `degrees()` présentées précédemment. Cependant, l'utilisation des fonctions trigonométriques basées sur les degrés est préférable, car elle évite les erreurs d'arrondi dans des cas particuliers comme `sind(30)`.

Le tableau 9.8 présente les fonctions hyperboliques disponibles.

Tableau 9.8. Fonctions hyperboliques

Fonction
Description Exemple(s)
$\sinh$ (double précision) $\rightarrow$ double précision sinus hyperbolique $\sinh(1) \rightarrow 1.1752011936438014$
$\cosh$ (double précision) $\rightarrow$ double précision Cosinus hyperbolique $\cosh(0) \rightarrow 1$
$\tanh$ (double précision) $\rightarrow$ double précision Tangente hyperbolique $\tanh(1) \rightarrow 0,7615941559557649$
asinh (double précision) $\rightarrow$ sinus hyperbolique inverse en double précision $\operatorname{asinh}(1) \rightarrow 0,881373587019543$

Fonction
Description Exemple(s)
acosh (double précision) → double précision cosinus hyperbolique inverse acosh(1) → 0
atanh (double précision) → double précision Tangente hyperbolique inverse atanh(0,5) → 0,5493061443340548

9.4. Fonctions et opérateurs de chaînes de caractères

Cette section décrit les fonctions et les opérateurs permettant d'examiner et de manipuler les chaînes de caractères. Dans ce contexte, les chaînes de caractères peuvent être de type caractère, caractère variable ou texte. Sauf indication contraire, ces fonctions et opérateurs acceptent et renvoient des valeurs de type texte. Ils acceptent également les arguments de type caractère variable. Les valeurs de type caractère sont converties en texte avant l'application de la fonction ou de l'opérateur, ce qui supprime les espaces de fin.

SQL définit certaines fonctions de chaînes de caractères qui utilisent des mots clés, plutôt que des virgules, pour séparer les arguments. Les détails se trouvent dans le tableau 9.9. PostgreSQL fournit également des versions de ces fonctions qui utilisent la syntaxe d'appel de fonction régulière (voir tableau 9.10).

Note

L'opérateur de concaténation de chaînes (||) acceptera une entrée non-chaîne, à condition qu'au moins une entrée soit de type chaîne, comme indiqué dans le tableau 9.9. Dans les autres cas, l'insertion d'une conversion explicite en texte peut être utilisée pour que l'entrée non-chaîne soit acceptée.

Tableau 9.9. Fonctions et opérateurs de chaînes SQL

Fonction/Opérateur
Description Exemple(s)
texte texte → texte Concatène les deux chaînes de caractères. 'Post' 'greSQL' → PostgreSQL
texte anynonarray → texte La fonction anynonarray text → text convertit l'entrée non-chaîne de caractères en texte, puis concatène les deux chaînes. (L'entrée non-chaîne de caractères ne peut pas être de type tableau, car cela créerait une ambiguïté avec les opérateurs sur les tableaux. Si vous souhaitez concaténer l'équivalent textuel d'un tableau, convertissez-le explicitement en texte.) 'Valeur : ' 42 → Valeur : 42
btrim (chaîne texte [, caractères texte]) → texte Supprime la chaîne la plus longue contenant uniquement des caractères dans caractères (un espace par défaut) du début et de la fin de la chaîne. btrim('xyxtrimyyx', 'xyz') → supprimer
Le texte n'est pas normalisé → booléen

Exemple(s) de description de fonction/opérateur
Vérifie si la chaîne est au format de normalisation Unicode spécifié. Le mot-clé optionnel `form` spécifie le format : NFC (par défaut), NFD, NFKC ou NFKD. Cette expression ne peut être utilisée que si l'encodage du serveur est UTF-8. Notez que la vérification de la normalisation à l'aide de cette expression est souvent plus rapide que la normalisation de chaînes potentiellement déjà normalisées. U&'\0061\0308bc' EST NORMALISÉ NFD → t
bit_length(texte) → entier Renvoie le nombre de bits dans la chaîne (8 fois la longueur de l'octet). bit_length('jose') → 32
longueur_caractère(texte) → entier longueur_caractère(texte) → entier Renvoie le nombre de caractères dans la chaîne. char_length('josé') → 4
lower (text) → text Convertit la chaîne en minuscules, conformément aux règles de la langue de la base de données. lower('TOM') → tom
lpad(string text, length integer [, fill text]) → text Étend la chaîne jusqu'à la longueur length en ajoutant les caractères fill (un espace par défaut) au début. Si la chaîne est déjà plus longue que length, elle est tronquée (à droite). lpad('salut', 5, 'xy') → xyxhi
ltrim(string text[, characters text]) → text Supprime la plus longue chaîne de caractères ne contenant que des caractères (un espace par défaut) au début de la chaîne. ltrim('zzzytest', 'xyz') → test
normaliser(texte[, formulaire]) → texte Convertit la chaîne au format de normalisation Unicode spécifié. Le mot-clé optionnel `form` spécifie le format : NFC (par défaut), NFD, NFKC ou NFKD. Cette fonction ne peut être utilisée que si l'encodage du serveur est UTF-8. normaliser(U&'\0061\0308bc', NFC) → U&'\00E4bc'
longueur_octet (texte) → entier Renvoie le nombre d'octets dans la chaîne. octet_length('josé') → 5 (si l'encodage du serveur est UTF-8)
octet_length(character) → entier Renvoie le nombre d'octets dans la chaîne. Cette version de la fonction acceptant directement le type caractère, elle ne supprime pas les espaces de fin. octet_length('abc :::character(4)) → 4
superposition (chaîne texte PLACING nouvelle sous-chaîne texte FROM entier de départ [POUR entier de comptage]) → texte Remplace la sous-chaîne de la chaîne commençant au caractère de départ et s'étendant sur une longueur de nombre de caractères par nouvelle sous-chaîne. Si le nombre de caractères est omis, sa longueur est par défaut celle de nouvelle sous-chaîne. superposition('Txxxxas' plaçant 'hom' de 2 pour 4) → Thomas
position (sous-chaîne texte DANS chaîne texte) → entier

Exemple(s) de description de fonction/opérateur
Renvoie le premier indice de début de la sous-chaîne spécifiée dans la chaîne, ou zéro si elle n'est pas présente. position('om' dans 'Thomas') → 3
rpads(string text, length integer [, fill text]) → text Étend la chaîne de caractères jusqu'à la longueur 'length' en ajoutant les caractères 'fill' (un espace par défaut). Si la chaîne est déjà plus longue que 'length', elle est tronquée. rpads('salut', 5, 'xy') → hixyx
rtrim (chaîne texte [, caractères texte]) → texte Supprime la chaîne la plus longue contenant uniquement des caractères dans caractères (un espace par défaut) à la fin de la chaîne. rtrim('testxxxz', 'xyz') → test
sous-chaîne (chaîne texte [DE l'entier de début] [POUR l'entier de comptage]) → texte Extrait la sous-chaîne de la chaîne commençant au caractère de départ spécifié (si celui-ci est indiqué) et s'arrêtant après le nombre de caractères spécifié (si celui-ci est indiqué). Fournissez au moins un des arguments de départ et de nombre. sous-chaîne('Thomas' de 2 pour 3) → hom sous-chaîne('Thomas' de 3) → omas sous-chaîne('Thomas' pour 2) → Th
sous-chaîne (chaîne texte FROM modèle texte) → texte Extrait la première sous-chaîne correspondant à l'expression régulière POSIX ; voir la section 9.7.3. sous-chaîne('Thomas' de '...\$') → mas
sous-chaîne (chaîne texte SIMILAIRE motif texte ÉCHAPPER texte d'échappement) → texte sous-chaîne (chaîne texte DE motif texte POUR texte d'échappement) → texte Extrait la première sous-chaîne correspondant à l'expression régulière SQL ; voir la section 9.7.2. La première forme est spécifiée depuis SQL:2003 ; la seconde forme n'était présente que dans SQL:1999 et doit être considérée comme obsolète. substring('Thomas' similaire '%"o_a#" ' escape '#') → oma
supprimer ([LEADING TRAILING BOTH] [caractères texte] DE la chaîne texte) → texte Supprime la chaîne la plus longue contenant uniquement des caractères entre parenthèses (un espace par défaut) au début, à la fin ou aux deux extrémités (LES DEUX sont la valeur par défaut) de la chaîne. supprimer(les deux 'xyz' de 'yxTomxx') → Tom
supprimer ([DÉBUT FIN LES DEUX] [DE] texte chaîne [, caractères texte]) → texte Il s'agit d'une syntaxe non standard pour trim(). supprimer(tous deux de 'yxTomxx', 'xyz') → Tom
unicode_assigned (texte) → booléen Renvoie vrai si tous les caractères de la chaîne ont un point de code Unicode ; faux sinon. Cette fonction ne peut être utilisée que si l'encodage du serveur est UTF-8.
majuscule (texte) → texte Convertit la chaîne en majuscules, conformément aux règles de la langue de la base de données. upper('tom') → TOM

Des fonctions et opérateurs de manipulation de chaînes supplémentaires sont disponibles et sont répertoriés dans le tableau 9.10. (Certaines d'entre elles sont utilisées en interne pour implémenter les fonctions de chaînes de caractères standard SQL répertoriées dans le tableau 9.9.) Il existe également des opérateurs de correspondance de modèles, qui sont décrits dans la section 9.7, et des opérateurs de recherche en texte intégral, qui sont décrits au chapitre 12.

Tableau 9.10. Autres fonctions et opérateurs de chaînes de caractères

Fonction/Opérateur	Description	Exemple(s)
<code>text ^@ text</code>	→ boolean Renvoie vrai si la première chaîne commence par la seconde (équivalent à la fonction <code>start-s_with()</code>). <code>'alphabet' ^@ 'alph'</code> → t	
<code>ascii(texte)</code>	→ entier : Renvoie le code numérique du premier caractère de l'argument. En encodage UTF-8, renvoie le point de code Unicode du caractère. Dans les autres encodages multioctets, l'argument doit être un caractère ASCII.	<code>ascii('x')</code> → 120
<code>`chr(integer)</code>	→ text Renvoie le caractère correspondant au code donné. En UTF-8, l'argument est interprété comme un point de code Unicode. Dans les autres encodages multioctets, l'argument doit désigner un caractère ASCII. <code>`chr(0)`</code> est interdit car les types de données texte ne peuvent pas stocker ce caractère.	<code>chr(65)</code> → A
<code>`concat(val1 "any" [, val2 "any" [, ...]])</code>	→ text Concatène les représentations textuelles de tous les arguments. Les arguments NULL sont ignorés. <code>`concat('abcde', 2, NULL, 22)</code> → abcde222`	
<code>concat_ws (sep text, val1 "any" [, val2 "any" [, ...]])</code>	→ texte Concatène tous les arguments sauf le premier, en les séparant. Le premier argument sert de séparateur et ne doit pas être NULL. Les arguments NULL sont ignorés. <code>`concat_ws(',', 'abcde', 2, NULL, 22)</code> → abcde,2,22`	
<code>format (formatstr text [, formatarg "any" [, ...]])</code>	→ text Formate les arguments selon une chaîne de format ; voir la section 9.4.1. Cette fonction est similaire à la fonction C <code>sprintf</code> .	<code>format('Bonjour %s, %1\$s', 'Monde')</code> → Bonjour Monde, Monde
<code>initcap(texte)</code>	→ texte Convertit la première lettre de chaque mot en majuscule et le reste en minuscule. Les mots sont des séquences de caractères alphanumériques séparées par des caractères non alphanumériques.	<code>initcap('hi THOMAS')</code> → Salut Thomas
<code>gauche (texte chaîne, n entier)</code>	→ texte Renvoie les n premiers caractères de la chaîne, ou lorsque n est négatif, renvoie tous les caractères sauf les n derniers.	<code>gauche('abcde', 2)</code> → ab
<code>longueur (texte)</code>	→ entier Renvoie le nombre de caractères dans la chaîne.	<code>longueur('jose')</code> → 4
<code>MD5 (texte)</code>	→ texte Calcule le hachage MD5 de l'argument, le résultat étant écrit en hexadécimal.	

Fonction/Opérateur
Description
Exemple(s)
MD5('abc') → 900150983cd24fb0d6963f7d28e17f72
analyser_ident (identifiant_qualifié texte [, mode_strict booléen DEFAULT true]) → texte[] Divise qualified_identifiant en un tableau d'identifiants, en supprimant les guillemets autour de chaque identifiant. Par défaut, les caractères supplémentaires après le dernier identifiant sont considérés comme une erreur ; si le deuxième paramètre est faux, ces caractères supplémentaires sont ignorés. (Ce comportement est utile pour analyser les noms d'objets tels que les fonctions.) Notez que cette fonction ne tronque pas les identifiants trop longs. Si vous souhaitez une troncature, vous pouvez convertir le résultat en `name[]`. `parse_ident('"'SomeSchema".someTable')` → {SomeSchema,sometable}`
pg_client_encoding() → nom Renvoie le nom de l'encodage client actuel. pg_client_encoding() → UTF8
quote_ident(texte) → texte : Renvoie la chaîne de caractères fournie, correctement encadrée par des guillemets, pour être utilisée comme identifiant dans une requête SQL. Les guillemets ne sont ajoutés que si nécessaire (c'est-à-dire si la chaîne contient des caractères non identifiants ou si elle est convertie en minuscules). Les guillemets imbriqués sont doublés. Voir également l'exemple 41.1 : quote_ident('Foo bar') → "Foo bar"
La fonction `quote_literal(text) → text` renvoie la chaîne de caractères fournie, correctement encadrée par des guillemets pour être utilisée comme littéral dans une requête SQL. Les guillemets simples et les barres obliques inverses sont doublés. Notez que `quote_literal` renvoie `null` si l'argument est nul ; si celui-ci peut être nul, `quote_nullable` est souvent plus approprié. Voir également l'exemple 41.1 : `quote_literal(E'O'Reilly')` → 'O"Reilly"
quote_literal (anyelement) → texte Convertit la valeur donnée en texte, puis la met entre guillemets. Les guillemets simples et les barres obliques inverses sont correctement doublés. quote_literal(42.5) → '42.5'
quote_nullable(text) → text Renvoie la chaîne de caractères fournie correctement encadrée par des guillemets pour être utilisée comme littéral dans une requête SQL ; ou, si l'argument est nul, renvoie NULL. Les guillemets simples et les barres obliques inverses sont correctement doublés. Voir aussi l'exemple 41.1. quote_nullable(NULL) → NULL
quote_nullable (tout élément) → texte Convertit la valeur donnée en texte, puis la place entre guillemets comme une valeur littérale ; ou, si l'argument est nul, renvoie NULL. Les guillemets simples et les barres obliques inverses sont correctement doublés. quote_nullable(42.5) → '42.5'
regexp_count (chaîne texte, motif texte [, entier de départ [, texte des indicateurs]]) → entier Renvoie le nombre de fois où le modèle d'expression régulière POSIX correspond dans la chaîne ; voir la section 9.7.3. regexp_count('123456789012', '\d\d\d', 2) → 3

Fonction/Opérateur
Description
Exemple(s)
<code>regex_instr (chaîne texte, motif texte [, entier de départ [, N entier [, endoption entier [, flags texte [, subexpr entier]]]])</code> → entier Renvoie la position dans la chaîne où se trouve la Nième occurrence du modèle d'expression régulière POSIX, ou zéro s'il n'y a pas d'occurrence ; voir la section 9.7.3. <code>regex_instr('ABCDEF', 'c(..)', 1, 1, 0, 'i') → 3</code> <code>regex_instr('ABCDEF', 'c(..)', 1, 1, 0, 'i', 2) → 5</code>
<code>regex_like (chaîne texte, motif texte [, drapeaux texte])</code> → booléen Vérifie si une correspondance avec le modèle d'expression régulière POSIX se trouve dans la chaîne ; voir la section 9.7.3. <code>regex_like('Hello World', 'world\$', 'i') → t</code>
<code>regex_match (chaîne texte, motif texte [, drapeaux texte])</code> → texte[] Renvoie les sous-chaînes correspondant à la première occurrence du modèle d'expression régulière POSIX dans la chaîne ; voir la section 9.7.3. <code>regex_match('foobarbequebaz', '(bar)(beque)') → {bar,beque}</code>
<code>regex_matches (chaîne texte, motif texte [, drapeaux texte])</code> → ensemble de texte[] Renvoie les sous-chaînes de la première correspondance du modèle d'expression régulière POSIX à la chaîne, ou les sous-chaînes de toutes ces correspondances si l'indicateur g est utilisé ; voir la section 9.7.3. <code>regex_matches('foobarbequebaz', 'ba.', 'g') →</code> <code>{bar}</code> <code>{baz}</code>
<code>regex_replace(string text, pattern text, remplacement text [, start in-eger] [, flags text])</code> → text Remplace la sous-chaîne qui correspond en premier au modèle d'expression régulière POSIX, ou toutes les correspondances si l'option g est utilisée ; voir la section 9.7.3. <code>regex_replace('Thomas', '[mN]a.', 'M') → ThM</code>
<code>regex_replace (texte de chaîne, texte de modèle, texte de remplacement, interface de démarrage)</code> <code>ger, N entier [, flags texte])</code> → texte Remplace la sous-chaîne qui correspond à la Nème correspondance au modèle d'expression régulière POSIX, ou toutes ces correspondances si N est zéro ; voir la section 9.7.3. <code>regex_replace('Thomas', '.', 'X', 3, 2) → ThoXas</code>
<code>regex_split_to_array (string text, pattern text [, flags text])</code> → texte[] Divise une chaîne de caractères en utilisant une expression régulière POSIX comme délimiteur, produisant un tableau de résultats ; voir la section 9.7.3. <code>regex_split_to_array('hello world', '\s+') → {hello,world}</code>
<code>regex_split_to_table (chaîne texte, motif texte [, drapeaux texte])</code> → ensemble de texte Divise une chaîne en utilisant une expression régulière POSIX comme délimiteur, produisant un ensemble de résultats ; voir la section 9.7.3. <code>regex_split_to_table('hello world', '\s+') →</code>

Fonction/Opérateur
Description
Exemple(s)
Bonjour monde
<code>regex_substr (chaîne texte, motif texte [, entier de départ [, N entier [, flags text [, subexpr integer]]]) → text</code> Renvoie la sous-chaîne de la chaîne correspondant à la Nième occurrence du modèle d'expression régulière POSIX, ou NULL en l'absence de correspondance ; voir la section 9.7.3. <code>regex_substr('ABCDEF', 'c(..)', 1, 1, 'i') → CDEF</code> <code>regex_substr('ABCDEF', 'c(..)', 1, 1, 'i', 2) → EF</code>
<code>repeat (string text, number integer) → text</code> Répète la chaîne le nombre de fois spécifié. <code>répéter('Pg', 4) → PgPgPgPg</code>
<code>replace (chaîne texte, de texte, à texte) → texte</code> Remplace toutes les occurrences dans la chaîne de sous-chaîne de de par sous-chaîne à. <code>remplacer('abcdefabcdef', 'cd', 'XX') → abXXefabXXef</code>
<code>reverse (text) → text</code> Inverse l'ordre des caractères dans la chaîne. <code>reverse('abcde') → edcba</code>
<code>droite (texte chaîne, n entier) → texte</code> Renvoie les n derniers caractères de la chaîne, ou lorsque n est négatif, renvoie tous les caractères sauf les n premiers. <code>droite('abcde', 2) → de</code>
<code>split_part (texte chaîne, texte délimiteur, n entier) → texte</code> Sépare la chaîne aux occurrences du délimiteur et renvoie le n-ième champ (en comptant à partir de 1), ou, si n est négatif, le n -ième champ en partant de la fin. <code>split_part('abc~@~def~@~ghi', '~@~', 2) → def</code> <code>split_part('abc,def,ghi,jkl', ',', -2) → ghi</code>
<code>commence_par (texte chaîne, texte préfixe) → booléen</code> Renvoie vrai si la chaîne commence par le préfixe. <code>starts_with('alphabet', 'alph') → t</code>
<code>string_to_array (string text, delimiter text [, null_string text]) → texte[]</code> Divise la chaîne aux occurrences du délimiteur et transforme les champs résultants en un tableau de texte. Si le délimiteur est NULL, chaque caractère de la chaîne devient un élément distinct du tableau. Si le délimiteur est une chaîne vide, la chaîne est traitée comme un seul champ. Si <code>null_string</code> est fourni et n'est pas NULL, les champs correspondants sont remplacés par NULL. Voir aussi <code>array_to_string</code>. <code>string_to_array('xx~yy~zz', '~', 'yy') → {xx,NULL,zz}</code>
<code>chaîne_en_tableau (texte de chaîne, texte de délimiteur [, texte de chaîne nulle]) → ensemble de texte</code> Cette fonction divise la chaîne de caractères aux occurrences du délimiteur et renvoie les champs résultants sous forme de lignes de texte. Si le délimiteur est NULL, chaque caractère de la chaîne constitue une ligne distincte. Si le délimiteur est une chaîne vide, celle-ci est traitée comme un seul champ. Si une chaîne nulle est fournie et n'est pas NULL, les champs correspondants sont remplacés par NULL.

240

Exemple(s) de description de fonction/opérateur
Remplace chaque caractère de la chaîne correspondant à un caractère de l'ensemble « from » par le caractère correspondant de l'ensemble « to ». Si « from » est plus long que « to », les caractères supplémentaires présents dans « from » sont supprimés. translate('12345', '143', 'ax') → a2x5
<code>`unistr(text) → text`</code> : Évalue les caractères Unicode échappés dans l'argument. Les caractères Unicode peuvent être spécifiés comme suit : <code>`\XXXX`</code> (4 chiffres hexadécimaux), <code>`\+XXXXXX`</code> (6 chiffres hexadécimaux), <code>`\uXXXX`</code> (4 chiffres hexadécimaux) ou <code>`\UXXXXXXXX`</code> (8 chiffres hexadécimaux). Pour spécifier une barre oblique inverse, saisissez-en deux. Tous les autres caractères sont interprétés littéralement. Si l'encodage du serveur n'est pas UTF-8, le point de code Unicode identifié par l'une de ces séquences d'échappement est converti en l'encodage réel du serveur ; une erreur est signalée si cela s'avère impossible. Cette fonction fournit une alternative (non standard) aux constantes de chaîne avec des caractères d'échappement Unicode (voir section 4.1.2.3). <code>unistr('d\0061t\+000061') → données</code> <code>unistr('d\u0061t\U00000061') → données</code>

Les fonctions `concat`, `concat_ws` et `format` sont variadiques, il est donc possible de passer les valeurs à concaténer ou à formater sous forme de tableau marqué avec le mot-clé `VARIADIC` (voir section 36.5.6).

Les éléments du tableau sont traités comme s'il s'agissait d'arguments ordinaires distincts de la fonction. Si l'argument de tableau variadique est `NULL`, `concat` et `concat_ws` renvoient `NULL`, mais `format` traite `NULL` comme un tableau vide.

Voir également la fonction d'agrégation `string_agg` dans la section 9.21 et les fonctions de conversion entre les chaînes et le type `bytea` dans le tableau 9.13.

9.4.1. Format

La fonction `format` produit une sortie formatée selon une chaîne de format, dans un style similaire à la fonction C `sprintf`.

`format(formatstr texte [, formatarg "any" [, ...] ])`

``formatstr`` est une chaîne de formatage qui spécifie la mise en forme du résultat. Le texte de cette chaîne est copié directement dans le résultat, sauf lorsque des spécificateurs de format sont utilisés. Ces spécificateurs servent d'espaces réservés dans la chaîne, définissant comment les arguments de fonction suivants doivent être formatés et insérés dans le résultat. Chaque argument ``formatarg`` est converti en texte selon les règles de sortie habituelles de son type de données, puis formaté et inséré dans la chaîne de résultat conformément au(x) spécificateur(s) de format.

Les spécificateurs de format sont introduits par le caractère `%` et ont la forme suivante :

`%[position][flags][width]type`

où les champs composants sont :

position (facultatif)

Une chaîne de caractères au format `n$` où `n` est l'indice de l'argument à afficher. L'indice 1 correspond au premier argument après `formatstr`. Si la position est omise, l'argument suivant est utilisé par défaut.

drapeaux (facultatif)

Options supplémentaires permettant de contrôler la mise en forme de la sortie du spécificateur de format.

Actuellement, seul le signe moins (-) est pris en charge ; il permet d'aligner la sortie du spécificateur de format à gauche.

Cela n'a aucun effet sauf si le champ de largeur est également spécifié.

largeur (facultatif)

Spécifie le nombre minimal de caractères à utiliser pour afficher la sortie du spécificateur de format. La sortie est complétée à gauche ou à droite (selon l'option -) par des espaces si nécessaire pour atteindre la largeur requise. Une largeur insuffisante n'entraîne pas de troncature de la sortie, mais est simplement ignorée. La largeur peut être spécifiée de l'une des manières suivantes : un entier positif ; un astérisque (*) pour utiliser le prochain argument de la fonction comme largeur ; ou une chaîne de caractères de la forme *n\$ pour utiliser le n-ième argument de la fonction comme largeur.

Si la largeur provient d'un argument de fonction, cet argument est utilisé avant celui servant à spécifier le format.

Si la largeur est négative, le résultat est aligné à gauche (comme si l'option '-' avait été spécifiée) dans un champ de longueur ``abs(width)``.

type (obligatoire)

Type de conversion de format à utiliser pour produire la sortie du spécificateur de format. Les types suivants sont pris en charge :

- La valeur de l'argument est formatée sous forme de chaîne de caractères simple. Une valeur nulle est traitée comme une chaîne vide.
- Je traite la valeur de l'argument comme un identifiant SQL, en la mettant entre guillemets si nécessaire. C'est une erreur. pour que la valeur soit nulle (équivalent à `quote_ident`).
- L'interprète la valeur de l'argument comme une chaîne littérale SQL. Une valeur nulle est affichée sous la forme de la chaîne NULL. sans guillemets (équivalent à `quote_nullable`).

En plus des spécificateurs de format décrits ci-dessus, la séquence spéciale %% peut être utilisée pour afficher un caractère % littéral.

Voici quelques exemples de conversions de format de base :

```
SELECT format('Bonjour %s', 'Monde');
```

Résultat : Bonjour le monde

```
SELECT format('Test %s, %s, %s, %%', 'un', 'deux', 'trois');
```

Résultat : Test un, deux, trois, %

```
SELECT format('INSERT INTO %I VALUES(%L)', 'Foo bar', E'O
'Reilly');
```

Résultat : INSERT INTO "Foo bar" VALUES('O"Reilly')

```
SELECT format('INSERT INTO %I VALUES(%L)', 'locations', 'C:\Program
Fichiers');
```

Résultat : INSERT INTO locations VALUES('C:\Program Files')

Voici des exemples utilisant les champs de largeur et l'indicateur - :

```
SELECT format('|%10s|', 'foo');
```

Résultat : | foo|

```
SELECT format('|%-10s|', 'foo');
```

Résultat : |foo |

```
SELECT format('|%*s|', 10, 'foo');
```

Résultat : | foo|

```
SELECT format('%s', -10, 'foo');
```

Résultat : |foo |

```
SELECT format('%*s', 10, 'foo');
```

Résultat : |foo |

```
SELECT format('%*s', -10, 'foo');
```

Résultat : |foo |

Ces exemples illustrent l'utilisation des champs de position :

```
SELECT format('Test %3$s, %2$s, %1$s', 'un', 'deux', 'trois');
```

Résultat : Test trois, deux, un

```
SELECT format('%*2$s', 'foo', 10, 'bar');
```

Résultat : | bar|

```
SELECT format('%1$*2$s', 'foo', 10, 'bar');
```

Résultat : | foo|

Contrairement à la fonction C standard `sprintf`, la fonction `format` de PostgreSQL permet de combiner des spécificateurs de format avec et sans champ de position dans une même chaîne de format. Un spécificateur de format sans champ de position utilise toujours l'argument suivant le dernier argument consommé. De plus, la fonction `format` n'exige pas que tous ses arguments soient utilisés dans la chaîne de format. Par exemple :

```
SELECT format('Test %3$s, %2$s, %s', 'un', 'deux', 'trois');
```

Résultat : Test trois, deux, trois

Les spécificateurs de format `%l` et `%L` sont particulièrement utiles pour construire en toute sécurité des instructions SQL dynamiques. Voir l'exemple 41.1.

9.5. Fonctions et opérateurs de chaînes binaires

Cette section décrit les fonctions et les opérateurs permettant d'examiner et de manipuler les chaînes binaires, c'est-à-dire les valeurs de type `bytea`. Nombre d'entre eux sont équivalents, par leur finalité et leur syntaxe, aux fonctions de manipulation de chaînes de caractères décrites dans la section précédente.

SQL définit certaines fonctions de chaînes de caractères qui utilisent des mots clés, plutôt que des virgules, pour séparer les arguments. Les détails se trouvent dans le tableau 9.11. PostgreSQL fournit également des versions de ces fonctions qui utilisent la syntaxe d'appel de fonction régulière (voir tableau 9.12).

Tableau 9.11. Fonctions et opérateurs de chaînes binaires SQL

Fonction/Opérateur	Description	Exemple(s)
<code>bytea bytea</code>	Concaténe les deux chaînes binaires.	<code>'\x123456'::bytea '\x789a00bcde'::bytea</code> → <code>\x123456789a00bcde</code>
<code>longueur_bit (octet)</code>	→ entier Renvoie le nombre de bits dans la chaîne binaire (8 fois la longueur de l'octet).	<code>bit_length('\x123456'::bytea)</code> → 24

Fonction/Opérateur
Description
Exemple(s)
btrim (octets bytea, bytesremoved bytea) → bytea Supprime la plus longue chaîne contenant uniquement des octets apparaissant dans bytesremoved au début et à la fin de bytes. btrim("\x1234567890'::bytea, '\x9012'::bytea) → \x345678
ltrim (octets bytea, bytesremoved bytea) → bytea Supprime la chaîne la plus longue contenant uniquement des octets apparaissant dans bytesremoved à partir du début de bytes. ltrim("\x1234567890'::bytea, '\x9012'::bytea) → \x34567890
longueur_octet (octet) → entier Renvoie le nombre d'octets dans la chaîne binaire. `octet_length`("\x123456'::bytea) → 3`
overlay (bytes bytea PLACING newsubstring bytea FROM start integer [FOR count integer]) → bytea` Remplace la sous-chaîne d'octets commençant à l'octet de départ et s'étendant sur `count` octets par `newsubstring`. Si `count` est omis, sa valeur par défaut est la longueur de `newsubstring`. `overlay`("\x1234567890'::bytea placed '\002\003'::bytea` de 2 pour 3) → \x12020390
position (substring bytea IN bytes bytea) → integer Renvoie le premier indice de début de la sous-chaîne spécifiée dans bytes, ou zéro si elle est absente. position("\x5678'::bytea in '\x1234567890'::bytea) → 3
rtrim (octets bytea, bytesremoved bytea) → bytea Supprime la chaîne la plus longue contenant uniquement des octets apparaissant dans bytesremoved à la fin de bytes. rtrim("\x1234567890'::bytea, '\x9012'::bytea) → \x12345678
substring (octets octet [DE l'entier de début] [POUR l'entier de comptage]) → La fonction `bytea` extrait la sous-chaîne d'octets commençant à l'octet de départ (si spécifié) et s'arrêtant après le nombre d'octets spécifié (si spécifié). Fournissez au moins un des arguments `start` et `count`. substring("\x1234567890'::bytea from 3 for 2) → \x5678
supprimer ([LEADING TRAILING BOTH] octets supprimés octets FROM octets octets) → La fonction `bytea` supprime la plus longue chaîne contenant uniquement des octets présents dans `bytesremoved` au début, à la fin ou aux deux extrémités (les deux étant la valeur par défaut) de `bytes`. `trim`("\x9012'::bytea from '\x1234567890'::bytea) → \x345678`
supprimer ([LEADING TRAILING BOTH] [FROM] bytes bytea, bytesremoved bytea) → bytea Ceci est une syntaxe non standard pour trim(). supprimer(tous deux de '\x1234567890'::bytea, '\x9012'::bytea) → \x345678

Des fonctions supplémentaires de manipulation de chaînes binaires sont disponibles et sont répertoriées dans le tableau 9.12. Certaines d'entre elles sont utilisées en interne pour implémenter les fonctions de chaînes standard SQL répertoriées dans le tableau 9.11.

Tableau 9.12. Autres fonctions de chaînes binaires

Fonction	Description	Exemple(s)
<code>bit_count(bytes bytea) → bigint</code>	Renvoie le nombre de bits à 1 dans la chaîne binaire (également appelé « popcount »).	<code>bit_count('\x1234567890'::bytea) → 15</code>
<code>get_bit(bytes bytea, n bigint) → entier</code>	Extrait le n-ième bit d'une chaîne binaire.	<code>get_bit('\x1234567890'::bytea, 30) → 1</code>
<code>get_byte(bytes bytea, n entier) → entier</code>	Extrait le n-ième octet d'une chaîne binaire.	<code>get_byte('\x1234567890'::bytea, 4) → 144</code>
<code>longueur (octet) → entier</code>	Renvoie le nombre d'octets dans la chaîne binaire.	<code>longueur('\x1234567890'::bytea) → 5</code>
<code>longueur (octets octet, nom d'encodage) → entier</code>	Renvoie le nombre de caractères dans la chaîne binaire, en supposant qu'il s'agit de texte dans l'encodage donné.	<code>longueur('jose'::bytea, 'UTF8') → 4</code>
<code>md5 (octet) → texte</code>	Calcule le hachage MD5 de la chaîne binaire et affiche le résultat en hexadécimal.	<code>md5('Th000omas'::bytea) → 8ab2d3c9689aaf18b4958c334c82d8b1</code>
<code>set_bit(bytes bytea, n bigint, newvalue integer) → bytea</code>	Définit le n-ième bit de la chaîne binaire à la valeur newvalue.	<code>set_bit('\x1234567890'::bytea, 30, 0) → \x1234563890</code>
<code>`set_byte(bytes bytea, n integer, newvalue integer) → bytea`</code>	Affecte la valeur newvalue au n-ième octet de la chaîne binaire.	<code>`set_byte('\x1234567890'::bytea, 4, 64) → \x1234567840`</code>
<code>sha224 (bytea) → bytea</code>	Calcule le hachage SHA-224 de la chaîne binaire.	<code>sha224('abc'::bytea) → \x23097d223405d8228642a477bda255b32aadbc4bda0b3f7e36c9da7</code>
<code>sha256(bytea) → bytea</code>	Calcule le hachage SHA-256 de la chaîne binaire.	<code>sha256('abc'::bytea) → \xba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad</code>
<code>sha384 (bytea) → bytea</code>	Calcule le hachage SHA-384 de la chaîne binaire.	<code>sha384('abc'::bytea) → \xcb00753f45a35e8bb5a03d699ac65007272c32ab0ded1631a8b605a43ff5bed8086072ba1e7cc2358bae-ca134c825a7</code>
<code>sha512 (bytea) → bytea</code>	Calcule le hachage SHA-512 de la chaîne binaire.	

Fonction
Description Exemple(s)
<pre>sha512('abc':bytea) → \xddaf35a193617abac- c417349ae20413112e6fa4e89a97ea20a9eeee64b55d39a 2192992a274fc1a836ba3c23a3feebbd 454d4423643ce80e2a9ac94fa54ca49f</pre>
<pre>substr(bytes bytea, start integer [, count integer]) → bytea</pre> Extrait la sous-chaîne d'octets commençant au `start`ième octet et s'étendant sur `count` octets si ce nombre est spécifié. (Équivalent à `substring(bytes from start for count)`.) <pre>substr("\x1234567890":bytea, 3, 2) → \x5678</pre>

Les fonctions `get_byte` et `set_byte` numérotent le premier octet d'une chaîne binaire comme octet 0. Les fonctions `get_bit` et `set_bit` numérotent les bits à partir de la droite dans chaque octet ; par exemple, le bit 0 est le bit de poids faible du premier octet et le bit 15 est le bit de poids fort du deuxième octet.

Pour des raisons historiques, la fonction `md5` renvoie une valeur hexadécimale de type texte, tandis que les fonctions `SHA-2` renvoient une valeur de type `bytea`. Utilisez les fonctions `encode` et `decode` pour convertir entre les deux. Par exemple, écrivez `encode(sha256('abc'), 'hex')` pour obtenir une représentation textuelle encodée en hexadécimal, ou `decode(md5('abc'), 'hex')` pour obtenir une valeur d'octet.

Les fonctions permettant de convertir des chaînes entre différents jeux de caractères (encodages) et de représenter des données binaires arbitraires sous forme textuelle sont présentées dans le tableau 9.13. Pour ces fonctions, un argument ou un résultat de type texte est exprimé dans l'encodage par défaut de la base de données, tandis que les arguments ou les résultats de type `bytea` sont dans un encodage nommé par un autre argument.

Tableau 9.13. Fonctions de conversion texte/chaîne binaire

Fonction
Description Exemple(s)
<pre>convert(bytes bytea, src_encoding name, dest_encoding name) → bytea</pre> Convertit une chaîne binaire représentant un texte encodé <code>src_encoding</code> en une chaîne binaire encodée <code>dest_encoding</code> (voir la section 23.3.4 pour les conversions disponibles). <code>convert('text_in_utf8', 'UTF8', 'LATIN1') → \x746578745f696e5f75746638</code>
<pre>convert_from(bytes bytea, src_encoding name) → text</pre> Convertit une chaîne binaire représentant du texte encodé <code>src_encoding</code> en texte encodé dans la base de données (voir la section 23.3.4 pour les conversions disponibles). <code>convert_from('text_in_utf8', 'UTF8')</code> <code>→ text_in_utf8</code>
<pre>convert_to(string text, dest_encoding name) → bytea</pre> Convertit une chaîne de caractères (encodée selon l'encodage de la base de données) en une chaîne binaire encodée selon l'encodage <code>dest_encoding</code> (voir la section 23.3.4 pour les conversions disponibles). <code>convert_to('some_text', 'UTF8') → \x736f6d655f74657874</code>
<pre>encode (bytes bytea, format text) → text</pre> Encode les données binaires en une représentation textuelle ; les valeurs de format prises en charge sont : <code>base64</code>, <code>escape</code>, <code>hex</code>. <code>encode('123\000\001', 'base64') → MTIzAAE=</code>
<pre>decode (string text, format text) → bytea</pre> Décode les données binaires à partir d'une représentation textuelle ; les valeurs de format prises en charge sont les mêmes que pour <code>encode</code>.

Fonction
Description
Exemple(s)
décoder('MTIzAAE=', 'base64') → \x3132330001

Les fonctions d'encodage et de décodage prennent en charge les formats textuels suivants :

base64

Le format base64 est conforme à la section 6.81 de la RFC 2045. Conformément à cette RFC, les lignes encodées sont coupées à 76 caractères. Cependant, au lieu du marqueur de fin de ligne MIME CRLF, seul un saut de ligne est utilisé. La fonction de décodage ignore les caractères de retour chariot, de saut de ligne, d'espace et de tabulation. Sinon, une erreur est générée lorsque des données base64 invalides sont fournies à la fonction de décodage, notamment lorsque le remplissage final est incorrect.

s'échapper

Le format d'échappement convertit les octets nuls et les octets dont le bit de poids fort est à 1 en séquences d'échappement octales (\nnn), et double les barres obliques inverses. Les autres valeurs d'octets sont représentées littéralement. La fonction de décodage génère une erreur si une barre oblique inverse n'est pas suivie d'une seconde barre oblique inverse ou de trois chiffres octaux ; elle accepte les autres valeurs d'octets sans modification.

hexagone

Le format hexadécimal représente chaque groupe de 4 bits de données par un chiffre hexadécimal, de 0 à f, en écrivant d'abord le chiffre de poids fort de chaque octet. La fonction d'encodage produit les chiffres hexadécimaux en minuscules. Comme la plus petite unité de données est de 8 bits, la fonction d'encodage renvoie toujours un nombre pair de caractères. La fonction de décodage accepte ces caractères en majuscules ou en minuscules. Une erreur est levée si la fonction de décodage reçoit des données hexadécimales invalides, notamment un nombre impair de caractères.

Voir également la fonction d'agrégation `string_agg` dans la section 9.21 et les fonctions d'objets volumineux dans la section 33.4.

9.6. Fonctions et opérateurs de chaînes binaires

Cette section décrit les fonctions et les opérateurs permettant d'examiner et de manipuler les chaînes de bits, c'est-à-dire les valeurs de type « bit » et « bit variable ». (Bien que seul le type « bit » soit mentionné dans ces tableaux, les valeurs de type « bit variable » peuvent être utilisées indifféremment.) Les chaînes de bits prennent en charge les opérateurs de comparaison usuels présentés dans le tableau 9.1, ainsi que ceux présentés dans le tableau 9.14.

Tableau 9.14. Opérateurs de chaînes binaires

Opérateur
Description
Exemple(s)
bit bit → bit
Enchaînement
B'10001' B'011' → 10001011
bit & bit → bit
Opération ET bit à bit (les entrées doivent être de même longueur)
B'10001' & B'01101' → 00001
bit bit → bit
OU bit à bit (les entrées doivent être de même longueur)

¹ <https://datatracker.ietf.org/doc/html/rfc2045#section-6.8>

Opérateur
Description
Exemple(s)
$B'10001' B'01101' \rightarrow 11101$
$\text{bit} \# \text{bit} \rightarrow \text{bit}$ OU exclusif bit à bit (les entrées doivent être de longueur égale) $B'10001' \# B'01101' \rightarrow 11100$
$\sim \text{bit} \rightarrow \text{bit}$ NON bit à bit $\sim B'10001' \rightarrow 01110$
$\text{bit} \ll \text{entier} \rightarrow \text{bit}$ Décalage bit à bit à gauche (la longueur de la chaîne est préservée) $B'10001' \ll 3 \rightarrow 01000$
$\text{bit} \gg \text{entier} \rightarrow \text{bit}$ Décalage bit à bit vers la droite (la longueur de la chaîne est préservée) $B'10001' \gg 2 \rightarrow 00100$

Certaines des fonctions disponibles pour les chaînes binaires sont également disponibles pour les chaînes de bits, comme indiqué dans le tableau 9.15.

Tableau 9.15. Fonctions de chaînes binaires

Fonction
Description
Exemple(s)
$\text{bit_count}(\text{bit}) \rightarrow \text{bigint}$ Renvoie le nombre de bits à 1 dans la chaîne binaire (également appelé « popcount »). $\text{bit_count}(B'10111') \rightarrow 4$
$\text{bit_length}(\text{bit}) \rightarrow \text{entier}$ Renvoie le nombre de bits dans la chaîne binaire. $\text{bit_length}(B'10111') \rightarrow 5$
$\text{longueur}(\text{bit}) \rightarrow \text{entier}$ Renvoie le nombre de bits dans la chaîne de bits. $\text{longueur}(B'10111') \rightarrow 5$
$\text{octet_length}(\text{bit}) \rightarrow \text{entier}$ Renvoie le nombre d'octets dans la chaîne de bits. $\text{octet_length}(B'1011111011') \rightarrow 2$
$\text{superposition}(\text{ bits bit PLACING nouvelle sous-chaîne bit FROM entier de départ [POUR entier de comptage] }) \rightarrow \text{bit}$ Remplace la sous-chaîne de bits commençant au bit de départ et s'étendant sur `count` bits par nouvelle sous-chaîne. Si `count` est omis, sa longueur est celle de nouvelle sous-chaîne. Exemple : $\text{superposition}(B'010101010101010' \text{ en plaçant } B'11111' \text{ à partir de } 2 \text{ pour } 3) \rightarrow 01111101010101010$
$\text{position}(\text{ sous-chaîne bit IN bits bit }) \rightarrow \text{entier}$ Renvoie le premier indice de début de la sous-chaîne spécifiée dans les bits, ou zéro si elle n'est pas présente.

Fonction
Description Exemple(s)
<code>position(B'010' dans B'000001101011') → 8</code>
<code>sous-chaîne (bits bit [DE l'entier de début] [POUR l'entier de comptage]) → bit</code> Extrait la sous-chaîne de bits commençant au bit de départ si celui-ci est spécifié, et s'arrête après le nombre de bits spécifié si celui-ci est spécifié. Fournissez au moins un des paramètres suivants : départ et nombre. <code>sous-chaîne(B'11001011111' de 3 pour 2) → 00</code>
<code>get_bit(bits bit, n entier) → entier</code> Extrait le n-ième bit d'une chaîne de bits ; le premier bit (le plus à gauche) est le bit 0. <code>get_bit(B'1010101010101010', 6) → 1</code>
<code>définir_bit (bits bit, n entier, nouvelle_valeur entier) → bit</code> Définit la valeur du n-ième bit de la chaîne binaire ; le premier bit (le plus à gauche) est le bit 0. <code>set_bit(B'1010101010101010', 6, 0) → 1010100010101010</code>

De plus, il est possible de convertir des valeurs entières en type bit et inversement. Convertir un entier en bit(n) copie les n bits les plus à droite. Convertir un entier en une chaîne de bits dont la largeur est supérieure à celle de l'entier lui-même étend le signe à gauche. Exemples :

<code>44::bit(10)</code>	<code>0000101100</code>
<code>44::bit(3) cast(-44</code>	<code>100</code>
<code>as bit(12)) '1110'::bit(4)::integer</code>	<code>111111010100</code>
	<code>14</code>

Notez que la conversion en « bit » signifie la conversion en bit(1), et ne fournira donc que le bit de poids faible de l'entier.

9.7. Correspondance des motifs

PostgreSQL propose trois méthodes distinctes de recherche de motifs : l'opérateur SQL LIKE traditionnel, l'opérateur SIMILAR TO plus récent (ajouté dans SQL:1999) et les expressions régulières de type POSIX. Outre les opérateurs de base (« cette chaîne correspond-elle à ce motif ? »), des fonctions permettent d'extraire ou de remplacer des sous-chaînes correspondantes et de découper une chaîne aux emplacements correspondants.

Conseil

Si vos besoins en matière de correspondance de modèles vont au-delà, envisagez d'écrire une fonction définie par l'utilisateur en Perl ou en Tcl.

Prudence

Bien que la plupart des recherches par expressions régulières s'exécutent très rapidement, il est possible de concevoir des expressions régulières dont le traitement peut nécessiter un temps et une quantité de mémoire arbitraires. Soyez prudent lorsque vous acceptez des modèles de recherche par expressions régulières provenant de sources non officielles. Si vous devez absolument le faire, il est conseillé de définir un délai d'expiration pour chaque requête.

Les recherches utilisant des modèles SIMILAR TO présentent les mêmes risques de sécurité, car SIMILAR TO offre bon nombre des mêmes capacités que les expressions régulières de style POSIX.

Les recherches LIKE, étant beaucoup plus simples que les deux autres options, sont plus sûres à utiliser avec des sources de modèles potentiellement hostiles.

Les opérateurs de correspondance de motifs des trois types ne prennent pas en charge les classements non déterministes. Si nécessaire, appliquez un classement différent à l'expression pour contourner cette limitation.

9.7.1. J'AIME

chaîne LIKE motif [ESCAPE caractère-d'échappement] chaîne NOT LIKE
motif [ESCAPE caractère-d'échappement]

L'expression LIKE renvoie vrai si la chaîne correspond au modèle fourni. (Comme prévu, l'expression NOT LIKE renvoie faux si LIKE renvoie vrai, et inversement. L'expression équivalente est NOT (chaîne LIKE modèle).)

Si le motif ne contient ni signe de pourcentage ni trait de soulignement, il représente uniquement la chaîne de caractères elle-même ; dans ce cas, LIKE se comporte comme l'opérateur d'égalité. Un trait de soulignement () dans le motif représente n'importe quel caractère unique ; un signe de pourcentage (%) représente une séquence de zéro ou plusieurs caractères.

Quelques exemples :

'abc' COMME 'abc' 'abc'	vrai
COMME 'a%' 'abc' COMME	vrai
'_b_' 'abc' COMME 'c'	vrai
	FAUX

La fonction LIKE s'applique toujours à la chaîne entière. Par conséquent, pour rechercher une séquence n'importe où dans une chaîne, le motif doit commencer et se terminer par un signe pourcentage.

Pour rechercher un tiret bas ou un signe pourcentage littéral sans rechercher d'autres caractères, le caractère correspondant dans le modèle doit être précédé du caractère d'échappement. Le caractère d'échappement par défaut est la barre oblique inverse, mais il est possible d'en sélectionner un autre à l'aide de la clause ESCAPE. Pour rechercher le caractère d'échappement lui-même, il faut en écrire deux.

Note

Si l'option `standard_conforming_strings` est désactivée, les barres obliques inverses utilisées dans les constantes de chaînes littérales devront être doublées. Consultez la section 4.1.2.1 pour plus d'informations.

Il est également possible de désactiver l'échappement en saisissant ESCAPE ". Cela désactive le mécanisme d'échappement, empêchant ainsi de désactiver la signification spéciale des signes de soulignement et de pourcentage dans le modèle.

Selon la norme SQL, l'omission de l'option ESCAPE signifie qu'aucun caractère d'échappement n'est présent (au lieu d'une barre oblique inverse par défaut), et une valeur ESCAPE nulle est interdite. Le comportement de PostgreSQL à cet égard est donc légèrement non standard.

Le mot-clé ILIKE peut être utilisé à la place de LIKE pour que la correspondance soit insensible à la casse et dépende des paramètres régionaux actifs. Cette fonctionnalité n'est pas standard SQL, mais constitue une extension PostgreSQL.

L'opérateur `~~` est équivalent à LIKE, et `~~*` correspond à ILIKE. Il existe également les opérateurs `!~` et `!~*` qui représentent respectivement NOT LIKE et NOT ILIKE. Tous ces opérateurs sont spécifiques à PostgreSQL. Vous pouvez rencontrer ces noms d'opérateurs dans la sortie de EXPLAIN et à des endroits similaires, car l'analyseur syntaxique traduit LIKE, etc., en ces opérateurs.

Les expressions LIKE, ILIKE, NOT LIKE et NOT ILIKE sont généralement traitées comme des opérateurs dans la syntaxe PostgreSQL ; par exemple, elles peuvent être utilisées dans les constructions d'opérateur d'expression ANY (sous-requête), bien qu'une clause ESCAPE ne puisse pas y être incluse. Dans certains cas particuliers, il peut être nécessaire d'utiliser les noms des opérateurs sous-jacents.

Voir également l'opérateur de correspondance `^@` et la fonction correspondante `starts_with()`, qui sont utiles dans les cas où il suffit de faire correspondre le début d'une chaîne.

9.7.2. SIMILAIRE AUX Expressions Régulières

chaîne SIMILAIRE AU motif [ESCAPE caractère-d'échappement] chaîne NON SIMILAIRE
AU motif [ESCAPE caractère-d'échappement]

L'opérateur `SIMILAR TO` renvoie vrai ou faux selon que son modèle corresponde ou non à la chaîne donnée. Il est similaire à `LIKE`, à la différence qu'il interprète le modèle selon la définition d'une expression régulière de la norme SQL. Les expressions régulières SQL constituent un croisement particulier entre la notation `LIKE` et la notation d'expression régulière courante (POSIX).

Tout comme `LIKE`, l'opérateur `SIMILAR TO` ne réussit que si son motif correspond à la chaîne entière ; contrairement aux expressions régulières classiques où le motif peut correspondre à n'importe quelle partie de la chaîne. De même que `LIKE`, `SIMILAR TO` utilise `_` et `%` comme caractères génériques désignant n'importe quel caractère unique et n'importe quel chaîne, respectivement (celles-ci sont comparables à `.` et `*` dans les expressions régulières POSIX).

En plus de ces fonctionnalités empruntées à `LIKE`, `SIMILAR TO` prend en charge ces métacaractères de correspondance de modèles empruntés aux expressions régulières POSIX :

- `|` indique une alternance (l'une ou l'autre des deux alternatives).
- `*` indique la répétition de l'élément précédent zéro ou plusieurs fois.
- `+` indique la répétition de l'élément précédent une ou plusieurs fois.
- `?` indique la répétition de l'élément précédent zéro ou une fois.
- `{m}` désigne la répétition de l'élément précédent exactement m fois.
- `{m,}` désigne la répétition de l'élément précédent m fois ou plus.
- `{m,n}` désigne la répétition de l'élément précédent au moins m fois et pas plus de n fois.
- Les parenthèses `()` peuvent être utilisées pour regrouper des éléments en un seul élément logique.
- Une expression entre crochets `[...]` spécifie une classe de caractères, tout comme dans les expressions régulières POSIX.

Notez que le point `.` n'est pas un métacaractère pour `SIMILAIRE À`.

Comme pour `LIKE`, la barre oblique inverse désactive la signification spéciale de ces métacaractères. Un autre caractère d'échappement peut être spécifié avec `ESCAPE`, ou la fonctionnalité d'échappement peut être désactivée en écrivant `ESCAPE ''`.

Selon la norme SQL, l'omission de l'option `ESCAPE` signifie qu'aucun caractère d'échappement n'est présent (au lieu d'une barre oblique inverse par défaut), et une valeur `ESCAPE` nulle est interdite. Le comportement de PostgreSQL à cet égard est donc légèrement non standard.

Une autre extension non standard consiste à faire suivre le caractère d'échappement d'une lettre ou d'un chiffre pour accéder aux séquences d'échappement définies pour les expressions régulières POSIX ; voir les tableaux 9.20, 9.21 et 9.22 ci-dessous.

Quelques exemples :

<code>'abc' SIMILAIRE À 'abc'</code>	vrai
<code>'abc' SIMILAIRE À 'a'</code>	FAUX
<code>'abc' SIMILAIRE À '%(b)d%' 'abc' SIMILAIRE À '(b </code>	vrai
<code>c)%' '-abc-' SIMILAIRE À '%\mabc\lM%' vrai</code>	FAUX

'xabcy' SIMILAIRE À '%\mabc\M%' faux

La fonction `substring`` à trois paramètres extrait une sous-chaîne correspondant à une expression régulière SQL. Elle s'écrit selon la syntaxe SQL standard.

sous-chaîne(chaîne de caractères similaires, caractère d'échappement)

ou en utilisant la syntaxe SQL:1999 désormais obsolète :

sous-chaîne(chaîne à partir du modèle pour le caractère d'échappement)

ou sous la forme d'une simple fonction à trois arguments :

sous-chaîne(chaîne, motif, caractère d'échappement)

Comme pour la fonction `SIMILAR TO`, le modèle spécifié doit correspondre à la totalité de la chaîne de données ; dans le cas contraire, la fonction échoue et renvoie null. Pour indiquer la partie du modèle dont la sous-chaîne de données correspondante est recherchée, le modèle doit contenir deux occurrences du caractère d'échappement suivies d'un guillemet double ("). Le texte correspondant à la portion du modèle comprise entre ces séparateurs est renvoyé en cas de correspondance réussie.

Les séparateurs d'échappement guillemets doubles divisent en fait le modèle de la sous-chaîne en trois expressions régulières indépendantes ; par exemple, une barre verticale (|) dans l'une des trois sections n'affecte que cette section. De plus, la première et la troisième de ces expressions régulières sont définies pour correspondre à la plus petite portion de texte possible, et non à la plus grande, en cas d'ambiguïté sur la portion de la chaîne de données correspondant à chaque motif. (En langage POSIX, les première et troisième expressions régulières sont dites non gourmandes.)

En tant qu'extension de la norme SQL, PostgreSQL autorise la présence d'un seul séparateur guillemet double d'échappement, auquel cas la troisième expression régulière est considérée comme vide ; ou l'absence de séparateurs, auquel cas les première et troisième expressions régulières sont considérées comme vides.

Voici quelques exemples, avec `#` délimitant la chaîne de retour :

sous-chaîne('foobar' similaire '%#o_b#%' échappement '#') sous-chaîne('foobar' similaire hors-sujet
'#o_b#%' échappement '#') NUL

9.7.3. Expressions régulières POSIX

Le tableau 9.16 répertorie les opérateurs disponibles pour la correspondance de modèles à l'aide d'expressions régulières POSIX.

Tableau 9.16. Opérateurs de correspondance d'expressions régulières

Opérateur	Description	Exemple(s)
texte ~ texte → booléen	La chaîne correspond à l'expression régulière, en tenant compte de la casse.	'thomas' ~ 't.*ma' → t
texte ~* texte → booléen	La chaîne correspond à l'expression régulière, sans tenir compte de la casse.	'thomas' ~* 'T.*ma' → t
texte !~ texte → booléen	La chaîne ne correspond pas à l'expression régulière (sensible à la casse).	

Opérateur
Description
Exemple(s)
'thomas' !~ 't.*max' → t
texte !~* texte → booléen La chaîne ne correspond pas à l'expression régulière, sans tenir compte de la casse 'thomas' !~* 'T.*ma' → f

Les expressions régulières POSIX offrent un moyen plus puissant de rechercher des motifs que les opérateurs LIKE et SIMILAR TO. De nombreux outils Unix, tels que egrep, sed ou awk, utilisent un langage de recherche de motifs similaire à celui décrit ici.

Une expression régulière est une séquence de caractères qui définit de manière abrégée un ensemble de chaînes de caractères (un ensemble régulier). Une chaîne correspond à une expression régulière si elle appartient à l'ensemble régulier décrit par cette expression. Comme pour l'opérateur LIKE, les caractères du motif correspondent exactement aux caractères de la chaîne, sauf s'il s'agit de caractères spéciaux du langage des expressions régulières ; toutefois, les expressions régulières utilisent des caractères spéciaux différents de ceux utilisés par LIKE. Contrairement aux motifs LIKE, une expression régulière peut correspondre à n'importe quel point d'une chaîne, sauf si elle est explicitement ancrée au début ou à la fin de la chaîne.

Quelques exemples :

'abcd' ~ 'bc' vrai

'abcd' ~ 'ac' vrai — le point correspond à n'importe quel caractère « abcd »

~ « a.*d » vrai — * répète l'élément de motif précédent 'abcd' ~ '(b|x)' vrai — | signifie

OU, groupe de parenthèses 'abcd' ~ '^a' ancre au début de la chaîne

vrai -

'abcd' ~ '^ (b|c)' faux — correspondrait sauf pour l'ancrage

Le langage de modèles POSIX est décrit plus en détail ci-dessous.

La fonction `substring` à deux paramètres, `substring(string from pattern)`, extrait une sous-chaîne correspondant à un motif d'expression régulière POSIX. Elle renvoie `null` en l'absence de correspondance, sinon la première portion du texte correspondant au motif. Si le motif contient des parenthèses, la portion du texte correspondant à la première sous-expression entre parenthèses (celle dont la parenthèse ouvrante est la plus proche) est renvoyée. Vous pouvez encadrer l'expression entière de parenthèses si vous souhaitez en utiliser à l'intérieur sans déclencher cette exception. Si vous avez besoin de parenthèses dans le motif avant la sous-expression à extraire, consultez la section sur les parenthèses non capturantes décrite ci-dessous.

Quelques exemples :

sous-chaîne('foobar' de 'ob') sous-chaîne('foobar' de 'o(.)b') hors-sujet 0

La fonction `regexp_count` compte le nombre d'occurrences d'une expression régulière POSIX dans une chaîne de caractères. Sa syntaxe est `regexp_count(string, pattern [, start [, flags]])`. La recherche de `pattern` dans `string` s'effectue généralement à partir du début de la chaîne, mais si le paramètre `start` est fourni, à partir de l'indice de ce caractère. Le paramètre `flags` est une chaîne de caractères optionnelle contenant zéro ou plusieurs indicateurs (composés d'une seule lettre) qui modifient le comportement de la fonction. Par exemple, inclure `'i'` dans `flags` indique une correspondance insensible à la casse. Les indicateurs pris en charge sont décrits dans le tableau 9.24.

Quelques exemples :

`regexp_count('ABCABCAXYaxy', 'A.')` 3

`regexp_count('ABCABCAXYaxy', 'A.', 1, 'i')` 4

La fonction `regexp_instr` renvoie la position de début ou de fin de la N-ième occurrence d'une expression régulière POSIX dans une chaîne de caractères, ou zéro en l'absence d'occurrence. Sa syntaxe est `regexp_instr(string, pattern [, start [, N [, endoption [, flags [, subexpr ]]])`. La recherche de `pattern` dans `string` s'effectue généralement à partir du début de la chaîne, sauf si le paramètre `start` est fourni, à partir de l'index de ce caractère. Si `N` est spécifié, la N-ième occurrence est recherchée ; sinon, c'est la première qui est recherchée. Si le paramètre `endoption` est omis ou égal à zéro, la fonction renvoie la position du premier caractère de l'occurrence. Sinon, `endoption` doit être égal à un, et la fonction renvoie la position du caractère suivant l'occurrence. Le paramètre `flags` est une chaîne de caractères optionnelle contenant zéro ou plusieurs indicateurs (composés d'une seule lettre) qui modifient le comportement de la fonction. Les indicateurs pris en charge sont décrits dans le tableau 9.24. Pour un motif contenant des sous-expressions entre parenthèses, `subexpr` est un entier indiquant la sous-expression recherchée : le résultat identifie la position de la sous-chaîne correspondant à cette sous-expression. Les sous-expressions sont numérotées selon l'ordre de leur parenthèse ouvrante. Si `subexpr` est omis ou égal à zéro, le résultat identifie la position de la correspondance complète, indépendamment des sous-expressions entre parenthèses.

Quelques exemples :

```
regexp_instr('numéro de votre rue, ville, code postal, FR', '[^,]+', 1, 2)
23
```

```
regexp_instr('ABCDEFGH', '(c..)(...)', 1, 1, 0, 'i', 2)
6
```

La fonction `regexp_like` vérifie si une chaîne de caractères correspond à une expression régulière POSIX et renvoie `true` ou `false`. Sa syntaxe est `regexp_like(chaîne, motif [, options])`. L'option `options` est une chaîne de caractères optionnelle contenant zéro ou plusieurs options (une seule lettre) qui modifient le comportement de la fonction. Les options prises en charge sont décrites dans le tableau 9.24. Si aucune option n'est spécifiée, cette fonction a le même comportement que l'opérateur `~`. Si seule l'option `i` est spécifiée, elle a le même comportement que l'opérateur `~*`.

Quelques exemples :

```
regexp_like('Hello World', 'world') regexp_like('Hello World',
'world', 'i') vrai FAUX
```

La fonction `regexp_match` renvoie un tableau de texte contenant les sous-chaînes correspondantes à la première occurrence d'une expression régulière POSIX dans une chaîne de caractères. Sa syntaxe est `regexp_match(chaîne, motif [, options])`. En l'absence de correspondance, le résultat est `NULL`. Si une correspondance est trouvée et que le motif ne contient aucune sous-expression entre parenthèses, le résultat est un tableau de texte à un seul élément contenant la sous-chaîne correspondant au motif entier. Si une correspondance est trouvée et que le motif contient des sous-expressions entre parenthèses, le résultat est un tableau de texte dont le n-ième élément est la sous-chaîne correspondant à la n-ième sous-expression entre parenthèses du motif (les parenthèses non capturantes ne sont pas prises en compte ; voir ci-dessous pour plus de détails). Le paramètre `options` est une chaîne de texte optionnelle contenant zéro ou plusieurs options d'une seule lettre qui modifient le comportement de la fonction. Les options prises en charge sont décrites dans le tableau 9.24.

Quelques exemples :

```
SELECT regexp_match('foobarbequebaz', 'bar.*que'); regexp_match
```

```
-----
{barbecue} (1
rangée)
```

```
SELECT regexp_match('foobarbequebaz', '(bar)(beque)'); regexp_match
```

```
-----
{bar,beque} (1
ligne)
```

Conseil

Dans le cas courant où vous souhaitez simplement extraire la sous-chaîne correspondante ou NULL en l'absence de correspondance, la meilleure solution consiste à utiliser `regexp_substr()`. Cependant, cette fonction n'est disponible qu'à partir de PostgreSQL version 15. Avec les versions antérieures, vous pouvez extraire le premier élément du résultat de `regexp_match()`, par exemple :

```
SELECT (regexp_match('foobarbequebaz', 'bar.*que'))[1]; regexp_match
```

```
barbecue (1
```

rangée)

La fonction `regexp_matches()` renvoie un ensemble de tableaux de texte contenant les sous-chaînes correspondantes à une expression régulière POSIX. Sa syntaxe est identique à celle de `regexp_match()`. Cette fonction ne renvoie aucune ligne en l'absence de correspondance, une ligne en cas de correspondance sans l'option `-g`, ou N lignes en cas de N correspondances avec l'option `-g`. Chaque ligne renvoyée est un tableau de texte contenant la sous-chaîne correspondante ou les sous-chaînes correspondant aux sous-expressions entre parenthèses de l'expression régulière, comme décrit précédemment pour `regexp_match()`. `regexp_matches()` accepte toutes les options présentées dans le Tableau 9.24, ainsi que l'option `-g` qui lui indique de renvoyer toutes les correspondances, et non seulement la première.

Quelques exemples :

```
SELECT regexp_matches('foo', 'not there'); regexp_matches
```

(0 lignes)

```
SELECT regexp_matches('foobarbequebazilbarfbonk', '(b[^b]+) (b[^b]+)', 'g'); regexp_matches
```

```
{bar,beque}
{bazil,barf} (2 lignes)
```

Conseil

Dans la plupart des cas, il est préférable d'utiliser `regexp_matches()` avec l'option `-g`, car si vous ne souhaitez conserver que la première occurrence, il est plus simple et plus efficace d'utiliser `regexp_match()`. Cependant, `regexp_match()` n'est disponible qu'à partir de PostgreSQL version 10. Pour les versions antérieures, une astuce courante consiste à placer l'appel à `regexp_matches()` dans une sous-requête, par exemple :

```
SELECT col1, (SELECT regexp_matches(col2, '(bar)(beque)'))
DE l'onglet ;
```

Cette fonction renvoie un tableau de texte en cas de correspondance, ou NULL dans le cas contraire, comme le ferait `regexp_match()`. Sans la sous-requête, cette requête ne produirait aucun résultat pour les lignes du tableau sans correspondance, ce qui n'est généralement pas le comportement souhaité.

La fonction `regexp_replace()` remplace les sous-chaînes correspondant aux expressions régulières POSIX par un nouveau texte. Sa syntaxe est `regexp_replace(source, motif, remplacement [, début [, N ]] [, options ])`. (Notez que N ne peut être spécifié que si début l'est, mais options peut être fournies dans tous les cas.) La chaîne source est renvoyée inchangée si aucune correspondance n'est trouvée avec le motif.

Si une correspondance est trouvée, la chaîne source est renvoyée avec la chaîne de remplacement substituée à la sous-chaîne correspondante. La chaîne de remplacement peut contenir `\\n`, où n est compris entre 1 et 9, pour indiquer que la sous-chaîne source correspondant à la n-ième sous-expression entre parenthèses du motif doit être insérée, et `\\&` pour indiquer que la sous-chaîne correspondant au motif entier doit être insérée. Utilisez `\\` si vous souhaitez insérer une barre oblique inverse littérale dans le texte de remplacement. La recherche du motif s'effectue dans la chaîne, généralement à partir du début, mais si le paramètre `start` est fourni, à partir de l'index de caractère spécifié. Par défaut, seule la première occurrence du motif est remplacée. Si N est spécifié et supérieur à zéro, la N-ième occurrence du motif est remplacée. Si l'option g est activée, ou si N est spécifié et égal à zéro, toutes les occurrences à partir de la position de départ sont remplacées. (L'option « g » est ignorée lorsque « N » est spécifié.) Le paramètre « flags » est une chaîne de caractères facultative contenant zéro ou plusieurs options d'une seule lettre qui modifient le comportement de la fonction. Les options prises en charge (à l'exception de « g ») sont décrites dans le tableau 9.24.

Quelques exemples :

```
regexp_replace('foobarbaz', 'b.', 'X') fooXbaz
```

```
regexp_replace('foobarbaz', 'b.', 'X', 'g')
fooXX
```

```
regexp_replace('foobarbaz', 'b(..)', 'X\\1Y', 'g')
fooXarYXazY
```

```
regexp_replace('Une fonction PostgreSQL', 'a|e|i|o|u', 'X', 1, 0,
'je')
X PXstgrXSQL fXnctXXn
```

```
regexp_replace('Une fonction PostgreSQL', 'a|e|i|o|u', 'X', 1, 3,
'je')
Une fonction PostgrXSQL
```

La fonction `regexp\_split\_to\_table` divise une chaîne de caractères en utilisant une expression régulière POSIX comme délimiteur. Sa syntaxe est `regexp\_split\_to\_table(chaîne, motif [, options])`. Si aucune correspondance n'est trouvée, la fonction renvoie la chaîne entière. S'il y a au moins une correspondance, elle renvoie, pour chaque correspondance, le texte allant de la fin de la dernière correspondance (ou du début de la chaîne) jusqu'au début de la correspondance suivante. Lorsqu'il n'y a plus de correspondance, elle renvoie le texte allant de la fin de la dernière correspondance jusqu'à la fin de la chaîne. Le paramètre `options` est une chaîne de caractères optionnelle contenant zéro ou plusieurs options d'une seule lettre qui modifient le comportement de la fonction. `regexp\_split\_to\_table` prend en charge les options décrites dans le tableau 9.24.

La fonction `regexp\_split\_to\_array` se comporte de la même manière que `regexp\_split\_to\_table`, à la différence qu'elle renvoie un tableau de texte. Sa syntaxe est `regexp\_split\_to\_array(string, pattern [, flags])`. Les paramètres sont identiques à ceux de `regexp\_split\_to\_table`.

Quelques exemples :

```
SELECT foo FROM regexp_split_to_table('the quick brown fox jumps over the lazy dog', '\\s+') AS foo;
```

```

nourriture
-----
le
brun
rapide
renard
sauts
sur
le
chien

paresseux (9 rangées)
```

```
SELECT regexp_split_to_array('the quick brown fox jumps over the lazy dog', '\s+'); regexp_split_to_array
```

```
-----
{le,rapide,renard,brun,saute,par-dessus,le,chien, paresseux}
(1 rangée)
```

```
SELECT foo FROM regexp_split_to_table('the quick brown fox', '\s*')
AS foo;
```

```
nourriture
```

```
-----
```

```
t
```

```
h
```

```
e
```

```
q
```

```
vous
```

```
je
```

```
c
```

```
k
```

```
b
```

```
r
```

```
o
```

```
w
```

```
n
```

```
f
```

```
o
```

```
x
```

```
(16 lignes)
```

Comme le montre le dernier exemple, les fonctions de segmentation d'expressions régulières ignorent les correspondances de longueur nulle qui se trouvent au début ou à la fin de la chaîne, ou immédiatement après une correspondance précédente. Ceci est contraire à la définition stricte de la correspondance d'expressions régulières implémentée par les autres fonctions d'expressions régulières, mais constitue généralement le comportement le plus pratique. D'autres systèmes logiciels, tels que Perl, utilisent des définitions similaires.

La fonction `regexp_substr` renvoie la sous-chaîne correspondant à un motif d'expression régulière POSIX, ou `NULL` en l'absence de correspondance. Sa syntaxe est `regexp_substr(string, pattern [, start [, N [, flags [, subexpr ]]]])`. Le paramètre `pattern` est recherché dans la chaîne, généralement à partir du début, mais si le paramètre `start` est fourni, à partir de l'indice de ce caractère. Si `N` est spécifié, la fonction renvoie la N-ième occurrence du motif ; sinon, elle renvoie la première. Le paramètre `flags` est une chaîne de caractères optionnelle contenant zéro ou plusieurs indicateurs (composés d'une seule lettre) qui modifient le comportement de la fonction. Les indicateurs pris en charge sont décrits dans le tableau 9.24. Pour un motif contenant des sous-expressions entre parenthèses, `subexpr` est un entier indiquant la sous-expression recherchée : le résultat est la sous-chaîne correspondant à cette sous-expression. Les sous-expressions sont numérotées selon l'ordre de leur parenthèse ouvrante. Si `subexpr` est omis ou égal à zéro, le résultat est la correspondance complète, quelles que soient les sous-expressions entre parenthèses.

Quelques exemples :

```
regexp_substr('numéro de votre rue, ville, code postal, FR', '[^,]+', 1, 2)
                                                                ville code
postal regexp_substr('ABCDEFGH', '(c..)(...)', 1, 1, 'i', 2)
                                                                FGH
```

9.7.3.1. Détails des expressions régulières

Les expressions régulières de PostgreSQL sont implémentées à l'aide d'un logiciel développé par Henry Spencer. La majeure partie de la description des expressions régulières ci-dessous est extraite intégralement de son manuel.

Les expressions régulières (RE), telles que définies dans POSIX 1003.2, se présentent sous deux formes : les expressions régulières étendues (ERE). (approximativement celles d'egrep), et les expressions régulières ou les expressions régulières de base (approximativement celles d'ed). PostgreSQL prend en charge les deux. il prend en charge les formulaires et implémente également certaines extensions qui ne figurent pas dans la norme POSIX, mais qui sont devenues courantes. largement utilisées en raison de leur disponibilité dans des langages de programmation tels que Perl et Tcl. Les expressions régulières utilisant ces Dans cette documentation , les extensions non POSIX sont appelées RE avancées ou ARE. Les ARE sont presque des Sur-ensemble exact des ERE, mais les BRE présentent plusieurs incompatibilités de notation (en plus d'être beaucoup plus nombreux). (plus limité). Nous décrivons d'abord les formulaires ARE et ERE, en soulignant les caractéristiques qui s'appliquent uniquement aux ARE, Décrivez ensuite en quoi les BRE diffèrent.

Note

PostgreSQL présume toujours initialement qu'une expression régulière suit les règles ARE. Cependant, les règles ERE ou BRE, plus limitées, peuvent être choisies en ajoutant une option intégrée .
 au modèle RE, tel que décrit dans la section 9.7.3.4. Cela peut être utile pour la compatibilité avec applications qui attendent exactement les règles POSIX 1003.2.

Une expression régulière est définie comme une ou plusieurs branches, séparées par `|`. Elle correspond à tout ce qui correspond à l'une des branches.

Une branche est constituée de zéro ou plusieurs atomes ou contraintes quantifiés, concaténés. Elle correspond à une correspondance pour la première correspondance, suivie d'une correspondance pour la seconde, etc. ; une branche vide correspond à la chaîne vide.

Un atome quantifié est un atome éventuellement suivi d'un seul quantificateur. Sans quantificateur, il correspond à une correspondance pour l'atome. Avec un quantificateur, elle peut correspondre à un certain nombre de correspondances pour l'atome. Un atome peut prendre n'importe quelle forme parmi celles présentées dans le tableau 9.17. Les quantificateurs possibles et leur signification sont : comme indiqué dans le tableau 9.18.

Une contrainte correspond à une chaîne vide, mais ne correspond que lorsque des conditions spécifiques sont remplies. Une contrainte peut être utilisé là où un atome pourrait être utilisé, sauf qu'il ne peut pas être suivi d'un quantificateur. Le simple Les contraintes sont présentées dans le tableau 9.19 ; d'autres contraintes sont décrites plus loin.

Tableau 9.17. Atomes d'expression régulière

Atome	Description
<code>(concernant)</code>	(où <code>re</code> est une expression régulière quelconque) correspond à une correspondance pour <code>re</code> , avec la correspondance notée pour possible reportage
<code>(?:concernant)</code>	comme ci-dessus, mais le match n'est pas signalé. (un ensemble de parenthèses « non capturantes ») (AREs seulement)
<code>.</code>	correspond à n'importe quel caractère unique
<code>[caractères]</code>	une expression entre crochets, correspondant à l'une quelconque des caractères (voir la section 9.7.3.2 pour plus de détails)
<code>k</code>	(où <code>k</code> est un caractère non alphanumérique) correspond à ce caractère pris comme un caractère ordinaire, par exemple, <code>\k</code> correspond à un caractère barre oblique inverse
<code>\c</code>	où <code>c</code> est alphanumérique (éventuellement suivi) (par d'autres personnages) est une échappatoire, voir la section 9.7.3.3 (ARE uniquement ; dans les ERE et BRE, ce correspondances <code>c</code>)
<code>{</code>	Lorsqu'il est suivi d'un caractère autre qu'un chiffre, il correspond à l'accolade ouvrante <code>{</code> ; lorsqu'il est suivi d'un chiffre, il marque le début d'une limite. (voir ci-dessous)

Atome	Description
x	où x est un caractère unique sans autre signification, correspond à ce caractère

Une expression régulière ne peut pas se terminer par une barre oblique inverse (\).

Note

Si l'option `standard_conforming_strings` est désactivée, toute barre oblique inverse que vous écrivez dans une chaîne littérale sera interprétée comme une erreur. Les constantes devront être doublées. Voir la section 4.1.2.1 pour plus d'informations.

Tableau 9.18. Quantificateurs d'expressions régulières

Quantificateur	Allumettes
*	une séquence de 0 ou plusieurs correspondances de l'atome
+	une séquence d'une ou plusieurs correspondances de l'atome
?	une séquence de 0 ou 1 correspondances de l'atome
{m}	une séquence de m correspondances exactes de l'atome
{m,}	une séquence de m ou plus correspondances de l'atome
{m,n}	une séquence de m à n (inclus) correspondances de l'atome ; m ne peut pas dépasser n
*?	version non gourmande de *
+ ?	version non gourmande de +
??	version non gourmande de ?
{m}?	Version non gourmande de {m}
{m,}?	version non gourmande de {m,}
{m,n}?	version non gourmande de {m,n}

Les formes utilisant {...} sont appelées bornes. Les nombres m et n à l'intérieur d'une borne sont non signés. entiers décimaux dont les valeurs autorisées vont de 0 à 255 inclus.

Les quantificateurs non gourmands (disponibles uniquement dans les ARE) correspondent aux mêmes possibilités que leurs homologues. homologues normaux (gourmands) , mais préfèrent le plus petit nombre plutôt que le plus grand nombre de correspondances. Voir la section 9.7.3.5 pour plus de détails.

Note

Un quantificateur ne peut pas suivre immédiatement un autre quantificateur, par exemple, ** est invalide. Un quantificateur ne peut pas commencer une expression ou une sous-expression ni suivre ^ ou |.

Tableau 9.19. Contraintes d'expression régulière

Contrainte	Description
^	correspondances au début de la chaîne
\$	correspondances à la fin de la chaîne
(?=re)	Les correspondances positives anticipées à tout moment où un Correspondance de sous-chaîne re commence (AREs uniquement)
(?!concernant)	Les correspondances négatives anticipées à tout moment où aucune sous-chaîne ne correspond à re commence (AREs uniquement)

Contrainte	Description
(?<=re)	un regard positif sur les matches à tout moment où un Correspondance de sous-chaîne se terminant par re (ARE uniquement)
(?<!re)	correspondances de lookbehind négatif à tout moment où aucune sous-chaîne ne correspond aux fins de re (ARE uniquement)

Les contraintes de pré-requête et de rétro-requête ne peuvent pas contenir de références arrière (voir section 9.7.3.3), et toutes

Les parenthèses à l'intérieur de ces parenthèses sont considérées comme non capturantes.

9.7.3.2. Expressions entre crochets

Une expression entre crochets est une liste de caractères encadrée par des crochets []. Elle correspond généralement à n'importe quel caractère unique de la liste (voir ci-dessous). Si la liste commence par ^, cela correspond à n'importe quel caractère unique ne figurant pas dans la liste. Si deux caractères de la liste sont séparés par un tiret (-), cela représente l'ensemble des caractères possibles. caractères compris entre ces deux (inclus) dans la séquence de classement, par exemple [0-9] dans les correspondances ASCII n'importe quel chiffre décimal. Il est interdit que deux intervalles partagent une même extrémité, par exemple l'as. Les intervalles sont très Ces méthodes dépendent de la séquence de classement ; les programmes portables doivent donc éviter de s'y fier.

Pour inclure une] littérale dans la liste, placez-la en premier caractère (après ^, si celui-ci est utilisé). Pour inclure un Une valeur littérale peut être le premier ou le dernier caractère, ou la seconde borne d'une plage. Pour utiliser une valeur littérale : la première extrémité d'une plage, entourez-la de [. et .] pour en faire un élément de classement (voir ci-dessous).

À l'exception de ces caractères, certaines combinaisons utilisant [(voir les paragraphes suivants) et les caractères d'échappement (AREs uniquement), tous les autres caractères spéciaux perdent leur signification particulière à l'intérieur d'une expression entre crochets. En particulier, \ n'est pas spécial lorsqu'il suit les règles ERE ou BRE, bien qu'il soit spécial (comme l'introduction une évasion) dans les ARE.

Dans une expression entre crochets, un élément de classement (un caractère, une séquence de plusieurs caractères qui se classe comme s'il s'agissait d'un seul caractère, ou un nom de séquence de classement pour l'un ou l'autre) entouré de [. et .] représente la séquence de caractères de cet élément de classement. La séquence est traitée comme un seul élément de la liste de l'expression entre crochets. Cela permet à une expression entre crochets contenant un élément de classement à plusieurs caractères de correspondre à plusieurs caractères, par exemple si la séquence de classement inclut un ch élément de classement, alors le RE [[.ch.]]*c correspond aux cinq premiers caractères de chchcc.

Note

PostgreSQL ne prend actuellement pas en charge les éléments de classement multi-caractères. Cette information décrit les comportements futurs possibles.

Dans une expression entre crochets, un élément de classement encadré par [= et =] constitue une classe d'équivalence, représentant les séquences de caractères de tous les éléments de classement équivalents à celui-ci, y compris lui-même.

Il n'existe pas d'autres éléments de classement équivalents, le traitement est comme si les délimiteurs englobants étaient [. et .].) Par exemple, si o et ^ sont les membres d'une classe d'équivalence, alors [[=o=]], [[=^=]], et [o^] sont tous synonymes. Une classe d'équivalence ne peut pas être une extrémité d'un intervalle.

Dans une expression entre crochets, le nom d'une classe de caractères encadré par [: et :] représente la liste des Tous les caractères appartenant à cette classe. Une classe de caractères ne peut pas être utilisée comme extrémité d'une plage. La norme POSIX définit les noms de classes de caractères suivants : alnum (lettres et chiffres), alpha (lettres), blank (espace et tabulation), cntrl (caractères de contrôle), digit (chiffres), graph (caractères imprimables), caractères sauf espace), minuscules, caractères imprimables (y compris l'espace), Les caractères suivants sont pris en charge : ponctuation, espace, majuscules et chiffres hexadécimaux. Leur comportement est généralement cohérent d'une plateforme à l'autre. pour les caractères de l'ensemble ASCII 7 bits. Indique si un caractère non-ASCII donné est considéré comme appartenant à L'une de ces classes dépend du classement utilisé pour la fonction ou l'opérateur d'expression régulière (voir section 23.2), ou par défaut du paramètre régional LC_CTYPE de la base de données (voir section 23.1). La classification des caractères non-ASCII peut varier d'une plateforme à l'autre, même dans des paramètres régionaux portant des noms similaires. (Mais la locale C ne considère jamais les caractères non-ASCII comme appartenant à l'une de ces classes.) En plus de ces classes de caractères standard, PostgreSQL définit la classe de caractères de mots, qui est

identique à alnum plus le caractère de soulignement (`_`), et la classe de caractères ASCII, qui contient Exactly l'ensemble ASCII 7 bits.

Il existe deux cas particuliers d'expressions entre crochets : les expressions entre crochets `[[:<:]]` et `[[:>:]]`.

sont des contraintes, correspondant respectivement à des chaînes vides au début et à la fin d'un mot. Un mot est défini comme une séquence de caractères alphanumériques qui n'est ni précédée ni suivie d'autres caractères alphanumériques.

Un caractère alphanumérique est tout caractère appartenant à la classe des caractères alphanumériques, c'est-à-dire toute lettre, tout chiffre ou toute lettre.

le trait de soulignement. Il s'agit d'une extension, compatible avec mais non spécifiée par POSIX 1003.2, et devrait à utiliser avec précaution dans les logiciels destinés à être portables sur d'autres systèmes. La contrainte s'échappe

Les méthodes décrites ci-dessous sont généralement préférables ; elles ne sont plus standardisées, mais sont plus faciles à saisir.

9.7.3.3. Échappements d'expressions régulières

Les séquences d'échappement sont des séquences spéciales commençant par `\` suivies d'un caractère alphanumérique.

sous plusieurs formes : saisie de caractères, raccourcis de classes, échappements de contraintes et références arrières. A `\`

Dans les ARE, la présence d'un caractère alphanumérique ne constituant pas une séquence d'échappement valide est illégale. Dans les ERE,

Il n'y a pas d'échappement : en dehors d'une expression entre crochets, un `\` suivi d'un caractère alphanumérique est simplement représente ce caractère comme un caractère ordinaire, et à l'intérieur d'une expression entre crochets, `\` est un caractère ordinaire caractère. (Ce dernier point constitue la seule véritable incompatibilité entre les ERE et les ARE.)

Les séquences d'échappement de caractères permettent de spécifier plus facilement les caractères non imprimables et autres caractères indésirables dans les expressions régulières. Elles sont présentées dans le tableau 9.20.

Les raccourcis de classe permettent de représenter de manière abrégée certaines classes de personnages couramment utilisées. comme indiqué dans le tableau 9.21.

Une échappement de contrainte est une contrainte, correspondant à la chaîne vide si des conditions spécifiques sont remplies, écrite comme moyen d'évasion. Ils sont présentés dans le tableau 9.22.

Une référence arrière (`\n`) correspond à la même chaîne de caractères que celle correspondant à la sous-expression précédente entre parenthèses.

spécifié par le nombre `n` (voir tableau 9.23). Par exemple, `([bc])\1` correspond à `bb` ou `cc` mais pas à `bc`

ou `cb`. La sous-expression doit précéder entièrement la référence arrière dans l'expression régulière. Les sous-expressions sont

Les éléments sont numérotés selon l'ordre de leur parenthèse ouvrante. Les parenthèses non capturantes ne définissent pas de

sous-expressions. La référence arrière ne prend en compte que les caractères de la chaîne correspondant à la sous-expression

référéncée, et non les contraintes qu'elle contient. Par exemple, `(^d)\1` correspond à `22`.

Tableau 9.20. Échappements de caractères pour les expressions régulières

S'échapper	Description
<code>\un</code>	caractère d'alerte (cloche), comme dans C
<code>b</code>	retour arrière, comme dans C
<code>\B</code>	synonyme de barre oblique inverse (<code>\</code>) pour aider à réduire le besoin de doubler la barre oblique inverse
<code>\cX</code>	(où X est un caractère quelconque) le caractère dont Les 5 bits de poids faible sont identiques à ceux de X, et dont les autres bits sont tous nuls
<code>\e</code>	le caractère dont le nom de séquence de classement est Échap, ou à défaut, le caractère de valeur octale 033
<code>\f</code>	alimentation de forme, comme dans C
<code>\n</code>	nouvelle ligne, comme dans C
<code>\r</code>	retour chariot, comme dans C
<code>\t</code>	tabulation horizontale, comme dans C
<code>\wxyz</code>	(où wxyz représente exactement quatre chiffres hexadécimaux) le caractère dont la valeur hexadécimale est 0xwxyz

S'échapper	Description
\Ustuvwxyz	(où stuvwxyz est exactement huit hexadécimaux chiffres) le caractère dont la valeur hexadécimale est 0xstuvwxyz
\v	tabulation verticale, comme dans C
\xhhh	(où hhh est une séquence quelconque de chiffres hexadécimaux) le caractère dont la valeur hexadécimale est 0xxxh (un seul caractère, quel que soit le nombre) (Les chiffres hexadécimaux sont utilisés)
\0	le caractère dont la valeur est 0 (l'octet nul)
\xy	(où xy représente exactement deux chiffres octaux, et n'est pas (une référence arrière) le caractère dont la valeur octale est 0xy
\xyz	(où xyz représente exactement trois chiffres octaux, et est (pas une référence arrière) le caractère dont l'octal La valeur est 0xyz

Les chiffres hexadécimaux sont 0 à 9, af et AF. Les chiffres octaux sont 0 à 7.

Les séquences d'échappement numériques spécifiant des valeurs en dehors de la plage ASCII (0-127) ont une signification. dépend de l'encodage de la base de données. Lorsque l'encodage est UTF-8, les valeurs d'échappement sont équivalentes à Les points de code Unicode, par exemple \u1234, signifient le caractère U+1234. Pour les autres codages multioctets, les séquences d'échappement de caractères spécifient généralement simplement la concaténation des valeurs d'octets du caractère. Si la valeur d'échappement ne correspond à aucun caractère autorisé dans l'encodage de la base de données, aucune erreur ne sera générée.

une valeur peut être générée, mais elle ne correspondra jamais à aucune donnée.

Les séquences d'échappement sont toujours interprétées comme des caractères ordinaires. Par exemple, \135 correspond à] en ASCII. mais \135 ne termine pas une expression entre crochets.

Tableau 9.21. Classe d'expressions régulières - Échappements abrégés

S'échapper	Description
\d	correspond à n'importe quel chiffre, comme [[:digit:]]
\s	correspond à n'importe quel caractère d'espacement, comme [[:espace:]]
\w	correspond à n'importe quel caractère de mot, comme [[:mot:]]
\D	correspond à tout caractère non numérique, comme [^[:digit:]]
\S	correspond à n'importe quel caractère non blanc, comme [^[:espace:]]
\W	correspond à n'importe quel caractère non alphabétique, comme [^[:mot:]]

Les séquences d'échappement abrégées fonctionnent également à l'intérieur des expressions entre crochets, bien que les définitions affichées Les constructions ci-dessus ne sont pas tout à fait syntaxiquement valides dans ce contexte. Par exemple, [ac\d] est équivalent à [ac[:digit:]].

Tableau 9.22. Échappements des contraintes d'expression régulière

S'échapper	Description
\UN	ne correspond qu'au début de la chaîne (voir Voir la section 9.7.3.5 pour plus de détails sur la différence avec ^)
\m	ne correspond qu'au début d'un mot
\M	ne correspond qu'à la fin d'un mot